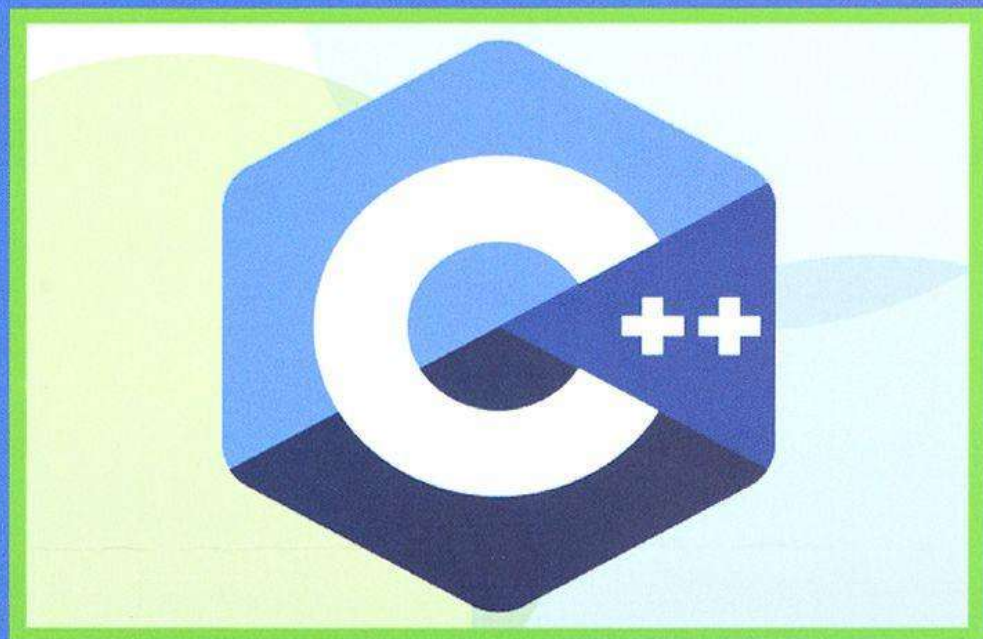


Васильев А. Н.

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ на C++



От простых объектно-ориентированных программ до
лямбда-вычислений, функторов,
итераторов, контейнеров
и многопоточного программирования

Нит
Издательство

СЕРИЯ — ПРОСТО О СЛОЖНОМ — СЕРИЯ



Наука и Техника

Санкт-Петербург
2016



ВАСИЛЬЕВ А. Н.

Объектно- ориентированное программирование на C++



Наука и Техника

Санкт-Петербург
2016

ISBN 978-5-94387-984-5

УДК 004.438

ВАСИЛЬЕВ А. Н.

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ НА C++ —

СПб.: Наука и Техника, 2016. — 544 с., ил.

Серия "Просто о сложном"

Представленная книга - о языке программирования C++. А еще эта книга - об объектно-ориентированном программировании (сокращенно ООП). Читатель научится создавать полноценные объектно-ориентированные программы. Мы рассмотрим все основные и наиболее важные конструкции C++, так что при желании читатель сможет создавать и обычные (не объектно-ориентированные) программы. Но случится это не само по себе. Книгу мало прочитать. С книгой нужно работать. В этом случае успех придет. Книга предназначена как тем, кто уже имеет некоторое представление о C++, так и тем, кто сталкивается с ним впервые и хочет освоить данный язык программирования.

Книга написана простым и доступным языком с большим количеством наглядных примеров.

Контактные телефоны издательства:

(812) 412 70 25, (812) 412 70 26, (044) 516 38 66

Официальный сайт: www.nit.com.ru

© Васильев А.Н., ПРОКДИ, 2016

© Наука и техника (оригинал-макет), 2016

Содержание

ВВЕДЕНИЕ.....	11
ВСТУПЛЕНИЕ	12
ЯЗЫК C++ И ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ	12
ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ.....	12
ОСОБЕННОСТИ ЯЗЫКА C++	15
НАШИ МЕТОДЫ	16
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ И СТАНДАРТЫ ЯЗЫКА	17
СРЕДА РАЗРАБОТКИ DEV C++	19
СРЕДА РАЗРАБОТКИ MICROSOFT VISUAL STUDIO EXPRESS	24
СРЕДА РАЗРАБОТКИ NETBEANS	30
ОБРАТНАЯ СВЯЗЬ	42
ГЛАВА 1. ПРОСТЫЕ ПРОГРАММЫ	43
1.1. ПРОГРАММИРОВАНИЕ БЕЗ ПРОГРАММИРОВАНИЯ	44
1.2. РЕАЛИЗУЕМ ПЕРВУЮ ОБЪЕКТНО-ОРИЕНТИРОВАННУЮ ПРОГРАММУ.....	48
1.3. ДОЛОЙ ОКОВЫ ООП	57
1.4. ЗНАКОМСТВО С КОНСТРУКТОРАМИ	59
1.5. РЕОРГАНИЗАЦИЯ ПРОГРАММНОГО КОДА.....	63
ГЛАВА 2. МЕТОДЫ	71
2.1. ПЕРЕГРУЗКА МЕТОДОВ	72
2.2. ПЕРЕГРУЗКА ФУНКЦИЙ	84
2.3. ОПЕРАТОРНЫЕ МЕТОДЫ	85

2.4. ОПЕРАТОРНЫЕ ФУНКЦИИ	96
ГЛАВА 3. НАСЛЕДОВАНИЕ И СОПУТСТВУЮЩИЕ МЕХАНИЗМЫ	103
3.1. ОСНОВЫ НАСЛЕДОВАНИЯ	104
3.2. ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДОВ И ВИРТУАЛЬНОСТЬ.....	111
3.3. КОНСТРУКТОР ПРОИЗВОДНОГО КЛАССА	121
ГЛАВА 4. НАСЛЕДОВАНИЕ: СЕКРЕТЫ И ОСОБЕННОСТИ.....	129
4.1. МНОЖЕСТВЕННОЕ НАСЛЕДОВАНИЕ	130
4.2. ВИРТУАЛЬНЫЕ БАЗОВЫЕ КЛАССЫ.....	137
4.3. АБСТРАКТНЫЕ КЛАССЫ И ЧИСТО ВИРТУАЛЬНЫЕ МЕТОДЫ.....	140
4.4. ПЕРЕМЕННЫЕ БАЗОВЫХ И ПРОИЗВОДНЫХ КЛАССОВ.....	145
ГЛАВА 5. ССЫЛКИ И УКАЗАТЕЛИ	155
5.1. ЗНАКОМСТВО СО ССЫЛКАМИ	156
5.2. ССЫЛКИ И НАСЛЕДОВАНИЕ.....	160
5.3. МЕХАНИЗМ ПЕРЕДАЧИ АРГУМЕНТОВ.....	164
5.4. МЕХАНИЗМ ПЕРЕДАЧИ АРГУМЕНТОВ И НАСЛЕДОВАНИЕ.....	171
5.5. ЗНАКОМСТВО С УКАЗАТЕЛЯМИ	175
ГЛАВА 6. ПАМЯТЬ, ДЕКТРУКТОРЫ И МАССИВЫ	183
6.1. ДИНАМИЧЕСКОЕ ВЫДЕЛЕНИЕ ПАМЯТИ	184
6.2. ДЕКТРУКТОР	187
6.3. ЗНАКОМСТВО С МАССИВАМИ	197
6.4. СТАТИЧЕСКИЕ МАССИВЫ	206

6.5. СИМВОЛЬНЫЕ МАССИВЫ	211
ГЛАВА 7. ВСЕ О МАССИВАХ	217
7.1. ИНДЕКСИРОВАНИЕ ОБЪЕКТОВ	218
7.2. ОСОБЕННОСТИ ИСПОЛЬЗОВАНИЯ ДИНАМИЧЕСКИХ МАССИВОВ В КЛАССАХ	227
7.3. ПЕРЕГРУЗКА ОПЕРАТОРА ПРИСВАИВАНИЯ	232
7.4. КОНСТРУКТОР СОЗДАНИЯ КОПИИ	237
ГЛАВА 8. ФУНКЦИИ И КЛАССЫ	243
8.1. ОБОБЩЕННЫЕ ФУНКЦИИ	244
8.2. ОБОБЩЕННЫЕ КЛАССЫ	253
8.3. ПЕРЕГРУЗКА И ЯВНАЯ СПЕЦИАЛИЗАЦИЯ ОБОБЩЕННЫХ ФУНКЦИЙ	259
8.4. ЯВНАЯ СПЕЦИАЛИЗАЦИЯ ОБОБЩЕННЫХ КЛАССОВ	264
8.5. ОБОБЩЕННЫЕ КЛАССЫ И НАСЛЕДОВАНИЕ	270
ГЛАВА 9. ФУНКТОРЫ	275
9.1. ЗНАКОМСТВО С ФУНКТОРАМИ	276
9.2. ФУНКТОРЫ С АРГУМЕНТАМИ И БЕЗ АРГУМЕНТОВ	279
9.3. РЕАЛИЗАЦИЯ ПОЛИНОМА ЧЕРЕЗ ФУНКТОР	282
9.4. КОНСТАНТНЫЕ МЕТОДЫ И АРГУМЕНТЫ	291
9.5. ФУНКТОР НА ОСНОВЕ ШАБЛОНА	298
9.6. ФУНКТОР НА ОСНОВЕ КЛАССА СО СТАТИЧЕСКИМ МАССИВОМ	305
ГЛАВА 10. ФУНКЦИЯ КАК АРГУМЕНТ И РЕЗУЛЬТАТ	311

10.1. УКАЗАТЕЛЬ НА ФУНКЦИЮ	312
10.2. РЕШЕНИЕ УРАВНЕНИЯ МЕТОДОМ ПОСЛЕДОВАТЕЛЬНЫХ ПРИБЛИЖЕНИЙ	316
10.3. ЗНАКОМСТВО С ЛЯМБДА-ФУНКЦИЯМИ	319
10.4. МАССИВ УКАЗАТЕЛЕЙ НА ФУНКЦИЮ	323
10.5. ФУНКЦИЯ КАК РЕЗУЛЬТАТ	327
10.6. УКАЗАТЕЛИ НА МЕТОДЫ	330
10.7. ВОЗВРАЩАЯСЬ К ФУНКТОРАМ	334
ГЛАВА 11. ВОЗВРАЩАЯСЬ К МАССИВАМ	345
11.1. ДВУМЕРНЫЙ СТАТИЧЕСКИЙ МАССИВ	346
11.2. ИМИТАЦИЯ НЕОГРАНИЧЕННОГО ДВУМЕРНОГО МАССИВА	352
11.3. ДИНАМИЧЕСКИЕ ДВУМЕРНЫЕ МАССИВЫ	354
11.4. СОЗДАНИЕ "РВАНОВОГО" ДВУМЕРНОГО МАССИВА	359
11.5. ДВУМЕРНЫЙ МАССИВ КАК ПОЛЕ ОБЪЕКТА	364
11.6. ПЕРЕДАЧА ДВУМЕРНОГО МАССИВА АРГУМЕНТОМ ФУНКЦИИ	374
ГЛАВА 12. КОНТЕЙНЕРЫ И ИТЕРАТОРЫ	381
12.1. ЗНАКОМСТВО С КОНТЕЙНЕРАМИ	382
12.2. ЗНАКОМСТВО С ИТЕРАТОРАМИ.....	395
12.3. СТАНДАРТНЫЕ ПОДХОДЫ.....	415
ГЛАВА 13. ОБРАБОТКА ИСКЛЮЧИТЕЛЬНЫХ СИТУАЦИЙ	423
13.1. ПРИМЕР С ОШИБКОЙ.....	424
13.2. ПЕРСОНАЛИЗИРУЕМ ОШИБКИ	429

13.3. ИСПОЛЬЗОВАНИЕ ОБЪЕКТА ИСКЛЮЧЕНИЯ	436
13.4. ГЕНЕРИРОВАНИЕ ИСКЛЮЧЕНИЙ	439
13.5. ПОДКЛАССЫ ОШИБОК	447
13.6. ПОЛЬЗОВАТЕЛЬСКИЕ КЛАССЫ ИСКЛЮЧЕНИЙ	449

ГЛАВА 14. МНОГОПОТОЧНОЕ ПРОГРАММИРОВАНИЕ 457

14.1. ЗНАКОМСТВО С ПОТОКАМИ	459
14.2. НЕСКОЛЬКО ДОЧЕРНИХ ПОТОКОВ	465
14.3. ПЕРЕДАЧА АРГУМЕНТОВ ФУНКЦИИ ПОТОКА	469
14.4. СОЗДАНИЕ ПОТОКА НА ОСНОВЕ ФУНКТОРА	471
14.5. СОЗДАНИЕ ПОТОКА НА ОСНОВЕ МЕТОДА КЛАССА	474
14.6. ВРЕМЕННАЯ ПРИОСТАНОВКА ПОТОКОВ	477
14.7. СИНХРОНИЗАЦИЯ ПОТОКОВ	480
14.8. ИДЕНТИФИКАЦИЯ ПОТОКОВ	484

ГЛАВА 15. ИНФОРМАЦИЯ К РАЗМЫШЛЕНИЮ 491

15.1. СТРУКТУРЫ	492
15.2. АЛЬТЕРНАТИВНОЕ НАЗВАНИЕ ДЛЯ ТИПА	495
15.3. ПЕРЕЧИСЛЕНИЯ	497
15.4. ВЫЗОВ КОНСТРУКТОРА В КОНСТРУКТОРЕ	501
15.5. ФАБРИКА ОБЪЕКТОВ	503
15.6. ДИНАМИЧЕСКАЯ ИДЕНТИФИКАЦИЯ ТИПОВ	507
15.7. ВИРТУАЛЬНЫЕ ДЕСТРУКТОРЫ	514
15.8. ЦИКЛ ПО КОЛЛЕКЦИИ	518

15.9. АВТОМАТИЧЕСКОЕ ОПРЕДЕЛЕНИЕ ТИПА	521
15.10. ОСОБЕННОСТИ ПЕРЕГРУЗКИ ОПЕРАТОРА ПРИСВАИВАНИЯ .	523
15.11. ПЕРЕГРУЗКА ОПЕРАТОРА ПРИВЕДЕНИЯ ТИПА.....	527
ГЛАВА 16. ЗАКЛЮЧЕНИЕ.....	531
16.1. О ЯЗЫКАХ ПРОГРАММИРОВАНИЯ	532
16.2. ПРИЛОЖЕНИЯ С ГРАФИЧЕСКИМ ИНТЕРФЕЙСОМ.....	533
16.3. ПРОГРАММИРОВАНИЕ И ЖИЗНЬ	534

ВВЕДЕНИЕ



Вступление

Язык C++ и объектно-ориентированное программирование

Замечательная идея! Что ж она мне самому в голову не пришла?

из к/ф "Ирония судьбы или с легким паром"

Представленная книга - о языке программирования C++. А еще эта книга - об объектно-ориентированном программировании (сокращенно ООП). Это если коротко. Если не очень коротко - то потребуются пояснения.

Объектно-ориентированное программирование

- Нет, ей об этом думать еще рано.

- Об этом думать никому не рано, и никогда не поздно.

из к/ф "Кавказская пленница"

Объектно-ориентированное программирование - особый, специфический способ организации программного кода. Принципы ООП универсальны и не имеют отношения к какому-то определенному языку программирования. С равным успехом говорят об ООП, например, в C++, Java или C#. Список можно продолжить и другими языками программирования.

В книге на простых и показательных примерах проиллюстрировано, как принципы ООП применяются при составлении программных кодов на языке C++. Имеет смысл "очертить" основные идеологические моменты, которые характерны именно для ООП.

Есть три принципа, положенных в основу ООП: *инкапсуляция*, *полиморфизм* и *наследование*. Стандартный программный код представляет собой последовательность команд (или инструкций), которые выполняются одна за другой. В этом смысле объектно-ориентированная программа мало чем отличается от обычной программы. Но отличия, разумеется, есть. Чтобы их понять, следует учесть, что программный код нужен не сам по себе, а для обработки данных. Данных может быть много или мало - это не принци-

ально. Важно, что они есть, и программа предназначена для работы с этими данными. Формально программный код делится на две категории, или два типа: есть код для *реализации данных*, и есть код для *обработки данных*. Способ организации программы - это вопрос "взаимодействия" кодов разных типов. При обычном стиле программирования (которое иногда называется процедурным или структурным) разные типы кода "существуют" независимо друг от друга, а их "объединение" происходит по мере необходимости обычно на финальной стадии создания программы. Здесь нет ничего плохого, но такой подход оправдывает себя при составлении относительно небольших программ. Если программа большая по размеру, ее становится все труднее "упорядочить". Это очевидная и в некотором смысле банальная проблема, связанная с объемом программного кода. Понятно, что чем больше программа содержит команд, тем сложнее их не то что оптимизировать, но и просто добиться корректной работы. Другими словами, даже у хорошего программиста есть предел восприятия. С одной стороны - это проблема программиста. С другой стороны данная проблема системная, и решать ее нужно системно. Системный подход, предназначенный в первую очередь для решения проблемы эффективного (и, главное, понятного) способа организации программного кода предлагается в рамках парадигмы ООП.



На заметку

Важный момент: необходимость в появлении ООП обусловлена не проблемами с программным или аппаратным обеспечением, а ограниченными человеческими возможностями.

Как отмечалось выше, ООП - особый способ организации программного кода. Сводится он к тому, что код разбивается на блоки. В каждом блоке объединяется программный код, через который реализуются данные, и программный код, предназначенный для работы с этими данными. Такие блоки программы могут "взаимодействовать" между собой и называются *объектами*. Принцип, согласно которому данные и код для их обработки объединяются в одно целое, называется *инкапсуляцией*. Это первый и наиважнейший принцип ООП.



На заметку

В ООП объединение кода, реализующего данные, и кода, предназначенного для обработки данных, выполняется на начальных этапах создания программы.

В программном коде принцип инкапсуляции реализуется через классы и объекты. *Класс* - это шаблон, по которому создаются объекты. Неплохая аналогия для класса - чертеж некоторого изделия, по которому затем выполняется реальный объект, или, например, план дома, на основании которого строится целый микрорайон из однотипных зданий.



На заметку

Наличие класса не означает создание объекта. Класс - штука воздушная, в некотором смысле бестелесная. Сам по себе класс - всего лишь схема, шаблон. По этому шаблону создается нечто реальное - объект. Объектов может быть несколько, а может не быть вовсе. Создание класса не обязывает нас создавать объект.

Даже если объекты создаются по одному шаблону (то есть создаются на основании одного класса), это все равно разные объекты. У них одинаковый набор характеристик (как количество окон в однотипных домах), но значения параметров персонализированы для каждого объекта (точно так же, как различаются окна в разных зданиях).

Важный механизм ООП - *наследование*. Название механизма достаточно точно соответствует его сути. Благодаря наследованию новые классы можно создавать не на пустом месте, а на основе уже существующих классов. Такой подход исключительно удобен, поскольку, во-первых, отпадает необходимость вносить изменения в программный код уже существующих классов (а это повышает уровень совместимости разных версий программного кода), а, во-вторых, экономится время и усилия на создание новых, "усовершенствованных" классов.

Полиморфизм накладывает на объектно-ориентированный код требование поддержки универсальных, однотипных интерфейсов. Суть этого принципа легче и лучше объяснять на простых аналогиях. Например, дверных замков в природе существует огромное множество. Но чтобы открыть или закрыть замок, достаточно вставить в замок ключ и провернуть его. При этом нас мало интересует "внутренняя начинка" замка. Другой пример - система управления автомобилем. Есть стандартный набор педалей, руль и прочие элементы управления. Принципы управления автомобилем универсальны и мало зависят от того, как автомобиль устроен "внутри". Новые автомобили создаются так, чтобы принцип управления ими оставался неизменным (если, конечно, речь не идет о революционных изменениях в автомобилестроении). В плане программирования, когда речь заходит о полиморфизме, то обычно вспоминают *перегрузку* и *переопределение* методов, с которыми мы познакомимся в основной части книги. Здесь лишь отметим, что благодаря перегрузке и переопределению удастся использовать методы с одинаковыми названиями, которые позволяют выполнять разные (но обычно однотипные) действия.

На первый взгляд все описанное выше может показаться запутанным и сложным. Однако легко заметить и общую тенденцию: основные усилия направлены на то, чтобы свести к минимуму объем необходимого программного кода и сделать его наиболее универсальным и структурированным. Важно и то, что ООП - подход, предназначенный для написания серьезных программ. Это как авианосец. Он большой и мощный. Он очень хорош для того, чтобы контролировать морские пути, например. Но плавать на нем по речке - идея плохая. Так и с ООП. Принципы ООП хороши при работе с большими и сложными проектами. Если программа небольшая и несложная, нет необходимости использовать ООП.



На заметку

Пикантность ситуации в том, что нам предстоит осваивать премудрости ООП на относительно несложных проектах. Но эта задача решаемая.

Особенности языка C++

Я артист больших и малых академических театров.

из к/ф "Иван Васильевич меняет профессию"

Язык C++ хорош во всех отношениях. Это современный, гибкий и мощный язык программирования, который является визитной карточкой любого уважающего себя программиста. Язык профессиональный в полном смысле этого слова. У него "лаконичный", понятный и удобный синтаксис. Если проследить историю развития и трансформации языка C в язык C++, то легко понять, что C++ есть ни что иное, как усовершенствованный язык C, "расширенный" для поддержки парадигмы ООП. С этой точки зрения нет никаких сомнений в том, что язык C++ подходит для нашего важного и сложного дела - изучения принципов программирования с использованием классов и объектов. Вместе с тем, используемый в книге подход не самый "классический". Обычно аргументом к изучению языка C++ служит то, что он "переходной": в C++ можно писать как обычные (не объектно-ориентированные программы), так и создавать полноценные объектно-ориентированные приложения.



На заметку

На первый взгляд может показаться, что "переходной" характер языка программирования вполне естественен. Тем не менее, это не так. Упомянутые ранее языки программирования Java и C# являются полностью объектно-ориентированными. При программировании на этих языках, даже при написании самой маленькой программы, придется придерживаться принципов ООП, и, соответственно, описать, по крайней мере, один класс.

Другими словами, при работе в C++ классы и объекты можно использовать, а можно не использовать. Все зависит от намерений программиста, конечной цели и конкретики решаемой задачи.

Изучение языка программирования C++ нередко подается в формате "от простого к сложному", то есть сначала излагаются "обычные" методы программирования в C++, а потом описываются классы, объекты и сопутствующие им технологии и конструкции. Мы отойдем от этой традиции и сразу начнем с рассмотрения объектно-ориентированных кодов. Это позволит, во-первых, сконцентрироваться на наиболее важных моментах, и, во-вторых, сократить объем учебного материала, существенно не понижая уровень изложения и охват основных тем.



На заметку

Закон Парето утверждает, что в деятельности любого рода 20% усилий приносят 80% результата, и остальные 80% усилий дают лишь 20% результата. Мы попытаемся "локализовать" те минимальные усилия, которые позволят в полной мере раскрыть потенциал и языка C++, и принципов ООП.

Однако не только классы и объекты будут предметом нашего изучения. В C++ есть много важных, полезных и эффективных конструкций, без знания и свободного владения которыми программирование в C++ представляется проблематичным. Важным вопросам, которые напрямую не относятся к ООП, но имеют непосредственное отношение к C++, в книге уделяется должное внимание.

Наши методы

*Пойдем простым логическим ходом.
из к/ф "Ирония судьбы или с легким паром"*

При изучении любого курса (не обязательно имеющего отношения к программированию) всегда есть одна дилемма - материала обычно много, а времени на его изучение обычно мало. Если мы говорим о книге с претензией на учебник, то проблема остается актуальной.

Научиться программировать получится, только если теория подкрепляется практикой. Причем с уверенностью можно утверждать, что первостепенным при изучении любого курса программирования являются практические навыки. Их нужно развивать и укреплять. Книга в этом деле может быть помощником. Хорошая книга может быть путеводителем - не более. Есть еще конечно, справочники, толстые и информативные. Но справочник и учебник - книги разные. В справочнике важен объем, полнота и достовер-

ность информации. Учебник характеризуется той пользой, которую он дает читателю.

Мы не ставим перед собой целью узнать все о языке C++ или ООП. Наша цель в ином - дать мощный толчок в изучении принципов ООП вообще, и языка C++ в частности. Еще мы определим общее направление, в котором разумно двигаться в дальнейшем. Однако сказанное не означает, что книга будет неполной или в каком-то смысле ущербной. Читатель научится создавать полноценные объектно-ориентированные программы. Мы рассмотрим все основные и наиболее важные конструкции C++, так что при желании читатель сможет создавать и обычные (не объектно-ориентированные) программы. Но случится это не само по себе. Книгу мало прочитать. С книгой нужно работать. В этом случае успех придет.

Учебный материал в книге подается исходя из того, что читатель не знаком ни с принципами ООП, ни с языком C++. Другими словами, априори предполагается, что читатель в C++ не программировал. Вместе с тем, книга будет полезной и тем, кто знаком с языком C++.



На заметку

При обсуждении синтаксических конструкций и утилит будут анализироваться наиболее важные моменты. Второстепенные детали мы нередко будем оставлять "за бортом". Это как раз то минимальное и неизбежное "зло", с которым придется смириться. Оправданием нам служит убеждение в эффективности данного подхода.

Программное обеспечение и стандарты языка

*Фигуры, может, и нет, а характер - налицо.
из к/ф "Девчата"*

Важный вопрос - программное обеспечение, которое будет или может быть использовано для составления и компиляции программных кодов. Далее мы поговорим именно об этом. Важно понимать, что выбор программного обеспечения, кроме прочего, связан также с поддержкой определенного стандарта языка C++.



На заметку

Программа, написанная на языке C++, должна соответствовать определенным правилам. Существует стандарт языка C++, который определяет основы синтаксиса и ряд смежных положений: базовые синтаксические конструкции, алгоритмы

и подходы, допустимые в языке. Программы, которые переводят (в том или ином виде) написанный в соответствии со стандартом языка C++ программный код в набор команд, понятных для компьютера (точнее, операционной системы или иной программы-посредника) называются компиляторами.

Непосредственно программный код можно набрать хоть в текстовом редакторе. Здесь мы практически не ограничены в возможностях. Затем код нужно *компилировать*. Для этого используется программа-компилятор. Компиляторов много и среди них есть как некоммерческие (свободно распространяемые), так и коммерческие. Проблема в том, что далеко не все компиляторы полностью поддерживают стандарт языка C++: некоторые "правильные" с точки зрения стандарта языка синтаксические конструкции компилятору могут быть "непонятны". В этом смысле выбор компилятора имеет значение. При выборе компилятора нелишне просмотреть информацию из его справочной системы.



На заметку

Достаточно долго существовал стандарт языка C++ от 2003 года. Новая существенная модификация стандарта произошла в 2011 году, а в 2014 году в стандарт языка C++ были внесены некоторые уточнения. По большому счету, изменения сводятся к расширению возможностей языка и добавлению новых синтаксических конструкций.

На момент написания книги последним утвержденным стандартом языка C++ является стандарт от 2014 года. Однако самые интересные новшества появились в стандарте 2011 года. Так что при разработке приложений можно смело ориентироваться на этот стандарт. Кроме того, следует учесть, что далеко не все компиляторы соответствуют не то что стандарту 2014 года, но даже стандарту 2011 года. Причина банальная - вносить "юридические" изменения в стандарт намного проще и быстрее, чем реализовать их в таком программном продукте, как компилятор. Со временем, разумеется, данная проблема будет решена.

В любом случае, перечисленные выше обстоятельства не должны смущать, поскольку стандарт 2003 года, поддерживаемый большинством компиляторов, "перекрывает" основы синтаксиса языка C++. Соответственно, материал книги в основной своей массе подходит для большинства наиболее популярных компиляторов. В тех случаях, когда программный код содержит существенные "новинки", для восприятия которых нужен наиболее продвинутый компилятор, в книге даются по этому поводу пояснения.

При разработке программного кода удобно пользоваться программами, которые называются *средами разработки* (часто используется аббревиатура *IDE* от английского *Integrated Development Environment*). Среда разработки - это на самом деле несколько программ (как минимум редактор кодов и компилятор) в одном "флаконе".



На заметку

Среда разработки - намного лучше, чем просто компилятор. Это целый набор утилит, среди которых компилятор является одним из компонентов. Обычно к услугам разработчика - специальный редактор, облегчающий набор программного кода, а также средства отладки, тестирования, компоновки и ряд других компонентов.

Существует много сред разработки, в том числе и распространяемых на некоммерческой основе. Мы полный обзор делать не будем. Остановимся лишь на трех: кратко рассмотрим среды разработки *Dev C++* и *Microsoft Visual Studio Express*, и еще рассмотрим методы работы со средой *NetBeans*.



На заметку

Стоит отметить, что все эти среды бесплатные и могут быть свободно установлены на компьютер пользователя.

Среда разработки Dev C++

*Чего не надо, того не сделают.
из к/ф "Покровские ворота"*

Среда разработки Dev C++ распространяется свободно: установочные файлы можно загрузить на сайте www.bloodshed.net компании *Bloodshed Software*. Страница загрузки установочных файлов среды разработки показана на рис. В.1.



Рис. В. 1. Страница загрузки среды разработки Dev C++

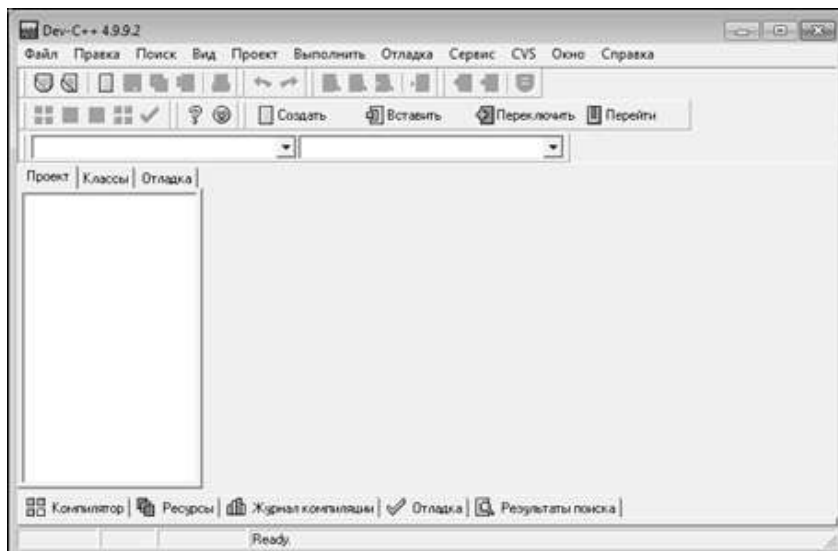


Рис. В.2. Окно среды разработки Dev C++

Среда поддерживает многоязыковой интерфейс (в том числе и русскоязычный). На рис. В.2 показано окно приложения Dev C++ (с русскоязычным интерфейсом).

Методы работы со средой Dev C++ описывать не будем. Во-первых, не это есть предмет нашей книги. Во-вторых, надо признать, что элементы настройки Dev C++ просты и понятны. Даже если нет четкого представления о назначении того или иного элемента, на интуитивном уровне обычно понятна его "миссия". Кратко опишем лишь процесс создания нового проекта.

Чтобы создать новый проект (новую программу), в меню **Файл** выбираем команду-меню **Создать**, и в раскрывающемся списке выбираем команду **Проект** (рис. В.3).

В результате открывается окно с названием **Новый проект**, в котором необходимо задать тип проекта и его название (рис. В.4).

Название проекта вводится в поле **Имя**, а для создания консольного приложения в верхней части окна на вкладке **Basic** следует выбрать пиктограмму **Console Application** (см. рис. В.4).



На заметку

Мы будем рассматривать программы, ввод и вывод информации в которых реализуется через консольное окно (или окно вывода, имитирующее консольное окно). Такие приложения называются консольными.

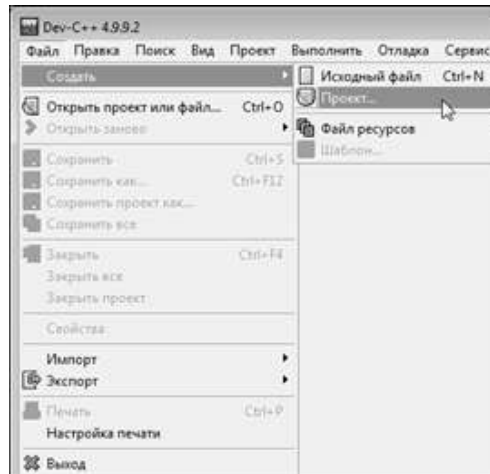


Рис. В.3. Начало создания нового проекта

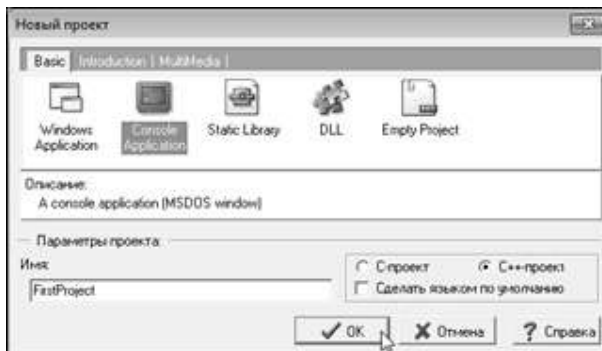


Рис. В.4. Создание консольного проекта Dev C++

После того, как тип и название приложения определены, следует сохранить проект. Место сохранения проекта выбирается с помощью диалогового окна **Create new project**, как показано на рис. В.5.

После создания проекта в окне редактора набираем программный код. На рис. В.5 показано окно среды разработки Dev C++ с несложной программой.

Программный код в окне редактора кодов приложения Dev C++ такой:

```
#include <iostream>
using namespace std;
int main() {
    cout<<"We use Dev-C++"<<endl;
```

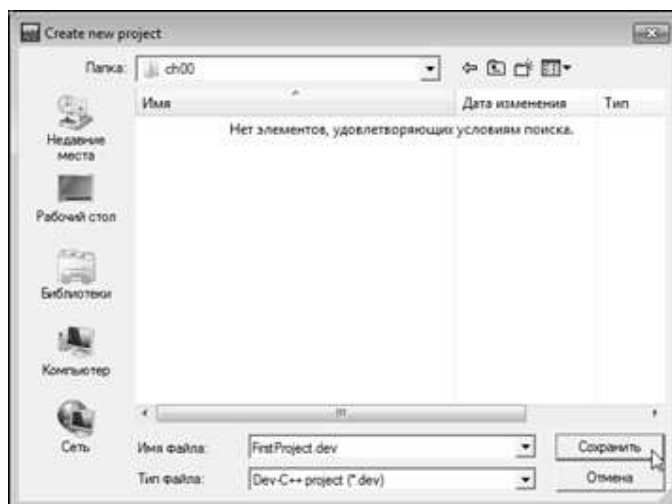


Рис. В.5. Задаем место сохранения проекта Dev C++

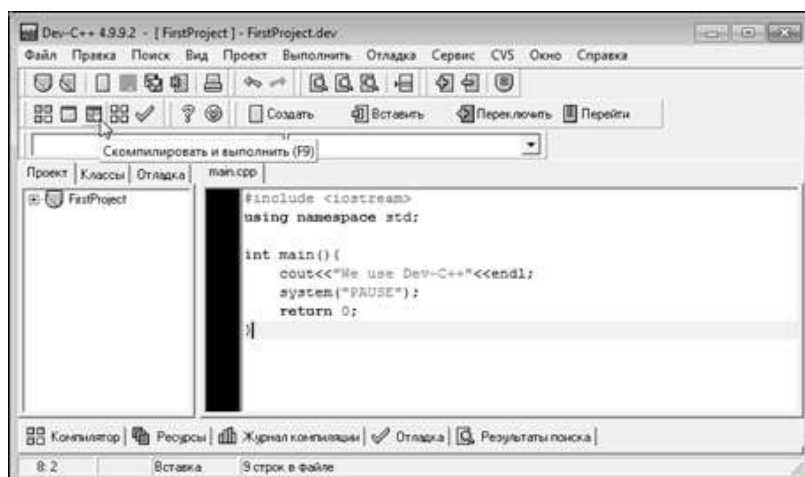


Рис. В.6. Окно среды разработки Dev C++ с программным кодом

```
system ("PAUSE");
return 0;
}
```

Для компиляции программы и запуска ее на выполнение достаточно выполнить одно из следующих действий:

- нажать клавишу <F9> на клавиатуре;

- щелкнуть пиктограмму, соответствующую команде компилирования и запуска на выполнение программы (по умолчанию данная пиктограмма третья слева на второй сверху панели меню, как показано на рис. В.6);
- воспользоваться командой **Скомпилировать и выполнить** в меню **Выполнить**.

В результате выполнения программы (если программный код не содержит ошибок и удачно откомпилирован) открывается консольное окно с сообщением We use Dev-C++ (рис. В.7).

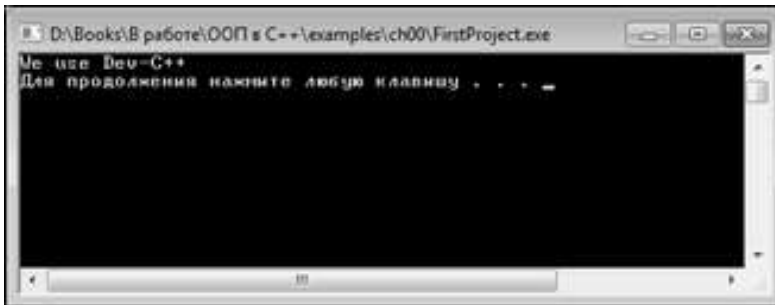


Рис. В.7. Результат выполнения консольного проекта Dev C++

Подробности

Если кому-то интересен программный код проекта (см. рис. В.6), то он достаточно простой и содержит всего несколько команд. Мы такого типа команды будем обсуждать в основной части книги. Сейчас же просто дадим краткое описание с минимальными пояснениями. Итак, инструкцией `#include <iostream>` подключается библиотека стандартного ввода-вывода. Команда `using namespace std` означает, что используется стандартное пространство имен. Непосредственно выполняемый программный код содержится в главной функции, которая называется `main`. Ключевое слово `int` перед названием функции свидетельствует о том, что функция в качестве результата возвращает целое число. Пустые круглые скобки после имени функции означают, что у функции нет аргументов. Программный код функции указывается внутри блока, ограниченного парой фигурных скобок (то есть между `{` и `}`). Команда `cout<<"We use Dev-C++"<<endl` представляет собой инструкцию вывода в консоль текста `We use Dev-C++` с последующим переходом к новой строке. Команда `system("PAUSE")` в данном случае нужна для того, чтобы консольное окно не пропадало с экрана сразу после вывода сообщения (иначе все произойдет настолько быстро, что мы ничего не успеем заметить). Инструкция `return 0` означает, что работа программы завершается с кодом `0` (нормальное завершение работы программы).

Вообще стоит отметить, что среда разработки Dev C++ простая, компактная, легко настраивается и свободно распространяется (что немаловажно). В общем и целом - выбор неплохой.



На заметку

Идеализировать среду разработки Dev C++ тоже особо не стоит. И за ней имеются некоторые "грешки". Например, в Dev C++ в качестве размера статического массива допускается указывать переменную (а не константу), что, мягко говоря, не очень соответствует стандартам языка C++. Хотя такой демократизм серьезно упрощает жизнь, следует понимать, что это уже будет не совсем стандартный программный код и при использовании другой среды разработки может появиться ошибка.

Среда разработки Microsoft Visual Studio Express

*И все-таки, поверьте историк: огорчить против желания нельзя.
из к/ф "Покровские ворота"*

У компании Microsoft есть коммерческий продукт *Visual Studio*. Более простая версия называется *Visual Studio Express* и распространяется бесплатно (но при условии регистрации). Лицензия *Visual Studio Express* предусматривает, что конечным пользователем продукт используется в некоммер-

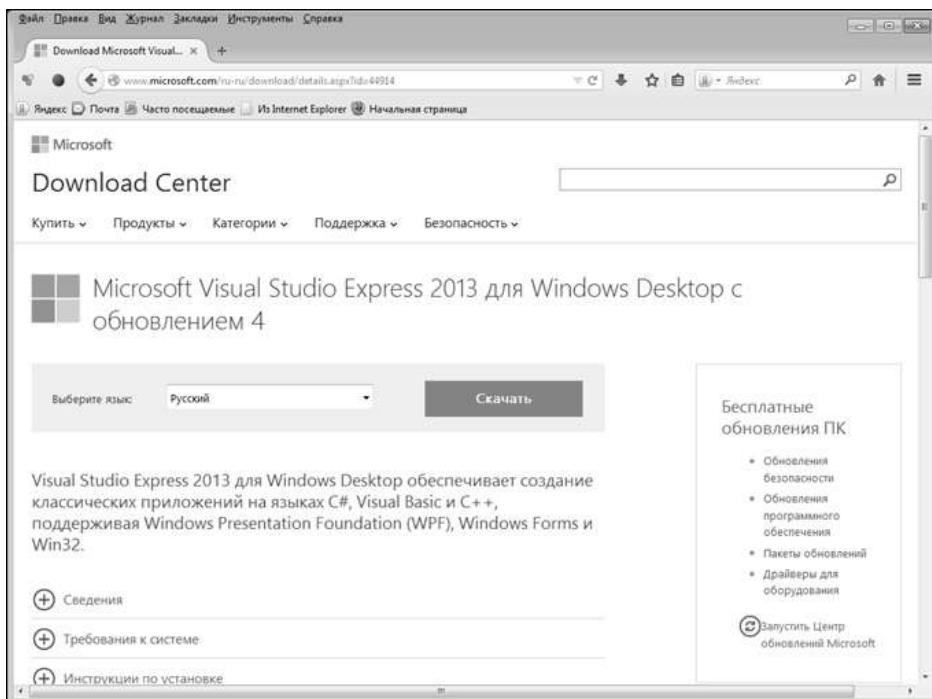


Рис. В.8. Окно загрузки Microsoft Visual Studio Express 2013

ческих целях (желающие могут уточнить юридические моменты на сайте www.microsoft.com компании Microsoft).

Для выхода на страницу загрузки среды Visual Studio Express имеет смысл начать "путешествие" с официальной страницы Microsoft или сразу выйти на сайт центра загрузок www.microsoft.com/ru-ru/download, а уже оттуда по гиперссылкам перейти на страницу загрузки приложения. На рис. В.8 показана страница загрузки приложения *Microsoft Visual Studio Express 2013*.



На заметку

Время от времени условия загрузки и регистрации приложения Microsoft Visual Studio Express меняются, поэтому не исключено, что читателю придется проявить некоторую изобретательность для того, чтобы стать счастливым пользователем данного программного продукта.

После запуска приложения открывается окно, подобное представленному на рис. В.9.

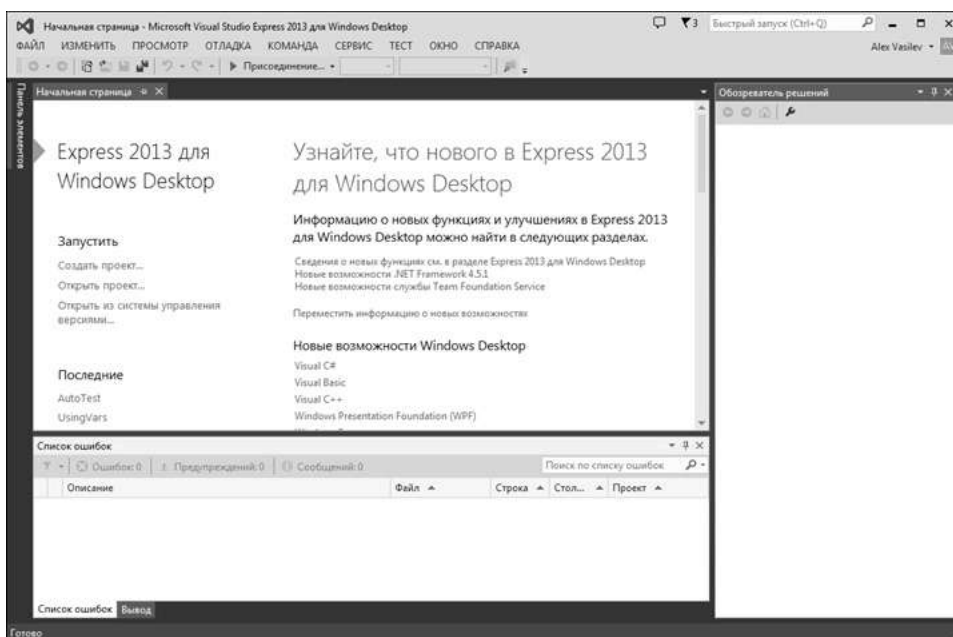


Рис. В.9. Окно приложения Microsoft Visual Studio Express 2013

Для создания нового проекта в среде разработки Microsoft Visual Studio Express в меню **Файл** выбираем команду **Создать проект** (рис. В.10).

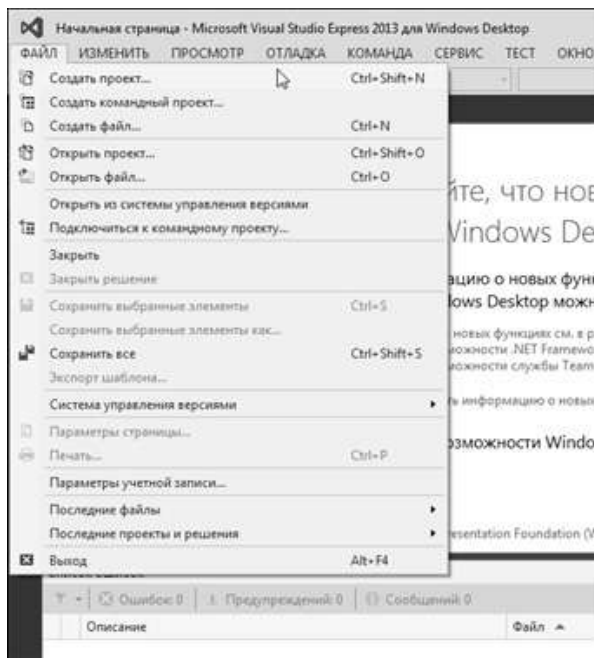


Рис. В.10. Создание нового проекта в среде Microsoft Visual Studio Express

Другой способ: на панели инструментов есть специальная пиктограмма. Также новый проект создается нажатием комбинации клавиш <Ctrl>+<Shift>+<N>.

При создании нового проекта откроется окно **Создать проект**, в котором следует задать тип приложения - например так, как показано на рис. В.11.

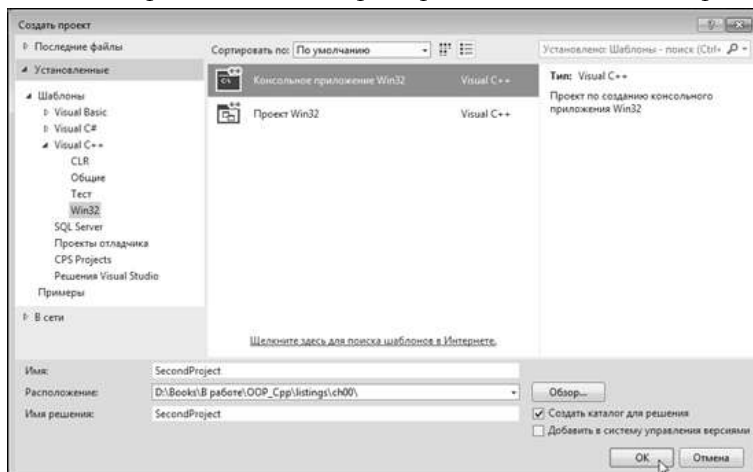


Рис. В.11. Определяем тип приложения

В поле **Имя** указывается название для приложения, а в поле **Расположение** - место для сохранения файлов проекта (имеется ряд других настроек, с которыми, хочется верить, читатель без труда разберется). В процессе подтверждения настроек появится еще несколько окон. Одно из них показано на рис. В.12.

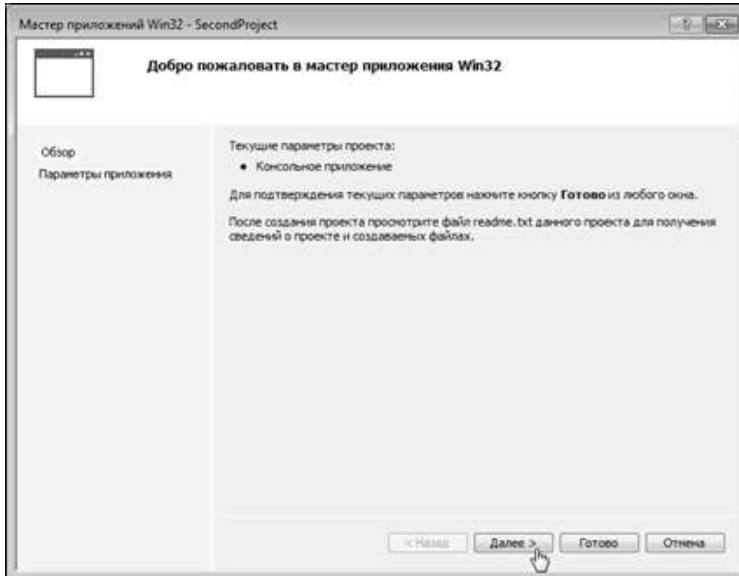


Рис. В.12. Создается консольное приложение

На рис. В.13 показано окно, в котором выполняется ряд дополнительных настроек.



На заметку

Имеет смысл обратить внимание на опцию Предварительно скомпилированный заголовок. По умолчанию эта опция установлена. Если так, то в шапке программы должна быть инструкция `#include "stdafx.h"`. Здесь и далее мы будем исходить из того, что при создании приложения данная опция отменена.

Наконец открывается окно редактора программных кодов для созданного проекта. На рис. В.14 показано, как может выглядеть окно среды разработки Microsoft Visual Studio Express с введенным программным кодом перед запуском проекта на выполнение.

В окне редактора содержится следующий программный код:

```
#include <iostream>
using namespace std;
```

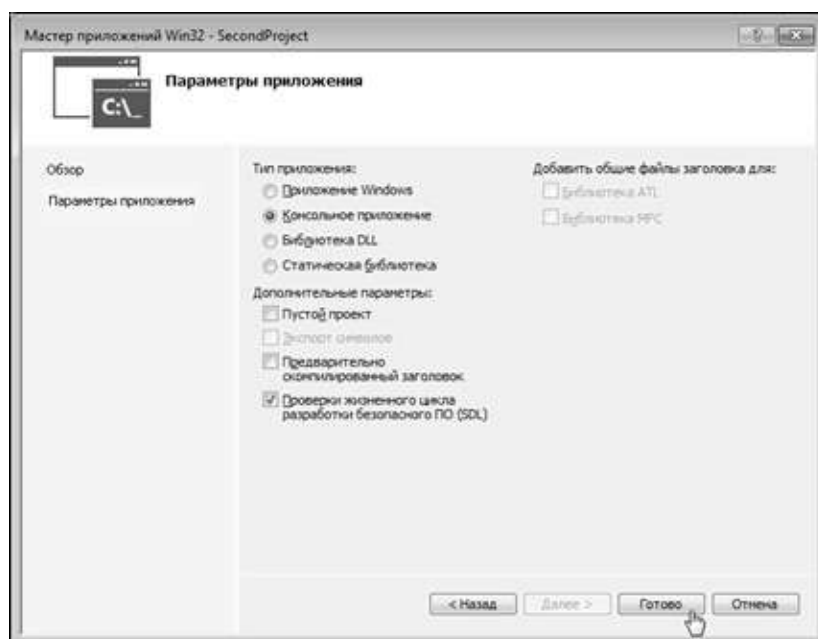


Рис. В. 13. Дополнительные параметры консольного приложения

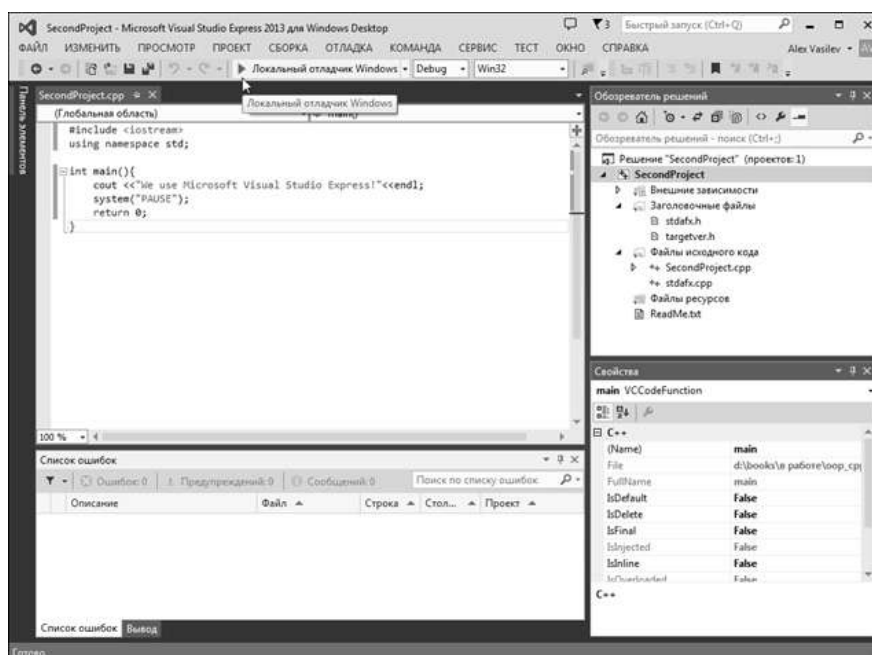


Рис. В. 14. Окно среды разработки Visual Studio Express 2013 перед запуском проекта на выполнение

```
int main() {
cout<<"We use Microsoft Visual Studio Express!"<<endl;
system ("PAUSE");
return 0;
}
```



На заметку

Программный код практически такой же, как и в разделе, в котором рассматривалась среда разработки Dev C++. Изменился только отображаемый при выполнении программы текст в консольном окне.

Для запуска программы на выполнение (с предварительной компиляцией, разумеется) можем воспользоваться пиктограммой с зеленой стрелкой на панели инструментов (см. рис. В.14), нажать клавишу <F5> на клавиатуре или прибегнуть к помощи команды **Начать отладку** в меню **Отладка** (рис. В.15).

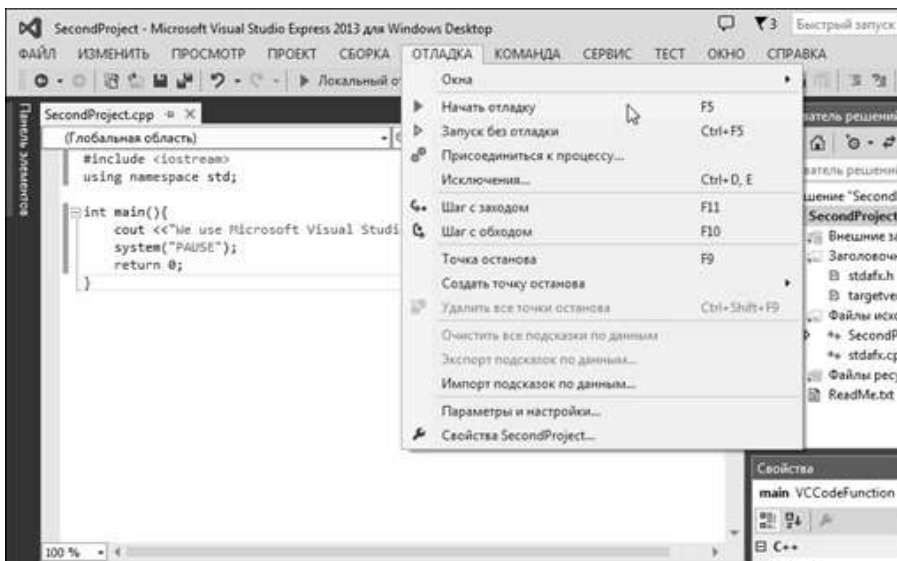


Рис. В.15. Запуск проекта на выполнение с помощью команды меню

В результате выполнения программы открывается консольное окно с сообщением `We use Microsoft Visual Studio Express!` (рис. В.16).

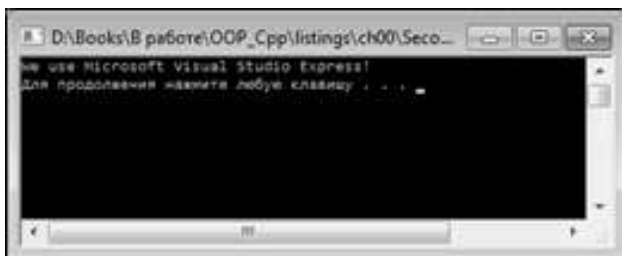


Рис. В.16. Результат выполнения программы

**На заметку**

Обратите внимание, что среда разработки Microsoft Visual Studio Express позволяет создавать приложения не только на языке C++, но и, например, на языках C# и Visual Basic.

Собственно, о среде разработки Microsoft Visual Studio Express можно рассуждать много. В наши планы это не входит. Наши планы больше связаны со средой разработки *NetBeans*.

Среда разработки NetBeans

*На всех языках как птица поет.
из к/ф "Покровские ворота"*

Откровенно говоря, основное предназначение среды разработки *NetBeans* - написание проектов на языке программирования Java. Вместе с тем, среда позволяет создавать проекты еще на нескольких языках, среди которых есть и язык C++. Примеры из книги тестировались именно в среде разработки NetBeans. Среда NetBeans многофункциональна и достаточно удобна в работе (хотя это, конечно, субъективное мнение). Но не это главное. Для разработки (и тестирования) примеров среда разработки NetBeans была выбрана, потому что:

- при работе с этой средой без особых усилий можно выводить кириллический текст в консоль;
- среда разработки позволяет использовать компилятор, поддерживающий (полностью или частично) новые стандарты языка программирования C++.

Теперь по каждому пункту "персонально". С выводом кириллического текста в консольное окно проблема такая: кодировка, используемая в консольном окне, отличается от кодировки, используемой в редакторе кодов.

Поэтому если в окне редактора кириллический текст выглядит нормально (читабельно), то в консольном окне появятся несуразные символы. Обратное, если текст вводится в консольном окне и считывается в программе, то нормально выглядящий при вводе текст будет иметь совершенно иной вид при обработке. Еще раз подчеркнем, что причина здесь кроется в различии кодировок редактора кодов и консольного окна. Выход из ситуации может быть таким:

- установить кодировку редактора кодов такую же, как и кодировка консольного окна;
- изменить кодировку консольного окна;
- добавить в программный код инструкции, позволяющие производить ввод и вывод текста в разных кодировках (в соответствии с кодировками консольного окна и редактора).

Каждый из вариантов имеет свои недостатки. В NetBeans эта проблема решается довольно просто. Во-первых, вывод осуществляется не в консольное окно, а в специальное *окно вывода*, что расширяет возможности по настройке параметров вывода. Во-вторых, для проекта можно в явном виде указать кодировку и избежать проблем с некорректным отображением кириллических текстов. Поскольку мы только учимся программировать, для нас такой подход вполне приемлем.

Что касается компилятора, то он устанавливается отдельно от среды NetBeans. Сначала лучше установить компилятор для языка C++, а уже затем среду разработки NetBeans. Это, мягко говоря, не очень удобно. Но здесь имеется и плюс: среда разработки NetBeans поддерживает работу с несколькими типами компиляторов, так что имеется определенный выбор. Далее узнаем, как загрузить среду разработки, компилятор, все это установить и наслаждаться возможностями NetBeans.

Загрузить среду разработки NetBeans можно на странице поддержки проекта www.netbeans.org (рис. В.17).

Переходя по гиперссылке, оказываемся на странице загрузки среды разработки (рис. В.18).

Обычно предлагается несколько вариантов комплектации. Выбрать нужно тот, в котором имеется поддержка C++. Выбираем, загружаем, устанавливаем. Однако как отмечалось выше, перед установкой среды настоятельно рекомендуется установить компилятор.



На заметку

В принципе, компилятор можно установить и потом (после установки NetBeans), но в этом случае придется выполнять настройки для компилятора самостоятельно.

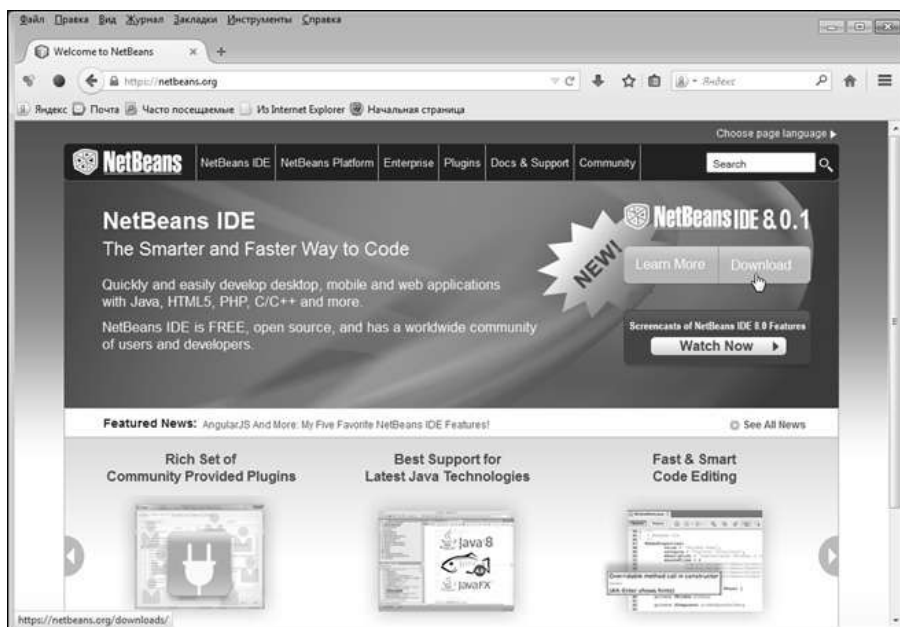


Рис. В.17. Страница поддержки проекта NetBeans

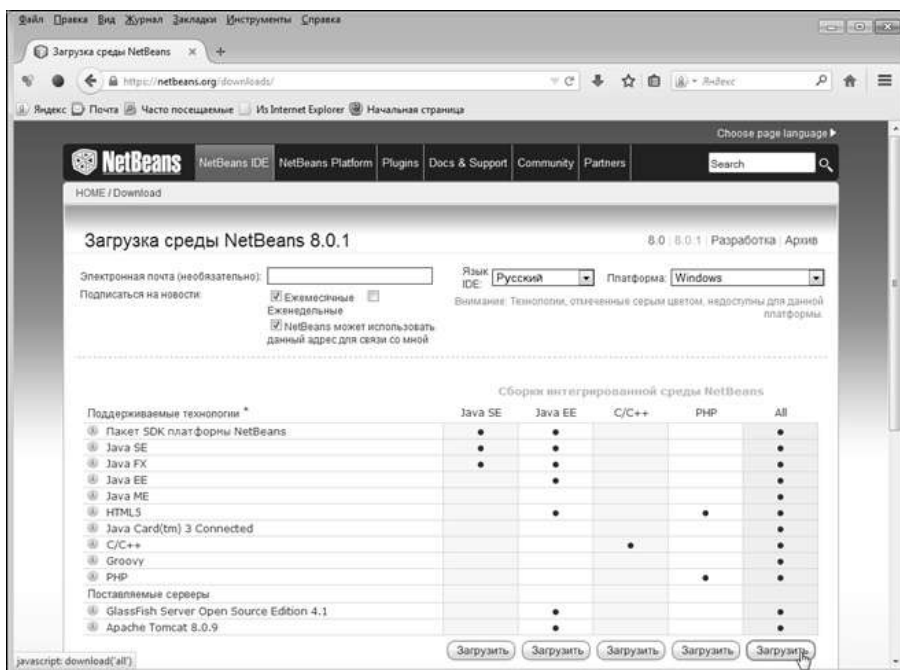


Рис. В.18. Страница загрузки среды разработки NetBeans

Если на момент установки среды NetBeans компилятор уже установлен, то, скорее всего, настройки будут выполнены в полуавтоматическом режиме (параметры определяются автоматически, а пользователю требуется только их подтвердить).

Вариантов с компиляторами достаточно много, но каждый требует индивидуального подхода. Для операционной системы Windows основные компиляторы среды NetBeans - это *Cygwin* и *MinGW*. Страница проекта Cygwin находится по адресу www.cygwin.com и показана на рис. В.19.

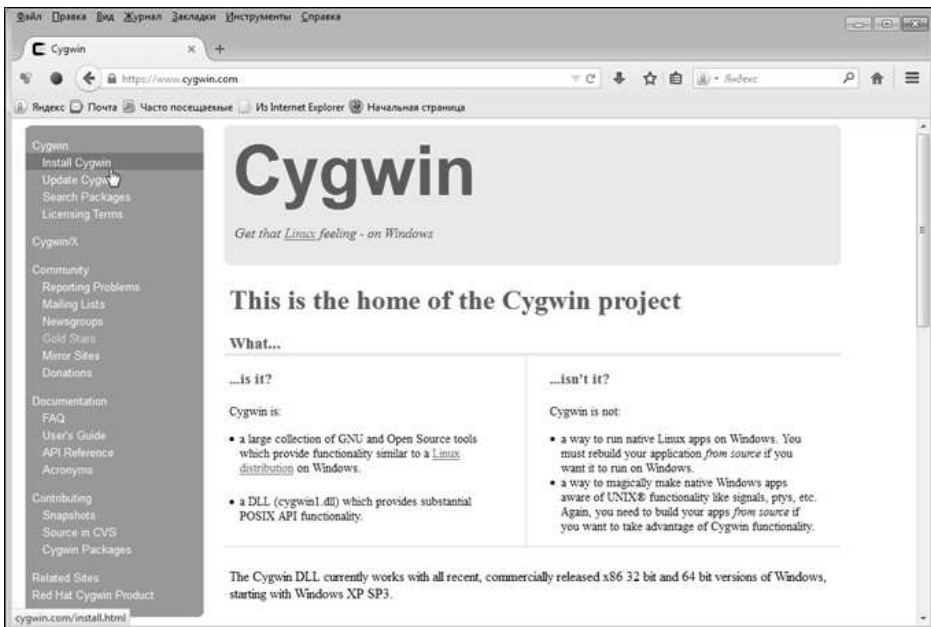


Рис. В.19.Страница проекта Cygwin

Страница проекта MinGW находится по адресу www.mingw.org. Окно браузера, открытое на странице проекта, показано на рис. В.20.

Хочется верить, что установка программных продуктов пройдет без эксцессов. На всякий случай приводим на рис. В.21 окно **Параметры** с настройками среды разработки NetBeans, связанными с используемым компилятором и прочими сопутствующими утилитами. Окно можно открыть, выбрав в меню **Сервис** команду **Параметры**. Сверху в окне в ряду пиктограмм следует выбрать пиктограмму **C/C++**.



На заметку

На самом деле при "установке компилятора" устанавливается не только компилятор, но и какое-то количество вспомогательных программ и библиотек.

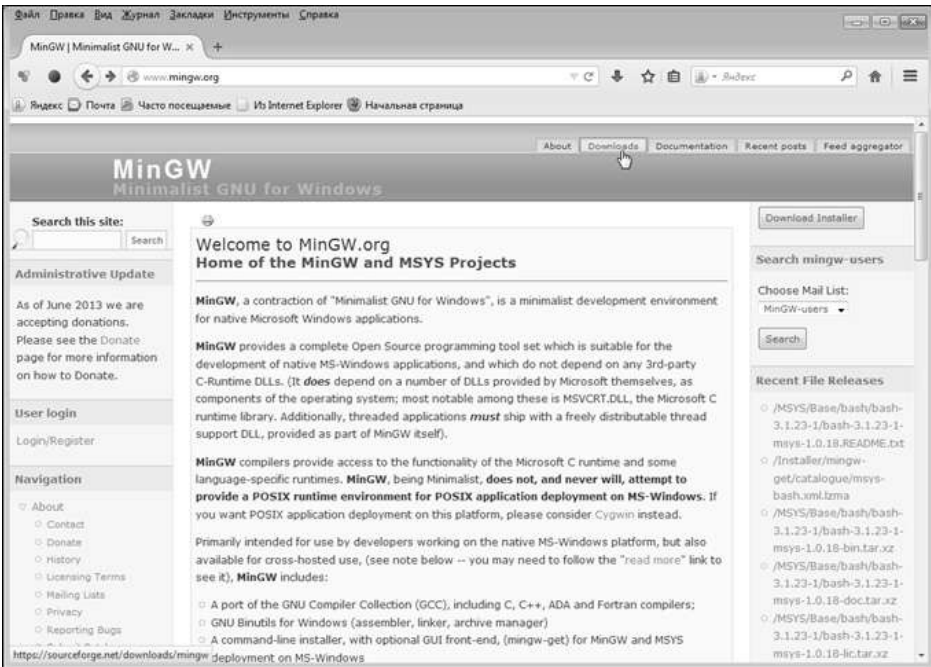


Рис. В.20. Страница проекта MinGW

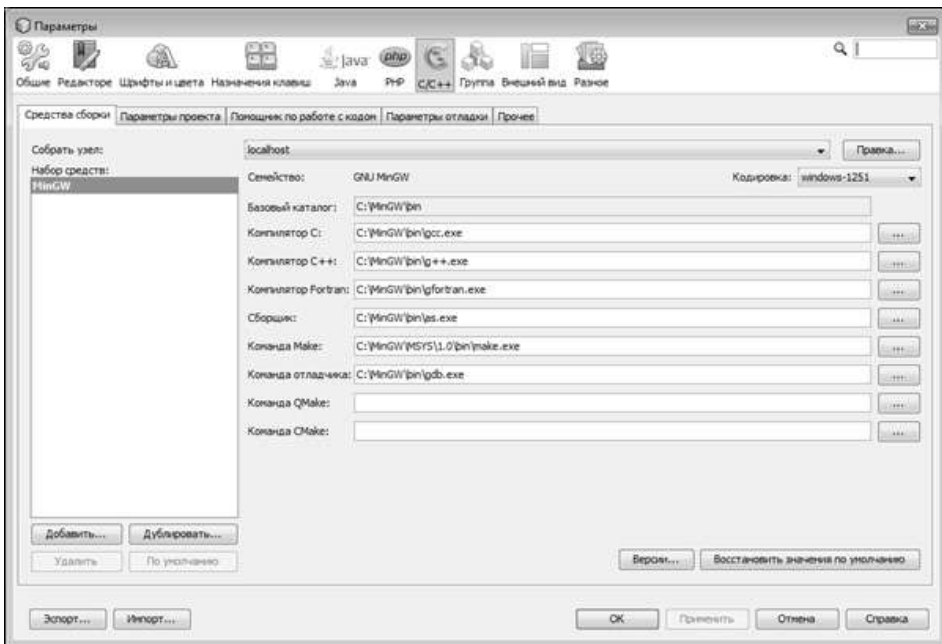


Рис. В.21. Окно настроек Параметры

Не следует пренебрегать информацией (включая справочные страницы) среды разработки NetBeans. На сайте www.netbeans.org можно найти информацию об особенностях настройки приложения для работы с различными типами компиляторов. Это вдвойне актуально, поскольку в новых версиях среды разработки NetBeans принципы выполнения настроек могут меняться.

Наконец, напомним, что среда NetBeans - это всего лишь один из возможных вариантов. При необходимости можно воспользоваться другой средой разработки.

Если приложение NetBeans установлено и все настройки выполнены корректно, мы готовы приступить к работе. Для этого запускаем приложение. Рабочее окно среды NetBeans показано на рис. В.22.

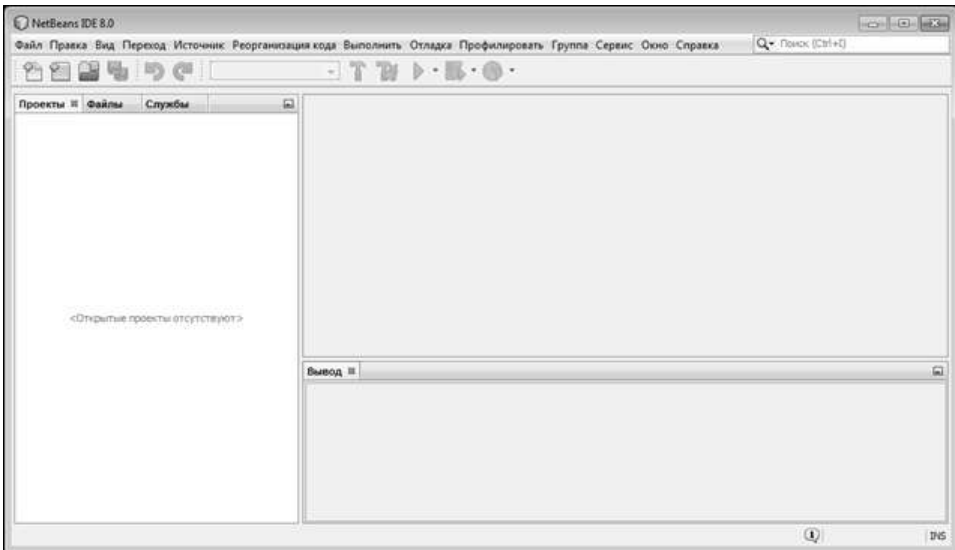


Рис. В.22. Рабочее окно среды разработки NetBeans

Для создания нового проекта в среде NetBeans в меню **Файл** выбираем команду **Создать проект**, как показано на рис. В.23.



На заметку

Также можно нажать комбинацию клавиш <Ctrl>+<Shift>+<N> или щелкнуть пиктограмму для данной команды (пиктограмма с желтой папкой и зеленым крестом).

Откроется окно **Создать проект**, в котором выбирается тип приложения: мы создаем приложение на C++, поэтому в разделе **Категории** выбираем позицию **С/C++**, а в разделе **Проекты** выбираем позицию **Приложение на C/C++** (рис. В.24).

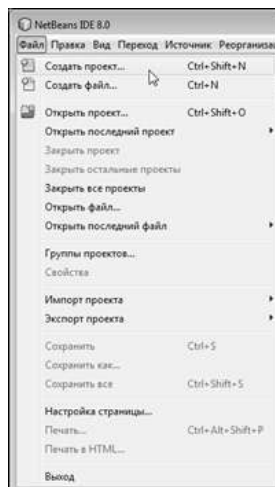


Рис. В.23. Создание нового проекта

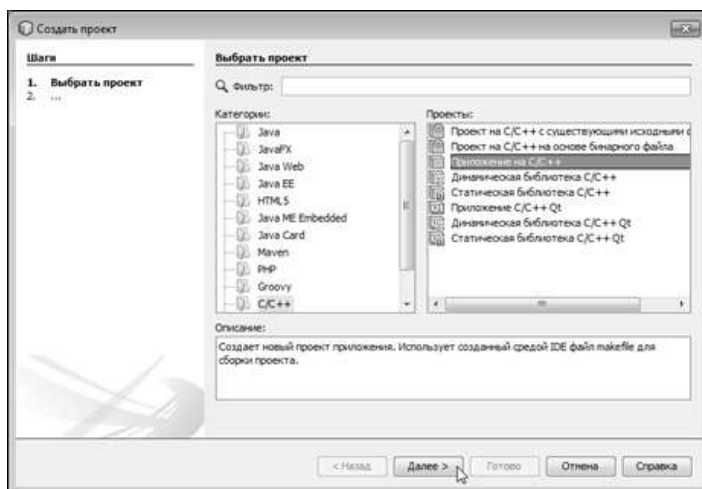


Рис. В.24. Выбор типа приложения

В следующем окне (рис. В.25) задается имя проекта, место сохранения файлов проекта, и выполняется ряд других настроек.

На этом фактически создание проекта как такового завершается. Но мы еще проиллюстрируем, как выполняются две важные настройки проекта, "отвечающие" за кодировку и версию компилятора.

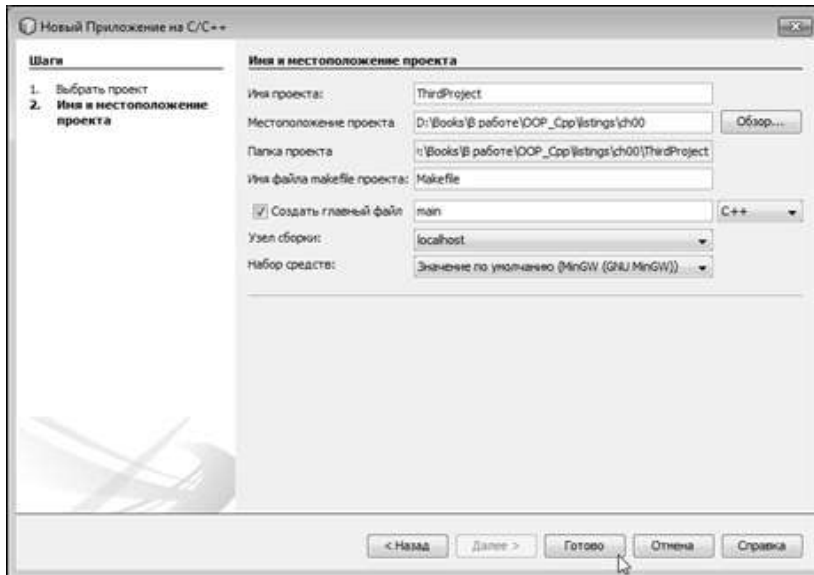


Рис. В.25. Задаем название проекта и место сохранения файлов проекта

Для выполнения указанных настроек выделяем позицию с названием проекта во внутреннем окне **Проекты**, щелкаем правую кнопку мыши и в раскрывшемся контекстном меню выбираем команду **Свойства** (рис. В.26).

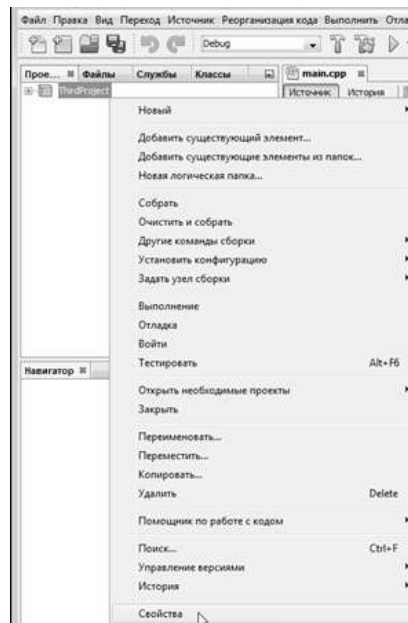


Рис. В.26. В контекстном меню проекта выбираем команду **Свойства**

Откроемся окно свойств проекта (рис. В.27), в котором в разделе **Категории** выбираем позицию **Общего назначения** и в нижней части окна в раскрывающемся списке **Кодировка** выбираем нужную кодировку (в данном случае это кодировка **windows-1251**).

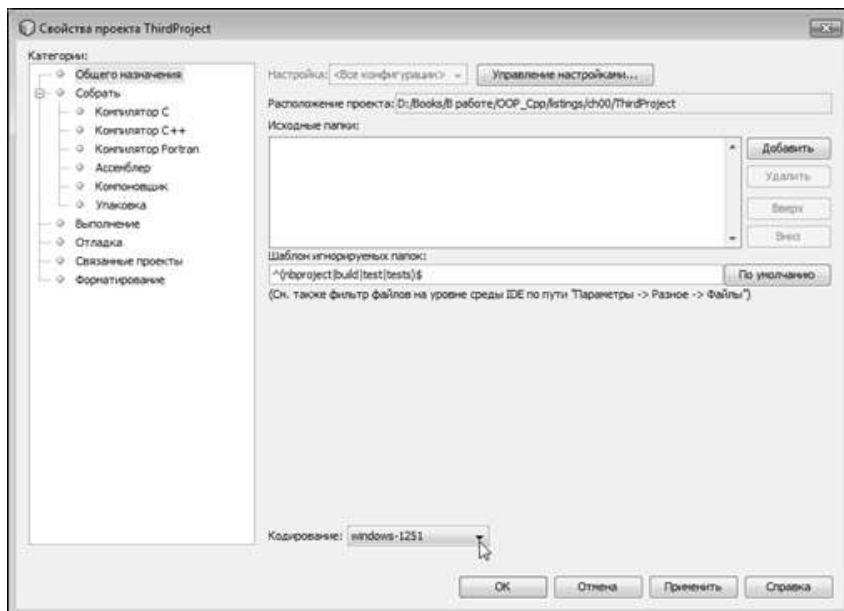


Рис. В.27. В окне свойств проекта определяется кодировка

Стандарт, поддерживаемый компилятором, задается через параметры **C Standard** и **C++ Standard**. На рис. В.28 в разделе **Категории** в группе **Собрать** выделена позиция **Компилятор C**, а в правой части окна напротив поля **C Standard** выбирается значение **C11** (что соответствует стандарту 2011 года).

На рис. В.29 аналогичная процедура выполняется для позиции **Компилятор C++** в группе **Собрать** раздела **Категории**: в раскрывающемся списке напротив поля **C++ Standard** выбирается значение **C++11**.

После выполнения настроек в окне свойств проекта следует щелкнуть кнопку **Применить** (рис. В.30) для немедленного применения этих настроек, и затем подтвердить завершение процесса выполнения настроек, щелкнув кнопку **ОК**.

Итак, проект создан и настройки выполнены. Осталось ввести программный код. Поступаем так (рис. В.31): во внутреннем окне **Проекты** раскрываем позицию с названием проекта, затем раскрываем группу **Исходные файлы**

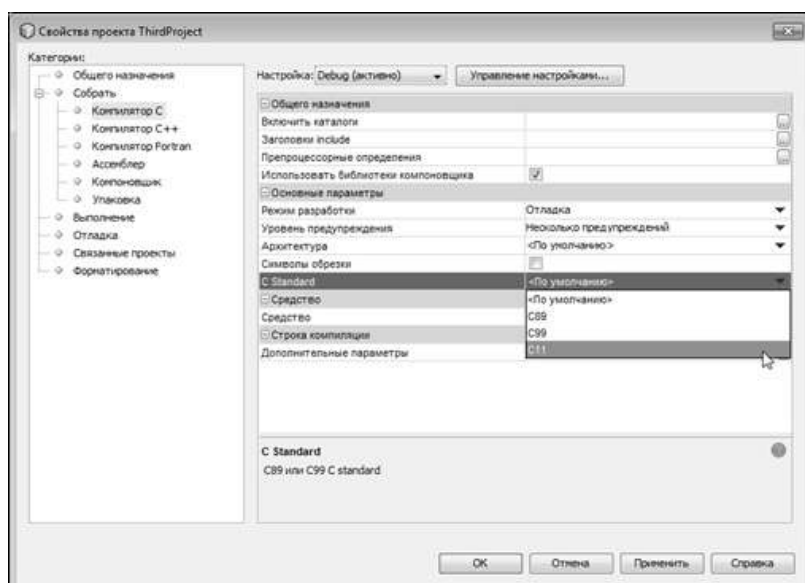


Рис. В.28. Определение значение параметра C Standard

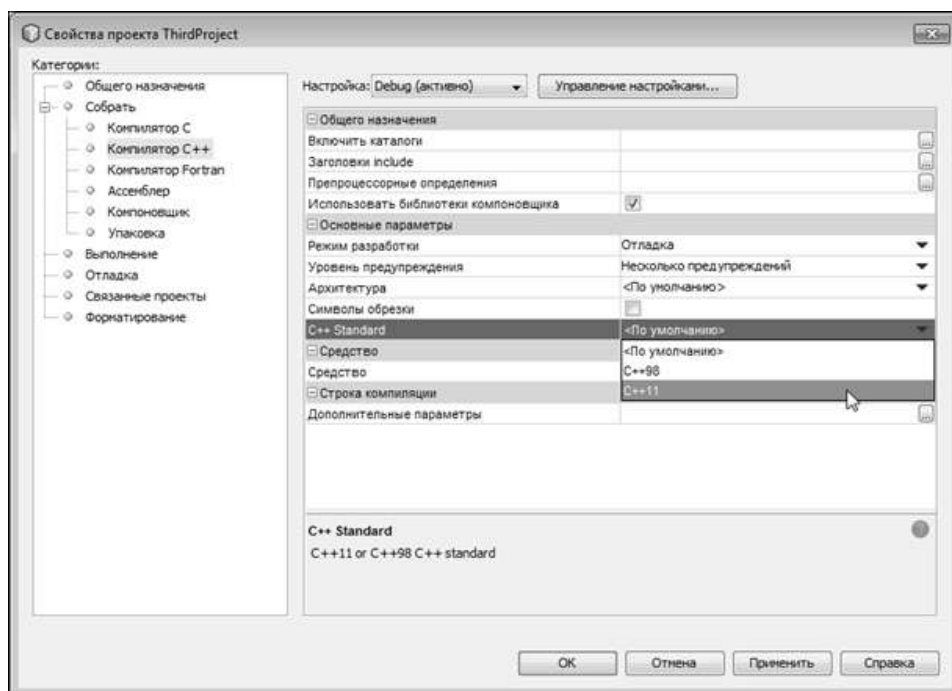


Рис. В.29. Определение значение параметра C++ Standard

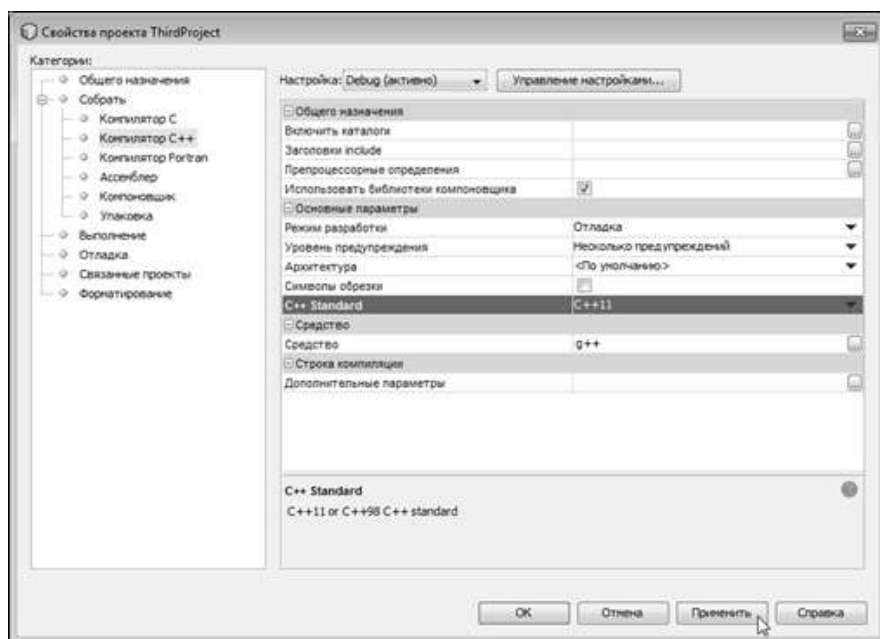


Рис. В.30. Применение выполненных настроек

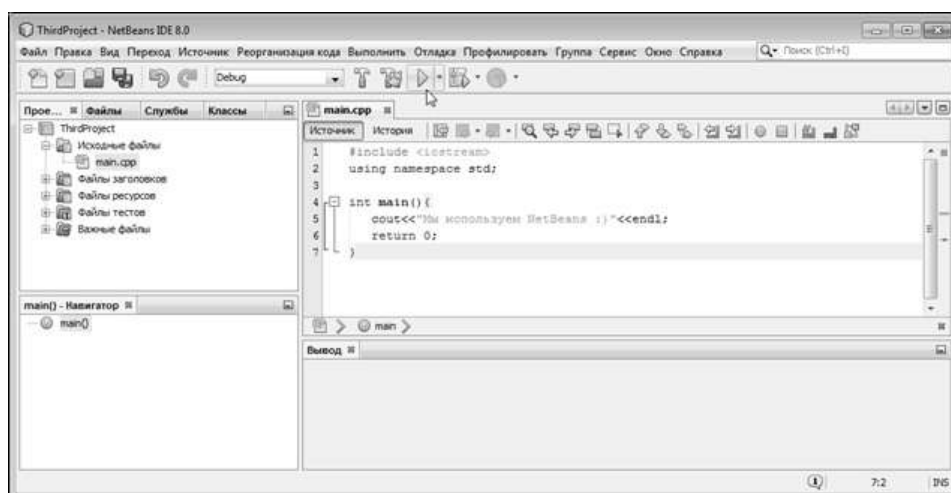


Рис. В.31. Проект с программным кодом

и двойным щелчком выделяем позицию с названием файла проекта (в данном случае **main.cpp**). В правой части откроется окно с названием файла, и во вкладке **Источник** вводим программный код (см. рис. В.31).

Здесь мы ввели такие команды:

```
#include<iostream>
using namespace std;
int main() {
    cout<<"Мы используем NetBeans :)"<<endl;
    return 0;
}
```

Для компилирования проекта и запуска его на выполнение щелкаем пиктограмму с зеленой стрелкой на панели инструментов (см. рис. В.31). Также можем нажать клавишу <F6> или в меню **Выполнить** выбрать команду **Запустить проект** (рис. В.32).

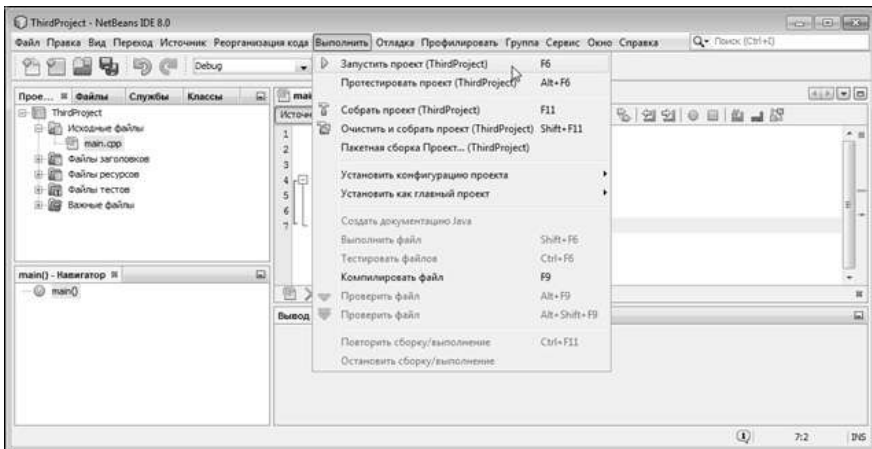


Рис. В.32. Запуск проекта на выполнение

Результат выполнения программы (если все прошло без ошибок) отображается во внутреннем окне **Вывод**, как показано на рис. В.33.



На заметку

После завершения выполнения программы окно вывода результата не исчезает, поэтому в программном коде команду `system("PAUSE")` можно не использовать - в ней нет необходимости.

Вкратце это те особенности среды разработки NetBeans, которые, скорее всего, понадобятся пользователю. Вообще же настроек, команд и прочих утилит достаточно много. Мы их описывать не будем, чтобы не выходить за

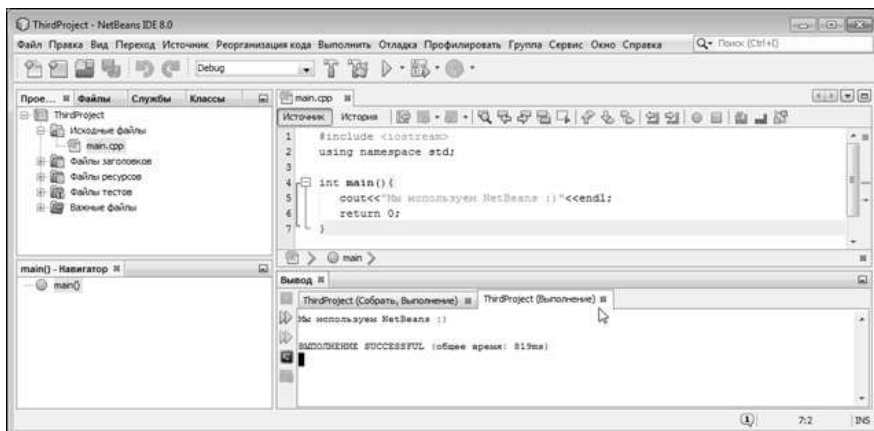


Рис. В.33. Отображение результата выполнения программы

рамки приличий и не уклоняться от "генеральной линии" (которая всецело связана с языком C++ и концепцией ООП, но никак не со средой NetBeans). Тем более что интерфейс среды NetBeans продуман неплохо и многие настройки интуитивно понятны даже малоподготовленному пользователю. В крайнем случае, можно обратиться к справке по данному программному продукту.

Обратная связь

*Гений, не гений - а тапочки заслужил.
из к/ф "Усатый нянь"*

То, как видят книгу читатели, совсем необязательно совпадает с тем, как книга видится автору. Поэтому очень важно получать "обратный отклик". Мнение читателей имеет принципиальное значение: во многом именно оно определяет тематику новых книг и учитывается, по возможности, при переиздании книг уже вышедших. Общение с читателями позволяет устранить и некоторые технические проблемы. Как бы там ни было, есть скромная, но настоятельная просьба по возможности принять посильное участие в улучшении книги (возможно, будущих ее изданий). Приветствуется конструктивная критика, которой можно "подвергнуть" автора по электронной почте vasilev@univ.kiev.ua или alex@vasilev.kiev.ua. Персональный сайт автора доступен по адресу www.vasilev.kiev.ua. На этом сайте (опять же, обычно по просьбе читателей) размещаются дополнительные материалы к книгам.

Глава 1.

ПРОСТЫЕ ПРОГРАММЫ



Иван Сильч! Говори нормальным человеческим языком!

из к/ф "Безумный день инженера Баркасова"

Начнем с того, что поближе познакомимся с принципами ООП и рассмотрим, как эти принципы реализуются в виде программного кода на языке C++. Подход наш будет простым и прагматичным. Каждое новое положение или прием мы будем рассматривать на небольшом примере. Так поступим и на этот раз. Однако перед тем, как начать рассмотрение программного кода, уделим некоторое внимание общим принципам программирования, имеющим достаточно универсальный характер и не относящимся непосредственно к языку C++.

1.1. Программирование без программирования

По-моему хорошо и в рифму.

из к/ф "Слезы капали"

Если по-хорошему, то программирование начинается не с составления программного кода, а с формализации задачи и разработки алгоритма, с помощью которого задача будет решаться. Разумеется, когда разрабатывается алгоритм, следует принимать в расчет особенности языка программирования (или конкретного программного пакета), который будет использован для реализации алгоритма. Однако обычно это обстоятельство не очень критичное. Во всяком случае возможности языка C++ настолько значительные, а сам язык настолько гибкий, что практически любой разумный алгоритм может быть реализован в C++. Так или иначе, но нас будут интересовать алгоритмы "в чистом виде" и составлять мы их будем в твердой уверенности, что любая наша идея может быть реализована в C++. И это недалеко от истины.

Чтобы проиллюстрировать объектно-ориентированный подход, рассмотрим небольшой пример-зарисовку. Общая постановка задачи будет такой: кто-то (будем называть его *вкладчиком*) хочет положить определенную сумму на счет в банк под проценты. Необходимо определить сумму, которую он получит (снимет со счета в банке) через указанный период времени. Все просто. Нам, понятное дело, для решения этой действительно несложной задачи необходимо написать программный код. Но предварительно нужно определиться с алгоритмом расчетов. В их основе лежит формула, по которой вычисляется сумма (с учетом начисленных процентов) на банковском

счету. Для конкретности предположим, что на депозит ложится сумма в M денежных единиц. Процентная ставка будет составлять величину в n процентов годовых, а депозит (денежный вклад) размещается на период времени t (выражается в годах). Тогда, по истечении периода времени t , вкладчик снимет с депозита сумму $M \left(1 + \frac{n}{100}\right)^t$. Если абстрагироваться от второстепенных деталей, то для вычисления результат нам необходимо знать значение трех параметров: вносимую на депозит величину M , годовую ставку процента n и время размещения депозита t . Но если бы все было так просто, мы бы не рассматривали данный пример.



На заметку

Приведенная выше формула называется формулой вычисления сложных процентов и означает, что проценты на банковский депозит начисляются не только на базовую сумму депозита (то есть на ту сумму, что вносит вкладчик), но и на проценты от этой суммы за предыдущие годы. Так, на каждую денежную единицу за первый год сумма начисляемых процентов составляет величину $\frac{n}{100}$ (где через n обозначена годовая ставка в процентном выражении). Поэтому если на начало года депозит был в 1 денежную единицу, то к концу года на счету вкладчика будет $1 + \frac{n}{100}$ денежных единиц. Еще через год на депозите будет $\left(1 + \frac{n}{100}\right)^2$, и так далее. Через t лет сумма на депозите составит величину $\left(1 + \frac{n}{100}\right)^t$. Напомним, это если вначале на депозит положили 1 денежную единицу. Если денежных единиц было M , то итоговая сумма будет $M \left(1 + \frac{n}{100}\right)^t$.

Нас, разумеется, интересует в первую очередь не технология расчетов, а стратегия организации программного кода. Здесь, как говорится, возможны варианты. Самый простой способ, который напрашивается сам собой, сводится к тому, чтобы получить (задать в программном коде или организовать ввод через клавиатуру) значения упомянутых выше трех параметров и на их основе вычислить нужное значение. В этом подходе нет ничего плохого, и он вполне корректный. Но не самый удобный, особенно если предполагается, что вычисления будут производиться несколько раз для разных значений начального взноса, ставки процента и периода времени. Поэтому разумнее, например, было бы создать специальную функцию с тремя аргументами (начальный взнос, процентная ставка и период времени), которая бы вычисляла итоговую сумму депозита.



На заметку

Есть два термина: аргумент и параметр. При вызове функции или метода те значения, которые передаются "на вход" обычно называют аргументами. При описании функции или метода значения, которые им передаются, называют параметрами. Мы в основном будем придерживаться термина аргумент. Другими словами, термин параметр будем использовать в общепринятом смысле.

В этом случае код функции описывается один раз, а использовать его можно многократно - каждый раз, когда необходимо провести соответствующие вычисления.



На заметку

Функция - это именованный блок программного кода, который можно вызывать (многократно). Имя программного блока называется именем функции. При вызове кода функции в соответствующем месте программы указывается имя функции и, если необходимо, аргументы (значения, переменные), с которыми функция вызывается. Функции удобно использовать, когда один и тот же фрагмент кода в программе выполняется несколько раз.

Это тоже хороший подход, но он не объектно-ориентированный. Что же в нем "не так"? Ответ вначале может показаться несколько надуманным, но на самом деле за ним скрывается глубокая философия (да, это философия ООП - правда, в несколько упрощенной форме). Так вот, даже если мы используем функцию для вычисления итоговой суммы депозита, нам каждый раз придется указывать значения, на основе которых рассчитывается результат. Особо плохого в этом ничего нет - нам в любом случае как-то придется их (значения) указать, потому что иначе вычисления теряют смысл. Вопрос в том, как "правильные" значения для характеристик депозита "находят" функцию для вычисления итоговой суммы. Суть проблемы легче понять, если исходить из предположения, что имеется не один, а несколько вкладчиков, и для каждого из них задается свое значение начальной суммы депозита, процентной ставки и периода времени, на который открывается депозит. Все эти данные нужно где-то хранить - например, в переменных (каких именно и как именно - сейчас не важно).

Таким образом, функция у нас одна, а данных для обработки много. И каждый раз, когда мы вызываем функцию, аргументами ей нужно передать значения трех параметров для определенного вкладчика, и ничего не перепутать. При наличии должного терпения и аккуратности эта задача вполне разрешимая. Но так уж устроен человек, что он всегда хочет большего. Мы не исключение. Мы хотим по возможности обезопасить себя от случайностей и прибегнем для этого к помощи ООП.

Чтобы исключить "перепутывание" данных при обработке, объединим их в одно целое с функцией, которая предназначена для обработки данных. Такое объединение выполняется в рамках объекта. То есть объект в нашем случае - это набор сведений о базовой сумме депозита, процентной ставке и времени депозита, а также программный код для вычисления на основе указанных данных итоговой суммы депозита. Такой объект в некотором смысле можем отождествлять с вкладчиком (или, по крайней мере, правильным

будет утверждение, что каждому вкладчику соответствует отдельный объект). При этом у читателя, скорее всего, возникает справедливое замечание: как же так, мы в каждый объект включаем, помимо уникальных данных, еще и блок кода для их обработки, и этот блок одинаков для всех объектов? Другими словами, мы многократно дублируем по сути один и тот же программный код, что вряд ли можно назвать оптимальным методом программирования. Откровенно говоря, это так и не так. Правда в том, что в каждый объект фактически "спрятана" одна и та же функция (точнее, это разные функции, но с одинаковым кодом). Но большой проблемы здесь нет, поскольку соответствующий программный код (то есть код для функции) нам придется набирать только один раз. Объяснение простое: объекты создаются по одному "шаблону" (который называется классом). Достаточно описать программный код в "шаблоне" (классе), и при создании объектов они автоматически получают нужные свойства и характеристики из "шаблона".



На заметку

Позже мы узнаем, что термин шаблон в C++ используется во вполне определенном смысле, как обозначение для обобщенных классов и функций. Здесь же в слово шаблон мы никакого особого смысла не вкладываем. Чтобы подчеркнуть это обстоятельство, слово шаблон заключено в двойные кавычки.

Теперь перейдем к более конкретным вещам. А именно, определимся с тем, как будем создавать объекты, и что с ними потом делать. В первую очередь, как отмечалось выше, нам необходим "шаблон", на основе которого будут создаваться объекты. Такой "шаблон" называется *классом*. Класс фактически содержит описание того, что может быть в объекте. А в объекте могут быть некоторые данные, и еще в объекте может содержаться программный код для работы с данными. Обычно данные содержатся в переменных. Но поскольку в данном случае речь идет не просто о данных, а о данных для объекта, то соответствующие переменные принято называть *полями*. Программный код для обработки данных представляет собой ни что иное, как описание функции. Функции, которые относятся к объектам, называются *методами*.



На заметку

Поля и методы класса называются членами класса.

Так вот, чтобы описать класс, достаточно указать, какие поля и методы содержатся в соответствующих объектах (то есть объектах, которые создаются на основе класса). Это общий принцип.

В данном конкретном случае у класса должны быть, очевидно, такие поля:

- поле для записи и хранения значения базовой суммы депозита;
- поле для значения годовой процентной ставки по депозиту;
- время, на которое открывается депозит.

Кроме этих трех полей, у класса должен быть метод, который в качестве результата, с использованием значений полей, будет вычислять итоговую сумму депозита.

Дальнейший план действий такой: мы описываем класс, создаем на основе этого класса нужное количество объектов (с определенными значениями полей) и затем, используя метод вычисления итоговой суммы результата, получаем значения для сумм депозитов по каждому вкладчику. Реализация всего этого великолепия в программе, написанной на языке C++, является вопросом техническим.

1.2. Реализуем первую объектно-ориентированную программу

*Форму будете создавать под моим личным контролем. Форме сегодня придается большое содержание.
из к/ф "Чародеи"*

Наступил черед составления программного кода. Мы начнем с описания класса. Класс в C++ (в общих чертах) описывается следующим образом:

- начинается описание класса с ключевого слова `class`;
- после ключевого слова `class` указывается имя, или название класса (имя класса придумываем сами - лучше, чтобы оно более-менее соответствовало назначению класса);
- программный код класса заключается в фигурные скобки (пара скобок `{ и }`, между которыми размещается код класса), а в конце (после последней фигурной скобки) указывается точка с запятой;
- поля класса описываются так: указывается специальный идентификатор, определяющий тип значения поля, и имя поля;
- при описании метода указывается идентификатор типа результата метода (тип значения, вычисляемого в результате выполнения метода), имя метода, в круглых скобках список аргументов метода (в данном случае таких нет, но круглые скобки все равно нужны), а также программный код метода (заклучается в фигурные скобки).

Программный код класса, который нам нужен для решения поставленной задачи, может иметь следующий вид:

```
classBankAccount{
public:
    double money;
    double rate;
    int time;
    double result(){
        double res=money;
        int i;
        for(i=1;i<=time;i++){
            res=res*(1+rate/100);
        }
        return res;
    }
};
```

Хотя этот код и небольшой, он содержит довольно много важных конструкций, которые и обсудим. Как и анонсировалось, описание класса начинается с ключевого слова `class`. После него указано имя класса `BankAccount`. Программный код класса заключен в фигурные скобки.



На заметку

В самом начале программного кода класса есть инструкция (ключевое слово) `public`, после которого стоит двоеточие. Эта инструкция определяет уровень доступа к полям и методам класса. Что такое уровень доступа и на что он влияет, мы поговорим немного позже, когда будем рассматривать программный код, в котором выполняется доступ к полям и методам объектов. Пока же для нас важно помнить, для чего в принципе нужна указанная инструкция.

В классе объявляется три числовых поля:

- Поле `money` относится к типу `double` (число с плавающей точкой). В это поле будем записывать значение базовой суммы депозита вкладчика.
- Поле `rate` также относится к типу `double` и предназначено для запоминания годовой процентной ставки по депозиту.
- Поле `time` является целочисленным (в качестве идентификатора типа указано ключевое слово `int`). В это поле будет записываться количество лет, на которые в банк помещается депозит.



На заметку

Как отмечалось ранее, поле - это практически то же, что и обычная переменная. Только поле описывается в теле класса. Думать о поле/переменной удобно как о

некотором значении, которое мы можем изменять и использовать при вычислениях. Технически переменная или поле представляют собой некоторую область в памяти компьютера. Эта область может содержать значение. Указывая в программном коде имя переменной, мы, тем самым, обращаемся к значению, которое записано в соответствующем месте памяти.

В языке C++ для каждого поля или переменной указывается тип. Технически такая необходимость обусловлена тем, что для записи различных значений нужен различный объем памяти. Другими словами, в зависимости от типа переменной в памяти выделяется соответствующий указанному типу объем памяти. Кроме того, тип переменной влияет на способ обработки значения, записанного в ячейке памяти. Мы достаточно часто будем использовать следующие типы данных: `int` (целые числа), `double` (действительные числа с плавающей точкой) и `char` (символ - то есть одна буква).

Потребности

В C++ существует несколько идентификаторов для обозначения базовых (наиболее простых) типов данных: `bool` (логические значения), `int` (целые числа), `float` (действительные числа), `double` (действительные числа двойной точности), `char` (символы) и `wchar_t` (двухбайтовые символы). Еще есть ключевое слово `void`, которое в основном используется для обозначения типа результата при описании функций и методов, не возвращающих результат.

Метод для вычисления итоговой суммы депозита описывается немного сложнее (по сравнению с тем, как описывались поля). Метод называется `result`. Пустые круглые скобки после имени метода означают, что аргументов у него нет. Ключевое слово `double` перед именем метода свидетельствует о том, что методом вычисляется значение (возвращается результат), и значение это относится к типу `double` - то есть это действительное число в формате с плавающей точкой. Тело метода (программный код, который выполняется при вызове метода) указан в блоке из фигурных скобок.

В теле метода командой `double res=money` объявляется переменная с именем `res`. Это переменная типа `double`. Значение переменной (на начальном этапе) такое же, как и у поля `money`.



На заметку

Оператором присваивания в языке C++ является знак равенства `=`. Команда вида `переменная=значение` означает, что переменной присваивается значение. Другими словами, значение записывается в переменную. Команду `res=money` следует понимать в том смысле, что текущее (на момент выполнения команды) значение поля `money` записывается в переменную `res`.

В данном случае мы объявляем переменную `res` и сразу присваиваем ей значение. Можно было бы просто объявить переменную. В этом случае для переменной выделяется место в памяти, но ничего туда не записывается.

Командой `int i` объявляется целочисленная переменная `i`, которую мы используем в *операторе цикла*. Оператор цикла, в свою очередь, нужен нам для того, чтобы вычислить результат метода. Признаком оператора цикла является ключевое слово `for`. В круглых скобках после этого ключевого слова есть три команды, разделяемые точкой с запятой. Первой командой `i=1` переменной `i` присваивается единичное значение. Эта команда выполняется только один раз, и только в начале выполнения оператора цикла. Вторая команда `i<=time` представляет собой условие, необходимое для выполнения оператора цикла. Другими словами, оператор цикла выполняется до тех пор, пока значение переменной `i` не превышает значение поля `time`. Командой `i++` значение переменной `i` увеличивается на единицу (команда `i++` эквивалентна инструкции `i=i+1`). Еще одна команда есть в блоке, выделенном фигурными скобками: речь идет об инструкции `res=res*(1+rate/100)`, в соответствии с которой текущее значение переменной `res` умножается на значение выражения $(1 + \text{rate}/100)$, и результат вычисленного выражения записывается в переменную `res`. Здесь мы знакомимся с несколькими арифметическими операторами: умножения (оператор `*`), сложения (оператор `+`) и деления (оператор `/`).

Потребности

Что касается оператора цикла, то он выполняется по следующей схеме:

- один раз в самом начале выполнения оператора цикла выполняется команда в первом блоке в круглых скобках после ключевого слова `for` (первый блок - до первой точки с запятой, - в данном случае это команда `i=1`);
- проверяется условие во втором блоке в круглых скобках после ключевого слова `for` (в данном случае условие `i<=time`);
- если условие ложно, работа оператора цикла завершается;
- если условие истинно, выполняются команды в теле оператора цикла (те, что в блоке в фигурных скобках - конкретно здесь это одна команда `res=res*(1+rate/100)`), а затем команда в третьем блоке в круглых скобках после ключевого слова `for` (в рассматриваемом примере команда `i++`);
- снова проверяется условие во втором блоке в круглых скобках после ключевого слова `for`, и так далее.

Выше описаны самые общие принципы работы оператора цикла в объеме, необходимом для понимания принципов выполнения кода. Оператор цикла на самом деле очень гибкий и может использоваться в самых разных форматах.

После того, как оператор цикла завершает свою работу, значение переменной `res` возвращается в качестве результата метода. Используем мы для этого команду `return res`.



На заметку

Инструкция `return` имеет удивительную особенность: она завершает работу метода (функции), из тела которого (которой) вызывается. Если после инструкции `return` указать значение, то это значение возвращается в качестве результата метода (функции).

Таким образом, методом `result()` при вызове будет вычисляться значение итоговой суммы депозита вкладчика. Полученное значение возвращается методом как результат. В качестве исходных данных используются значения полей объекта, из которого вызывается метод.



На заметку

В теле (в программном коде) метода `result()` используются значения полей `money`, `rate` и `time`. Кроме этого, в методе объявляются переменные `res` и `i`. Про последние говорят, что это локальные переменные метода. Локальная переменная и поле - далеко не одно и то же. Разница между ними в области доступности и времени существования. Поля доступны в любой точке программного кода класса. Существуют поля до тех пор, пока существует объект. Что касается локальных переменных, то область их доступности определяется тем блоком, в котором они объявлены (блок, в свою очередь, определяется парой фигурных скобок). Область доступности упомянутых выше локальных переменных `res` и `i` ограничивается телом метода `result()`. Существуют эти переменные, пока выполняется метод. После того, как выполнение метода завершено, все локальные переменные удаляются из памяти.

Хотя мы создали класс, мы еще не создали ни одного объекта этого класса. Создавать и использовать объекты класса будем в *главной функции программы*. Дело в том, что при выполнении программы в C++ на самом деле выполняется программный код функции, которая имеет имя `main()` в программном коде, а "в жизни" называется главной функцией программы. Мы будем использовать следующий шаблон для этой функции:

```
int main() {
    // Программный код
    return 0;
}
```

Перед именем функции указывается идентификатор `int`, который означает, что функция возвращает целочисленное значение. Целое число передается операционной системе и определяет режим завершения программы. Если все прошло "в штатном режиме", программа возвращает операционной системе нулевое значение. Поэтому программный код функции `main()` мы будем завершать командой `return 0`.



На заметку

Иногда возникает острая необходимость написать в программном коде что-то не для компилятора, а "для себя". Такая возможность есть. Речь идет о комментариях - текстовых вставках в программный код, которые предназначены для программиста (или любого иного, кому есть дело до программного кода) и компилятором полностью игнорируются. Комментарий нужно специальным образом выделить. В C++ существует два типа комментариев: однострочные и многострочные. Для создания однострочного комментария слева от него размещаются две косые черты `//`. Все, что находится справа от двух косых черт, при компилировании в расчет не принимается. Если комментарий занимает больше одной строки, то удобнее применить следующий метод комментирования: вначале комментария указывается комбинация символов `/*`, а завершается комментарий комбинацией символов `*/`.

Но и это еще не все. Вначале программы обычно размещаются "заголовочные" инструкции: например, команды подключения библиотек и инструкция определения *пространства имен*. В большинстве случаев по умолчанию мы будем начинать программный код инструкцией `# include <iostream>`, которой подключается стандартная *библиотека ввода/вывода*. Также мы будем работать со стандартным пространством имен (называется `std`), в силу чего в программном коде появляется команда `using namespace std`.



На заметку

Для того чтобы различать, например, классы, они, очевидно, должны иметь разные названия. В какой-то момент это становится проблемой, поскольку чтобы подобрать информативные и, самое главное, разные названия для большого числа классов, уже мало иметь хорошую фантазию. Поэтому используется такой прием: выполняют разбивку множества всех именовании на области, которые называются пространствами имен. В пределах одного пространства имен названия должны быть уникальными. В разных пространствах имен названия могут совпадать. Чтобы указать, какое используется пространство имен, в программный код помещают команду в формате `using namespace пространство_имен`. Стандартное пространство имен называется `std`.

Конечный функциональный (то есть готовый к использованию) программный код, в котором описывается класс, а также создаются и используются объекты этого класса, представлен в листинге 1.1.

Листинг 1.1. Класс BankAccount и его объекты

```

#include <iostream>
using namespace std;
// Начало описания класса:
class BankAccount{
    // Открытые члены класса:
public:
    // Поле для записи базовой суммы депозита:
    double money;
    // Поле для записи процентной ставки:
    double rate;
    // Время, на которое размещается депозит:
    int time;
    // Метод для вычисления итоговой суммы депозита:
    double result(){
        // Локальная переменная
        // для записи результата метода:
        double res=money;
        // Локальная переменная для оператора цикла:
        int i;
        // Оператор цикла:
        for(i=1;i<=time;i++){
            // Вычисление результата:
            res=res*(1+rate/100);
        }
        // Результат метода:
        return res;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Создаем первый объект:
    BankAccount ivanov;
    // Создаем второй объект:
    BankAccount petrov;
    // Значения полей первого объекта:
    ivanov.money=100;
    ivanov.rate=13;
    ivanov.time=3;
    // Значения полей второго объекта:
    petrov.money=90;
    petrov.rate=18;
    petrov.time=4;
    // Итоговая сумма депозита для первого вкладчика:
    cout<<"Иванов: "<<ivanov.result()<<endl;
}

```



```
// Итоговая сумма депозита для второго вкладчика:
cout<<"Петров: "<<petrov.result()<<endl;
// Завершение программы:
return 0;
}
```

В главной функции программы первые две команды дают нам пример создания объектов. Создаются объекты фактически так же, как объявляются переменные, только вместо идентификатора типа указывается имя класса. Так, в рассматриваемом примере командами `BankAccount ivanov` и `BankAccount petrov` создаются два объекта класса `BankAccount`: один объект называется `ivanov`, а другой объект называется `petrov`. Но создание объектов, как и в случае с переменными, означает лишь, что под них в памяти выделяется место. Это место нужно чем-то заполнить или, проще говоря, полям созданных объектов нужно присвоить значения. Значения полям присваиваются так же просто, как и значения локальным переменным: слева от оператора присваивания указывается поле, а справа от оператора присваивания указывается присваиваемое полю значение. Правда, одно формальное отличие все же есть.

Поскольку у разных объектов имеются поля с одинаковыми названиями, необходимо как-то различать поля разных объектов. Другими словами, если мы обращаемся к полю, то нужно указать к полю какого объекта мы обращаемся. Для этого используется так называемый "точечный" синтаксис: сначала указывается имя объекта, и затем, через точку, имя поля, то есть в формате `объект.поле`. В таком же формате выполняем обращение к методам объекта: перед инструкцией вызова метода указывается имя объекта. Имя объекта и имя метода разделяются точкой. Например, командой `ivanov.money=100` полю `money` объекта `ivanov` присваивается значение 100, а командой `petrov.money=90` полю `money` объекта `petrov` присваивается значение 90. Аналогично, для вызова метода `result()` из объекта `ivanov` используем инструкцию `ivanov.result()`, а для вызова метода `result()` из объекта `petrov` используем команду `petrov.result()`.



На заметку

Команды вида `объект.поле` или `объект.метод` можно использовать в главной функции программы (или в любом ином месте программного кода за пределами тела соответствующего класса) только в том случае, если поле или метод являются открытыми. Чтобы поля и методы класса были открытыми, их необходимо описывать в блоке, который начинается ключевым словом `public`. Именно с этого ключевого слова начинается программный код в теле класса `BankAccount`. Поэтому мы можем использовать для инициализации полей (присваивания значения полям) инструкции с явным обращением к полям. Если бы в программном коде класса `BankAccount` ключевое слово `public` отсутствовало, то команды вида `ivanov.money` или `petrov.result()` привели бы к ошибкам при компиляции, поскольку в C++ по умолчанию все поля и методы класса являются закрытыми.

После того, как полям объектов `ivanov` и `petrov` присвоены значения, мы вычисляем для каждого из этих объектов итоговую сумму депозита. Для этого нам достаточно вызвать из соответствующего объекта метод `result()`.



На заметку

Если мы вызываем метод `result()` из объекта `ivanov` (команда `ivanov.result()`), то при вычислении значения метода используются поля объекта `ivanov`. Если метод `result()` вызывается из объекта `petrov` (команда `petrov.result()`), то при вычислениях используются поля объекта `petrov`. Таким образом, метод использует поля объекта, из которого вызывается. Поэтому нет необходимости беспокоиться о том, что результат будет рассчитан на основе данных для "другого" вкладчика.

Для вывода результата в консольное окно нами используется *оператор вывода* `<<`. Справа от оператора вывода указывается значение (текст заключается в двойные кавычки), которое выводится (на экран в данном случае). Слева от оператора вывода в общем случае указывается идентификатор устройства, в которое осуществляется вывод информации. Инstrukция `cout` (сокращение от *console output*) является идентификатором консольного устройства вывода (по умолчанию это экран компьютера). Допускается использование сразу нескольких операторов вывода в одной команде. Поэтому вполне законной, например, является следующая инструкция `cout<<"Иванов: "<<ivanov.result()<<endl`, которой на экран выводится текст "Иванов: ", затем числовое значение `ivanov.result()`, являющееся результатом вызова метода `result()` из объекта `ivanov`, и, наконец, выполняется переход к новой строке (следствие наличия в команде ключевого слова `endl` - сокращение от *end of line*). Аналогично выполняется команда `cout<<"Петров: "<<petrov.result()<<endl`.

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 1.1)

Иванов: 144.29

Петров: 174.49

В консольном окне (окне вывода) появляется два сообщения с итоговыми суммами депозитов для двух различных вкладчиков. Для получения данной информации нам, по большому счету, достаточно было вызвать из соответствующего объекта метод `result()`. В принципе, это удобно. Но даже невооруженным взглядом сразу видно, что в рассмотренном примере есть, скажем так, некоторые неудобные моменты. В первую очередь довольно утомительно выглядят команды, которыми заполняются поля объектов. На

практике так обычно не поступают. То есть вариант, когда полям в явном виде присваиваются значения, вполне приемлем, но только если полей и/или объектов не очень много. Если это не так, то одним из способов решения проблемы может быть использование *конструктора*.

1.3. Долой оковы ООП

Наше повеление: "Этот танец не вяжется с королевской честью". Мы запрещаем его на веки веков.

из к/ф "31 июня"

Истина, как известно, познается в сравнении. Интересно было бы рассмотреть программный код, в котором решается та же задача, что и выше, но без использования принципов ООП. Понятно, что вариантов тут много. Мы рассмотрим наиболее простой. Код представлен в листинге 1.2.

Листинг 1.2. Программа без классов и объектов

```
#include<iostream>
using namespace std;
// Функция для вычисления итоговой суммы депозита:
double result(double money,double time,double rate){
// Локальная переменная
// для записи результата функции:
double res=money;
// Локальная переменная для оператора цикла:
int i;
// Оператор цикла:
for(i=1;i<=time;i++){
// Вычисление результата:
res=res*(1+rate/100);
}
// Результат функции:
return res;
}
// Главная функция программы:
int main(){
// Значения переменных (первый вкладчик):
double ivanov_money=100;// Вклад
double ivanov_rate=13;    // Процентная ставка
double ivanov_time=3;     // Время
// Значения переменных (второй вкладчик):
double petrov_money=90;   // Вклад
double petrov_rate=18;    // Процентная ставка
double petrov_time=4;     // Время
```

```
// Итоговая сумма депозита для первого вкладчика:
cout<<"Иванов: "<<result(ivanov_money,ivanov_time,ivanov_
rate)<<endl;
// Итоговая сумма депозита для второго вкладчика:
cout<<"Петров: "<<result(petrov_money,petrov_time,petrov_
rate)<<endl;
// Завершение программы:
return 0;
}
```

Результат выполнения данного программного кода выглядит следующим образом:

Результат выполнения программы (из листинга 1.2)

```
Иванов: 144.29
Петров: 174.49
```

Видим, что результаты в обоих случаях совпадают. Однако программы принципиально разные. Во втором случае мы совершенно не использовали классы и объекты. Вместо этого описана функция `result()` с тремя аргументами (начальный вклад, процентная ставка и время размещения вклада). Результатом функция возвращает значение итоговой суммы депозита.

В главной функции программы для каждого из вкладчиков определяется по три переменных. Эти переменные передаются аргументами функции `result()`.



На заметку

Метод `result()` описывается в теле класса. При создании объектов этого класса каждый объект получает свой "персональный" метод. Этот метод имеет доступ к полям "своего" объекта. Поскольку метод имеет доступ к полям объекта, то аргументы методу не нужны. Образно выражаясь, метод `result()` получает необходимую ему информацию через поля объекта. Когда мы описываем функцию `result()`, то она как бы "сама по себе". У функции нет связанного с ней объекта. А для выполнения калькуляций функции нужны параметры (вклад, ставка процента и время вклада). Эти параметры передаются функции в виде аргументов. Поэтому функция `result()` описана с тремя аргументами, которые играют ту же фактически роль, что поля объекта в первом примере.



На заметку

Может показаться, что программа без классов и объектов проще и понятней. Возможно, это где-то так и есть. Но тут важно понять, что пример мы рассматривали простой. А представляете, что будет, если вкладчиков не два, а двадцать? Да и всю гибкость и эффективность методов ООП мы еще осознали. Например, мы еще не познакомились с конструкторами. Но это со временем придет.

1.4. Знакомство с конструкторами

Эх, погубят тебя слишком широкие возможности.

из к/ф "Айболит 66"

Конструктор - штука мощная и удобная. Во всяком случае, работать с классами и объектами и не использовать конструкторы - просто нереально. Формально конструктор - это метод, который вызывается автоматически при создании объекта. Поэтому если в конструкторе разместить кокой-то программный код, то этот программный код будет выполняться каждый раз, когда создается новый объект. Более того, у конструктора могут быть аргументы. Да и вообще, конструкторов у класса может быть несколько. Обо всем этом и поговорим.

Итак, поскольку конструктор - это все-таки метод (хоть и особенный), его в классе можно описать. Есть определенные правила описания конструктора в классе. Их немного, но они непреклонные:

- Имя конструктора совпадает с именем класса.
- Конструктор не возвращает результат, а идентификатор типа результата для него не указывается.
- У конструктора могут быть аргументы.
- У класса может быть несколько конструкторов.

Все достаточно просто. Например, если создается конструктор для класса с названием `BankAccount`, то конструктор класса должен называться точно так же, то есть `BankAccount()`. У конструктора могут быть аргументы, а может не быть. И еще, как отмечалось выше, мы можем описать сразу несколько конструкторов в классе.



На заметку

Процесс создания нескольких конструкторов в классе является примером более общего механизма, который называется *перегрузкой* методов и позволяет создавать несколько версий метода с одним и тем же именем. Перегрузка методов обсуждается немного позже.

Далее мы немного усовершенствуем программный код класса `BankAccount` и добавим в этот класс два конструктора. Соответственно, изменится и программный код главной функции программы `main()`. Новую, усовершенствованную версию класса, назовем так же, как и предыдущую - то есть `BankAccount`. Программный код приведен в листинге 1.3.

Листинг 1.3. Класс BankAccount с конструктором

```

#include <iostream>
using namespace std;
// Начало описания класса:
class BankAccount{
    // Открытые члены класса:
public:
    // Поле для записи базовой суммы депозита:
    double money;
    // Поле для записи процентной ставки:
    double rate;
    // Время, на которое размещается депозит:
    int time;
    // Метод для вычисления итоговой суммы депозита:
    double result(){
        // Локальная переменная
        // для записи результата метода:
        double res=money;
        // Локальная переменная для оператора цикла:
        int i;
        // Оператор цикла:
        for(i=1;i<=time;i++){
            // Вычисление результата:
            res=res*(1+rate/100);
        }
        // Результат метода:
        return res;
    }
    // Конструктор класса (без аргументов):
    BankAccount(){
        // Присваиваем значения полям:
        money=100;
        rate=13;
        time=3;
    }
    // Конструктор класса (с тремя аргументами):
    BankAccount(double m,double r,int t){
        // Присваиваем значения полям:
        money=m;
        rate=r;
        time=t;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){

```

```
// Создаем первый объект:
BankAccount ivanov;
// Создаем второй объект:
BankAccount petrov(90,18,4);
// Итоговая сумма депозита для первого вкладчика:
cout<<"Иванов: "<<ivanov.result()<<endl;
// Итоговая сумма депозита для второго вкладчика:
cout<<"Петров: "<<petrov.result()<<endl;
// Завершение программы:
return 0;
}
```

По сравнению с исходной версией (см. листинг 1.1), мы внесли в программный код минимальные изменения. В частности, в классе `BankAccount`, помимо трех полей и метода, которые там были и раньше, появилось еще два конструктора (правильнее будет сказать - две версии конструктора). Мы рассмотрим программный код конструктора с тремя аргументами, а конструктор без аргументов "организован" аналогично. Результат выполнения программного кода такой же, как и программного кода из листинга 1.1:

Результат выполнения программы (из листинга 1.3)

```
Иванов: 144.29
Петров: 174.49
```

Поскольку класс называется `BankAccount`, то конструктор класса называется так же, то есть `BankAccount()` (пустые круглые скобки для конструктора мы указываем для того, чтобы подчеркнуть, что это метод - во всяком случае, такая сложилась традиция). У конструктора три аргумента. Аргументы конструктора (впрочем, как и других методов), описываются так: указывается тип аргумента и его название. Если аргументов несколько, они разделяются запятыми. Даже если несколько аргументов имеют одинаковые типы, для каждого аргумента тип все равно указывается персонально.



На заметку

Аргументы метода или конструктора имеют "силу" локальных переменных. Это означает, что доступны они только в теле соответствующего метода/конструктора.

В данном случае у конструктора три аргумента: аргумент `m` типа `double`, аргумент `r` типа `double` и аргумент `t` типа `int`. Предполагается, что аргумент `m` определяет значение базовой суммы, вносимой на депозит вкладчиком, аргумент `r` определяет процентную ставку по вкладу, а аргумент

`t` задает время, на которое в банк помещается депозит. В соответствии с этими "предположениями" в теле конструктора полям класса присваиваются значения: полю `money` присваивается значение первого аргумента конструктора `m` (команда `money=m`), полю `rate` присваивается значение второго аргумента конструктора (команда `rate=r`), а полю `time` присваивается значение третьего аргумента конструктора `t` (команда `time=t`). Таким образом, в результате выполнения конструктора полям создаваемого объекта присваиваются те значения, что переданы конструктору аргументами. Возникает естественный вопрос: как конструктору передать аргументы? Делается это очень просто. Чтобы передать конструктору аргументы при создании (объявлении) объекта в круглых скобках после имени объекта указываются аргументы конструктора. Например, команда `BankAccount petrov(90,18,4)` в функции `main()` означает, что создается объект `petrov` класса `BankAccount`, и при этом конструктору класса `BankAccount` необходимо передать аргументы 90, 18 и 4.

У класса, помимо конструктора с тремя аргументами, есть еще и конструктор без аргументов. В этом случае при создании объекта аргументы конструктору не передаются. Поля объекта все равно получают значения, но на этот раз значения определяются не аргументами (которых нет), а являются фиксированными значениями (команды `money=100`, `rate=13` и `time=3` в теле конструктора). На ситуацию можно посмотреть и по-другому: если мы создаем объект, не передавая аргументы конструктору (как, например, в команде `BankAccount ivanov`), то поля создаваемого объекта по умолчанию получают значения 100 для поля `money`, 13 для `rate` и 3 для поля `time`.



На заметку

Сразу отметим, что это далеко не единственный способ задать значения по умолчанию. Существуют и иные, в некотором смысле даже более элегантные подходы. И мы с ними познакомимся.

Даже из этой несложной иллюстрации видно, что конструктор достаточно удобен и прост в использовании. По крайней мере, любой претендующий на профессиональный уровень класс должен содержать несколько различных конструкторов. Хотя, конечно, многое зависит от назначения класса. С другой стороны, совершенно очевидно, что рассмотренный нами программный код можно усовершенствовать и усовершенствовать - практически до бесконечности. Мы пойдем несколько иным путем. А именно, попробуем немного иначе организовать функциональные возможности класса, для чего добавим в него несколько новых методов. Кроме того, поля сделаем закры-

тыми, а доступ к этим полям (для присваивания значений) реализуем через открытые методы.

1.5. Реорганизация программного кода

*Видел чудеса техники, но такого...
из к/ф "Иван Васильевич меняет профессию"*

Как отмечалось выше, мы внесем некоторые, в определенном смысле важные, изменения в программный код. Основные изменения сводятся к следующему:

- В класс `BankAccount` будет добавлено текстовое поле для записи в него имени вкладчика.
- Все четыре поля (три числовых и одно текстовое) класса станут закрытыми.
- Для присваивания значения числовым полям в программный код класса `BankAccount` добавлен метод `set()` с тремя аргументами, имеющими значения по умолчанию.
- Для присваивания значения текстовому полю в класс `BankAccount` добавлен метод `setName()` с текстовым аргументом.
- Метод `show()` предназначен для отображения полной информации по вкладчику. А именно, при выполнении метода в консольное окно выводится имя вкладчика (значение текстового поля), а также "технические" параметры вклада (начальная сумма вклада, процентная ставка по вкладу и время, на которое деньги размещаются на депозит). Кроме этого, отображается и итоговая сумма депозита. Для этого в теле метода `show()` вызывается метод `result()`.
- У класса `BankAccount`, как и ранее, два конструктора, но теперь у них изменились аргументы. В частности, в классе имеется конструктор с одним текстовым аргументом, а также конструктор с одним текстовым и тремя числовыми аргументами. Дополнительный текстовый аргумент, как несложно догадаться, предназначен для определения (при создании объекта) имени вкладчика.
- Помимо определения значений полей в конструкторе класса (в обеих версиях) вызывается метод `show()`, в результате чего создание объект означает автоматическое отображение в консольном окне информации о соответствующем вкладчике.

В листинге 1.4 приведен программный код новой версии программы, в которой использован класс `BankAccount`.

Листинг 1.4. Класс BankAccount с закрытыми полями

```
#include <iostream>
#include <string>
using namespace std;
// Начало описания класса:
class BankAccount{
    // Закрытые члены класса:
private:
    // Поле для записи имени вкладчика:
    string name;
    // Поле для записи базовой суммы депозита:
    double money;
    // Поле для записи процентной ставки:
    double rate;
    // Время, на которое размещается депозит:
    int time;
    // Открытые члены класса:
public:
    // Метод для присваивания значения числовым полям:
    void set(double m=100,double r=13,int t=3){
        // Значения полей:
        money=m;
        rate=r;
        time=t;
    }
    // Метод для определения значения текстового поля
    // (имя вкладчика):
    void setName(string n){
        // Значение текстового поля:
        name=n;
    }
    // Метод для вычисления итоговой суммы депозита:
    double result(){
        // Локальная переменная
        // для записи результата метода:
        double res=money;
        // Локальная переменная для оператора цикла:
        int i;
        // Оператор цикла:
        for(i=1;i<=time;i++){
            // Вычисление результата:
            res=res*(1+rate/100);
        }
        // Результат метода:
        return res;
    }
};
```

```

    }
    // Метод для отображения данных вкладчика:
void show(){
    // Отображается имя вкладчика:
cout<<name;
    // Отображаются значения числовых полей:
cout<<"<<money<<"<<rate<<"<<time;
    // Вычисляется и отображается
    // итоговая сумма депозита:
cout<<"<<result()<<endl;
}
    // Конструктор класса (с одним аргументом):
BankAccount(string name){
    // Задаем имя вкладчика:
setName(name);
    // Присваиваем значения полям:
set();
    // Отображаем данные вкладчика:
show();
}
    // Конструктор класса (с четырьмя аргументами):
BankAccount(string name,double m,double r,int t){
    // Задаем имя вкладчика:
setName(name);
    // Присваиваем значения полям:
set(m,r,t);
    // Отображаем данные вкладчика:
show();
}
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Создаем первый объект:
BankAccount ivanov("Иванов Иван");
    // Создаем второй объект:
BankAccount petrov("Петров Петр",90,18,4);
    // Завершение программы:
return 0;
}

```

Описание класса `BankAccount` начинается с инструкции `private`, после которой через двоеточие описываются четыре поля класса: с тремя числовыми полями мы уже знакомы, а еще одно новое поле имеет название `name` и объявлено как такое, что имеет тип `string`. Если быть до конца откровенным, то `string` - это название класса. Чтобы этот класс стал до-

ступен, в начале программы (во второй строке) мы поместили команду `#include <string>`, которой подключается библиотека для работы с текстом. Инструкция `string name`, таким образом, означает ни что иное, как принадлежность поля `name` к славной когорте объектов класса `string`. Другими словами, теперь у класса `BankAccount` есть поле, которое само является объектом класса (в данном случае это класс `string`, но мог быть и какой-то другой). Такой подход (когда поля класса являются объектами) вполне законный и часто используется на практике. Для нас пока что важно, что поле `name` текстовое, и в качестве значения этому полю можно присвоить текст.



На заметку

В C++ существует еще один популярный способ реализации текста - в виде символического массива.

Что касается инструкции `private`, то ее можно было бы не указывать - по умолчанию, если идентификатор уровня доступа членов не указан, они считаются закрытыми.

Поскольку теперь все четыре поля класса являются закрытыми, мы не сможем к ним обратиться в программном коде вне пределов тела класса. То есть обращаться к закрытым полям класса мы можем только в теле методов, описываемых в этом классе. Отсюда сразу возникают две очевидные проблемы: как полям присваивать значения и как узнать значения полей? Выход, разумеется, есть. Состоит он в использовании открытых методов доступа к закрытым полям класса. Поскольку такие методы открытые, то, с одной стороны, к этим методам можно обратиться вне пределов класса, а с другой стороны, эти методы имеют доступ ко всем членам "своего" класса (в том числе и к закрытым полям и методам).



На заметку

Таким образом, для работы с закрытыми полями вводятся своеобразные "посредники". Напрямую работать с полем (вне класса) мы не можем, но можем обратиться к помощи метода-посредника. На первый взгляд может показаться, что это слишком запутанная и ненужная схема. Но на самом деле все не так. Дело в том, что "закрывая" члены класса и вводя методы доступа, мы создаем своеобразный предохранительный щит или барьер для несанкционированных (нежелательных, недопустимых) действий с закрытыми членами класса. Ведь метод доступа позволяет выполнять с полями, например, только те операции, которые запрограммированы в теле метода, и никакой "самодеятельности". В этом смысле класс (или его объект) напоминает "черный ящик" с определенной начинкой (в виде закрытых членов класса), а функциональность этого "черного ящика" определяется набором всяких кнопок и переключателей, роль которых играют открытые методы доступа.

В первую очередь интерес представляет процесс присваивания значения полям. Для этого мы в классе объявляем два метода.

На заметку

Все методы в классе, в том числе и две версии конструктора, описываются в блоке, озаглавленном инструкцией `public`, и поэтому являются открытыми.

Метод `set()` предназначен для присваивания значений трем числовым полям (`money`, `rate` и `time`). Метод не возвращает результат (то есть назначение метода состоит в том, чтобы выполнить некоторые действия, и при этом "на выходе" никакие значения нам не нужны), поэтому в качестве идентификатора типа для результата метода указано ключевое слово `void`.



На заметку

Ключевое слово `void` означает, что метод результат не возвращает. То, что для конструктора (который, как известно, тоже не возвращает результат) идентификатор `void` не указывается - это исключение из правил. Еще одно исключение из правил - деструктор. Для него, как и для конструктора, идентификатор типа не указывается. Деструкторы будем обсуждать позже. Если же речь идет об обычном методе (не конструкторе или деструкторе), и метод не возвращает результат, то в качестве идентификатора типа результата указываем ключевое слово `void`.

У метода `set()` три аргумента. Причем у этих аргументов есть значения по умолчанию. Что это означает? При вызове метода, если один или несколько аргументов явно не указаны, для недостающего аргумента (или аргументов) будет использовано значение по умолчанию. При объявлении метода значения по умолчанию для аргументов указываются через знак равенства после имени аргумента. Поэтому, например, если метод `set()` вызывать с тремя аргументами, то эти аргументы будут определять значения полей объекта, из которого вызывается метод. Если не указать ни одного аргумента, то будут использованы значения по умолчанию.



На заметку

Значения по умолчанию могут иметь все аргументы метода, а могут иметь только некоторые аргументы метода. Правило такое: в списке аргументов те, что имеют значения по умолчанию, должны быть в конце. То есть сначала указываются аргументы без значений по умолчанию, а затем указываются аргументы со значениями по умолчанию.

Метод для определения значения текстового поля называется `setName()`. Он не возвращает результат и у него всего один текстовый аргумент (объект класса `string`). В теле метода командой `name=n` (через `n` обозначен аргумент метода) присваивается значение полю `name`.

**На заметку**

Если абстрагироваться от того, что `n` и `name` - это объекты, то команда `name=n` особых вопросов не вызывает. Вопросы могут возникнуть, если вспомнить, что на самом деле речь идет об объектах. Но проблемы, разумеется, нет. В C++, если объекты относятся к одному классу (а `n` и `name` являются объектами класса `string`), то одному объекту в качестве значения можно присвоить другой объект. При этом по умолчанию присваивание выполняется через механизм побитового копирования - то есть "содержимое" одного объекта копируется в "содержимое" другого объекта.

Метод для отображения данных вкладчика называется `show()`. У него нет аргументов, и методом не возвращается результат. При выполнении метода в консольное окно последовательно выводятся значения полей объекта. Для удобства инструкция вывода значений разбита на несколько команд. Например, для вывода значения текстового поля `name` использована команда `cout<<name`. При вычислении значения итоговой суммы депозита мы используем инструкцию вызова метода `result()`. В данном случае, поскольку метод `result()` вызывается в теле класса, ссылка на объект, из которого вызывается метод, необязательна. По умолчанию подразумевается, что этот тот же объект, из которого вызывается метод `show()`.

Конструктор класса с текстовым аргументом (инструкция `string name` в списке аргументов конструктора) содержит две команды: командой `setName(name)` задаем имя вкладчика, а командой `set()` присваиваем значения числовым полям. Поскольку в данном случае метод `set()` вызывается без аргументов, то в качестве каждого из трех аргументов метода используется соответствующее значение по умолчанию. Инструкция вызова метода `show()` в теле конструктора приводит к тому, что сразу после заполнения полей при создании объекта отображается информация о "содержимом" объекта. Объект вызова для методов `setName()`, `set()` и `show()` не указывается, и это означает, что вызываются они из того же объекта, для которого вызывается конструктор.

Принципиальное отличие конструктора с четырьмя аргументами в том, что в теле конструктора вместо вызова метода `set()` без аргументов, этот метод вызывается с тремя аргументами. Тем самым явно задаются значения трех числовых полей создаваемого объекта.

Поскольку программный код класса `BankAccount` изменился достаточно сильно, это отразилось и на программном коде функции `main()`. А именно, теперь в главной функции программы, если не считать последней "стандартной" инструкции, всего две команды, которыми создаются объекты класса `BankAccount`. Один объект создается командой `BankAccount ivanov("Иванов Иван")`, а для создания второго объекта использована команда `BankAccount petrov("Петров Петр")`,

90, 18, 4). При этом в консольном окне отображаются параметры созданных объектов. Результат выполнения программы такой:

Результат выполнения программы (из листинга 1.4)

```
Иванов Иван: 100: 13: 3: 144.29
Петров Петр: 90: 18: 4: 174.49
```

Несложно догадаться, что сообщения, которые появляются в консольном окне, являются следствием вызова метода `show()`. Метод этот, напомним, вызывается каждый раз при создании нового объекта. Команда вызова метода `show()` размещена в программном коде конструктора класса `BankAccount`.



На заметку

Мы уже видели, что при наличии нескольких версий конструктора объекты можно создавать по-разному. Решение о том, какой именно вариант конструктора вызывается при создании объекта, принимается на основании команды создания объекта - по количеству и типу аргументов, которые передаются конструктору. Также необходимо иметь в виду еще один достаточно важный момент: если в классе не описан конструктор, то при создании объекта используется так называемый конструктор по умолчанию. У него нет аргументов и при его вызове никакие дополнительные операции, кроме непосредственно создания объекта, не выполняются. Именно благодаря наличию конструктора по умолчанию мы могли создавать объекты, не описывая конструктор в классе. Но как только в классе описан хотя бы один конструктор, конструктор по умолчанию больше не доступен. Поэтому в рассмотренном выше примере, в котором у класса `BankAccount` два варианта конструктора соответственно с одним и четырьмя аргументами, при создании объекта нужно указать один или четыре аргумента. Создать объект, не передавая аргументы конструктору, не получится.

Глава 2.

МЕТОДЫ



Вы сюда приехали, чтобы записывать сказки, понимаете ли, а мы здесь работаем, чтобы сказку сделать былью, понимаете ли.
из к/ф "Кавказская пленница"

В предыдущей главе, при рассмотрении конструкторов, мы уже упоминали такой полезный механизм, как *перегрузка методов*. В этой главе мы обсудим его подробнее. Помимо этого, мы узнаем, что такое *операторный метод* и *перегрузка операторов*. Так что, в общем и целом, эта глава посвящена методам - и немножко функциям. Помимо этого, мы познакомимся с *условным оператором*, а также узнаем, что такое *рекурсия*. А еще мы увидим, как используется оператор цикла `while`.

2.1. Перегрузка методов

Это же вам не лезгинка, а твист!
из к/ф "Кавказская пленница"

Перегрузка методов - это механизм, который позволяет создавать несколько методов с одним и тем же именем. В результате создается иллюзия, что можно вызывать метод с разными аргументами. Мы, если это не вызывает путаницы, будем говорить о таких методах как о разных версиях одного метода (хотя на самом деле методы разные).



На заметку

Мы говорим о перегрузке методов, хотя все сказанное относится и к функциям.

Технически перегрузка реализуется очень просто - достаточно описать несколько методов, у которых одно и то же имя, при этом имеются различия по типу возвращаемого результата и/или списку аргументов (имеется в виду количество и/или тип аргументов). Другими словами, разные версии одного метода должны как-то отличаться - настолько, чтобы при вызове метода по списку переданных ему аргументов и контексту вызова метода можно было однозначно определить, какая версия метода используется.



На заметку

Тип результата метода, его имя и список аргументов - все вместе называется *прототипом метода*. Таким образом, при перегрузке разные версии метода имеют общее имя, но отличаются прототипами.

Перегрузку методов проиллюстрируем все на том же примере с банковским депозитом. На этот раз задачу немного усложним. А именно, мы позволим вкладчику, по прошествии определенного времени после внесения на депозит базовой суммы, сделать еще один взнос (или снять часть суммы - формально это сводится к размещению на депозите отрицательной суммы). Помимо этого, мы внесем в программный код ряд технических новшеств. В частности, появится несколько новых вспомогательных методов. Большинство членов класса станут закрытыми. Но самое главное изменение, конечно, связано с наличием в программном коде перегруженных методов.



На заметку

Перегрузка может выполняться и для метода, аргументы которого имеют значения по умолчанию. Объединение двух механизмов (перегрузка и значения аргументов по умолчанию), хотя это и не запрещено, нередко приводит к ошибкам. Например, у метода всего один аргумент, и у аргумента есть значение по умолчанию. Мы перегружаем метод, создавая версию метода без аргументов. В этом случае, если метод вызывается без аргументов, то совершенно неясно, что имеется в виду: первая версия метода с пропущенным аргументом (то есть нужно использовать первую версию метода со значением аргумента по умолчанию) или вторая версия метода (версия без аргументов). Хотя такого рода ошибки отслеживаются на этапе компиляции, лучше их не допускать вовсе.

Также здесь мы познакомимся с *условным оператором*, который позволяет выполнять различные блоки команд в зависимости от того, истинно или нет некоторое условие. Программный код, о котором идет речь, представлен в листинге 2.1.

Листинг 2.1. Класс BankAccount с перегрузкой методов

```
#include <iostream>
#include <string>
using namespace std;
// Начало описания класса:
class BankAccount{
    // Закрытые члены класса:
private:
    // Поле для записи имени вкладчика:
    string name;
    // Поле для записи базовой суммы депозита:
    double money;
    // Поле для записи процентной ставки:
    double rate;
    // Время, на которое размещается депозит:
    int time;
    // Метод для вычисления итоговой суммы депозита:
    double findValue(double m,double r,double t){
```

```

// Локальная переменная
// для записи результата метода:
double res=m;
// Оператор цикла:
for(int i=1;i<=t;i++){
// Вычисление результата:
res=res*(1+r/100);
}
// Результат метода:
return res;
}
// Метод для отображения полей таблицы
void showTable(bool style=false){
// Отображаем строку заголовков таблицы:
cout<<"| Вкладчик\t"<<"| Сумма\t"<<"| Время\t";
if(style){
cout<<"| Сумма\t"<<"| Время\t";
}
cout<<"| % \t"<<"| Всего | \n";
}
// Метод (с двумя аргументами) для отображения
// фактических данных вкладчика:
void showData(double m,int t){
// Отображается имя вкладчика:
cout<<"| "<<name<<"\t";
// Отображаем базовую сумму вклада:
cout<<"| "<<money<<"\t";
// Отображаем время размещения
// базовой суммы вклада:
cout<<"| "<<time<<"\t";
// Отображаем дополнительную сумму вклада:
cout<<"| "<<m<<"\t";
// Отображаем время размещения дополнительной
// суммы вклада:
cout<<"| "<<time-t<<"\t";
// Отображаем процентную ставку:
cout<<"| "<<rate<<"\t";
// Отображаем итоговую сумму депозита:
cout<<"| "<<result(m,t)<<" | \n";
// Пустая строка:
cout<<endl;
}
// Метод (без аргументов) для отображения
// данных таблицы:
void showData(){
// Отображается имя вкладчика:

```

```

cout<<"| "<<name<<"\t";
    // Отображаем базовую сумму вклада:
cout<<"| "<<money<<"\t";
    // Отображаем время размещения
    // базовой суммы вклада:
cout<<"| "<<time<<"\t";
    // Отображаем процентную ставку:
cout<<"| "<<rate<<"\t";
    // Отображаем итоговую сумму депозита:
cout<<"| "<<result()<<" |\n";
    // Пустая строка:
cout<<endl;
    }
    // Метод (без аргументов) для вычисления
    // итоговой суммы депозита:
double result(){
    // Результат метода:
    return findValue(money,rate,time);
}

    // Метод (с двумя аргументами) для вычисления
    // итоговой суммы депозита:
double result(double m,int t){
    // Локальные переменные для записи отдельных сумм:
    double m1,m2;
    // "Основной" вклад:
    m1=findValue(money,rate,time);
    // Проверяем корректность параметров:
    if(t>time){
        // Результат метода:
        return m1;
    }else{
        // "Дополнительный" вклад:
        m2=findValue(m,rate,time-t);
    // Результат метода:
    return m1+m2;
    }
}

    // Открытые члены класса:
public:
    // Метод (четыре аргумента) для присваивания
    // значения полям:
    void setAll(string n,double m,double r,int t){
    // Значения полей:
        name=n;
        money=m;
        rate=r;
    }
}

```

```

time=t;
    }
    // Метод (с одним аргументом) для присваивания
// значения полям:
void setAll(string n){
// Вызываем версию метода с четырьмя аргументами:
setAll(n,100,13,3);
    }
    // Метод (с двумя аргументами) для отображения
// данных вкладчика:
void show(double m,int t){
// Отображаем строку заголовков таблицы:
showTable(true);
    // Отображаем данные вкладчика:
showData(m,t);
    }
    // Метод (с одним аргументом) для отображения
// данных вкладчика:
void show(double m){
// Вызываем версию метода с двумя аргументами:
show(m,1);
    }
    // Метод (без аргументов) для отображения
// данных вкладчика:
void show(){
    // Отображаем строку заголовков таблицы:
showTable();
    // Отображаем данные вкладчика:
showData();
    }
    // Конструктор класса (с одним аргументом):
BankAccount(stringname){
    // Присваиваем значения полям:
setAll(name);
// Отображаем данные вкладчика:
show();
    }
    // Конструктор класса (с четырьмя аргументами):
BankAccount(string name,double m,double r,int t){
// Присваиваем значения полям:
setAll(name,m,r,t);
    // Отображаем данные вкладчика:
show();
    }
}; // Окончание описания класса
// Главная функция программы:

```

```
int main() {
    // Создаем первый объект:
    BankAccount ivanov("Иванов И.И.");
    // Проверяем эффект от дополнительного вклада:
    ivanov.show(30, 2);
    // Еще одна попытка:
    ivanov.show(29);
    // Создаем второй объект:
    BankAccount petrov("Петров П.П.", 90, 18, 4);
    // Меняем параметры вклада и вкладчика:
    petrov.setAll("Сидоров С.С.");
    // Проверяем результат:
    petrov.show();
    // Обработка некорректной ситуации:
    petrov.show(50, 5);
    // Завершение программы:
    return 0;
}
```

Результат выполнения этой программы представлен ниже:

Результат выполнения программы (из листинга 2.1)

Вкладчик	Сумма	Время	%	Всего	
Иванов И.И.	100	3	13	144.29	
Вкладчик	Сумма	Время	Сумма	Время	%
Иванов И.И.	100	3	30	1	13
Вкладчик	Сумма	Время	Сумма	Время	%
Иванов И.И.	100	3	29	2	13
Вкладчик	Сумма	Время	%	Всего	
Петров П.П.	90	4	18	174.49	
Вкладчик	Сумма	Время	%	Всего	
Сидоров С.С.	100	3	13	144.29	
Вкладчик	Сумма	Время	Сумма	Время	%
Сидоров С.С.	100	3	50	-2	13

Имеет смысл проанализировать основные блоки программы. Как и в предыдущем случае, у класса `BankAccount` имеется четыре закрытых поля: текстовое поле `name` для записи имени вкладчика, а также числовые поля `money` (сумма депозита), `rate` (процентная ставка по депозиту) и `time` (время, на которое размещается депозит). Что касается вычисления итого-

вой суммы депозита, то здесь произошли некоторые изменения. А именно, мы для удобства описываем закрытый метод `findValue()`. У метода три аргумента, определяющие, соответственно, сумму вклада (аргумент `m`), процентную ставку (аргумент `r`) и время размещения депозита (аргумент `t`). На основании этих параметров вычисляется итоговая сумма депозита, которая и возвращается как результат метода.



На заметку

Фактически, методом `findValue()` выполняются те же самые вычисления, что в предыдущих примерах выполнялись методом `result()`. Принципиальное отличие в том, что метод `result()` для вычислений использовал значения полей, а в методе `findValue()` используются аргументы. По большому счету, в методе `findValue()` мы реализовали алгоритм вычисления итоговой суммы депозита. Необходимые для реализации этого алгоритма числовые значения передаются аргументами методу.

Данные о вкладчике мы планируем выводить в консольное окно в виде импровизированной таблицы. Для этой таблицы необходимо отобразить строку заголовков. Здесь мы создали вспомогательный закрытый метод `showTable()`, которым в консоль выводится строка заголовков таблицы, и который будет вызываться в методе `show()`. Предполагается два режима вывода информации: для случая, когда вкладчик не делает дополнительный вклад, и для случая, когда такой вклад делается. В каждом из этих случаев таблица выглядит по-разному. Так, если дополнительного вклада нет, то у таблицы в столбцах названием

Вкладчик отображается имя вкладчика (значение поля `name`), в столбце с названием Сумма отображается первоначальная сумма вклада (значение поля `money`), в столбце с названием Время отображается время, на которое размещается вклад (значение поля `time`), значение процентной ставки отображается в столбце с названием % (символ процента), а в столбце Всего отображается итоговая сумма по депозиту. В случае, когда в расчет нужно принять еще и дополнительный вклад, пара столбиков Сумма и Время отображается дважды. Во втором столбце Сумма отображается сумма дополнительного вклада, а во втором столбце Время отображается время, на которое этот дополнительный вклад вносится. Поэтому у метода `showTable()` имеется аргумент, значение которого определяет режим вывода. Аргумент называется `style` и относится к логическому типу (тип `bool`). Переменные логического типа могут принимать два значения: `true` (истина) или `false` (ложь). Аргумент `style` имеет значение по умолчанию `false`. То есть если вызвать метод `showTable()` без аргумента, то метод будет выполняться так, как если бы аргументом было указано значение `false`.

В теле метода командой `cout<<" | Вкладчик\t"<<" | Сумма\t"<<" | Время\t"` начинаем отображать строку заголовков таблицы. Специальный символ `\t` (считается, что это один символ) в тексте является инструкцией *табуляции*. После выполнения команды в консольное окно будет выведено название трех столбиков таблицы (разделенные вертикальной чертой). На следующем этапе в дело вступает *условный оператор*. После ключевого слова `if` в круглых скобках указывается логическое значение (в данном случае это аргумент `style` метода). Если логическое значение равно `true`, выполняется блок команд после ключевого слова `if`: в рассматриваемом коде там всего одна команда `cout<<" | Сумма\t"<<" | Время\t"`, которой в консольном окне отображается название еще двух столбцов таблицы. Если логическое значение равно `false`, эти команды не выполняются.

Подробности

Здесь мы имеем дело с сокращенной формой условного оператора. В общем случае структура условного оператора такая: после ключевого слова `if` в круглых скобках указывается проверяемое условие. Если условие истинно (значение `true`), то выполняется блок команд после ключевого слова `if`. Если условие ложно (значение `false`), то выполняется блок команд после ключевого слова `else`. Допускается форма условного оператора без `else`-блока. Именно с таким оператором мы имеем дело.

Также стоит учесть, что в качестве условия может использоваться не только выражение логического типа `bool`, но и числовое значение. При этом отличное от нуля значение интерпретируется как истина (значение `true`), а нулевое значение интерпретируется как ложь (значение `false`).

Для отображения фактических данных вкладчика предназначен метод `showData()`. У этого метода две версии - с двумя аргументами и без аргументов. Таким образом, имеет место перегрузка метода `showData()`. В версии метода с двумя аргументами первый определяет величину дополнительного вклада (аргумент `m` типа `double`), а второй определяет время, через которое после размещения основного депозита вносится дополнительный платеж (аргумент `t` типа `int`). Эта версия метода будет вызываться "в связке" вместе с вызовом метода `showTable()` с аргументом `true` - то есть для случая, когда кроме основного вклада на депозит вносится и дополнительный вклад.

В теле метода последовательно отображаются значения полей `name`, `money` и `time`. Затем отображается значение параметра `m`, а также значение выражения `time-t`. Последнее выражение, в котором вычисляется разность значения поля `time` и аргумента `t` - это время, на которое под проценты размещается дополнительная сумма. После этого отображается значение поля `rate` (процентная ставка), а также вычисляется и отображается итоговая сумма депозита. Для вычисления итоговой величины депозита мы вы-

зываем метод `result()` с аргументами `m` и `t` (инструкция `result(m, t)`). Ранее (в предыдущих примерах) метод `result()` вызывался нами только без аргументов. Здесь мы его перегрузили. Детально код этого метода рассматривается позже.



На заметку

Комбинация `\n` в тексте, выводимом в консольное окно, является инструкцией перехода к новой строке.

Версия метода `showData()` без аргументов организована проще по сравнению с версией этого же метода, но с двумя аргументами. Если говорить в общем, то теперь отсутствуют команды отображения значений суммы дополнительного вклада и времени его размещения (таких параметров просто нет), а при вычислении итоговой денежной суммы на депозите вызывается метод `result()` без аргументов.

Что касается последнего, то он определен следующим образом. При вызове метода `result()` без аргументов в теле метода вызывается метод `findValue()` с аргументами `money, rate` и `time`, а полученное значение возвращается результатом метода `result()`. Вся инструкция выглядит как `return findValue(money, rate, time)`. Версия метода `result()` с двумя аргументами определена немного сложнее. В теле метода объявляются две локальные переменные `m1` и `m2`, обе типа `double`. Командой `m1=findValue(money, rate, time)` в переменную `m1` записывается значение итоговой суммы по основному вкладу. Затем в условном операторе проверяется условие `t>time` (соответствующее выражение указано в круглых скобках после ключевого слова `if`).

Истинность условия означает, что параметр `t`, переданный вторым аргументом методу `result()` (и означающий время, по прошествии которого на депозит вносится дополнительная сумма), превышает время (поле `time`), на которое на депозит помещается основная сумма вклада. В нашем представлении такая ситуация некорректна. В этом случае мы исходим из того, что дополнительный вклад банком не принимается. Другими словами, попытка внести дополнительную сумму на депозит игнорируется. Командой `return m1` рассчитанное ранее значение переменной `m1` возвращается как результат метода.

Напомним, это происходит в случае, если выполнено условие `t>time`. Если указанное условие не выполняется, в игру вступает блок программного кода, указанный после инструкции `else` в условном операторе. В частности, `else`-блок содержит две команды. Командой `m2=findValue(m, rate, time-t)` вычисляется итоговая сумма на депозите по дополнительному вкладу, а за-

тем результат метода возвращается командой `return m1+m2` как сумма итоговых значений по основному и дополнительному вкладам.



На заметку

Выше при расчетах мы исходили из следующих соображений: если сначала вносится сумма M на срок T , а затем через время t вносится дополнительная сумма m , то это эквивалентно двум отдельным вкладам (под одни и те же проценты): один сумма M на время T , второй сумма m на время $T-t$.

Таким образом, мы использовали перегрузку метода `result()` и описали две версии метода: без аргументов и с двумя аргументами. Это означает, что метод `result()` можно вызывать без аргументов или с двумя аргументами.

Среди открытых членов класса `BankAccount` есть перегруженный метод `setAll()` (две версии - с четырьмя аргументами и одним аргументом), перегруженный метод `show()` (три версии - с двумя аргументами, с одним аргументом и без аргументов) и перегруженный конструктор класса (две версии - с одним аргументом и четырьмя аргументами).

Метод `setAll()` предназначен для присваивания значения полям объекта класса. Метод не возвращает результат. Если у метода четыре аргумента (один текстовый и два числовых), то значения, переданные аргументами методу, последовательно записываются в поля `name`, `money`, `rate` и `time`. Если же у метода только один текстовый аргумент, то этот аргумент определяет значение поля `name`. Для остальных полей используются постоянные значения (для поля `name` значение 100, для поля `rate` значение 13, а для поля `time` значение 3). Тело метода `setAll()` в этом случае содержит всего одну команду `setAll(n, 100, 13, 3)`. То есть в теле версии метода с одним аргументом `n` вызывается версия этого же метода, но с четырьмя аргументами. Другими словами, описывая метод, мы в теле метода вызываем его же, но с другим набором аргументов. На первый взгляд это может показаться неправильным, хотя конечно код корректный. Чтобы понять идеологию такого подхода, полезно вспомнить, что методы с одинаковыми названиями и разным количеством аргументов - это на самом деле разные методы. Поэтому рассматриваемый случай принципиально не отличается от ситуации, когда в теле одного метода вызывается другой метод.



На заметку

Существует такое понятие, как *рекурсия*. В этом случае при определении метода в теле этого метода вызывается он же, но с другим аргументом (или аргументами). Но при рекурсии и описываемый, и вызываемый метод - это один и тот же метод, просто у них изменяются *значения аргументов*. В рассмотренном выше случае о рекурсии речь не идет, поскольку вызываются разные версии метода. У них разное количество аргументов. Технически это разные методы.

Метод `show()` предназначен для отображения данных по вкладу. Если метод вызывается с двумя аргументами, то первый аргумент означает денежную сумму дополнительного вклада, а второй аргумент определяет время размещения дополнительного вклада. В теле метода командой `showTable(true)` отображается расширенная строка заголовков таблицы, а затем командой `showData(m, t)` отображаются фактические данные вкладчика.

Версия метода `show()` с одним аргументом описана так, что в этом случае вызывается версия метода `show()` с двумя аргументами: первый аргумент определяется по аргументу исходного метода, а в качестве второго аргумента указано постоянное (единичное) значение. Ситуация аналогична к предыдущей, когда мы рассматривали разные версии метода `setAll()`.

Если метод `show()` вызывается без аргументов, то, в соответствии с программным кодом этой версии метода, сначала для отображения строки заголовков таблицы вызывается (без аргументов) метод `showTable()`, а затем вызывается (также без аргументов) метод `showData()`, в результате чего отображаются данные по вкладу.

Конструктор класса с одним аргументом (параметр `name`) содержит команду `setAll(name)`, которой полям присваиваются значения, а затем командой `show()` значения характеристик (полей) объекта выводятся в консольное окно в виде небольшой таблицы.



На заметку

Возможно, кого-то смутит то обстоятельство, что имя аргумента `name` конструктора совпадает с именем поля класса. Правило здесь такое: если локальная переменная (а аргумент конструктора или метода "имеют силу" локальной переменной) называется так же, как поле, то приоритет остается за локальной переменной. Другими словами, в теле конструктора с аргументом `name` ссылка `name` означает этот аргумент, несмотря на то, что у класса есть поле с таким же названием. Как в подобных случаях обращаться к полям, мы узнаем немного позже.

Конструктор класса с четырьмя аргументами содержит команду вызова метода `setAll()` с этими же самыми аргументами, после чего командой `show()` отображается информация о созданном объекте.

Весь этот программный код определяет функциональность объектов класса `BankAccount`. Чтобы понять, что происходит при выполнении программы, необходимо проанализировать программный код главной функции `main()`.

В теле функции `main()` командой `BankAccount ivanov("Иванов И.И.")` создается объект `ivanov` класса `BankAccount`. Поскольку конструктору передается только один аргумент (определяющий имя вкладчи-

ка - значение поля `name` объекта), то числовые поля получают значения 100 (сумма начального вклада - поле `money`), 13 (процентная ставка - поле `rate`) и 3 (время, на которое размещается сумма - поле `time`). Также стоит напомнить, что при создании объекта автоматически отображается информация о нем - все благодаря наличию инструкции вызова метода `show()` в конструкторе класса (в каждой из двух его версий). Поэтому первая импровизированная таблица в консольном окне появляется вследствие создания объекта `ivanov`.

Чтобы увидеть, каким будет эффект от дополнительного вклада, используем команду `ivanov.show(30, 2)`. В данном случае первый аргумент метода `show()` определяет сумму дополнительного вклада, а второй аргумент определяет промежуток времени, по прошествии которого этот вклад вносится на депозит. В результате выполнения команды объект `ivanov` не меняется, но в консольном окне появляется новая таблица - вторая сверху. По сравнению с предыдущей таблицей, у этой на два столбика больше.

Если воспользоваться командой `ivanov.show(29)`, то это все равно, как если бы при вызове метода `show()` мы указали вторым аргументом значение 1. Результатом выполнения команды является третья сверху таблица в консольном окне вывода программы.

Второй объект создаем командой `BankAccount petrov ("Петров П.П.", 90, 18, 4)`, явно указав все четыре аргумента для конструктора. В результате создается объект `petrov`, а информация о нем отображается в консольном окне (четвертая таблица сверху в консоли).

Если мы воспользуемся командой `petrov.setAll("Сидоров С.С.")`, то в результате будут изменены значения полей объекта `petrov`. В данном случае методу `setAll()` передан только один текстовый аргумент. Соответствующее значение присваивается полю `name` объекта `petrov`. Значения для полей `money`, `rate` и `time` явно не указаны. Эти поля получают соответственно значения 100, 13 и 3. Чтобы проверить результат, используем команду `petrov.show()` (пятая таблица сверху в консольном окне вывода).

Еще одна, некорректная с нашей точки зрения, ситуация тестируется командой `petrov.show(50, 5)`. В данной инструкции второй аргумент метода `show()` превышает значение поля `time` объекта `petrov`, из которого вызывается метод. В таком случае, напомним, дополнительный вклад при вычислении итоговой суммы депозита игнорируется (см. программный код метода `result()` в листинге 2.1). Результат вызова метода `show()` с такими аргументами из объекта `petrov` представлен в консоли вывода программы самой нижней таблицей. Хотя дополнительный взнос при вычислении итоговой суммы депозита не учитывается, две дополнительные колонки с

суммой дополнительного взноса и формально отрицательным временем, на которое как бы размещается эта сумма, в таблице все же отображаются.

2.2. Перегрузка функций

Я мечтал об этом всю свою сознательную жизнь.

из к/ф "Ирония судьбы или с легким паром"

Перегружать можно и функции. Как иллюстрацию рассмотрим небольшой пример, представленный в листинге 2.2.

Листинг 2.2. Перегрузка функций

```
#include <iostream>
using namespace std;
// Функция без аргументов:
void show(){
    cout<<"Функция show() без аргументов!\n";
}
// Функция с одним аргументом:
void show(string txt){
    cout<<"Аргумент функции show(): "+txt<<endl;
}
// Главная функция:
int main(){
    show();
    show("текст");
    return 0;
}
```

В результате выполнения этого программного кода в консольном окне отображается два сообщения:

Результат выполнения программы (из листинга 2.2)

```
Функция show() без аргументов!
Аргумент функции show(): текст
```

Здесь мы описываем две версии функции `show()`. В одном случае функция описана без аргументов. Во втором случае у функции текстовый аргумент. Функция в обоих случаях не возвращает результат. В зависимости от того, передан ли функции аргумент при вызове, отображается сообщение о том, что у функции нет аргумента, или сообщение с указанием значения аргумента.

2.3. Операторные методы

Были демоны, не отрицаю. Но они самоликвидировались.

из к/ф "Иван Васильевич меняет профессию"

Удивительная возможность, которая есть в C++, связана с *операторными методами*. Операторные методы - это методы, которые определяют "поведение" основных арифметических (и не совсем арифметических) операторов в случае, когда среди операндов есть объекты пользовательских классов. Речь идет о том, что такие операции, например, как сложение или умножение, на самом деле можно интерпретировать как вычисление некоторой функции, аргументами которой являются операнды соответствующего выражения.



На заметку

Операторы бывают бинарные и унарные (правда, в C++ есть еще и один тернарный оператор). У бинарных операторов (таких, как +, - или =) два операнда, а у унарных операторов (таких, как ++ или --) - один операнд. Операндами называются переменные, к которым применяется соответствующая операция. Для бинарного оператора операнды указываются справа и слева от оператора. Для унарного оператора операнд указывается перед или после оператора - в зависимости от того, как используется оператор.

Когда мы говорим об *операторном методе* для некоторого *бинарного оператора*, то имеется в виду метод, результат вызова которого идентичен применению соответствующего оператора. Роль операндов играют объект, из которого вызывается метод (первый операнд) и аргумент операторного метода (второй операнд). У операторных методов для *унарных операторов* аргументов нет - единственным операндом выступает объект, из которого вызывается метод.

Помимо операторных методов, существуют *операторные функции*. Общая идея такая же, как с операторными методами, только в случае функций все операнды реализуются через аргументы операторной функции. Как это выглядит, будет показано на небольшом примере в конце главы.

Такая "интерпретация" имеет свое практическое выражение в том, что при описании классов можно создавать специальные операторные методы. Это откроет путь для применения к объектам данного класса арифметических (в основном) операторов. Поясним сказанное на примере бинарного оператора, который формально обозначим символом ☺. Предположим, мы хотим

обеспечить корректную обработку выражений вида $A \odot B$, где A является объектом описываемого нами класса (для удобства будем называть его `Класс`). Второй операнд B может относиться к базовому типу или быть объектом какого-то иного класса (в том числе и класса `Класс`). В этом случае в классе `Класс` необходимо описать операторный метод для оператора $\odot$. Операторный метод описывается точно так же, как и обычный (не операторный метод). Просто у операторного метода специальное название, которое получается объединением ключевого слова `operator` и самого оператора. В данном случае операторный метод для оператора $\odot$ должен называться `operator \odot`. В круглых скобках после имени операторного метода указывается аргумент, который является вторым операндом. Первым операндом является объект, из которого вызывается операторный метод.



На заметку

Еще раз подчеркнем: для бинарного оператора операторный метод описывается с одним аргументом (второй операнд для оператора, первый операнд - объект вызова). Для унарного оператора операторный метод описывается без аргументов (единственный операнд для унарного оператора - объект вызова операторного метода). Во всем остальном описание операторного метода мало чем отличается от описания обычного метода.

Соответственно, операторная функция для бинарного оператора описывается с двумя аргументами, а операторная функция для унарного оператора описывается с одним аргументом. Но это будет потом.

В качестве иллюстрации к использованию операторных методов рассмотрим пример - вариацию на тему размещения денежной суммы на депозит. Поскольку нас будут интересовать в первую очередь операторные методы, мы по возможности упростим весь прочий программный код. Что касается операторных методов, то мы опишем их несколько. В частности, мы добьемся следующих эффектов:

- прибавляя к объекту класса `BankAccount` целое число, получим новый объект класса `BankAccount`, у которого по сравнению с исходным объектом поле `time` увеличено на значение, определяемое целочисленным операндом;
- прибавляя к объекту класса `BankAccount` действительное число (число в формате с плавающей точкой), получим новый объект класса `BankAccount`, у которого по сравнению с исходным объектом поле `money` увеличено на значение, определяемое числовым операндом;

- умножая объект класса `BankAccount` на число, получим новый объект класса `BankAccount`, у которого по сравнению с исходным объектом поле `rate` увеличено на значение, определяемое числовым множителем;
- вычисляя разность двух объектов класса `BankAccount`, получаем действительное число, определяющее разность итоговых денежных сумм на депозитах, определяемых соответствующими объектами (то есть объектами, разность которых вычисляется);
- также мы опишем операторный метод для унарного оператора! (восклицательный знак - он же логический *оператор отрицания*) таким образом, что применение этого оператора к объекту класса `BankAccount` возвращает значение итоговой суммы депозита по вкладу, параметры которого определяются объектом - операндом.

Программный код, в котором реализованы все эти возможности, представлен в листинге 2.3.

Листинг 2.3. Класс `BankAccount` с операторными методами

```
#include <iostream>
#include <string>
using namespace std;
// Класс с перегрузкой операторных методов:
class BankAccount{
    // Закрытые члены класса:
private:
    // Поля:
    string name;
    double money;
    double rate;
    int time;
    // Метод для вычисления итоговой суммы вклада
    // (используется рекурсивный вызов):
    double findValue(double m, double r, int t){
        // Если период размещения вклада равен нулю:
        if(t==0){
            return m;
            // Период размещения вклада отличен от нуля:
        }else{
            // Используем рекурсию:
            return findValue(m, r, t-1) * (1+r/100);
        }
    }
    // Вычисление итоговой суммы вклада:
```

```

double total(){
return findValue(money,rate,time);
}
// Открытые члены класса:
public:
// Метод для присваивания значений полям объекта:
void set(string n,double m,double r,int t){
// Значения полей:
    name=n;
    money=m;
    rate=r;
    time=t;
}
// Конструктор с четырьмя аргументами:
BankAccount(string n,double m,double r,int t){
// Значения полей:
set(n,m,r,t);
}
// Конструктор класса без аргументов:
BankAccount(){
// Значения полей:
set("Anonimus",0,0,0);
}
// Метод для отображения данных об объекте:
void show(){
    cout<<"Данные по банковскому счету.\n";
    cout<<"Вкладчик:\t"<<name<<endl;
    cout<<"Сумма:\t"<<money<<endl;
    cout<<"Ставка %:\t"<<rate<<endl;
    cout<<"Время:\t"<<time<<endl;
    cout<<"Всего:\t"<<total()<<endl;
    cout<<endl;
}
// Операторный метод для сложения объекта
// и целого числа:
BankAccount operator+(int t){
// Локальный объект:
BankAccount tmp;
// Значения полей локального объекта:
tmp.set(name,money,rate,time+t);
// Результат операторного метода:
return tmp;
}
// Операторный метод для сложения объекта
// и действительного числа:
BankAccount operator+(double m){

```

```

        // Локальный объект:
BankAccount tmp;
        // Значения полей локального объекта:
tmp.set(name,money+m,rate,time);
// Результат операторного метода:
return tmp;
    }
    // Операторный метод для умножения объекта
    // на число:
BankAccount operator*(double x){
    // Локальный объект:
BankAccount tmp;
    // Значения полей локального объекта:
tmp.set(name,money,rate+x,time);
// Результат операторного метода:
return tmp;
    }
    // Операторный метод для унарного оператора
    // "восклицательный знак" (то есть !):
double operator!(){
    // Результат операторного метода:
return total();
    }
    // Операторный метод для вычитания объектов:
double operator-(BankAccount tmp){
    // Результат операторного метода:
return total()-!tmp;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Создание объекта:
BankAccount ivanov("Иванов И.И.",100,12,3);
// Отображаем параметры объекта:
ivanov.show();
    // Еще один объект:
BankAccount fellow;
    // Сложение объекта и целого числа:
fellow=ivanov+2;
    // Результат:
fellow.show();
// Увеличиваем сумму вклада
// и процентную ставку:
((ivanov+25.0)*4).show();
// Разность двух объектов:
double dif=fellow-ivanov;

```

```
// Проверяем результат:
cout<<"Разница итоговых сумм:\t"<<endl;
return 0;
}
```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 2.3)

Данные по банковскому счету.

```
Вкладчик:      Иванов И.И.
Сумма:         100
Ставка %:      12
Время:        3
Всего:         140.493
```

Данные по банковскому счету.

```
Вкладчик:      Иванов И.И.
Сумма:         100
Ставка %:      12
Время:        5
Всего:         176.234
```

Данные по банковскому счету.

```
Вкладчик:      Иванов И.И.
Сумма:         125
Ставка %:      16
Время:        3
Всего:         195.112
```

Разница итоговых сумм: 35.7414

Далее мы кратко проанализируем программный код, не имеющий отношения к операторным методам, и более подробно - программный код, связанный с реализацией и использованием операторных методов.

Как всегда, в классе `BankAccount` есть четыре закрытых поля (текстовое поле `name` и числовые поля `money`, `rate` и `time`). Также есть закрытый метод `findValue()` для вычисления итоговой суммы вклада. Такой метод у класса был и ранее, но теперь вместо явного вычисления итоговой суммы с помощью оператора цикла мы использовали *рекурсивный вызов*. А именно, метод `findValue()` описан с тремя аргументами: аргументы `m` и `r` типа `double`, а также аргумент `t` типа `int`. В операторе цикла (в условном операторе) проверяется равенство нулю значения аргумента `t`. Если соответствующее условие истинно, это формально означает, что вклад разме-

щается на нулевой промежуток времени и итоговая сумма по депозиту равна начальной сумме вклада, то есть определяется параметром `m`. Поэтому в блоке, выполняемом при истинном условии `t==0`, имеется единственная команда `return m`, которой работа метода завершается, а в качестве результата возвращается значение параметра `m`. В противном случае, то есть если условие `t==0` ложно, выполняется команда `return findValue(m, r, t-1) * (1+r/100)` в `else`-блоке. Здесь для вычисления результата метода вызывается этот же метод, но теперь на единицу уменьшен его третий аргумент.



На заметку

При вычислении итоговой суммы по депозиту в соответствии с принципом рекурсивного вызова метода мы исходили из следующих соображений. Для удобства обозначим через $F(M, r, T)$ итоговую сумму по вкладу M на период T при фиксированной процентной ставке r (в процентах). За период времени $T - 1$, по определению, итоговая сумма на депозите равна $F(M, r, T - 1)$. За год эта сумма превратится в $(1 + \frac{r}{100})F(M, r, T - 1)$. Это есть ни что иное, как конечная сумма на депозите, полученная на основе первоначального вклада M на период T , то есть $F(M, r, T)$. Таким образом, имеем рекуррентное соотношение $F(M, r, T) = (1 + \frac{r}{100})F(M, r, T - 1)$. Именно этим соотношением мы воспользовались при определении метода `findValue()`.

При вызове метода `findValue()` программный код выполняется в общих чертах так:

- Проверяется значение третьего аргумента метода. Если это значение равно нулю, то методом возвращается результат - значение первого аргумента метода.
- Если значение третьего аргумента метода не равно нулю, начинается вычисляться выражение в `else`-блоке условного оператора. Это выражение, кроме прочего, содержит инструкцию вызова метода `findValue()`, но третий аргумент в этом случае на единицу меньше. Поэтому при запущенном методе `findValue()` этот же метод запускается еще раз, но с уменьшенным на единицу третьим аргументом. Чтобы различить процессы, исходный метод будем называть "альфа", а вызываемый при его вычислении метод - "браво".
- При выполнении программного кода метода "браво" `findValue()` проверяется третий аргумент метода. Если аргумент равен нулю, вычисляется значение метода "браво", и это значение используется

для вычисления значения метода "альфа". Процесс на этом завершается.

- Если третий аргумент метода "браво" не равен нулю, при вычислении значения метода "браво" снова вызывается метод `findValue()` (назовем его "чарли"), и его третий аргумент еще на единицу меньше.
- Вычисление значения метода "чарли" начинается с проверки третьего аргумента. Если аргумент равен нулю, вычисляется значение метода "чарли", это значение используется для получения значения метода "браво", а значение метода "браво", в свою очередь, позволяет вычислить значение метода "альфа".
- Если третий аргумент метода "чарли" не равен нулю, для вычисления значения метода вызывается метод `findValue()` (назовем его "дельта" - его третий аргумент на единицу меньше, чем при вызове метода "чарли"), и так далее. Последовательность "матрешечных" вызовов метода `findValue()` продолжается до тех пор, пока при очередном вызове метода третий аргумент метода не окажется равен нулю. После этого методы начинают возвращать значения (в обратной последовательности к тому, как они вызывались - то есть в соответствии с принципом стека).



На заметку

Хотя рекурсивное определение методов обычно выглядит простым и элегантным, насчет эффективности могут возникнуть серьезные опасения. Дело в том, что реализованные через рекурсивный вызов методы очень серьезно расходуют системные ресурсы и нередко влияют, не в лучшую сторону, на скорость выполнения программного кода.

Что касается использованных выше "идентификаторов" для обозначения вызовов методов, то *Alpha* (альфа), *Bravo* (браво), *Charlie* (чарли) и *Delta* (дельта) - это из западноевропейского фонетического алфавита.

Однако описанный выше метод `findValue()` напрямую для вычисления суммы депозита не используется. Мы его вызываем в теле метода `total()`. При этом аргументами методу `findValue()` передаются значения полей объекта.



На заметку

В принципе, использованная нами схема (вызова метода в методе) может показаться нелогичной. Но для реализации рекурсии она удобна. Дело в том, что при

рекурсивном вызове метода необходимо изменять его аргументы. Если аргументами выступают поля объекта, это сделать довольно проблематично. Мы применили "военную хитрость": сначала через рекурсию описали метод `findValue()` с аргументами, никак не связанными с полями объекта, а затем описали метод `total()`, в котором выполняется вызов метода `findValue()`, а аргументами указываются поля объекта.

Но это еще не все. Внимательный читатель, скорее всего, задастся вопросом о том, что будет происходить с полями объекта при вызове метода `findValue()` в теле метода `total()`? Ведь аргументами методу `findValue()` передаются поля объекта. И хотя формально в теле метода эти поля не меняются, вопрос об их "уязвимости" остается открытым. А ответ простой: на полях объекта это никак не отразится. Причина в том, что по умолчанию в C++ методу аргументы передаются по значению: на самом деле передается не та переменная, которая формально указана аргументом, а ее копия. Все вычисления в теле метода выполняются с копией аргумента. Сам аргумент в вычислениях не участвует. Подробнее о режимах передачи аргументов методам мы еще будем говорить в книге.

Среди открытых членов класса есть метод `set()` с четырьмя аргументами. Назначение метода простое. С его помощью полям присваиваются значения. Также у класса имеется перегруженный конструктор: версия с четырьмя аргументами и версия без аргументов. В обоих случаях в теле конструктора вызывается метод `set()`, аргументами которому передаются значения для полей объекта.



На заметку

Конструктор без аргументов нам нужен для удобства. Дело в том, что при описании операторных методов мы создаем локальные объекты. Чтобы каждый раз не передавать конструктору четыре аргумента (которые на самом деле все равно будут переопределяться), мы и описали в классе `BankAccount` версию конструктора без аргументов. Какие при этом значения присваиваются полям объекта, в данном конкретном примере особой роли не играет.

Также в классе `BankAccount` описан метод `show()`, которым выполняется отображение в консольном окне данных об объекте. Данные отображаются построчно в формате *название поля/значение поля*. Все остальное в классе - описание операторных методов.

Операторный метод для сложения объекта и целого числа имеет формальное название `operator+`, в качестве типа результата для него указано имя класса `BankAccount` (результат метода - объект класса `BankAccount`), а аргумент метода называется `t` и относится к типу `int` (то есть это целое число). В теле метода командой `BankAccount tmp` создается локальный объект `tmp` класса `BankAccount`. Нам этот объект нужен, чтобы "вычислить" результат метода. Схема такая: мы создаем локаль-

ный объект, задаем нужные значения полям этого объекта, а затем возвращаем этот объект как результат метода. В данном случае командой `tmp.set(name, money, rate, time+t)` для объекта `tmp` значения всех полей (за исключением поля `time`) устанавливаются такими же, как и у объекта, являющегося первым операндом (напомним, по определению объект вызова является первым операндом для бинарного оператора - здесь это оператор сложения `+`). Поле `time` у объекта `tmp` на величину `t` больше, чем у первого операнда (объект, из которого вызывается метод). Командой `return tmp` объект `tmp` возвращается в качестве результата операторного метода.

Похожим образом описан операторный метод для вычисления суммы объекта и действительного числа типа `double` (в формате с плавающей точкой). По сравнению с предыдущим случаем различия такие: аргумент метода имеет тип `double`, а не `int`, а при присваивании значений полям локального объекта "добавку" получает поле `money`, а не поле `time`.



На заметку

Таким образом, результат добавления к объекту целого числа отличается от результата добавления к объекту числа в формате с плавающей точкой. Фактически, мы *перегрузили* операторный метод для оператора сложения.

Операторный метод для умножения объекта на число имеет название `operator*`. В качестве результат объект возвращает объект класса `BankAccount`, а аргументом метода (второй операнд для оператора умножения) является значение типа `double`. Метод описан так, что при умножении объекта на число получаем новый объект. Все поля этого объекта-результата определяются значениями полей объекта-операнда с той лишь разницей, что поле `rate` увеличивается на значение второго операнда.

Несколько иначе определяется операторный метод для унарного оператора! (вообще это оператор *логического отрицания* - в своем "классическом" варианте он *инвертирует* логические значения). Соответствующий операторный метод называется `operator!`. Поскольку оператор `!` унарный и первым операндом является объект вызова оператора, то аргументов у операторного метода нет. Более того, в качестве результата операторный метод возвращает значение типа `double`. Что это за значение, легко понять по единственной команде `return total()` в теле метода, которой результатом метода возвращается значение итоговой суммы по депозиту. Другими словами, применение унарного оператора `!` к объекту класса `BankAccount`

приведет к тому, что будет вычислено и возвращено как результат значение, получаемое вызовом метода `total()` (из данного объекта-операнда).

Унарный оператор `!` мы используем, когда определяем операторный метод `operator-` для оператора вычитания `-`. Операторный метод `operator-` определен так, что его аргумент - объект `tmp` класса `BankAccount`. В качестве результата методом возвращается число: результат вычисления выражения `total() - !tmp`. Учитывая, как определен операторный метод для оператора `!`, несложно догадаться, что выражение `total() - !tmp` есть ни что иное, как `total() - tmp.total()`. То есть это разность итоговых сумм по депозитам для разных объектов: от итоговой суммы депозита первого операнда вычитается итоговая сумма депозита второго операнда.

В главной функции программы `main()` сначала командой `BankAccount ivanov("Иванов И.И.", 100, 12, 3)` создается объект `ivanov` класса `BankAccount`. Командой `ivanov.show()` мы проверяем значение полей этого объекта. Затем создаем еще один объект, для чего используем команду `BankAccount fellow`. В команде `fellow=ivanov+2` к объекту `ivanov` прибавляется целое число, и результат записывается в объект `fellow`. Благодаря тому, что в классе `BankAccount` описан соответствующий операторный метод, такая команда является корректной. В результате значения полей объекта `ivanov` будут скопированы в поля объекта `fellow`, а поле `time` будет увеличено на 2. Результат проверяем с помощью команды `fellow.show()`.

Команда `((ivanov+25.0)*4).show()` может показаться странной, но она, разумеется, тоже корректна. Разберем это выражение. В результате вычисления инструкции `ivanov+25.0` получаем объект класса `BankAccount`. В данном случае в игру вступает операторный метод, описанный для случая, когда вторым операндом при сложении является число в формате с плавающей точкой.

Напомним, что в этом случае в новом объекте увеличивается значение поля `money`. И хотя результат операции `ivanov+25.0` мы ни в одну переменную не записываем, все равно "на выходе" имеем объект класса `BankAccount`. Этот объект умножается на 4 (инструкция `(ivanov+25.0)*4`). Понятно, что для вычисления результата такой операции будет задействован операторный метод `operator*()`. В результате получим еще один объект, из которого вызывается метод `show()`. Поэтому вся команда выглядит как `((ivanov+25.0)*4).show()`. Наконец, разность двух объектов `fellow-ivanov` дает нам действительное число, которое записывается в переменную `dif` и затем выводится в консольное окно.

2.4. Операторные функции

- А зачем вы мне это говорите?

- Сигнализирую.

из к/ф "Девчата"

Выше мы отмечали, что кроме операторных методов могут быть описаны и операторные функции. Название операторной функции формируется так же, как и название операторного метода: ключевое слово `operator` и непосредственно символ оператора. Аргументы операторной функции - это операнды соответствующего выражения. Поэтому у операторной функции для бинарного оператора аргументов два, а у операторной функции для унарного оператора аргумент один.

В листинге 2.4 показан пример с решением фактически той же самой задачи, что была рассмотрена выше. Но теперь вместо описания в классе операторных методов мы используем операторные функции. Несколько изменится и описание класса `BankAccount`. Кроме того, что в классе теперь нет операторных методов, мы "убрали" одну из версий конструктора (без аргументов), добавив при этом в версию конструктора с четырьмя аргументами значения для аргументов по умолчанию. Также "прекратил свое существование" вспомогательный метод `findValue()`. В предыдущем примере он вызывался в теле метода `total()`. Теперь в методе `total()` для вычисления итоговой денежной суммы запускается *оператор цикла* `while`, в котором собственно и выполняются необходимые калькуляции.



На заметку

Как уже отмечалось, вместо операторных методов на сцену выходят операторные функции. Но проблема в том, что обращаться нужно к полям объекта, и поля эти закрытые. У операторных методов проблем с доступом к закрытым полям не возникает. Но функции не описаны в классе и не являются членами объекта. Поэтому к закрытым полям у них доступа нет. Разумеется, проблему можно решить, сделав поля открытыми. Но есть и другой выход. Можно объявить функции дружественными классу `BankAccount`. Они при этом останутся внешними к классу функциями, но будут иметь доступ к закрытым членам класса. Для объявления функции дружественной к классу, в теле класса указывается прототип функции с ключевым словом `friend`. Дружественной может быть как обычная функция, так и операторная функция. В программном коде класса `BankAccount` операторные функции объявляются как дружественные.

Также стоит обратить внимание на то, что программный код функции `main()` по сравнению с предыдущим примером не изменился (равно как и результат выполнения программы).

Теперь рассмотрим программный код:

Листинг 2.4. Класс BankAccount и операторные функции

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class BankAccount{
// Закрытые члены класса:
private:
    // Поля:
    string name;
    double money;
    double rate;
    int time;
    // Вычисление итоговой суммы вклада:
double total(){
double s=money;    // Начальная сумма вклада
int t=1;           // Переменная цикла
// Оператор цикла (выполняется, пока значение
// переменной t не превышает значение поля time):
while(t<=time){
// "Начисление процентов" за год:
s*=(1+rate/100);
// Увеличение значения переменной цикла:
t++;
}
return s; // Результат метода
}
// Открытые члены класса:
public:
    // Метод для присваивания значений полям объекта:
void set(string n,double m,double r,int t){
// Значения полей:
name=n;
money=m;
rate=r;
time=t;
}
// Конструктор с четырьмя аргументами:
BankAccount(string n="Anonimus",double m=0,double r=0,int t=0)
{
// Значения полей:
set(n,m,r,t);
}
// Метод для отображения данных об объекте:
void show(){
```

```

cout<<"Данные по банковскому счету.\n";
cout<<"Вкладчик:\t"<<name<<endl;
cout<<"Сумма:\t"<<money<<endl;
cout<<"Ставка %:\t"<<rate<<endl;
cout<<"Время:\t"<<time<<endl;
cout<<"Всего:\t"<<total()<<endl;
cout<<endl;
}
// Дружественные операторные функции:
friend BankAccount operator+(BankAccount,int);
friend BankAccount operator+(BankAccount,double);
friend BankAccount operator*(BankAccount,double);
friend double operator!(BankAccount);
friend double operator-(BankAccount,BankAccount);
}; // Окончание описания класса
// Операторная функция для сложения объекта
// и целого числа:
BankAccount operator+(BankAccount obj,int t){
// Локальный объект:
BankAccount tmp;
// Значения полей локального объекта:
tmp.set(obj.name,obj.money,obj.rate,obj.time+t);
// Результат операторного метода:
return tmp;
}
// Операторная функция для сложения объекта
// и действительного числа:
BankAccount operator+(BankAccount obj,double m){
// Локальный объект:
BankAccount tmp;
// Значения полей локального объекта:
tmp.set(obj.name,obj.money+m,obj.rate,obj.time);
// Результат операторного метода:
return tmp;
}
// Операторная функция для умножения объекта
// на число:
BankAccount operator*(BankAccount obj,double x){
// Локальный объект:
BankAccount tmp;
// Значения полей локального объекта:
tmp.set(obj.name,obj.money,obj.rate+x,obj.time);
// Результат операторного метода:
return tmp;
}
// Операторная функция для унарного оператора

```

```
// "восклицательный знак" (то есть !):
double operator!(BankAccount obj){
    // Результат операторного метода:
    return obj.total();
}
// Операторная функция для вычитания объектов:
double operator-(BankAccount obj,BankAccount tmp){
    // Результат операторного метода:
    return !obj-!tmp;
}
// Главная функция программы:
int main(){
    // Создание объекта:
    BankAccount ivanov("Иванов И.И.",100,12,3);
    // Отображаем параметры объекта:
    ivanov.show();
    // Еще один объект:
    BankAccount fellow;
    // Сложение объекта и целого числа:
    fellow=ivanov+2;
    // Результат:
    fellow.show();
    // Увеличиваем сумму вклада
    // и процентную ставку:
    ((ivanov+25.0)*4).show();
    // Разность двух объектов:
    double dif=fellow-ivanov;
    // Проверяем результат:
    cout<<"Разница итоговых сумм:\t"<<dif<<endl;
    return 0;
}
```

Как отмечалось выше, результат выполнения программы такой же, как и в предыдущем случае:

Результат выполнения программы (из листинга 2.4)

Данные по банковскому счету.

Вкладчик:	Иванов И.И.
Сумма:	100
Ставка %:	12
Время:	3
Всего:	140.493

Данные по банковскому счету.

Вкладчик:	Иванов И.И.
-----------	-------------

```
Сумма:          100
Ставка %:       12
Время:         5
Всего:          176.234
```

Данные по банковскому счету.

```
Вкладчик:       Иванов И.И.
Сумма:          125
Ставка %:       16
Время:         3
Всего:          195.112
```

```
Разница итоговых сумм:  35.7414
```

В первую очередь нас интересуют операторные функции. Их код похож на код соответствующих операторных методов из предыдущего примера. Но имеется три принципиальных отличия:

- Функции описываются вне класса и не относятся ни к классу, ни к объектам, которые создаются на основе этого класса (методы описываются в классе и при создании объектов являются членами этих объектов).
- По сравнению с методами, у функций появляется по одному дополнительному аргументу. Речь идет о первом аргументе операторных функций, который является объектом класса `BankAccount` и у всех функций называется `obj` (но этот никак не требование, а просто "совпадение"). Этот первый аргумент отождествляется первым операндом в соответствующем операторном выражении. Другими словами, если при использовании операторного метода первый операнд по умолчанию отождествляется с объектом, из которого вызывается операторный метод, то в операторной функции первый операнд указывается явно в виде аргумента функции.
- Там, где в теле операторных методов обращение к полям объекта выполняется просто через указание названия поля, в коде операторных функций выполняется обращение к этому полю в полном "точечном" формате, с указанием объекта (первый аргумент `obj` операторной функции) и названия поля.



На заметку

При объявлении функций как дружественных в теле класса соответствующая инструкция начинается с ключевого слова `friend`, после которого указывается прототип операторной функции. В прототипе названия аргументов можно не указывать

(только их тип). Например: `friend BankAccount operator+(BankAccount,int)` или `friend double operator!(BankAccount).`

Также прокомментируем использование *оператора цикла* `while`. Синтаксис этого оператора простой и интуитивно понятный:

- после ключевого слова `while` в круглых скобках указывается условие, истинность которого является необходимой предпосылкой для выполнения оператора цикла;
- в фигурных скобках размещаются команды, которые выполняются за каждый цикл.

Выполняется оператор цикла так: проверяется условие, и если оно истинно, выполняются команды в фигурных скобках, после чего снова проверяется условие. И так далее. Работа оператора цикла прекращается, если при очередной проверке условия оно окажется ложным.

Подробности

Помимо оператора цикла `while`, есть еще оператор цикла `do-while`. Если шаблон использования оператора цикла `while` такой:

```
while(условие) {
    // команды
}
```

то у оператора `do-while` структура шаблона несколько иная:

```
do{
    // команды
} while(условие);
```

Оператор цикла `do-while` начинает выполняться с команд в фигурных скобках после ключевого слова `do`. Затем проверяется условие в круглых скобках после ключевого слова `while`. Если условие истинно, снова выполняются команды в фигурных скобках, и опять проверяется условие. Так продолжается до тех пор, пока условие не окажется ложным.

Принципиальная разница между операторами цикла `while` и `do-while` состоит в том, что в операторе `while` сначала проверяется условие, а затем выполняются команды, а в операторе `do-while` сначала выполняются команды, а затем проверяется условие. Поэтому в операторе `do-while` команды в теле оператора будут выполнены, по меньшей мере, один раз.

Глава 3.

НАСЛЕДОВАНИЕ И СОПУТСТВУЮЩИЕ МЕХАНИЗМЫ



*Первый раз таких одиночников вижу.
из к/ф "Девчата"*

В предыдущих главах мы кратко познакомились с основными приемами, которые позволяют описывать классы, создавать на их основе объекты и, наконец, использовать эти объекты в программе. Разумеется, таким скромным перечнем возможности C++ в области ООП не ограничиваются. В этой главе мы расширим познания как в плане синтаксиса и организации C++, так и в отношении общей концепции ООП. В первую и основную очередь нас будет интересовать такой мощный механизм ООП, как *наследование*, а также сопутствующие наследованию технологии (как, например, *переопределение методов*). Помимо этого мы познакомимся с *виртуальными методами*, а также узнаем, как описывается *конструктор производного класса*. Короче говоря, планы у нас большие. Поэтому сразу же переходим к делу.

3.1. Основы наследования

*Если мы допустим беспорядок в документации, потомки нам этого не простят.
из к/ф "Гостья из будущего"*

Наследование - механизм, который позволяет одним классам включать в себя поля и методы других классов (то есть наследовать их). На практике это сводится к тому, что новый класс создается не на пустом месте, а на основе уже существующего класса (или существующих классов).



На заметку

В отличие от таких языков программирования, как Java и C#, в языке C++ класс можно создавать на основе не только одного, но сразу нескольких классов. Такой тип наследования называется *множественным*. Множественное наследование - штука мощная и где-то даже опасная. О множественном наследовании мы еще поговорим.

Класс, на основе которого создается новый класс, называется *базовым*. Новый класс, который создается на основе базового класса, называется *производным* классом или *подклассом*. Вначале мы рассмотрим ситуацию, когда базовый класс один (как отмечалось выше, при множественном наследовании базовых классов может быть несколько).

Для того чтобы создать класс не "с нуля", а на основе уже существующего класса, в описании нового класса после имени класса нужно указать имя базового класса. Кроме имени базового слова указывается ключевое слово, определяющее *режим наследования*. Имя производного и базового классов разделяются двоеточием. Общий шаблон создания производного класса (на основе одного базового класса) такой:

```
class имя: режим наследования имя_базового_класса{
    // программный код производного класса
};
```

Ключевых слов, которые определяют режим наследования, всего три: `public` (*открытое наследование*), `private` (*закрытое наследование*) и `protected` (*защищенное наследование*). Такие же ключевые слова используются при описании уровня доступа к членам класса (в `public`-блоке описываются *открытые* члены класса, в `private`-блоке описываются *закрытые* члены класса, а в `protected`-блоке описываются *защищенные* члены класса). Режим наследования влияет на то, какие члены базового класса, в зависимости от уровня их доступа, будут наследованы в производном классе. Другими словами, наследуемость или ненаследуемость члена базового класса в производном классе зависит от того, какой у этого члена уровень доступа в базовом классе и какой задействован режим наследования. Но если быть более точным, то от указанных характеристик зависит уровень доступа унаследованного члена в производном классе. Разобраться в ситуации поможет таблица 3.1.

Таблица 3.1. Режимы наследования

Режим наследования Уровень доступа	public	private	protected
не указан	не наследуется	не наследуется	не наследуется
public	public	private	protected
private	не наследуется	не наследуется	не наследуется
protected	protected	private	protected

Для запоминания особенности разных режимов наследования разумно воспользоваться следующими правилами:

- закрытые члены базового класса (описанные в классе со спецификатором доступа `private` или без спецификатора доступа) не наследуются, независимо от режима наследования;
- если спецификатор уровня доступа не указан, то это все равно, как если бы было указано ключевое слово `private`;
- при `public`-наследовании (открытое наследование) наследуемые члены базового класса не меняют своего уровня доступа: `public`-члены остаются `public`-членами, а `protected`-члены остаются `protected`-членами;
- при `private`-наследовании (закрытое наследование) все наследуемые члены из базового класса в производном классе становятся закрытыми членами;
- при `protected`-наследовании (защищенное наследование) все наследуемые из базового класса члены в производном классе становятся защищенными членами (то есть `protected`-членами).



На заметку

Здесь и далее важно понимать, что мы имеем в виду под выражением "член класса не наследуется". Речь идет о том, что такой член класса в производном классе недоступен. То есть на него нельзя сослаться даже внутри производного класса. При этом такие члены никуда не пропадают. Технически они присутствуют в производном классе. Просто к ним нет прямого доступа (а вот непрямой доступ обычно имеется). Как все это выглядит на практике, мы увидим немного позже, когда будем разбирать примеры.

Что касается ключевого слова `protected`, которое может использоваться не только как идентификатор режима наследования, но и как спецификатор уровня доступа члена класса: соответствующие члены класса называются защищенными. Они так же, как и закрытые члены класса (описаны с идентификатором `private` или без идентификатора) доступны только в пределах класса. Но, в отличие от закрытых членов класса, защищенные члены класса наследуются.

При создании производного класса путем наследования базового класса в теле производного класса описываются его "дополнительные" члены - то есть те члены, которые есть у производного класса, и которых нет у базового класса. Все члены базового класса (кроме закрытых) автоматически включаются в производный класс и к ним можно обращаться так, как если бы они были явно описаны в производном классе. В качестве иллюстрации рассмотрим небольшой пример, в котором применяется наследование. Идея такая: сначала мы описываем класс для реализации точки на плоскости.

Точка -это такой геометрический объект. Тем, кто помнит геометрию, здесь должно быть все понятно. Кто геометрию не помнит, переживать по этому поводу не должен, потому что на понимании происходящего это никак не скажется. В любом случае для нас важно, что точка, которую мы собираемся описывать, имеет две ключевые характеристики - координаты. Другими словами, каждую точку на плоскости можно задать парой действительных чисел, которые называются координатами точки. Логично предположить, что для этой цели в классе будет два числовых поля. Также у класса будет несколько методов. Один метод будет выводить в консольное окно информацию о точке (значения ее координат). Специальный метод будет вычислять расстояние от начала координат до точки. Еще один метод будет выводить полную информацию о точке: координаты точки и расстояние от точки до начала координат. Так сказать, два в одном.



На заметку

Уместно будет напомнить, что если в декартовой системе координат (а мы имеем дело именно с такой системой координат) точка имеет координаты x и y , то расстояние от этой точки до точки начала координат (точка с обеими нулевыми координатами) дается выражением $\sqrt{x^2 + y^2}$.

Класс для реализации точки на плоскости будет называться `Point2D`. Но точка совсем необязательно должна находиться на плоскости. Точнее, мы не обязаны ограничивать пространство наших возможностей плоскостью. Никто не запрещает нам "размещать" точки в пространстве (трехмерном). От случая плоскости (двумерное пространство) принципиальное отличие в том, что теперь у точки не две, а три координаты. Для реализации точки в трехмерном пространстве мы опишем класс `Point3D`. Причем создавать класс `Point3D` будем на основе класса `Point2D` путем наследования и учтем, что по сравнению с классом `Point2D`, в классе `Point3D` появится еще одно поле (третья координата точки). Теперь рассмотрим программный код, представленный в листинге 3.1.

Листинг 3.1. Создание класса путем наследования

```
#include <iostream>
#include <cmath>
using namespace std;
// Базовый класс (реализация точки на плоскости):
class Point2D{
    // Открытые члены класса:
    public:
        // Числовые поля:
        double x,y;
```

```

        // Метод для присваивания значений полям:
void set(double x1,double y1){
// Значения полей (декартовы координаты точки):
    x=x1;
    y=y1;
}
// Метод для отображения значения полей:
void show(){
    // Значения полей (координаты точки):
cout<<"Точка 2D ("<<x<<" "<<y<<").\n";
}

// Метод для вычисления расстояния до точки:
double dist(){
    // Расстояние от начала координат до точки:
return sqrt(x*x+y*y);
}

// Метод для отображения координат точки и
// расстояния до нее от начала координат:
void getInfo(){
    // Координаты точки:
    show();
// Расстояние до точки:
cout<<"Расстояние до точки "<<dist()<<endl;
}
}; // Окончание описания базового класса
// Производный класс (реализация точки в пространстве).
// Создается путем открытого наследования базового класса:
class Point3D: public Point2D{
    // Открытые члены класса:
public:
    // Третья координата точки:
    double z;
}; // Окончание описания производного класса
// Главная функция программы:
int main(){
    // Создаем объект (точка на плоскости):
    Point2D A;
    // Координаты точки:
    A.set(3,4);
    // Выводим информацию для точки:
    A.getInfo();
    // Создаем объект (точка в пространстве):
    Point3D B;
    // Две координаты точки:
    B.set(3,4);
    // Третья координата точки:

```

```

    B.z=6;
    // Вызываем наследуемый метод:
    B.getInfo();
    // Проверяем параметры объекта (координаты точки):
    cout<<"Точка 3D ("<<B.x<<" "<<B.y<<" "<<B.z<<") .\n";
    // Расстояние до точки:
    cout<<"Расстояние до точки ";
    cout<<sqrt(B.x*B.x+B.y*B.y+B.z*B.z)<<endl;
    // Завершение программы:
    return 0;
}

```

Проанализируем представленный программный код. В базовом классе `Point2D` объявлены открытые числовые (тип `double`) поля `x` и `y`, которые мы отождествляем с координатами точки. Для присваивания значения полям в классе описан открытый метод `set()`. Метод не возвращает результат и у него два параметра. Они определяют значения полей объекта. Также мы описываем метод с красноречивым названием `show()`. У метода нет параметров, он не возвращает результат. В теле метода командой `cout<<"Точка 2D ("<<x<<" "<<y<<") .\n"` в консольное окно выводится сообщение со значениями координат точки (в окружении вспомогательного текста).

Метод `dist()` не имеет аргументов и в качестве результата возвращает числовое значение типа `double`. Это вычисленное по значениям полей `x` и `y` расстояние от точки до начала координат: для вычисления результата использована команда `return sqrt(x*x+y*y)`.

Подробности

Для вычисления квадратного корня мы использовали математическую функцию `sqrt()` из математической библиотеки `cmath` (которая, кстати, содержит много других полезных математических функций). Для подключения библиотеки в шапке программы добавлена инструкция `#include <cmath>`.

Метод `getInfo()` содержит всего две команды: сначала вызывается метод `show()`, а затем командой `cout<<"Расстояние до точки "<<dist()<<endl`, содержащей инструкцию вызова метода `dist()`, отображается информация о расстоянии от начала координат до точки.

Таким образом, класс `Point2D` состоит в общей сложности из двух полей и четырех методов, причем один метод (метод `getInfo()`) вызывает два других метода (метод `show()` и метод `dist()`).

Намного проще программный код класса `Point3D`, который является производным от класса `Point2D`. Данное обстоятельство нашло свое проявление в том, что в описании класса `Point3D` после имени класса указано

двоеточие, ключевое слово `public` (режим наследования открытый) и имя базового класса `Point2D`. Учитывая открытость полей базового класса, а также открытый режим наследования, несложно догадаться, что этой небольшой инструкции достаточно, чтобы производный класс получил от базового "в наследство" два поля (`x` и `y`) и четыре метода (`set()`, `show()`, `dist()` и `getInfo()`). Другими словами, хотя ничего из перечисленного выше явно в классе `Point3D` не описано, оно все там есть. Но не только это, а что-то еще. В классе `Point3D` явно описано открытое поле `z` типа `double`. В это поле будет записываться третья координата точки в трехмерном пространстве. Следовательно, с формальной точки зрения класс `Point3D` отличается от класса `Point2D` наличием дополнительного поля `z`.

В функции `main()` командой `Point2D A` создаем объект `A` класса `Point2D`. Командой `A.set(3, 4)` полям объекта `A` присваиваются значения, а командой `A.getInfo()` информация для точки выводится в консольное окно. Результатом выполнения всех этих инструкций являются первые две строки вывода в консольном окне:

Результат выполнения программы (из листинга 3.1)

```
Точка 2D (3;4).
Расстояние до точки 5
Точка 2D (3;4).
Расстояние до точки 5
Точка 3D (3;4;6).
Расстояние до точки 7.81025
```

Теперь создаем объект производного класса. Поможет нам команда `Point3D B`. Объект `B` является объектом класса `Point3D`, который, в свою очередь, является производным классом от класса `Point2D`.



На заметку

Хотя между классами `Point3D` и `Point2D` имеется "наследственная" связь, на наличие связи между объектами `B` и `A` это никак не сказывается. Правильнее будет сказать, что между объектами `A` и `B` нет никакой связи. Это совершенно разные объекты и у каждого из них своя собственная "судьба".

Поскольку у объекта `B` имеется метод `set()`, никто не запрещает нам вызвать этот метод. После выполнения команды `B.set(3, 4)` поля `x` и `y` объекта `B` получат соответственно значения 3 и 4. Но у объекта `B` еще есть поле `z`, и ему нужно присвоить значение. Делаем это явно, воспользовавшись командой `B.z=6`.

Если мы прибегнем к помощи команды `B.getInfo()`, то получим не очень адекватный результат (третья и четвертая строки вывода в консольном окне), хотя и вполне ожидаемый. Внешне такое впечатление, что объект `B` описывает точку в двумерном пространстве: результаты выполнения команд `A.getInfo()` и `B.getInfo()` идентичны. Дело в том, что при вызове метода `getInfo()` из объекта `B`, в соответствии с программным кодом этого метода, вызывается метод `show()`, а затем (в более сложном выражении) и метод `dist()`. Но оба эти метода описаны в классе `Point2D` и о третьей координате `z` ничего "не знают". Поэтому совершенно ожидаемо поле `z` объекта `B` при вызове метода `getInfo()` игнорируется. Тем не менее, поле `y` объекта есть и оно вполне функциональное. Чтобы убедиться в этом, воспользуемся командами `cout<<"Точка 3D ("<<B.x<<" ; "<<B.y<<" ; "<<B.z<<" ) .\n"` (проверяем параметры объекта - координаты точки) `,cout<<"Расстояние до точки "` и `cout<<sqrt(B.x*B.x+B.y*B.y+B.z*B.z)<<endl` (вычисляем расстояние до точки). Результат представлен выше (две последние строки вывода).



На заметку

Расстояние от точки с координатами x , y и z до точки начала координат в трехмерном пространстве вычисляется как корень квадратный из суммы квадратов координат точки, то есть по формуле $\sqrt{x^2 + y^2 + z^2}$. Этой формулой мы воспользовались в программном коде при вычислении расстояния до точки в трехмерном пространстве.

Таким образом, в этом примере мы познакомились с методами реализации наследования классов. С другой стороны, стала очевидной и проблема, связанная с "несоответствием" унаследованных методов новым "требованиям" в производном классе. Разрешаются такого рода неудобства по-разному, но чаще всего - с помощью *переопределения методов*.

3.2. Переопределение методов и виртуальность

*Что ей надо, я тебе потом скажу.
из к/ф "Бриллиантовая рука"*

Переопределение метода - это явное описание унаследованного метода в производном классе. То есть чтобы переопределить метод, мы в производном классе "переписываем" программный код того метода, который унаследован из базового класса. В рассматриваемом примере в классе `Point3D` из класса `Point2D` наследуются четыре метода. Два из них мы перепишем

в классе `Point3D`, тем самым переопределив их. Еще один метод будет перегружен в производном классе.



На заметку

Между *переопределением* метода и *перегрузкой* метода имеется принципиальная разница, это не одно и то же. При перегрузке метода создается несколько версий одного метода: эти версии имеют одинаковые названия, но разные прототипы. То есть они должны чем-то отличаться: списком аргументов и/или типом результата. При переопределении метода, во-первых, должно быть наследование, и, во-вторых, у описываемого в производном классе метода совпадать с методом из базового класса должно не только название, но и весь прототип (название, тип результата, количество и тип аргументов). Таким образом, о переопределении методов можем говорить только в контексте наследования. Нет наследования - нет переопределения.

Если мы перегружаем метод, то, как отмечалось ранее, на самом деле создается несколько разных методов. Каждая версия перегруженного метода - это отдельный метод. Просто называются они одинаково. При переопределении метода исходная версия метода из базового класса, унаследованная в производном классе, тоже никуда не девается. Просто она "замещается" другой версией. Если сказать иначе, то когда мы переопределяем в производном классе метод, новая явно описанная версия метода будет вызываться каждый раз в производном классе при обращении к методу. Старая версия при этом тоже существует. Более того, ее можно вызвать. Как это сделать, мы скоро узнаем.

Новый программный код, в котором производный класс `Point3D` содержит перегрузку метода `set()` и переопределение методов `show()` и `dist()`, представлен в листинге 3.2.

Листинг 3.2. Переопределение методов

```
#include <iostream>
#include <cmath>
using namespace std;
// Базовый класс (реализация точки на плоскости):
class Point2D{
    // Открытые члены класса:
    public:
        // Числовые поля:
        double x,y;
        // Метод для присваивания значений полям:
    void set(double x1,double y1){
        // Значения полей (декартовы координаты точки):
```

```

        x=x1;
        y=y1;
    }
    // Метод для отображения значения полей:
    void show() {
        // Значения полей (координаты точки):
        cout<<"2D-точка ("<<x<<" "<<y<<").\n";
    }

    // Метод для вычисления расстояния до точки:
    double dist() {
        // Расстояние от начала координат до точки:
        return sqrt(x*x+y*y);
    }

    // Метод для отображения координат точки и
    // расстояния до нее от начала координат:
    void getInfo() {
        // Координаты точки:
        show();
    }
    // Расстояние до точки:
    cout<<"2D-расстояние "<<dist()<<endl;
    // Пустая строка:
    cout<<endl;
}
}; // Окончание описания базового класса
// Производный класс (реализация точки в пространстве).
// Создается путем открытого наследования базового класса:
class Point3D: public Point2D{
    // Открытые члены класса:
public:
    // Третья координата точки:
    double z;
    // Перегрузка метода для присваивания значений полям:
    void set(double x1,double y1,double z1){
    // Вызываем версию метода из базового класса:
        Point2D::set(x1,y1);
        // Значение третьего поля:
        z=z1;
    }
    // Переопределяем метод для отображения координат:
    void show() {
        // Значения полей (координаты точки):
        cout<<"3D-точка ("<<x<<" "<<y<<" "<<z<<").\n";
    }

    // Переопределяем метод для вычисления
    // расстояния до точки:
    double dist() {

```

```

        return sqrt(x*x+y*y+z*z);
    }

    /* Начало комментария
    void getInfo(){
        show();
        cout<<"Расстояние до точки "<<dist()<<endl;
        cout<<endl;
    }
    Окончание комментария */
}; // Окончание описания производного класса
// Главная функция программы:
int main(){
    // Создаем объект (точка на плоскости):
    Point2D A;
    // Координаты точки:
    A.set(3,4);
    // Выводим информацию для точки:
    A.getInfo();
    // Создаем объект (точка в пространстве):
    Point3D B;
    // Определяем координаты точки:
    B.set(3,4,5);
    // Координаты точки:
    B.show();
    // Расстояние до точки:
    cout<<"3D-расстояние "<<B.dist()<<endl;
    // Пустая строка:
    cout<<endl;
    // Вызываем наследуемый метод:
    B.getInfo();
    // Завершение программы:
    return 0;
}

```

В классе `Point2D`, по сравнению с предыдущей версией, произошли минимальные изменения, в основном "декоративного" характера: в методе `show()` изменился выводимый в консоль вспомогательный текст, а в методе `getInfo()`, кроме аналогичного изменения выводимого текста, еще отображается пустая строка (после того, как в консоль выведена "полезная" информация). Все это сделано, чтобы легче было различать результаты выполнения методов объектов разных классов.

Класс `Point3D` существенно расширился. Теперь, кроме поля `z`, в классе явно описан метод `set()`. Но здесь речь о переопределении метода `set()` не идет. Дело в том, что в классе `Point2D` метод `set()` описан с двумя

аргументами, а в классе `Point3D` метод `set()` описывается с тремя аргументами. Поэтому в данном случае имеем дело скорее с *перегрузкой* метода. Причем перегрузка (назовем этот так) достаточно "специфическая": мы как бы перегружаем метод, унаследованный из базового класса.



На заметку

Ситуация не такая простая, как может показаться на первый взгляд. Если подойти к вопросу формально, то класс `Point3D` получает две версии метода `set()`: одна версия (с двумя аргументами) наследуется из класса `Point2D`, а другая версия (с тремя аргументами) явно описывается в классе `Point3D`. Но если попытаться вызвать из объекта класса `Point3D` версию метода `set()` с двумя аргументами (унаследованная версия), появится сообщение об ошибке. Такой неприятный сюрприз связан с правилом сокрытия имен, которое действует в C++. Если кратко, то при объявлении члена (это может быть поле или метод) в производном классе соответствующее имя члена "перекрывает" такие же имена в базовом классе. Таким образом, если мы описали хоть какую-то версию метода `set()`, то все другие версии этого метода, до этого свободно наследуемые из базового класса в производном, будут "спрятаны". Последнее означает, что при вызове метода `set()` из объекта производного класса поиск подходящей версии метода будет осуществляться только в пределах производного класса.

Ситуация неприятная, но не критичная. Обратиться к "базовой" версии метода можно, воспользовавшись оператором расширения контекста `::` (два двоеточия). Перед оператором указывается имя базового класса, а после оператора - вызываемый метод. Также можно в теле производного класса добавить инструкцию `using`, после которой указать имя метода в формате `базовый_класс::метод`.

В теле метода `set()`, описанного в классе `Point3D` с тремя аргументами (`x1`, `y1` и `z1`), командой `Point2D::set(x1, y1)` вызываем версию метода с двумя аргументами, описанную в классе `Point2D`. В этой команде мы явно указали, что вызывается версия метода из класса `Point2D`: перед инструкцией вызова метода `set(x1, y1)`, через оператор расширения контекста `::`, указано имя класса `Point2D`, в котором описан метод (точнее, вызываемая версия метода). Затем командой `z=z1` присваивается значение третьему полю.



На заметку

Если мы захотим создать объект класса `Point3D` (пусть это будет объект `B`), а затем нам понадобится вызвать из этого объекта метод `set()`, но только ту его версию, у которой два аргумента и которая описана в классе `Point2D`, то команда вызова будет иметь вид `B.Point2D::set(аргумент1, аргумент2)`.

Чтобы каждый раз при вызове "базовой" версии метода `set()` не приходилось указывать имя базового класса, в теле производного класса `Point3D` можно добавить инструкцию `using Point2D::set` (помещается в `public`-блоке, заканчивается точкой с запятой).

Метод `show()` в классе `Point3D` переопределяем таким образом, что теперь при вызове метода отображаются все три координаты точки (значения трех полей объекта). Достигается это командой `cout<<"3D-расстояние ("<<x<<" "<<y<<" "<<z<<").\n"`. Отображаемый вместе с координатами точки вспомогательный текст позволяет идентифицировать версию метода.

Метод `dist()` переопределяем для вычисления расстояния от точки в трехмерном пространстве до точки начала координат. Результат метода вычисляется командой `return sqrt(x*x+y*y+z*z)`. Здесь мы воспользовались тем, что расстояние до точки - это корень квадратный из суммы квадратов ее декартовых координат.



На заметку

В классе `Point3D` есть закомментированный блок с программным кодом метода `getInfo()`. Если для этого блока комментирование отменить (достаточно удалить строки с текстом `Начало комментария` и `Конец комментария`), то получится, что в классе `Point3D` мы переопределили метод `getInfo()`. Правда, переопределяем мы его практически так же, как этот метод определялся в базовом классе `Point2D`. Но пока что комментарий убирать не нужно.

Подробности

Бывают комментарии однострочные: все, что находится справа от двойной косой черты `//`, является комментарием. Комментарии бывают многострочными. Многострочный комментарий начинается символами `/*`, и заканчивается символами `*/`.

В главной функции программы командой `Point2D A` создаем объект `A` класса `Point2D`. Значения полям объекта присваиваем командой `A.set(3,4)`. Информацию об объекте выводим в консольное окно с помощью команды `A.getInfo()` (первые две строки вывода в консольном окне вывода):

Результат выполнения программы (из листинга 3.2)

2D-точка (3;4).

2D-расстояние 5

3D-точка (3;4;5).

3D-расстояние 7.07107

2D-точка (3;4).

2D-расстояние 5

Пока что ничего неожиданного не происходит. Далее создаем объект производного класса. Воспользуемся командой `Point3D B`. Поля объекта `B` определяем с помощью метода `set()`, передав ему три аргумента, как это сделано в команде `B.set(3, 4, 5)`. После этого командой `B.show()` выводим в консольное окно информацию о координатах точки, а расстояние до точки узнаем, если воспользуемся командой `cout<<"3D-расстояние "<<B.dist()<<endl`. Результатом выполнения этих двух команд являются третья и четвертая (непустые) строки в консольном окне вывода.

Здесь тоже нет ничего неожиданного. Методы `set()`, `show()` и `dist()`, вызываемые из объекта класса `Point3D`, работают вполне корректно и ожидаемо. Но если мы вызовем команду `B.getInfo()`, которая по идее должна была бы продублировать результат вызова методов `show()` и `dist()`, выполненные на предыдущем этапе, получим довольно неожиданный результат: мы вызывали метод `getInfo()` из объекта `B`, а результат такой, как если бы метод вызывался из объекта класса `Point2D`. Несмотря на то, что метод `getInfo()` не описывается явно в классе `Point3D`, он в этом классе наследуется и содержит, в свою очередь, команды вызова методов `show()` и `dist()`, которые мы в классе `Point3D` переопределили.

Корректность работы этих переопределенных методов нами проверена на предыдущем этапе. Поэтому объяснение к происходящему может быть таким: при вызове унаследованной версии метода `getInfo()` в этом методе вызываются не переопределенные, а исходные версии методов `show()` и `dist()`, которые описаны в базовом классе `Point2D`. Подтверждение сказанному легко найти, если убрать комментарий в последнем блоке программного кода (см. листинг 3.2.), добавив тем самым в класс `Point3D` переопределение метода `Point3D`.

Важно то, что при переопределении в классе `Point3D` метод `getInfo()` практически "дословно" дублирует код метода из класса `Point2D`. Тем не менее, результат разительный. Ниже показан результат выполнения программы, в которой в классе `Point3D` переопределен метод `getInfo()`:

Результат выполнения программы (из листинга 3.2)

2D-точка (3;4).

2D-расстояние 5

3D-точка (3;4;5).

3D-расстояние 7.07107

3D-точка (3;4;5).

Расстояние до точки 7.07107

Переопределенный метод `getInfo()` вызывает переопределенные версии методов `show()` и `dist()`. И хотя проблема решена, общий подход для ее решения не может быть приемлемым: нам пришлось дублировать (почти) в производном классе программный код наследуемого метода, что не может радовать.

Выход найдем, если сделаем переопределяемые методы *виртуальными*. Технически все сводится к тому, что методы в базовом классе, которые мы собираемся переопределять в производном классе, описываются с ключевым словом `virtual`. Новая версия программного кода с виртуальными переопределяемыми методами представлена в листинге 3.3 (наиболее важные изменения выделены жирным шрифтом).

Листинг 3.3. Виртуальные методы

```
#include <iostream>
#include <cmath>
using namespace std;
// Базовый класс (реализация точки на плоскости):
class Point2D{
    // Открытые члены класса:
    public:
        // Числовые поля:
        double x,y;
        // Виртуальный метод для присваивания значений полям:
virtual void set(double x1,double y1){
// Значения полей (декартовы координаты точки):
        x=x1;
        y=y1;
    }
    // Виртуальный метод для отображения значения полей:
virtual void show(){
        // Значения полей (координаты точки):
        cout<<"2D-точка ("<<x<<" "<<y<<").\n";
    }
    // Виртуальный метод для вычисления
    // расстояния до точки:
virtual double dist(){
        // Расстояние от начала координат до точки:
        return sqrt(x*x+y*y);
    }
    // Метод для отображения координат точки и
    // расстояния до нее от начала координат:
    void getInfo(){
        // Координаты точки:
        show();
    }
}
```



```
// Расстояние до точки:
cout<<"Расстояние до точки "<<dist()<<endl;
// Пустая строка:
cout<<endl;
}
}; // Окончание описания базового класса
// Производный класс (реализация точки в пространстве).
// Создается путем открытого наследования базового класса:
class Point3D: public Point2D{
    // Открытые члены класса:
public:
    // Используем скрытое имя:
using Point2D::set;
    // Третья координата точки:
    double z;
    // Перегрузка метода для присваивания значений полям:
    void set(double x1,double y1,double z1){
        // Вызываем версию метода из базового класса:
set(x1,y1);
        // Значение третьего поля:
        z=z1;
    }
    // Переопределяем метод для отображения координат:
    void show(){
        // Значения полей (координаты точки):
        cout<<"3D-точка ("<<x<<";"<<y<<";"<<z<<").\n";
    }
    // Переопределяем метод для вычисления
    // расстояния до точки:
    double dist(){
        return sqrt(x*x+y*y+z*z);
    }
}; // Окончание описания производного класса
// Главная функция программы:
int main(){
    // Создаем объект (точка на плоскости):
    Point2D A;
    // Координаты точки:
    A.set(3,4);
    // Выводим информацию для точки:
    A.getInfo();
    // Создаем объект (точка в пространстве):
    Point3D B;
    // Определяем координаты точки:
    B.set(3,4,5);
    // Выводим информацию для точки:
```

```

B.getInfo();
    // Завершение программы:
    return 0;
}

```

Результат выполнения данного программного кода представлен ниже:

Результат выполнения программы (из листинга 3.3)

```

2D-точка (3;4) .
Расстояние до точки 5

3D-точка (3;4;5) .
Расстояние до точки 7.07107

```

Мы внесли следующие изменения в программный код:

- в базовом классе `Point2D` добавили ключевое слово `virtual` в описание методов `set()`, `show()` и `dist()`;
- в описании производного класса `Point3D` добавили инструкцию `using Point2D::set`;
- в классе `Point3D` в описании метода `set()` команду `Point2D::set(x1,y1)` заменили на команду `set(x1,y1)`;
- удалили блок (в комментарии) с описанием метода `getInfo()` в классе `Point3D`;
- в главной функции `main()` удалили команды `B.show()`, `cout<<"3D-расстояние "<<B.dist()<<endl` и `cout<<endl`.

Как несложно заметить, теперь метод `getInfo()`, когда вызывается из объекта класса `Point3D`, вполне сносно справляется со своей задачей. Таким образом, использование виртуальных переопределяемых методов позволяет вызывать "правильные" версии методов из производного класса.

На заметку



Виртуальность метода - характеристика наследуемая. То есть если метод объявлен как виртуальный, то сколько бы раз он ни наследовался (здесь имеется в виду *многоуровневое наследование*, когда производный класс сам является базовым для другого класса), виртуальным он и останется. Поэтому если метод, описанный в базовом классе, впоследствии предполагается переопределять, то лучше сразу его сделать виртуальным.

Еще одна особенность рассмотренного выше примера - мы не использова-

ли в нем конструкторов, хотя это было бы вполне логичным. Использование конструкторов при наследовании классов имеет некоторые особенности. Их мы и обсудим.

3.3. Конструктор производного класса

Ничего особенного. Обыкновенная контрабанда.

из к/ф "Бриллиантовая рука"

Описание конструктора для базового класса особых проблем не представляет - конструктор в этом случае описывается, как и для обычного класса (который не является базовым). Образно выражаясь, вся тяжесть ноши по реализации гибкой и непротиворечивой схемы реализации конструкторов при наследовании ложится на производный класс. Здесь важно помнить главное правило, которое состоит в том, что при создании объекта производного класса сначала вызывается конструктор базового класса.

Если у конструктора базового класса аргументов нет, то вроде нет и особых проблем. Но сразу возникает вопрос: что делать, если конструктору базового класса нужно передавать аргументы? В C++ предусмотрен механизм, который позволяет решить эту проблему. В частности, если мы описываем конструктор производного класса, то после имени конструктора (но перед фигурными скобками, определяющими тело конструктора) через двоеточие указывается инструкция вызова конструктора базового класса: имя базового класса, а в круглых скобках аргументы конструктора базового класса.

На заметку



Новые стандарты языка C++ позволяют вызывать в конструкторе класса другую версию конструктора того же самого класса. Подробнее эта тема обсуждается в последней главе книги.

В случае если производный класс создается на основе одного базового класса, шаблон описания конструктора производного класса выглядит следующим образом:

```
имя_класса(аргументы) : имя_базового_класса(аргументы) {
    // программный код конструктора производного класса
}
```

Как иллюстрацию к использованию конструкторов при наследовании рассмотрим пример с классами `Point2D` и `Point3D`, но только теперь в обеих

классах будет по несколько конструкторов. Соответствующий программный код представлен в листинге 3.4.

Листинг 3.4. Конструкторы и наследование

```
#include <iostream>
#include <cmath>
using namespace std;
// Базовый класс (реализация точки на плоскости):
class Point2D{
    // Открытые члены класса:
    public:
        // Числовые поля:
        double x,y;
        // Конструктор (с двумя аргументами):
        Point2D(double x1,double y1){
            // Сообщение о создании объекта:
            cout<<"2D-объект создан (2 аргумента): ";
            // Присваиваем значения полям:
                x=x1;
                y=y1;
            // Выводим информацию об объекте:
            show();
        }
        // Конструктор (с одним аргументом):
        Point2D(double x1){
            // Сообщение о создании объекта:
            cout<<"2D-объект создан (1 аргумент): ";
            // Присваиваем значения полям:
                x=x1;
                y=x1;
            // Выводим информацию об объекте:
            show();
        }
        // Конструктор (без аргументов):
        Point2D(){
            // Сообщение о создании объекта:
            cout<<"2D-объект создан (без аргументов): ";
            // Присваиваем значения полям:
                x=0;
                y=0;
            // Выводим информацию об объекте:
            show();
        }
        // Виртуальный метод для отображения значения полей:
```

```

        virtual void show(){
            // Значения полей (координаты точки):
            cout<<"2D-точка ("<<x<<" "<<y<<").\n";
        }
}; // Окончание описания базового класса
// Производный класс (реализация точки в пространстве).
// Создается путем открытого наследования базового класса:
class Point3D: public Point2D{
    // Открытые члены класса:
public:
    // Третья координата точки:
    double z;
    // Конструктор (с тремя аргументами):
    Point3D(double x1,double y1,double z1):Point2D(x1,y1){
        // Сообщение о создании объекта:
        cout<<"3D-объект создан (3 аргумента): ";
        // Присваиваем значение полю:
        z=z1;
        // Выводим информацию об объекте:
        show();
    }
    // Конструктор (с одним аргументом):
    Point3D(double x1):Point2D(x1){
        // Сообщение о создании объекта:
        cout<<"3D-объект создан (1 аргумент): ";
        // Присваиваем значение полю:
        z=x1;
        // Выводим информацию об объекте:
        show();
    }
    // Конструктор (без аргументов):
    Point3D():Point2D(){
        // Сообщение о создании объекта:
        cout<<"3D-объект создан (безаргументов): ";
        // Присваиваем значение полю:
        z=0;
        // Выводим информацию об объекте:
        show();
    }
    // Конструктор (аргумент - объект):
    Point3D(Point2D obj){
        // Сообщение о создании объекта:
        cout<<"3D-объект создан (аргумент - объект): ";
        // Присваиваем значения полям:
        x=obj.x;
        y=obj.y;
    }
};

```

```

        z=0;
        // Выводим информацию об объекте:
        show();
    }
    // Переопределяем метод для отображения координат:
    void show() {
        // Значения полей (координаты точки):
        cout<<"3D-точка ("<<x<<" "<<y<<" "<<z<<").\n";
        cout<<"-----"<<endl;
    }
}; // Окончание описания производного класса
// Главная функция программы:
int main() {
    // Создаем объекты (точки на плоскости):
    Point2D A;
    Point2D B(1);
    Point2D C(2,3);
    // Разделитель:
    cout<<"*****"<<endl;
    // Создаем объекты (точки в пространстве):
    Point3D D;
    Point3D E(4);
    Point3D F(5,6,7);
    Point3DG(C);
    // Завершение программы:
    return 0;
}

```

Поскольку нас интересуют конструкторы, то чтобы не загромождать лишними конструкциями программный код, мы его немного упростили: у классов остались только поля и метод `show()` для отображения значений полей. Ну и еще, конечно, конструкторы.

В базовом классе `Point2D` (кроме полей `x` и `y`, а также метода `show()`) описаны три варианта конструктора класса: с двумя аргументами, с одним аргументом и без аргументов. Качественно программный код в этих конструкторах не отличаются: сначала выводится сообщение о том, что создан новый объект (причем имеется намек на то, что объект - в двумерном пространстве, а также указано количество аргументов конструктора), присваиваются значения полям, после чего информация о созданном объекте выводится в консольное окно. Если конструктору переданы два аргумента, то они определяют значения полей объекта. Если у конструктора один аргумент, то оба поля объекта получают одинаковые значения. Наконец, если при создании объекта конструктору не переданы аргументы, оба поля у объекта будут с нулевыми значениями.

В производном классе `Point3D`, который наследует класс `Point2D`, как и ранее объявляется дополнительное числовое поле `z` и переопределяется метод `show()`. Также мы описываем в производном классе четыре варианта конструкторов: с тремя числовыми аргументами, с одним числовым аргументом, без аргументов и с одним аргументом - объектом класса `Point2D`. Например, при описании конструктора класса `Point3D` с тремя аргументами, как и в обычном (не производном) классе, мы описываем после имени конструктора три параметра (в данном случае это переменные `x1`, `y1` и `z1` - все типа `double`). Далее следует двоеточие и инструкция `Point2D(x1, y1)`. Эта инструкция означает, что при создании объект производного класса сначала вызывается конструктор базового класса. Первые два из трех аргументов конструктора класса `Point3D` передаются аргументами конструктору класса `Point2D`. Другими словами, когда мы передаем конструктору класса `Point3D` три аргумента, то первый и второй аргументы будут переданы конструктору класса `Point2D`, а третий будет непосредственно использован в теле конструктора `Point3D` (для присваивания значения полю `z`).



На заметку

То, что мы использовали первые два аргумента для передачи конструктору базового класса, никак не означает, что мы не можем их использовать и в теле конструктора производного класса. Просто в данном случае в этом нет необходимости.

Во всем остальном конструктор производного класса прост: третьему полю объекта на основе значения аргумента конструктора присваивается значение, после чего с помощью переопределенного метода `show()` информация отображается в консольном окне.



На заметку

В теле конструктора класса `Point3D` в явном виде присваивается значение только третьему полю `z`. Значения полям `x` и `y` присваиваются при вызове конструктора класса `Point2D`.

В конструкторе производного класса выводится сообщение о том, что создается объект класса `Point3D`. Присваивание значения третьему полю выполняется только после того, как выводится это сообщение. Конструктор базового класса вызывается еще раньше. И в этом конструкторе первой является команда вывода сообщения о том, что создается объект класса `Point2D`. Таким образом, при создании объекта производного класса (на основе конструктора с тремя аргументами) общая последовательность действий следующая:

- выводится сообщение о создании объекта класса `Point2D`;
- присваиваются значения полям `x` и `y`;
- выводится информация об объекте класса `Point2D` (используется только два поля объекта);
- выводится сообщение о создании объекта класса `Point3D`;
- присваивается значение полю `z`;
- выводится информация об объекте класса `Point3D` (используются все три поля объекта).

Примерно такая же схема реализуется и в прочих конструкторах класса `Point3D`. Например, в случае конструктора с одним аргументом этот аргумент передается конструктору базового класса. Затем выводится сообщение о создании объекта и присваивается значение третьему полю (такое же, как и первым двум).

В конструкторе производного класса без аргументов вызывается конструктор базового класса без аргументов. В результате поля `x` и `y` получают нулевые значения. Нулевое значение полю `z` присваиваем непосредственно в теле конструктора производного класса.

Несколько выпадает из этой схемы конструктор создания объекта класса `Point3D` на основе объекта класса `Point2D`. Если абстрагироваться от программирования, то ситуация выглядит вполне логично: чтобы из точки на плоскости создать точку в пространстве достаточно добавить третью координату -например, с нулевым значением. Именно такой подход мы и реализуем в конструкторе. Он имеет некоторые особенности:

- аргументом конструктора класса `Point3D` указан объект `obj` класса `Point2D`;
- конструктор базового класса явно не вызывается - в этом случае автоматически вызывается конструктор базового класса без аргументов;
- после того, как выводится сообщение о создании объекта класса `Point3D`, в явном виде присваиваются значения полям объекта: полю `x` создаваемого объекта присваивается значение поля `x` объекта-аргумента `obj` конструктора (команда `x=obj.x`), поле `y` получает значение `obj.y`, а полю `z` присваивается значение `0`;
- информация о созданном объекте выводится в консоль с помощью метода `show()`.



На заметку

Уточняем: в данном случае, как отмечалось выше, мы в явном виде конструктор базового класса не вызываем и это означает, что на самом деле вызывается конструктор базового класса без аргументов. При вызове этого конструктора полям `x` и `y` объекта присваиваются нулевые значения. Затем в теле конструктора производного класса эти поля получают новые значения.

В функции `main()` размещена серия команд, которыми создаются объекты класса `Point2D` (используются конструкторы без аргументов, с одним аргументом и двумя аргументами). Результат выполнения этих команд представлен первыми тремя строками в консольном окне вывода:

Результат выполнения программы (из листинга 3.4)

```
2D-объект создан (без аргументов): 2D-точка (0;0).
2D-объект создан (1 аргумент): 2D-точка (1;1).
2D-объект создан (2 аргумента): 2D-точка (2;3).
*****
2D-объект создан (без аргументов): 2D-точка (0;0).
3D-объект создан (без аргументов): 3D-точка (0;0;0).
----->
2D-объект создан (1 аргумент): 2D-точка (4;4).
3D-объект создан (1 аргумент): 3D-точка (4;4;4).
----->
2D-объект создан (2 аргумента): 2D-точка (5;6).
3D-объект создан (3 аргумента): 3D-точка (5;6;7).
----->
2D-объект создан (без аргументов): 2D-точка (0;0).
3D-объект создан (аргумент - объект): 3D-точка (2;3;0).
----->
```

После этого выводится импровизированный разделитель (ряд из звездочек) и затем создается четыре объекта класса `Point3D`. Здесь мы последовательно используем конструкторы класса `Point3D` с тремя числовыми аргументами, одним числовым аргументом, без аргументов и одним аргументом - объектом класса `Point2D`. Создание каждого объекта сопровождается (в силу описанных причин) выводом двух строк текста в консольное окно. Читатель может сравнить свои ожидания от выполнения программного кода с реальными результатами выполнения программы, представленными выше.

Глава 4.

НАСЛЕДОВАНИЕ: СЕКРЕТЫ И ОСОБЕННОСТИ



Он начинает новую жизнь, дайте ему возможность вспомнить все лучшее.

из к/ф "Покровские ворота"

Наследование можно обсуждать очень долго. Тема эта многогранная, и в ней есть множество разных аспектов и "тонких моментов", которые критически важны для понимания базовых принципов реализации механизма наследования. Некоторые вопросы мы уже успели рассмотреть. В основном речь шла о приемах и принципах, касающихся реализации наследования в самых простых формах. Теперь настало время несколько расширить наши горизонты.

4.1. Множественное наследование

Я вся такая внезапная, такая противоречивая вся.

из к/ф "Покровские ворота"

Ранее мы создали производный класс на основе одного базового класса. Но на самом деле базовых классов может быть несколько.



На заметку

В отличие от таких языков, как Java и C#, в языке C++ производный класс можно создавать на основе нескольких базовых классов.

При этом производный класс через механизм наследования получает "во владение" поля и методы всех своих базовых классов. В данном случае говорят о *множественном наследовании*. Множественное наследование - мощный и удобный механизм создания классов. Вместе с тем он и достаточно опасный, поскольку чреват созданием некорректного программного кода. Но как бы там ни было, мы познакомимся с множественным наследованием и рассмотрим некоторые примеры по этому поводу.

Если производный класс создается на основе нескольких базовых классов, то при описании производного класса после его имени перечисляются (через запятую) все базовые классы, на основе которых он создается. Для каждого базового класса отдельно указывается механизм наследова-

ния. Например, если класс `Charlie` создается на основе базовых классов `Alpha` и `Bravo`, причем для класса `Alpha` используется открытый механизм наследования, а для класса `Bravo` защищенный механизм, то объявление класса `Charlie` выполняется по такому шаблону:

```
class Charlie: public Alpha, protected Bravo{
// Программный код класса Charlie
};
```

В этом случае класс `Charlie` получит, в соответствии с режимом наследования, поля и методы классов `Alpha` и `Bravo`. Из класса `Alpha` будут "получены" все незакрытые члены, и уровень доступа для них в классе `Charlie` будет такой же, какой был в классе `Alpha`. Из класса `Bravo` также будут получены незакрытые поля и методы, но уровень доступа к ним в классе `Charlie` будет защищенный, вне зависимости от уровня доступа в классе `Bravo` (главное, чтобы они наследовались). Для большей конкретики рассмотрим пример из листинга 4.1. В нем реализована очень простая схема множественного наследования, в которой производный класс создается на основе двух базовых классов.

Листинг 4.1. Множественное наследование

```
#include<iostream>
#include <string>
using namespace std;
// Первый базовый класс:
class Alpha{
private:
    // Текстовое поле:
    string name;
public:
    // Конструктор:
    Alpha(string txt){
        name=txt; // Значение текстового поля
    }
    // Метод:
    void hi(){
        cout<<"Объект "<<name<<" класса Alpha!\n";
    }
}; // Окончание описания первого базового класса
// Второй базовый класс:
class Bravo{
private:
    // Текстовое поле:
    string name;
```

```

public:
    // Конструктор:
    Bravo(string txt){
        name=txt; // Значение текстового поля
    }
    // Метод:
    void hello(){
        cout<<"Объект "<<name<<" классаBravo!\n";
    }
}; // Окончание описания второго базового класса
// Производный класс:
class Charlie: public Alpha, protected Bravo{
public:
    // Конструктор производного класса:
    Charlie(string x,string y):Alpha(x), Bravo(y){
        cout<<"Создан объект класса Charlie!\n";
    }
    // Метод:
    void show(){
        // Вызов метода из первого базового класса:
        hi();
        // Вызов метода из второго базового класса:
        hello();
    }
}; // Окончание описания второго базового класса
// Главная функция программы:
int main(){
    // Создание объекта производного класса:
    Charlie obj("Альфа","Браво");
    // Вызов методов из объекта производного класса:
    obj.show();
    obj.hi();
    return 0; // Завершение выполнения программы
}

```

Результат выполнения программного кода такой:

Результат выполнения программы (из листинга 4.1)

```

Создан объект класса Charlie!
Объект Альфа класса Alpha!
Объект Браво класса Bravo!
Объект Альфа класса Alpha!

```

У класса Alpha имеется закрытое текстовое поле name и открытый метод hi(), которым выводится приветствие, содержащее значение поля name.



На заметку

Текстовое поле в данном случае реализуется в виде объекта класса `string`. Для использования класса `string` в шапке программы командой `#include <string>` подключается заголовок `<string>`.

Конструктору класса `Alpha` передается один текстовый аргумент, который определяет значение поля `name` создаваемого объекта.

Аналогичную структуру имеет класс `Bravo`. Так же, как и у класса `Alpha`, у класса `Bravo` есть закрытое поле `name`, значение которому присваивается при вызове конструктора. Метод `hello()` выводит сообщение, при формировании которого используется значение поля `name` объекта класса.

При наследовании базовых классов в классе `Charlie` последний "получает" методы `hi()` и `hello()`. Причем первый метод наследуется как открытый, а второй - как защищенный. Другими словами, метод `hi()` можно вызывать как в классе `Charlie`, так и вне класса, а метод `hello()` доступен только в классе. Поэтому, например, в методе `show()`, который описан в классе `Charlie`, вызов обоих методов вполне законен, а вот в главной функции `main()` из объекта `obj` класса `Charlie` метод `hello()` вызвать не получится.

Нетривиальным образом обстоят дела с полями `name` (одно описано в классе `Alpha`, а другое описано в классе `Bravo`). Во-первых, эти поля в своих классах объявлены как закрытые, поэтому в производном классе как бы не наследуются. Но в методах `hi()` и `hello()` выполняется обращение к полю `name` соответствующего класса. Более того, в конструкторах классов `Alpha` и `Bravo` присваиваются значения полям `name`. В конструкторе класса `Charlie` вызываются конструкторы классов `Alpha` и `Bravo`: в описании конструктора класса `Charlie` после двоеточия указываются, через запятую, названия классов `Alpha` и `Bravo` (что означает вызов конструкторов соответствующих классов), а в круглых скобках конструкторам передаются аргументы. Другими словами, и в конструкторе класса `Charlie`, и в методах `hi()` и `hello()` в теле класса неявно используются поля `name`, описанные в классах `Alpha` и `Bravo`. Здесь для нас важны два обстоятельства:

- Мы имеем дело с полями, которые не наследуются в производном классе, но при этом все же в производном классе используются.
- Поля имеют одинаковые названия, что скрывает за собой потенциальную проблему идентификации полей с совпадающими названиями.

Что касается "ненаследуемых", но "используемых" полей, то все становится на свои места, если под термином "не наследуется" понимать невозможность явного (то есть непосредственного обращения через указание имени) использования полей и методов. В отношении ненаследуемых в классе `Charlie` полей данное замечание означает, что поля технически существуют, но в теле класса `Charlie` к этим полям обратиться напрямую не получится: использование идентификатора `name` в теле класса приведет к ошибке. Вместе с тем, к полям могут обращаться методы, которые описаны в базовых классах и которые содержат в себе код обращения к соответствующим полям. Важно и то, что полей именно два, а не одно: то есть совпадение названий полей не означает, что они каким-либо образом "сливаются", "объединяются" или "перекрываются". Доказательство такое: при вызове из объекта класса `Charlie` метода `show()` в нем, через методы `hi()` и `hello()`, выполняется обращение к полям `name`, причем каждый из методов `hi()` и `hello()` получает свое значение соответствующего поля. Вывод простой - поля разные, хотя и имеют совпадающие названия.

Здесь мы подходим к важному моменту: что было бы, если поля `name` из классов `Alpha` и `Bravo` наследовались в классе `Charlie`? Ведь совершенно очевидно, что при множественном наследовании вполне реальна ситуация, когда в производном классе из нескольких базовых классов наследуются поля или методы с одинаковыми названиями. Более того, не исключается, что член с таким же именем определен в производном классе. То есть ситуация может быть очень запутанной. Общий рецепт ее разрешения состоит в том, что при обращении к такому полю или методу следует явно указывать класс, из которого наследован соответствующий член. Если класс, из которого наследуется член, не указать, то автоматически будет подразумеваться член, описанный в производном классе. Как иллюстрацию рассмотрим небольшой пример, представленный в листинге 4.2.

Листинг 4.2. Совпадающие названия наследуемых членов

```
#include <iostream>
#include <string>
using namespace std;
// Первый базовый класс:
class Alpha{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    Alpha(string txt){
        name=txt; // Значение текстового поля
```



```

    }
    // Метод для отображения значения текстового поля:
void getName(){
cout<<"Поле name из класса Alpha: "<<name<<endl;
}
}; // Окончание описания первого базового класса
// Второй базовый класс:
class Bravo{
public:
    // Текстовое поле:
string name;
    // Конструктор:
Bravo(string txt){
name=txt; // Значение текстового поля
}
    // Метод для отображения значения текстового поля:
void getName(){
cout<<"Поле name из класса Bravo: "<<name<<endl;
}
}; // Окончание описания второго базового класса
// Производный класс:
class Charlie: public Alpha, public Bravo{
public:
    // Текстовое поле:
string name;
    // Конструктор производного класса:
Charlie(string x,string y,string z):Alpha(x), Bravo(y){
name=z; // Значение текстового поля
}
    // Метод для отображения значения текстовых полей:
void getName(){
cout<<"Метод getName() класса Charlie:\n";
// Поле из первого базового класса:
Alpha::getName();
    // Поле из второго базового класса:
Bravo::getName();
    // Поле из производного класса:
cout<<"Значение поля name: "<<name<<endl;
cout<<"Выполнение метода завершено.\n";
}
}; // Окончание описания производного класса
// Главная функция программы:
int main(){
    // Создание объекта производного класса:
Charlie obj("Альфа","Браво","Чарли");
// Вызов метода из производного класса:

```

```

obj.getName();
// Вызов метода из первого базового класса:
obj.Alpha::getName();
// Вызов метода из второго базового класса:
obj.Bravo::getName();
cout<<"Проверяем значения полей.\n";
// Обращение к полю из первого базового класса:
cout<<"Поле Alpha::name: "<<obj.Alpha::name<<endl;
// Обращение к полю из второго базового класса:
cout<<"Поле Bravo::name: "<<obj.Bravo::name<<endl;
// Обращение к полю из производного класса:
cout<<"Значение поля name: "<<obj.name<<endl;
return 0;
}

```

Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 4.2)

```

Метод getName() класса Charlie:
Поле name из класса Alpha: Альфа
Поле name из класса Bravo: Браво
Значение поля name: Чарли
Выполнение метода завершено.
Поле name из класса Alpha: Альфа
Поле name из класса Bravo: Браво
Проверяем значения полей.
Поле Alpha::name: Альфа
Поле Bravo::name: Браво
Значение поля name: Чарли

```

В данном случае речь идет о трех классах: класс `Charlie` создается путем наследования классов `Alpha` и `Bravo`. В обоих случаях механизм наследования открытый (`public`-наследование). В каждом из классов `Alpha` и `Bravo` объявлено открытое текстовое (объект класса `string`) поле `name` и метод `getName()`. Вызов метода `getName()` из объекта класса `Alpha` или `Bravo` приводит к тому, что отображается информация о классе, в котором объявлен метод, и значении поля `name` объекта, из которого метод вызывается. Таким образом, в классе `Charlie` наследуется два поля `name` и два метода `getName()`. Но это еще не все. В самом классе `Charlie` объявляется поле `name` и переопределяется метод `getName()`. Таким образом, в классе `Charlie` три поля с названием `name` и три версии метода `getName()` (точнее, три метода с названием `getName()`). Чтобы различать эти поля и методы, используем явную идентификацию класса, из которого насле-

дованы поля и методы. В частности, если в теле класса `Charlie` мы при обращении к полю используем идентификатор `name`, а при обращении к методу используем инструкцию `getName()`, то речь идет соответственно о поле и методе, описанных непосредственно в классе `Charlie`. Инструкции `Alpha::name` и `Alpha::getName()` представляют собой обращение к полю `name` и методу `getName()`, определенным в классе `Alpha`. Аналогично, инструкции `Bravo::name` и `Bravo::getName()` позволяют получить доступ к полю `name` и методу `getName()`, определенным в классе `Bravo`. Это же правило остается справедливым, если мы обращаемся к полям и методам не в классе `Charlie`, а через объект этого класса. Примеры таких команд есть в программном коде.

4.2. Виртуальные базовые классы

*Делом надо заниматься серьезно или не заниматься им вообще.
из к/ф "Служебный роман"*

При множественном наследовании теоретически может сложиться ситуация, когда один и тот же базовый класс наследуется (через разные цепочки наследования) в производном классе несколько раз. Обычно в этом ничего хорошего нет, поскольку нередко такие ситуации приводят к ошибкам. С другой стороны, совсем избежать подобных ситуаций бывает проблематично. Но выход есть, и состоит он в том, чтобы при создании цепочки наследования использовать *виртуальные базовые классы*.

Эффект виртуальности базового класса создается просто: при указании механизма наследования следует использовать ключевое слово `virtual`. Например, представим, что класс `Base` является базовым для классов `Alpha` и `Bravo`: то есть и класс `Alpha`, и класс `Bravo` создаются путем наследования класса `Base`. Класс `Charlie` создается наследованием классов `Alpha` и `Bravo`. Таким образом, класс `Base` наследуется в классе `Charlie` двумя путями: через класс `Alpha` и через класс `Bravo`.

В классе `Base` объявлено открытое текстовое поле `name`. В классах `Alpha` и `Bravo` ничего не объявляется и не описывается. Зато в классах `Alpha` и `Bravo` наследуется поле `name` из класса `Base`. И в теле класса `Charlie` ничего не объявляется. В классе `Charlie` также наследуется поле `name` - а точнее, два поля: из класса `Alpha` и класса `Bravo`. Причем "первоисточник" у этого "раздвоившегося" поля один - это класс `Base`. Ситуацию иллюстрирует программный код в листинге 4.3.

Листинг 4.3. Многократное наследование базового класса

```

#include <iostream>
#include <string>
using namespace std;
// Базовый класс:
class Base{
public:
string name; // Текстовое поле
};
// Производный класс от класса Base:
class Alpha: public Base{};
// Производный класс от класса Base:
class Bravo: public Base{};
// Производный класс создается
// на основе двух базовых классов:
class Charlie: public Alpha, public Bravo{};
int main(){
Charlie obj; // Объект класса
// Значение поля name, унаследованное через
// класс Alpha:
obj.Alpha::name="Альфа";
// Значение поля name, унаследованное через
// класс Bravo:
obj.Bravo::name="Браво";
// Проверяем значения полей:
cout<<obj.Alpha::name<<endl;
cout<<obj.Bravo::name<<endl;
return 0;
}

```

В главной функции командой `Charlie obj` создается объект `obj` класса `Charlie`. Затем командами `obj.Alpha::name="Альфа"` и `obj.Bravo::name="Браво"` присваиваются значения полям `name`, унаследованным из классов `Alpha` и `Bravo` соответственно. Значения этих полей выводятся в консольное окно. Результат выполнения программы такой:

Результат выполнения программы (из листинга 4.3)

```

Альфа
Браво

```

Теперь мы несколько изменим ситуацию, воспользовавшись виртуальным наследованием класса `Base`. Измененный программный код представлен в

листинге 4.4 (жирным шрифтом выделено ключевое слово `virtual` в программном коде).

Листинг 4.4. Виртуальное наследование базового класса

```
#include <iostream>
#include <string>
using namespace std;
// Базовый класс:
class Base{
public:
string name; // Текстовое поле
};
// Производный класс от класса Base
// (виртуальное наследование):
class Alpha: virtual public Base{};
// Производный класс от класса Base
// (виртуальное наследование):
class Bravo: virtual public Base{};
// Производный класс создается
// на основе двух базовых классов:
class Charlie: public Alpha, public Bravo{};
int main(){
Charlie obj; // Объект класса
// Значение поля name, унаследованное через
// класс Alpha:
obj.Alpha::name="Альфа";
// Значение поля name, унаследованное через
// класс Bravo:
obj.Bravo::name="Браво";
// Проверяем значения полей:
cout<<obj.Alpha::name<<endl;
cout<<obj.Bravo::name<<endl;
return 0;
}
```

Схема в данном случае практически та же, как и в предыдущем случае, однако при наследовании класса `Base` в классах `Alpha` и `Bravo` указано ключевое слово `virtual`. Результат выполнения программы изменится:

Результат выполнения программы (из листинга 4.4)

Браво
Браво

Поле `name` класса `Charlie` теперь одно, а не два, как это было в предыдущем примере. Другими словами, поле `name`, наследуемое через класс `Alpha`, и поле `name`, наследуемое через класс `Bravo`, теперь интерпретируются как одно и то же поле. Поэтому команда `obj.Alpha::name="Альфа"` "перекрывается" командой `obj.Bravo::name="Браво"`: сначала поле `name` получает значение "Альфа" в результате выполнения первой команды, а затем поле `name` получает значение "Браво" в результате выполнения второй команды.

4.3. Абстрактные классы и чисто виртуальные методы

*Работа секретная. Думаю, с космосом связанная. Так что читайте газеты!
из к/ф "Усатый нянь"*

Метод в классе можно не описывать, а только объявить. Другими словами, в классе допустимо указать тип результата метода, его название и аргументы, а программный код тела метода не указывать. Такие методы называются *чисто виртуальными* и описываются с ключевым словом `virtual`. Заканчивается описание чисто виртуального метода знаком равенства и нулем. Общий шаблон объявления чисто виртуального метода следующий (жирным шрифтом выделены ключевые элементы):

```
virtual тип_результата имя_метода(список_аргументов)=0;
```

Если класс содержит хотя бы один чисто виртуальный метод, то класс называется *абстрактным*. Абстрактный класс отличается от обычного класса тем, что содержит, по меньшей мере, хотя бы один чисто виртуальный метод.

У абстрактных классов есть (в известном смысле очевидная) особенность: на основе абстрактного класса нельзя создать объект. Причину понять не сложно - если в классе метод только объявлен, но не описан (то есть если метод чисто виртуальный), то даже если бы на основе такого класса можно было создать объект, данный объект содержал бы не определенные методы. Проку здесь мало. Поэтому абстрактный класс представляет интерес как таковой, а не как лекало для создания объектов. Обычно с помощью абстрактных классов создается стандарт, на основе которого путем наследования создаются новые классы. Другими словами, на основе абстрактного класса нельзя создать объект, но можно создать производный класс. Это и есть основное предназначение абстрактных классов.



На заметку

Если производный класс создается на основе абстрактного класса, то в производном классе должны быть определены все чисто виртуальные методы, объявленные в абстрактном классе. Если хоть один из чисто виртуальных методов, объявленных в базовом абстрактном классе, в производном классе не описан, то созданный производный класс также будет абстрактным.

Как иллюстрацию рассмотрим пример из листинга 4.5. В этой программе описывается абстрактный класс, а затем на основе абстрактного класса создаются производные классы.



На заметку

В программе реализована идея о вычислении объема и площади поверхности нескольких геометрических фигур: речь идет о кубе, сфере и правильном тетраэдре. Каждой из этих геометрических фигур соответствует отдельный класс (*Cube*, *Sphere* и *Tetrahedron*). Классы создаются путем наследования абстрактного класса *Base*. В абстрактном классе *Base* описано несколько полей и методов.

Листинг 4.5. Наследование абстрактного класса

```
#include <iostream>
#include <string>
using namespace std;
// Базовый абстрактный класс:
class Base{
public: // Открытые члены класса
double pi; // Число "пи"
double s2; // Корень квадратный из двух
double s3; // Корень квадратный из трех
double size; // Линейные размеры
string name; // Название фигуры
    // Чисто виртуальный метод (вычисление объема):
virtual double volume()=0;
    // Чисто виртуальный метод (площадь поверхности):
virtual double area()=0;
    // Описание метода, предназначенного для
    // присваивания значения полям:
void set(double L,string txt){
size=L; // Линейный размер фигуры
name=txt; // Название фигуры
}
    // Описание метода для отображения
    // характеристик геометрической фигуры:
void show(){
cout<<"Фигура: "<<name<<endl;
```

```

cout<<"Линейныйразмер: "<<size<<endl;
cout<<"Объем: "<<volume()<<endl;
cout<<"Площадьповерхности: "<<area()<<endl;
}

// Конструкторкласса:
Base(double L,string txt){
pi=3.141592; // Значение числа "пи"
s2=1.414213; // Значение корня квадратного из двух
s3=1.732050; // Значение корня квадратного из трех
    // Вызывается метод для присваивания значения
    // полям size и name:
set(L,txt);
}
};

// Класс создается на основе абстрактного класса (куб):
class Cube: public Base{
public: // Открытыечленыкласса
// Конструктор производного класса:
Cube(double L): Base(L,"куб"){
show(); // Отображение параметров фигуры
}
// Описание унаследованного из базового класса
// чисто виртуального метода (объем куба):
double volume(){
return size*size*size;
}
// Описание унаследованного из базового класса чисто
// виртуального метода (площадь поверхности куба):
double area(){
return 6*size*size;
}
};

// Класс создается на основе абстрактного класса (сфера):
class Sphere: publicBase{
public: // Открытыечленыкласса
// Конструктор производного класса:
Sphere(double L): Base(L,"сфера"){
show(); // Отображение параметров фигуры
}
// Описание унаследованного из базового класса
// чисто виртуального метода (объем сферы):
double volume(){
return 4*pi*size*size*size/3;
}
// Описание унаследованного из базового класса чисто
// виртуального метода (площадь поверхности сферы):

```



```
double area(){
return 4*pi*size*size;
}
};
// Класс создается на основе
// абстрактного класса (тетраэдр):
class Tetrahedron: public Base{
public: // Открытые члены класса
// Конструктор производного класса:
Tetrahedron(double L): Base(L,"тетраэдр"){
show(); // Отображение параметров фигуры
}
// Описание унаследованного из базового класса чисто
// виртуального метода (объем тетраэдра):
double volume(){
return s2*size*size*size/12;
}
// Описание унаследованного из базового класса чисто
// виртуального метода (площадь поверхности тетраэдра):
double area(){
return s3*size*size;
}
};
// Главная функция программы:
int main(){
Cube a(10); // Объект класса для куба
Sphere b(10); // Объект класса для сферы
Tetrahedron c(10); // Объект класса для тетраэдра
return 0;
}
```

При выполнении программы получаем такой результат:

Результат выполнения программы (из листинга 4.5)

```
Фигура: куб
Линейный размер: 10
Объем: 1000
Площадь поверхности: 600
Фигура: сфера
Линейный размер: 10
Объем: 4188.79
Площадь поверхности: 1256.64
Фигура: тетраэдр
Линейный размер: 10
Объем: 117.851
```

Площадь поверхности: 173.205

Итак, в программном коде описывается абстрактный базовый класс `Base`. В этом классе есть несколько полей и методов. Некоторые методы обычные, а некоторые методы - чисто виртуальные. Назначение четырех полей типа `double` такое: в поле `pi` записывается приближенное значение числа π , в поле `s2` записывается приближенное значение для числа $\sqrt{2}$, в поле `s3` записывается приближенное значение числа $\sqrt{3}$, а поле `size` предназначено для записи в него линейного размера фигуры. Текстовое поле `name` понадобится для записи названия геометрической фигуры (мы будем использовать значения "куб", "сфера" и "тетраэдр").

Метод `set()` описан с двумя аргументами, определяющими значения полей `size` и `name` соответственно. Также в классе `Bravo` есть метод `show()`, предназначенный для отображения информации о фигуре (ее название, объем, и площадь поверхности). В теле метода `show()` вызываются методы `volume()` и `area()`, которые являются чисто виртуальными. Данные методы описываются в производных классах.

Хотя класс `Base` является абстрактным, в нем описан конструктор. В теле конструктора присваиваются значения полям `pi`, `s2` и `s3`, а также через вызов метода `set()` присваиваются значения полям `size` и `name`.

Классы `Cube`, `Sphere` и `Tetrahedron` описаны однотипно, и отличия между ними несущественные. Так, в классе `Cube` описан конструктор с одним аргументом. Этот аргумент определяет значение поля `size` и передается первым аргументом конструктору класса `Base`. Вторым аргументом конструктору класса `Base` передается текстовое значение "куб". Также в теле конструктора класса `Cube` вызывается метод `show()`. Метод `volume()` определен так, что результатом возвращается третья степень значения поля `size` (объем куба). Методом `area()` возвращается умноженный на 6 квадрат значения поля `size` (площадь поверхности куба). Аналогичная структура кода и у классов `Sphere` и `Tetrahedron` - разумеется, с поправкой на значения, которые возвращаются методами `volume()` и `area()` в каждом из этих классов.

В главной функции программы командами `Cube a(10)`, `Sphere b(10)` и `Tetrahedron c(10)` создаются объекты классов `Cube`, `Sphere` и `Tetrahedron`. Поскольку в конструкторах этих классов вызывается метод `show()`, то при создании объектов автоматически появляются сообщения с информацией о создаваемых объектах.

4.4. Переменные базовых и производных классов

Посторонние разговоры прекратить. Операция началась. За мной!
из к/ф "Старики-разбойники"

Допустим, некоторая переменная объявлена как принадлежащая к определенному типу. Тогда записать в эту переменную можно только значение соответствующего типа (если не считать ситуаций, когда имеет место *автоматическое приведение типов* - например, когда переменной типа `double` присваивается значение типа `int`). Но это если речь не идет об объектах. С объектами не все так просто. Имеется очень важная и очень полезная особенность, связанная с объектами базовых и производных классов. Состоит она в том, что *значением переменной базового класса может быть объект производного класса*.



На заметку

Важный момент, связанный с терминологией. До этого мы объявляли объекты командами вида `класс объект (аргументы)`. Мы знаем: так создается объект соответствующего класса. Но на самом деле ситуация "более хитрая", чем это кажется на первый взгляд. Сам процесс создания объекта - это как минимум выделение памяти под объект (все остальное - в зависимости от конструктора класса). А то, что мы называем *объектом* и обрабатываем в программном коде как объект, есть ни что иное, как переменная, через которую мы получаем доступ к объекту. В принципе, когда мы переменную отождествляем с объектом, то в этом ничего неправильного нет. Но чтобы понять тот механизм, который обсуждается далее, нам разумнее внести некоторые коррективы в нашу "объектную" концепцию. А именно, мы будем различать переменную и объект, к которому получаем доступ через эту переменную. Пожалуй, самый оптимальный вариант - думать об объекте как о значении переменной, которую мы будем называть для большей наглядности *объектной переменной*. Тип *объектной переменной* - это тот класс, для которого данная переменная объявляется. В принципе, значением такой *объектной переменной* может быть объект того же класса. Ранее всегда так и было. Но теперь все несколько иначе: значением *объектной переменной*, которая относится к одному классу, может быть объект, который относится к другому классу. Но это при условии, что класс объекта является производным от класса *объектной переменной*. Примерно так.

Можно сказать и так: переменной базового класса в качестве значения разрешается присвоить переменную производного класса. Например, если есть некоторый класс с названием *Базовый*, и на его основе путем наследования создается класс *Производный*, то командами `Базовый А` и `Производный В` создаются объекты А и В соответственно базового и производного классов. И в этом нет ничего особенного. Специфика "родственной" связи между базовым и производным классами состоит в том, что допустимой становится

команда `A=B`, в результате выполнения которой значением переменной `A` будет объект класса `Производный`. Это копия того объекта, который является значением переменной `B`. Важно понимать, что речь идет именно о копии. Поэтому если мы впоследствии изменим параметры объекта `B`, то на объекте `A` это не скажется никак.

Есть еще одно важное обстоятельство: если значением переменной базового класса является объект производного класса, то через такую переменную можно получить доступ только к тем полям и методам объекта-значения, которые объявлены в базовом классе. Поле и методы, описанные непосредственно в производном классе, через переменную базового класса недоступны.



На заметку

Стандартная процедура присваивания объектов подразумевает побитовое копирование объектов: при выполнении команды `объект_1=объект_2` в переменную `объект_1` записывается точная копия объекта, определяемого переменной `объект_2`. Правда, такое "поведение по умолчанию" при присваивании может быть изменено - например, путем перегрузки оператора присваивания для объектов соответствующего класса. Перегрузка оператора присваивания еще будет обсуждаться в книге.

Для более конкретного исследования ситуации с присваиванием переменным базового класса объектов производного класса рассмотрим программный код, представленный в листинге 4.6. В этом примере описан базовый класс `Base` с конструктором, текстовым полем `name` и методом `show()` для отображения значения поля `name`. На основе класса `Base` создаются классы `Alpha` и `Bravo`. В этих классах переопределяется метод `show()`. В главной функции программы создается объект `obj` класса `Base`, а также объекты `a` и `b` производных классов `Alpha` и `Bravo` соответственно. Затем переменной `obj` последовательно присваиваются значения переменных `a` и `b`, и каждый раз из переменной `obj` вызывается метод `show()`.

Листинг 4.6. Переменные базовых и производных классов

```
#include <iostream>
#include <string>
using namespace std;
// Базовый класс:
class Base{
public:
    // Текстовое поле:
    string name;
    // Конструктор класса:
```

```
Base(string txt){
name=txt;
}
// Метод для отображения значения поля:
void show(){
cout<<"Класс Base. Полename: "<<name<<endl;
}
}; // Окончание описания базового класса
// Первый производный класс:
class Alpha: public Base{
public:
// Конструктор производного класса:
Alpha(string txt): Base(txt){}
// Переопределение метода из базового класса:
void show(){
cout<<"Класс Alpha. Полename: "<<name<<endl;
}
}; // Окончание описания первого производного класса
// Второй производный класс:
class Bravo: public Base{
public:
// Конструктор производного класса:
Bravo(string txt): Base(txt){}
// Переопределение метода из производного класса:
void show(){
cout<<"Класс Bravo. Полename: "<<name<<endl;
}
}; // Окончание описания второго производного класса
// Главная функция программы:
int main(){
// Объект базового класса:
Base obj("Красный");
// Объект первого производного класса:
Alpha a("Желтый");
// Объект второго производного класса:
Bravo b("Зеленый");
// Метод вызывается из объекта базового класса:
obj.show();
// Метод вызывается из объекта
// первого производного класса:
a.show(); // Вызывается метод из базового объекта
// Метод вызывается из объекта
// второго производного класса:
b.show();
// Переменной базового класса присваивается объект
// первого производного класса:
```

```

obj=a;
    // Метод вызывается из объекта базового класса:
obj.show();
    // Переменной базового класса присваивается объект
    // второго производного класса:
obj=b;
    // Метод вызывается из объекта базового класса:
obj.show();
// Полю nameобъекта bприсваивается новое значение:
b.name="Синий";
b.show(); // Метод show() вызывается из объекта b
obj.show();// Метод show() вызывается из объекта obj
return 0;
}

```

В результате выполнения программы получаем следующее:

Результат выполнения программы (из листинга 4.6)

```

Класс Base. Поле name: Красный
Класс Alpha. Поле name: Желтый
Класс Bravo. Поле name: Зеленый
Класс Base. Поле name: Желтый
Класс Base. Поле name: Зеленый
Класс Bravo. Поле name: Синий
Класс Base. Поле name: Зеленый

```

Чтобы понять смысл происходящего, разумно выделить наиболее важные моменты. Они такие:

- В классе `Base` описано текстовое поле `name`, и это поле наследуется в классах `Alpha` и `Bravo`.
- В классе `Base` есть метод `show()`, которым отображается значение поля `name`. В классах `Alpha` и `Bravo` переопределяется метод `show()`. Методом `show()` в классах `Alpha` и `Bravo` также отображается значение поля `name` (которое наследуется из базового класса).

Различие между версиями метода `show()` из классов `Base`, `Alpha` и `Bravo` лишь во вспомогательном тексте, который отображается при вызове метода. По этому тексту удастся однозначно определить, о какой версии метода идет речь.



На заметку

В базовом классе `Base` метод `show()` объявлен как не виртуальный. Если сделать метод виртуальным (добавив инструкцию `virtual` в описание метода), результат выполнения программы не изменится.

Переменная `obj` в главной функции программы объявлена как объект класса `Base`. Но значениями ей могут присваиваться объекты производных классов `Alpha` и `Bravo`. Собственно так и происходит: в программе создается объект класса `Alpha` и объект класса `Bravo`, а затем эти объекты по очереди присваиваются значениями переменной `obj`. После каждого присваивания из объекта `obj` вызывается метод `show()`.

Разберем последствия выполнения команды присваивания `obj=a` и команды `obj.show()`, которая выполняется после этого. Объект `a` создается со значением "Желтый" поля `name`. Значение поля `name` у объекта `obj` при создании равно "Красный". В результате выполнения команды `obj.show()` (после присваивания `obj=a`) появляется сообщение Класс `Base`. Поле `name`: Желтый. Следовательно, вызывается версия метода `show()`, описанная в классе `Base`, но значение поля `name` взято из объекта `a` (точнее, копии этого объекта, которая присваивается переменной `obj`). Аналогичная ситуация имеет место при выполнении команд `obj=b` и `obj.show()`: вызывается версия метода `show()` из класса `Base`, а значение поля `name` определяется объектом `b`. Почему так происходит? Объяснение попытаемся дать на "научно-популярном" уровне.

Итак, когда создается объект базового класса, для него в памяти выделяется место. В этом месте "спрятаны" поля и методы объекта. Когда создается объект производного класса, то место в памяти для такого объекта выделяется как для объекта базового класса (назовем это "основной" памятью), и еще немножко "дополнительной" памяти. В "дополнительную" память "прячутся" поля и методы, описанные непосредственно в производном классе. Те поля и методы, что наследуются из базового класса, "спрятаны" в "основной" памяти.

Когда переменной базового класса присваивается объект производного класса, происходит копирование объекта производного класса в область памяти, выделенную для объекта базового класса. Но здесь явно имеется проблема: памяти для объекта производного класса нужно как минимум не меньше, чем для объекта базового класса. Очевидно, что не удастся скопировать всю информацию из области памяти, выделенной для объекта производного класса - банально не хватит места в области памяти, в которую выполняется копирование. Проблема решается "отбрасыванием" информации из "дополнительной" памяти, выделенной для объекта производного

класса. Другими словами, копируется не весь объект производного класса, а только та его "часть", что унаследована из базового класса.

Возвращаясь к рассматриваемому примеру легко сообразить, что при выполнении, например, команды `obj=a` поле `name` из объекта `a` копируется (поскольку оно наследовано из базового класса), а новая версия метода `show()` "теряется в пути" (при том, что старая версия метода `show()`, описанная в классе `Base`, остается).

Подробности

Откровенно говоря, с "копированием" методов не все так просто. Но в данном случае мы можем смело отрываться от реальности и исходить из того, что методы "копируются" так же, как и поля.

Стоит обратить внимание и вот еще на что: после того, как выполнена команда `obj=b`, которой переменной `obj` присваивается объект `b`, командой `b.name="Синий"` полю `name` объекта `b` присваивается новое значение. Но проверка командой `obj.show()` значения поля `name` объекта `obj` показывает, что оно не изменилось. Причина как раз в том, что при выполнении команды `obj=b` в переменную `obj` копируется значение переменной `b`, поэтому переменные `obj` и `b` - это разные объекты.



На заметку

Последнее замечание относится скорее к читателям, знакомым с языками программирования Java и/или C#. В этих языках доступ к объектам реализуется фактически через ссылку и команды, подобные приведенным выше, приводят к совершенно иным результатам.

Подробности

Желающие могут провести следующий небольшой эксперимент: добавить в описание метода `show()` в классе `Base` ключевое слово `virtual`, сделав тем самым метод виртуальным, и запустить программу на выполнение. Как отмечалось ранее, результат выполнения программы не изменится.

Впоследствии мы познакомимся со ссылками и указателями и узнаем, что доступ к объекту можно получить не только через переменную, но и через указатель или ссылку. У указателей и ссылок есть особенность, аналогичная рассмотренной выше: указатель/ссылка базового класса может ссылаться на объект производного класса. И если в производном классе переопределяется виртуальный метод из базового класса, то через указатель/ссылку базового класса можно получить доступ к переопределенной версии метода. Но это так, на будущее.

Итак, если переменной базового класса присваивается объект производного класса, то через такую переменную можно получить доступ только к тем полям и методам объекта производного класса, которые объявлены в базовом классе. Ситуация становится нетривиальной, если речь идет о множественном наследовании. Небольшая иллюстрация к сказанному представлена в программном коде в листинге 4.7.

Листинг 4.7. Переменные базовых и производных классов при множественном наследовании

```
#include <iostream>
#include <string>
using namespace std;
// Первый базовый класс:
class Alpha{
public:
int code; // Числовое поле
// Конструктор класса:
Alpha(int n=0){
code=n; // Полю присваивается значение
}
// Метод для отображения значения поля:
void show(){
cout<<"Поле code: "<<code<<endl;
}
// Метод для отображения сообщения:
void hi(){
cout<<"Вас приветствует класс Alpha!\n";
}
};
// Второй базовый класс:
class Bravo{
public:
string name; // Текстовое поле
// Конструктор класса:
Bravo(string txt=""){
name=txt; // Полю присваивается значение
}
// Метод для отображения значения поля:
void show(){
cout<<"Поле name: "<<name<<endl;
}
// Метод для отображения сообщения:
void hello(){
cout<<"Вас приветствует класс Bravo!\n";
```

```

}
};
// Производный класс (создается на основе двух базовых):
class Charlie: public Alpha, public Bravo{
public:
    // Конструктор производного класса:
    Charlie(int n,string txt): Alpha(n),Bravo(txt){}
    // Метод для отображения приветствия:
    void say(){
        cout<<"Вас приветствует класс Charlie!\n";
        Alpha::show(); // Метод из первого базового класса
        Bravo::show(); // Метод из второго базового класса
    }
};
// Главная функция программы:
int main(){
    Alpha a; // Объект первого базового класса
    Bravo b; // Объект второго базового класса
    // Объект производного класса:
    Charlie c(123,"Браво");
    // Метод доступен только через переменную
    // производного класса:
    c.say();
    // Переменной первого базового класса
    // присваивается объект производного класса:
    a=c;
    // Переменной второго базового класса
    // присваивается объект производного класса:
    b=c;
    // Вызов методов, доступных через переменную
    // первого базового класса:
    a.hi();
    a.show();
    // Вызов методов, доступных через переменную
    // второго базового класса:
    b.hello();
    b.show();
    return 0;
}

```

Результат выполнения программы следующий:

Результат выполнения программы (из листинга 4.7)

Вас приветствует класс Charlie!
Поле code: 123

```
Поле name: Браво
Вас приветствует класс Alpha!
Поле code: 123
Вас приветствует класс Bravo!
Поле name: Браво
```

В программном коде объявлены два базовых класса Alpha и Bravo. В классе Alpha описано числовое поле code, а также методы show() и hi(). Методом hi() отображается сообщение, а методом show() отображается значение поля code. Аналогичный метод (метод с названием show()) описан в классе Bravo, но только в этом классе метод предназначен для отображения значения поля name. Также в классе описан метод hello(), которым отображается сообщение.

Классы Alpha и Bravo являются базовыми для класса Charlie. В классе Charlie наследуются поля code и name, а также методы hi(), hello() и два метода с одинаковыми названиями show(). Также в классе Charlie описан метод say(). В теле метода вызываются обе версии метода show() (унаследованные из классов Alpha и Bravo).

В главной функции программы создаются объекты a, b и соответственно классов Alpha, Bravo и Charlie. Переменным a и b присваивается как значение объект c. После этого через переменную a доступны методы show() (та версия, что унаследована в классе Charlie из класса Alpha) и hi(), а также поле code. Через переменную b доступны методы show() (та версия, что унаследована в классе Charlie из класса Bravo) и hello(), а также поле name. А вот метод say(), надо отметить, доступен только через переменную c.

Глава 5.

ССЫЛКИ И УКАЗАТЕЛИ



- А для чего глухонемому иностранцу переводчик?

- Много будешь знать - скоро состаришься.

из к/ф "Старики-разбойники"

В этой главе речь пойдет о нескольких важных механизмах, которые напрямую отношения к ООП не имеют, но принципиально важны для понимания всей концепции программирования в C++. Мы познакомимся со *ссылками*, *указателями*, а также обсудим *режимы* или *механизмы передачи аргументов* методам (и функциям). Поскольку нашей стратегической целью является все же ООП, то обсуждать будем в основном ссылки на объекты и указатели на объекты. При рассмотрении режимов передачи аргументов акцент делается на передачу аргументов методам.

5.1. Знакомство со ссылками

Ясность – одна из форм полного тумана.

из к/ф "Семнадцать мгновений весны"

Ссылка - это альтернативный способ обратиться к переменной, будь то переменная базового типа или объект. Нас ссылки будут интересовать в первую очередь в контексте работы с объектами. Поэтому рассмотрим ситуации, в которых речь идет о *ссылках на объекты*.

Прежде, чем приступить к разбору ссылок, рассмотрим операцию присваивания вида `objV=objA`, которой в объектную переменную `objV` копируется объект `objA`. Для простоты будем полагать, что объектная переменная `objV` относится к тому же классу, что и объектная переменная `objA`. Прежде чем команда `objV=objA` будет выполнена, объекты `objA` и `objV` должны быть созданы. При объявлении объектных переменных `objA` и `objV` для них в памяти выделяется место. А поскольку речь идет об объектных переменных, то фактически при их объявлении создаются объекты. При выполнении команды `objV=objA` происходит побитовое копирование объекта `objA` в область памяти, выделенную под объект `objV`. В результате создается копия объекта `objA`, которая записывается в объектную переменную `objV`. Вся

эта схема имеет место по умолчанию и реализуется в том случае, когда речь идет об обычной объектной переменной, не о ссылке.

Теперь посмотрим на ситуацию с другой точки зрения. Допустим, что имеется некоторый объект `obj` - или объектная переменная `obj`, что, в общем-то, практически одно и то же. Каждый раз, когда мы используем в программном коде объектную переменную `obj`, выполняется обращение к соответствующему объекту. Наша задача состоит в том, чтобы создать переменную `ref`, которая бы ссылалась на тот же самый объект. Важно еще раз отметить: речь идет не о копии объекта, а об одном и том же объекте. Воспользоваться командой вида `ref=obj` не получится, поскольку в этом случае будет создана копия объекта `obj` и записана в переменную `ref`. Чтобы создать переменную, при вызове которой выполняется обращение к объекту `obj` (то есть к тому же объекту, который записан в переменную `obj`), создают *ссылку*. Фактически по определению ссылка - это переменная, которая "указывает" или "ссылается" на уже существующую переменную. Можно также сказать, что ссылка - это *синоним* или *дополнительное название* для некоторой переменной.

Ссылка не может существовать сама по себе - она должна обязательно указывать на какое-то место в памяти. Поэтому при объявлении ссылки сразу же нужно указать переменную, на которую ссылка выполняется. Чтобы отличить ссылку от обычной переменной, при объявлении ссылки перед ее именем указывается символ `&`. Шаблон объявления (и инициализации) ссылки такой:

```
тип &имя_ссылки=переменная;
```

В данном случае речь идет о создании ссылки, которая относится к заданному типу, и ссылается на указанную переменную. Более конкретно, если нам нужно создать ссылку `ref` на объектную переменную `obj` класса `MyClass`, то соответствующая команда будет выглядеть так:

```
MyClass &ref=obj;
```

В результате и переменная `obj`, и переменная `ref` будут отождествляться с одним и тем же объектом.



На заметку

Объявление обычной переменной и объявление ссылки - процессы принципиально разные. При объявлении переменной для этой переменной выделяется место в памяти. Туда записывается значение переменной. При объявлении ссылки место в памяти не выделяется. Ссылка "получает" и "сберегает" свое значение в области

памяти, выделенной для другой переменной. Поэтому объявление ссылки подразумевает некоторую "индикацию" того, что это именно ссылка, а не обычная переменная. Таким "индикатором" и служит инструкция `&`, которая указывается перед именем ссылки при ее объявлении.

Работа со ссылками имеет свои особенности. Нам сейчас важно обратить внимание вот на что:

- При описании ссылки необходимо указать переменную, для которой ссылка будет "синонимом".
- Через ссылку можно обращаться к указанной переменной (считать и присваивать значение), но "перебросить" ссылку на другую переменную не получится.



На заметку

Другими словами, переменная, для которой выполняется ссылка, определяется один раз при создании ссылки и впоследствии изменена быть не может.

Небольшой пример, в котором используется ссылка, представлен в листинге 5.1. В программе описывается класс `MyClass`. В классе есть текстовое поле `name`, описан конструктор с одним аргументом (определяет значение поля `name`), а еще имеется метод `show()` для отображения значения поля `name`. В главной функции программы создаются два объекта `objA` и `objB` класса `MyClass`. Также создается ссылка `ref` на объект `objA`. Через эту ссылку из объекта `objA` вызывается метод `show()`. Затем через ссылку `ref` меняется значение поля `name` объекта `objA`, а для проверки результата метод `show()` вызывается из объекта `objA`.

Далее переменной-ссылке `ref` присваивается новое значение: объект `objB`. После выполнения команды `ref=objB` в переменную `objA` записывается значение `objB`. Поэтому в результате выполнения команды `ref.name="новое значение"` на самом деле изменяется значение поля `name` объекта `objA`. Как следствие при выполнении команд `ref.show()` и `objA.show()` отображается новое значение поля `name` объекта `objA`. Значение поля `name` объекта `objB` остается неизменным. В последнем убеждаемся, воспользовавшись командой `objB.show()`. Рассмотрим программный код примера:

Листинг 5.1. Использование ссылок

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
```



```

class MyClass{
public:
string name; // Текстовое поле
// Конструктор класса:
MyClass(string txt){
name=txt; // Значение поля
}

// Метод для отображения значения поля:
void show(){
cout<<"Поле name: "<<name<<endl;
}
};

// Главная функция программы:
int main(){
MyClass objA("объект А"); // Первый объект
MyClass objB("объект В"); // Второй объект
MyClass &ref=objA; // Ссылка на первый объект
ref.show(); // Вызов метода через ссылку
ref.name="ссылка"; // Изменение поля через ссылку
objA.show(); // Вызов метода из первого объекта
ref=objB; // Изменение значения через ссылку
ref.name="новое значение";
ref.show(); // Вызова метода через ссылку
objB.show(); // Вызов метода из второго объекта
objA.show(); // Вызов метода из первого объекта
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 5.1)

```

Поле name: объект А
Поле name: ссылка
Поле name: новое значение
Поле name: объект В
Поле name: новое значение

```

Итак, обращение (включая считывание значений, вызов методов и изменение значения полей) через ссылку к объекту, на который она ссылается, эквивалентно обращению к этому же объекту через "родную" переменную.

В рассмотренном примере мы, по большому счету, с помощью переменной-ссылки "альтернативно" обращались к объекту, что в некоторых случаях может быть удобным. Однако существуют и более "утонченные" способы использования ссылок.

5.2. Ссылки и наследование

*Этого объяснить я Вам не могу, потому что сам толком ничерта не понимаю.
из к/ф "Семнадцать мгновений весны"*

В рассматриваемом далее примере создается базовый класс `Base`, у класса есть текстовое поле `name` и метод `show()`. При вызове метода отображается информация о классе и значение поля `name`. На основе класса `Base` создаются производные классы `Alpha` и `Bravo`. В этих классах переопределяется метод `show()`: изменяется текст, отображаемый при вызове метода.



На заметку

В классе `Base` метод `show()` описан как виртуальный - с ключевым словом `virtual`. Это важно, поскольку мы планируем обращаться к объектам производных классов через ссылки базового класса.

Для каждого из классов (`Base`, `Alpha` и `Bravo`) создается по объекту, и на каждый из этих объектов создается ссылка. Причем независимо от класса объекта, ссылка относится к классу `Base`. Это возможно, поскольку класс `Base` является базовым для классов `Alpha` и `Bravo`. Через ссылки на объекты вызывается метод `show()`. Причем перед тем, как вызывать метод `show()`, полю `name` каждого из объектов присваивается значение. То есть последовательность действий такая: сначала на объекты выполняются ссылки, затем полям `name` объектов присваиваются значения, а после этого через ссылки на объекты вызывается метод `show()`, которым отображается значение поля `name` соответствующего объекта. Программный код примера приведен в листинге 5.2.

Листинг 5.2. Ссылки и наследование

```
#include <iostream>
#include <string>
using namespace std;
// Базовый класс:
class Base{
public:
    string name; // Текстовое поле
    // Виртуальный метод:
    virtual void show(){
        // Отображение значения поля:
        cout<<"Класс Base. Поле name: "<<name<<endl;
    }
}
```

```
};
// Первый производный класс:
class Alpha: public Base{
public:
// Переопределение метода:
void show(){
cout<<"Класс Alpha. Полename: "<<name<<endl;
}
};
// Второй производный класс:
class Bravo: public Base{
public:
// Переопределение метода:
void show(){
cout<<"Класс Bravo. Полename: "<<name<<endl;
}
};
// Главная функция программы:
int main(){
Base obj;    // Объект базового класса
Alpha objA; // Объект первого производного класса
Bravo objB;  // Объект второго производного класса
// Ссылка на объект базового класса:
Base &ref=obj;
// Ссылка на объект первого производного класса:
Base &refA=objA;
// Ссылка на объект второго производного класса:
Base &refB=objB;
// Значение поля name объекта базового класса:
obj.name="Базовый";
// Значение поля nameобъекта
// первого производного класса:
objA.name="Альфа";
// Значение поля nameобъекта
// второго производного класса:
objB.name="Браво";
// Вызов метода show() через ссылки на объекты:
ref.show();
refA.show();
refB.show();
return 0;
}
```

При выполнении программы получаем такой результат:

Результат выполнения программы (из листинга 5.2)

Класс Base. Поле name: Базовый

Класс Alpha. Поле name: Альфа

Класс Bravo. Поле name: Браво

Нечто похожее мы наблюдали в предыдущей главе, когда рассматривали присваивание объектным переменным базового класса объектов производного класса. Однако в данном случае имеется одно очень важное отличие: через ссылку базового класса мы получаем доступ к переопределенной в производном классе версии виртуального метода. С обычными переменными такой "номер" не проходил.

Подробности

Напомним, что при копировании объекта производного класса в переменную базового класса "дополнительная" информация теряется (см. пояснения в предыдущей главе). В случае со ссылками все по-другому. Создание ссылки не предполагает выделение памяти. Соответственно, если для ссылки базового класса при инициализации в качестве значения указывается объект производного класса, то копирования этого объекта не происходит. Ссылка просто "указывает" на данный объект. Другими словами, "дополнительная" информация не теряется, и к ней в принципе можно получить доступ. Нечто похожее имеет место при работе с указателями.

Вывод простой: как и в случае с объектной переменной, ссылка базового класса может ссылаться на объект производного класса. Но у ссылки есть преимущество: через нее получаем доступ к переопределенным в производном классе виртуальным методам. Более того, базовый класс, для которого объявляется ссылка, может быть абстрактным.

**На заметку**

Напомним, что для абстрактного класса объект создать нельзя. Зато можно создать ссылку базового абстрактного класса на объект производного класса.

Несколько измененный программный код из предыдущего примера представлен в листинге 5.3. Принципиальное новшество состоит в том, что базовый класс является абстрактным.

Листинг 5.3. Ссылки и абстрактные классы

```
#include <iostream>
#include <string>
using namespace std;
// Базовый абстрактный класс:
class Base{
```

```
public:
string name; // Текстовое поле
// Чисто виртуальный метод:
virtual void show()=0;
};
// Первый производный класс:
class Alpha: public Base{
public:
// Определение метода:
void show(){
cout<<"Класс Alpha. Поле name: "<<name<<endl;
}
};
// Второй производный класс:
class Bravo: public Base{
public:
// Определение метода:
void show(){
cout<<"Класс Bravo. Поле name: "<<name<<endl;
}
};
// Главная функция программы:
int main(){
Alpha objA; // Объект первого производного класса
Bravo objB; // Объект второго производного класса
// Ссылка на объект первого производного класса:
Base &refA=objA;
// Ссылка на объект второго производного класса:
Base &refB=objB;
// Значение поля name объекта
// первого производного класса:
objA.name="Альфа";
// Значение поля name объекта
// второго производного класса:
objB.name="Браво";
// Вызов метода show() через ссылки на объекты:
refA.show();
refB.show();
return 0;
}
```

Получаем такой результат:

Результат выполнения программы (из листинга 5.3)

Класс Alpha. Поле name: Альфа
Класс Bravo. Поле name: Браво

Класс `Base` является абстрактным из-за того, что в нем метод `show()` объявлен как чисто виртуальный. Поэтому мы не можем создать объект класса `Base`, но можем создать ссылку класса `Base`. Ссылка может выполняться на объект класса, производного от базового (хотя бы и абстрактного). Собственно, так мы и поступили выше.

Помимо ссылок есть еще один механизм, который позволяет получать "альтернативный" доступ к объектам и обычным переменным. Причем механизм этот намного более гибкий и эффективный, по сравнению со ссылками. Имеются в виду *указатели*. Они обсуждаются далее. Но прежде мы уделим внимание вопросу, имеющему определенное отношение к ссылкам. Речь пойдет о способе (или *механизме*) *передачи аргументов* методам и функциям. Хотя на первый взгляд может показаться, что данная тема мало-значительная, на самом деле все как раз наоборот. Без понимания того, как аргументы передаются в метод или функцию, крайне сложно разобраться во многих важных моментах реализации объектно-ориентированной парадигмы программирования в языке C++.

5.3. Механизм передачи аргументов

Так скучно, что даже простой сквозняк - развлечение.

из к/ф "Приключения принца Флоризеля"

Истина проста и лаконична: в C++ существует два *механизма передачи аргументов*. По умолчанию, если некоторая переменная указана как аргумент функции или метода, то автоматически создается копия такой переменной, и уже эта копия передается в метод или функцию для проведения всех необходимых вычислений. По завершении выполнения метода/функции все локальные переменные удаляются из памяти, в том числе и копии аргументов. Такой механизм передачи аргументов называется *передачей аргументов по значению*. Если нас такой вариант не устраивает, то можно использовать другой механизм передачи аргументов *- по ссылке*. При передаче аргументов по ссылке в метод передаются непосредственно те переменные, которые указаны аргументами. То есть никакие копии для переменных-аргументов не создаются и все вычисления происходят непосредственно с аргументами.

Когда важен механизм передачи аргументов? Обычно это имеет значение, если при вызове метода предпринимается попытка в том или ином виде изменить аргументы этого самого метода. Поэтому общее правило гласит: если при выполнении метода или функции изменять аргументы не планируется, то о механизме передачи аргументов можно, как правило, не беспокоиться.



На заметку

Это правило неплохо работает, когда речь идет о переменных (аргументах) базовых типов. Если аргументами являются объекты, то все намного сложнее. Дело в том, что для объектов создание копии - процесс не всегда тривиальный. По умолчанию копия объекта создается путем побитового копирования. С другой стороны, в классе может быть явно описан конструктор создания копии объекта. Если так, то процесс создания копии определяется конструктором создания копии. В случае, когда аргументом методу передан объект и используется механизм передачи аргумента по значению (а такой механизм, напомним, используется по умолчанию), для создания "технической" копии объекта и передачи ее аргументом методу будет вызван конструктор создания копии. И многое (если не все) зависит от того, как именно описан данный конструктор. Но все эти вопросы будут обсуждаться немного позже. Сейчас нас интересуют базовые отличия двух механизмов передачи аргументов.

Чтобы аргумент методу или функции передавался по ссылке, а не по значению, в описании метода или функции перед именем этого аргумента следует указать символ `&`.

Различия в механизмах передачи аргументов проиллюстрируем на нескольких небольших примерах. Начнем с программного кода из листинга 5.4, в котором описан класс `MyClass` текстовым полем `name`. У класса есть конструктор с одним аргументом (которым присваивается значение полю `name`), и метод `show()` для отображения значения поля. Еще у класса есть метод `swap()`. Он не возвращает результат и у него один аргумент - объект класса `MyClass`. При вызове метода `swap()` значение поля `name` объекта, переданного аргументом методу, присваивается объекту, из которого вызывается метод. В свою очередь, "старое" значение поля `name` объекта, из которого вызывается метод, присваивается полю `name` объекта, переданного аргументом методу `swap()`. Происходит такой своеобразный "обмен" значениями полей. Обратимся теперь к программному коду:

Листинг 5.4. Передача аргумента по значению

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    MyClass(string txt){
    // Значение поля:
    name=txt;
    }
```

```

    // Метод для отображения значения поля:
void show(){
cout<<"\tОбъектсполем "<<name<<endl;
}

    // Методдля "обмена" полями:
void swap(MyClassobj){
cout<<"\tВыполняетсяметодswap(). ";
cout<<"Значения полей до \"обмена\".\n";
cout<<"\tОбъект, из которого вызывается метод swap():\n\t";
// Отображение значения поля объекта,
    // из которого вызывается метод:
show();
cout<<"\tОбъект - аргумент метода swap():\n\t";
    // Отображение значения поля объекта,
    // переданного аргументом методу:
obj.show();
    // Переменная для запоминания "старого" значения
    // поля объекта, из которого вызывается метод:
string txt;
    // Запоминается исходное значение поля:
txt=name;
    // Новое значение поля:
name=obj.name;
    // Новое значение поля nameобъекта, переданного
    // аргументом методу swap():
obj.name=txt;
cout<<"\tОбъекты \"обменялись\" полями.\n\t";
cout<<"\tОбъект, из которого вызывается метод swap():\n\t";
// Отображение значения поля объекта, из которого
    // вызывается метод swap():
show();
cout<<"\tОбъект - аргумент метода swap():\n\t";
// Отображение значения поля объекта, переданного
    // аргументом методу swap():
obj.show();
cout<<"\tМетодswap() выполнен.\n";
}
};

// Главная функция программы:
int main(){
// Создаются объекты:
MyClass A("Красный");
MyClass B("Зеленый");
cout<<"После создания объектов:\n";
// Проверяются значения полей созданных объектов:
A.show();

```



```

B.show();
// Объекты "обмениваются" значениями полей:
A.swap(B);
cout<<"После \"обмена\" полями:\n";
// Проверяются значения полей объектов
// после "обмена" значениями:
A.show();
B.show();
return 0;
}

```



На заметку

Для выполнения отступов при отображении сообщений мы использовали в текстовых литералах символ табуляции `\t`.

Для вставки в текст символа двойных кавычек использовалась инструкция `\`. Использовать просто двойные кавычки нельзя, поскольку в C++ двойные кавычки применяются для выделения текстовых литералов.

Ниже приведен результат выполнения программы:

Результат выполнения программы (из листинга 5.4)

```

После создания объектов:
    Объект с полем Красный
    Объект с полем Зеленый
Выполняется метод swap(). Значения полей до "обмена".
    Объект, из которого вызывается метод swap():
        Объект с полем Красный
    Объект - аргумент метода swap():
        Объект с полем Зеленый
Объекты "обменялись" полями.
    Объект, из которого вызывается метод swap():
        Объект с полем Зеленый
    Объект - аргумент метода swap():
        Объект с полем Красный
Метод swap() выполнен.
После "обмена" полями:
    Объект с полем Зеленый
    Объект с полем Зеленый

```

Что тут, собственно, происходит? Мы создаем два объекта (А и В) класса MyClass. У объекта А поле name имеет значение "Красный", а у объекта В поле name имеет значение "Зеленый" (значения полей передаются конструктору при создании объектов). Затем командой `A.swap(B)` из объекта

Авызывается метод `swap()` с аргументом - объектом `B`. В теле метода значение поля `name` объекта `B` присваивается полю `name` объекта `A` и наоборот - значение поля `name` объекта `A` присваивается полю `name` объекта `B`. Во всяком случае, мы на это очень рассчитываем. Более того, при проверке в теле метода результата "обмена" значениями полей все вроде выглядит вполне ожидаемо: по тем сообщениям, которые появляются в консольном окне, можно сделать вывод, что "обмен" состоялся. Но как только метод `swap()` завершает работу, и мы пытаемся проверить значения полей `name` объектов `A` и `B` с помощью метода `show()`, то оказывается, что у объекта `A` значение поля `name` изменилось, в то время как значение поля `name` объекта `B` осталось неизменным.

Чтобы понять причины происходящего, необходимо учесть, как объект передается аргументом методу `swap()`. А именно, создается копия этого объекта. У объекта-копии все точно такое, как у объекта `B`. Но только это разные объекты. Объект-копия существует, пока выполняется метод `swap()`. Во всех командах в теле метода `swap()` имя `obj`, формально обозначающее аргумент метода `swap()` и отождествляемое в случае команды `A.swap(B)` с объектом `B`, на самом деле обозначает копию объекта `B`. Поэтому когда значение `obj.name` присваивается полю `name` объекта, из которого вызывается метод `swap()`, то все выполняется ожидаемо. Ведь у копии объекта `B` и самого объекта `B` значения полей `name` совпадают. Как результат поле `name` объекта `A` получает новое значение.



На заметку

Поле `name` объекта, из которого вызывается метод `swap()`, не является аргументом этого метода. Поэтому команда `name=obj.name` в теле метода `swap()` выполняется так: берется "оригинал" поля `name` объекта, из которого вызван метод, и ему присваивается значение поля `name` копии объекта, переданного аргументом функции.

Совсем иная ситуация, когда выполняется команда вида `obj.name=txt`. Этой командой значение `txt` присваивается полю `name` *копии* объекта, переданного аргументом методу `swap()`. Когда в теле метода `swap()` выполняется команда `obj.show()`, то отображается значение поля `name` *копии* объекта-аргумента метода `swap()`, а не самого объекта-аргумента. После завершения выполнения метода `swap()` копия объекта-аргумента удаляется из памяти, а исходный объект (объект `B` в данном случае) остается неизменным.

Чтобы принципиально изменить ситуацию, несколько видоизменим программный код. Более конкретно, опишем в классе `MyClass` метод `swap()`, внося следующие изменения в прототип метода (изменения выделены жирным шрифтом):

```
void swap(MyClass&obj){
    // Тело метода - без изменений
}
```

Здесь в описании метода `swap()` перед именем аргумента `obj` указан символ `&`, что означает передачу аргумента по ссылке. Весь программный код будет выглядеть так, как показано в листинге 5.5 (для удобства жирным шрифтом выделено место, в котором появились изменения, а большинство комментариев удалено для сокращения объема кода - заодно читатель сможет проверить себя на предмет понимания кода программы).

Листинг 5.5. Передача аргумента по ссылке

```
#include <iostream>
#include <string>
using namespace std;
class MyClass{
public:
    string name;
    MyClass(string txt){
        name=txt;
    }
    void show(){
        cout<<"\tОбъект с полем "<<name<<endl;
    }
    // Аргумент передается по ссылке:
    void swap(MyClass &obj){
        cout<<"\tВыполняется метод swap(). ";
        cout<<"Значения полей до \"обмена\".\n";
        cout<<"\tОбъект, из которого вызывается метод swap():\n\t";
        show();
        cout<<"\tОбъект - аргумент метода swap():\n\t";
        obj.show();
        string txt;
        txt=name;
        name=obj.name;
        obj.name=txt;
        cout<<"\tОбъекты \"обменялись\" полями.\n\t";
        cout<<"\tОбъект, из которого вызывается метод swap():\n\t";
        show();
        cout<<"\tОбъект - аргумент метода swap():\n\t";
        obj.show();
        cout<<"\tМетодswap() выполнен.\n";
    }
};
```

```

int main() {
    MyClass A("Красный");
    MyClass B("Зеленый");
    cout<<"После создания объектов:\n";
    A.show();
    B.show();
    A.swap(B);
    cout<<"После \"обмена\" полями:\n";
    A.show();
    B.show();
    return 0;
}

```

Теперь результат выполнения программы такой:

Результат выполнения программы (из листинга 5.5)

После создания объектов:

Объект с полем Красный

Объект с полем Зеленый

Выполняется метод `swap()`. Значения полей до "обмена".

Объект, из которого вызывается метод `swap()`:

Объект с полем Красный

Объект - аргумент метода `swap()`:

Объект с полем Зеленый

Объекты "обменялись" полями.

Объект, из которого вызывается метод `swap()`:

Объект с полем Зеленый

Объект - аргумент метода `swap()`:

Объект с полем Красный

Метод `swap()` выполнен.

После "обмена" полями:

Объект с полем Зеленый

Объект с полем Красный

Видим, что в данном случае "обмен" значениями полей произошел вполне удачно. Причина - аргумент методу `swap()` передается по ссылке, поэтому все манипуляции выполняются непосредственно с тем объектом, который указан аргументом метода.



На заметку

Чтобы аргумент передавался по ссылке, символ `&` указывается только один раз при описании метода. Синтаксис команды вызова метода (с передачей аргумента по ссылке) не изменяется по сравнению со случаем, когда аргумент передается по значению.

5.4. Механизм передачи аргументов и наследование

*Ну, зачем такие сложности?!
из к/ф "Приключения Шерлока Холмса и
доктора Ватсона"*

Мы рассмотрим еще один пример, в котором неявно используется возможность присваивать значению объектной переменной базового класса объект производного класса. В программном коде, представленном в листинге 5.6, описывается базовый класс Alpha и производный класс Bravo. В классе Alpha есть текстовое поле name и виртуальный метод show(), которым отображается значение поля name (ну и еще небольшой дополнительный текст). В классе Bravo наследуется поле name и переопределяется метод show() (он по-прежнему отображает значение поля name, но дополнительный текст - другой). А еще в программе описана функция getInfo() с аргументом - объектом базового класса Alpha. В теле функции из объекта, переданного аргументом, вызывается метод show().

Все самое интересное происходит в функции main(). Там создаются два объекта: объект класса Alpha и объект b класса Bravo. Полям name каждого из этих объектов присваиваются значения, после чего вызывается функция getInfo(): сначала ей аргументом передается объект a, а затем аргументом функции передается объект b. Так можно делать именно благодаря тому, что класс Bravo, к которому относится объект b, является производным от класса Alpha.



На заметку

Аргумент функции getInfo() передается по значению (такой механизм используется по умолчанию). В данном случае это важно!

Рассмотрим программный код примера:

Листинг 5.6. Аргумент функции - объект базового класса

```
#include <iostream>
#include <string>
using namespace std;
// Базовый класс:
class Alpha{
public:
    // Текстовое поле:
    string name;
    // Метод для отображения значения поля:
```

```

virtual void show(){
cout<<"Базовый класс: "<<name<<endl;
}
};
// Производный класс:
class Bravo: public Alpha{
public:
// Переопределение метода для отображения
// значения поля:
void show(){
cout<<"Производный класс: "<<name<<endl;
}
};
// Функция с аргументом - объектом базового класса:
void getInfo(Alpha obj){
cout<<"Информация об объекте:"<<endl;
// Вызов метода из объекта - аргумента функции:
obj.show();
}
// Главная функция программы:
int main(){
// Объект базового класса:
Alpha a;
// Значение поля объекта базового класса:
a.name="Альфа";
// Объект производного класса:
Bravo b;
// Значение поля объекта производного класса:
b.name="Браво";
// Функция вызывается с аргументом - объектом
// базового класса:
getInfo(a);
// Вызов метода из объекта базового класса:
a.show();
// Функция вызывается с аргументом - объектом
// производного класса:
getInfo(b);
// Вызов метода из объекта производного класса:
b.show();
return 0;
}

```

Результат выполнения программы такой (жирным шрифтом выделено место, на которое следует обратить внимание):

Результат выполнения программы (из листинга 5.6)

Информация об объекте:
 Базовый класс: Альфа
 Базовый класс: Альфа
 Информация об объекте:
Базовый класс: Браво
 Производный класс: Браво

Есть два важных момента, которые стоило бы подчеркнуть, исходя из анализа результатов выполнения программного кода. Во-первых, аргументом функции `getInfo()` можно передавать не только объект класса `Alpha`, но и объект класса `Bravo`. Во-вторых, при передаче аргументом функции `getInfo()` объекта класса `Bravo` вызывается версия метода `show()`, описанная в классе `Alpha`.



На заметку

То, что у объекта `b` (класс `Bravo`) метод `show()` переопределен, легко проверить с помощью команды `b.show()` (что, собственно, в программном коде и делается).

Здесь разумно вспомнить, что аргумент функции `getInfo()` передается по значению. Фактически происходит примерно следующее:

- Для копии объекта, который передается аргументом функции `getInfo()`, в памяти выделяется место. Причем место это для объекта класса `Alpha`. Можно сказать и так: техническая переменная, которой в качестве значения будет присваиваться копия объекта-аргумента, относится к классу `Alpha`.
- Технической переменной как значение присваивается объект класса `Bravo`. Переменная, напомним, относится к классу `Alpha`. Но поскольку класс `Bravo` является производным от класса `Alpha`, то такое в принципе возможно. Через переменную базового класса `Alpha` можно получить доступ к тем членам объекта производного класса `Bravo`, которые описаны в классе `Alpha`. Это объясняет, почему вызывается версия метода `show()` из класса `Alpha`, несмотря на виртуальность метода.

Теперь мы выясним, что изменится в результате выполнения программы, если аргумент функции `getInfo()` будет передаваться не по значению, а по ссылке. Если аргумент передается по ссылке, то код описания функции будет таким (перед названием аргумента функции указана инструкция `&`):

```
void getInfo(Alpha &obj){
    cout<<"Информация об объекте:"<<endl;
```

```
obj.show();
}
```

Минимально измененный (по сравнению с предыдущим примером) программный код приведен в листинге 5.7 (удалены комментарии и добавлена инструкция `&` перед именем аргумента `obj` в описании функции `getInfo()` -соответствующее место и комментарий выделены в программном коде жирным шрифтом).

Листинг 5.7. Объект базового класса передается по ссылке

```
#include <iostream>
#include <string>
using namespace std;
class Alpha{
public:
string name;
virtual void show(){
cout<<"Базовый класс: "<<name<<endl;
}
};
class Bravo: public Alpha{
public:
void show(){
cout<<"Производный класс: "<<name<<endl;
}
};
// Аргументом передается по ссылке:
void getInfo(Alpha &obj){
cout<<"Информация об объекте:"<<endl;
obj.show();
}
int main(){
Alpha a;
a.name="Альфа";
Bravo b;
b.name="Браво";
getInfo(a);
a.show();
getInfo(b);
b.show();
return 0;
}
```


При выполнении программы получим следующий результат (жирным шрифтом выделено место, достойное особого внимания):

Результат выполнения программы (из листинга 5.7)

Информация об объекте:
 Базовый класс: Альфа
 Базовый класс: Альфа
 Информация об объекте:
Производный класс: Bravo
 Производный класс: Bravo

Результат практически такой же, как в предыдущем случае, но теперь при передаче функции `getInfo()` аргументом объекта `бкласса Bravo` вызывается переопределенная версия метода `show()` (версия, описанная в классе `Bravo`). Причина в том, что обращение к аргументу - объекту производного класса, выполняется фактически через ссылку базового класса. Мы с подобной ситуацией уже сталкивались ранее. В этом случае решение о вызове версии метода принимается на основе типа объекта, а не типа объектной переменной. Поэтому в результате вызывается версия метода `show()`, переопределенная в производном классе.

5.5. Знакомство с указателями

*Если бы на каждую печь была отдельная
 кочерга, тогда бы придирались. А так – не
 гарантирую!*
из к/ф "Безумный день инженера Барка-
сова"

До этого нам неоднократно приходилось иметь дело с переменными. Некоторые переменные были числовыми, некоторые содержали текст, а некоторые обозначали объекты. Многообразие и сила переменных очевидна. Также мы познакомились со ссылками. Но даже этим возможности для эффективного создания программ не ограничиваются. Точнее, возможности не ограничиваются *такими переменными*. Здесь мы поговорим еще об одном типе переменных, которые в некотором смысле являются особенными и успешное их "приручение" является залогом к эффективному программированию в C++. Далее обсудим *указатели*.

Если подойти к указателям формально, то указатель - это переменная, значением которой является *адрес* области памяти. Здесь подразумевается область памяти, которая может содержать значение определенного типа. Понятно, что для разных значений нужен разный объем памяти. По этому

принципу классифицируют указатели. То есть когда мы говорим об указателе, то важно (явно или неявно) иметь в виду, данные какого типа предполагается записывать в ячейку памяти, адрес которой будет записан в указателе.



На заметку

Понять, что такое указатели и в чем их главная особенность, попробуем на примере из банковской деятельности. Предположим у нас в банке есть вклад. Для удобства будем думать, что банк на самом деле хранит деньги. Как в принципе мы получаем доступ к своим деньгам? Приходим в банк (представляемся оператору), называем номер счета (или оператор его нам называет - не принципиально). Дальше выполняются нужные операции с вкладом. Мы, образно выражаясь, можем просто узнать сумму на счете или изменить ее (снять/добавить денег). В этом случае мы должны идентифицировать счет (по имени вкладчика и/или номеру счета). Где технически находятся наши деньги не столь важно: они могут быть в сейфе за стеной или в хранилище, или в хозяйственной сумке в багажнике машины директора банка. Мы знаем номер счета и по номеру можем проверить и/или изменить его состояние. Здесь уместно провести аналогию с обычной переменной (не указателем). Номер счета - это имя переменной, а состояние счета (сумма на счету) - это значение переменной. Знаем номер счета - имеем доступ к деньгам. Знаем имя переменной - имеем доступ к ее значению.

Возможна и другая ситуация. Представим себе сотрудника банка, у которого, в силу его функциональных обязанностей, есть ключ от определенной банковской ячейки (ячейка с определенным номером). Он может ячейку открыть и закрыть. Может туда что-то положить, а может и забрать. И ему все равно, деньги с какого счета хранятся в ячейке. Это аналог указателя. Указатель "знает" адрес области памяти (номер ячейки), и с помощью указателя можно менять данные в памяти (деньги в ячейке). Но важно понимать, что значение указателя - это "номер ячейки", а не "сумма денег", которая в ней хранится.

Теория обычно выглядит проще, чем практика. В практическом плане значимыми являются следующие вопросы:

- Как создать указатель?
- Как присвоить указателю значение и что с ним можно делать (операции с указателями)?
- Как узнать адрес области памяти, в которой хранится обычная переменная?
- Как по значению указателя (адресу памяти) определить значение, записанное в этой памяти?

Будем отвечать на эти вопросы постепенно. Итак, как же создается указатель? Мы уже знаем, что это переменная. Переменные объявляются так: указывается тип переменной и ее название. Название мы выбираем сами. Как быть с типом? Поскольку для указателя важно, какого типа данные будут (или могут) записываться в область памяти, на которую ссылается указатель, то, очевидно, что этот тип должен как-то присутствовать в объявлении указателя. Если просто указать идентификатор типа при объявлении, получим обычную переменную данного типа. Нужна "метка", которая позволит отличить указатель от обычной переменной. Такой "меткой" служит звездочка *. Поэтому если мы описываем указатель, то в качестве идентификатора типа указывается тип значения, которое может записываться в соответствующую ячейку памяти, а чтобы отличить указатель от обычной переменной, перед именем указателя ставится *. Так, инструкцией `int number` объявляется целочисленная переменная с именем `number`. А вот инструкцией `int *pointer` объявляется указатель `pointer`, который как значение может содержать адрес ячейки, которая, в свою очередь, может содержать как значение целое число.



На заметку

В одной инструкции можно объявлять одновременно как обычные переменные, так и указатели. Но следует учесть, что оператор звездочка * "действует" только на ту переменную, возле которой он указан. Например, инструкцией `int *pointer, number` объявляется указатель `pointer` на целочисленное значение и обычная переменная `number` типа `int`.

Также обращаем внимание, что именем указателя является идентификатор без звездочки: то есть звездочка, которую мы используем при объявлении указателя, частью имени указателя не является.

Присваиваются значения указателям так же, как и иным переменным - с помощью оператора присваивания. Но присваивать нужно адрес. Возникает вопрос: откуда его взять? Обычно приходится иметь дело с адресами переменных, то есть адресами тех ячеек, в которые записаны переменные. Получить адрес переменной можно с помощью оператора `&`. Например, результатом выражения `&number` является адрес области памяти, в которой хранится значение переменной `number`. Такой адрес можно присвоить в качестве значения указателю `pointer`. Соответствующая команда выглядит как `pointer=&number`. После ее выполнения указатель `pointer` "знает" адрес переменной `number`. Если нам нужно узнать значение, записанное по некоторому адресу, используем оператор *. Так, значением инструкции `*pointer` является (если предварительно выполнена команда `pointer=&number`) значение переменной `number` - то есть то

значение, которое записано по адресу, хранящемуся в указателе `pointer`.



На заметку

С помощью инструкции `*pointer` можно не только "прочитать" значение, записанное по адресу, хранящемуся в указателе `pointer`, но и изменить значение в соответствующей ячейке памяти.

Что касается операций с указателями, то, помимо присваивания, здесь вариантов для деятельности не так уж и много. Среди допустимых *арифметических операций с указателями* можно выделить такие:

- К указателю разрешается прибавить целое число. Результатом операции `указатель+число` является адрес ячейки памяти. Эта ячейка отстоит (в "направлении" увеличения адреса) относительно адреса указателя на количество ячеек, определяемое числом. Например, если `pointer` - это указатель на целое число (объявлен как `int *pointer`), то результатом выражения `pointer+3` будет указатель на целочисленную ячейку, отстоящую на 3 позиции "вправо" (в "направлении" увеличения адреса) от той ячейки, на которую ссылается указатель `pointer`.
- От указателя разрешается отнять целое число. Результатом операции `указатель-число` является адрес ячейки памяти, отстоящий (в "направлении" уменьшения адреса) относительно адреса указателя на количество ячеек, определяемое числом. Например, результатом выражения `pointer-2` будет указатель на целочисленную ячейку, отстоящую на 2 позиции "влево" (в "направлении" уменьшения адреса) от той ячейки, на которую ссылается указатель `pointer`.
- Можно вычислять разность указателей. Результатом выражения `указатель_1-указатель_2` является целое число, определяющее количество ячеек между адресами, записанными в указателях. Например, если `p` и `q` - указатели (скажем, на целочисленные ячейки), то результатом выражения `q-p` будет целое число, равное количеству целочисленных ячеек, которые находятся между ячейками, на которые ссылаются указатели `q` и `p`.
- Указатели можно индексировать. Индекс указывается в квадратных скобках после имени указателя. Значение индекса - целое число (не обязательно положительное). Результатом выражения `указатель[индекс]` является *значение* ячейки, которая отстоит от ячейки с адресом указателя на количество позиций, определяемое индексом в квадратных скобках. Например, результатом

выражения `pointer[3]` будет значение целочисленной ячейки, отстоящей на 3 позиции "вправо" (в "направлении" увеличения адреса) от той ячейки, на которую ссылается указатель `pointer`. А результат выражения `pointer[-2]` - значение целочисленной ячейки, отстоящей на 2 позиции "влево" (в "направлении" уменьшения адреса) от той ячейки, на которую ссылается указатель `pointer`.

Такого типа правила называются *адресной арифметикой*. Мы о ней вспомним, когда будем иметь дело с *массивами*, и еще будем разбирать более детально.

Указатели могут создаваться не только для простых типов данных, но и для объектов. Объявление указателей на объекты подчиняется общему правилу: в качестве типа значения указывается класс объекта, а перед именем переменной-указателя ставится звездочка *. Так, указатель на объект некоторого класса можно создать командой вида `класс *указатель`. Адрес объекта можно получить, поставив амперсанд & перед именем объектной переменной. Скажем, результатом команды `указатель=&объект` станет присваивание указателю адреса объекта. Понятно, что объект должен относиться к тому же классу, который указан при создании указателя. Чтобы по указателю на объект получить сам объект, достаточно перед указателем поставить звездочку * - речь идет об инструкции вида `*указатель`. Но если нужно обратиться к полю или методу объекта, существует упрощенная форма обращения через указатель на объект: после имени переменной-указателя помещается оператор стрелки `->`, а после него - имя поля или инструкция вызова метода. Например, если у объекта есть поле, а указатель ссылается на этот объект, то обращение к полю через указатель выполняется так: `указатель->поле`. Другими словами, альтернативой к инструкции `объект.поле` является инструкция `указатель->поле`. Аналогично выполняется вызов метода через указатель - командой вида `указатель->метод(аргументы)`.

Как иллюстрацию к использованию указателей рассмотрим небольшой пример. В этом примере описывается класс `MyClass`, создается объект `obj` класса, и указатель `pnt` на объект `obj`. Обращение к полям и вызов метода объекта `obj` осуществляется через указатель `pnt` на объект. Соответствующий программный код приведен в листинге 5.8.

Листинг 5.8. Указатель на объект

```
#include <iostream>
#include <string>
using namespace std;
```

```
// Класс:
class MyClass{
public:
    string name; // Текстовое поле
    int code;    // Числовое поле
    // Метод для отображения значения полей:
    void show(){
        cout<<"Поле name: "<<name<<endl;
        cout<<"Поле code: "<<code<<endl;
    }
};
// Главная функция программы:
int main(){
    // Создание объекта класса:
    MyClass obj;
    // Указатель на объект класса:
    MyClass *pnt;
    // Указателю присваивается значение:
    pnt=&obj;
    // Присваивание значения полю name (через указатель):
    pnt->name="объект";
    // Присваивание значения полю code (через указатель):
    pnt->code=123;
    // Вызов метода show() (через указатель):
    pnt->show();
    return 0;
}
```

Результат выполнения программного кода будет таким:

Результат выполнения программы (из листинга 5.8)

```
Поле name: объект
Поле code: 123
```

У класса `MyClass` есть текстовое поле `name` и числовое поле `code`, а также метод `show()`, которым отображаются значения полей. В главной функции программы создается объект `obj` класса `MyClass`, а также указатель `pnt` на объект этого класса. Поскольку для класса не описан конструктор, то значения полям присваиваются после создания объекта в главной функции программы. Для этого, как отмечалось, создается указатель `pnt`. Адрес объекта узнаем с помощью инструкции `&obj`. Данное значение присваивается указателю `pnt`. Присваивание значений полям через указатель выполняется командами `pnt->name="объект"` и `pnt->code=123`, а метод `show()` вызывается командой `pnt->show()`.

Еще одна особенность работы с указателями иллюстрируется в программном коде в листинге 5.9. В этой программе создается два объекта одного класса и объявляется указатель на объект того же типа. Указателю последовательно присваиваются адреса первого и второго объектов и изменяются параметры этих объектов.

Листинг 5.9. Указатель и разные объекты

```
#include<iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
string name; // Текстовое поле
// Метод для отображения значения поля:
void show(){
cout<<"Поле name: "<<name<<endl;
}
};
// Главная функция программы:
int main(){
    // Создаются объекты класса:
    MyClass objA,objB;
    // Указатель на объект класса:
    MyClass *pnt;
    // Указатель на первый объект:
    pnt=&objA;
    // Поле первого объекта:
    pnt->name="объект А";
    // Указатель на второй объект:
    pnt=&objB;
    // Поле второго объекта:
    pnt->name="объект В";
    // Проверка содержимого объектов:
    objA.show();
    objB.show();
}
```

В результате выполнения программы получаем следующее:

Результат выполнения программы (из листинга 5.8)

```
Поле name: объект А
Поле name: объект В
```

Мы описываем класс `MyClass`, у которого есть текстовое поле `name` и метод `show()`, предназначенный для отображения значения поля. В главной функции программы создаются объекты `objA` и `objB`, а также указатель `pnt` на объект класса `MyClass`. Сначала командой `pnt=&objA` в указатель `pnt` записывается адрес объекта `objA`, а затем командой `pnt->name="объект А"` полю `name` объекта `objA` присваивается значение. Далее командой `pnt=&objB` в указатель `pnt` записывается адрес объекта `objB`. Командой `pnt->name="объект В"` полю `name` объекта `objB` присваивается значение. Проверка содержимого объектов выполняется командами `objA.show()` и `objB.show()`.

Важно здесь то, что используя указатель, мы получаем доступ к уже существующему объекту, а не к его копии. Так, если бы у нас была переменная `obj` класса `MyClass` и мы воспользовались командой `obj=objA`, то в таком случае была бы создана копия объекта `objA` и записана в переменную `obj`. Изменение объекта `obj` никак не повлияет на объект `objA`. Напротив, воспользовавшись командой `pnt=&objA`, мы получаем доступ непосредственно к объекту `objA` - хотя бы и через указатель. А вообще следует отметить, что указатель на объект может быть достаточно удобен во многих отношениях.



На заметку

На будущее отметим, что есть ключевое слово `this`, которое является указателем на объект, из которого вызывается метод. Это ключевое слово используют в программном коде класса, в описании методов. Например, если у класса имеется поле `name`, то обращение к этому полю в программном коде метода, описанного в классе, выглядит просто как `name` (другими словами, мы банально указываем имя поля). Но это сокращенная форма обращения к полю. На самом деле, когда речь идет о поле объекта, то обычно указывается, какого именно объекта. Если метод обращается к полю того самого объекта, из которого вызывается, то можно использовать сокращенную форму обращения (указав одно лишь имя объекта). Если в силу каких-то причин нам понадобилось бы явно идентифицировать объект, можно было воспользоваться ключевым словом `this`. Например, полная форма обращения к полю `name` объекта, из которого вызывается метод, могла бы выглядеть как `this->name`. К этому вопросу мы еще вернемся немного позже.

Глава 6.

ПАМЯТЬ, ДЕСТРУКТОРЫ И МАССИВЫ



- Ученый совет должен быть в полном составе!

- Кота ученого приглашать будем?
из к/ф "Чародеи"

Далее мы обсудим ряд подходов, которые очень полезны (а иногда и просто незаменимы) при составлении программ, в том числе и в рамках парадигмы ООП. Если более конкретно, то мы:

- научимся в динамическом режиме *выделять память* (в том числе динамически создавать и удалять объекты);
- узнаем, что такое *деструктор* и как он используется;
- познакомимся с *массивами*.

При этом мы предполагаем во множестве использовать *указатели*. Без них в данном случае не обойтись, поскольку на применении указателей базируются рассматриваемые далее механизмы и методики. Исключением, возможно, являются деструкторы - в том смысле, что они не имеют непосредственного отношения к указателям. Но мы задействуем указатели, когда будем иллюстрировать методы работы с деструкторами.

Как бы там ни было, несмотря на заявленную тематику главы, указателей будет много, появляются они в разных местах программного кода и в разных, так сказать, "ипостасях". Начнем с *динамического выделения памяти*.

6.1. Динамическое выделение памяти

- Что за вздор. Как Вам это в голову взбрело?
- Да не взбрело бы, но факты, как говорится,
упрямая вещь.

из к/ф "Чародеи"

Ранее мы имели дело со *статическими переменными*. Это самые обычные переменные. Термин *статические* используется в том смысле, что память под такие переменные выделяется на этапе компиляции программы. Существует и иной способ выделения памяти - *динамический*. При динамическом выделении памяти она распределяется не на этапе компиляции, а уже в процессе выполнения программы.

Для выделения памяти в динамическом режиме используют оператор `new`. После оператора `new` указывается тип переменной, для которой выделяется память. При выполнении такой инструкции под переменную выделяется память, а результатом возвращается указатель на область памяти, которая выделена. Ниже приведен пример объявления указателя `p` на значение целочисленного типа, а затем в указатель записывается адрес области памяти, которая выделяется динамически:

```
int *p;
p=new int;
```

В результате выполнения команд в памяти выделяется место под значение типа `int`, и адрес области памяти записывается в указатель `p`. Теперь к данной области памяти можно обращаться через указатель `p`. В частности, чтобы получить доступ к значению, записанному по соответствующему адресу, используем инструкцию `*p`. Откровенно говоря, это не всегда удобно, а потому для переменных базовых типов память динамически выделяют не очень часто. Другое дело, если речь идет об объектах. Здесь принцип выделения памяти точно такой же, только в качестве типа указывается имя класса для объекта, а результатом возвращается указатель на объект.

Подробности

Если быть более точным, то при динамическом создании объекта после оператора `new` указывается конструктор класса, на основе которого динамически создается объект. В круглых скобках после имени класса передаются аргументы конструктора. Если конструктор без аргументов, то пустые круглые скобки можно не использовать.

Например, если необходимо динамически выделить память под объект класса `MyClass` и записать адрес этого объекта в указатель `pnt`, то соответствующий блок программного кода мог бы выглядеть так:

```
MyClass *pnt;
pnt=new MyClass;
```

Как мы знаем, при создании объектов вызывается конструктор. Имя класса, указанное после инструкции `new` фактически представляет собой вызов конструктора. В случае, когда при создании объекта конструктору необходимо передавать аргументы, они указываются после имени класса, то есть используется команда вида `указатель=new класс (аргументы)`.

Если память под объект (или обычную переменную) выделялась динамически, то по завершении использования переменной или объекта выделенную

память принято освобождать (что фактически эквивалентно удалению переменной или объекта). Для удаления переменной или объекта (освобождения динамически выделенной памяти) используют оператор `delete`. После оператора `delete` следует указатель на область памяти, которая освобождается. Например, если для целочисленного указателя `p` память выделялась командой `p=new int`, то освободить эту область памяти (фактически удалить соответствующую переменную) можно командой `delete p`. Если речь идет об объекте класса, то процедура освобождения памяти такая же: после того, как память выделена командой `pnt=new MyClass` (динамическое выделение памяти под объект класса `MyClass` и запись адреса объекта в указатель `pnt`), она освобождается командой `delete pnt`.



На заметку

Память освобождается после того, как потребность в переменной или объекте пропадает. Делается это в целях экономии, поскольку если для работы программы памяти выделять нужно много, то она, как ни странно, может закончиться. С другой стороны, если переменную (или объект) из памяти не удалить, то ничего страшного, скорее всего, не произойдет. Поэтому вопрос удаления переменных и объектов (по завершении их использования) в известном смысле относится к правилам хорошего тона в программировании.

Небольшой пример для иллюстрации динамического выделения памяти при создании объекта представлен в листинге 6.1.

Листинг 6.1. Динамическое выделение памяти для объекта

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
    string name; // Текстовое поле
    // Конструктор:
    MyClass(string txt){
        name=txt; // Значение текстового поля
    }
    // Метод для отображения текстового поля:
    void show(){
        cout<<"Поле name: "<<name<<endl;
    }
};
// Главная функция программы:
int main(){
    MyClass *pnt; // Указатель на объект
```

```
// Динамическое создание объекта:
pnt=new MyClass("динамический объект");
// Вызов метода объекта через указатель на объект:
pnt->show();
// Удаление динамического объекта:
delete pnt;
return 0;
}
```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 6.1)

Поле name: динамический объект

В программе объявляется класс `MyClass`, в котором имеется текстовое поле, конструктор с одним аргументом, а также метод для отображения значения текстового поля. В главной функции программы объявляется указатель `pnt` на объект класса `MyClass`, а затем командой `pnt=new MyClass("динамический объект")` создается динамический объект: динамически выделяется память для объекта, вызывается конструктор класса с аргументом "динамический объект" и адрес объекта записывается в указатель `pnt`. После того, как указателю `pnt` присвоено значение, командой `pnt->show()` из объекта, определяемого указателем `pnt`, вызывается метод `show()`. Затем, перед завершением работы программы, командой `delete pnt` выполняется удаление объекта (память, выделенная под объект, помечается как неиспользуемая, вследствие чего она может быть выделена под другой объект). Что касается "внешних эффектов", то удаление объекта проходит незаметно. Во всяком случае, если из программного кода убрать команду удаления объекта, то на видимом результате выполнения программы это никак не скажется. Тем не менее, удаление объекта из памяти - процесс важный и нетривиальный. Ну, или, по крайней мере, мы можем сделать его нетривиальным. В этом нам поможет *деструктор*.

6.2. Деструктор

*Это экспонаты. Отходы, так сказать, магического производства.
из к/ф "Чародеи"*

Если коротко, то *деструктор* - это метод, который автоматически вызывается при удалении объекта. Деструктор, как и конструктор, можно описать в классе. До этого мы с деструкторами дела не имели и в классах их не описывали. Поэтому удаление объектов происходило тихо и незаметно. Но

если описать деструктор в классе, то при удалении объекта могут выполняться некоторые дополнительные действия.



На заметку

Деструктор вызывается при удалении не только динамических, но и статических (то есть таких, что создаются стандартными методами - не динамически) объектов. Статические объекты, если они созданы в главной функции программы, удаляются при завершении выполнения программы. Внутренние (локальные) объекты, созданные при вызове методов и функций, удаляются из памяти при завершении выполнения методов и функций.

Деструктор в классе описывается как обычный метод, но он должен удовлетворять некоторым критериям. Вот они:

- Название деструктора совпадает с именем класса с тильдой (символ ~) в начале. Например, если класс называется `MyClass`, то название деструктора будет `~MyClass`.
- Деструктор не возвращает результат, и идентификатор типа результата для него не указывается (все, как и в случае с конструктором).
- У деструктора нет аргументов. Это простое обстоятельство имеет важное последствие - деструктор не перегружается. Другими словами, деструктор в классе может быть только один.

Небольшой пример, в котором используется деструктор, представлен в программном коде из листинга 6.2.

Листинг 6.2. Деструктор

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
string name; // Текстовое поле
// Конструктор:
MyClass(string txt){
name=txt; // Значение текстового поля
cout<<"Создание объекта с полем name: "<<name<<endl;
}
// Деструктор
~MyClass(){
cout<<"Удаление объекта с полем name: "<<name<<endl;
}
```

```
};
// Главная функция программы:
int main() {
    cout<<"Начало выполнения программы.\n";
    MyClass *pnt; // Указатель на объект
    // Динамическое создание объекта:
    pnt=new MyClass("динамический объект");
    cout<<"Объект создан. Теперь объект удаляется.\n";
    // Удаление динамического объекта:
    delete pnt;
    cout<<"Завершение выполнения программы.\n";
    return 0;
}
```

В результате выполнения программы получаем такое:

Результат выполнения программы (из листинга 6.2)

Начало выполнения программы.
 Создание объекта с полем name: динамический объект
 Объект создан. Теперь объект удаляется.
 Удаление объекта с полем name: динамический объект
 Завершение выполнения программы.

В этом примере описывается класс `MyClass`, в котором есть текстовое поле `name`, конструктор и деструктор. У конструктора один текстовый аргумент, которым определяется значение поля `name`. Также при вызове конструктора отображается сообщение о создании объекта и значение поля `name` этого объекта.

Деструктор содержит всего одну команду `cout<<"Удаление объекта с полем name: "<<name<<endl`, которой отображается сообщение об удалении объекта и значение поля `name` удаляемого объекта.

В главной функции программы объявляется указатель `pnt` на объект класса `MyClass`. Сам объект создается динамически командой `pnt=new MyClass("динамический объект")`. При этом автоматически вызывается конструктор, и, как следствие, появляется сообщение о создании объекта. Перед завершением выполнения программы объект из памяти удаляется командой `delete pnt`, и при автоматическом вызове деструктора появляется сообщение об удалении объекта.

Как уже отмечалось, деструктор вызывается при удалении не только динамических, но и статических объектов. Правда, со статическими объектами не всегда просто определить, когда именно они будут удаляться из памяти,

хотя некоторые "рецепты" на этот счет имеются. Скажем, локальные объекты (те, что создавались в теле метода при его вызове) существуют до тех пор, пока выполняется код метода. Это же замечание, по большому счету, относится и к объектам, созданным в главной функции программы. Перед завершением программы все созданные объекты из памяти будут удалены. И каждый раз будет вызываться деструктор. Небольшой пример, в котором иллюстрируется процесс выполнения деструктора при удалении объектов, представлен в листинге 6.3. В этом примере используется класс с деструктором и несколько объектов, созданных "при разных обстоятельствах". В частности, имеется динамический объект, статический объект (созданный в главной функции), а также локальный статический объект (создается при вызове метода). При удалении каждого из этих объектов вызывается деструктор.

Листинг 6.3. Деструктор и различные объекты

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
    string name; // Текстовое поле
    // Конструктор класса:
    MyClass(string txt){
        name=txt; // Присваивание значения полю
        // Отображение значения поля:
        cout<<"Создан объект \\"<name<<"\n";
    }
    // Деструктор:
    ~MyClass(){
        // Сообщение об удалении объекта:
        cout<<"Удален объект \\"<name<<"\n";
    }
    // Метод с созданием локального объекта:
    void newObj(){
        // Сообщение о начале выполнения метода:
        cout<<"Выполняется метод newObj().\n";
        // Создание локального объекта:
        MyClass tmp("Локальный Объект");
        // Сообщение о завершении выполнения метода:
        cout<<"Завершение выполнения метода newObj().\n";
    }
};
```



```
// Главная функция программы:
int main() {
    cout<<"Начало выполнения программы.\n";
    // Создание статического объекта:
    MyClass obj("Статический Объект");
    // Создание динамического объекта:
    MyClass *pnt=new MyClass("Динамический Объект");
    // Вызов метода из динамического объекта:
    pnt->newObj();
    // Удаление динамического объекта:
    delete pnt;
    // Вызов метода из статического объекта:
    obj.newObj();
    cout<<"Завершение выполнения программы.\n";
    return 0;
}
```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 6.3)

```
Начало выполнения программы.
Создан объект "Статический Объект".
Создан объект "Динамический Объект".
Выполняется метод newObj().
Создан объект "Локальный Объект".
Завершение выполнения метода newObj().
Удален объект "Локальный Объект".
Удален объект "Динамический Объект".
Выполняется метод newObj().
Создан объект "Локальный Объект".
Завершение выполнения метода newObj().
Удален объект "Локальный Объект".
Завершение выполнения программы.
Удален объект "Статический Объект".
```

В программе мы традиционно создаем класс `MyClass` с текстовым полем `name`, методом `newObj()`, конструктором и деструктором. В конструкторе присваивается значение текстовому полю и выводится сообщение о создании объекта (с указанием значения текстового поля). При вызове деструктора отображается сообщение об удалении объекта (и также указывается значение текстового поля этого объекта). Здесь ситуация для нас вполне знакомая. Новшество связано с тем, что в классе описан метод `newObj()`, который не возвращает результат, и у которого нет аргументов. В теле мето-

да командой `MyClass tmp("Локальный Объект")` создается локальный объект. Перед созданием объекта выводится сообщение о начале выполнения метода, а после создания объекта выводится сообщение о завершении выполнения метода. Собственно, это все выполняемые при вызове метода действия.

В главной функции программы сначала отображается сообщение о начале выполнения программы. После этого командой `MyClass obj("Статический Объект")` создается статический объект. При этом вызывается конструктор и в консольном окне появляется сообщение `Создан объект "Статический Объект" ..` Затем командой `MyClass *pnt=new MyClass("Динамический Объект")` объявляется указатель `pnt`, создается динамический объект, и адрес этого объекта записывается в указатель. При динамическом создании объекта вызывается конструктор и появляется сообщение `Создан объект "Динамический Объект" ..` При вызове метода `newObj()` из объекта, на который ссылается указатель `pnt`, происходит следующее:

- отображается сообщение `Выполняется метод newObj() .;`
- при создании локального объекта вызывается конструктор, которым выводится сообщение `Создан объект "Локальный Объект" .;`
- затем отображается сообщение `Завершение выполнения метода newObj() .` о завершении метода - причем сообщение появляется при выполнении формально последней команды в теле метода.

При завершении выполнения метода `newObj()` (после того, как выполнены все команды в теле метода) из памяти выгружаются локальные переменные и, в частности, удаляется локальный объект, созданный в теле метода. В этом случае вызывается деструктор. Результатом упомянутых процессов является сообщение `Удален объект "Локальный Объект" ..`

Динамически созданный объект удаляется из памяти командой `delete pnt`. Следствие вызова деструктора при выполнении данной команды - сообщение `Удален объект "Динамический Объект" .` в консольном окне.

После удаления динамического объекта, метод `newObj()` вызывается из статического объекта `obj`. Ситуация аналогична той, что имела место при вызове метода `newObj()` из динамического объекта: появляются сообщения:

Выполняется метод `newObj()`. Создан объект "Локальный Объект"..
 Завершение выполнения метода `newObj()`.
 Удален объект "Локальный Объект"..

Последней командой в функции `main()` отображается сообщение `Завершение выполнения программы..` Но кроме этого сообщения еще выводится сообщение `Удален объект "Статический Объект"..` Дело в том, что при завершении программы из памяти выгружаются все созданные объекты. В данном случае был создан статический объект `obj`. Перед тем, как программа завершает работу, объект удаляется из памяти. Указанное сообщение появляется при вызове деструктора для объекта.

В обоих рассмотренных примерах деструкторы играли роль скорее декоративную. В реальности спектр возможностей при использовании деструкторов намного шире, чем отображение сообщений. Об этом следующий пример.

В программном коде, представленном в листинге 6.4, создается своеобразная структура из объектов, которую можно было бы назвать "цепочкой" объектов. Здесь важны два момента. Во-первых, класс `MyClass`, на основе которого создаются объекты, содержит два поля: целочисленное поле `code`, и поле `next`, представляющее собой указатель на объект класса `MyClass`. Таким образом, один объект класса `MyClass` может "ссылаться" на другой объект того же класса. Получается своеобразная "цепочка" объектов, в которой первый объект в своем поле `next` содержит адрес второго объекта, а тот, соответственно, в своем уже поле `next` содержит адрес третьего объекта, и далее по цепочке. Последний объект ни на что не ссылается (значение поля `next` последнего объекта будет равно `NULL`).

Во-вторых, программный код организован так, что у нас есть прямой доступ только к первому объекту в цепочке. Теоретически доступ к прочим объектам можно было бы получить, последовательно считывая адреса объектов, записанные в полях `next`. Но нас интересует не вопрос доступа к объектам, а вопрос их удаления. Все объекты создаются динамически. Поэтому по завершении использования объектов их надо удалить. Проблема в том, что если мы удалим первый объект в цепочке, то формально потеряем доступ к прочим объектам. Выход найдем в использовании деструктора: мы определим деструктор так, что при удалении объекта автоматически удаляется и объект, адрес которого записан в поле `next`.

Листинг 6.4. Цепочка объектов

```
#include <iostream>
using namespace std;
// Класс:
```

```

class MyClass{
private:
int code; // Закрытое числовое поле
public:
MyClass *next; // Открытое поле - указатель
    // Конструктор класса:
MyClass(int m){
code=m; // Присваивание значения полю
cout<<"Создан объект №"<<code<<endl;
}
    // Деструктор:
~MyClass(){
if(next!=NULL){ // Если ссылка не пустая
delete next; // Удаление объекта по указателю
}
cout<<"Удаляется объект №"<<code<<endl;
}
};
// Главная функция программы:
int main(){
cout<<"Начинаем создавать объекты.\n";
    // Количество создаваемых объектов:
intn=10;
    // Создание первого объекта:
MyClass *first=newMyClass(1);
    // Переменная-указатель на объект:
MyClass *p;
    // Начальное значение указателя - адрес
    // первого объекта:
p=first;
    // Создание цепочки объектов:
for(inti=1;i<n;i++){
    // Создание нового объекта:
p->next=new MyClass(i+1);
    // Переопределение ссылки на объект:
p=p->next;
}
    // Пустая ссылка в последнем объекте:
p->next=NULL;
cout<<"Удаляемобъект first:\n";
// Удаление первого объекта:
delete first;
cout<<"Все объекты удалены.\n";
return 0;
}

```

Результат выполнения программы выглядит так:

Результат выполнения программы (из листинга 6.4)

```
Начинаем создавать объекты.
Создан объект №1
Создан объект №2
Создан объект №3
Создан объект №4
Создан объект №5
Создан объект №6
Создан объект №7
Создан объект №8
Создан объект №9
Создан объект №10
Удаляем объект first:
Удаляется объект №10
Удаляется объект №9
Удаляется объект №8
Удаляется объект №7
Удаляется объект №6
Удаляется объект №5
Удаляется объект №4
Удаляется объект №3
Удаляется объект №2
Удаляется объект №1
Все объекты удалены.
```

В классе `MyClass` описывается закрытое целочисленное поле `code`. Этому полю присваивается значение при вызове конструктора класса. Также при вызове конструктора (после того, как присвоено значение полю `code`) отображается сообщение о создании объекта с указанием значения поля `code` этого объекта. Еще у класса есть поле `next`, которое является указателем на объект класса `MyClass`.

В деструкторе класса `MyClass` две команды: условный оператор и инструкция отображения сообщения об удалении объекта и значении поля `code` этого объекта. В условном операторе проверяется выражение `next != NULL`, которое возвращает значение `true`, если значение поля-указателя `next` удаляемого объекта не является пустым.



На заметку

Выражение `NULL` означает "пустое значение" или "отсутствие значения". В данном случае поле `next` объекта в цепочке содержит адрес следующего объекта. Но последнему объекту в цепочке не на что ссылаться, поэтому в поле `next` этого

объекта будет записано значение `NULL`. Условие `next!=NULL` фактически является проверкой того, не является ли объект последним в цепочке.

Вместо выражения `NULL`, в соответствии с соглашением о нулевом указателе, можно было бы использовать нулевое значение (то есть использовать команды `next!=0` и `p->next=0` вместо `next!=NULL` и `p->next=NULL` соответственно). В определенном смысле так было бы даже лучше.

Если поле `next` не пустое, то командой `delete next` удаляется объект, адрес которого записан в указателе `next`.

Подробности

Здесь есть важный концептуальный момент. Связан он с тем, как удаляются объекты в цепочке. А именно, если удаляется какой-то объект, то, естественно, вызывается деструктор. Но в этом деструкторе есть команда удаления объекта, адрес которого записан в указателе `next`. Поэтому прежде, чем будет удален текущий объект, должен быть удален объект, на который он ссылается через поле `next`. Но при удалении указанного объекта вызывается деструктор, в котором есть команда удалить следующий объект, и так далее - вплоть до последнего объекта в цепочке. Последний объект удаляется сразу, поскольку его поле `next` содержит значение `NULL` и в этом случае команда в условном операторе в деструкторе не выполняется. Другими словами, если мы попытаемся удалить объект в цепочке, то будут автоматически удалены все объекты до конца цепочки. Причем удаляются они в обратном порядке: сначала последний объект в цепочке, затем предпоследний, затем тот, что перед ним, ну и так до самого первого объекта (того, что мы пытаемся удалить явно).

В главной функции программы объявляется целочисленная переменная `n`, которая определяет количество объектов в цепочке. Затем командой `MyClass *first=new MyClass(1)` динамически создается первый объект (с полем `code` равным 1). Адрес этого объекта записывается в указатель `first`.



На заметку

В данном случае мы объединили в одну команду инструкцию объявления указателя и инструкцию создания объекта (с присваиванием адреса объекта указателю).

Также объявляется указатель `p` на объект класса `MyClass`. Этот указатель нам понадобится при создании цепочки объектов (в операторе цикла). Начальное значение указателя `p` такое же, как и значение указателя `first` (другими словами, вначале оба указателя содержат адрес одного и того же объекта - первого в цепочке объектов). Для создания цепочки объектов запускается оператор цикла, в котором индексная переменная `i` пробегает значения от 1 до `n-1` включительно. За каждый цикл командой `p->next=new MyClass(i+1)` динамически создается новый объект со

значением поля `code` равным `i+1` (при этом автоматически вызывается конструктор класса и появляется сообщение о создании объекта с соответствующим полем `code`). Адрес объекта записывается в поле `next` того объекта, на который в данный момент ссылается указатель `p`. Командой `p=p->next` указателю `p` присваивается новое значение: это записанный в поле `next` адрес объекта, который был создан предыдущей командой.

Как все это работает? На первой итерации указатель `p` содержит адрес первого объекта (на который ссылается указатель `first`). В поле `next` этого объекта записывается адрес нового объекта, созданного со значением 2 для поля `code` (значение выражения `i+1` на первой итерации). Затем адрес данного объекта заносится в указатель `p`. На этом первая итерация заканчивается. На второй итерации происходят аналогичные события, но только теперь "отправной точкой" является второй объект в цепочке (тот объект, что был создан на предыдущей итерации). На заключительной итерации создается последний объект в цепочке. Переменная-указатель `p` содержит адрес данного объекта. Но вот значение полю `next` объекта не присваивается. Посему после оператора цикла командой `p->next=NULL` эта досадная оплошность устраняется. Означенной командой завершается процесс создания цепочки объектов. Удаление объектов "стартует" благодаря команде `delete first`.

6.3. Знакомство с массивами

*Очень убедительно. Мы подумаем, к кому это применить.
из к/ф "31 июня"*

Массив - это набор элементов (переменных, объектов), объединенных общим именем и идентифицируемых с помощью этого имени и индекса (или индексов). В зависимости от того, как для массива выделяется память, различают массивы *статические* и *динамические*. Размер (количество элементов) при создании статического массива должен быть известен на момент компиляции программы. Размер динамического массива может быть определен в процессе выполнения программы (но, естественно, до того, как массив объявлен).

Мы начнем с динамических массивов. Затем постепенно перейдем к обсуждению статических массивов. Самый простой тип массивов - *одномерные* массивы. В таких массивах для идентификации элементов достаточно одного индекса. Одномерные массивы удобно представлять в виде цепочки из элементов определенного типа (это может быть как базовый тип, так и класс - в последнем случае речь идет о массиве объектов).

Для создания динамического массива используется оператор `new`, после которого указывается тип элементов массива, а в квадратных скобках - размер массива. Результатом выполнения команды является адрес первого элемента массива. Данный адрес можно записать в указатель соответствующего типа.



На заметку

Динамический массив создается практически так же, как и динамическая переменная, только в случае массива нужно указать количество элементов, для которых выделяется память (для одной переменной этого делать не нужно).

Подробности

Для создания массива объектов необходимо, чтобы в классе, на основе которого создаются объекты, имелся конструктор без аргументов. Например, представим, что нам нужно создать динамический массив `objs` из 10-ти объектов класса `MyClass`. Команда, которой создается такой массив, могла бы выглядеть как `MyClass *objs=new MyClass[10]`. Фактически в данном случае после оператора `new` указано название класса (без аргументов в круглых скобках), что означает создание объекта вызовом конструктора без аргументов. Поэтому необходимо, чтобы такой конструктор существовал. Данное замечание в равной степени относится и к статическим массивам объектов.

Есть два важных обстоятельства:

- как уже отмечалось ранее, указатели в C++ индексируются;
- место в памяти для элементов массива выделяется последовательно, так что элементы массива в памяти размещаются один за другим.

Все это позволяет успешно использовать в качестве имени массива имя переменной-указателя на первый элемент массива. Далее под именем динамического массива мы будем подразумевать переменную-указатель, в которую записан адрес первого элемента массива.

Шаблон команды создания динамического массива такой:

```
тип *указатель=new тип[размер];
```

Данной командой создается массив элементов, каждый из которых имеет указанный тип, а количество элементов определяется параметром `размер`. Адрес первого элемента в массиве записывается в переменную указатель (переменную указатель отождествляем с именем динамического массива). Например, следующей командой создается динамический целочисленный (каждый элемент относится к типу `int`) массив `nums` из 20 элементов:


```
int *nums=new int[20];
```

Обращение к элементам массива выполняется просто: необходимо указать имя массива, а после имени массива в квадратных скобках - индекс элемента массива. Индексация элементов начинается с нуля - то есть первый элемент массива имеет индекс 0. Индекс последнего элемента в массиве на единицу меньше количества элементов в массиве. Скажем, если массив `nums` состоит из 20-ти элементов, то первый элемент массива - это `nums[0]`, а последний элемент массива - это `nums[19]`.



На заметку

В C++ нет автоматического контроля выхода за пределы массива. Поэтому задача "не выскочить" из массива всецело ложится на плечи программиста. В известном смысле все логично, поскольку на самом деле обращаясь к элементам массива, мы фактически индексируем указатель. Базовое положение, которое делает тождественным имя массива и имя указателя на первый элемент массива - это то, что выделяемые для массива ячейки следуют подряд одна за другой. Индексируя указатель мы, тем самым, перебираем ячейки массива.

Здесь нелишним будет напомнить правило индексирования указателей: если после имени указателя в квадратных скобках указать целое число, то результатом такого выражения будет значение ячейки, которая отстоит от ячейки, определяемой указателем, на количество позиций, определяемых значением индекса.

После того, как динамический массив в программе больше не нужен, он удаляется из памяти. Для этого используют оператор `delete`, после которого указывают пустые квадратные скобки и имя переменной-указателя на первый элемент массива. Шаблон команды удаления динамического массива, адрес первого элемента которого записан в `указатель`, выглядит так:

```
delete [] указатель;
```

Так, если удаляется массив `nums`, то команда выглядит следующим образом:

```
delete [] nums;
```

В результате удаляется не только ячейка, на которую ссылается указатель `nums`, но и все остальные ячейки, зарезервированные под соответствующий массив.

**На заметку**

Если вместо команды `delete [] nums` воспользоваться командой `delete nums`, будет удален только первый элемент массива. Остальные ячейки массива продолжат находиться "в резерве". В принципе ничего катастрофического здесь нет, но соответствующая память, скорее всего, "выйдет из игры" на время выполнения программы. А это не очень разумно.

Небольшой пример создания и использования динамического числового массива представлен в листинге 6.5. В программе реализуется класс, в котором "спрятан" массив с числами Фибоначчи (первых два числа в последовательности равны единице, а каждое следующее вычисляется как сумма двух предыдущих). Массив динамический. Для реализации массива использованы два закрытых поля - целое число и указатель на целое число. В целочисленное поле при создании объекта записывается значение, определяющее количество элементов в массиве. В поле-указатель записывается адрес первого элемента в массиве.

**На заметку**

Вообще, для однозначной "идентификации" массива (статического или динамического) нужны два параметра. Во-первых, потребуется "точка входа" (например, адрес первого элемента массива). Во-вторых, понадобится количество элементов в массиве (или параметр, позволяющий определить количество элементов в массиве).

Массив создается и заполняется в конструкторе. Также в классе описан метод, которым отображается содержимое массива. При вызове деструктора выводится сообщение об удалении объекта с указанием количества элементов в массиве и собственно удаляется сам динамический массив.

Подробности

В программе встречается ключевое слово `this`. Вообще инструкция `this` используется в программных кодах методов класса, и является указателем на объект, из которого вызывается метод.

Рассмотрим программный код примера:

Листинг 6.5. Числовой массив

```
#include <iostream>
using namespace std;
// Класс с полем-указателем (для создания
// динамического массива):
class MyClass{
private:    // Закрытые члены класса
```

```

int n;          // Целочисленное поле
int *fibs;      // Поле-указатель (динамический массив)
public:         // Открытые члены класса
    // Конструктор:
MyClass(int n){
    // Присваивание значения целочисленному полю:
    if(n<3){ // Проверка значения аргумента конструктора
        this->n=3; // Использован указатель this
    }
    else{
        // Слева - ссылка на поле класса,
        // справа - аргумент конструктора:
        this->n=n;
    }
    // Создание динамического массива:
    fibs=new int[this->n];
    // Первый элемент массива:
    fibs[0]=1;
    // Второй элемент массива:
    fibs[1]=1;
    // Заполнение прочих элементов массива:
    for(int i=2;i<this->n;i++){
        // Значение элемента массива:
        fibs[i]=fibs[i-1]+fibs[i-2];
    }
    // Деструктор:
    ~MyClass(){
        cout<<"Удаляется объект. ";
        cout<<"Количество элементов: "<<n<<endl;
        delete [] fibs; // Удаление динамического массива
    }
    // Метод для отображения содержимого массива:
    void show(){
        for(int i=0;i<n;i++){
            // Отображение значения элемента массива:
            cout<<fibs[i]<<" ";
        }
        // Переход к новой строке:
        cout<<endl;
    }
};

int main(){
    // Создание двух объектов:
    MyClass objA(15),objB(-5);
    // Отображение содержимого массивов, "спрятанных"

```

```
// в объектах:
objA.show();
objB.show();
return 0;
}
```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 6.5)

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610
1 1 2
Удаляется объект. Количество элементов: 3
Удаляется объект. Количество элементов: 15
```

В этом примере описан класс `MyClass`, у которого два закрытых поля: поле `n` типа `int` и указатель `fibs` на целочисленное значение. Поле-указатель отождествляем с именем массива. Поскольку оба поля закрытые, то доступ к ним есть только из тела класса. Все самое интересное происходит в конструкторе. У конструктора один аргумент, который определяет размер создаваемого массива. В самом начале кода конструктора в условном операторе проверяется значение аргумента: если оно меньше 3, то создается массив из трех элементов. В противном случае количество элементов создаваемого массива определяется аргументом конструктора. Адрес первого элемента созданного таким образом динамического массива записывается в переменную `fibs`.

Подробности

В программном коде название аргумента конструктора совпадает с названием поля `n`. Аргумент конструктора "имеет силу" локальной переменной. Действует такое правило: если имя локальной переменной совпадает с названием поля (или глобальной переменной), то локальная переменная "перекрывает" поле и обращение по имени переменной означает обращение именно к локальной переменной. Поэтому в теле конструктора имя `n` означает аргумент конструктора. Чтобы получить доступ к одноименному полю класса, необходимо использовать команду с явным указанием объекта вызова. В этом случае используем указатель `this` на объект, из которого вызывается метод - в случае конструктора подразумевается создаваемый объект. Скажем, инструкция обращения к полю `n` объекта выглядит как `this->n`. Именно ее мы используем в теле конструктора, когда необходимо прочитать значение поля `n` объекта или присвоить этому полю значение.

А вот в методе `show()` для обращения к полю `n` объекта инструкцию `this->n` использовать необязательно, достаточно просто указать имя поля `n`. Причина проста: поскольку в методе `show()` локальной переменной с

именем `n` нет, то имя `n` означает соответствующее поле объекта, из которого вызывается метод `show()`.

Поскольку в созданном массиве не меньше трех элементов, то в конструкторе явно первым двум элементам массива присваиваются единичные значения, после чего запускается оператор цикла, в котором вычисляются все последующие элементы динамического массива.

Метод `show()` предназначен для отображения элементов массива. В теле метода запускается оператор цикла. В нем индексная переменная перебирает элементы массива `fib`s, а верхняя граница диапазона изменения индексной переменной определяется значением поля `n` (напомним, это количество элементов в массиве, а индекс последнего элемента на единицу меньше).

Деструктор содержит несколько команд, которыми отображается сообщение об удалении объекта. В этом сообщении указано, сколько элементов содержит удаляемый объект (имеется в виду количество элементов в массиве удаляемого объекта). Для удаления самого массива использована команда `delete [] fibs`.

В главной функции создается два объекта класса `MyClass`. Для одного объекта аргументом конструктора указано значение `15`, а для другого аргументом указано значение `-5`. Поэтому в первом объекте "спрятан" массив из `15` элементов, а во втором - массив из `3` элементов. Содержимое этих массивов видим, когда вызываем из объектов метод `show()`. По завершении выполнения программы созданные объекты выгружаются из памяти. Для каждого удаляемого объекта вызывается деструктор. При выполнении деструкторов появляются сообщения об удалении объектов. Несложно заметить, что объекты из памяти удаляются в обратном порядке к тому, как они создавались (то есть первым удаляется объект, который создавался последним).

Другой пример создания массива, - на этот раз массива объектов, - представлен в листинге 6.6. В данном случае мы все так же имеем дело с последовательностью Фибоначчи, но несколько изменили подход: мы описываем класс `MyClass` с целочисленным полем `num`. Для запоминания последовательности чисел Фибоначчи создается массив объектов, в числовое поле каждого такого объекта записывается число из последовательности. Поле `num` описано как закрытое, поэтому для присваивания значения полю и считывания значения поля в классе `MyClass` создаются открытые методы `set()` и `get()` соответственно. Также в классе описан деструктор: при удалении объекта появляется сообщение с указанием значения числового поля `num` удаляемого объекта.

Листинг 6.6. Массив объектов

```

#include<iostream>
using namespace std;
// Класс:
class MyClass{
private: // Закрытые члены
int num; // Целочисленное поле
public: // Открытые члены
    // Метод для присваивания значения полю:
void set(int num){
    // Слева - поле класса, справа - аргумент метода:
this->num=num;
}
    // Метод для считывания значения поля:
int get(){
return num; // Результат метода - значение поля
}
    // Деструктор:
~MyClass(){
cout<<"Удаляется объект с полем "<<num<<endl;
}
};
// Главная функция программы:
int main(){
    // Индексная переменная и количество
    // элементов массива:
int i,n=10;
    // Указатель на объект (имя массива):
MyClass *obj;
    // Создание динамического массива:
obj=newMyClass[n];
    // Значение поля для первого объекта в массиве:
obj[0].set(1);
    // Значение поля для второго объекта в массиве:
obj[1].set(1);
    // Присваивание значений полям объектов из массива:
for(i=2;i<n;i++){
    // Вычисление и присваивание значения полю объекта:
obj[i].set(obj[i-1].get()+obj[i-2].get());
}
    // Отображение значений полей объектов из массива:
for(i=0;i<n;i++){
cout<<obj[i].get()<<" ";
}
cout<<endl;

```

```
// Удаление массива объектов:
delete [] obj;
return 0;
}
```

В главной функции программы объявляются две переменные: индексная переменная `i` используется в операторе цикла, а переменная `n` со значением 10 задает количество объектов в создаваемом массиве. Для создания массива объектов сначала командой `MyClass *obj` объявляется указатель `obj` на объект класса `MyClass`. Затем командой `obj=new MyClass[n]` собственно создается массив из `n` объектов класса `MyClass` и адрес первого объекта в массиве записывается в переменную `obj`.

Теперь под `obj` можно смело подразумевать массив объектов. Если нам впоследствии понадобится один из объектов, входящих в массив, то после имени массива в квадратных скобках необходимо указать индекс объекта в массиве (индексация, напомним, начинается с нуля). Например, командами `obj[0].set(1)` и `obj[1].set(1)` из объектов `obj[0]` и `obj[1]` вызывается метод `set()`. Аргументом методу передается число, присваиваемое полю `num` соответствующего объекта.

Таким способом задаются два первых числа последовательности. Далее запускается оператор цикла, в котором через индекс перебираются все оставшиеся объекты в массиве `obj`. Командой `obj[i].set(obj[i-1].get()+obj[i-2].get())` полю `num` каждого из объектов присваивается значение.



На заметку

Поскольку поле `num` закрытое, то для считывания его значения мы из объекта вызываем метод `get()` (результат метода - значение поля `num` объекта, из которого вызывается метод). Чтобы присвоить значение полю `num`, вызывается метод `set()`.

После присваивания значений числовым полям объектов массива запускается оператор цикла, в котором отображаются значения этих полей. Здесь мы также используем метод `get()`. Наконец, командой `delete [] obj` массив объектов удаляется из памяти. При удалении каждого объекта массива вызывается деструктор. Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 6.6)

```
1 1 2 3 5 8 13 21 34 55
```

```

Удаляется объект с полем 55
Удаляется объект с полем 34
Удаляется объект с полем 21
Удаляется объект с полем 13
Удаляется объект с полем 8
Удаляется объект с полем 5
Удаляется объект с полем 3
Удаляется объект с полем 2
Удаляется объект с полем 1
Удаляется объект с полем 1

```

Судя по сообщениям, отображаемым деструкторами, объекты в массиве удаляются, начиная с самого последнего.



На заметку

Обращаем внимание, что при создании массива объектов каждый отдельный объект в массиве создается путем вызова конструктора без аргументов. Чтобы такое было в принципе возможно, в соответствующем классе должен быть конструктор без аргументов. Это может быть или конструктор по умолчанию (существует, если в классе явно не описан конструктор), или же конструктор без аргументов должен быть описан явно.

Выше мы использовали динамические массивы. Но кроме *динамических*, существуют еще и *статические* массивы.

6.4. Статические массивы

*Тумбочек у нас - завались, а вот по части подушек бедствуем.
из к/ф "Девчата"*

С формальной точки зрения *статический массив* мало чем отличается от динамического. Принципиальное отличие одно - размер статического массива должен быть известен на момент компиляции программы и, соответственно, не может определяться переменной (только константой). Объявляется одномерный статический массив просто: после идентификатора типа элементов массива указывается название массив и количество элементов массива (в квадратных скобках после имени массива). Шаблон объявления статического одномерного массива такой:

```
тип массив[размер];
```

Например, одномерный статический целочисленный массив `nums` из 10 элементов создается следующей командой:


```
int nums[10];
```

Статические массивы явным образом из памяти удалять не нужно. Как и в случае с динамическим массивом, имя статического массива является указателем на его первый элемент, а элементы статического массива располагаются в памяти подряд, один за другим. Поэтому методы работы со статическим массивом фактически такие же, как и с динамическим массивом.

Статические массивы объектов создаются аналогично тому, как создаются статические массивы с элементами базовых типов. Далее приведен фрагмент кода, в котором объявляется массив `obj` объектов класса `MyClass` (размер массива определяется константой `n`):

```
const int n=10;
MyClass obj[n];
```

При создании массива объектов вызывается конструктор без аргументов (у соответствующего класса такой конструктор должен быть). Небольшой пример с использованием статического массива объектов приведен в листинге 6.7. В программе описывается класс с целочисленным полем. Значением полю при создании объекта присваивается случайное число (в диапазоне значений от 0 до 99 включительно).

За присваивание значения полю "отвечает" конструктор класса. Также в теле конструктора имеется команда, которой отображается сообщение о создании объекта и отображается значение числового поля объекта. У конструктора аргументов нет. При вызове деструктора также отображается сообщение, но уже об удалении объекта. Как и при создании объекта, при удалении объекта в консоль выводится значение его числового поля.

Алгоритм выполнения программы такой:

- Создается статический массив объектов. При создании каждого из объектов значением числовому полю объекта присваивается случайное число. Сообщение о создании соответствующего объекта отображается в консольном окне.
- Запускается оператор цикла, в котором меняется значение числового поля для каждого из объектов в массиве.
- Запускается еще один оператор цикла, которым отображается новое значение числового поля объектов в массиве.

Теперь рассмотрим подробнее программный код примера:

Листинг 6.7. Статический массив объектов

```

#include <iostream>
#include <cstdlib>
using namespace std;
// Класс:
class MyClass{
public:
    // Целочисленное поле:
    int n;
    // Конструктор:
    MyClass(){
        // Присваивание значения полю:
        this->n=rand()%100;
        // Отображение сообщения:
        cout<<"Создан объект с полем "<<n<<endl;
    }
    // Деструктор:
    ~MyClass(){
        // Отображение сообщения:
        cout<<"Удаляется объект с полем "<<n<<endl;
    }
    // Метод для отображения значения поля:
    void show(){
        // Отображение сообщения:
        cout<<"Объект с полем "<<n<<endl;
    }
};
// Главная функция программы:
int main(){
    // Размер статического массива:
    const int n=5;
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Статический массив объектов:
    MyClass obj[n];
    // Указатели на объект:
    MyClass *first,*last;
    // Изменение значения полей объектов из массива:
    for(first=obj,last=obj+n;first!=last;first++){
        first->n/=10; // Новое значение поля
    }
    // Отображение значения полей объектов из массива:
    for(int i=0;i<n;i++){
        // Вызов метода из объекта - элемента массива:
        obj[i].show();
    }
}

```

```
    }
    return 0;
}
```

В конструкторе класса `MyClass` присваивание значения целочисленному полю `n` выполняется командой `this->n=rand()%100`. Здесь вызывается функция `rand()`, которой генерируются целые случайные числа (чтобы функция стала доступной, подключается заголовок `<cstdlib>`). Мы вычисляем остаток от деления сгенерированного числа на 100, поэтому в результате получаем целое число в диапазоне значений от 0 до 99. Инициализация генератора случайных чисел осуществляется командой `srand(2014)` в главной функции программы.

Подробности

Функция `rand()` предназначена для генерирования равномерно распределенных целых случайных чисел. Функции `srand()` аргументом передается число, которое используется для запуска процесса генерирования чисел. Другими словами, для генерирования случайных чисел необходимо задать некоторое числовое значение (какое именно - неважно). Важно другое: для одного и того же "затравочного" значения получаем одну и ту же последовательность случайных чисел.

В принципе, можно не вызывать функцию `srand()`. Тогда число для инициализации генератора случайных чисел выбирается автоматически без участия пользователя/программиста.

Размер статического массива объектов, который создается командой `MyClass obj[n]`, задается константой `n` (в примере значение константы равно 5).

В главной функции объявлены два указателя (`first` и `last`) на объект класса `MyClass`. Эти указатели мы используем в операторе цикла, в котором изменяется значение числового поля объектов, формирующих массив. Для перебора объектов в массиве мы поступаем следующим образом:

- В блоке инициализации оператора цикла командами `first=obj` и `last=obj+n` указателям `first` и `last` присваиваются значения. Указатель `first` в результате содержит адрес первого элемента массива, а указатель `last` содержит адрес ячейки, которая размещается после последнего элемента массива (то есть значение указателя `last` - это адрес *первой не входящей в массив* ячейки). При вычислении значения для указателя `last` мы воспользовались тем, что имя массива `obj` является указателем на первый элемент массива, и учли правила адресной арифметики (правило прибавления к указателю целого числа).

- Оператор цикла выполняется до тех пор, пока истинно условие `first!=last`. Условие означает, что адрес в указателе `first` отличается от адреса в указателе `last`. Поскольку в третьем блоке оператора цикла командой `first++` за каждую итерацию значение указателя `first` перебрасывается на ячейку "справа" (в "направлении" увеличения адреса) от текущей, то условие станет ложным, когда указатель `first` будет содержать адрес ячейки за пределами массива. А это означает, что все элементы массива перебраны.
- В теле оператора цикла командой `first->n/=10` полю `n` объекта, адрес которого записан в переменную `first`, присваивается новое значение: текущее значение поля делится нацело на 10, и полученный результат записывается в поле.

Подробности

Мы воспользовались одной из форм составного оператора присваивания. Речь об операторе `/=`. Выражение вида `x/=y` эквивалентно выражению `x=x/y`. Существуют и другие аналогичные формы составного оператора присваивания. Например, выражение `x+=y` эквивалентно выражению `x=x+y`, выражение `x*=y` эквивалентно выражению `x=x*y`, выражение `x%=y` эквивалентно выражению `x=x%y`, ну и так далее. Другими словами, если $\odot$ - некоторый оператор, то выражение `x $\odot$ y` означает команду `x=x $\odot$ y`.



На заметку

В главной функции есть константа `n`. У объектов класса `MyClass` есть поле `n`. И это разные "переменные". Если выполняется обращение к полю объекта, необходимо указать объект (например, через указатель на объект). Если не указать объект, то речь, очевидно, будет идти о константе `n`.

Еще на одно обстоятельство стоит обратить внимание. Касается оно указателя `first`. Начальное значение этого указателя - адрес первого элемента массива, совпадающее со значением указателя `obj` (имя массива). Фактически, мы создаем "копию" указателя на первый элемент массива. Может показаться, что такая операция лишняя. Однако все дело в том, что значение указателя `first` можно менять в процессе выполнения программы (что мы и делаем в операторе цикла). А вот изменить значение указателя `obj` не получится - это постоянный указатель (аналог константы), значение которого неизменно.

После того, как значение числового поля объектов массива изменено, запускается еще один оператор цикла, в котором перебор элементов осуществляется с помощью индексной переменной `i`. В теле этого оператора для отображения значения поля `n` объекта используется команда `obj[i].show()` - метод `show()` предназначен для отображения значения поля объекта. Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 6.7)

```

Создан объект с полем 15
Создан объект с полем 30
Создан объект с полем 15
Создан объект с полем 78
Создан объект с полем 96
Объект с полем 1
Объект с полем 3
Объект с полем 1
Объект с полем 7
Объект с полем 9
Удаляется объект с полем 9
Удаляется объект с полем 7
Удаляется объект с полем 1
Удаляется объект с полем 3
Удаляется объект с полем 1

```

Поскольку мы имеем дело со статическим массивом, то явно удалять массив из памяти не нужно. Это происходит автоматически при завершении работы программы.

На заметку

Объекты массива создаются в порядке возрастания индекса -от первого элемента до последнего. Порядок удаления объектов из массива противоположный - от последнего элемента к первому.

6.5. Символьные массивы

- Вот по этому поводу первый тост.
 - Сейчас запишу.
 - Потом запишешь.
 из к/ф "Кавказская пленница"

Символьные массивы (массивы, элементами которых являются значения типа `char`) в C++ имеют особый статус, поэтому уделим им немного внимания. Символьные массивы -основной механизм реализации текстовых значений в C++. Другими словами, базовый механизм представления текста в программе - массив из символов (букв). Несложно догадаться, что если мы текст представляем в виде массива букв, то это именно те буквы, из которых состоит текст.

Фундаментальная проблема, с которой сталкиваемся при использовании символьных массивов в качестве контейнеров для текста, связана с тем, что размер массива и размер текста, записанного в массив - в общем случае разные. Правда, на первый взгляд может показаться, что такой проблемы как бы и нет: достаточно определить размер массива по количеству символов в тексте. И так действительно можно сделать. Но пикантность ситуации в том, что нам в программе нужны не просто текстовые значения, а "переменные", в которые можно было бы записывать текст. А специфика текста такова, что при изменении текстового значения вообще-то изменяется количество символов в тексте. Поэтому нужен контейнер, размер которого был бы достаточен для записи текста различной длины (в разумных пределах, разумеется). Если для записи текста используется массив, то его размер должен быть не меньше размера текста, который предполагается записывать в массив. Это, в свою очередь, означает, что размер текста меньше размера массива. Отсюда и проблема: мы можем "запомнить" размер массива, но как быть с размером текста? Как его определить?

Для определения длины текста в символьном массиве используют специальный "маркер", который называется *нуль-символом* (не путать с нулем!). Нуль-символ обозначается как `'\0'` (считается, что это один символ) и имеет нулевой код в кодовой таблице символов. Правило такое: текст, который записывается в символьный массив, должен заканчиваться нуль-символом.

Если бы нуль-символ приходилось заносить в массив вручную, такой подход был бы крайне утомительным. Благо, обычно к этому прибегать не приходится. Все потому, что символьные массивы обрабатываются по-особому. Наиболее важные, пожалуй, два момента:

- Символьный массив можно инициализировать текстовым литералом (значение, заключенное в двойные кавычки). При инициализации символьного массива текстовым литералом буквы из литерала автоматически заносятся в инициализируемый массив. Нуль-символ в конце текста также добавляется автоматически. Например, в результате выполнения команды `char str[100]="Текст"` в массив `str` побуквенно заносится фраза "Текст" (заполняются первые пять элементов массива `str`), и в конце добавляется нуль-символ `'\0'` (шестой элемент массива `str`).
- Если справа от оператора вывода `<<` указать имя символьного массива, то будет отображен текст, записанный в этом массиве. Текст отображается до того места, где в символьном массиве записан нуль-символ. Например, при выполнении команды `cout<<str` в окне вывода отобра-

жается текстовое значение, записанное в массив `str` (в данном случае слово "Текст").

Подробности

Если быть более точным, то ситуация следующая. Если справа от оператора вывода находится указатель на значение типа `char`, то отображается не адрес, записанный в указателе, а символы, начиная с того, на который ссылается указатель, и вплоть до нуля-символа. Напомним, что имя массива является указателем на первый элемент массива. Таким образом, имя символьного массива - указатель на значение типа `char`.

Небольшая иллюстрация к использованию символьных массивов приведена в листинге 6.8.

Листинг 6.8. Символьные массивы

```
#include <iostream>
using namespace std;
int main() {
    // Символьный массив:
    char txt[100]="Символьный массив";
        // Отображение содержимого
        // символьного массива:
    cout<<txt<<endl;
        // Отображается текст,
        // начиная с 12-й буквы:
    cout<<txt+11<<endl;
        // Замена буквы 'м' на 'М':
    for(int i=0;txt[i];i++){
        if(txt[i]=='м') txt[i]='М';
    }
        // Содержимое массива:
    cout<<txt<<endl;
        // Добавляем нуль-символ в текст:
    txt[10]='\0';
        // Отображается текст:
    cout<<txt<<endl;
    cout<<txt+11<<endl;
    return 0;
}
```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 6.8)

Символьный массив
массив
СиМвольный Массив
СиМвольный
Массив

Проанализируем программный код и результаты его выполнения. В самом начале командой `char txt[100]="Символьный массив"` создается символьный массив `txt` из 100 элементов и инициализируется текстовым значением из двух слов "Символьный массив":

- первая буква текста 'С' присваивается элементу `txt[0]`;
- символ 'и' присваивается элементу `txt[1]`;
- и так далее - например, пробел ' ' (это тоже символ) присваивается элементу `txt[10]`;
- последняя буква текста 'в' записывается в элемент `txt[16]`;
- нуль-символ '\0' записывается в элемент `txt[17]`.

Указанное текстовое значение появляется в окне вывода при выполнении команды `cout<<txt<<endl`.

Результатом выполнения команды `cout<<txt+11<<endl` является текст "массив" в окне вывода. Объяснение простое: значение выражения `txt+11` есть указатель на 12-й символ в тексте (элемент `txt[11]`, или буква 'м'). Поэтому отображение текста начинается именно с этого места и пока не встретится нуль-символ (элемент `txt[17]`).

Для замены строчных (маленьких) букв 'м' в тексте на прописные (большие) буквы 'М' запускается оператор цикла, в котором индексная переменная `i` получает начальное нулевое значение и за каждый цикл ее значение увеличивается на единицу. Интерес представляет условие: это выражение `str[i]`. Здесь мы воспользовались тем, что, во-первых, нуль-символ имеет нулевой код, и, во-вторых, отличное от нуля числовое значение интерпретируется как `true`, а нулевое числовое значение интерпретируется как `false`. Значение выражения `str[i]` представляет собой символ в тексте `txt` с индексом `i`. Если это не нуль-символ, то его код отличен от нуля и `str[i]` интерпретируется как `true`. Если это нуль-символ, то `str[i]` интерпретируется как `false`. Таким образом, оператор цикла выполняется, пока не будет прочитан нуль-символ.

В теле оператора цикла есть условный оператор. Если текущий символ равен 'м' (условие `txt[i]=='м'`), то его значение меняется на 'М' (коман-

да `txt[i]='M'`). Характер внесенных в текст изменений можем проверить с помощью команды `cout<<txt<<endl`.

Следующий эксперимент состоит в том, чтобы заменить в тексте пробел (элемент `txt[10]`) на нуль-символ. Для этого выполняется команда `txt[10]='\0'`. Если после выполнения данной команды попытаться отобразить текст инструкцией `cout<<txt<<endl`, получим только первое слово. Причину понять несложно: текст отображается, начиная с первого символа, и пока не встретится нуль-символ. Но теперь нуль-символ встречается в элементе `txt[10]`! Дальше этого символа текст не отображается. Но вот если вызвать команду `cout<<txt+11<<endl`, в окне вывода увидим второе слово из текста. Объяснение практически такое же, как и в предыдущем случае: текст отображается, начиная с символа `txt[11]` до ячейки массива с нуль-символом. В данном случае это элемент `txt[17]`.

Глава 7.

ВСЕ О МАССИВАХ



*Ведь это же настоящая тайна! Ты потом
никогда себе не простишь!
из к/ф "Гостья из будущего"*

В этой главе отправной точкой для нас послужат массивы (в основном динамические). Мы обсудим, как массивы могут использоваться в качестве членов класса. Все прочие темы, нашедшие отражение в главе (среди них *перегрузка оператора []* для индексирования объектов, *перегрузка оператора присваивания* и *конструктор создания копии*), рассматриваются в контексте работы с массивами, являющимися членами класса.

7.1. Индексирование объектов

*Это великая победа дедуктивного метода.
из к/ф "Гостья из будущего"*

При одновременном использовании массивов и объектов очень полезной может быть перегрузка операторов. Мы уже знаем, как создать класс с полем-массивом (динамическим, например). При обращении к элементам такого массива мы использовали точечный синтаксис: указывали объект и, через точку, имя массива и индекс элемента. Означенный подход не всегда удобен. Перегрузив оператор `[]` (квадратные скобки) можем добиться эффекта, когда индексируется непосредственно объект. Другими словами, указав после имени объекта квадратные скобки и индекс, получим доступ к элементу массива, "спрятанного" в объекте.



На заметку

У бинарного оператора `[]` первый операнд -та переменная, что указывается перед квадратными скобками (то есть индексируемая переменная), а второй операнд -индекс, который указывается в квадратных скобках: в выражении `переменная[индекс]` первым операндом является переменная, а вторым операндом является индекс.

Реализация такого подхода представлена в программном коде из листинга 7.1. В программе описывается класс `MyClass`, с двумя закрытыми полями: поле `nums` представляет собой указатель на целочисленное значение и предназначено для создания массива (название поля можем отождествлять с названием массива), а также целочисленное поле `size`, определяющее размер массива. У конструктора класса один аргумент, определяющий размер создаваемого динамического массива. Массив заполняется случайными числами в диапазоне значений от 0 до 99.



На заметку

Для генерирования случайных чисел использована функция `rand()`. Инициали-

зация генератора случайных чисел выполняется в главной функции программы с помощью функции `srand()`. Для использования указанных функций подключается заголовок `<cstdlib>`.

Деструктор содержит всего одну команду для удаления динамического массива. Здесь ситуация нам уже знакомая. А вот перегрузка оператора

[] требует некоторых комментариев. Вот так выглядит описание соответствующего операторного метода:

```
int &operator[](int i){
    // Результат операторного метода:
    return nums[i%size];
}
```

Данный операторный метод называется `operator[]` поскольку это именно метод и описывается в классе, то первым операндом автоматически считается объект, перед которым будут размещаться квадратные скобки. Указанный в квадратных скобках индекс, описывается как аргумент операторного метода (переменная `i`). В данном случае второй аргумент (индекс) мы определяем как целое число. В теле операторного метода всего одна инструкция, возвращающая результатом метода значение элемента массива `nums` с соответствующим индексом.

Но если быть более точным, то индекс элемента немного отличается от аргумента операторного метода: при обращении к элементу массива индексом указывается остаток от деления аргумента операторного метода на размер массива (инструкция `i%size`). Благодаря такому нехитрому приему при индексировании объектов можно указывать практически любой индекс: после вычисления остатка от целочисленного деления реальный индекс, через который выполняется обращение к элементу массива, неизбежно попадает в "правильный" диапазон значений от 0 до `size-1`. Например, если при индексировании объекта указать индекс `size`, то на самом деле это будет означать обращение к элементу массива с индексом 0. Аналогично, индекс `size+1` эквивалентен индексу 1, и так далее.

Еще одно важное обстоятельство, заслуживающее внимания - наличие инструкции `&` перед названием операторного метода. Наличие инструкции `&` означает, что методом возвращается не просто значение соответствующего элемента массива, а ссылка на массив. Если бы мы хотели только считывать значения массива, то без инструкции `&` в названии операторного метода можно было обойтись. Если же нужно, чтобы через индексирование объекта можно было присваивать значение элементу массива, то операторный метод должен возвращать ссылку на элемент.

Подробности

Проще говоря, если в названии операторного метода инструкцию `&` не использовать, то прочитать значение элемента массива `nums` из объекта `obj` с индексом `i` удалось бы командой `obj[i]`, но попытка присвоить значение выражению `obj[i]` не имела бы успеха. Чтобы понять, почему так происходит, нужно иметь хотя бы общее представление о том, как методы (и функции) возвращают результат.

Если метод возвращает результат, то при вызове метода для возвращаемого методом значения выделяется в памяти место. После того, как результат метода вычислен, соответствующее значение записывается в выделенную ячейку памяти. Значение из ячейки используется в дальнейших вычислениях. Если результатом метода является значение элемента массива, то данное значение считывается и записывается в ячейку, предназначенную для результата метода. Если нам необходимо только прочитать значение элемента, то здесь особых проблем не возникнет. А вот если мы попытаемся присвоить значение элементу, то значение "записывается" в ячейку, предназначенную для результата метода, а вовсе не в ячейку, в которую записан элемент массива. Чтобы этого не происходило, метод результатом должен возвращать не просто значение элемента массива, а ссылку на элемент.

Для отображения содержимого массива `nums` в классе `MyClass` описан метод `show()`. У метода есть один целочисленный аргумент с нулевым значением по умолчанию. Аргумент метода определяет индекс, начиная с которого отображаются значения элементов. В теле метода обращение к элементу массива с индексом `i` выполняется в формате `(*this)[i]`. В данной инструкции использован перегруженный операторный метод. Поскольку `this` представляет собой указатель на объект, из которого вызывается метод, то `*this` — непосредственно объект. Следовательно, инструкция `(*this)[i]` (круглые скобки использованы для соблюдения правильного приоритета операторов `*` и `[]`) представляет собой индексирование объекта, выполняемое в соответствии с тем, как описан операторный метод `operator[]()`. Напомним, что индексирование объекта выполняется с учетом цикличности индекса. Поскольку индексная переменная `i` пробегает значения в диапазоне от значения `start` до значения `start+size-1` (включительно), то перебираются все элементы массива.

В главной функции программы:

- инициализируется генератор случайных чисел;
- создается объект `obj` класса `MyClass`;
- из объекта `obj` вызывается метод `show()` без аргументов (в результате чего отображаются значения элементов массива из объекта `obj`);

- изменяются значения двух первых элементов массива из объекта obj;
- с помощью оператора цикла отображаются значения элементов массива из объекта obj;
- с помощью метода show() (с ненулевым аргументом) значения элементов массива из объекта obj циклически отображаются (начиная с середины массива).

Весь программный код выглядит так:

Листинг 7.1. Перегрузка оператора []

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Класс:
class MyClass{
private:    // Закрытые поля
int size;  // Размер массива
int *nums; // Имя массива
public:    // Открытые члены класса
    // Конструктор:
MyClass(int n){
    size=n; // Размермассива
    nums=new int[size]; // Создаетсямассив
    for(int i=0;i<size;i++){
        // Значение элемента массива:
        nums[i]=rand()%100;
    }
}
    // Деструктор:
~MyClass(){
    // Удаление динамического массива:
    delete [] nums;
}
    // Перегрузка оператора [] для
    // обращения к элементу массива:
int &operator[](int i){
    // Результат операторного метода:
    return nums[i%size];
}
    // Метод для отображения содержимого массива:
void show(int start=0){
    for(int i=start;i<start+size;i++){
        // При обращении к элементу массива использован
```

```

        // перегруженный оператор [] и указатель this:
cout<<(*this)[i]<<" ";
    }
cout<<endl; // Переход к новой строке
}
};
// Главная функция программы:
int main(){
    // Инициализация генератора случайных чисел:
srand(2014);
    // Размер массива:
int n=10;
    // Создается объект:
MyClass obj(n);
cout<<"Исходный массив:\n";
    // Отображение элементов массива из объекта:
obj.show();
    // Изменяются значения элементов массива из объекта:
obj[0]=100;
obj[n+1]=200;
cout<<"Массив после изменения значений элементов:\n";
    // Отображение значений элементов массива из объекта:
for(int i=0;i<n;i++){
    // Использован перегруженный оператор:
cout<<obj[i]<<" ";
}
cout<<"\n Циклическое отображение значений элементов:\n";
    // Циклическое отображение значений элементов
    // начиная со середины массива:
obj.show(n/2);
    return 0;
}

```

Результат выполнения программы такой (с поправкой на генерируемые случайные числа):

Результат выполнения программы (из листинга 7.1)

```

Исходный массив:
15 30 15 78 96 43 53 26 73 40
Массив после изменения значений элементов:
100 200 15 78 96 43 53 26 73 40
Циклическое отображение значений элементов:
43 53 26 73 40 100 200 15 78 96

```

Есть еще одна достаточно распространенная ситуация, когда бывает раз-

умно перегрузить оператор []. Дело в том, что метод или функция в качестве результата не могут возвращать массив. Зато они могут возвращать в качестве результата объект. Если массив "спрятать" в объект и перегрузить оператор [] так, чтобы объект можно было индексировать, то тем самым создается полная иллюзия, что результатом метода или функции является массив. Небольшая иллюстрация к сказанному представлена в листинге 7.2.

Листинг 7.2. Индексируемый объект как результат метода

```
#include <iostream>
using namespace std;
// Глобальная константа определяет размер
// используемых массивов:
const int size=5;
// Класс с полем - статическим массивом и методом,
// возвращающем объект класса:
class MyNums{
private: // Закрытые члены класса
    // Поле - статический массив:
    double nums[size];
    // Метод для вычисления среднего значения:
    double avr(){
        // Локальная переменная для записи результата:
        double s=0;
        // Оператор цикла для вычисления суммы элементов:
        for(int i=0;i<size;i++){
            s+=nums[i];
        }
        // Сумма значений элементов делится
        // на количество элементов:
        s/=size;
        return s;
    }
public: // Открытые члены класса
    // Конструктор (аргумент - массив):
    MyNums(double n[size]=NULL){
        // Если аргумент не указан, то поле-массив
        // не заполняется:
        if(n==NULL) retur n;
        // Заполнение поля-массива:
        for(int i=0;i<size;i++){
            nums[i]=n[i];
        }
    }
}
```

```

    // Перегрузка оператора [] для возможности
    // индексирования объектов:
double &operator[](inti){
    // Значение индексированного объекта:
    return nums[i];
}
    // Метод для отображения значений
    // элементов поля-массива:
void show(){
    cout<<"| "; // Начальный символ
    for(int i=0;i<size;i++){
        cout<<nums[i]<<" | ";
    }
    cout<<endl; // Переход на новую строку
}
    // Метод возвращает результатом объект:
MyNums shift(){
    // Локальный массив:
double m[size];
    // Среднее значение по массиву:
double a=avr();
    // Значения элементов локального массива:
for(int i=0;i<size;i++){
    m[i]=nums[i]-a;
}
    // Создание локального объекта:
MyNums tmp(m);
    // Результат метода - объект:
return tmp;
}
};
// Главная функция программы:
int main(){
    // Массив числовых значений:
double x[size]={1.1,3.3,-0.2,2.7,5.1};
    // Объект создается на основе массива:
MyNums obj(x);
    cout<<"Исходный массив:\n";
    // Отображение значений элементов массива
    // из созданного объекта:
obj.show();
    cout<<"После сдвига на среднее:\n";
    // Отображение элементов массива из
    // объекта - результата метода show():
obj.shift().show();
    // Новый объект (с пустым массивом):

```

```
MyNums array;
    // Значением объекту присваивается результат
    // вызова метода show() из объекта obj:
array=obj.shift();
    cout<<"В обратном порядке:\n";
cout<<"| ";
// Отображение (в обратном порядке)
    // значений элементов массива из объекта array:
for(int i=size-1;i>=0;i--){
    cout<<array[i]<<" | ";
}
cout<<endl;
return 0;
}
```

В программе мы используем статические массивы. Чтобы все они были одного размера, мы определяем глобальную (объявлена вне главной функции, поэтому доступна везде - и в классе, и в главной функции) целочисленную константу `size` (со значением 5).

В классе `MyNums` описано закрытое поле-массив (статический) с названием `nums` размера `size` (тип элементов `double`). В классе есть закрытый метод `avr()`, который в качестве результата возвращает значение типа `double`. Методом вычисляется среднее арифметическое значение по элементам поля-массива. В теле метода в локальную переменную `s` сначала записывается (с помощью оператора) сумма значений элементов массива, а затем полученное значение делится на количество элементов в массиве. Полученное значение возвращается методом как результат.

Конструктор класса принимает один аргумент - числовой массив. Аргумент имеет значение по умолчанию `NULL` (пустая ссылка). В теле конструктора в условном операторе проверяется значение аргумента. Если значение аргумента равно `NULL` (например, аргумент не указан), то командой `return` выполнение конструктора прекращается. В этом случае массив `nums` создаваемого объекта заполняться не будет. Если аргументом конструктору передан указатель (имя массива), то с помощью оператора цикла значения из массива, переданного аргументом конструктору, копируются в массив, являющийся полем создаваемого объекта.



На заметку

Напомним, что вместо выражения `NULL` для обозначения пустой ссылки разрешается использовать нулевое значение (нулевой указатель).

Перегрузка оператора `[]` выполнена так, что при индексировании объекта класса выполняется обращение к соответствующему элементу массива, "спрятанного" в объекте.

Метод `show()` не имеет аргументов, не возвращает результат, и предназначен для отображения значений элементов массива-поля объекта.

Наконец, метод `shift()` позволяет создать новый объект класса `MyNums` (возвращаемый результатом метода). В теле метода объявляется локальный числовой массив `m` размера `size`. В переменную `a` записывается результат вызова метода `avr()` - таким образом, эта переменная содержит среднее значение по элементам массива. Запускается оператор цикла, и в нем командой `m[i]=nums[i]-a` значение элемента массива `m` вычисляется как разность значения соответствующего элемента массива `nums` и среднего значения `a`. После заполнения массива `m`, командой `MyNums tmp(m)` на основе массива создается локальный объект `tmp` класса `MyNums`, возвращаемый инструкцией `return tmp` как результат метода.

В главной функции программы объявляется и сразу инициализируется числовой массив `x` размера `size`.



На заметку

Если массив инициализируется одновременно с объявлением, то значения, присваиваемые элементам массива, указываются, разделенные запятыми, в фигурных скобках - такая конструкция "присваивается" массиву. Количество элементов в фигурных скобках должно соответствовать размеру массива. Если размер массива явно не указать, он будет определен автоматически по количеству значений в фигурных скобках.

На основе массива `x` командой `MyNums obj(x)` создается объект `obj`. Командой `obj.show()` отображается содержимое массива из объекта `obj`. Командой `obj.shift().show()` отображается содержимое массива, который вычисляется и возвращается методом `shift()` на основе другого массива - того, что спрятан в объекте `obj`. В данном случае мы воспользовались тем, что результатом выражения `obj.shift()` является объект класса `MyNums`, и из него, следовательно, получится вызвать метод `show()`. Но можно поступить и иначе. Например, сначала командой `MyNums array` создается новый объект (аргумент конструктору не передается, поэтому массив в объекте не заполнен - но в данном случае это не принципиально), а затем командой `array=obj.shift()` данному объекту присваивается в качестве значения результат вызова метода `shift()` из объекта `obj`. Учитывая наличие перегруженного оператора `[]` для объектов класса `MyNums` (к ним относится и объект `array`), можем индексировать объект `array`, что и используется в операторе цикла для отображения (в обратном поряд-

ке) значений элементов массива из объекта `array`. Результат выполнения всего программного кода будет таким:

Результат выполнения программы (из листинга 7.2)

```
Исходный массив:
| 1.1 | 3.3 | -0.2 | 2.7 | 5.1 |
После сдвига на среднее:
| -1.3 | 0.9 | -2.6 | 0.3 | 2.7 |
В обратном порядке:
| 2.7 | 0.3 | -2.6 | 0.9 | -1.3 |
```

Здесь мы в полях класса не случайно использовали статические массивы: мы имели дело с копированием объектов, и в таком случае статические массивы "надежнее". С динамическими массивами не все так однозначно.

7.2. Особенности использования динамических массивов в классах

*Как говорит наш дорогой шеф, в нашем деле
главное - этот самый реализм.
из к/ф "Бриллиантовая рука"*

Чтобы понять суть проблемы, которая может возникнуть при использовании в классах динамических массивов, рассмотрим небольшой, но показательный пример. А именно, создадим класс с конструктором, деструктором и динамическим массивом. Затем создадим два объекта класса, один объект присвоим в качестве значения другому и посмотрим, что из этого получится. Программный код приведен в листинге 7.3.

Листинг 7.3. Присваивание объектов класса с динамическим массивом

```
#include <iostream>
using namespace std;
// Класс с динамическим массивом:
class MyClass{
public:
int *nums; // Указатель на массив
int size; // Размер массива
// Конструктор:
MyClass(int n){
// Присваивание значения полю:
size=n;
// Создание динамического массива:
nums=new int[size];
// Заполнение массива нулями:
```

```

for(int i=0;i<size;i++){
nums[i]=0; // Значение элемента массива
}
}
// Деструктор:
~MyClass(){
// Удаление массива:
delete [] nums;
}
// Метод для отображения содержимого массива:
void show(){
for(int i=0;i<size;i++){
// Отображение значения элемента массива:
cout<<nums[i]<<" ";
}
cout<<endl; // Переход к новой строке
}
};
// Главная функция программы:
int main(){
// Первый объект:
MyClass A(5);
// Второй объект:
MyClass B(10);
// Значения элементов массива из первого объекта:
A.show();
// Значения элементов массива из второго объекта:
B.show();
// Второму объекту в качестве значения
// присваивается первый объект:
B=A;
// Значения элементов массива из второго объекта
// после присваивания:
B.show();
// Изменяем значение элемента массива
// из первого объекта:
A.nums[2]=123;
// Значения элементов массива из первого объекта
// после изменения значения элемента массива
// из первого объекта:
A.show();
// Значения элементов массива из второго объекта
// после изменения значения элемента массива
// из первого объекта:
B.show();
return 0;

```

```
}
```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 7.3)

```
0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0
0 0 123 0 0
0 0 123 0 0
```

В главной функции программы создаются два объекта `A` и `B`, соответственно с массивами из 5 и 10 элементов. В конструкторе значениям всех элементов присваиваются нулевые значения. Из объектов `A` и `B` вызывается метод `show()`. В результате появляется два ряда нулей - в одном пять, в другом десять. Затем командой `B=A` второму объекту в качестве значения присваивается первый объект. Командой `B.show()` проверяем, что объект `B` действительно изменился (теперь вместо десяти нулей отображается пять). Затем мы изменяем значение одного из элементов массива из объекта `A`: командой `A.nums[2]=123` элементу с индексом 2 присваивается значение 123.

Если теперь проверить содержимое массива `nums` объекта `A`, то получим вполне ожидаемый результат: соответствующий элемент массива изменил свое значение. Но вот результат выполнения команды `B.show()` может стать сюрпризом: элемент массива `nums` с индексом 2 данного объекта также изменил свое значение с нулевого на 123. То есть, изменяя значение элемента командой `A.nums[2]=123` мы тем самым изменили и элемент массива в объекте `B`. А если быть совсем точным, то в обоих случаях речь идет об одном и том же элементе и, соответственно, одном и том же массиве. Чтобы понять причину происходящего, необходимо учесть "технология" операции присваивания объектов. Если коротко, то выполняется побитовое копирование значений полей объектов.

Для удобства будем думать, что речь идет об объектах одного класса (напомним, что еще может быть вариант, когда объектной переменной базового класса присваивается объект производного класса, но суть дела от этого мало меняется). Если объекты относятся к одному классу, то у них одинаковый набор полей. При присваивании объектов выполняется копирование значений полей: значения полей объекта справа от оператора присваивания присваиваются полям объекта слева от оператора присваивания.

Теперь посмотрим, что происходит при выполнении команды $B=A$. При выполнении команды значения полей объекта A присваиваются соответствующим полям объекта B . У объектов A и B два поля: числовое поле `size` и поле-указатель `nums`. После выполнения команды $B=A$ значение поля `size` объекта B такое же, как значение поля `size` объекта A . Аналогично, значение поля `nums` объекта B совпадает со значением поля `nums` объекта A . С полем `size` все достаточно просто - это числовое поле, поэтому происходит присваивание числового значения. С полем `nums` тоже все просто, но только данное поле - указатель, посему речь идет о присваивании адресов. Адрес в поле `nums` объекта B будет таким же, как и адрес поля `nums` объекта A , и это адрес первого элемента динамического массива, сгенерированного при создании объекта A . Отсюда и получается, что объекты A и B содержат ссылку на один и тот же массив.

Со статическими массивами, надо отметить, таких проблем не возникает. Небольшой пример приведен в листинге 7.4. Представленный код похож на предыдущий, но только теперь в классе используется статический массив.

Листинг 7.4. Присваивание объектов класса со статическим массивом

```
#include <iostream>
using namespace std;
// Размер массива:
const int size=5;
// Класс со статическим массивом:
class MyClass{
public:
    int nums[size]; // Поле-массив
// Конструктор:
    MyClass() {
        // Заполнение массива нулями:
        for(int i=0;i<size;i++){
            nums[i]=0; // Значение элемента массива
        }
        // Метод для отображения содержимого массива:
        void show(){
            for(int i=0;i<size;i++){
                // Отображение значения элемента массива:
                cout<<nums[i]<<" ";
            }
            cout<<endl; // Переход к новой строке
        }
    };
};
```



```
// Главная функция программы:
int main(){
    // Первый объект:
    MyClass A;
    // Второй объект:
    MyClass B;
    // Значения элементов массива из первого объекта:
    A.show();
    // Значения элементов массива из второго объекта:
    B.show();
    // Второму объекту в качестве значения
    // присваивается первый объект:
    B=A;
    // Изменяем значение элемента массива
    // из первого объекта:
    A.nums[2]=123;
    // Значения элементов массива из первого объекта
    // после изменения значения элемента массива
    // из первого объекта:
    A.show();
    // Значения элементов массива из второго объекта
    // после изменения значения элемента массива
    // из первого объекта:
    B.show();
    return 0;
}
```

В результате выполнения программного кода получим следующее:

Результат выполнения программы (из листинга 7.4)

```
0 0 0 0 0
0 0 0 0 0
0 0 123 0 0
0 0 0 0 0
```

Видим, изменение значения элемента массива `nums` из объекта `Ана` массиве из объекта `Вника` не сказывается. Причина в том, теперь массив `nums` является статическим: при создании объекта класса под элементы такого массива автоматически выделяется память, а при присваивании объектов выполняется побитовое копирование значений элементов массива `nums` из одного объекта в другой.

7.3. Перегрузка оператора присваивания

Не надо меня щадить: пусть самое страшное, но правда.

из к/ф "Бриллиантовая рука"

Но вернемся к динамическим массивам. Разумеется, есть рецепт, позволяющий разумными средствами решить проблему копирования объектов. Стоит он в том, чтобы перегрузить оператор присваивания.



На заметку

Оператор присваивания = может перегружаться. Операндами этого бинарного оператора являются переменные (объекты), указанные справа и слева от оператора присваивания. Другими словами в выражении вида `B=A` операндами являются `B` и `A`.

Чтобы перегрузить оператор присваивания, в классе описываем операторный метод с названием `operator=`. Под первым операндом здесь подразумевается объект слева от оператора присваивания (то есть тот объект, которому присваивается значение). Соответственно, второй операнд описывается как аргумент операторного метода. Поэтому при вычислении выражения `B=A` из объекта `B` вызывается операторный метод `operator=()` с аргументом `A`. Поскольку в C++ оператор присваивания возвращает значение, то оператор присваивания перегружают так, чтобы значение возвращалось (обычно значение, возвращаемое оператором присваивания - объект слева от оператора). Это позволяет использовать конструкции вида `B=A` операндами в более сложных выражениях.

Пример программного кода с классом, содержащим динамический массив (поле-указатель ссылается на динамический массив) с перегруженным оператором присваивания представлен в листинге 7.5.

Листинг 7.5. Класс с динамическим массивом и перегруженным оператором присваивания

```
#include <iostream>
using namespace std;
// Класс с динамическим массивом:
class MyClass{
public:
    int *nums; // Указатель на массив
    int size; // Размер массива
    // Конструктор:
    MyClass(int n){
        // Присваивание значения полюsize:
        size=n;
        // Создание динамического массива:
        nums=new int[size];
        // Заполнение массива нулями:
```

```

for(int i=0;i<size;i++){
nums[i]=0; // Значение элемента массива
}
}
// Деструктор:
~MyClass(){
// Удаление массива:
delete [] nums;
}
// Метод для отображения содержимого массива:
void show(){
for(int i=0;i<size;i++){
// Отображение значения элемента массива:
cout<<nums[i]<<" ";
}
cout<<endl; // Переход к новой строке
}
// Перегрузка оператора присваивания.
// Результатом метода является ссылка на объект.
// Аргумент также передается по ссылке:
MyClass &operator=(MyClass &obj){
// Удаление старого массива:
delete [] this->nums;
// Новое значение поля size:
this->size=obj.size;
// Создается новый массив:
this->nums=new int[this->size];
// Копирование элементов массива:
for(int i=0;i<this->size;i++){
// Значения полей объекта слева от оператора
// присваивания такие же, как и у объекта
// справа от оператора присваивания:
this->nums[i]=obj.nums[i];
}
// Результат операторного метода:
return *this;
}
};
// Главная функция программы:
int main(){
// Первый объект:
MyClass A(5);
// Второй объект:
MyClass B(10);
// Значения элементов массива из первого объекта:
A.show();

```

```

// Значения элементов массива из второго объекта:
B.show();
// Второму объекту в качестве значения
// присваивается первый объект:
B=A;
// Значения элементов массива из второго объекта
// после присваивания:
B.show();
// Изменяем значение элемента массива
// из первого объекта:
A[nums[2]]=123;
// Значения элементов массива из первого объекта
// после изменения значения элемента массива
// из первого объекта:
A.show();
// Значения элементов массива из второго объекта
// после изменения значения элемента массива
// из первого объекта:
B.show();
return 0;
}

```

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 7.5)

```

0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0
0 0 123 0 0
0 0 0 0 0

```

Здесь мы фактически добавили в программный код класса MyClass описание операторного метода `operator=()`. Для удобства приведем отдельно соответствующий программный код (комментарии удалены):

```

MyClass &operator=(MyClass &obj) {
delete [] this->nums;
this->size=obj.size;
this->nums=new int[this->size];
for(int i=0;i<this->size;i++){
this->nums[i]=obj.nums[i];
}
return *this;
}

```

Видим, что у метода один аргумент, и он передается по ссылке (перед именем аргумента `obj` указан символ `&`). Метод возвращает результат, и этот результат - объект класса `MyClass`. Более того, в прототипе перед названием операторного метода есть символ `&`. Поэтому метод не просто возвращает объект, а ссылку на объект. Какой это объект, понять несложно, взглянув на `return`-инструкцию в теле метода. Соответствующая команда имеет вид `return *this`. Ключевое слово `this`, как мы знаем, является указателем на объект, из которого вызывается метод. Если перед указателем разместить `*`, получим значение, записанное по адресу из указателя. В данном случае `*this` означает объект, из которого вызывается операторный метод. То есть если мы говорим о команде вида `B=A`, то метод `operator=()` вызывается из объекта `B`, аргументом методу передается (по ссылке) объект `A`, а результатом выражения `B=A` будет объект `B` (не копия объекта, а именно сам объект).

Понять, почему аргумент операторному методу передается по ссылке и почему результатом операторного метода должна возвращаться ссылка на объект, на первый взгляд не так уж и просто. Хотя именно здесь кроется ключ к пониманию "механики" работы с объектами - во всяком случае, в той части, что касается присваивания, копирования, создания и удаления объектов, а также использования методов совместно с объектами.

Итак, если коротко, то мы пытаемся избежать ситуации, когда создается копия объекта. Чтобы понять, почему это плохо, сначала проанализируем ситуацию с аргументом метода. Предположим, что аргумент операторному методу передается по значению. В данном случае для объекта, переданного аргументом методу, создается копия. Копия создается побитовым копированием содержимого исходного объекта в объект-копию: грубо говоря, происходит обычное дублирование. Но проблема в том, что мы используем в объектах динамические массивы, связанные с объектом только полем-указателем.

При создании копии объекта значение поля копируется, а новый массив не создается. Поэтому и объект-оригинал, и объект-копия ссылаются на один и тот же массив. Непосредственно здесь большой проблемы или опасности нет. Проблема возникает, когда операторный метод завершает работу и все локальные переменные, в том числе и копия объекта-аргумента, удаляются из памяти. И вот когда объект-копия удаляется из памяти, то вызывается деструктор. А в деструкторе есть команда удаления массива, на который ссылается поле-указатель. Вопрос: какой массив удаляется при удалении объекта-копии? Правильно - массив, на который ссылается объект, переданный аргументом операторному методу. Причина в том, что поле-указатель в объекте-копии содержит тот же адрес, что и поле-указатель в объек-

те-аргументе. А это уже катастрофа. Ничего подобного не происходит, если аргумент операторному методу передается через ссылку.

Подытоживая, можем констатировать, что опасность создания копии связана с тем, что при удалении копии вызывается деструктор. Похожая ситуация и с результатом метода. Напомним, что результатом операторного метода возвращается ссылка на объект, которому присваивается значение. Чтобы понять важность данного обстоятельства, поступим, аналогично к случаю исследования "роли аргумента": предположим, что возвращается не ссылка на объект, а "просто" объект. И тут есть важный идеологический момент: необходимо четко понимать, как метод возвращает результат.

Нелишним будет освежить в памяти основные моменты. Итак, мы говорим о методе, возвращающем некоторый объект (но не ссылку на объект). Поскольку метод возвращает результат, то под него выделяется место в памяти. При выполнении инструкции `return` с указанием возвращаемого в качестве результата объекта, "возвращаемый" объект копируется в область памяти, зарезервированную под результат. То есть фактически для результата метода создается техническая копия. Здесь есть некоторая аналогия с передачей аргумента по значению. Дальнейшие события также разворачиваются по аналогии с "историей про аргумент".

Копия объекта-результата удаляется из памяти, как только в ней исчезает необходимость. А необходимость исчезает сразу после использования результата метода в программе (по факту при завершении работы метода). В данном конкретном примере копия объекта-результата содержит поле-указатель такое же, как и объект, из которого вызывается операторный метод. При удалении копии объекта-результата вызывается деструктор. Он удаляет динамический массив, на который через свое поле-указатель ссылается исходный объект (объект, которому присваивается значение и из которого вызывается операторный метод). Если же операторный метод возвращает ссылку на объект, то копия для объекта-результата не создается: результат возвращается "напрямую", так сказать, "без посредников".



На заметку

Несложно заметить, что "все неприятности" в данном примере связаны с деструктором класса `MyClass`. Если удалить из программного кода деструктор, то можно также удалить оба символа `&` в описании операторного метода `operator=()` (как следствие аргумент передается по значению, а результатом будет объект, а не ссылка на объект). Правда, тогда при удалении объектов из памяти динамические массивы не удаляются.

Проанализировав ситуацию, легко заметить, что критическим моментом в описанных выше схемах является создание копии объекта. Мы можем су-

щественно облегчить себе задачу, если изменим используемый по умолчанию способ копирования объектов (имеется в виду создание копии объекта на основе уже существующего объекта). Для этого в классе необходимо описать *конструктор создания копии*.

7.4. Конструктор создания копии

*В этом есть известное изящество.
из к/ф "Покровские ворота"*

Конструктор создания копии - это конструктор для создания нового объекта на основе уже существующего объекта. У такого конструктора один аргумент, являющийся объектом того же класса, для которого описывается конструктор создания копии.



На заметку

Если в классе конструктор создания копии не описан, то копия объекта создается побитовым копированием. Если в классе описан конструктор создания копии, то каждый раз, когда явно или неявно создается копия объекта, вызывается данный конструктор.

Рассмотрим вариант решения предыдущей задачи, когда в классе `MyClass` описан конструктор создания копии (комментарии удалены):

```
MyClass(MyClass &obj) {
    size=obj.size;
    nums=new int[size];
    for(int i=0;i<size;i++){
        nums[i]=obj.nums[i];
    }
}
```

Аргументом конструктора указан объект `obj` класса `MyClass`, причем аргумент передается по ссылке (перед названием аргумента указан `&`). Конструктор создания копии *всегда* описывается с аргументом, передаваемым по ссылке.

Подробности

Если забыть инструкцию `&` перед аргументом конструктора, то при попытке создать копию объекта для аргумента, переданного конструктору, будет создаваться копия. Для создания копии вызывается конструктор создания копии, а аргументом конструктору передается копия для копии объекта. Чтобы создать "копию для копии" снова вызывается конструктор создания копии, и так до бесконечности. Поэтому аргумент конструктору создания копии передается по ссылке.

В теле конструктора создания копии на основе объекта-аргумента определяется размер динамического массива, создается новый динамический массив (для создаваемого объекта), и значения элементов созданного массива заполняются на основе значений элементов массива объекта, переданного аргументом конструктору.

Весь программный код представлен в листинге 7.6. Стоит обратить внимание, что теперь при описании операторного метода для оператора присваивания нет необходимости передавать аргумент по ссылке и возвращать в качестве результата ссылку на объект (и мы эти механизмы теперь не используем).



На заметку

В операторном методе в предыдущем примере на поля объекта, из которого вызывается метод, использовались полные ссылки через указатель `this`: например `this->size` или `this->nums`. Здесь мы от подобной практики отказались.

Листинг 7.6. Конструктор создания копии

```
#include <iostream>
using namespace std;
// Класс с динамическим массивом:
class MyClass{
public:
    int *nums; // Указатель на массив
    int size;  // Размер массива
    // Конструктор:
    MyClass(int n){
        // Присваивание значения полю size:
        size=n;
        // Создание динамического массива:
        nums=new int[size];
        // Заполнение массива нулями:
        for(int i=0;i<size;i++){
            nums[i]=0; // Значение элемента массива
        }
        // Конструктор создания копии.
        // Аргумент передается по ссылке:
        MyClass(MyClass &obj){
            // Присваивание значения полю size:
            size=obj.size;
            // Создание динамического массива:
            nums=new int[size];
            // Заполнение массива:
```



```

for(int i=0;i<size;i++){
nums[i]=obj.nums[i]; // Значение элемента массива
}
}
// Деструктор:
~MyClass(){
// Удаление массива:
delete [] nums;
}
// Метод для отображения содержимого массива:
void show(){
for(int i=0;i<size;i++){
// Отображение значения элемента массива:
cout<<nums[i]<<" ";
}
cout<<endl; // Переход к новой строке
}
// Перегрузка оператора присваивания:
MyClass operator=(MyClass obj){
// Удаление старого массива:
delete [] nums;
// Новое значение поля size:
size=obj.size;
// Создается новый массив:
nums=new int[size];
// Копирование элементов массива:
for(int i=0;i<size;i++){
// Значения полей объекта слева от оператора
// присваивания такие же, как и у объекта
// справа от оператора присваивания:
nums[i]=obj.nums[i];
}
// Результат операторного метода:
return *this;
}
};
// Главная функция программы:
int main(){
// Первый объект:
MyClass A(5);
// Второй объект:
MyClass B(10);
// Значения элементов массива из первого объекта:
A.show();
// Значения элементов массива из второго объекта:
B.show();

```

```

// Второму объекту в качестве значения
// присваивается первый объект:
V=A;
// Значения элементов массива из второго объекта
// после присваивания:
V.show();
// Изменяем значение элемента массива
// из первого объекта:
A.nums[2]=123;
// Значения элементов массива из первого объекта
// после изменения значения элемента массива
// из первого объекта:
A.show();
// Значения элементов массива из второго объекта
// после изменения значения элемента массива
// из первого объекта:
V.show();
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 7.6)

```

0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0
0 0 123 0 0
0 0 0 0 0

```

Как видим, код выполняется вполне корректно.



На заметку

Читатель, наверное, заметил, что между деструктором, оператором присваивания и конструктором создания копии прослеживается четкая "идеологическая" связь. Дело в том, что деструктор, конструктор создания копии и операторный метод для оператора присваивания обычно сильно привязаны друг к другу. Существует так называемое "*Правило большой тройки*" (под тройкой имеется в виду деструктор, конструктор и оператор присваивания). Состоит оно в том, что если в программном коде класса описан хотя бы один из трех упомянутых методов, то следует описать и два других. Иначе говоря, обычно в классе или описывают деструктор, конструктор создания копии и перегружают оператор присваивания, или не делают ничего из перечисленного. Логика здесь такая: если не описать ничего, то соответствующие операции выполняются в режиме "по умолчанию". В данном режиме функции деструктора, конструктора создания копии и оператора присваивания выполняются по наиболее простому и относительно безопасному алгоритму. Но

лишь только в классе что-то из указанной великолепной тройки описано, значительно возрастает вероятность, что могут возникнуть проблемы при выполнении программного кода.

На этом мы заканчиваем главу, но не заканчиваем наше знакомство с массивами. Просто кроме массивов есть много других интересных и перспективных тем. Им тоже имеет смысл уделить внимание. А к массивам мы еще вернемся.

Глава 8.

ФУНКЦИИ И КЛАССЫ



Они хотят Галактику завоевать. Неужели непонятно?

из к/ф "Гостья из будущего"

Пришло время нам познакомиться с очень мощным механизмом, базирующимся на создании *обобщенных функций*, или даже *обобщенных классов*. Все это богатство часто называют общим словом - *шаблоны*. Главная идея, положенная в основу создания и использования шаблонов, сводится к тому, что тип данных, который принято указывать в явном виде, в обобщенной функции или классе описывается через параметр, а затем уже при вызове функции или создании объекта параметр определяется явно или неявно по контексту соответствующей команды. Сначала посмотрим, как эта идея реализуется в функциях.

8.1. Обобщенные функции

Скучно, господа, скучно. Вот в Средние Века скучно не было: тогда в каждом доме был домовый, а в каждой церкви - Бог.

из к/ф "Сталкер"

Нередко случается так, что алгоритм выполнения функции достаточно универсальный в том смысле, что не зависит от типа используемых данных. Классическая задача: функция с двумя аргументами, и при вызове функции аргументы "обмениваются" значениями. Понятно, что аргументами функции могут быть числа, символы, текст или объекты некоторого класса. Алгоритм реализации "обмена" значениями аргументов не зависит от типа аргументов (главное, чтобы у аргументов были эти самые "значения" и допустимой была процедура присваивания). Кратко алгоритм, реализуемый в функции, мог бы быть описан следующим образом:

- В теле функции создается переменная того же типа, что и аргументы функции.
- Локальной переменной присваивается значение первого аргумента функции.
- Первому аргументу функции присваивается значение второго аргумента функции.

- Второму аргументу функции присваивается значение локальной переменной (в нее ранее было записано значение первого аргумента).
- Функция не возвращает результат, а аргументы функции передаются по ссылке (чтобы можно было изменить их значения).

Если мы хотим, чтобы функция могла вызываться с аргументами разных типов (при условии, что у первого и второго аргументов типы совпадают), то могли бы перегрузить функцию - создать версию функции для каждого из возможных типов аргументов. Но здесь есть две небольшие проблемы. Во-первых, не всегда заранее получается определить, о каких типах идет речь. Во-вторых, программный код разных версий функции будет отличаться только идентификаторами типов в соответствующих местах кода. Подобный подход не очень "интеллектуальный". Намного продуктивнее создать одну функцию, в которой тип данных сам описывается как параметр. Это и есть обобщенная (или шаблонная) функция.

При описании обобщенной функции используют конструкцию следующего вида (наиболее важные элементы выделены жирным шрифтом):

```
template <class параметр> тип имя (аргументы) {  
    // тело функции  
}
```

В принципе обобщенная функция описывается практически так же, как и обычная функция - но есть некоторые "дополнительные" элементы. Главное отличие состоит в том, что описание обобщенной функции начинается с ключевого слова `template`, а далее в угловых скобках указывается ключевое слово `class` и формальный параметр, обозначающий тип данных. Затем следует самое обычное описание функции, с одной лишь поправкой: в описании можно использовать параметр для обозначения типа данных. Таким образом, получается, что тип данных используется в функции как некий абстрактный идентификатор. Впоследствии при вызове функции вместо означенного идентификатора (то есть параметра) используется конкретный тип - какой именно, определяется по контексту вызова функции. Далее, если это не приведет к недоразумениям, будем отождествлять параметр, обозначающий тип данных, с *обобщенным типом*.



На заметку

В обобщенной функции разрешается использовать несколько параметров для обозначения разных типов (то есть обобщенная функция может содержать несколько обобщенных типов). В таком случае в угловых скобках через запятую перечисляются конструкции вида `class параметр`, где каждый из параметров обозначает некоторый (обобщенный) тип данных.

В частности, описанная выше обобщенная функция для "обмена" значениями аргументов могла бы выглядеть так:

```
// Обобщенная функция. Через X обозначен тип данных.
// Аргументы функции передаются по ссылке:
template<class X> void SwapData(X &a,X&b){
X tmp; // Локальная переменная типа X
tmp=a; // Локальной переменной присваивается
      // значение первого аргумента
a=b;   // Первому аргументу присваивается
      // значение второго аргумента
b=tmp; // Второму аргументу присваивается
      // значение локальной переменной
}
```

При вызове обобщенной функции `SwapData()` тип обобщенного параметра определяется по вызову функции. Например, при вызове функции с целочисленными аргументами ее программный код выполняется, как если бы вместо параметра `X` был указан тип `int`. При вызове функции с аргументами типа `char` ее код выполнялся, как если бы параметром `X` был тип `char`, ну и так далее. В листинге 8.1 приведен программный код, иллюстрирующий применение обобщенной функции `SwapData()` (для простоты комментарии в теле функции удалены).

Листинг 8.1. Обобщенная функция

```
#include <iostream>
#include <string>
using namespace std;
// Первый класс:
class First{
public:
// Текстовое поле:
string name;
// Конструктор класса (со значением аргумента
// по умолчанию):
First(string txt=""){
// Полю присваивается значение:
name=txt;
}
// Метод для отображения значения поля:
void show(){
cout<<"Текстовое поле: "<<name<<endl;
}
}; // Завершение описания первого класса
// Второй класс:
```



```

class Second{
public:
    // Целочисленное поле:
    int code;
    // Конструктор класса (со значением аргумента
    // по умолчанию):
    Second(int n=0){
        // Полю присваивается значение:
        code=n;
    }
    // Метод для отображения значения поля:
    void show(){
        cout<<"Числовое поле: "<<code<<endl;
    }
}; // Завершение описания второго класса
// Обобщенная функция:
template <class X> void SwapData(X &a,X &b){
    X tmp;
    tmp=a;
    a=b;
    b=tmp;
}
// Главная функция программы:
int main(){
    cout<<"Аргументы - целые числа.\n";
    // Целочисленные переменные:
    int a=5,b=10;
    // Вызов обобщенной функции с
    // целочисленными аргументами:
    SwapData(a,b);
    // Проверка значений целочисленных переменных
    // после вызова обобщенной функции:
    cout<<"a = "<<a<<endl;
    cout<<"b = "<<b<<endl;
    cout<<"Аргументы - символы.\n";
    // Символьные переменные:
    char x='A',y='Я';
    // Вызов обобщенной функции с
    // символьными аргументами:
    SwapData(x,y);
    // Проверка значений символьных переменных
    // после вызова обобщенной функции:
    cout<<"x = "<<x<<endl;
    cout<<"y = "<<y<<endl;
    cout<<"Аргументы - объекты класса First.\n";
    // Объекты класса First:

```

```

First A("объект A"), B("объект B");
    // Вызов обобщенной функции с
    // аргументами - объектами класса First:
SwapData(A, B);
// Вызов метода show() для проверки значений
// полей объектов после вызова обобщенной функции:
A.show();
B.show();
cout<<"Аргументы - объекты класса Second.\n";
// Объекты класса Second:
Second objA(100), objB(200);
// Вызов обобщенной функции с
// аргументами - объектами класса Second:
SwapData(objA, objB);
// Вызов метода show() для проверки значений
// полей объектов после вызова обобщенной функции:
objA.show();
objB.show();
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 8.1)

```

Аргументы - целые числа.
a = 10
b = 5
Аргументы - символы.
x = Я
y = А
Аргументы - объекты класса First.
Текстовое поле: объект B
Текстовое поле: объект A
Аргументы - объекты класса Second.
Числовое поле: 200
Числовое поле: 100

```

В программе описывается два класса с названиями `First` и `Second`. У каждого из классов имеется поле: у класса `First` объявлено текстовое поле `name`, а у класса `Second` есть целочисленное поле `code`. Еще у классов описан метод `show()`, предназначенный для отображения значения поля: для класса `First` отображается значение поля `name`, а для класса `Second` отображается значение поля `code`. В каждом классе описан конструктор с одним аргументом. Важное обстоятельство: у аргументов конструкторов

заданы значения по умолчанию, поэтому при создании объекта как класса `First`, так и класса `Second` конструктору аргумент можно не передавать.



На заметку

Почему это важно? Потому что мы планируем передавать объекты классов `First` и `Second` аргументами функции `SwapData()`. В теле функции имеется команда объявления локальной переменной `tmp` того же типа, что и тип аргументов (обобщенный тип `X`). Если аргументы являются объектами, то и локальная переменная `tmp` будет объектом. Но в команде создания объекта (имеется в виду команда `X tmp`) передача аргументов конструктору не предусмотрена. Поэтому если при создании объекта конструктору обязательно нужно передавать аргумент (или аргументы), то возникнет ошибка (программный код просто не скомпилируется). Отсюда и острая необходимость в наличии у класса конструктора без аргументов (или конструктора с аргументами, имеющими значение по умолчанию).

В главной функции программы обобщенная функция `SwapData()` последовательно вызывается с целочисленными аргументами, с символьными аргументами, а также с аргументами - объектами классов `First` и `Second` (сначала оба аргумента относятся к классу `First`, а затем оба аргумента относятся к классу `Second`). Во всех этих случаях значение параметра `X`, обозначающего обобщенный тип, определяется автоматически исходя из контекста вызова функции. Так, при целочисленных аргументах значение параметра `X` равно `int`, при символьных аргументах функции параметр `X` тождественен типу `char`, когда аргументами функции являются объекты класса `First` - параметр `X` обозначает класс `First`, а если функции `SwapData()` аргументами передаются объекты класса `Second`, то параметр `X` следует полагать равным названию данного класса.

Как отмечалось выше, у обобщенной функции может быть несколько обобщенных типов. Небольшой пример подобной функции (и сопутствующего программного кода) приведен в листинге 8.2. В программе описывается два класса (называются `IntClass` и `CharClass`). В каждом классе есть по одному полю: в классе `IntClass` описано целочисленное поле `code`, а в классе `CharClass` есть символьное поле `symb`. У каждого класса описан метод `show()`, отображающий значение поля (для каждого класса свое). Также в каждом классе описан конструктор, позволяющий при создании объекта задать значение поля объекта. У аргумента конструктора есть значение по умолчанию, поэтому при создании объекта конструктору передается один аргумент или аргументы не передаются.

Но понятно, что это все "преамбула". Основной интерес представляет обобщенная функция `getObject()`, описанная с двумя обобщенными типами (параметры обозначены через `X` и `Y`). Первый аргумент обозначен как такой, что относится к типу `X`, а второй аргумент функции относится к типу

Y. Неявно предполагается, что X - объектный тип (то есть первый аргумент функции - объект). Более того, мы исходим из того, что через параметр X обозначен не просто класс, а класс с методом `show()`. В теле функции динамически создается объект обобщенного типа X, причем аргументом конструктору передается второй аргумент обобщенной функции. Из созданного объекта вызывается метод `show()`, а указатель на созданный объект возвращается результатом функции.

Теперь рассмотрим программный код:

Листинг 8.2. Обобщенная функция с двумя параметрами

```
#include <iostream>
using namespace std;
// Класс с целочисленным полем:
class IntClass{
public:
    // Поле:
    int code;
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение поля code = "<<code<<endl;
    }
    // Конструктор:
    IntClass(int =0){
        code=a; // Полю присваивается значение
    }
}; // Завершение описания класса
// Класс с символьным полем:
class CharClass{
public:
    // Поле:
    char symb;
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение поля symb = "<<symb<<endl;
    }
    // Конструктор:
    CharClass(char s='a'){
        symb=s; // Присваивание значения полю
    }
}; // Завершение описания класса
// Обобщенная функция:
template<class X,class Y> X *getObject(X obj,Y field){
    cout<<"Начинает выполняться функция getObject().\n";
```

```

X *tmp; // Указатель на значение обобщенного типа
// Динамически создается объект:
tmp=new X(field);
// Вызов метода из локального объекта:
tmp->show();
cout<<"Выполнение функции getObject() завершено.\n";
// Результат функции - указатель на объект:
return tmp;
}
// Главная функция программы:
int main(){
    // Объект класса IntClass:
    IntClass objInt;
    // Объект класса CharClass и указатель на объект:
    CharClass objChar, *pnt;
    // Вызов метода через указатель на объект,
    // который возвращается обобщенной функцией:
    getObject(objInt,100)->show();
    // Результат обобщенной функции записывается
    // в указатель на объект:
    pnt=getObject(objChar,'z');
    // Вызов метода через указатель на объект:
    pnt->show();
    return 0;
}

```

При выполнении программного кода в окне вывода появляются следующие сообщения:

Результат выполнения программы (из листинга 8.2)

```

Начинает выполняться функция getObject().
Значение поля code = 100
Выполнение функции getObject() завершено.
Значение поля code = 100
Начинает выполняться функция getObject().
Значение поля symb = z
Выполнение функции getObject() завершено.
Значение поля symb = z

```

В первую очередь более детально обсудим программный код обобщенной функции `getObject()`. Ниже отдельно приведен код функции (с удаленными комментариями):

```
template<class X, class Y> X *getObject(X obj, Y field) {
    cout<<"Начинает выполняться функция getObject().\n";
    X *tmp;
    tmp=new X(field);
    tmp->show();
    cout<<"Выполнение функции getObject() завершено.\n";
    return tmp;
}
```

Как отмечалось выше, в функции использовано два обобщенных типа: первый аргумент `obj` функции описан с идентификатором типа `X`, второй аргумент `field` функции описан с обобщенным типом `Y`. Результат функции - указатель (звездочка `*` перед именем функции) на объект типа `X`.



На заметку

Фактически, первый аргумент функции `getObject()` используется исключительно для идентификации обобщенного типа `X`. При вызове функции, исходя из того, к какому классу относится объект, переданный первым аргументом функции, определяется тип создаваемого в теле функции объекта и, соответственно, тип результата функции (ссылка на объект).

Командой `X *tmp` в теле функции создается указатель `tmp` на объект класса `X` (самого объекта пока еще нет). Объект создается динамически с помощью команды `tmp=new X(field)`. В данном случае аргумент `field` обобщенной функции `getObject()` передается аргументом конструктору, а адрес объекта записывается в указатель `tmp`. Указатель возвращается результатом функции (команда `return tmp`), но перед этим командой `tmp->show()` из созданного объекта вызывается метод `show()`.



На заметку

Команда `tmp=new X(field)` предполагает, что у класса, обозначенного формально через `X`, есть конструктор с одним аргументом. Выполнение команды `tmp->show()` возможно только в том случае, если в классе, обозначенном параметром `X`, описан метод `show()`. Классы `IntClass` и `CharClass` удовлетворяют обоим этим критериям.

В главной функции программы создается объект `objInt` класса `IntClass`, объект `objChar` класса `CharClass`, а еще указатель `pnt` на объект класса `CharClass`. Есть и нетривиальные команды. Например, такая: `getObject(objInt, 100)->show()`. Здесь вызывается функция `getObject()`, возвращающая результатом указатель на объект того же класса, что и объект `objInt` (то есть речь идет о классе `IntClass`), а поле `code`

объекта равно 100 (второй аргумент функции `getObject()`). Через указатель на объект (выражение `getObject(objInt, 100)`) вызывается метод `show()` объекта. В результате отображается значение его поля `code`.

Результат выражения `getObject(objChar, 'z')` - указатель на объект класса `CharClass` (поскольку к данному классу относится объект `objChar`, переданный первым аргументом обобщенной функции `getObject()`). Значение поля `symbol` обобщенного объекта равно 'z' (второй аргумент обобщенной функции `getObject()`). В данном случае результат вызова обобщенной функции записывается указатель `pnt` (команда `pnt=getObject(objChar, 'z')`), а затем командой `pnt->show()` метод `show()` вызывается из объекта и появляется сообщение о значении его поля `symbol`.

В обоих рассмотренных примерах мы описывали классы со сходной структурой. Понятно, что в нашем случае ситуация создана искусственно. Тем не менее, на практике подобные случаи также встречаются достаточно часто. Альтернативой к тому, чтобы каждый раз описывать в явном виде все классы является создание *обобщенного класса*.

8.2. Обобщенные классы

Ну, хватит! Что Вы словно мальчик пускаете туман? Или Вас зовут Монте-Кристо? из к/ф "Семнадцать мгновений весны"

Обобщенный класс - класс, в котором тип данных представлен в виде параметра. Ситуация очень сходна с тем, как все происходит с обобщенными функциями. Описание обобщенного класса начинается с ключевого слова `template`, после чего в угловых скобках в комбинации с ключевым словом `class` перечисляются параметры, обозначающие обобщенные типы данных. Эти параметры используются в описании класса. Класс описывается стандартно, с поправкой на наличие в описании параметров для обозначения обобщенных типов. Синтаксическая "конструкция" для объявления обобщенного класса (с одним обобщенным типом) выглядит следующим образом (жирным шрифтом выделены ключевые элементы):

```
template <class параметр > class название{
// код класса
};
```

Например, ниже приведен пример формального объявления обобщенного класса `Alpha` с одним обобщенным типом:

```
template <class X> class Alpha{
// код класса
};
```

Еще один пример - обобщенный класс `Bravo` с двумя обобщенными типами:

```
template <class X, class Y> class Bravo{
// код класса
};
```

При создании объекта обобщенного класса необходимо в явном виде указать, какие типы следует использовать вместо параметров обобщенных типов. Посему после название класса перед именем объекта (в команде объявления объекта) в угловых скобках указывается идентификатор типа (или идентификаторы, если в классе обобщенных типов несколько). Для класса с одним обобщенным типом команда создания объекта выглядит, в общем и целом, так:

```
класс<тип> имя_объекта;
```

Скажем, объекты на основе упомянутого выше обобщенного класса `Alpha` могли бы создаваться следующими командами:

```
Alpha<int> objA;
Alpha<char> objB;
```

Объект `objA` создается так, что вместо параметра `X` используется идентификатор `int` -то есть в качестве обобщенного типа используется тип целочисленный. Объект `objB` при создании получает вместо обобщенного типа `X` символьный тип `char`.

Рассмотрим команды:

```
Bravo<int, char> objC;
Bravo<bool, double> objD;
```

Они означают буквально следующее: при создании объекта `objC` на основе обобщенного класса `Bravo` вместо обобщенного типа `X` следует использовать целочисленный тип `int`, а вместо обобщенного типа `Y` должен быть задействован символьный тип `char`. Аналогично, при создании объекта



место обобщенных типов `X` и `Y` используются соответственно типы `:double`.

На заметку

С обобщенными функциями все было несколько проще: решение о том, какие типы следует использовать вместо обобщенных параметров, принималось на основе команды вызова функции. При создании объектов класса подобный подход неприменим. Поэтому приходится в явном виде указывать идентификаторы типов данных.

Далее рассмотрим небольшой пример с обобщенным классом, у которого один обобщенный тип. Данный пример - вариация на тему самого первого примера из этой главы. Там (см. листинг 8.1) рассматривалась обобщенная функция, выполняющая обмен значениями аргументов, переданных функции. В программе, представленной в листинге 8.3, описывается обобщенный класс `MyClass` с одним параметром для обобщенного типа (обозначен как `X`). В классе описано поле `data` обобщенного типа `X`, есть конструктор с одним аргументом (аргумент конструктора относится к обобщенному типу `X` и определяет значение поля `data` создаваемого объекта). В теле конструктора присваивается значение полю `data` и вызывается метод `show()`. Методом `show()` отображается значение поля `data` объекта.

В обобщенном классе `MyClass` описан метод `swap()`. Метод не возвращает результат и ему передается (по ссылке) один аргумент (объект того же класса, что и объект, из которого вызывается метод). В результате вызова метода `swap()` происходит обмен значениями полей объекта, из которого вызывается метод, и объекта, переданного методу аргументом. В функции `main()` проверяется "функциональность" созданного обобщенного класса. Весь программный код выглядит так:

Листинг 8.3. Обобщенный класс с одним параметром

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный класс:
template <class X> class MyClass{
public:
    // Поле (обобщенного типа):
    X data;
    // Конструктор класса
    // (с аргументом обобщенного типа):
    MyClass(X field){
        // Полю присваивается значение:
        data=field;
```

```

        // Отображение значения поля:
show();
    }
    // Метод для отображения значения поля:
void show(){
cout<<"Значениеполя: "<<data<<endl;
}

    // Метод для "обмена" значениями полей объектов.
    // Аргумент метода - объект обобщенного класса,
    // который передается по ссылке:
void swap(MyClass<X>&obj){
// Создается локальная переменная:
    X tmp;
    // "Запоминается" значение поля
// объекта - аргумента метода:
tmp=obj.data;
    // Новое значение поля
    // объекта -аргумента метода:
obj.data=this->data;
    // Новое значение поля объекта,
    // из которого вызывается метод:
this->data=tmp;
}
}; // Завершение описания обобщенного класса
// Главная функция программы:
int main(){
    // Объекты создается на основе обобщенного класса:
MyClass<int>a(100),b(200);
    // Вызов метода для обмена значениями полей объектов:
a.swap(b);
cout<<"После вызова метода swap():\n";
    // Проверка значений полей объектов:
a.show();
b.show();
// Создание объектов на основе обобщенного класса:
MyClass<string> x("первый");
MyClass<string> y("второй");
// Вызов метода для обмена значениями полей объектов:
x.swap(y);
cout<<"После вызова метода swap():\n";
    // Проверка значений полей объектов:
x.show();
y.show();
return 0;
}

```

Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 8.3)

```
Значение поля: 100
Значение поля: 200
После вызова метода swap():
Значение поля: 200
Значение поля: 100
Значение поля: первый
Значение поля: второй
После вызова метода swap():
Значение поля: второй
Значение поля: первый
```

Есть несколько обстоятельств, достойные особого внимания. В первую очередь это способ описания аргумента для метода `swap()`. Пикантность ситуации связана с тем, что, как мы предполагаем, аргумент метода должен относиться к тому же классу, что и объект, из которого вызывается метод. А объект, из которого вызывается метод, создается на основе обобщенного класса `MyClass`. При создании объекта на основе обобщенного класса помимо названия класса важно и то, какие типы данных используются. Ведь понятно, что если взять один и тот же обобщенный класс, но при создании объекта использовать разные типы данных (вместо обобщенных типов), получим принципиально разные объекты. Для нас же важно, чтобы оба объекта (тот, из которого вызывается метод и тот, который передается аргументом методу) создавались на основе одного и того же обобщенного класса с использованием одного и того же типа. Мы в качестве типа объекта, переданного аргументом метода `swap()`, указали выражение `MyClass<X>`. Поскольку параметр `X` обозначает обобщенный тип, указываемый при создании объекта класса, то в данном случае инструкция `MyClass<X>` обеспечивает "единство типа" для объектов (имеются в виду объект, из которого вызывается метод и объект, переданный аргументом методу).



На заметку

Вообще, при работе с объектами, созданными на основе обобщенного класса `MyClass`, выражения вида `MyClass<int>` или `MyClass<string>` можно отождествлять с "типом" соответствующего объекта.

Что касается главной функции программы, то в ней на основе обобщенного класса `MyClass` создается несколько объектов: при создании двух объектов использован тип `int`, а два других создаются с использованием типа `string`. Из одного объекта вызывается метод `swap()`, а другой объект того же типа передается аргументом методу. Затем проверяются значения поля `data` каждого из объектов. С этой целью вызывается метод `show()`.

Еще один небольшой пример представлен в листинге 8.4. В программном коде описан обобщенный класс `MyClass` с двумя обобщенными типами (параметры `X` и `Y`). У класса имеется два поля (поле `first` обобщенного типа `X` и поле `second` обобщенного типа `Y`), конструктор с двумя аргументами (значения полей), а также метод `show()`, отображающий значения полей соответствующего объекта. Метод `show()` вызывается в конструкторе класса.

Для второго параметра (имеется в виду параметр `Y`) в описании обобщенного класса задано значение по умолчанию: после названия параметра через знак равенства указано ключевое слово `int`. Последнее означает, что если при создании объекта на основе обобщенного класса второй параметр (идентификатор типа) явно не указан, то вместо параметра `Y` используется тип `int`.

В главной функции программы на основе обобщенного класса `MyClass` создается несколько объектов. Рассмотрим программный код примера:

Листинг 8.4. Обобщенный класс с двумя обобщенными типами

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный класс:
template<class X, class Y=int> class MyClass{
public:
    // Поля (обобщенного типа):
    X first;
    Y second;
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение поля first: "<<first<<endl;
        cout<<"Значение поля second: "<<second<<endl;
    }
    // Конструктор:
    MyClass(Xf, Ys){
        first=f; // Полю присваивается значение
        second=s; // Полю присваивается значение
        show(); // Отображаются значения полей
    }
}; // Завершение описания обобщенного класса
// Главная функция программы:
int main(){
    // Объекты создаются на основе
    // обобщенного класса MyClass:
    MyClass<double, char> objA(3.5, 'A');
```

```
MyClass<int,bool> objB(10,true);
MyClass<string> objC("текст",123);
return 0;
}
```

Ниже приведен результат выполнения программного кода:

Результат выполнения программы (из листинга 8.4)

```
Значение поля first: 3.5
Значение поля second: A
Значение поля first: 10
Значение поля second: 1
Значение поля first: текст
Значение поля second: 123
```

Поскольку в конструкторе класса вызывается метод `show()`, то при создании объектов отображаются значения полей `first` и `second`. В главной функции программы создается три объекта. В первых двух случаях используемые вместо обобщенных типов идентификаторы указываются явно (то есть указывается два идентификатора типов), а при создании третьего объекта в угловых скобках указывается только один идентификатор типа. В этом случае по умолчанию второй тип будет целочисленным.

8.3. Перегрузка и явная специализация обобщенных функций

*Перезагрузка завершена. Счастливого пути!
из к/ф "Гостя из будущего"*

Обобщенные функции можно *перегружать*. Ранее мы имели дело с перегрузкой функций, но тогда речь шла об обычных (не обобщенных) функциях. Если функция обобщенная, то у нее, фактически, два набора "аргументов": непосредственно аргументы функции и параметры, обозначающие обобщенные типы. Правило перегрузки обобщенных функций состоит в том, что допускается создание нескольких функций с одинаковыми названиями, но разным набором аргументов и параметров. Другими словами, перегрузка обобщенных функций может затрагивать не только непосредственно аргументы функции, но и параметры, обозначающие обобщенные типы.

Помимо перегрузки обобщенных функций, существует понятие *явной специализации* обобщенных функций. Явная специализация - это когда для обобщенной функции явно описывается необобщенная версия. Необоб-

щенная версия обобщенной функции - функция с таким же названием, только все типы в ней указаны явно и однозначно. Необобщенных версий может быть несколько.



На заметку

В известном смысле явную специализацию обобщенной функции допустимо рассматривать как своеобразное "исключение из правил": существует определенный шаблон (определяемый обобщенной функцией) с некоторым общим алгоритмом выполнения функции, но для определенных типов (тех, что использованы при явной специализации) алгоритм выполнения функции несколько отличается от шаблонного.

В качестве иллюстрации к перегрузке обобщенных функций и явной их специализации рассмотрим программный код, представленный в листинге 8.5.

Листинг 8.5. Перегрузка и явная специализация обобщенных функций

```
#include <iostream>
#include <string>
using namespace std;
// Классстекстовымполем:
class Alpha{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    Alpha(string txt){
        name=txt; // Полю присваивается значение
    }
    // Объекты класса:
}objA("первый"), objB("второй");
// Обобщенная функция с двумя параметрами
// и двумя аргументами разного типа:
template <class X, class Y> void show(X a, Y b){
    cout<<"У функции два аргумента разного типа!\n";
    cout<<"1-й аргумент: "<<a<<endl;
    cout<<"2-й аргумент: "<<b<<endl;
}
// Версия обобщенной функции с одним параметром
// и двумя аргументами одного типа:
template <class X> void show(X a, X b){
    cout<<"У функции два аргумента одного типа!\n";
    cout<<"1-й аргумент: "<<a<<endl;
    cout<<"2-й аргумент: "<<b<<endl;
```

```

}
// Версия обобщенной функции с одним параметром
// и одним аргументом:
template <class X> void show(X a){
cout<<"У функции один аргумент!\n";
cout<<"Значение аргумента: "<<a<<endl;
}
// Версия функции с двумя аргументами.
// Первый аргумент - объект класса Alpha.
// Тип второго аргумента определяется параметром:
template<class Y> void show(Alpha a,Y b){
cout<<"У функции два аргумента (первый - объект)!\n";
cout<<"Поле name объекта: "<<a.name<<endl;
cout<<"2-й аргумент: "<<b<<endl;
}
// Версия функции с двумя аргументами.
// Тип первого аргумента определяется параметром.
// Второй аргумент - объект класса Alpha:
template<class X> void show(X a,Alpha b){
cout<<"У функции два аргумента (второй - объект)!\n";
cout<<"1-й аргумент: "<<a<<endl;
cout<<"Поле name объекта: "<<b.name<<endl;
}
// Версия функции с двумя аргументами - объектами
// класса Alpha:
void show(Alpha a,Alpha b){
cout<<"Аргументы функции - объекты класса Alpha!\n";
cout<<"Поле name 1-го объекта: "<<a.name<<endl;
cout<<"Поле name 2-го объекта: "<<b.name<<endl;
}
// Версия функции с одним аргументом - объектом
// класса Alpha:
void show(Alpha a){
cout<<"Аргумент функции - объект класса Alpha!\n";
cout<<"Значение поля name объекта: "<<a.name<<endl;
}
// Версия функции без аргументов:
void show(){
cout<<"У функции нет аргументов!\n";
}
// Главная функция:
int main(){
    // Различные способы вызова функции show().
    // Аргументы типа int и string:
    show(100,"текст");
    cout<<endl;

```

```

    // Аргументы типа double:
show(12.5,10.3);
cout<<endl;
    // Аргумент типа int:
show(200);
cout<<endl;
    // Объект класса Alpha и тип char:
show(objA,'Z');
cout<<endl;
// Аргумент типа char и объект класса Alpha:
show('A',objB);
cout<<endl;
    // Объекты класса Alpha:
show(objA,objB);
cout<<endl;
    // Объект класса Alpha:
show(objA);
cout<<endl;
    // Безаргументов:
show();
return 0;
}

```

Результат выполнения программы следующий:

Результат выполнения программы (из листинга 8.5)

У функции два аргумента разного типа!

1-й аргумент: 100

2-й аргумент: текст

У функции два аргумента одного типа!

1-й аргумент: 12.5

2-й аргумент: 10.3

У функции один аргумент!

Значение аргумента: 200

У функции два аргумента (первый - объект)!

Поле name объекта: первый

2-й аргумент: Z

У функции два аргумента (второй - объект)!

1-й аргумент: A

Поле name объекта: второй

Аргументы функции - объекты класса Alpha!
 Поле name 1-го объекта: первый
 Поле name 2-го объекта: второй

Аргумент функции - объект класса Alpha!
 Значение поля name объекта: первый

У функции нет аргументов!

Программа очень простая: мы описали класс `Alpha` с текстовым полем `name` и несколько версий функции `show()`. В частности, описана обобщенная функция с таким именем и двумя аргументами. Есть версия функции с одним аргументом обобщенного типа, а также описана версия функции с двумя аргументами одного обобщенного типа. Еще имеется вариант функции без аргументов, с одним аргументом - объектом класса `Alpha`, двумя аргументами -объектами класса `Alpha`, и два варианта функции, у которой один из аргументов является объектом класса `Alpha`, а второй определяется через параметр (относится к обобщенному типу).

В общем и целом функцией `show()` при вызове отображаются значения аргументов функции (с разными поясняющими комментариями). При вызове функции нужная ее версия определяется на основе команды вызова. Например, если аргументами функции указаны значения типа `int` и `string`, то вызывается обобщенная функция `show()` с двумя параметрами, вместо которых соответственно используются типы `int` и `string`.

При обработке команды с вызовом функции `show()` с двумя аргументами типа `double` будет задействована версия функции с одним обобщенным параметром и двумя аргументами. Тип аргументов определяется указанным параметром.



На заметку

Если бы в программе не была описана версия обобщенной функции `show()` с одним параметром и двумя аргументами одного типа, то при вызове функции `show()` с двумя аргументами типа `double` использовалась бы версия обобщенной функции с двумя параметрами, и каждый из них интерпретировался бы как тип `double`.

В случае, когда один или оба аргумента функции являются объектами класса `Alpha`, то для такого объекта (или объектов) отображается значение поля `name`.



На заметку

Объекты `objA` и `objB` класса `Alpha` создаются не в функции `main()`, а перечисляются в списке объектов сразу после описания класса.

Вообще же принцип перегрузки и явной специализации обобщенных функций понять несложно: описываются различные варианты функции, а при вызове выбирается подходящая версия функции. Главное, чтобы этот выбор идентифицировался однозначно. Например, предположим, что в программе была описана версия функции `show()` с первым аргументом - объектом класса `Alpha` и вторым аргументом обобщенного типа, а также версия функции `show()` с первым аргументом обобщенного типа и вторым аргументом - объектом класса `Alpha`. Если при этом не описана версия функции с двумя аргументами - объектами класса `Alpha`, то вызов функции `show()` с двумя объектами класса `Alpha` в качестве аргументов привел бы к ошибке. Причина в том, что невозможно определить, какая версия функции должна быть вызвана: та, в которой аргумент обобщенного типа второй или первый.

8.4. Явная специализация обобщенных классов

*Когда человек теряет чувство юмора, невозможно предугадать, что он натворит.
из к/ф "Старики-разбойники"*

При работе с обобщенными классами помимо "основной" обобщенной версии класса (версии, содержащей параметры для обобщенных типов) можно описать версии классов для каких-то конкретных значений параметров (то есть для конкретных типов данных описывается "отдельная" версия обобщенного класса). По аналогии с обобщенными функциями, будем говорить в этих случаях о *явной специализации* обобщенного класса.



На заметку

Не следует путать явную специализацию класса с использованием значения по умолчанию для параметров обобщенного класса. При явной специализации класса создается особая версия класса для случая, когда обобщенный тип принимает то или иное значение. Если же для параметров класса указаны значения по умолчанию, то во всех случаях используется один и тот же шаблонный код, просто значение параметров разрешается явно не указывать (тогда используются значения по умолчанию).

При явной специализации обобщенного класса соответствующая версия класса описывается как обобщенная (начинается с ключевого слова `template`), но в угловых скобках ключевое слово `class` и параметр для обобщенного типа не указываются - угловые скобки пустые. А вот после имени класса в угловых скобках указывается тип, для которого выполняет-

ся явная специализация класса. В общем, явная специализация для класса выполняется так (жирным шрифтом выделены ключевые элементы):

```
template<>classназвание<тип>{
// код класса
};
```

Пример с иллюстрацией явной специализации класса, представлен в листинге 8.6.

Листинг 8.6. Явная специализация класса

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный класс (параметр типа
// имеет значение по умолчанию):
template <class X=char> class MyClass{
public:
    // Поле обобщенного типа:
    X data;
    // Конструктор:
    MyClass(X d){
        // Полю присваивается значение:
        data=d;
    }
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значениеполя data: "<<data<<endl;
    }
}; // Завершение описания обобщенного класса
// Явная специализация обобщенного класса:
template <> class MyClass<string>{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    MyClass(string s){
        // Полю присваивается значение:
        name=s;
    }
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение текстового поля name: "<<name<<endl;
    }
}; // Завершение описания класса
```

```
// Главная функция программы:
int main() {
    // Объект класса с типом int:
    MyClass<int> A(123);
    A.show();
    // Объект класса с типом string:
    MyClass<string> B("текст");
    B.show();
    // Объект класса с типом char (значение по умолчанию):
    MyClass<> C('Я');
    C.show();
    return 0;
}
```

Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 8.6)

```
Значение поля data: 123
Значение текстового поля name: текст
Значение поля data: Я
```

В программе описан обобщенный класс `MyClass` с одним параметром, причем параметр имеет значение по умолчанию `char`. Последнее обстоятельство означает, что если при создании объекта класса явно тип не указать, то для параметра обобщенного типа используется значение `char`.

В классе `MyClass` описан конструктор с одним аргументом, и в конструкторе переданное аргументом значение присваивается полю с названием `data`. Поле относится к обобщенному типу. В классе `MyClass` есть метод `show()`, отображающий значение поля `data`.

Явная специализация класса `MyClass` выполняется для типа `string`. В этом случае вместо поля `data` появляется текстовое поле `name`. В остальных программные коды разных версий класса `MyClass` схожи. Принцип создания объектов обобщенного класса следующий: если класс создается для типа, отличного от `string`, то используется "основной" код обобщенного класса. Если же при создании объекта обобщенного класса указан тип `string`, то используется версия класса `MyClass`, описанная для явной его специализации. Примеры различных способов создания объектов на основе обобщенного класса `MyClass` представлены в функции `main()`.

Ситуация может быть не такой однозначной. Далее мы рассмотрим пример, в котором обобщенный класс содержит два параметра для обозначения обобщенных типов, а явная специализация выполняется для двух случаев:

- если обобщенные типы равны `int` и `string`;
- если второй обобщенный тип равен `string` (а первый - произвольный);

То есть во втором случае выполняется явная специализация только по второму параметру (первый остается заданным формально). Если так, то будем говорить о *явной частичной специализации класса*.

Рассмотрим программный код, представленный в листинге 8.7.

Листинг 8.7. Явная частичная специализация класса

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный класс (с двумя параметрами типа):
template <class X, class Y> class MyClass{
public:
    // Первое поле обобщенного типа:
    X first;
    // Второе поле обобщенного типа:
    Y second;
    // Конструктор:
    MyClass(X f, Y s){
    // Полям присваиваются значения:
    first=f;
    second=s;
    }
    // Метод для отображения значений полей:
    void show(){
    cout<<"Значениеполя first: "<<first<<endl;
    cout<<"Значениеполя second: "<<second<<endl;
    }
}; // Завершение описания обобщенного класса
// Явная частичная специализация обобщенного класса:
template <class X> class MyClass<X, string>{
public:
    // Первое поле обобщенного типа:
    X first;
    // Текстовое поле:
    string name;
    // Конструктор:
    MyClass(X f, string s){
    // Полям присваиваются значения:
    first=f;
    name=s;
```

```

    }
    // Метод для отображения значений полей:
void show(){
cout<<"Значение поля first: "<<first<<endl;
cout<<"Значение текстового поля name: "<<name<<endl;
}
}; // Завершение описания класса
// Явная специализация класса:
template<> class MyClass<int,string>{
public:
// Целочисленное поле:
int code;
// Текстовое поле:
string name;
// Конструктор:
MyClass(int c,string s){
// Полям присваиваются значения:
code=c;
name=s;
}
// Метод для отображения значений полей:
void show(){
cout<<"Значение целочисленного поля: "<<code<<endl;
cout<<"Значение текстового поля: "<<name<<endl;
}
}; // Завершение описания класса
// Главная функция программы:
int main(){
// Объект класса с типами int и char:
MyClass<int,char> A(123,'Я');
A.show();
// Объект класса с типами char и string:
MyClass<char,string> B('A',"текст");
B.show();
// Объект класса с типами int и string:
MyClass<int,string> C(100,"новый текст");
C.show();
return 0;
}

```

В результате выполнения программного кода получаем:

Результат выполнения программы (из листинга 8.7)

Значение поля first: 123
Значение поля second: Я

Значение поля `first`: А
 Значение текстового поля `name`: текст
 Значение целочисленного поля: 100
 Значение текстового поля: новый текст

Обобщенный класс `MyClass` описан с двумя параметрами типа `X` и `Y`. В классе есть два поля (поле `first` обобщенного типа `X` и поле `second` обобщенного типа `Y`). Конструктору передаются два аргумента, присваиваемые значениями полям. В классе описан метод `show()`. При его вызове отображаются значения полей объекта, из которого вызывается метод.

Явная специализация класса `MyClass` выполняется для типов `int` и `string`. Заголовок описания класса теперь выглядит как `template <> class MyClass<int, string>`: после ключевого слова `template` пустые угловые скобки, затем ключевое слово `class`, название класса и в угловых скобках названия типов, для которых выполняется явная специализация. Что касается программного кода данной версии класса `MyClass`, то в теле класса пописаны поля `code` и `name` - соответственно целочисленное и текстовое. Конструктором присваиваются значения полям, а при вызове метода `show()` значения полей отображаются в окне вывода.

При явной частичной специализации класса `MyClass` в описании версии класса использован заголовок `template <class X> class MyClass<X, string>`. Фактически мы имеем дело с шаблоном, в котором один обобщенный тип `X`, а другой тип указан явно. Здесь первое поле `first` класса относится к обобщенному типу, а второе поле с названием `name`, относится к текстовому типу `string`. Как и в предыдущих случаях метод `show()` предназначен для отображения значений полей.

При создании в главной функции программы объектов на основе обобщенного класса `MyClass` традиционно указываются типы данных для использования при создании объекта обобщенного класса.



На заметку

Имеет смысл напомнить, что хотя разные объекты создаются на основе одного и того же обобщенного класса, принципиальное значение имеет то, какие именно типы данных были использованы для передачи параметрами классу. Если, например, один объект создается командой `MyClass<int, char> A(123, 'Я')`, а другой создается командой `MyClass<char, string> B('А', "текст")`, то речь фактически идет об объектах разных типов: объект `A` относится к типу `MyClass<int, char>`, в то время как объект `B` относится к типу `MyClass<char, string>`.

8.5. Обобщенные классы и наследование

*Ну, а это довесок к кошмару.
из к/ф "Старики-разбойники"*

Обобщенные классы можно использовать при наследовании. Мы рассмотрим простой пример, в котором на основе одного обобщенного базового класса создается, путем наследования, другой базовый класс - тоже обобщенный. Соответствующий программный код приведен в листинге 8.8.

Листинг 8.8. Обобщенные классы и наследование

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный базовый класс с двумя параметрами:
template<class X,class Y> class Alpha{
public:
    // Поля:
    X dataX;
    Y dataY;
    // Конструктор:
    Alpha(X dx,Ydy) {
    // Полям присваиваются значения:
    dataX=dx;
    dataY=dy;
    }
    // Метод для отображения значения полей:
    void show(){
    cout<<"Первое поле: "<<dataX<<endl;
    cout<<"Второе поле: "<<dataY<<endl;
    }
}; // Завершение описания базового обобщенного класса
// Обобщенный производный класс:
template <class X,classY,class Z> class Bravo: public
Alpha<X,Y>{
public:
    // Поле:
    Z dataZ;
    // Конструктор:
    Bravo(Xdx,Ydy, Zdz): Alpha<X,Y>(dx,dy){
        // Полю присваивается значение:
        dataZ=dz;
        // Вызов метода для отображения значений полей:
        show();
    }
};
```



```
// Переопределение метода из базового класса:
void show(){
    // Вызов версии метода из базового класса:
    Alpha<X,Y>::show();
    // Отображение значения поля:
    cout<<"Третье поле: "<<dataZ<<endl;
}
}; // Завершение описания производного обобщенного класса
// Главная функция программы:
int main(){
    // Создание объекта на основе
    // производного обобщенного класса
    Bravo<int,char,string>obj(100,'A',"текст");
    return 0;
}
```

Ниже приведен результат выполнения программы:

Результат выполнения программы (из листинга 8.8)

```
Первое поле: 100
Второе поле: A
Третье поле: текст
```

В программе описывается обобщенный класс `Alpha` (заголовок `template <class X,class Y> class Alpha`) с двумя параметрами `X` и `Y` для обозначения обобщенных типов, используемых в классе. Данный класс мы используем в качестве базового, создав на его основе производный обобщенный класс `Bravo`.



На заметку

В классе `Alpha` описаны два поля `dataX` и `dataY` соответственно обобщенных типов `X` и `Y`. У конструктора класса `Alpha` два аргумента, определяющие значения полей `dataX` и `dataY`. Методом `show()` отображаются значения полей `dataX` и `dataY`.

При создании класса `Bravo` путем наследования класса `Alpha` использован заголовок `template <class X,class Y,class Z> class Bravo: public Alpha<X,Y>`. Здесь есть несколько моментов, достойных внимания. Во-первых, производный класс мы описываем как обобщенный, с указанием трех параметров для обозначения обобщенных типов (параметры `X`, `Y` и `Z`). Во-вторых, базовый класс, на основе которого создается производный класс, определен как `Alpha<X,Y>`.

Мы указываем не просто имя базового класса, а имя базового класса с обозначением значений параметров обобщенных типов, используемых в классе `Alpha`. В данном случае первые два параметра (`X` и `Y`) из класса `Bravo` "передаются" в класс `Alpha` и используются в качестве значений параметров для обобщенных типов.



На заметку

Вообще, программный код данного примера легче понять, если думать, что класс `Bravo` создается на основе класса `Alpha<X,Y>`. Другими словами, описание класса `Bravo` - это описание класса, наследующего класс `Alpha<X,Y>`.

В теле класса `Bravo` описывается поле `dataZ` обобщенного типа `Z`. Еще в классе `Bravo` наследуются два поля `dataX` и `dataY` из класса `Alpha`.

У конструктора класса `Bravo` три аргумента - в соответствии с количеством полей. Каждый аргумент конструктора - значение соответствующего поля. Значения полям `dataX` и `dataY` присваиваются при вызове конструктора базового класса. Формально инструкция вызова конструктора имеет вид `Alpha<X,Y>(dx,dy)`: эта инструкция указывается после названия конструктора класса `Bravo` (через `dx` и `dy` обозначены первые два аргумента конструктора класса `Bravo`). Здесь нет ничего особенного, за исключением пожалуй того момента, что название базового класса указано в виде `Alpha<X,Y>`.

Значение полю `dataZ` присваивается в теле конструктора командой `dataZ=dz`. Также в теле конструктора вызывается метод `show()`. Но это не та версия метода `show()`, которая наследуется в классе `Bravo` из класса `Alpha`. Все дело в том, что метод `show()` в классе `Bravo` переопределяется. Причина очевидна: в классе `Alpha` метод описан таким образом, что при его вызове отображаются значения полей `dataX` и `dataY`. В классе `Bravo` появляется еще одно поле, поэтому желательно, чтобы методом `show()` значение данного поля отображалось тоже. Как следствие, метод переопределяется, причем переопределяется так: сначала вызывается старая версия метода (та версия, что описана в классе `Alpha`), а затем отдельной командой отображается значение поля `dataZ`. Старая версия метода `show()` вызывается командой `Alpha<X,Y>::show()`. В данном случае перед названием метода `show()`, через оператор расширения контекста (оператор `::`), указывается название класса с описанным методом - в нашем случае это класс `Alpha<X,Y>`.

В главной функции программы командой `Bravo<int,char,string> obj(100,'A',"текст")` создается объект `obj` на основе обобщенного производного класса `Bravo`. Поскольку в конструкторе класса вызывается

метод `show()`, то в окне вывода отображается информация о значении полей созданного объекта.

На основе обобщенного класса путем наследования можно создавать не только обобщенные, но и обычные классы. Небольшой пример по этому поводу приведен в листинге 8.9. В программе описывается шаблонный класс `Alpha` с одним параметром, а затем на основе класса создается два производных класса: класс `Bravo` создается на основе класса `Alpha<int>`, а класс `Charlie` создается на основе класса `Alpha<char>`.

Листинг 8.9. Создание обычных классов наследованием обобщенного класса

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный базовый класс:
template <class X> class Alpha{
public:
    // Поле обобщенного типа:
    X data;
    // Конструктор:
    Alpha(X d){
        // Значение поля:
        data=d;
    }
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение поля data: "<<data<<endl;
    }
}; // Завершение описания обобщенного базового класса
// Производный класс:
class Bravo: public Alpha<int>{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    Bravo(int n): Alpha<int>(n){
        // Значение поля:
        name="Объект класса Bravo";
        // Отображение значений полей:
        show();
    }
    // Переопределение метода:
    void show(){
        // Отображение значения поля name:
        cout<<name<<endl;
```

```
// Вызов исходной версии метода:
Alpha<int>::show();
}
}; // Завершение описания производного класса
// Производный класс:
class Charlie: public Alpha<char>{
public:
    // Текстовое поле:
    string name;
    // Конструктор:
    Charlie(char n): Alpha<char>(n){
    // Значение текстового поля:
    name="Объект класса Charlie";
    // Отображение значений полей:
    show();
    }
    // Переопределение метода:
    void show(){
        // Значение поля:
        cout<<name<<endl;
    // Вызов исходной версии метода:
    Alpha<char>::show();
    }
}; // Завершение описания производного класса
// Главная функция программы:
int main(){
    // Объект класса Bravo:
    Bravo objB(100);
    //Объект класса Charlie:
    Charlie objC('A');
    return 0;
}
```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 8.9)

```
Объект класса Bravo
Значение поля data: 100
Объект класса Charlie
Значение поля data: A
```

Классы `Bravo` и `Charlie`, полученные на основе обобщенного класса `Alpha`, являются обычными (не обобщенными), хотя при этом имеют поля разного типа и идентичную "структуру", которая во многом определяется базовым обобщенным классом.

Глава 9.

ФУНКТОРЫ



Нас всех губит отсутствие дерзости в перспективном видении проблем. Мы не можем себе позволить фантазировать.

из к/ф "Семнадцать мгновений весны"

В этой главе мы обсудим вопросы, связанные с использованием *функторов*. Если кратко, то функтор - объект, который вызывается подобно функции. Выражение "использование функторов" следует понимать в самом широком смысле, поскольку мы затронем аспекты, на первый взгляд имеющие весьма отдаленное отношение к функторам. Однако, как будет показано далее, в умелых руках практически любой механизм окажется полезным. Традиционно, основное внимание уделяется методам ООП.

9.1. Знакомство с функторами

Софистика, пастор, софистика!

из к/ф "Семнадцать мгновений весны"

Итак, *функтор* - объект со свойствами функции. Главное свойство функции - ее можно вызывать. Поэтому не будет большой ошибкой назвать функтором объект, допускающий вызов, как обычная функция: если после имени объекта указать круглые скобки (с аргументами или без), получим некоторый результат (или не получим, если результат не предусмотрен).

Технически для того, чтобы объект можно было вызывать наподобие функции, в классе, на основе которого создается объект, должен быть перегружен оператор `()` ("круглые скобки"). Другими словами, в таком классе должен быть описан операторный метод с именем `operator()`. Операторный метод предназначен для обработки выражений вида `объект(аргумент)`. Объект, указанный перед круглыми скобками, автоматически считается первым операндом выражения. Это объект, из которого вызывается операторный метод. Аргумент операторного метода - значение, указываемое в круглых скобках.



На заметку

Как отмечалось ранее, для перегрузки операторов могут использоваться не только операторные методы, но и операторные функции. Однако есть группа операторов, которые перегружаются только с помощью операторных методов. Среди опера-

торов, рассматриваемых (и перегружаемых) в книге, к этой группе относятся оператор присваивания=, оператор "квадратные скобки" [] и оператор "круглые скобки" ().

Следовательно, оператор "круглые скобки" перегружается явным описанием в классе операторного метода с именем `operator()`. Только так, и никак иначе.

Начнем с рассмотрения очень простой ситуации: со степенной зависимости вида $f(x) = x^n$. Вместо описания в программном коде функции с двумя аргументами (имеются в виду параметры x и n) мы опишем класс с закрытым целочисленным полем (значение параметра n) и перегруженным оператором «круглые скобки». Последний описывается с числовым аргументом, соответствующим параметру x . Поскольку в классе перегружен оператор с названием `operator()`, то объекты данного класса вызываются, как функции: после имени объекта в круглых скобках указывается значение параметра x , а результатом всего выражения является числовое значение x^n , причем значение показателя степени n функтор «считывает» со своего одноименного закрытого поля.

Программный код примера приведен в листинге 9.1. Там описан класс `MyPow` закрытым целочисленным полем `n`, конструктором с одним аргументом (аргумент конструктора определяет значение закрытого поля), а также методом `help()`, отображающем справку по объекту (имеется в виду информация о значении закрытого поля `n`). Еще класс содержит перегрузку операторного метода. Весь программный код выглядит так:

Листинг 9.1. Функтор для вычисления степенной зависимости

```
#include <iostream>
using namespace std;
// Класс для создания функтора:
class MyPow{
private:
    // Закрытое поле класса:
    int n;
public:
    // Конструктор с одним аргументом:
    MyPow(int n){
        // Значение поля:
        this->n=n;
    }
    // Метод для отображения "справки":
```

```

void help(){
cout<<"Функтор для возведения в степень "<<n<<endl;
}
    // Перегрузка операторного метода "круглые скобки".
    // Результат метода - аргумент x в целочисленной
    // степени n (закрытое поле функтора):
double operator()(double x){
double s=1;
for(int i=1;i<=n;i++){
s*=x;
}
return s;
}
}; // Завершение описания класса
// Главная функция программы:
int main(){
    // Переменная для передачи аргументом функтору:
double x=3;
    // Создание объекта-функтора:
MyPowf(5);
    // Информация о функторе:
f.help();
    // Вызов функтора:
cout<<"x="<<x<<": "<<f(x)<<endl;
// Новое значение переменной:
x=2;
    // Создание объекта-функтора:
MyPowF(7);
    // Информация о функторе:
F.help();
    // Вызов функтора:
cout<<"x="<<x<<": "<<F(x)<<endl;
return 0;
}

```

Программный код операторного метода с названием `operator()` достаточно прост. В качестве результата методом возвращается значение типа `double`, которое вычисляется (с помощью оператора цикла) как n -кратное произведение аргумента x метода.



На заметку

Класс `MyPow` описан так, что значение закрытого поля `n` определяется единожды при создании объекта класса путем передачи аргумента конструктору. После этого изменить значение поля нельзя. Его можно лишь прочитать с помощью метода `help()`.

В главной функции программы создаются два объекта класса `MyPow` (функтуры), а затем с помощью "вызова" этих объектов вычисляются степенные выражения. Результат выполнения программы такой:

Результат выполнения программы (из листинга 9.1)

```
Функтор для возведения в степень 5
x=3: 243
Функтор для возведения в степень 7
x=2: 128
```

Понятно, что выше представлен очень простой и "прямолинейный" способ описания и использования функтора. Могут быть более специфичные ситуации. Но все они в конечном итоге базируются на перегрузке операторного метода для "круглых скобок". Для большей наглядности рассмотрим еще несколько примеров.

9.2. Функторы с аргументами и без аргументов

*Это простейшая цепь рассуждений.
из к/ф "Приключения Шерлока Холмса и
доктора Ватсона"*

"История" следующей программы простирается к самым первым главам книги, где мы рассматривали задачу о вкладчике и его счете в банке.



На заметку

Напомним, что речь идет о размещении вклада в денежных единиц под годовую процентную ставку r (в процентах) на время t (в годах). В этом случае итоговая сумма вклада вычисляется по формуле $M \left(1 + \frac{r}{100} \right)^t$.

Ранее мы описывали класс `BankAccount` с тремя полями `money`, `rate` и `time` (соответственно величина начального вклада, процентная ставка и время, на которое размещается вклад), а для вычисления итоговой суммы вклада описывался специальный метод. Теперь же мы пойдем несколько иным путем и вместо метода предусмотрим возможность вызова объекта класса. Рассмотрим программный код в листинге 9.2.

Листинг 9.2. Функтор на основе класса BankAccount

```

#include <iostream>
using namespace std;
// Класс:
class BankAccount{
// Закрытые поля класса:
private:
    // Величина вклада:
    double money;
    // Процентная ставка:
    double rate;
    // Время:
    int time;
    // Открытые члены класса:
public:
    // Операторный метод для "круглых скобок"
    // (версия без аргументов):
    double operator() () {
        double res=money;
        int i;
        for(i=1;i<=time;i++){
            res=res*(1+rate/100);
        }
        return res;
    }
    // Операторный метод для "круглых скобок"
    // (версия с тремя аргументами):
    void operator() (double m,double r,double t){
        money=m;
        rate=r;
        time=t;
    }
    // Конструктор класса (с тремя аргументами):
    BankAccount(double m=100,double r=13,int t=3){
        // Неявный вызов операторного метода:
        (*this)(m,r,t);
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Первый объект:
    BankAccount ivanov;
    // Вызывается операторный метод без аргументов:
    cout<<"Иванов: "<<ivanov()<<endl;
    // Вызывается операторный метод с тремя аргументами:

```

```
ivanov(120,12,2);
// Вызывается операторный метод без аргументов:
cout<<"Иванов: "<<ivanov()<<endl;
// Второй объект:
BankAccount petrov(90,18,4);
// Вызывается операторный метод без аргументов:
cout<<"Петров: "<<petrov()<<endl;
// Вызывается операторный метод с тремя аргументами:
petrov(130,15,5);
// Вызывается операторный метод без аргументов:
cout<<"Петров: "<<petrov()<<endl;
return 0;
}
```

В классе `BankAccount` (нынешней его версии) три закрытых действительных поля (`money`, `rate` и `time`), а также в открытом блоке конструктор и две версии операторного метода `operator()()` (одна круглая скобка относится к названию метода, вторая - формальный признак метода). Одна версия операторного метода описывается без аргументов и возвращает результат: это значение итоговой суммы депозита, вычисляемое на основе закрытых полей объекта (функтора), из которого вызывается метод. Другая версия операторного метода не возвращает результат и принимает три аргумента. Данная версия операторного метода предназначена для присваивания значений закрытым полям объекта (функтора).



На заметку

Поля у объекта закрытые, но операторный метод описан в открытом блоке, поэтому можем использовать операторный метод для доступа к закрытым полям.

В теле конструктора выполняется присваивание значений полям объекта. Причем для выполнения этой процедуры в конструкторе вызывается операторный метод с тремя аргументами - цель достигается с помощью команды `(*this)(m,r,t)`. Здесь фактически мы используем неявный вызов операторного метода с тремя аргументами. Инstrukция `*this` означает объект вызова (в данном случае создаваемый объект). В круглых скобках после инструкции `*this` указаны аргументы конструктора. Обработывается такое выражение через вызов операторного метода `operator()()` с тремя аргументами, в результате чего полям объекта присваиваются значения.



На заметку

Инstrukция `*this` взята в круглые скобки с учетом приоритета операторов, чтобы выражение обрабатывалось правильно.

В главной функции программы создаются два объекта. Эти объекты "вызываются" с аргументами и без, соответственно, для вычисления итоговой суммы депозита и присваивания новых значений полям объектов. Результат выполнения программы такой:

Результат выполнения программы (из листинга 9.2)

Иванов: 144.29
Иванов: 150.528
Петров: 174.49
Петров: 261.476

9.3. Реализация полинома через функтор

*Это мелочи. Но нет ничего важнее мелочей!
из к/ф "Приключения Шерлока Холмса и
доктора Ватсона"*

Следующий пример, по сравнению с предыдущими, имеет большую математическую направленность: в нем программными методами, путем создания специального класса и перегрузки ряда операторов (включая и оператор "круглые скобки"), реализуются некоторые операции с полиномами.



На заметку

Полиномом степени n от переменной x называется выражение $P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$. Параметры $a_0, a_1, \dots, a_n$ называются коэффициентами полинома и однозначно определяют его как функциональную зависимость. Чтобы вычислить значение полинома в заданной точке x достаточно знать коэффициенты полинома.

Программный код примера представлен в листинге 9.3. Там описан класс `Polynomial`. В классе набор коэффициентов полинома реализуется в виде динамического массива. Кроме непосредственно операторного метода `operator()()`, в классе описаны операторные методы для операторов присваивания и индексирования объектов. Все это значительно облегчает работу с объектами данного класса. Рассмотрим программный код:

Листинг 9.3. Функтор для работы с полиномами

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Класс для реализации полиномов:
class Polynomial{
```

```

    // Закрытые поля класса:
private:
    // Указатель на массив с коэффициентами полинома:
double *a;
    // Степень полинома:
int power;
    // Открытые члены класса:
public:
    // Конструктор (аргументы - указатель на массив
    // и степень полинома):
Polynomial(double *b,int n){
power=n; // Степень полинома
    // Создается динамический массив
    // для коэффициентов полинома:
a=new double[power+1];
    // Заполнение массива:
for(int i=0;i<=power;i++){
a[i]=b[i];
    }
}
    // Конструктор (аргумент - степень полинома):
Polynomial(int n=0){
    // Степень полинома:
power=n;
    // Массив для коэффициентов полинома:
a=new double[power+1];
    // Заполнение массива нулями:
for(int i=0;i<=power;i++){
a[i]=0;
    }
}
    // Конструктор создания копии:
Polynomial(Polynomial &P){
    // Степень полинома:
power=P.getPower();
    // Создание динамического массива
    // для коэффициентов полинома:
a=new double[power+1];
    // Заполнение массива:
for(int i=0;i<=power;i++){
a[i]=P[i];
    }
}
    // Деструктор:
~Polynomial(){
    // Удаление массива:

```

```

delete [] a;
}
// Метод возвращает значение для степени полинома:
int getPower(){
return power;
}
// Метод для отображения коэффициентов полинома:
void coefs(){
for(int i=0;i<=power;i++){
cout<<a[i]<<"\t";
}
cout<<endl;
}
// Операторный метод для индексирования объектов:
double &operator[](int k){
return a[k];
}
// Операторный метод для "вызова" объектов:
double operator()(double x){
// Локальные переменные
// (значение полинома и степенное слагаемое):
double s=0,q=1;
// Вычисление значения полинома:
for(int i=0;i<=power;i++){
s+=a[i]*q; // Добавка к сумме
q*=x;      // Степенное слагаемое
}
// Значение полинома:
return s;
}
// Операторный метод для умножения полиномов:
Polynomial &operator*(Polynomial P){
// Указатель на объект:
Polynomial *t;
// Степень полинома - аргумента метода:
int n=P.getPower();
// Степень полинома - результата произведения:
int m=power+n;
// Новый динамический объект
// (все коэффициенты нулевые):
t=new Polynomial(m);
// Заполнение массива с коэффициентами для
// полинома - результата произведения.
// Перебор коэффициентов 1-го полинома:
for(int i=0;i<=power;i++){
// Перебор коэффициентов 2-го полинома:

```

```

for(int j=0;j<=n;j++){
// "Уточнение" коэффициентов
    // полинома - результата произведения:
    (*t)[i+j]+=a[i]*P[j];
}
}
// Результат метода - объект:
return *t;
}
// Операторприсваивания:
Polynomial operator=(PolynomialP){
// Удаление массива коэффициентов:
delete [] a;
// Степень полинома:
power=P.getPower();
// Новый массив коэффициентов:
a=new double[power+1];
// Заполнениемассива:
for(int i=0;i<=power;i++){
a[i]=P[i];
}
// Результат метода:
return *this;
}
}; // Завершение описания класса
// Главная функция программы:
int main(){
// Инициализация генератора
// случайных чисел:
srand(2014);
// Степень первого полинома:
const int n=2;
// Массив коэффициентов для полинома:
double nums[n+1]={3,-2,1};
cout<<"Полиномы\n";
// Объект для первого полинома:
Polynomial P(nums,n);
cout<<"P: ";
// Коэффициенты первого полинома:
P.coefs();
// Объект для второго полинома:
Polynomial Q(n+1);
cout<<"Q: ";
// Коэффициенты второго полинома:
Q.coefs();
// Коэффициенты полинома - случайные числа:

```

```

for(int i=0;i<=n+1;i++){
    Q[i]=rand()%5-2;
}
cout<<"Q: ";
// Коэффициенты второго полинома:
Q.coefs();
// Объект для третьего полинома:
Polynomial R;
// Произведение полиномов:
R=P*Q;
cout<<"R: ";
// Коэффициенты третьего полинома:
R.coefs();
cout<<"Значения полиномов\n";
// Аргумент для полиномов и
// приращение для аргумента:
double x=-2,dx=1;
// Значения полиномов в разных точках:
for(int k=1;k<=5;k++){
    cout<<"P("<<x<<" ) = "<<P(x)<<"\t";
    cout<<"Q("<<x<<" ) = "<<Q(x)<<"\t";
    cout<<"R("<<x<<" ) = "<<R(x)<<endl;
    x+=dx; // Изменение значения аргумента
}
return 0;
}

```

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 9.3)

Полиномы

```

P: 3      -2      1
Q: 0      0      0      0
Q: -2     -2     -2      1
R: -6     -2     -4      5      -4      1

```

Значения полиномов

```

P(-2) = 11      Q(-2) = -14      R(-2) = -154
P(-1) = 6       Q(-1) = -3       R(-1) = -18
P(0) = 3        Q(0) = -2        R(0) = -6
P(1) = 2        Q(1) = -5        R(1) = -10
P(2) = 3        Q(2) = -6        R(2) = -18

```

В классе `Polynomial`, как отмечалось, два закрытых поля: указатель `a` отождествляется с массивом (динамическим) коэффициентов полинома, степень же полинома "запоминается" в целочисленном поле `power`.

**На заметку**

Стоит заметить, что количество коэффициентов в полиноме на единицу больше степени полинома. Поэтому количество элементов в массиве `a` на единицу больше значения поля `power`. Во всяком случае, наша задача состоит в том, чтобы это соотношение выполнялось для любого объекта класса `Polynomial`.

Прочие члены класса открытые. Среди них особую группу составляют конструкторы. В классе `Polynomial` описано несколько конструкторов:

- Есть конструктор, позволяющий создавать объект на основе массива элементов и числового значения, определяющего степень соответствующего полинома (количество элементов в массиве на единицу больше показателя степени полинома). В этом случае коэффициенты полинома определяются значениями элементов массива, на основе которого создается объект.
- Описан конструктор с одним числовым аргументом, имеющим нулевое значение по умолчанию и определяющим количество коэффициентов в полиноме. Все коэффициенты получают нулевые значения.
- В классе имеется конструктор создания копии, когда новый объект создается на основе уже существующего объекта. Для нового объекта создается динамический массив с таким же количеством элементов и с такими же значениями элементов, как и в объекте, на основе которого создается новый объект.

**На заметку**

В конструкторе создания копии аргумент передается по ссылке. В операторе цикла, в котором выполняется заполнение элементов массива создаваемого объекта, при обращении к элементам исходного объекта выполняется индексирование объекта (имеется в виду команда `a[i]=P[i]`, где через `a` обозначен динамический массив, а через `P` обозначен аргумент конструктора создания копии).

В классе `Polynomial` описан деструктор. В теле деструктора всего одна команда `delete []` для удаления динамического массива.

**На заметку**

Поля, определяющие степень полинома и коэффициенты полинома, являются закрытыми. Поэтому просто так к ним обратиться не получится. Для считывания значения поля `power` (степень полинома) описан метод `getPower()`. Отобразить набор коэффициентов полинома можно с помощью метода `coefs()`.

Операторный метод `operator[]()` для индексирования объектов возвращает в качестве результата ссылку на элемент `a[k]` (при условии, что `k` - целочисленный аргумент операторного метода).

Операторный метод `operator()()` определен так, что результатом метода возвращается значение полинома. При этом результат вычисляется на основе значения аргумента операторного метода и коэффициентов полинома, "спрятанных" в динамический массив объекта, из которого вызывается операторный метод.

Еще один операторный метод призван обеспечить возможность вычислять произведение полиномов. Речь идет об операторном методе `operator*()`. Аргумент метода - объект класса `Polynomial`. Результатом метода также является объект класса `Polynomial`. Причем речь идет о ссылке на объект. Причина последнего в том, что в теле метода создается динамический объект, и непосредственно данный объект возвращается результатом метода. Именно поэтому мы воспользовались механизмом возвращения ссылки на объект.



На заметку

Нелишним будет напомнить, как в общем случае возвращается результат метода или функции. Значение, возвращаемое результатом, на самом деле копируется в заранее выделенную для результата функции или метода ячейку памяти, и уже это значение возвращается результатом. То есть фактически возвращается копия того значения, которое вычислено как финальное значение результата. В данном случае нас подобный вариант не устраивает и метод `operator*()` описан так, что возвращается непосредственно объект, созданный в теле метода. Эту ситуацию мы еще прокомментируем и рассмотрим более детально.

Переопределение оператора умножения для объектов класса `Polynomial` непосредственного отношения к функторам не имеет, но в определенном смысле иллюстрирует преимущества функторов по сравнению с обычными функциями: для функторов можно задавать операции, которые проблематично реализовать с обычными функциями.

Подробности

Возможно, понадобятся пояснения относительно вычисления произведения полиномов. Допустим, имеется полином $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$ степени n и полином $Q(x) = b_0 + b_1x + b_2x^2 + \dots + b_mx^m$ степени m , и мы хотим вычислить произведение этих полиномов $R(x) = P(x)Q(x)$. Важно то, что произведение двух полиномов - тоже полином. Поэтому если исходные полиномы реализуются в виде объектов некоторого класса, то произведение таких полиномов также можно реализовать в виде объекта того же класса. Степень полинома-произведения равняется сумме степеней пере-

множаемых полиномов (в наших обозначениях $n + m$). Действительно, $R(x) = (a_0 + a_1x + a_2x^2 + \dots + a_nx^n)(b_0 + b_1x + b_2x^2 + \dots + b_mx^m) = c_0 + c_1x + c_2x^2 + \dots + c_{n+m}x^{n+m}$

, причем коэффициенты этого полинома $c_k = \sum_{i,j:i+j=k} (a_i b_j)$. Другими словами,

коэффициент c_k с индексом k представляет собой сумму попарных произведений коэффициентов умножаемых полиномов, сумма индексов которых равна k . Например, $c_0 = a_0b_0$, $c_1 = a_0b_1 + a_1b_0$, и так далее. Именно этим свойством коэффициентов полинома-произведения мы воспользовались при описании метода `operator*()`.

В теле операторного метода `operator*()` объявляется указатель `t` на объект класса `Polynomial`. Из объекта `P`, переданного аргументом операторному методу, вызывается метод `getPower()` и результат записывается в переменную `n`. Это степень второго из перемножаемых полиномов. Если сложить данное значение со значением поля `power` объекта, из которого вызывается операторный метод, то получим степень полинома, являющегося результатом вычисления произведения. Собственно указанное значение вычисляется и записывается в переменную `m`. Далее командой `t=new Polynomial(m)` создается динамический объект с динамическим массивом из `m` элементов. Если посмотреть программный код соответствующего конструктора, то несложно сообразить, что значения всех элементов в массиве будут нулевыми. Эти значения нужно "уточнить" - заполнить элементы так, чтобы получились коэффициенты вычисляемого полинома.

Запускаются вложенные операторы цикла. Во внешнем цикле перебираются коэффициенты первого полинома, поэтому индексная переменная `i` в операторе цикла пробегает значения от 0 до `power` включительно. Во внутреннем цикле индексная переменная `j` пробегает значения от 0 до `n`, перебирая тем самым коэффициенты второго полинома. При фиксированных значениях `i` и `j` командой `(*t)[i+j]+=a[i]*P[j]` добавляется слагаемое к значению элемента с индексом `i+j` полинома, реализованного через динамический объект.



На заметку

Обращение к элементам массива объекта, из которого вызывается операторный метод, выполняется в явном виде (инструкция `a[i]`). При обращении к элементам массивов объекта `P` (аргумент операторного метода) и динамического объекта (инструкция `*t`) использовано индексирование объекта (инструкции `P[j]` и `(*t)[i+j]` соответственно). Напомним, что индексирование объектов возможно благодаря перегрузке операторного метода `operator[]()`.

Сформированный таким образом динамический объект командой `return *t` возвращается как результат метода.

Чтобы при присваивании объектов класса `Polynomial` не "терялись" их динамические массивы, перегружаем оператор присваивания. В теле опера-

торного метода `operator=()` командой `delete []` а сначала удаляется динамический массив, записанный в объекте на данный момент.



На заметку

Первым операндом в выражении присваивания является объект слева от оператора присваивания. Операторный метод `operator=()` вызывается из данного объекта. Поэтому объектом вызова в операторном методе является фактически тот объект, которому присваивается значение. Объект, переданный аргументом операторному методу `operator=()` является присваиваемым объектом. В данном случае нам необходимо добиться, чтобы объект, которому присваивается значение, содержал такой же массив, как и объект, значение которого присваивается. Также у этих объектов должны совпадать значения полей `power`.

Новая степень полинома вычисляется командой `power=P.getPower()` (через `P` обозначен аргумент операторного метода - то есть присваиваемый объект). Создается новый динамический массив (команда `a=new double[power+1]`), после чего запускается оператор цикла, и в нем значения элементам массива `a` присваиваются такие же, как значения элементов массива в присваиваемом объекте (команда `a[i]=P[i]` в теле оператора цикла). Результатом операторного метода возвращается объект, которому присваивалось значение (объект вызова, инструкция `*this`).

В главной функции программы значения коэффициентов для одного из полиномов присваиваются случайным образом, для чего используется генератор случайных чисел. Инициализируется генератор случайных чисел инструкцией `srand(2014)`. Кроме этого, определяется целочисленная константа `n` (на ее основе будут вычисляться степени полиномов), а также явно инициализируется массив `nums` с коэффициентами для первого полинома. Полином (объект для полинома) создается командой `Polynomial P(nums,n)`. Другой полином создается с помощью команды `Polynomial Q(n+1)`. У созданного таким образом объекта `Q` все элементы во внутреннем динамическом массиве равны нулю. Для изменения значений элементов массива запускается оператор цикла, где командой `Q[i]=rand()%5-2` каждому из элементов массива в качестве значения присваивается целое случайное число в диапазоне значений от `-2` до `2` включительно. В данном случае обращение к элементам внутреннего динамического массива выполняется через индексирование объекта.

Для отображения содержимого внутреннего динамического массива из соответствующего объекта вызывается метод `coefs()`.

Объект для третьего полинома создается командой `Polynomial R`. Затем командой `R=P*Q` вычисляется произведение полиномов, а результат записывается в объект `R`. Убедиться в том, что речь идет именно о произведении

полиномов можно, сравнив коэффициенты исходных полиномов P и Q с коэффициентами полинома R .

В операторе цикла для нескольких значений действительной переменной x вычисляются значения полиномов P , Q и R . Значение вычисляется командами вида $P(x)$, $Q(x)$ и $R(x)$. В данном случае, очевидно, речь идет о вызове объектов - то есть объекты используются как функторы.



На заметку

Для каждого фиксированного значения аргумента x выполняется соотношение $R(x) = P(x)Q(x)$.

Есть один момент, который не связан непосредственно с функторами, но на который все же хотелось бы обратить внимание. Речь идет о способе реализации класса для работы с полиномами. Интересующие нас позиции следующие:

- Конструктор создания копии.
- Операторный метод для оператора присваивания.
- Операторный метод для оператора умножения.

Большинство "особенностей" кода обусловлено тем, что в классе `Polynomial` используется динамический массив. Поскольку технически динамический массив связан с классом (объектом) через указатель, то при побитовом копировании объектов возникают определенные проблемы.



На заметку

Мы используем динамические массивы, поскольку при работе с динамическим массивом размер массива (а значит и степень соответствующего полинома) можно определять в процессе выполнения программы. Это означает, что одного класса достаточно для работы с полиномами разных степеней.

9.4. Константные методы и аргументы

- Какая гадость.
- Это не гадость. Это последние достижения современной науки.
из к/ф "31 июня"

В рассмотренном выше примере операторный метод для оператора умножения был описан так, что результатом (по ссылке) возвращался динамический массив, созданный при вызове метода. Если бы мы захотели описать метод, чтобы он возвращал статический объект, пришлось бы внести

небольшие, но существенные изменения в программный код конструктора создания копии. Чтобы не быть голословными, рассмотрим модификацию предыдущего примера в листинге 9.4 (лишние комментарии в этом случае удалены, а основные изменения выделены жирным шрифтом).

Листинг 9.4. Новая версия старого примера

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Polynomial{
private:
double *a;
int power;
public:
Polynomial(double *b,int n){
power=n;
    a=new double[power+1];
    for(int i=0;i<=power;i++){
a[i]=b[i];
    }
}
Polynomial(int n=0){
power=n;
    a=new double[power+1];
    for(int i=0;i<=power;i++){
a[i]=0;
    }
}
// Конструктор создания копии
// (аргумент с идентификатором const):
Polynomial(const Polynomial &P){
power=P.getPower();
    a=new double[power+1];
    for(int i=0;i<=power;i++){
a[i]=P[i];
    }
}
~Polynomial(){
delete [] a;
}
// Константный метод возвращает значение
// для степени полинома:
int getPower() const{
return power;
}
```

```

    }
void coefs() {
for(int i=0; i<=power; i++) {
cout<<a[i]<<"\t";
    }
cout<<endl;
}
// Константный операторный метод для
// индексирования объектов:
double &operator[](int k) const {
return a[k];
}
double operator()(double x) {
double s=0, q=1;
for(int i=0; i<=power; i++) {
    s+=a[i]*q;
    q*=x;
}
return s;
}
// Операторный метод для умножения полиномов.
// Результатом возвращается значение локального
// статического объекта:
Polynomial operator*(Polynomial P) {
int n=P.getPower();
int m=power+n;
    // Локальный объект
    // (все коэффициенты нулевые):
Polynomial t(m);
for(int i=0; i<=power; i++) {
for(int j=0; j<=n; j++) {
t[i+j]+=a[i]*P[j];
    }
}
return t;
}
Polynomial operator=(Polynomial P) {
delete [] a;
power=P.getPower();
a=new double[power+1];
for(int i=0; i<=power; i++) {
a[i]=P[i];
    }
return *this;
}
};

```

```

int main() {
    srand(2014);
    const int n=2;
    double nums[n+1]={3,-2,1};
    cout<<"Полиномы\n";
        Polynomial P(nums,n);
    cout<<"P: ";
    P.coefs();
        Polynomial Q(n+1);
    cout<<"Q: ";
    Q.coefs();
    for(int i=0;i<=n+1;i++){
        Q[i]=rand()%5-2;
    }
    cout<<"Q: ";
    Q.coefs();
        Polynomial R;
        R=P*Q;
    cout<<"R: ";
    R.coefs();
    cout<<"Значения полиномов\n";
    double x=-2,dx=1;
    for(int k=1;k<=5;k++){
        cout<<"P("<<x<<" ) = "<<P(x)<<"\t";
        cout<<"Q("<<x<<" ) = "<<Q(x)<<"\t";
        cout<<"R("<<x<<" ) = "<<R(x)<<endl;
        x+=dx;
    }
    return 0;
}

```

Что нового в этом примере? Главная функция программы не изменилась вовсе (за исключением удаленных комментариев). Не очень сильно изменился и программный код класса `Polynomial`. Основные изменения произошли в операторном методе `operator*()`. Вместо создания динамического объекта создается локальный статический объект `t` класса `Polynomial` (в предыдущем примере через `t` обозначался указатель на динамический объект). Элементы внутреннего динамического массива объекта `t` заполняются, после чего объект возвращается результатом операторного метода. С точки зрения здоровой логики здесь все корректно. Однако если никаких других изменений в программный код не вносить, то на этапе компиляции возникнет ошибка, связанная, по большому счету, с выполнением команды `R=P*Q` в главной функции программы. Возникает вопрос: почему? Ответ, как ни странно может показаться, обращается вокруг конструктора создания копии. Дело в том, что аргумент конструктору создания копии переда-

ется по ссылке. Причину мы обсуждали ранее: чтобы не было бесконечного вызова конструктора создания копии при попытке создать копию для аргумента конструктора. Пикантность ситуации в том, что вызов конструктора создания копии автоматически блокирует создание другой копии, кроме той, что создается конструктором. Если перефразировать иначе, то при вызове конструктора создания копии на основе объекта-оригинала создается копия, и пока она создается, создание других копий блокируется. Примерно так.

Само по себе это не страшно. Но бывают ситуации, когда "пикантность процедуры" со счетов сбросить не получится. И такая ситуация случается, когда результат произведения объектов P и Q записывается в переменную R (команда $R=P*Q$). При вычислении выражения $P*Q$ вызывается операторный метод `operator*()`. При выполнении метода создается локальный объект класса `Polynomial`, который и возвращается результатом метода. Но на самом деле возвращается не этот объект, а его копия.

Для создания копии вызывается конструктор создания копии. До этого момента никаких проблем не возникает. Но вот если результат выражения $P*Q$ присваивается другому объекту (переменная R), то вызывается операторный метод `operator=()` для оператора присваивания. Результат выражения $P*Q$ передается аргументом операторному методу `operator=()` (путем создания копии - если аргумент передается по значению). Получается, что создается копия для копии, и такой процесс блокируется. Причем передача по ссылке аргумента операторному методу `operator=()` тоже не решает проблему: заблокировано как "транзитное" копирование исходного объекта, так и его "транзитная" передача по ссылке.

Решение проблемы может быть таким: в описании конструктора создания копии используем *константный аргумент*. Технически все сводится к добавлению инструкции `const` в описании аргумента конструктора. Весь код конструктора в данном случае выглядит следующим образом:

```
Polynomial(const Polynomial &P){
    power=P.getPower();
    a=new double[power+1];
    for(int i=0;i<=power;i++){
        a[i]=P[i];
    }
}
```

Наличие инструкции `const` в аргументе означает, что он "защищен" от изменения: попытка изменить аргумент метода (в данном случае конструктора) приведет к ошибке. Это как бы формальная сторона вопроса. Есть и

неформальная сторона. Состоит она в том, что теперь (когда явно указано, что объект-оригинал при создании копии неизменен) описанные выше "блокировки" снимаются.



На заметку

Важно понимать, что конструктор создания копии с прототипом `Polynomial(Polynomial &P)` отличается по своим "возможностям" от конструктора создания копии с прототипом `Polynomial(const Polynomial &P)`. Описание конструктора создания копии с константным аргументом (версия конструктора с идентификатором `const` в описании аргумента) считается более предпочтительной.

Но здесь тоже не так все просто. Многое зависит от программного кода конструктора. В данном случае в теле конструктора создания копии из объекта-аргумента `P`, который передается по ссылке и должен быть неизменным (не может меняться при создании копии), вызываются методы: метод `getPower()` вызывается при выполнении команды `power=P.getPower()`, а операторный метод `operator[]()` вызывается при выполнении команды `a[i]=P[i]` в операторе цикла. Нам необходимо "дать гарантии", что при вызове этих методов объект `P` не изменится. Мы можем сделать это, описав методы `getPower()` и `operator[]()` как константные. Для этого прототип метода нужно завершить инструкцией `const`. Формально константность метода означает, что при его вызове объект, из которого вызывается метод, не изменяется. В этом смысле с методом `getPower()` все просто. Его код будет таким:

```
int getPower() const{
return power;
}
```

Методом считывается значение поля `power` объекта, и поэтому очевидно, что никаких изменений объект не претерпевает. С методом `operator[]()` не все так просто. Код этого метода следующий:

```
double &operator[](int k) const{
return a[k];
}
```

Формально здесь возвращается значение соответствующего элемента массива. Но все дело в том, что на самом деле возвращается не просто значение, а ссылка на значение. Поэтому теоретически через такую ссылку элемент массива может быть не только считан, но и изменен - что, собственно, и происходит, правда в других местах программного кода. Мы же, не испытывая

ни капли смущения, объявляем операторный метод `operator[]()` как константный. Учитывая сделанное выше замечание, может показаться, что мы, мягко говоря, приукрашаем реальность. Но на самом деле криминала здесь нет. Дело в том, что даже если при вызове операторного метода меняется значение того или иного элемента динамического массива, с формальной точки зрения объект со "спрятанным" в нем массивом не меняется: все поля объекта остаются с теми самими значениями, как и до вызова метода.



На заметку

У объекта класса `Polynomial` два поля: значением поля `power` является степень соответствующего полинома, а значением поля `a` является адрес первого элемента динамического массива. При вызове операторного метода `operator[]()` не меняется значение ни одного из этих полей.

На этом основные изменения в программном коде исчерпываются. Результат выполнения программы такой же, как и в предыдущем случае:

Результат выполнения программы (из листинга 9.4)

Полиномы

```
P: 3      -2      1
Q: 0      0      0      0
Q: -2     -2     -2      1
R: -6     -2     -4      5      -4      1
```

Значения полиномов

P(-2) = 11	Q(-2) = -14	R(-2) = -154
P(-1) = 6	Q(-1) = -3	R(-1) = -18
P(0) = 3	Q(0) = -2	R(0) = -6
P(1) = 2	Q(1) = -5	R(1) = -10
P(2) = 3	Q(2) = -6	R(2) = -18

Итак, мы уже рассмотрели два подхода к описанию класса `Polynomial`, различия между которыми сводятся в основном к способу описания конструктора создания копии и некоторых операторных методов. В обоих случаях принципиальным моментом остается использование динамических массивов. Однако возможен качественно иной подход решения данной задачи. Такой подход основан на использовании шаблонов.

9.5. Функтор на основе шаблона

Хотите обмануть мага? Боже, какая детская непосредственность. Я же вижу Вас насквозь.

из к/ф "31 июня"

На новом этапе нашей профессиональной эволюции рассмотрим задачу о создании функтора для реализации полинома с привлечением обобщенного класса. Ранее мы уже использовали обобщенные классы и функции. Через параметры в обобщенных классах и функциях передавались обобщенные типы. В рассматриваемом далее примере имеется существенная особенность: через параметр в обобщенный класс передается не тип данных, а числовое значение, определяющее степень полинома. Иными словами мы описываем обобщенный класс с параметром, определяющим размер статического массива, содержащего коэффициенты полинома. Программный код примера приведен в листинге 9.5.

Листинг 9.5. Функтор на основе шаблона

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Обобщенный класс для реализации функтора.
// Параметр шаблона - степень полинома:
template <intpower> class Polynomial{
// Закрытые поля:
private:
    // Статический массив:
    double a[power+1];
    // Открытые члены:
public:
    // Конструктор создания объекта на основе массива:
    Polynomial(double *b){
        for(int i=0;i<=power;i++){
            a[i]=b[i]; // Заполнение массива
        }
    }
    // Конструктор без аргументов:
    Polynomial(){
        for(int i=0;i<=power;i++){
            a[i]=0; // Заполнение массива
        }
    }
    // Метод возвращает степень полинома:
    int getPower(){
```

```

return power;
    }
    // Операторный метод для индексирования объектов:
double &operator[] (int k) {
return a[k];
}

    // Операторный метод для "вызова" объектов:
double operator() (double x) {
double s=0,q=1;
// Вычисление значения полинома:
for(int i=0;i<=power;i++){
s+=a[i]*q;
q*=x;
    }
return s;
}
// Обобщенный операторный метод для
// вычисления произведения полиномов:
template<int n>
Polynomial<power+n> operator*(Polynomial<n> P) {
// Степень полинома - результата произведения:
const int m=power+n;
// Локальный объект создается на
// основе обобщенного класса:
Polynomial<m>tmp;
    // Вычисление коэффициентов
    // полинома - результата произведения:
for(int i=0;i<=power;i++){
for(int j=0;j<=n;j++){
tmp[i+j]+=a[i]*P[j];
    }
    }
// Результат операторного метода - объект:
return tmp;
}
// Метод для отображения коэффициентов полинома:
void coefs() {
for(int i=0;i<=power;i++) {
cout<<a[i]<<"\t";
    }
cout<<endl;
}
}; // Завершение описания обобщенного класса
// Главная функция программы:
int main() {
    // Инициализация генератора случайных чисел:

```

```

srand(2014);
    // Степень первого полинома:
const int n=2;
    // Степень второго полинома:
const int m=3;
    // Массив с коэффициентами полинома:
double nums[n+1]={3,-2,1};
cout<<"Полиномы\n";
// Создание на основе обобщенного класса
    // объекта для полинома:
Polynomial<n>P(nums);
cout<<"P:\t";
// Коэффициенты первого полинома:
P.coefs();
    // Создание на основе обобщенного класса
    // объекта для второго полинома:
Polynomial<m>Q;
cout<<"Q:\t";
// Коэффициенты второго полинома:
Q.coefs();
    // Присваивание коэффициентам второго полинома
    // случайных значений:
for(int i=0;i<=Q.getPower();i++){
Q[i]=rand()%5-2; // Случайное число от -2 до 2
}
cout<<"Q:\t";
    // Коэффициенты второго полинома:
Q.coefs();
cout<<"P*Q:\t";
// Коэффициенты для произведения полиномов:
(P*Q).coefs();
// Создание на основе обобщенного класса
    // объекта для полинома:
Polynomial<n+m>R;
    // Произведение двух полиномов:
R=P*Q;
cout<<"R:\t";
// Коэффициенты полинома - произведения
    // двух полиномов:
R.coefs();
cout<<"Значения полиномов\n";
    // Аргумент для вычисления значения полинома и
    // приращение для аргумента:
double x=-2,dx=1;
    // Значения полиномов при разных аргументах:
for(int k=1;k<=5;k++){

```

```
cout<<"P("<<x<<" ) = "<<P(x)<<"\t";
cout<<"Q("<<x<<" ) = "<<Q(x)<<"\t";
cout<<"R("<<x<<" ) = "<<R(x)<<"\t";
cout<<"(P*Q)("<<x<<" ) = "<<(P*Q)(x)<<endl;
x+=dx;
}
return 0;
}
```

В результате выполнения программного кода в окне вывода отображается следующая информация:

Результат выполнения программы (из листинга 9.5)

Полиномы

P:	3	-2	1			
Q:	0	0	0	0		
Q:	-2	-2	-2	1		
P*Q:	-6	-2	-4	5	-4	1
R:	-6	-2	-4	5	-4	1

Значения полиномов

P(-2) = 11	Q(-2) = -14	R(-2) = -154	(P*Q)(-2) = -154
P(-1) = 6	Q(-1) = -3	R(-1) = -18	(P*Q)(-1) = -18
P(0) = 3	Q(0) = -2	R(0) = -6	(P*Q)(0) = -6
P(1) = 2	Q(1) = -5	R(1) = -10	(P*Q)(1) = -10
P(2) = 3	Q(2) = -6	R(2) = -18	(P*Q)(2) = -18

Теперь проанализируем программный код примера. Основу его составляет описание обобщенного класса `Polynomial`. Прототип описания класса такой: `template<int power> class Polynomial`. Традиционно присутствует ключевое слово `template`, но вот в угловых скобках вместо ключевого слова `class` и названия параметра указана инструкция `int power`. Означает это буквально следующее: при создании объекта класса необходимо будет после названия класса в угловых скобках указать целочисленный параметр. В программном коде класса обращение к параметру выполняется через имя `power`.

Закрытое поле у класса всего одно: статический массив, который определяется инструкцией `double a[power+1]`. Здесь мы используем целочисленный параметр `power` для определения размера статического массива.



На заметку

Поскольку массив статический, то нет необходимости описывать конструктор создания копии и операторный метод для присваивания объектов: вполне приемлема используемая по умолчанию процедура побитового копирования. Также мы не бу-

дем описывать деструктор: нет динамического массива, и поэтому отпадает необходимость освобождать память, занимаемую массивом, при удалении объекта. Для статического массива это делается автоматически.

У класса есть два конструктора и несколько открытых методов. Один из конструкторов позволяет создавать объект на основе числового массива. Элементы числового массива определяют коэффициенты полинома. Массив передается конструктору через указатель на первый элемент.



На заметку

Как известно, для передачи массива аргументом функции необходимо два параметра: указатель на первый элемент массива и параметр, определяющий количество элементов в массиве. В данном случае конструктору передается только указатель на первый элемент массива. Необходимости передавать параметр, определяющий количество элементов в массиве, нет, поскольку данный параметр указывается при создании объекта обобщенного класса (имеется в виду параметр `power`).

Еще один конструктор, описанный в классе, позволяет создавать объект с массивом, у которого нулевые значения. Аргументы конструктору в этом случае не передаются.



На заметку

Поскольку речь идет о статическом массиве, то весь код конструктора (первого и второго) сводится, фактически, к присваиванию значений элементам массива. Просто в первом случае присваиваемые элементам массива значения берутся из массива, переданного аргументом конструктору, а во втором случае элементам массива присваиваются нулевые значения.

Метод `getPower()` возвращает значение параметра `power`, а с помощью метода `coefs()` отображаются значения элементов массива `a`. Нет особой интриги в описании операторных методов `operator[]()` и `operator()()`. Операторным методом `operator[]()` возвращается ссылка на элемент массива с соответствующим индексом, а операторный метод `operator()()` фактически делает объект класса `Polynomial` функтором: благодаря описанию этого метода в классе объект класса можно вызывать как функцию. Результатом будет, напомним, значение полинома в точке, определяемой аргументом операторного метода.

Наибольший интерес, пожалуй, вызывает операторный метод `operator*()`, предназначенный для вычисления произведения двух полиномов.

В первую очередь следует разобраться с тем, что у нас (точнее, операторного метода) есть "на входе", и что мы хотим получить "на выходе". Итак, имеется два объекта класса `Polynomial`, которые мы отождествляем с

полиномами. Один из этих объектов - тот, из которого вызывается операторный метод. Степень соответствующего полинома определяется параметром `power`. Второй объект передается аргументом операторному методу. Он обозначен через `P`. Данному объекту соответствует полином некоторой степени. Показатель степени полинома (обозначаем его как `n`, и, что важно, - его значение нам наперед неизвестно) указывается в угловых скобках после имени класса `Polynomial` в описании аргумента операторного метода `operator*()`. Если так, то результатом операторного метода будет полином степени, определяемой суммой степеней перемножаемых полиномов - то есть суммой параметров `power` и `n`. Но если "происхождение" параметра `power` в общем понятно (параметр, указываемый при создании объекта, из которого вызывается метод), то параметр `n` определяется типом объекта, передаваемого аргументом операторному методу.



На заметку

Если мы создаем объекты на основе класса `Polynomial`, но при этом в угловых скобках указаны разные параметры, то по большому счету имеем дело с разными типами объектов. Другими словами, инструкции `Polynomial<2>` и `Polynomial<3>` обозначают разные типы. Здесь речь идет фактически о том, что методу `operator*()` аргументом передается объект, и показатель степени полинома (параметр `n`), реализуемого этим объектом, "спрятан" в типе объекта.

При описании метода нам необходимо каким-то образом определиться с параметром `n` - его нужно где-то объявить. Естественный выход из ситуации - использовать шаблон. Поэтому заголовок операторного метода `operator*()` выглядит так: `template<int n> Polynomial<power+n> operator*(Polynomial<n> P)`. После ключевого слова `template` в угловых скобках объявлен целочисленный параметр `n` - точно такой же, как в описании типа объекта, передаваемого аргументом методу. Тип результата метода определяется выражением `Polynomial<power+n>`. Здесь мы учли, что степень полинома-произведения равняется сумме степеней перемножаемых полиномов.

Что касается программного кода операторного метода `operator*()`, то он достаточно простой. Командой `const int m=power+n` определяется (ради удобства) константа, определяющая степень полинома-результата, затем создается объект для такого полинома (команда `Polynomial<m> tmp`) и запускаются вложенные операторы цикла, с помощью которых вычисляются значения элементов массива для объекта `tmp`. Объект возвращается в качестве результата метода.

В главной функции программы, по сравнению с предыдущими примерами, много схожих моментов, но есть и некоторые отличия. На них и остановимся.

Степень первого полинома определяется константой n , для определения степени второго полинома объявляется константа m . Объект P для первого полинома создается с помощью инструкции `Polynomial<n> P(nums)`. Поскольку аргументом конструктору передан массив `nums`, то массив для объекта P получит такие же значения. Объект Q создается командой `Polynomial<m> Q`. Элементы массива данного объекта при создании получают нулевые значения. Затем с помощью оператора цикла им присваиваются случайные значения (как это делалось и в предыдущих примерах).

Отдельных пояснений заслуживает процедура вычисления произведения полиномов, выполняемая на примере объектов P и Q . Так, результатом выражения $P*Q$ является объект класса `Polynomial` (с соответствующим значением параметра). Поэтому из выражения $P*Q$ можно вызвать, например, метод `coefs()` (команда `(P*Q).coefs()`), в результате чего в окне вывода будут отображены коэффициенты полинома-произведения. А можно узнать значение такого полинома в какой-то конкретной точке x (команда вида `(P*Q)(x)`).



На заметку

В последнем случае важно понимать, что выражение $P(x)*Q(x)$ и $(P*Q)(x)$ - далеко не одно и то же самое (хотя числовые значения этих выражений должны совпадать). Выражение $P(x)*Q(x)$ вычисляется так: рассчитываются значения выражений $P(x)$ и $Q(x)$, после чего они перемножаются. Это соответствует ситуации, когда вычисляются значения каждого из полиномов в точке, а потом полученные значения умножаются одно на другое.

Что касается выражения $(P*Q)(x)$, то здесь сначала вычисляется объект $P*Q$, и уже затем данный объект "вызывается" с аргументом x . На языке операций с полиномами ситуация выглядит следующим образом: сначала определяется результирующий полином (как функциональная зависимость), а затем вычисляется его значение.

Здесь мы, кстати, не очень явно столкнулись с более фундаментальной проблемой: как передавать функцию аргументом в другую функцию. К данному вопросу мы вернемся, но немного позже.

Использование выражения наподобие $P*Q$ приемлемо, но не всегда удобно. Можем результат выражения $P*Q$ записать в переменную (присвоить в качестве значения объекту). Но предварительно необходимо создать объект. Делаем это с помощью команды `Polynomial<n+m> R`. На следующем этапе выполняем присваивание `R=P*Q`, после чего объект R отождествляется с результатом произведения объектов P и Q .

**На заметку**

Чтобы объекту R можно было присвоить значение произведения $P \cdot Q$, объект R должен быть создан с передачей "правильного" параметра для обобщенного класса. Степень полинома, реализованного через объект P , равняется n . Степень полинома, реализованного через объект Q , равняется m . Поэтому объекту, являющемуся произведением P и Q , соответствует полином степени $n+m$. Именно такое значение параметра было передано обобщенному классу при создании объекта R командой `Polynomial<n+m> R`.

Хотя использованный нами подход в известном смысле удобен, все же не сложно заметить и некоторые его недостатки. Как минимум обращает на себя внимание не самый простой способ описания операторного метода для вычисления произведения объектов, а также достаточно "замысловатый" подход к определению объекта, которому в качестве значения присваивается результат произведения объектов. Если смотреть глубже, то легко сообразить, что концептуальная проблема подхода, основанного на использовании обобщенного класса, связана с тем, что степень полинома при создании объекта указывается единожды и впоследствии не может быть изменена. При создании объектов на основе классов с динамическими массивами подобной проблемы не было (зато там были другие "неожиданности").

**На заметку**

Откровенно говоря, ситуация такая, что объекты для реализации полиномов разных степеней создаются на основе разных классов, хотя формально во всех таких случаях используется один обобщенный класс. Просто каждый раз классу передается какое-то значение параметра, и в результате получаем по большому счету разные классы.

Используя обобщенный класс, мы имеем дело с чем-то средним между классом с динамическим массивом и классом со статическим массивом. Хотя со статическими массивами тоже не все так просто. Рассмотрим пример.

9.6. Функтор на основе класса со статическим массивом

Ну и что Вы скажете обо всем этом, Ватсон?

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

Если создать класс с полем, являющимся статическим массивом, автоматически исчезнет ряд "тактических" проблем, но появится одна "стратегическая": размер массива фиксирован, а полиномы могут быть разной степени.

Самый простой выход - взять массивы достаточного размера, чтобы туда поместился практически любой (в разумных пределах) полином. Естественно, это приведет к нерациональному расходованию системных ресурсов (памяти), но за простоту приходится чем-то платить.

В листинге 9.6 приведен программный код, в котором реализована идея класса со статическим массивом. В программе объявлена константа `Nmax`, определяющая размер статических массивов в объектах класса `Polynomial`. В классе есть поле `power`, определяющее степень полинома, который реализуется через данный объект. Другими словами, несмотря на то, что массив имеет размер `Nmax`, используются только элементы массива с индексами от 0 до `power` включительно. Здесь заключается основная идея в использовании класса со статическим массивом. Теперь рассмотрим программный код:

Листинг 9.6. Функтор на основе класса со статическим массивом

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Размер массивов:
const int Nmax=100;
// Класс для реализации полиномов:
class Polynomial{
    // Закрытые поля класса:
private:
    // Массив с коэффициентами полинома:
    double a[Nmax];
    // Степень полинома:
    int power;
    // Открытые члены класса:
public:
    // Конструктор (аргументы - указатель на массив
    // и степень полинома):
    Polynomial(double *b,int n){
        power=n; // Степень полинома
        // Заполнение массива:
        for(int i=0;i<=power;i++){
            a[i]=b[i];
        }
        // Конструктор (аргумент - степень полинома):
        Polynomial(int n=0){
            // Степень полинома:
            power=n;
            // Заполнение массива нулями:
```

```

for(int i=0;i<=power;i++){
a[i]=0;
}
}
// Метод возвращает значение для степени полинома:
int getPower(){
return power;
}
// Метод для отображения коэффициентов полинома:
void coefs(){
for(int i=0;i<=power;i++){
cout<<a[i]<<"\t";
}
cout<<endl;
}
// Операторный метод для индексирования объектов:
double &operator[](int k){
return a[k];
}
// Операторный метод для "вызова" объектов:
double operator()(double x){
double s=0,q=1;
for(int i=0;i<=power;i++){
s+=a[i]*q;
q*=x;
}
return s;
}
// Операторный метод для умножения полиномов:
Polynomial operator*(Polynomial P){
// Степень полинома - аргумента метода:
int n=P.getPower();
// Степень полинома - результата произведения:
int m=power+n;
// Локальный объект:
Polynomial t(m);
// Вычисление элементов массива:
for(int i=0;i<=power;i++){
for(int j=0;j<=n;j++){
t[i+j]+=a[i]*P[j];
}
}
return t;
}
}; // Завершение описания класса
// Главная функция программы:

```

```

int main() {
    // Инициализация генератора
    // случайных чисел:
    srand(2014);
    // Степень первого полинома:
    const int n=2;
    // Массив коэффициентов для полинома:
    double nums[n+1]={3,-2,1};
    cout<<"Полиномы\n";
    // Объект для первого полинома:
    Polynomial P(nums,n);
    cout<<"P: ";
    // Коэффициенты первого полинома:
    P.coefs();
    // Объект для второго полинома:
    Polynomial Q(n+1);
    cout<<"Q: ";
    // Коэффициенты второго полинома:
    Q.coefs();
    // Коэффициенты полинома - случайные числа:
    for(int i=0;i<=n+1;i++){
        Q[i]=rand()%5-2;
    }
    cout<<"Q: ";
    // Коэффициенты второго полинома:
    Q.coefs();
    // Объект для третьего полинома:
    Polynomial R;
    // Произведение полиномов:
    R=P*Q;
    cout<<"R: ";
    // Коэффициенты третьего полинома:
    R.coefs();
    cout<<"Значения полиномов\n";
    // Переменные:
    double x=-2,dx=1;
    // Значения полиномов в разных точках:
    for(int k=1;k<=5;k++){
        cout<<"P("<<x<<" ) = "<<P(x)<<"\t";
        cout<<"Q("<<x<<" ) = "<<Q(x)<<"\t";
        cout<<"R("<<x<<" ) = "<<R(x)<<endl;
        x+=dx;
    }
    return 0;
}

```

Думается, особых комментариев данный код не требует. Результат выполнения программы такой же, как и в некоторых предыдущих примерах:

Результат выполнения программы (из листинга 9.6)

Полиномы

P:	3	-2	1			
Q:	0	0	0	0		
Q:	-2	-2	-2	1		
R:	-6	-2	-4	5	-4	1

Значения полиномов

P(-2) = 11	Q(-2) = -14	R(-2) = -154
P(-1) = 6	Q(-1) = -3	R(-1) = -18
P(0) = 3	Q(0) = -2	R(0) = -6
P(1) = 2	Q(1) = -5	R(1) = -10
P(2) = 3	Q(2) = -6	R(2) = -18

На этом мы рассмотрение функторов временно завершаем, хотя тема функций и "функциональных" объектов еще будет подниматься в книге.

Глава 10.

ФУНКЦИЯ КАК АРГУМЕНТ И РЕЗУЛЬТАТ



Холмс, это исключено. Сразу видно, что Вы мало читаете.

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

В данной главе мы продолжим "функциональную" тему. Правда, посмотрим на задачу использования функций несколько под иным углом. В первую очередь обсудим один важный вопрос, который берет свое начало из простого, но не очевидного утверждения: "имя функции является указателем на функцию". Это действительно так и далее обсудим, как выполняется указатель на функцию, как он используется, да и вообще, неплохо было бы разобраться, о чем в принципе идет речь.

10.1. Указатель на функцию

Именно так выражается ее потребность в мировой гармонии.

из к/ф "Покровские ворота"

Мы в принципе знаем, что такое указатель на обычную переменную или даже указатель на объект. Если кратко, то указатель можно отождествлять с некоторым адресом памяти. Адрес записывается в переменную. Тип такой переменной (переменной-указателя) важен не менее чем значение, записанное в переменную. Тип переменной-указателя влияет на результат арифметических операций с этим указателем. Поэтому желательно знать не только адрес ячейки памяти, но и тип значения, которое может быть записано в соответствующую область памяти.

Если речь идет о функции, то на самом деле мало что меняется. Имеется "точка входа", определяемая адресом области памяти, в которой записан код функции. Данный адрес является значением указателя на функцию. Остается определиться с "типом" такого указателя. Здесь уместно задаться вопросом: какие характеристики функции важны? Как минимум, определяющее значение имеют количество и тип аргументов, а также тип возвращаемого результата. Это как раз параметры, декларируемые при создании указателя на функцию. Указатель на функцию создается (объявляется) для функций определенного *типа*. Тип функции в данном случае определяется типом результата и количеством и типами аргументов. Если данные харак-

теристики у некоторых функций совпадают, то такие функции относятся к одному типу (в том смысле, что можно объявить указатель и этому указателю в качестве значения присвоить имя той или иной функции). Общий шаблон объявления указателя на функцию выглядит так:

```
тип_результата (*название_указателя) (типы_аргументов);
```

Указывается тип результата, возвращаемого функцией (на которую может указывать указатель), название указателя (имя переменной-указателя), а в круглых скобках через запятую перечисляются типы аргументов функции (на которую может указывать указатель). Перед именем переменной-указателя должна присутствовать звездочка *, а вся конструкция из звездочки и имени указателя заключается в круглые скобки (иначе синтаксис команды будет неоднозначным). Например, ниже приведено объявление указателя `f` на функцию, имеющую один аргумент типа `int` и в качестве результата возвращающую значение типа `int`:

```
int (*f) (int);
```

Если указатель объявлен, то ему можно присвоить значение. Значение указателя - адрес ячейки в памяти, через которую получаем доступ к функции (можно полагать, что это адрес, где записана функция). Как узнать/получить такой адрес? Адрес функции возвращается именем функции. Другими словами, имя функции (без круглых скобок) возвращает адрес функции. Вывод из сказанного простой: указателю на функцию в качестве значения можно присвоить имя функции. Главное, чтобы тип функции соответствовал типу указателя на функцию.

После присваивания указателю на функцию значения, с указателем можно обращаться как с функцией - то есть мы можем вызвать указатель. При этом будет вызываться та функция, имя которое присвоено указателю в качестве значения. Вызывается указатель точно так же, как и функция: после имени указателя в круглых скобках передаются аргументы (если аргументов нет, то после имени указателя просто указываются пустые круглые скобки). Небольшой пример приведен в листинге 10.1.

В программе объявляются две функции: функцией `factorial()` вычисляется факториал числа, а функцией `dfactorial()` вычисляется двойной факториал числа. Обе эти функции подпадают под один "типаж": у них один аргумент типа `int` и результатом они возвращают значение типа `int`.

**На заметку**

Факториалом числа n называется произведение чисел от 1 до n включительно: $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n - 1) \cdot n$. Например, $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.

Двойной факториал числа n - это произведение через одно число всех натуральных чисел от 1 (для нечетных чисел) или 2 (для четных чисел) до n включительно: $n!! = n \cdot (n - 2) \cdot (n - 4) \cdot \dots$. Например, $5!! = 5 \cdot 3 \cdot 1 = 15$, а $6!! = 6 \cdot 4 \cdot 2 = 48$.

В главной функции программы создается указатель `f` на функцию соответствующего типа, этому указателю присваиваются поочередно значения `factorial` и `dfactorial`, и каждая из этих функций вызывается через указатель `f`. Рассмотрим программный код примера:

Листинг 10.1. Указатель на функцию

```
#include <iostream>
using namespace std;
// Функция для вычисления факториала числа:
int factorial(int n){
    int res=1;
    for(int i=1;i<=n;i++){
        res*=i;
    }
    return res;
}
// Функция для вычисления двойного факториала числа:
int dfactorial(int n){
    int res=1;
    for(int i=n;i>=1;i-=2){
        res*=i;
    }
    return res;
}
// Главная функция программы:
int main(){
    int m=5; // Аргумент для функции
    // Указатель на функцию:
    int (*f)(int);
    // Указателю на функцию присваивается значение:
    f=factorial;
    // Функция вызывается через указатель:
    cout<<"f("<<m<<")="<<f(m)<<endl;
    // Указателю на функцию присваивается значение:
```

```
f=dfactorial;
    // Функция вызывается через указатель:
cout<<"f ("<<m<<" )="<<f(m)<<endl;
return 0;
}
```

В результате выполнения программного кода получаем такое:

Результат выполнения программы (из листинга 10.1)

```
f(5)=120
f(5)=15
```

Указатель `f` объявляется командой `int (*f)(int)`. Такому указателю можно присвоить в качестве значения имя функции, у которой один аргумент типа `int` и результат типа `int`. Функции `factorial()` и `dfactorial()` подпадают под это определение. Поэтому команды `f=factorial` и `f=dfactorial` вполне законны. После выполнения команды `f=factorial` указатель `f` ссылается на функцию `factorial()`. В этом смысле допустимо рассматривать функцию `f()` в качестве "синонима" функции `factorial()`. Вызов `f(m)` эквивалентен вызову `factorial(m)` (здесь `m` -целочисленная переменная). После выполнения команды `f=dfactorial` указатель `f` ссылается на функцию `dfactorial()` и команда `f(m)` теперь эквивалентна команде `dfactorial(m)`.

Рассмотренный выше пример достаточно искусственный. Вместе с тем, механизм создания указателей на функции открывает перед разработчиком поистине уникальные возможности. Есть несколько классов очень важных с практической точки зрения задач, решение которых с применением указателей на функции выполняется легко, эффективно и в чем-то даже красиво. Некоторые ситуации мы рассмотрим. В основном речь будет идти о "математических" задачах, поскольку при решении именно таких задач вся мощь указателей на функции проявляется в полной мере.

10.2. Решение уравнения методом последовательных приближений

*Вот что крест животворящий делает!
из к/ф "Иван Васильевич меняет профессию"*

Рассмотрим задачу о решении уравнения вида $x = f(x)$. Функцию $f(x)$ полагаем заданной. Нам необходимо найти такое значение x , что $x = f(x)$. Например, если $f(x) = \sqrt{x+6}$, то речь идет об уравнении $x = \sqrt{x+6}$. У этого уравнения есть решение $x = 3$: если подставить данное значение в выражение $x = \sqrt{x+6}$, то оно превратится в тождество ($\sqrt{3+6} = \sqrt{9} = 3$).

Не существует универсального алгоритма, позволяющего находить в общем случае аналитическое решение уравнения вида $x = f(x)$. Однако есть алгоритм, который позволяет находить числовое решение уравнения. Для этого необходимо указать начальное приближение x_0 для корня уравнения. Если начальное приближение задано, то выполняются последовательные итерации, или приближения:

- на основе нулевого приближения x_0 вычисляется первое приближение для корня $x_1 = f(x_0)$;
- на основе первого приближения x_1 вычисляется второе приближение $x_2 = f(x_1)$, и так далее;
- на основе n -го приближения вычисляется следующее приближение по формуле $x_{n+1} = f(x_n)$.

Процесс продолжается до тех пор, пока не будет достигнута нужная точность в вычислении корня уравнения.

Подробности

Описанная выше процедура выполнения последовательных приближений для корня сходится к корню уравнения далеко не всегда. Для сходимости процесса необходимо, чтобы в области поиска корня производная $f'(x)$ от функции $f(x)$ по абсолютной величине была меньше единицы, то есть, чтобы выполнялось условие $|f'(x)| < 1$.

Что здесь интересного с точки зрения программирования? Дело в том, что процесс поиска решения универсален для различных функций $f(x)$ (разумеется, при условии, что эти функции удовлетворяют критериям применимости метода последовательных итераций). Поэтому мы могли бы написать

программный код, определив в нем функцию для решения *различных* уравнений методом последовательных приближений. При этом функция $f(x)$, однозначно определяющая решаемое уравнение, могла бы передаваться (через указатель) аргументом той функции, которая "решает" уравнение. Такой подход реализован в программе, представленной в листинге 10.2.

Листинг 10.2. Решение уравнения методом последовательных приближений

```
#include <iostream>
#include <cmath>
using namespace std;
// Функция для решения уравнений
// методом последовательных приближений.
// Аргументы - указатель на функцию уравнения,
// начальное приближение и количество итераций:
double findRoot(double (*f)(double), double x0, int n){
// Начальное приближение для корня:
double x=x0;
// Последовательные итерации:
for(int i=1; i<=n; i++){
x=f(x); // Новое приближение для корня
}
return x;
}
// Функция уравнения:
double F(double x){
return sqrt(x+6);
}
// Функция уравнения:
double G(double x){
return (x*x+10)/11;
}
// Главная функция программы:
int main(){
// Начальные приближения для поиска корней уравнений:
double x0=0.0, z0=5.0;
cout<<"x = F(x)"<<"\t"<<"x = G(x)"<<endl;
cout<<"0: "<<x0<<"\t\t"<<z0<<endl;
// Решения на основе разного количества итераций:
for(int n=1; n<=5; n++){
cout<<n<<": "<<findRoot(F, x0, n)<<"\t";
cout<<findRoot(G, z0, n)<<endl;
}
return 0;
}
```

В программе описывается функция `findRoot()`, у которой три аргумента и которая возвращает в качестве результата значение типа `double`. Первым аргументом функции `findRoot()` передается указатель на функцию уравнения (имеется в виду функция $f(x)$ в правой части уравнения $x = f(x)$).

Указатель на функцию в аргументе другой функции описывается фактически так же, как объявляется указатель на функцию в ином месте программы: сначала указывается тип результата функции, затем в круглых скобках со звездочкой - название указателя, а в круглых скобках после данной конструкции - типы аргументов функции (в данном случае аргумент один). В рассматриваемом примере первый аргумент функции `findRoot()` описан выражением `double (*f)(double)`. Это означает буквально следующее: первым аргументом, который называется `f`, передается имя функции (указатель на функцию), у которой один аргумент типа `double` и такого же типа результат. В теле функции `findRoot()` можем отождествлять `f` с названием функции, определяющей правую часть решаемого уравнения.

Второй и третий аргументы функции `findRoot()` - соответственно начальное приближение для корня уравнения и количество итераций, по которым вычисляется корень. Результатом функцией возвращается вычисленное приближенное значение для корня уравнения.

В теле функции `findRoot()` со значением начального приближения для корня уравнения инициализируется переменная `x`, после чего запускается оператор цикла, и в нем всего одна команда `x=f(x)`. Каждый раз при выполнении команды вычисляется и записывается в переменную `x` новое приближение для корня уравнения. По завершении выполнения цикла значение переменной `x` возвращается в качестве результата функции `findRoot()`.

Помимо функции `findRoot()` в программе описаны функции `F()` и `G()`, определяющие правые части уравнений $x = \sqrt{x+6}$ и $x = \frac{x^2+10}{11}$ соответственно. В главной функции программы данные уравнения решаются вызовом функции `findRoot()` с разными аргументами: для первого уравнения это команды вида `findRoot(F, x0, n)`, а второе уравнение решается с помощью команд вида `findRoot(G, z0, n)`. Причем переменная `n`, определяющая количество итераций, на основе которых строится решение, пробегает последовательно набор значений от 1 до 5. Последнее дает возможность проследить, как точность решения меняется с увеличением количества итераций. Результат выполнения программы следующий:

Результат выполнения программы (из листинга 10.2)

$x = F(x)$	$x = G(x)$
0: 0	5
1: 2.44949	3.18182
2: 2.9068	1.82945
3: 2.98443	1.21335
4: 2.9974	1.04293
5: 2.99957	1.00797

Несложно заметить, что решения найдены вполне корректно (особенно с учетом того, что было выполнено всего 5 итераций).

Подробности

Решением уравнения $x = \sqrt{x+6}$, как отмечалось ранее, является значение $x = 3$. Поиск корня этого уравнения начинается с начального приближения $x_0 = 0$. Функция $f(x) = \sqrt{x+6}$ имеет производную $f'(x) = \frac{1}{2\sqrt{x+6}}$, для которой условие $|f'(x)| < 1$ выполняется при $x > -5.75$. Начальное приближение $x_0 = 0$ для корня уравнения попадает в нужный диапазон.

Уравнение $x = \frac{x^2+10}{11}$ на самом деле является квадратным уравнением $x^2 - 11x + 10 = 0$ с корнями $x = 1$ и $x = 10$. Для функции $f(x) = \frac{x^2+10}{11}$ производная $f'(x) = \frac{2x}{11}$. Условие $|f'(x)| < 1$ дает $-5.5 < x < 5.5$. В данный диапазон попадает только корень $x = 1$. Именно его мы пытаемся найти, указав в качестве начального приближения значение $x_0 = 5$.

10.3. Знакомство с лямбда-функциями

Приедут тут всякие: без профессии, без подушек...

из к/ф "Девчата"

В предыдущем примере функции, определяющие решаемые уравнения, описывались в явном виде в программе. Другими словами, если мы хотим решить какое-то уравнение, то нам необходимо описать функцию, определяющую уравнение. Такое положение дел в принципе вполне приемлемо. Однако имеются и иные возможности. В данном конкретном случае (имеется в виду предыдущий пример) вместо того, чтобы описывать функцию для каждого из решаемых уравнений, можем передать первым аргументом функции `findRoot()` анонимную функцию, или лямбда-функцию.

На заметку

Лямбда-функции появились в стандарте языка C++ начиная с 2011 года (то есть в стандарте C++ 11).

Лямбда-функция - это, по большому счету, функция без имени. Данное утверждение может показаться странным, а само наличие лямбда-функций нелогичным и неоправданным, но на самом деле бывают ситуации, когда "функция без имени" - как раз то, что нужно. Один из таких примеров - когда функция нужна для "одноразового" использования (например, когда функция передается аргументом другой функции, как в нашем примере).

Есть несколько способов описания лямбда-функции. Мы начнем с самого простого. Общий шаблон описания лямбда-функции выглядит так:

```
[ ] (аргументы) {return значение;}
```

Описание начинается с пустых квадратных скобок [] (которые замещают фактически название функции), затем следуют круглые скобки с описанием аргументов функции (подобно обычной функции), а затем в фигурных скобках указывается инструкция `return` и значение, возвращается результатом функции. Например, объявление лямбда-функции, в качестве результата возвращающей квадрат аргумента, может выглядеть следующим образом:

```
[ ] (double x) {return x*x;}
```

**На заметку**

По большому счету лямбда-функция описывается как самая обычная функция, только вместо имени используются пустые квадратные скобки []. В фигурных скобках может быть только одна инструкция, причем это должна быть `return`-инструкция. Тип лямбда-функции явно не указывается и определяется автоматически на основе возвращаемого функцией значения.

Также подчеркнем, что все перечисленные правила относятся к данному способу описания лямбда-функции. Лямбда-функции можно описывать и по-другому. Об этом мы тоже поговорим.

В предыдущем примере мы рассматривали уравнения с функциями

$F(x) = \sqrt{x+6}$ и $G(x) = \frac{x^2+10}{11}$ в правых частях. Эти функции при определении их в виде лямбда-функций будут выглядеть так:

```
[ ] (double x) {return sqrt(x+6);}
```

для функции $F(x) = \sqrt{x+6}$ и

```
[](double x){return (x*x+10)/11;}
```

для функции $G(x) = \frac{x^2+10}{11}$.

Естественным образом возникает вопрос: что можно делать с лямбда-функцией? Как ее использовать? Чтобы получить ответ, имеет смысл отождествить инструкцию описания лямбда-функции с названием функции. Тогда становится понятно, как лямбда-функцию передать аргументом другой функции: в том месте, где должно быть название функции-аргумента следует разместить описание лямбда-функции. Именно такой подход использован в примере, представленном в листинге 10.3. Данный пример - модификация предыдущей задачи, только теперь функции, определяющие решаемые уравнения, не описываются явно, а определяются через лямбда-функции. Для удобства восприятия лишние комментарии в программном коде удалены, а наиболее важные места выделены жирным шрифтом.

Листинг 10.3. Использование лямбда-функции при передаче аргумента

```
#include <iostream>
#include <cmath>
using namespace std;
double findRoot(double (*f)(double), double x0, int n){
    double x=x0;
    for(int i=1; i<=n; i++){
        x=f(x);
    }
    return x;
}
int main(){
    double x0=0.0, z0=5.0;
    cout<<"x=sqrt(x+6)"<<"\t"<<"x=(x*x+10)/11"<<endl;
    cout<<"0: "<<x0<<"\t\t"<<z0<<endl;
    for(int n=1; n<=5; n++){
        cout<<n<<" ";
        // Использование лямбда-функции:
        cout<<findRoot([](double x){return sqrt(x+6);}, x0, n);
        cout<<"\t";
        // Использование лямбда-функции:
        cout<<findRoot([](double x){return (x*x+10)/11;}, z0, n);
        cout<<endl;
    }
    return 0;
}
```

}

Ниже приведен результат выполнения данного программного кода:

Результат выполнения программы (из листинга 10.3)

<code>x=sqrt(x+6)</code>	<code>x=(x*x+10)/11</code>
0: 0	5
1: 2.44949	3.18182
2: 2.9068	1.82945
3: 2.98443	1.21335
4: 2.9974	1.04293
5: 2.99957	1.00797

За исключением незначительных "декоративных" элементов, результат выполнения программы (по сравнению с предыдущим примером) не изменился. Также должна быть знакома и понятна основная часть программного кода. Изменений на самом деле не много: отсутствует описание функций, определяющих уравнения, и присутствуют инструкции с описанием лямбда-функций. Речь идет о выражениях `findRoot([](double x){return sqrt(x+6);},x0,n)` и `findRoot([](double x){return (x*x+10)/11;},z0,n)`, в которых функция `findRoot()` вызывается с аргументами, заданными через лямбда-функции.



На заметку

В описании лямбда-функции можно явно указать тип возвращаемого результата. Если лямбда-функция описывается с явным указанием результата, то между круглыми скобками с аргументами функции и фигурными скобками с кодом функции размещается оператор стрелка (`->`) и идентификатор типа результата, возвращаемого функцией. Например, инструкцией `[] (double x, int n) -> double {double z; z=pow(x,n)/n; return z;}` описывается лямбда-функция с двумя аргументами (один типа `double`, а другой типа `int`), возвращающая результатом значение типа `double`. Очевидно, речь идет о функциональной зависимости

$$f(x, n) = \frac{x^n}{n}.$$

Вообще же лямбда-функции представляют собой элегантный, но достаточно ограниченный (в плане реализационных возможностей) инструмент. Концепция лямбда-функций ориентирована на парадигму *функционального программирования*. В рамках парадигмы функционального программирования значение функции определяется исключительно ее аргументами, не может зависеть от значения "внешних" факторов и не подразумевает никаких "внешних" эффектов. Поэтому, например, при определении лямбда-функции не получится использовать значения внешних переменных.

Ситуация усугубляется еще и тем, что в C++ нет внутренних функций (то есть нельзя описать функцию в функции). Как следствие, вернуть в качестве результата функции лямбда-функцию не получится. Из всего сказанного вывод простой и очевидный: особых надежд на лямбда-функции возлагать не стоит. В этом отношении намного более эффективным и продуктивным является подход, который базируется на использовании функторов.

10.4. Массив указателей на функцию

*Начинаю действовать без шума и пыли по вновь утвержденному плану.
из к/ф "Бриллиантовая рука"*

Элементами массива могут быть указатели на функцию. В принципе, такая ситуация довольно экзотическая, но в то же время и вполне реальная. Далее как небольшую иллюстрацию рассмотрим программный код, в котором описывается три функции `my_exp()`, `my_sin()` и `my_cos()` соответственно для вычисления экспоненты, синуса и косинуса.



На заметку

Для вычисления экспоненты использован ряд Тейлора

$$\exp(x) \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}.$$

Синус вычисляется по формуле

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots + \frac{(-1)^n x^{2n+1}}{(2n+1)!},$$

а для вычисления косинуса использована формула

$$\cos(x) \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots + \frac{(-1)^n x^{2n}}{(2n)!}.$$

Точность вычислений тем выше, чем больше значение для верхней границы ряда n .

В главной функции программы создается массив указателей на функцию, состоящий из трех элементов. Значениями элементам присваиваются имена упомянутых функций. Затем функции для вычисления значений экспоненты, синуса и косинуса вызываются через указатели из массива. В листинге 10.4 приведен программный код примера.

Листинг 10.4. Массив указателей на функцию

```
include <iostream>
#include <string>
#include <cstdio>
using namespace std;
// Верхняя граница ряда для вычисления
// экспоненты, синуса и косинуса:
const int n=100;
// Постоянная "пи":
const double pi=3.141592;
```

```

// Функция для вычисления экспоненты:
double my_exp(double x){
double s=0,q=1;
for(int k=0;k<=n;k++){
    s+=q;
    q*=x/(k+1);
}
return s;
}
// Функция для вычисления синуса:
double my_sin(double x){
double s=0,q=x;
for(int k=0;k<=n;k++){
    s+=q;
    q*=(-1)*x*x/(2*k+2)/(2*k+3);
}
return s;
}
// Функция для вычисления косинуса:
double my_cos(double x){
double s=0,q=1;
for(int k=0;k<=n;k++){
    s+=q;
    q*=(-1)*x*x/(2*k+1)/(2*k+2);
}
return s;
}
// Главная функция программы:
int main(){
    // Массив указателей на функцию:
    double (*f[3])(double);
    // Текстовый массив с названиями функций:
    string names[3]={"exp","sin","cos"};
    // Числовой массив со значениями аргументов
    // для передачи функциям:
    double x[6]={0,pi/6,pi/4,pi/3,1,pi/2};
    // Значение элементов массива:
    f[0]=my_exp;
    f[1]=my_sin;
    f[2]=my_cos;
    // Вызов функций через указатели - элементы массива:
    for(int i=0;i<3;i++){
        cout<<names[i]<<":\t";
        for(int j=0;j<6;j++){
            // Команда с вызовом функции через
            // указатель - элемент массива:

```

```
printf("%f", f[i] (x[j]));
printf("%s", "\t");
    }
cout<<endl;
}
return 0;
}
```

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 10.4)

```
exp: 1.000000 1.688092 2.193280 2.849653 2.718282 4.810476
sin: 0.000000 0.500000 0.707107 0.866025 0.841471 1.000000
cos: 1.000000 0.866025 0.707107 0.500000 0.540302 0.000000
```

Здесь мы встречаемся с несколькими, относительно новыми моментами. В первую очередь стоит обратить внимание на инструкцию `double (*f[3]) (double)`, которой объявляется массив `f` указателей на функцию. После имени массива указывается в квадратных скобках размер массива. Слева от имени массива есть звездочка `*`. Вся эта конструкция заключается в круглые скобки. Справа в круглых скобках ключевое слово `double` свидетельствует о том, что речь идет об указателях на функцию с одним `double`-аргументом. Наконец, ключевое слово `double` слева в самом начале всей инструкции говорит о том, что функция, на которую ссылается указатель из массива, в качестве результата возвращает значение типа `double`.



На заметку

Инструкция `double (*f[3]) (double)` достаточно запутанная и понять ее смысл не так просто. В подобных случаях можно пользоваться несколькими формальными правилами. Кратко сводятся они вот к чему:

- при "расшифровке" сложного объявления квадратные скобки `[]` означают "массив";
- звездочка `*` означает "указатель";
- круглые скобки (с идентификатором типа или без) означают "функцию";
- "расшифровку" следует начинать с идентификатора имени, а заканчивается она идентификатором типа, с которого начинается объявление;
- "расшифровка" выполняется по принципу "маятника": сначала слева направо (до окончания инструкции или до закрывающей круглой скобки), затем справа налево (до открывающей круглой скобки), затем снова слева направо, и так далее.

В данном случае "расшифровку" инструкции `double (*f[3]) (double)` начинаем с имени `f`. Далее движемся вправо и встречаем инструкцию `[3]`, что означает

"массив из 3 элементов". После этого встречаем закрывающую скобку, поэтому движемся влево и встречаем *. Звездочка означает "указатель". В итоге мы уже получили: "f это массив из 3 элементов, которые являются указателями". Слева от звездочки - открывающая круглая скобка. Поэтому движение влево прекращается, и начинается движение вправо. Там осталась инструкция (double). Инструкция означает функцию с аргументом типа double. Получаем фразу "f это массив из 3 элементов, которые являются указателями на функцию, у которой аргумент типа double". Дальше двигаться вправо некуда, поэтому направление движения - влево до самой первой инструкции double. Это результат, возвращаемый функцией. Итак, финальная фраза такая: "f это массив из 3 элементов, которые являются указателями на функцию, у которой аргумент типа double и которая возвращает результат типа double".

Кроме массива указателей на функцию в программе командами `string names[3]={"exp","sin","cos"}` и `double x[6]={0,pi/6,pi/4,pi/3,1,pi/2}` объявляется и инициализируется текстовый массив `names` с названиями функций и числовой массив `x` со значениями аргумента (для вычисления значений каждой из трех функций). Командами `f[0]=my_exp`, `f[1]=my_sin` и `f[2]=my_cos` элементам массива `f` присваиваются в качестве значений имена функций `my_exp()`, `my_sin()` и `my_cos()`.



На заметку

Программный код функций `my_exp()`, `my_sin()` и `my_cos()` фактически отличается только начальным значением переменной `q` и командой в теле оператора цикла, которой изменяется значение этой переменной. Понять это несложно, если принять во внимание, что во всех трех случаях речь идет о вычислении суммы вида

$q_0 + q_1 + q_2 + \dots + q_n$ (индекс возле q нумерует итерацию). Для экспонен-

ты $q_k = \frac{x^k}{k!}$, для синуса $q_k = \frac{(-1)^k x^{2k+1}}{(2k+1)!}$, а для косинуса $q_k = \frac{(-1)^k x^{2k}}{(2k)!}$.

Чтобы получить значение параметра q для следующей итерации, текущее значение необходимо умножить на величину $\frac{q_{k+1}}{q_k}$. Для экспоненты $\frac{q_{k+1}}{q_k} = \frac{x}{k+1}$, для

синуса $\frac{q_{k+1}}{q_k} = \frac{(-1)x^2}{(2k+2)(2k+3)}$, для косинуса $\frac{q_{k+1}}{q_k} = \frac{(-1)x^2}{(2k+1)(2k+2)}$.

После присваивания значений элементам массива `f`, запускается оператор цикла, в котором через указатели в массиве вычисляются значения функций для разных аргументов. Для аргумента `x[j]` вычисление значения функции выполняется командой `f[i](x[j])`.

На заметку

Исключительно ради удобства, и исходя из эстетических соображений, для отображения данных в окне вывода (наряду с оператором вывода), нами использова-

на функция `printf()`. Для этого нам пришлось подключить заголовок `<cstdio>`.

Функция `printf()` позволяет выполнять форматированный вывод. Первым аргументом функции передается текстовая строка, определяющая формат отображения данных. Сами отображаемые данные - второй аргумент функции `printf()`.

Строка форматирования начинается с символа `%`. Буква после символа `%` определяет тип данных, которые отображаются. Буква `s` соответствует текстовым значениям, а буква `f` соответствует действительным числам.

10.5. Функция как результат

Дорогу осилит идущий: Вам необходим творческий непокой.

из к/ф "Покровские ворота"

Ранее отмечалось, что функция в качестве результата может возвращать практически все, за исключением массива. В принципе, результатом функции может быть "функция". Другой вопрос, как, в каком виде, эту "функцию" реализовать? Мы рассмотрим два фундаментальных подхода. В первом случае результатом функции возвращается указатель на функцию. Во втором случае результатом функции будет функтор. Здесь рассмотрим ситуацию, когда результатом функции возвращается указатель на функцию.



На заметку

Как отмечалось ранее, в C++ нельзя использовать внутренние функции. Поэтому когда мы говорим о том, что функция возвращает результатом указатель на другую функцию, то эта "другая" функция должна быть внешней.

В листинге 10.5 представлена программа, в которой описаны три функции: `first()`, `second()` и `third()`. У каждой из них один аргумент типа `char`, а результатом каждая функция возвращает значение типа `string`. Кроме этих функций в программе описана функция `getFunction()` с целочисленным аргументом, возвращающая результатом указатель на функцию. В зависимости от значения целочисленного аргумента, который передается функции `getFunction()`, функцией возвращается указатель на одну из функций `first()`, `second()` или `third()`.

В главной функции программы объявляется указатель `f` на функцию. Запускается оператор цикла, в нем указателю `f` присваивается значение, после чего через этот указатель вызывается соответствующая функция. Весь код выглядит так:

Листинг 10.5. Указатель на функцию как результат функции

```
#include <iostream>
#include <string>
```

```

using namespace std;
// Первая функция:
string first(char sym){
string txt="Первая функция: ";
return txt+sym;
}
// Вторая функция:
string second(char sym){
string txt="Вторая функция: ";
return txt+sym;
}
// Третья функция:
string third(char sym){
string txt="Третья функция: ";
return txt+sym;
}
// Функция с результатом - указателем на функцию:
string (*getFunc(int m))(char){
// Циклическая перестановка индекса:
double k=(m-1)%3+1;
// Результат - указатель на функцию first():
if(k==1) return first;
// Результат - указатель на функцию second():
if(k==2) return second;
// Результат - указатель на функцию third():
return third;
}
// Главная функция программы:
int main(){
// Указатель на функцию:
string (*f)(char);
// Вызов разных функций через указатель на функцию:
for(int i=1;i<=6;i++){
// Указателю присваивается значение:
f=getFunc(i);
// Вызов функции через указатель:
cout<<f('A'+i-1)<<endl;
}
return 0;
}

```

Обращает на себя внимание способ описания функции `getFunc()`. В первую очередь это конечно касается типа результата функции. В данном случае заголовок функции описан как `string (*getFunc(int m))(char)`. Если вспомнить правило расшифровки таких выражений, то получается,

что `getFunc` -функция с целочисленным аргументом, которая является указателем на функцию с аргументом типа `char` и результатом типа `string`.

В теле функции командой `k=(m-1)%3+1` переменной `k` присваивается значение в диапазоне от 1 до 3 включительно. Если аргумент функции `m` принимает значения из того же диапазона, то значения `k` и `m` совпадают. Если значение аргумента `m` выходит за пределы диапазона, то, как бы, выполняется циклическая перестановка индекса. Такой нехитрый прием обеспечивает нам уверенность в том, что при любом аргументе (целочисленном, разумеется) система последовательных условных операторов даст корректный результат. А результат формируется на основе значения `k` и представляет собой указатель на одну из функций `first()`, `second()` или `third()`.

В главной функции программы командой `string (*f)(char)` объявляется указатель `f` на функцию с одним `char`-аргументом, возвращающую результатом `string`-значение. В операторе цикла, который сразу после этого запускается, индексная переменная `i` пробегает значения от 1 до 6 включительно. Эта индексная переменная передается аргументом функции `getFunc()`. Последняя вызывается в команде присваивания `f=getFunc(i)`. После выполнения команды указатель `f` тождественен с названием одной из функций: `first()`, `second()` или `third()`.

С учетом циклической перестановки аргумента функции `getFunc()` каждая из указанных трех функций "выбирается" дважды. Каждый раз при этом выполняется вызов означенной функции в команде `cout<<f('A'+i-1)<<endl`. Здесь мы воспользовались тем, что к символьным значениям можно прибавлять целые числа. Соответствующая арифметическая операция выполняется с кодами символов, и результат интерпретируется как символ с соответствующим кодом. Поэтому хотя вызываемые функции будут циклически чередоваться, символьный аргумент в буквенном выражении каждый раз "увеличивается на единицу".

В итоге при выполнении программы получим такой результат:

Результат выполнения программы (из листинга 10.5)

Первая функция: A
 Вторая функция: B
 Третья функция: C
 Первая функция: D
 Вторая функция: E
 Третья функция: F

Выше мы имели дело с функциями (в том смысле, что они не были методами) и указателями на функции. Когда речь заходит о методах, то ситуация несколько меняется. Хотя на метод также можно создать указатель, но указатель на метод и указатель на функцию - далеко не одно и то же. Рассмотрим этот вопрос более детально.

10.6. Указатели на методы

Многие меня уважают. Некоторые даже боятся.

из к/ф "Служебный роман"

В предыдущих примерах мы смело использовали то обстоятельство, что имя функции является указателем на функцию - фактически, ее адресом. В общем-то, имя метода тоже является указателем - на метод. Но если в случае с функцией указатель представляет собой *абсолютный* адрес функции, то через указатель для метода реализуется *относительный* адрес метода: адрес метода определяется через смещение относительно указателя `this` на объект вызова. Но если абстрагироваться от технических подробностей, то главный вывод состоит в том, что указатель на метод сам по себе не позволяет вызвать данный метод - нужно указать объект, из которого вызывается метод. Эта особенность указателей на методы находит проявление в способе их объявления.

Рецепт состоит в том, чтобы явно указать класс метода. Например, если речь идет о *методе* определенного *класса*, то указатель на метод получают с помощью инструкции `&класс::метод`. Здесь перед именем метода (через оператор расширения контекста `::`) не только указывается имя класса, но еще и используется оператор `&` получения адреса.

Указатель на метод некоторого *класса* объявляется так же, как и указатель на функцию, но только вместо звездочки `*` перед именем указателя используется выражение вида `класс::*`. Вызов метода через указатель выполняется с явным указанием объекта и с использованием оператора `*` для получения значения по указателю. Так, если нужно вызвать метод через *указатель* на метод, и вызов выполняется через некоторый *объект*, то команда вызова метода будет иметь вид `(объект.*указатель)(аргументы)` (скобки использованы для соблюдения нужной последовательности операций).

Все сказанное иллюстрирует небольшой пример, представленный в листинге 10.6. Фактически это вариация на тему предыдущего примера, но только теперь с использованием класса `MyClass` и указателей на методы класса. В классе описаны три закрытых метода `first()`, `second()` и `third()`. У

каждого из методов символьный аргумент и каждый из методов возвращает в качестве результата текстовое значение. Причем текст, возвращаемый методами, содержит значение текстового поля `name` того объекта, из которого вызывается метод (это позволяет однозначно идентифицировать объект вызова).

Открытый метод `getMeth()` на основании целочисленного аргумента (который, фактически, играет роль индекса) возвращает в качестве результата указатель на один из методов `first()`, `second()` или `third()`.

В главной функции программы создаются два объекта (А и В) класса `MyClass` и объявляется указатель `f` на метод класса. С помощью оператора цикла и метода `getMeth()` указателю `f` присваиваются различные значения и через указатель (и один из объектов класса `MyClass`) выполняется вызов методов `first()`, `second()` и `third()`. Рассмотрим детальнее программный код данного примера:

Листинг 10.6. Указатель на метод

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
private:
    // Текстовое поле:
    string name;
    // Первый метод:
    string first(char sym){
        string s="Объект "+name+". ";
        string txt="Первый метод: ";
        return s+txt+sym;
    }
    // Второй метод:
    string second(char sym){
        string s="Объект "+name+". ";
        string txt="Второй метод: ";
        return s+txt+sym;
    }
    // Третий метод:
    string third(char sym){
        string s="Объект "+name+". ";
        string txt="Третий метод: ";
        return s+txt+sym;
    }
public:
```

```

    // Конструктор:
    MyClass(string name){
    this->name=name;
    }

    // Метод, результат которого - указатель на метод:
    string (MyClass::*getMeth(int m))(char){
    // Циклическая перестановка индекса:
    int k=(m-1)%3+1;
    // Результат - указатель на первый метод:
    if(k==1) return &MyClass::first;
    // Результат - указатель на второй метод:
    if(k==2) return &MyClass::second;
    // Результат - указатель на третий метод:
    return &MyClass::third;
    }
}; // Завершение описания класса
// Главная функция программы:
int main(){
    // Первый объект:
    MyClass A("Альфа");
    // Второй объект:
    MyClass B("Браво");
    // Указатель на метод:
    string (MyClass::*f)(char);
    // Вызов методов через указатель:
    for(int i=1;i<=6;i++){
        // Значение указателя на метод:
        f=A.getMeth(i);
        // Вызов метода из первого объекта через указатель:
        cout<<(A.*f)('A'+i-1)<<endl;
        // Вызов метода из второго объекта через указатель:
        cout<<(B.*f)('a'+i-1)<<endl;
        cout<<endl;
    }
    return 0;
}

```

Конструктор класса `MyClass` описан так, что при создании объекта необходимо указать один текстовый аргумент. Аргумент определяет значение поля `name` создаваемого объекта.

Прототип метода `getMeth()` достаточно нетривиален: `string (MyClass::*getMeth(int m))(char)`. Хотя с другой стороны, похожие конструкции мы видели. Просто теперь выражение `MyClass::*` слева от имени метода означает, что это не просто указатель на функцию, а указатель на метод класса `MyClass`.



На заметку

Хотя метод вызывается из объекта, указатель на метод выполняется со ссылкой на имя класса. Это не случайно. Ранее отмечалось, что указатель на метод является не абсолютным, а относительным. Поэтому для вызова метода в качестве "отправной точки" нужен объект. Но относительный адрес метода определяется кодом класса, на основе которого создается объект. Как следствие, значение указателя на метод определяется классом. Фактически, речь идет о том, что для разных объектов один и тот же метод имеет один и тот же относительный адрес. В результате значение указателя на метод одно и то же для всех объектов класса. Но когда мы вызываем метод, указываем объект. У разных объектов разные адреса. Определяя при вызове методы на основе относительных адресов по отношению к разным абсолютным адресам разных объектов, получаем доступ к методам этих объектов.

Если провести аналогию с автомобилем, то ситуация примерно такая: для автомобилей данной модели рычаг коробки передач находится на определенном расстоянии от рулевого колеса. То есть, какой бы автомобиль мы ни взяли, коробка передач относительно рулевого колеса всегда на одном и том же расстоянии (имеется в виду не только само расстояние, но и направление). Но разные автомобили в пространстве размещены по-разному. Если мы указали "координаты" автомобиля, то определить, где находится коробка передач автомобиля, не составит проблемы. Автомобиль - аналог объекта, рычаг коробки передач - аналог метода, расстояние от рулевого колеса до рычага коробки передач - аналог относительного адреса метода.

В теле метода `getMeth()` командой `int k=(m-1)%3+1` в переменную `k` записывается одно из значений (от 1 до 3 включительно). В зависимости от значения переменной `k` результатом метода возвращается указатель на один из методов `first()`, `second()` или `third()` -соответственно значения `&MyClass::first`, `&MyClass::second` и `&MyClass::third`. Результат метода формируется так: перед именем метода, черезоператор расширения контекста `::`, указывается имя класса `MyClass`, и перед этой конструкцией указывается оператор `&` получения адреса. Получается указатель на метод.

В главной функции командой `string (MyClass::*f)(char)` объявляется указатель `f` на метод класса `MyClass`. В отличие от объявления указателя на функцию, при объявлении указателя на метод мы указываем имя класса.

В операторе цикла переменная `i` пробегает значения от 1 до 6, а командой `f=A.getMeth(i)` указателю на метод присваивается значение. При этом адрес метода, на который выполняется ссылка, получаем вызовом метода `getMeth()` из объекта `A`. Метод, на который ссылается указатель `f`, вызывается в командах `cout<<(A.*f) ('A'+i-1)<<endl` и `cout<<(B.*f) ('a'+i-1)<<endl`. Если в первом случае результат ожидаем (метод вызывается из объекта `A`), то во втором случае вызывается метод из объекта `B`. В последнем несложно убедиться, взглянув на результат выполнения программы:

Результат выполнения программы (из листинга 10.6)

Объект Альфа. Первый метод: A
 Объект Bravo. Первый метод: a

Объект Альфа. Второй метод: B
 Объект Bravo. Второй метод: b

Объект Альфа. Третий метод: C
 Объект Bravo. Третий метод: c

Объект Альфа. Первый метод: D
 Объект Bravo. Первый метод: d

Объект Альфа. Второй метод: E
 Объект Bravo. Второй метод: e

Объект Альфа. Третий метод: F
 Объект Bravo. Третий метод: f

Достоинно внимание следующее: значение указателю `f` присваивается через объект `A` и по логике присваиваемое указателю значение должно представлять собой адрес метода из данного объекта. Практика вызова методов через указатель `f` и объект `A` вроде бы подтверждают такую версию. Но вот если мы вызываем метод через тот же указатель `f`, но с привлечением объекта `B`, то вызывается метод объекта `B`. Объяснение простое: мы имеем дело с *относительными адресами*, поэтому объект, относительно которого "отсчитывается" адрес метода, имеет принципиальное значение. Эта особенность указателей на методы уже упоминалась ранее.

10.7. Возвращаясь к функторам

Это был титанический труд. На нем-то вы и надорвались.

из к/ф "Старики-разбойники"

Хотя указатели на функции и методы - механизм достаточно удобный и элегантный, многие задачи легче решать с привлечением функторов. В предыдущей главе функторы уже обсуждались. Здесь мы продолжим исследование темы функторов и как небольшую иллюстрацию рассмотрим несколько примеров (точнее, два). В первом примере через функторы реализуется массив функций. Точнее, мы создадим иллюзию того, что имеем дело с массивом функций, хотя на самом деле, конечно, все это реализуется через объект. Во втором примере мы рассмотрим задачу о вычислении

производной. Это математическая задача, а решать мы ее будем, определив функтор, позволяющий вычислить значение для производной от заданной при создании объекта функции.

В листинге 10.7 представлена программа, в которой реализуется идея о создании массива функций. Здесь мы встречаемся с *внутренним классом*.



На заметку

В теле класса можно описать другой класс. Класс, описанный внутри другого класса, называют внутренним классом. Класс, в котором описан внутренний класс, называется внешним классом.

Вкратце мы использовали следующий подход:

- В программе создается класс `FArray`, в котором описаны три метода `my_exp()`, `my_sin()` и `my_cos()`, предназначенные для вычисления экспоненты, синуса и косинуса соответственно (аналогичные функции были рассмотрены в одном из примеров в этой главе). При вызове методов передается значение аргумента, для которого вычисляется результат. Верхняя граница ряда, на основе которого вычисляется результат, запоминается в целочисленном поле объекта класса.
- В классе `FArray` описывается внутренний класс `F`. Если объект класса `FArray` используется для реализации (иллюзии) массива функций, то через объекты внутреннего класса `F` реализуются элементы такого массива.
- Объекты внутреннего класса `F` можно вызывать - то есть они являются функторами. При вызове объекта класса `F` вызывается один из методов `my_exp()`, `my_sin()` или `my_cos()` класса `FArray`. Какой именно вызывается метод, определяется на основе значения целочисленного поля `index` объекта внутреннего класса. Значение поля `index` задается при создании объекта внутреннего класса.
- Во внешнем классе `FArray` описан операторный метод для квадратных скобок, что позволяет индексировать объекты класса `FArray`. Благодаря данному обстоятельству с объектами класса `FArray` можно обращаться как с массивами (но элементы получится только считывать). При этом индексирование объекта класса `FArray` приводит к созданию объекта внутреннего класса с соответствующим значением индекса.

Весь программный код примера приведен ниже:

Листинг 10.7. Реализация "массива функций" с помощью функтора

```

#include <iostream>
using namespace std;
// Постоянная "пи":
const double pi=3.141592;
// Класс для реализации массива функций:
class FArray{
private:
    // Целочисленное поле - верхняя граница ряда
    // при вычислении значений экспоненты, синуса
    // икосинуса:
    int n;
    // Метод для вычисления экспоненты:
    double my_exp(double x){
    double s=0,q=1;
    for(int k=0;k<=n;k++){
        s+=q;
        q*=x/(k+1);
    }
    return s;
    }
    // Метод для вычисления синуса:
    double my_sin(double x){
    double s=0,q=x;
    for(int k=0;k<=n;k++){
        s+=q;
        q*=(-1)*x*x/(2*k+2)/(2*k+3);
    }
    return s;
    }
    // Метод для вычисления косинуса:
    double my_cos(double x){
    double s=0,q=1;
    for(int k=0;k<=n;k++){
        s+=q;
        q*=(-1)*x*x/(2*k+1)/(2*k+2);
    }
    return s;
    }
    // Внутренний класс:
    class F{
    private:
        // Целочисленное поле - индекс
        // "функции" в "массиве":
        int index;

```

```

        // Указатель на объект внешнего класса:
FArray *p;
public:
        // Конструктор внутреннего класса:
F(int m,FArray *p){
    // Значение индекса:
    index=m%3;
        // Указатель на объект:
    this->p=p;
    }
        // Перегрузка оператора "круглые скобки"
        // для объекта внутреннего класса:
double operator()(double x){
    if(index==0) return p->my_exp(x);
    if(index==1) return p->my_sin(x);
    return p->my_cos(x);
    }
}; // Окончание описания внутреннего класса
public:
        // Конструктор класса:
FArray(int n){
        // Значение поля:
    this->n=n;
    }
        // Перегрузка оператора "квадратные скобки":
F operator[](int k){
        // Локальный объект внутреннего класса:
    F tmp(k,this);
        // Результат операторного метода:
    return tmp;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Объект для реализации "массива функций":
    FArray mf(10);
        // Вызов "функций" из "массива":
    cout<<"exp(1)="<<mf[0](1)<<endl;           // Экспонента
    cout<<"sin(pi/6)="<<mf[1](pi/6)<<endl; // Синус
    cout<<"cos(pi/3)="<<mf[2](pi/3)<<endl; // Косинус
    return 0;
}

```

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 10.7)

```
exp(1)=2.71828
sin(pi/6)=0.5
cos(pi/3)=0.5
```

На что хочется обратить внимание? Конструктор внешнего класса `FArray` описан так, что при создании объекта класса необходимо указать аргумент - целое число, определяющее верхнюю границу ряда при вычислении математических функций. Чем больше соответствующее значение, тем выше точность вычислений. Разные объекты класса `FArray` отличаются значением данного поля. Но это на самом деле технический, непринципиальный момент.

**На заметку**

Программный код методов для вычисления экспоненты, синуса и косинуса здесь анализировать не будем - мы делали уже это ранее в другом примере. Правда, там речь шла о функциях, но различия на самом деле "косметические".

Операторный метод `operator[]()` в классе `FArray` описан так, что аргументом методу передается целое число (индекс в квадратных скобках после имени объекта), а результатом возвращается объект внутреннего класса `F`. Аргументами конструктору при создании объекта внутреннего класса передаются индекс (аргумент операторного метода) и указатель `this` на индексируемый объект.

Подробности

То есть получается так: объект класса `FArray` можно индексировать. При индексировании объекта класса `FArray` создается и возвращается объект внутреннего класса `F`, который можно вызывать.

При создании объекта класса `F` конструктору передаются два аргумента: целочисленное значение, соответствующее индексу элемента массива, который "имитируется" объектом, а также указатель `this` на индексируемый объект класса `FArray` (наш "импровизированный" массив). Зачем нужен указатель `this`? Все дело в том, что при вызове объекта внутреннего класса `F` на самом деле будет вызываться один из методов (`my_exp()`, `my_sin()` или `my_cos()`) внешнего класса `FArray`. Но эти методы обычные (не статические). При вызове методов необходимо указывать объект, из которого они вызываются - объект класса `FArray`. Поэтому объекту класса `F` при создании и передается указатель `this`.

Как отмечалось, у внутреннего класса `F` два поля: в целочисленное поле `index` заносится значение индекса элемента, который отождествляется с объектом класса, а указатель `p` на объект внешнего класса `FArray` содер-

жит адрес того объекта, из которого предстоит вызывать методы `my_exp()`, `my_sin()` и `my_cos()`.



На заметку

Подчеркнем еще раз: объект внешнего класса - это тот объект, через который реализуется массив функций (или который мы отождествляем с массивом функций).

Конструктору класса `F` передаются аргументами значения, присваиваемые полям `index` и `p`. Причем при присваивании значения полю `index` выполняется процедура вычисления остатка от деления на 3 (чтобы индекс элемента с неизбежностью попадал в диапазон допустимый значений от 0 до 2).

Перегрузка оператора "круглые скобки" выполняется так: проверяется значение поля `index` и в зависимости от значения данного поля вызывается тот или иной из методов `my_exp()`, `my_sin()` или `my_cos()`. Аргументом вызываемому методу передается значение, указанное в круглых скобках при вызове объекта класса `F`. Поскольку речь идет о вызове методов объекта внешнего класса, указатель `p` на объект явно присутствует в командах вызова методов.

В главной функции программы командой `FArray mf(10)` создается объект класса `FArray`, через который мы реализуем "массива функций".



На заметку

Математические функции, рассчитываемые на основе данного объекта, вычисляются на основе рядов с верхней границей суммы, равной 10.

Еще раз подчеркнем, что имеем дело с объектом, не массивом. Просто этот объект можно индексировать, а результат индексирования можно вызывать: как, например, в инструкциях `mf[0](1)` (вычисление экспоненты $\exp(1) \equiv e \approx 2.718281828$), `mf[1](pi/6)` (вычисление синуса $\sin(\frac{\pi}{6}) = 0.5$) и `mf[2](pi/3)` (вычисление косинуса $\cos(\frac{\pi}{3}) = 0.5$).

В следующем примере решается задача о нахождении производной.



На заметку

Производной для функции $f(x)$ называется такая функция $f'(x)$ (или $\frac{df}{dx}$), которая равна пределу отношения приращения функции к приращению аргумента при стремлении последнего к нулю: записывается как $f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$.

При числовых расчетах обычно используют приближенное выражение для произ-

водной $f'(x) \approx \frac{f(x+\Delta x) - f(x)}{\Delta x}$, причем, чем меньше приращение Δx , тем большей точности можно ожидать при вычислении производной.

Программа, в которой на основе исходной (дифференцируемой) функции вычисляется производная, представлена в листинге 10.8.

Листинг 10.8. Вычисление производной

```
#include <iostream>
#include <cmath>
using namespace std;
// Класс для вычисления производной:
class DF{
private:
    // Поле - указатель на дифференцируемую функцию:
    double (*f)(double);
    // Поле - значение приращения аргумента для
    // вычисления производной:
    double dx;
public:
    // Конструктор (аргументы: указатель на
    // дифференцируемую функцию и приращение аргумента):
    DF(double (*f)(double), double dx){
    // Значения полей:
    this->f=f;
    this->dx=dx;
    }
    // Операторный метод для "вызова" объекта.
    // Результат - значение производной:
    double operator()(double x){
        // Значение производной:
        return (f(x+dx)-f(x))/dx;
    }
}; // Окончание описания класса
// Функция для вычисления производной.
// Аргументы - указатель на дифференцируемую функцию и
// приращение аргумента:
DF diff(double (*f)(double), double dx){
    // Объект (функтор) для вычисления производной:
    DF obj(f, dx);
    // Результат функции - функтор:
    return obj;
}
// Функция для дифференцирования:
double G(double x){
```

```

return exp(x*x/2);
}
// Функция - результат дифференцирования:
double g(double x){
return x*exp(x*x/2);
}
// Главная функция программы:
int main(){
    // Объект (функтор) для вычисления производной:
    DF df(G,0.001);
    cout<<"Производная для функции y(x)=exp(x*x/2):\n";
    // Вычисление значения производной:
    for(double z=0;z<=2;z+=0.5){
    cout<<df(z)<<" vs. "<<g(z)<<endl;
    }
    cout<<"Производная для функции y(x)=x*(1-x):\n";
    // Вызов функции для вычисления производной:
    df=diff([](double x)->double{return x*(1-x);},0.001);
    // Вычисление значения производной:
    for(double z=0;z<=1;z+=0.25){
    cout<<df(z)<<" vs. "<<[](double x)->double{return 1-2*x;}
    (z)<<endl;
    }
    return 0;
}

```

Основу программы составляет класс `DF` с полем `f`, являющимся ссылкой на функцию, у которой один аргумент типа `double` и результатом является значение типа `double`. В поле `f` записывается адрес дифференцируемой функции (функции, для которой вычисляется производная). Поле `dx` типа `double` определяет значение аргумента, на основе которого вычисляется значение производной. Конструктору класса аргументами передаются адрес функции для дифференцирования и значение приращения аргумента.

Операторный метод для круглых скобок позволяет вызывать объекты класса `DF`. Результатом операторного метода возвращается выражение для значения производной $(f(x+dx) - f(x)) / dx$ (через x обозначен аргумент операторного метода - то есть это то значение аргумента, для которого вычисляется значение производной).



На заметку

Таким образом, при создании объекта класса `DF` указывается адрес функции, для которой нужно вычислить производную. Вызывая такой объект, получаем значение производной.

Помимо класса `DF`, исключительно ради удобства и наглядности описана функция `diff()`. Аргументами функции передаются указатель на дифференцируемую функцию и значение приращения аргумента. Результатом функции возвращается объект класса `DF`, через который реализуется производная. Аргументы функции `diff()` передаются аргументами конструктору класса `DF` при создании объекта, возвращаемого результатом функции.

Функция `G()`, описанная в программе, соответствует функциональной зависимости $y(x) = \exp(x^2)$, а функция `g()` соответствует ее производной $y'(x) = x \cdot \exp(x^2)$.

В главной функции программы командой `DF df(G, 0.001)` создается объект `df`, позволяющий вычислять значение производной для функции, определяемой программной функцией `G()`. Затем запускается оператор цикла, и в нем для разных значений аргумента `z` вычисляется приближенное `df(z)` и точное `g(z)` значения для производной.

На следующем этапе выполняется команда `df=diff([](double x)->double{return x*(1-x);}, 0.001)`, которой объекту `df` присваивается в качестве значения результат вызова функции `diff()`. Причем первым аргументом функции `diff()` передана лямбда-функция `[](double x)->double{return x*(1-x);}` (что соответствует функции $y(x) = x(1 - x)$). После этого объект `df` при вызове возвращает значение для производной $y'(x) = 1 - 2x$. Для сравнения точного и приближенных значений для производной вызывается еще один оператор цикла. Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 10.8)

Производная для функции $y(x) = \exp(x^2/2)$:

```
0.0005 vs. 0
0.567283 vs. 0.566574
1.65037 vs. 1.64872
4.62533 vs. 4.62033
14.7966 vs. 14.7781
```

Производная для функции $y(x) = x*(1-x)$:

```
0.999 vs. 1
0.499 vs. 0.5
-0.001 vs. 0
-0.501 vs. -0.5
-1.001 vs. -1
```


Как видим, результат вычисления производной достаточно неплохой (в плане точности). Но в любом случае в рассмотренных выше примерах нас интересовала не столько математическая сторона проблемы, сколько программные подходы по ее решению.

Глава 11.

ВОЗВРАЩАЯСЬ К МАССИВАМ



Вы убедили меня. Я понял, что вам еще нужен.

из к/ф "Старики-разбойники"

В этой главе мы возвращаемся к теме массивов. С массивами мы имели дело неоднократно. Мы их создавали и использовали в самых разных ситуациях. Однако некоторые аспекты работы с массивами остались вне пределов нашего внимания. Здесь мы восполним, так сказать, пробелы.

Ранее обсуждались статические и динамические *одномерные массивы*. В одномерном массиве для однозначной идентификации элемента массива достаточно одного индекса. Далее мы остановимся на *двумерных массивах*. В двумерном массиве элемент массива идентифицируется посредством имени массива и двух индексов.



На заметку

Размерность массива может быть большей, чем два. Но такие массивы (их называют многомерными) на практике используются не очень часто, поэтому рассматривать их не будем. Тем не менее, принципы работы с многомерными массивами логично вытекают из тех подходов, которые используются при работе с массивами двумерными. Так что в случае необходимости читатель, скорее всего, без труда действует и многомерные массивы.

11.1. Двумерный статический массив

Ну, импрессионистов - так импрессионистов. Для тебя я на все готов.

из к/ф "Старики-разбойники"

При объявлении *двумерного статического массива* указывается тип элементов массива, название массива и размер массива по каждому из индексов (для каждого индекса используются отдельные квадратные скобки):

```
тип имя_массива [размер] [размер] ;
```

Например, следующей инструкцией объявляется двумерный массив `nums` из целых чисел (тип `int`), с размером 3 по первому индексу и размером 5 по второму индексу:

```
int nums[3][5];
```

Двумерный массив удобно представлять как таблицу или матрицу из элементов: размер по первому индексу определяет количество строк такой импровизированной таблицы, а размер по второму индексу определяет количество столбцов. Именно к такой аналогии мы будем прибегать в дальнейшем.

Обращение к элементам массива выполняется очень просто: после имени массива в квадратных скобках указываются индексы элемента массива. По каждому из индексов индексация начинается с нуля. Каждый индекс указывается в отдельных квадратных скобках.

Как иллюстрацию к использованию двумерных массивов рассмотрим небольшой пример, представленный в листинге 11.1. В программе создается статический двумерный массив и заполняется случайными числами. Затем значения элементов массива выводятся в окно вывода.

Листинг 11.1. Знакомство с двумерными массивами

```
#include <iostream>
#include <cstdlib>
using namespace std;
int main(){
    // Размеры массива:
    const int m=3,n=5;
    // Статический массив:
    int nums[m][n];
    int i,j;
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Заполнение массива случайными числами:
    for(i=0;i<m;i++){
        for(j=0;j<n;j++){
            // Присваивание значения элементу:
            nums[i][j]=rand()%10;
            // Отображение значения элемента:
            cout<<nums[i][j]<<" ";
        }
        cout<<endl;
    }
    return 0;
}
```

Ниже приведен возможный результат выполнения программы:

Результат выполнения программы (из листинга 11.1)

```
5 0 5 8 6
3 3 6 3 0
2 6 8 4 0
```

В данном случае все просто и особых пояснений ситуация, пожалуй, не требует. Но на практике дела обстоят несколько сложнее. Чтобы при работе с двумерными массивами не возникало проблем, следует учесть несколько "моментов". Главный, пожалуй, состоит в том, что с концептуальной точки зрения двумерный массив - это массив из массивов. Другими словами, о двумерном массиве можно думать как об одномерном массиве, элементами которого являются массивы.

Мы знаем, что имя одномерного массива представляет собой указатель на его первый элемент. В этом смысле ничего не поменялось: имя двумерного массива является указателем на первый элемент (однако нужно помнить, что таким элементом является целая строка). Данное свойство двумерных массивов часто используется на практике. Например, как и в случае одномерного массива, двумерный массив можно "спрятать" в объект и переопределить оператор "квадратные скобки" (операторный метод `operator[]()`) так, чтобы объекты индексировались.

Небольшой пример по этому поводу приведен в листинге 11.2. В примере описывается класс `Nums`, и в классе описано поле `nums`, являющееся двумерным статическим массивом. В конструкторе класса элементы массива заполняются случайными числами (диапазон значений от 1 до 9 включительно). В классе описан метод `show()` для отображения значений элементов массива, а также операторный метод `operator[]()`. Поэтому объекты класса `Nums` индексируются: внешне все выглядит так, как если бы объект класса `Nums` был двумерным массивом.

Далее имеет смысл подробнее рассмотреть соответствующий программный код:

Листинг 11.2. Индексирование объекта с полем - двумерным массивом

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Размеры массива:
const int m=3,n=5;
// Класс с полем - двумерным массивом:
class Nums{
public:
    // Поле - двумерный массив:
```

```

int nums[m][n];
    // Конструктор:
Nums() {
    // Заполнение массива случайными числами:
    for(int i=0;i<m;i++){ // Перебираются строки
    for(int j=0;j<n;j++){ // Перебираются столбцы
        // Элементу присваивается значение
        // (случайное число от 1 до 9).
        // Использована индексация объекта:
        (*this)[i][j]=rand()%9+1;
    }
    }
    // Метод для отображения содержимого массива:
void show(){
    for(int i=0;i<m;i++){ // Перебираются строки
    for(int j=0;j<n;j++){ // Перебираются столбцы
        // Отображается значение элемента массива.
        // Использована индексация объекта:
        cout<<(*this)[i][j]<<" ";
    }
    cout<<endl; // Переход к новой строке
}
    // Операторный метод для индексации объекта:
int *operator[](int k){
    // Результат метода - указатель
    // на строку массива:
    return nums[k];
}
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Создается объект с полем - массивом:
    Nums obj;
    cout<<"Двумерный массив:\n";
    // Содержимое массива:
    obj.show();
    // Присваивание элементам массива значений:
    obj[0][0]=0;
    obj[m-1][0]=0;
    obj[0][n-1]=0;
    obj[m-1][n-1]=0;
    cout<<"После внесения изменений:\n";

```

```
// Содержимое массива:
obj.show();
return 0;
}
```

Важное место в программном коде занимает операторный метод `operator[]()`. Способ описания метода в данном случае несколько отличается от подхода, применявшегося ранее.

Подробности

В рассматриваемых ранее примерах операторный метод в качестве значения обычно возвращал ссылку на элемент одномерного массива, который, в свою очередь, был полем класса. В таких случаях речь, во-первых, шла об одном элементе, и, во-вторых, возвращалось не просто значение элемента, а ссылка на элемент (чтобы была возможность элементу присвоить значение через процедуру индексирования объектов). В рассматриваемом здесь примере ситуация принципиально иная. Нам необходимо "легализовать" выражения вида `объект[индекс][индекс]` (две квадратных скобки после имени объекта). Проблема в том, что операторный метод для квадратных скобок "обрабатывает" выражения вида `объект[индекс]` (одна квадратная скобка после имени объекта). Чтобы выражение `объект[индекс][индекс]` имело смысл, необходимо, чтобы в результате вычисления выражения `объект[индекс]` возвращалось "нечто", допускающее индексирование. Другими словами нам необходимо так определить операторный метод `operator[]()`, чтобы результатом выражения вида `объект[индекс]` получился другой объект или массив, и для этого другого объекта или массива можно было бы указать квадратные скобки с индексом, и это все имело бы смысл. Примерно так.

В данном конкретном случае результатом операторного метода `объект[индекс]` формально является указатель на целое число, но на самом деле это указатель на первый элемент одномерного массива, который, в свою очередь, является строкой в двумерном массиве. Понятно, что одномерный массив индексируется. Так что все приличия соблюдены.

В соответствии с описанием метода `operator[]()` он возвращает указатель на целочисленное значение. Если посмотреть на программный код метода, то легко понять, что результатом возвращается выражение `nums[k]` (через `k` обозначен аргумент операторного метода). Напомним, что `nums` - это двумерный массив. Если после имени двумерного массива указать два индекса, получим значение соответствующего элемента. Возникает вопрос: что будет, если после имени двумерного массива указать только один индекс?

Чтобы получить ответ, следует вспомнить, что двумерный массив - это одномерный массив, элементами которого являются одномерные массивы. Если мы индексируем одномерный массив, получаем элемент массива. В данном случае таким элементом будет массив, который мы отождествляем со строкой в двумерном массиве. Поэтому `nums[k]` - строка в двумерном массиве с индексом `k`. Эта строка является одномерным массивом. А мы помним, что имя одномерного массива - указатель на его первый элемент.

Тогда выражение `nums[k]` мы можем интерпретировать как имя одномерного массива, возвращаемого в качестве результата операторного метода. Если проиндексировать этот одномерный массив, получим в качестве результата значение элемента массива - то есть число.



На заметку

В конструкторе при присваивании значений элементам двумерного массива и в методе `show()` при отображении значений элементов двумерного массива использована индексация объекта, возможная благодаря переопределению операторного метода `operator[]()`. Речь идет об инструкциях вида `(*this)[i][j]`, в которых обращение к объекту, из которого вызывается метод, реализовано через указатель `this`.

При выполнении программы командой `Nums obj` создается объект `obj` класса `Nums`. При создании объекта заполняется случайными числами двумерный массив этого объекта. Проверить содержимое массива удастся командой `obj.show()`. Обращаться к элементам массива можно индексирова объект (двумя индексами). Например, командами `obj[0][0]=0`, `obj[m-1][0]=0`, `obj[0][n-1]=0` и `obj[m-1][n-1]=0` присваиваются нулевые значения "угловым" элементам массива. Вообще же результат выполнения программы возможен следующий:

Результат выполнения программы (из листинга 11.2)

Двумерный массив:

```
1 6 4 9 9
5 1 4 9 6
3 3 5 3 3
```

После внесения изменений:

```
0 6 4 9 0
5 1 4 9 6
0 3 5 3 0
```

Подход, который мы использовали в рассмотренном примере, базировался на том, что у индексированного объекта одно из полей является двумерным массивом. Другими словами, при индексировании объекта двумерный массив все же используется, хотя и не напрямую. На самом деле нередко удается обойтись и без массива.

11.2. Имитация неограниченного двумерного массива

Слушайте, что это Вы со мной загадками разговариваете? Честное слово, мне ничего не ясно!

из к/ф "Семнадцать мгновений весны"

Не очень часто, но встречаются ситуации, когда значение элементов двумерного массива удастся вычислить на основе индексов элемента. Если сформулировать это несколько иначе, то утверждение выглядит так: иногда существует функциональная зависимость, позволяющая на основе значений индексов элемента вычислить значение элемента. Конечно, в таком случае можно просто использовать соответствующую функцию и вызывать ее каждый раз, когда необходимо узнать значение того или иного элемента. Но никто не запрещает на ситуацию посмотреть по-другому и попытаться создать иллюзию двумерного массива, причем это может быть массив "бесконечного" размера, или двумерный неограниченный массив. Последнее утверждение означает буквально следующее: индексы у элементов такого "ненастоящего" массива могут быть любыми. Именно такую задачу и рассмотрим далее.

Итак, мы опишем класс, объекты которого индексируются с помощью двух индексов. Если для объекта указано два индекса (как для двумерного массива), то результатом подобного выражения будет целое число, отождествляемое со значением элемента массива. Чтобы быть более конкретными, рассмотрим ситуацию, когда для элемента с индексами i и j значение элемента вычисляется как $i \wedge j$, где через $\wedge$ обозначен оператор *побитового исключающего или*.



На заметку

Оператор $\wedge$ побитового исключающего или является бинарным. Результат выражения $A \wedge B$ вычисляется следующим образом:

- сравниваются соответствующие биты в двоичном представлении чисел A и B ;
- результатом сравнения битов является 1, если один из битов равен 1, а другой равен 0;
- результатом сравнения битов является 0, если оба бита имеют одинаковые значения (оба равны 0 или оба равны 1);
- полученный бинарный код является числовым результатом выражения.

Схема, используемая нами, проста: описывается класс, в котором имеется внутренний класс. У внутреннего класса есть целочисленное поле. Цело-

численное поле внутреннего класса - это первый индекс "элемента массива". Для внешнего и внутреннего класса описаны операторные методы `operator[]()`, поэтому объекты каждого из классов можно индексировать.

При индексировании объектов внешнего класса возвращается объект внутреннего класса. Причем значение индекса, указанного в квадратных скобках при индексировании объекта внешнего класса, присваивается целочисленному полю объекта внутреннего класса (объекта, возвращаемого результатом операторного метода). При индексировании объекта внутреннего класса возвращается числовое значение, которое вычисляется на основе целочисленного поля объекта внутреннего класса (значение поля - аналог первого индекса "элемента массива") и аргумента операторного метода (аналог второго индекса "элемента массива"). Программный код примера приведен в листинге 11.3.

Листинг 11.3. Имитация неограниченного двумерного массива

```
#include <iostream>
using namespace std;
// Класс для реализации неограниченного
// двумерного массива:
class Array2D{
private:
    // Внутренний класс:
    class Array1D{
private:
        // Целочисленное поле:
        int index;
public:
        // Конструктор внутреннего класса:
        Array1D(int index){
            this->index=index;
        }
        // Операторный метод для индексирования
        // объекта внутреннего класса:
        int operator[](int k){
            // Значение элемента двумерного массива:
            return index^k;
        }
    }; // Окончание описания внутреннего класса
public:
    // Операторный метод для индексирования
    // объекта внешнего класса:
    Array1D operator[](int k){
        // Объект внутреннего класса:
        Array1D tmp(k);
```

```

        // Результат метода - объект внутреннего класса:
return tmp;
    }
}; // Окончание описания класса
// Главная функция программы:
int main() {
    // Объект для имитации массива:
Array2D obj;
    // Отображение значений "элементов массива":
for(int i=0;i<5;i++){
for(int j=0;j<7;j++){
cout<<obj[i][j]<<" ";
        }
cout<<endl;
    }
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 11.3)

```

0 1 2 3 4 5 6
1 0 3 2 5 4 7
2 3 0 1 6 7 4
3 2 1 0 7 6 5
4 5 6 7 0 1 2

```

Стоит обратить внимание, что в главной функции программы при создании объекта `obj` класса `Array2D` размеры "массива" никак и нигде не указываются, и при вызове вложенных операторов цикла границы изменения индексных переменных выбираются в принципе произвольно.

11.3. Динамические двумерные массивы

*Какое глубокое проникновение в суть вещей!
Впрочем, принц всегда тонко анализировал
самую сложную ситуацию.
из к/ф "Приключения принца Флоризеля"*

Теперь рассмотрим вопрос о том, как создаются *динамические двумерные массивы*. Общий подход состоит в том, что сначала создается одномерный массив из указателей (тип указателей определяется типом элементов двумерного массива). А потом для каждого указателя создается динамический массив уже непосредственно из тех элементов, которые должны сформиро-

вать двумерный динамический массив. То есть фактически каждая строка двумерного динамического массива создается отдельно. Каждая такая строка - массив из элементов определенного типа. Имя массива - указатель на первый элемент в строке. Имя массива присваивается как значение одному из элементов в массиве указателей. В результате получается динамический одномерный массив, состоящий из имен динамических одномерных массивов.

Подробности

Поясним это на более конкретной ситуации. Допустим, мы хотим создать двумерный динамический массив из целых чисел (тип элементов `int`). Допустим, речь идет о массиве из 2 строк и 3 столбцов. Тогда для начала создаем массив из указателей на целые числа (тип элементов массива `int*`). Например, командой `int **nums=new int*[2]` создается динамический массив `nums` из двух элементов. Поскольку элементами массива являются указатели на целые числа (тип `int*`), то имя такого массива - указатель на указатель на целое число. Такой тип описывается выражением `int**`. Именно поэтому перед идентификатором `nums` указаны две звездочки.

Фактически массив `nums` предназначен для "запоминания" имен одномерных массивов, формирующих строки двумерного массива целых чисел. Эти одномерные массивы нужно создать. Сделать это можно, например, так: `nums[0]=new int[3]` и `nums[1]=new int[3]`. В результате выполнения каждой из этих команд создается одномерный динамический массив из трех элементов. Указатель на первый элемент первого массива записывается в переменную `nums[0]`, а указатель на первый элемент второго массива записывается в переменную `nums[1]`. А эти переменные, обе, являются элементами массива `nums`.

В результате получаем "структуру", которая полностью имитирует двумерный массив. В частности, обращение к элементу такого массив выполняется командой вида `nums[i][j]`, где индекс `i` принимает значения от 0 до 1 включительно, а индекс `j` принимает значения от 0 до 2 включительно. Почему такое возможно? На самом деле все просто. Выражение `nums[i]` - элемент одномерного массива `nums` с индексом `i`. Массив `nums` состоит из указателей. Поэтому `nums[i]` - указатель. Указатель индексируется. Командой `nums[i][j]` выполняется индексация указателя. А поскольку `nums[i]` не просто указатель, а указатель на первый элемент одномерного массива, то результатом выражения `nums[i][j]` является соответствующий элемент как бы двумерного массива.

Небольшой пример, в котором создается двумерный целочисленный массив и заполняется случайными числами, представлен в листинге 11.4.



На заметку

В этом примере мы также познакомимся с функцией, которой аргументом передается двумерный массив. Речь идет о функции `show()`, предназначенной для отображения значений элементов двумерного массива. Аргументами функции передается имя двумерного массива и два целочисленных значения, определяющие размеры двумерного массива.

Листинг 11.4. Двумерный динамический массив

```

#include <iostream>
#include <cstdlib>
using namespace std;
// Функция для отображения значений
// элементов двумерного динамического массива.
// Аргументы функции - имя двумерного массива и
// размеры массива:
void show(int **p,int s1,int s2){
    int i,j;
    for(i=0;i<s1;i++){
        for(j=0;j<s2;j++){
            // Обращение к элементу двумерного массива:
            cout<<p[i][j]<<" ";
        }
        cout<<endl;
    }
}
// Главная функция программы:
int main(){
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Размеры массива и индексные переменные:
    int m=3,n=5,i,j;
    // Двумерный массив (указатель на указатель):
    int **nums;
    // Создание одномерного массива указателей:
    nums=new int*[m];
    // Создание строк массива и заполнение массива:
    for(i=0;i<m;i++){
        // Создание очередной строки двумерного массива:
        nums[i]=new int[n];
        for(j=0;j<n;j++){
            // Значение элемента двумерного массива:
            nums[i][j]=rand()%10;
        }
    }
    // Отображение элементов двумерного массива:
    show(nums,m,n);
    // Удаление строк двумерного массива:
    for(i=0;i<m;i++){
        // Удаляется строка двумерного массива:
        delete [] nums[i];
    }
    // Удаление "внешнего" массива (массива указателей):

```

```
delete [] nums;
return 0;
}
```

Результат выполнения программы может быть таким (конкретные числовые значения зависят от особенностей генератора случайных чисел):

Результат выполнения программы (из листинга 11.4)

```
5 0 5 8 6
3 3 6 3 0
2 6 8 4 0
```



На заметку

При завершении работы с динамическими массивами (как читатель, должно быть, помнит) выделенную под массивы память рекомендуется освобождать. В данном случае, поскольку двумерный массив представляет собой массив из одномерных массивов, то сначала удаляется каждая строка двумерного массива, а затем удаляется "внешний" одномерный массив из указателей на строки двумерного массива.

Код программы достаточно простой. При выполнении программы инициализируется генератор случайных чисел и объявляется несколько целочисленных переменных: в переменные `mi` и `m` записываются размеры двумерного массива. Командой `int **nums` объявляется переменная `nums`, которая формально является указателем на указатель на целое число, а на самом деле это будет двумерный массив - имя массива мы будем отождествлять с переменной `nums`. Далее командой `nums=new int*[m]` создается динамический массив из `m` элементов, и каждый такой элемент является указателем на целочисленное значение. Адрес созданного динамического массива из указателей записывается в переменную `nums`.



На заметку

На этом этапе двумерный массив еще не создан. Создан только одномерный массив из указателей, но этим указателям пока не присвоены значения. Значениями элементов-указателей будут имена массивов-строк, которые предстоит создать.

Массив `nums` нужно заполнить. Для этого необходимо создать одномерные массивы, играющие роль строк двумерного массива. Адрес (указатель на первый элемент) массива-строки присваивается в качестве значения соответствующему элементу-указателю из массива `nums`. Для решения поставленной задачи необходимо перебрать все элементы массива `nums` и каждому из них присвоить значение, предварительно создав массив-стро-

ку. Также необходимо заполнить каждую строку случайными числами. Мы совмещаем оба процесса (создание строк и заполнение их случайными числами), для чего запускаются вложенные операторы цикла. Во внешнем цикле индексная переменная `i` пробегает значения от 0 до `m-1`. Во внутреннем цикле (при фиксированном значении переменной `i`) индексная переменная `j` пробегает значения от 0 до `n-1`. В теле внешнего оператора цикла, но до запуска внутреннего оператора цикла, выполняется команда `nums[i]=new int[n]`, которой создается динамический массив из `n` целочисленных элементов (то есть создается строка для двумерного массива), а указатель на первый элемент массива записывается в переменную `nums[i]`. После того, как строка создана, она заполняется случайными числами, для чего запускается внутренний оператор цикла, и в нем командой `nums[i][j]=rand()%10` присваивается значение элементу с индексом `j` в массиве, ссылка на первый элемент которого записана в `i`-й элемент массива `nums` -проще говоря, присваивается значение элементу с `j`-м индексом в строке с индексом `i`.

Для отображения элементов массива вызывается функция `show()`. Речь идет о команде `show(nums, m, n)`.

Подробности

В описании функции `show()` интерес представляет, пожалуй, способ объявления аргументов функции. Первым аргументом функции является имя двумерного массива. Объявлен этот аргумент как указатель на указатель на целое число. Поскольку по имени массива определить размер массива весьма и весьма проблематично, нужны еще два параметра, которые бы определяли количество строк и столбцов в массиве. Отсюда у функции появляется еще два целочисленных аргумента.

В теле функции запускаются вложенные операторы цикла, а в них перебираются индексы элементов в двумерном массиве и значения элементов отображаются (построчно) в окне вывода.

Далее проиллюстрировано, как можно удалить двумерный динамический массив из памяти. Ранее отмечалось, что сначала удаляется каждая строка двумерного массива, а затем удаляется массив, непосредственно в который записывались адреса строк (указатели на первые элементы массивов-строк) двумерного массива. Строки удаляются командой `delete [] nums[i]`, выполняемой в теле оператора цикла. В операторе перебираются элементы массива `nums`. После выполнения оператора цикла "уничтожены" массивы, игравшие роль строк в двумерном массиве. Но еще остался массив указателей `nums`. Его удаляем командой `delete [] nums`.

11.4. Создание "рваного" двумерного массива

Скажите, доктор Ватсон, Вы понимаете всю важность моего открытия?

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

Процедура, использованная нами выше для построения двумерного динамического массива, наводит на простую мысль: создать двумерный массив, у которого в разных строках разное количество элементов (такие массивы иногда называют "рваными"). Это возможно. Причем делается достаточно просто. Чтобы не быть голословными, сразу рассмотрим небольшой пример, представленный в листинге 11.5.

Листинг 11.5. Создание "рваного" двумерного массива

```
#include <iostream>
using namespace std;
// Главная функция программы:
int main() {
    // Целочисленные переменные:
    int n=5,i,j,s=1;
    // Указатель на указатель на целое число:
    int **nums;
    // Одномерный динамический массив из указателей:
    nums=new int*[n];
    cout<<"Первый массив:\n";
    for(i=0;i<n;i++){
        // Одномерный числовой массив. Количество элементов
        // различно для разных циклов:
        nums[i]=new int[i+1];
        for(j=0;j<=i;j++){
            // Значение элемента двумерного массива:
            nums[i][j]=s;
            // Отображается значение элемента массива:
            cout<<nums[i][j]<<" ";
            // Следующее натуральное число
            // (для значения элемента массива):
            s++;
        }
        // Переход к новой строке:
        cout<<endl;
    }
    // Удаление строк двумерного "рваного" массива:
    for(i=0;i<n;i++){
```

```

        // Удаляется строка:
delete [] nums[i];
    }
// Удаление массива указателей:
delete [] nums;
    // Создается новый массив указателей:
nums=new int*[3];
    // Первый статический массив:
int a[]={1,2,3};
    // Второй статический массив:
int b[]={4,5};
    // Третий статический массив:
int c[]={6,7,8,9};
    // Четвертый статический массив:
int d[]={10,11,12,13,14,15};
// Первая строка двумерного массива:
nums[0]=a;
    // Вторая строка двумерного массива:
nums[1]=b;
    // Третья строка двумерного массива:
nums[2]=c;
    // Массив с размерами строк "рваного" массива:
int size[]={3,2,4};
cout<<"Второй массив:\n";
    // Отображение значений элементов
    // "рваного" массива:
for(i=0;i<3;i++){
    for(j=0;j<size[i];j++){
        // Отображение значения элемента:
        cout<<nums[i][j]<<" ";
    }
    // Переход к новой строке:
    cout<<endl;
}
    // Замена строки в двумерном массиве:
nums[1]=d;
// Изменение значения размера второй строки
// "рваного" массива:
size[1]=6;
cout<<"Второй массив (после замены строки):\n";
// Отображение значений элементов "рваного"
// массива после замены второй строки:
for(i=0;i<3;i++){
    for(j=0;j<size[i];j++){
        // Отображение значения элемента:
        cout<<nums[i][j]<<" ";
    }
}

```

```

    }
    // Переход к новой строке:
    cout<<endl;
}
// Удаление массива указателей:
delete [] nums;
return 0;
}

```

В результате выполнения программы получаем буквально следующее:

Результат выполнения программы (из листинга 11.5)

```

Первый массив:
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
Второй массив:
1 2 3
4 5
6 7 8 9
Второй массив (после замены строки):
1 2 3
10 11 12 13 14 15
6 7 8 9

```

Программа состоит как бы из нескольких частей (точнее, двух). Сначала объявляется ряд целочисленных переменных и командой `int **nums` объявляется переменная `nums`, которой суждено стать именем двумерного "рваного" массива. Для реализации концепции "рваного" массива, так же как и в случае обычного двумерного динамического массива, нам нужно создать одномерный массив указателей. Такой массив создается командой `nums=new int*[n]`.

Далее, как и в предыдущем примере, мы запускаем оператор цикла, и в нем создаются строки массива, а каждая вновь созданная строка заполняется значениями. Новаций две: у создаваемых строк теперь разный размер, а значения элементов строки сразу после заполнения отображаются в окне вывода. Второй момент чисто технический. Нас интересуют строки разной длины. Но здесь все просто: в теле внешнего оператора цикла при заданном значении индексной переменной `i` (принимает значения от 0 до `n-1` включительно) выполняется команда `nums[i]=new int[i+1]`, которой создается одномерный массив из `i+1` элементов. Таким образом, первая строка состоит из одного элемента, вторая - из двух, третья - из трех, и так далее.

Подробности

Значениями элементов "рваного" массива присваиваются натуральные числа, начиная с 1. Процесс реализован так: объявляется целочисленная переменная `s` с начальным значением 1, и значение этой переменной во внутреннем операторе цикла присваивается элементу массива, после чего командой `s++` значение переменной `s` увеличивается на 1. В результате элементы "рваного" массива построчно получают значения 1, 2, 3, и так далее.

При отображении значений элементов массива можно проследить, сколько элементов в каждой строке. После того, как значения элементов массива выведены в консоль, начинается построчное удаление "рваного" массива. Выполняется удаление массива так же, как и в предыдущем примере: запускается оператор цикла, в котором удаляются строки, а уже затем удаляется массив из указателей. На этом "первая часть" программы заканчивается, и сразу начинается "вторая часть".

Командой `nums=new int*[3]` создается динамический массив из трех указателей, и указатель на первый элемент массива записывается в переменную `nums`.



На заметку

То есть переменная все та же, а массив уже другой.

Объявляются и инициализируются четыре статических целочисленных массива (`a`, `b`, `c` и `d`). В каждом из этих массивов разное количество элементов (соответственно 3, 2, 4 и 6). Размеры первых трех статических массивов заносятся в массив `size`. В результате значение `size[0]` соответствует размеру массива `a`, значение `size[1]` соответствует размеру массива `b`, а значение `size[2]` соответствует размеру массива `c`. При выполнении команд `nums[0]=a`, `nums[1]=b` и `nums[2]=c` в массив `nums` заносятся указатели на статические массивы. В результате получается, что значениями элементов-указателей динамического массива `nums` являются адреса (указатель на первый элемент) статических массивов, причем массивы имеют разный размер.



На заметку

Фактически мы объединили три статических массива "под эгидой" динамического массива, и в результате получилась иллюзия двумерного "рваного" массива.

Для отображения значений элементов такого импровизированного массива запускаем вложенные операторы цикла. Во внешнем операторе цикла индексная переменная `i` пробегает значения от 0 до 2, перебирая тем самым строки двумерного "рваного" массива, в котором строка с индексом `i` имеет

размер `size[i]`. Поэтому во внутреннем операторе цикла индексная переменная пробегает значения от 0 до `size[i]-1`.

Следующий этап состоит в "замене" второй строки в "рваном" двумерном массиве. Для этого достаточно присвоить новое значение элементу `nums[1]`. После выполнения команды `nums[1]=d` второй элемент в массиве `nums` является указателем на статический массив (первый его элемент) `d`, а не массив `b`, как это было раньше. Кроме того командой `size[1]=6` изменяется значение элемента `size`, определяющее размер второй строки рваного массива. Затем опять запускаются вложенные операторы цикла для отображения значений элементов "рваного" массива после замены в нем второй строки.

Перед завершением программы командой `delete [] nums` удаляется динамический массив указателей `nums`.



На заметку

Строки двумерного массива удалять не нужно, поскольку в данном случае они реализованы не через динамические, а через статические массивы.

Подробности

Подход, описанный выше, применим для создания двумерного статического массива. Речь идет о том, чтобы создать статический массив из указателей, а затем значениям элементам массива присвоить имена других статических массивов. Например, командой `int *nums[2]` объявляется массив `nums` из двух указателей на целые числа. Далее, допустим, имеется два целочисленных массива, созданных командами `int a[]={1,2,3}` и `int b[]={4,5,6,7,8}`. Массив `a` состоит из трех, а массив `b` состоит из пяти элементов (если при объявлении с одновременной инициализацией размер массива явно не указан, то он определяется автоматически по количеству элементов в списке инициализации в фигурных скобках). Если теперь выполнить команды `nums[0]=a` и `nums[1]=b`, то массив `nums` можно рассматривать как двумерный массив из двух строк: в первой строке три элемента, а во второй строке пять элементов. Хотя еще раз стоит подчеркнуть: это скорее иллюзия двумерного массива. Просто значения элементов `nums[0]` и `nums[1]` одномерного массива `nums` являются указателями, а указатели, как известно, индексируются. Поэтому имеют смысл такие выражения, как `nums[0][1]` или `nums[1][3]`, ну и так далее. Опять же, если в какой-то момент мы присвоим новое значение элементу `nums[0]` или `nums[1]`, получим "двумерный" массив с другой структурой. Очень часто, кстати, это бывает удобно.

11.5. Двумерный массив как поле объекта

Сначала меня это забавляло - с одной стороны. Но с другой - я понимал, что пора объясниться.

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

Далее рассмотрим небольшую иллюстрацию к тому, как динамические двумерные массивы могут использоваться в качестве полей в классах. Принципиально нового здесь ничего нет. Вообще, главная особенность использования двумерных динамических массивов собственно такая же, как и особенность использования одномерных динамических массивов: необходимо корректно выполнять процедуру создания копии объекта с полем, которое является динамическим массивом, и а также отслеживать процедуру присваивания таких объектов и их удаления.



На заметку

Напомним, что для определения способа копирования объектов в классе описывается конструктор создания копии, за присваивание объектов "отвечает" операторный метод `operator=()` (метод для оператора присваивания). Помимо этого, в "комплект" для явного описания в классе входит еще и деструктор.

Чтобы не дублировать многие, достаточно очевидные, вещи, мы не будем рассматривать создание копии объектов и присваивание статических объектов. Вместо этого сосредоточим наше внимание на других операциях с объектами, содержащими двумерные динамические массивы в качестве полей.

Далее рассматривается задача, в которой через объекты класса `Matrix` реализуются квадратные матрицы. Непосредственно матрицы представляются двумерными динамическими массивами (у них размеры по каждому из индексов одинаковые). Благодаря тому, что массивы динамические, ранг матрицы (размер массива по каждому из индексов) можно задавать непосредственно при создании объекта. В классе описан конструктор, которому при создании объекта передается целочисленный аргумент. Аргумент конструктора определяет ранг матрицы, реализуемой данным объектом. Все элементы матрицы получают нулевые значения (двумерный массив заполняется нулями).

Подробности

Поскольку при создании объекта класса `Matrix` динамически создается двумерный массив, то в классе, кроме конструктора, также описан и деструктор, в котором выполняется удаление двумерного динамического массива.

Класс `Matrix` для реализации квадратных матриц мы описываем не просто так, а для выполнения такой важной и интересной процедуры, как вычисление *определителя матрицы*. Определитель матрицы вычисляется с помощью метода `det()`. В этом методе, в свою очередь, вызывается метод `getMinor()`, возвращающий в качестве результата указатель на объект класса `Matrix`.

Подробности

Определитель, или детерминант, представляет собой числовое значение, которое вычисляется на основе элементов квадратной матрицы. Чтобы минимизировать математические выкладки, опишем (по возможности максимально просто) процесс вычисления определителя для квадратной матрицы. Это будет тот алгоритм, который использован в программе для вычисления определителя. Итак, если речь идет о квадратной матрице размерами «2 на 2» (то есть две строки и два столбца), то в этом случае определитель вычисляется как разность произведений диагональных и недиагональных элементов. Более конкретно: пусть матрица $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$, то определитель такой матрицы $\det(A) = a_{11}a_{22} - a_{12}a_{21}$. Если ранг матрицы больше двух, то вычислять определитель сложнее. Нам понадобится еще одно определение.

Рассмотрим матрицу ранга n , состоящую из n строк и n столбцов: $A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{pmatrix}$. Через a_{ij} обозначим элементы этой матрицы (индексы $i, j = 1, 2, \dots, n$). Обозначим через A_{ij} матрицу, которая получается из матрицы A вычеркиванием i -й строки и j -го столбца. Для удобства будем называть матрицу A_{ij} минорной, хотя это и не совсем корректный термин (но так удобнее будет объяснить алгоритм выполнения программы). Определитель $\det(A_{ij})$ матрицы A_{ij} называется минором. Очевидно, что ранг матрицы A_{ij} на единицу меньше ранга матрицы A и равен $n - 1$.

Определитель матрицы A вычисляется на основе определителей ее минорных матриц (миноров) по рекуррентной формуле: $\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{ij} \det(A_{ij})$ или $\det(A) = \sum_{i=1}^n (-1)^{i+j} a_{ij} \det(A_{ij})$. При этом в первой формуле индекс i можно брать любой, а во второй формуле может быть произвольным (но фиксированным) индекс j . Первая формула представляет собой разложение по строке, а вторая - разложение по столбцу. Идея очень простая: для вычисления определителя выбирается произвольный столбец или строка (для выполнения разложения), и определитель вычисляется так: берется сумма элементов из выбранной строки или столбца, умноженных на определитель минорной матрицы, получающейся из исходной вычеркиванием строки и столбца, на пересечении которых находится соответствующий элемент. Знаки (плюс или минус) слагаемых в сумме чередуются.

Мы будем использовать разложение по первой строке, поэтому наша рабочая формула выглядит как $\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{1j})$. Здесь только нужно учесть, что в программе для двумерного массива индексация элементов начинается с нуля.

Вообще же ситуация такая: для вычисления определителя матрицы ранга $n > 2$ нужно вычислить определители матриц ранга $n - 1$. Если ранг этих матриц тоже больше двух, то для вычисления определителей этих матриц следует вычислить определители матриц ранга $n - 2$, и так далее. Процесс завершается, когда ранг матриц, для которых вычисляется определитель, равен 2. В этом случае используется явная формула для матрицы ранга два. Получается своеобразный рекурсивный подход, когда для вычисления определителя необходимо вычислить определитель, но другого объекта. Эта идеология находит свое отражение в программном коде, рассматриваемом далее.



На заметку

Вообще с матрицами можно выполнять много самых разных операций. Для этого в классе `Matrix` пришлось бы описать ряд дополнительных методов. Но в данном конкретном случае для нас интерес представляет процедура вычисления определителя матрицы, и именно эта задача решается.

Для удобства в классе `Matrix` описан метод `show()`, которым отображается содержимое двумерного массива, а также определен операторный метод для индексирования объектов. Программный код примера приведен в листинге 11.6.

Листинг 11.6. Двумерный динамический массив как поле класса

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Класс с динамическим двумерным массивом
// для реализации квадратных матриц:
class Matrix{
private:
    // Поле - имя двумерного массива:
    double **matrix;
    // Поле - ранг (размер) матрицы:
    int rank;
    // "Технический" метод. Возвращает 1 или 0:
    int s(int a,int b){
        if(b<a) return 0;
        else return 1;
    }
public:
    // Конструктор с целочисленным аргументом:
    Matrix(int r){
        // Ранг матрицы:
        rank=r;
        // Создание массива указателей:
```



```

matrix=new double*[rank];
    // Создание строк для двумерного массива:
for(int i=0;i<rank;i++){
    // Создание массива-строки:
matrix[i]=new double[rank];
        // Элементам массива присваиваются
        // нулевые значения:
for(int j=0;j<rank;j++){
matrix[i][j]=0;
        }
    }
}
// Деструктор:
~Matrix(){
// Удаление строк массива:
for(int i=0;i<rank;i++){
delete [] matrix[i];
    }
    // Удаление массива указателей:
delete [] matrix;
}
// Операторный метод для индексирования объектов:
double *operator[](int k){
    // Результат метода - указатель на числовой
    // массив (строка двумерного массива):
return matrix[k];
}
// Метод для отображения содержимого
// двумерного массива:
void show(){
for(int i=0;i<rank;i++){
for(int j=0;j<rank;j++){
// Отображение значения элемента:
cout<<matrix[i][j]<<" ";
        }
        // Переход к новой строке:
cout<<endl;
    }
}
// Метод для вычисления минорной матрицы:
Matrix *getMinor(int i,int j){
int k,l; // Индексные переменные
    // Ранг минорной матрицы:
int r=rank-1;
    // Указатель на объект класса Matrix:
Matrix *t;

```

```

// Создание динамического объекта
// класса Matrix (объект для минорной матрицы):
t=new Matrix(r);
    // Заполнение элементов минорной матрицы:
for(k=0;k<r;k++){
for(l=0;l<r;l++){
// Значение элемента минорной матрицы:
        (*t)[k][l]=matrix[k+s(i,k)][l+s(j,l)];
}
    }
    // Результат метода - указатель на объект
    // минорной матрицы (объект класса Matrix):
return t;
}
// Метод для вычисления определителя матрицы:
double det(){
// Определитель матрицы ранга 2
    // (матрица "2 на 2"):
if(rank==2){
return matrix[0][0]*matrix[1][1]-matrix[0][1]*matrix[1][0];
}
    // Если ранг матрицы больше двух:
else{
// "Технический" множитель для формулы
    // вычисления определителя (знак слагаемого):
int sgn=1;
// Значение определителя матрицы:
double d=0;
    // Указатель на объект класса Matrix:
Matrix *t;
    // Вычисление определителя матрицы:
for(int j=0;j<rank;j++){
// Создание объекта для минорной матрицы:
t=getMinor(0,j);
        // Прибавление очередного слагаемого
        // при вычислении определителя:
d+=sgn*matrix[0][j]*(t->det());
// Изменяется знак для следующего слагаемого:
sgn*=-1;
        // Удаляется динамический объект
        // для уже использованной минорной матрицы:
delete t;
    }
    // Результат метода:
return d;
}

```

```

    }
}; // Окончание описания класса
// Главная функция программы:
int main() {
    // Размер статического массива:
    const int n=3;
    // Индексные переменные:
    int i,j;
    // Двумерный статический массив:
    double m[][n]={ {1,2,3}, {4,5,6}, {7,8,9} };
    // Статический объект класса Matrix (матрица):
    Matrix A(n);
    // Заполнение матрицы на основе значений
    // статического массива:
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            // Значение элемента матрицы:
            A[i][j]=m[i][j];
        }
    }
    cout<<"МатрицаA:\n";
    // Отображение содержимого матрицы:
    A.show();
    // Определитель матрицы:
    cout<<"det (A) =";
    cout<<A.det()<<endl;
    // Размер матрицы:
    int k=5;
    // Создание объекта для матрицы:
    Matrix B(k);
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Заполнение матрицы случайными числами:
    for(i=0;i<k;i++){
        for(j=0;j<k;j++){
            // Значение элемента матрицы (0,1 или 2):
            B[i][j]=rand()%3;
        }
    }
    cout<<"МатрицаB:\n";
    // Отображение содержимого матрицы:
    B.show();
    // Определитель матрицы:
    cout<<"det (B) =";
    cout<<B.det()<<endl;
    return 0;
}

```

```
}
```

Результат выполнения программы может быть таким:

Результат выполнения программы (из листинга 11.6)

Матрица A:

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

```
det(A)=0
```

Матрица B:

```
0 2 0 2 2
```

```
1 0 0 2 2
```

```
2 2 1 2 2
```

```
2 2 1 0 1
```

```
0 1 0 0 0
```

```
det(B)=-2
```

Наиболее принципиальным местом в программном коде класса `Matrix` является, пожалуй, два метода: `getMinor()`, которым вычисляется минорная матрица, и метод `det()`, которым собственно вычисляется определитель исходной матрицы. Код других методов в принципе должен быть понятен без особых пояснений.

Подробности

На всякий случай кратко выделим основные моменты. В частности, в закрытое целочисленное поле `rank` записывается ранг матрицы. Значение поля присваивается при вызове конструктора. Также в классе объявлено поле `double **matrix`, которое формально является указателем на указатель на число типа `double`. На самом же деле в переменную `matrix` записывается адрес первого элемента массива указателей (команда `matrix=new double*[rank]` в конструкторе). В операторе цикла в теле конструктора при фиксированном значении индексной переменной `i` командой `matrix[i]=new double[rank]` создается новый числовой массив (строка для динамического массива) и адрес первого элемента созданного массива присваивается значению элементу массива указателей. Элементам числового массива присваиваются нулевые значения: для этого запускается еще один (внутренний) оператор цикла.

В деструкторе сначала запускается оператор цикла (переменная `i` перебирает индексы всех элементов массива указателей), и за каждый цикл командой `delete [] matrix[i]` удаляется числовой динамический массив, являющийся строкой двумерного массива. По завершении оператора цикла командой `delete [] matrix` удаляется массив указателей. Тем самым двумерный динамический массив прекращает свое существование, чего мы и добивались.

Операторный метод для индексирования объектов `operator[]()` для задан-

ного целочисленного аргумента `k` возвращает в качестве результата значение `matrix[k]`. Это элемент массива `matrix`. Тип элемента - указатель на число типа `double`. Поэтому прототип операторного метода начинается с ключевого слова `double` и содержит звездочку `*` (это означает, что метод результатом возвращает указатель на значение типа `double`).

Что касается метода `show()`, то код у него простой: с помощью вложенных операторов цикла перебираются элементы двумерного массива и построчно отображаются в окне вывода.

Метод `getMinor()` для вычисления минорной матрицы "создает" динамический объект класса `Matrix`. Результатом метод возвращает указатель на данный объект. Поэтому прототип метода начинается с названия класса `Matrix`, а перед именем метода указана звездочка `*`. Аргументами методу передаются два целочисленных значения (обозначены переменными `i` и `j`), определяющие индексы строки и столбца, которые следует "вычеркнуть" в исходной матрице для получения минорной матрицы.

Код метода достаточно простой. Объявляются две целочисленные индексные переменные `k` и `l`, а переменной `r` присваивается значение `rank-1` (ранг минорной матрицы на единицу меньше ранга исходной матрицы). Также создается указатель `t` на объект класса `Matrix` (команда `Matrix *t`). Командой `t=new Matrix(r)` создается динамический объект класса `Matrix`, а адрес этого объекта записывается в указатель `t`. У созданного объекта есть двумерный массив с нулевыми элементами. Нам нужно присвоить значения этим элементам такие, чтобы объект соответствовал минорной матрице. Для заполнения двумерного массива запускаются вложенные операторы цикла, в них индексные переменные `k` и `l` пробегает значения от 0 до `r-1` включительно (переменная `r` соответствует рангу минорной матрицы и это та переменная, что передавалась аргументом конструктору при создании динамического объекта). В теле внутреннего оператора цикла выполняется всего одна команда `(*t)[k][l]=matrix[k+s(i,k)][l+s(j,l)]` для присваивания значения элементу (с индексами `k` и `l`) двумерного массива из динамического объекта, на который указывает указатель `t`.



На заметку

В выражении `(*t)[k][l]` используется индексация динамического объекта, на который ссылается указатель `t`.

Значения элементов динамического массива, "спрятанного" в динамическом объекте, определяются на основе значений элементов массива `matrix` того объекта, из которого вызывается метод `getMinor()`. Правила присваивания очень простые:

- если первый индекс k меньше значения первого аргумента i метода `getMinor()`, то в массиве `matrix` первый индекс должен быть k ;
- если первый индекс k больше или равен значению первого аргумента i метода `getMinor()`, то в массиве `matrix` первый индекс должен быть $k+1$;
- если второй индекс l меньше значения второго аргумента j метода `getMinor()`, то в массиве `matrix` второй индекс должен быть l ;
- наконец, если второй индекс l больше или равен значению второго аргумента j метода `getMinor()`, то в массиве `matrix` второй индекс должен быть $l+1$.

Чтобы добиться нужного эффекта мы используем "технический" метод `s()`, в качестве результата возвращающий значение 0 или 1. У метода два аргумента. Если второй аргумент меньше первого, возвращается значение 0, а в противном случае возвращается значение 1. Поэтому индексы для элемента массива `matrix`, соответствующего элементу с индексами k и l динамического объекта, определяются соответственно как $k+s(i, k)$ и $l+s(j, l)$.

После того, как элементы массива динамического объекта определены, командой `return t` указатель на динамический объект возвращается результатом метода.

Метод `det()` для вычисления определителя матрицы состоит из условного оператора, в котором проверяется значение ранга матрицы (значение поля `rank`). Если матрица второго ранга (условие `rank==2` истинно), то в качестве результата методом возвращается значение выражения `matrix[0][0]*matrix[1][1]-matrix[0][1]*matrix[1][0]`. Это явное выражение для определителя матрицы второго ранга (матрицы размеров "2 на 2"). Если же речь идет о матрице более высокого ранга (условие `rank==2` ложно), то выполняется `else`-ветка условного оператора. Здесь команд уже побольше, чем в предыдущем случае. В первую очередь с начальным значением 1 инициализируется целочисленная переменная `sgn`. Эта переменная "отвечает" за знак слагаемых в сумме для вычисления значения определителя.



На заметку

Напомним, что знак слагаемых должен поочередно меняться.

Значение определителя будет записываться в переменную `d` (начальное значение этой переменной равно 0). Также командой `Matrix *t` объявляется указатель на объект класса `Matrix`.

Подробности

Для вычисления определителя матрицы ранга, большего чем два, используется рекуррентное выражение, в которое входят определители матриц рангов, меньших ранга исходной матрицы на единицу. Эти матрицы (минорные матрицы) в теле метода будут реализовываться в виде динамических объектов. Адрес таких объектов необходимо куда-то записывать. Для этой цели и объявляется указатель `t` на объект класса `Matrix`.

Для вычисления определителя матрицы необходимо вычислить сумму. Количество слагаемых в данной сумме совпадает с рангом матрицы. Поэтому запускается оператор цикла, а в нем индексная переменная `j` пробегает значения от 0 до `rank-1` включительно. За каждый цикл к значению переменной `d` прибавляется одно слагаемое `sgn*matrix[0][j]*(t->det())`. Но перед этим командой `t=getMinor(0,j)` создается объект для минорной матрицы и адрес данного объекта записывается в указатель `t`. Чтобы поменялся знак у следующего слагаемого, выполняется команда `sgn*=-1` и, наконец, командой `delete t` удаляется объект для минорной матрицы.

Подробности

За каждый цикл создается объект для новой минорной матрицы. После того, как команды цикла выполнены, потребность в таком объекте отпадает, поэтому он удаляется из памяти. Вкратце схема такая: имеется указатель `t`, для каждого цикла создается новый объект класса `Matrix`, и адрес объекта записывается в указатель `t`. По завершении цикла объект удаляется. Получается, что за разные циклы указатель `t` ссылается на разные объекты.

По завершении оператора цикла переменная `d` возвращается как результат метода `det()`.

На этом описание программного кода класса `Matrix` заканчиваем. Проанализируем код функции `main()`.

В главной функции программы объявлена константа `n`, которая определяет размер по второму индексу двумерного статического массива `m`. Массив инициализирован набором числовых значений.



На заметку

При объявлении с одновременной инициализацией двумерного статического массива `m` размер по первому индексу не указан (пустые квадратные скобки), а по второму задан константой `n`. В таком случае размер по первому индексу определяется автоматически. Напомним, что при объявлении и инициализации одномерно-

го статического массива размер массива разрешается не указывать (определяется по количеству значений в списке инициализации). Для двумерного массива, как видим, ситуация похожая, но не совсем. Дело в том, что по второму индексу размер должен указываться обязательно, причем размер должен задаваться константой. Здесь уместно вспомнить, что двумерный статический массив - это массив из массивов. Поэтому важно знать не только тип элементов "внутренних" массивов (тех массивов, что являются элементами внешнего массива), но и их размер (количество элементов во "внутреннем" массиве).

Командой `Matrix A(n)` создается статический объект класса `Matrix`, отождествляемый с квадратной матрицей. Элементы матрицы заполняются на основе значений элементов массива `m`. Заполнение выполняется так: запускаются вложенные операторы цикла и во внутреннем цикле выполняется команда `A[i][j]=m[i][j]` для присваивания значения элементу массива, "спрятанного" в объекте `A`. Убедиться в том, что заполнение выполнено правильно, можем с помощью команды `A.show()`. Определитель матрицы вычисляется командой `A.det()`.

Еще один пример вычисления определителя показан для случая матрицы ранга 5 (ранг матрицы определяется значением переменной `k`), заполненной случайными целыми числами 0, 1 или 2. В данном случае важно то, что ранг матрицы не обязательно задавать константой - вполне сойдет и переменная. Такоереально благодаря использованию динамического двумерного массива.

Анализируя способ присваивания значений элементам динамических массивов, естественно задаться вопросом: почему бы не использовать для данной цели функцию? Разумеется, такую функцию легко создать. Но есть один момент: способ передачи двумерного массива аргументом функции. И здесь, как говорится, возможны варианты.

11.6. Передача двумерного массива аргументом функции

- Ладно, все. Надо что-то делать. Давай-ка, может быть, сами изобретем.

*- Витя, не надо! Я прошу тебя. Не дразни начальство!
из к/ф "Чародеи"*

В принципе ничего сложного или особенного в передаче двумерного массива аргументом функции или методу нет. Тем не менее, способ передач зависит от того, как именно реализован двумерный массив. Более конкретно, важен тип массива: динамический или статический. Если речь идет о динамическом массиве, то необходимо знать три параметра: название массива

и два числовых значения, определяющие размер по каждому из индексов. Собственно, эти параметры и передаются функции.

Подробности

Понять такой "триумвират" несложно, если вспомнить, как реализуется двумерный динамический массив. На самом деле это массив указателей, и каждый указатель ссылается на одномерный массив. Чтобы "идентифицировать" первый массив (массив указателей), нужно знать "точку входа" (название массива), и количество элементов в массиве. Это два параметра. Еще один параметр - размер массивов (разумеется, при условии, что все эти одномерные массивы имеют одинаковый размер), на которые ссылаются указатели в массиве указателей.

Если речь идет о статическом массиве, то формально нужно два параметра: название массива и количество элементов в массиве. Но только теперь под элементом массива подразумеваются одномерные массивы (строки двумерного массива). Для этих одномерных массивов должен быть указан размер. Размер задается константой. То есть на самом деле все равно получается три параметра, причем один параметр - константа.



На заметку

Статический двумерный массив может быть организован по принципу организации динамического массива. В этом случае сначала объявляется статический массив указателей и еще несколько статических массивов, которые должны послужить строками двумерного массива. Затем каждому указателю из массива указателей в качестве значения присваивается имя того или иного статического одномерного массива. В результате исходный статический массив указателей можно обрабатывать как двумерный массив. Понятно, что способ передачи такого массива аргументом функции, с одной стороны, может напоминать способ передачи аргументом функции динамического массива. Но у статического массива, организованного описанным выше способом, есть особенность: его размеры (по каждому индексу) определяются константами. Более конкретно, если создается статический массив из указателей, то его размер должен быть задан константой. Размер статических одномерных массивов, служащих строкам двумерного статического массива, также должны определяться константой. Если соответствующие константы объявлены как глобальные, то аргументами функции их можно не передавать.

Кратко остановимся на вопросе передачи двумерного массива аргументом функции. Начнем сразу с примера, представленного в листинге 11.7. В примере описано несколько функций, аргументами которым передаются массивы. В частности, есть функция для отображения содержимого массива, функция для заполнения двумерного массива случайными числами, функция для создания динамического массива и функция для удаления динамического массива. У функций, предназначенных для отображения содержимого массива и заполнения массива случайными числами по две версии: для работы со статическими и динамическими массивами.

Листинг 11.7. Передача двумерного массива аргументом функции

```

#include <iostream>
#include <cstdlib>
using namespace std;
// Размер статического массива
// по второму индексу:
const int size=4;
// Функция для отображения значений элементов
// динамического массива:
void show(int **p,int m,int n){
cout<<"Динамический массив:\n";
for(int i=0;i<m;i++){
for(int j=0;j<n;j++){
cout<<p[i][j]<<" ";
}
cout<<endl;
}
}
// Функция для отображения значений элементов
// статического массива:
void show(int p[][size],int m){
cout<<"Статический массив:\n";
for(int i=0;i<m;i++){
for(int j=0;j<size;j++){
cout<<p[i][j]<<" ";
}
cout<<endl;
}
}
// Функция для присваивания случайных значений
// элементам динамического массива:
void set(int **p,int m,int n){
for(int i=0;i<m;i++){
for(int j=0;j<n;j++){
p[i][j]=rand()%10;
}
}
}
// Функция для присваивания случайных значений
// элементам статического массива:
void set(int p[][size],int m){
for(int i=0;i<m;i++){
for(int j=0;j<size;j++){
p[i][j]=rand()%10;
}
}
}

```

```

    }
}
// Функция для создания динамического массива:
int **create(int m,int n){
int **p;
    p=new int*[m];
    for(int i=0;i<m;i++){
p[i]=new int[n];
    }
    return p;
}
// Функция для удаления динамического массива:
void del(int **p,int m,int n){
    for(int i=0;i<m;i++){
delete [] p[i];
    }
delete [] p;
}
// Главная функция программы:
int main(){
    // Инициализация генератора случайных чисел:
srand(2015);
    // Имя динамического массива:
int **A;
    // Создается динамический массив:
A=create(3,5);
    // Элементам динамического массива
    // присваиваются значения (случайные):
set(A,3,5);
cout<<"Массив A. ";
// Отображение значений элементов
// динамического массива:
show(A,3,5);
    // Статический массив из указателей:
int *B[2];
    // Статический числовой массив:
int  nums[]={1,2,3,4,5,6,7,8,9};
    // Адрес первого элемента статического числового
    // массива присваивается значением первому элементу
    // статического массива указателей:
B[0]=nums;
    // Значение первого элемента динамического
    // массива указателей присваивается второму элементу
    // статического массива указателей:
B[1]=A[0];
cout<<"Массив B. ";

```

```

// Отображение 2-х строк и 5-ти столбцов
// импровизированного "рваного" массива:
show(B,2,5);
// Присваивание значений "рваному" массиву
// (2 строки и 5 столбцов):
set(B,2,5);
cout<<"Массив В (случайные числа).\n";
// Отображение значений элементов "рваного"
// массива (2 строки и 5 столбцов):
show(B,2,5);
cout<<"Массив А (изменилась 1-я строка).\n";
// Отображение значений элементов
// динамического массива:
show(A,3,5);
// Статический числовой массив:
int C[2][size];
// Заполнение статического числового массива:
set(C,2);
cout<<"Массив С. ";
// Отображение значений элементов статического
// числового массива:
show(C,2);
// Удаление динамического массива:
del(A,3,5);
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 11.7)

```

Массив А. Динамический массив:
8 0 1 2 0
0 6 6 4 0
4 0 9 2 4
Массив В. Динамический массив:
1 2 3 4 5
8 0 1 2 0
Массив В (случайные числа).
Динамический массив:
6 9 7 0 4
3 5 8 3 6
Массив А (изменилась 1-я строка).
Динамический массив:
3 5 8 3 6
0 6 6 4 0

```

4 0 9 2 4

Массив С. Статический массив:

6 7 0 3

3 7 7 4

В программе объявляется целочисленная константа `size`, определяющая количество элементов в строках двумерных статических массивов, которые будут использованы в программе.

Функция `show()` предназначена для отображения содержимого массива. Если массив динамический, то функции передается имя массива и два числа - размеры массива. Если массив статический, то первый аргумент функции описывается как `int p[][size]`. Здесь размер массива по второму индексу указан явно, причем через константу, определенную до описания функции. Вторым аргументом функции `show()` определяется количество строк в двумерном статическом массиве. Совершенно аналогично описываются аргументы в функции `set()`, предназначенной для заполнения массивов случайными числами.

При вызове функции `create()` создается динамический массив. Аргументами функции передаются размеры массива. При выполнении функции массив создается, а имя этого массива возвращается как результат функции. Поскольку речь идет о целочисленном двумерном динамическом массиве, то имя такого массива - указатель на указатель на целое число. Отсюда в прототипе функции ключевое слово `int` и две звездочки `**`.

Для удаления динамического массива может быть использована функция `del()`. Аргументы функции - имя удаляемого массива и его размеры. Функция не возвращает результат.

В главной функции программы проиллюстрированы методы работы с описанными функциями. Так, динамический целочисленный массив создается командой `A=create(3,5)`. Заполнение случайными числами массива выполняется командой `set(A,3,5)`. Содержимое массива отображается командой `show(A,3,5)`. Для удаления массива в конце программы использована команда `del(A,3,5)`.

Командой `int *B[2]` в программе объявляется статический массив указателей из двух элементов. Также командой `int nums[]={1,2,3,4,5,6,7,8,9}` инициализирован статический числовой массив. После выполнения команд `B[0]=nums` массив `nums` становится первой строкой в массиве `B` (значением элемента `B[0]` является адрес первого элемента в массиве `nums`). Выполнение команды `B[1]=A[0]` приводит к тому, что первая строка массива `A` становится второй строкой массива `B` (при этом она все равно продолжает оставаться первой строкой

массива `A`). Массив `B` получился "рваным": его первая и вторая строки содержат разное количество элементов.

Подробности

Важный момент связан вот с чем. На самом деле двумерного массива `B` как бы нет совсем: это одномерный массив из указателей. Но значения указателям присвоены такие, что создается иллюзия двумерного массива. При этом изменяя значения в "двумерном массиве" `B` на самом деле мы будем изменять значения в массиве `nums` и первой строке двумерного динамического массива `A`.

Командой `show(B, 2, 5)` отображается по пять элементов в каждой из двух строк двумерного массива `B`. Но на самом деле будут отображены пять элементов массива `nums` и пять элементов массива `A[0]`. Командой `set(B, 2, 5)` указанным элементам присваиваются случайные значения. Но опять же, реально будут присваиваться новые значения элементам массивов `nums` и `A[0]`. Проверить справедливость данного утверждения несложно: достаточно, например, воспользоваться командами `show(B, 2, 5)` и `show(A, 3, 5)`.



На заметку

Желающие могут самостоятельно проследить, как изменился массив `nums`.

Командой `int C[2][size]` создается статический числовой массив из двух строк. Количество элементов в каждой строке определяется константой `size`. Для заполнения такого массива случайными числами использована команда `set(C, 2)`, а отображается содержимое массива командой `show(C, 2)`. Стоит обратить внимание, что первым аргументом функциям передается только имя массива. Второй аргумент - количество строк в массиве. Количество столбцов указывалось только при описании функции. Получается, что этот параметр как бы "спрятан" в типе статического массива.

Глава 12.

КОНТЕЙНЕРЫ И ИТЕРАТОРЫ



- Этнографическая экспедиция.
 - Понятно. Нефть ищете?
 из к/ф "Кавказская пленница"

В этой главе мы познакомимся с такими новыми для нас понятиями, как *контейнеры* и *итераторы*. В принципе, ничего особо сложного здесь нет. Скорее, мы обсудим уже знакомые вещи, но, так сказать, на несколько ином уровне абстракции. Мы будем расширять горизонты наших знаний постепенно. Начнем с контейнеров.

12.1. Знакомство с контейнерами

- Где я?
 - Там же, где и я.
 - А вы где?
 - В аэропорту.
 из к/ф "Ирония судьбы или с легким паром"

Если кратко, то *контейнер* - это объект, способный "хранить" в себе другие объекты. Мы с подобием контейнера уже сталкивались, когда имели дело с объектами, внутри которых были "спрятаны" динамические массивы. Но у таких объектов есть два недостатка. Во-первых, размер динамического массива, хотя и может определяться на стадии создания объекта, при работе с объектом обычно остается фиксированным (ну, или, во всяком случае, мы особо не пытались его изменить). Для эффективного использования контейнеров важно, чтобы имела возможность добавлять в контейнер новые объекты, а также "извлекать" (получать значение) и удалять объекты, уже находящиеся в контейнере.



На заметку

Конкретные операции, допустимые с контейнером, зависят от типа последнего.

Во-вторых, в массиве сберегаются значения только определенного типа. Для контейнера более характерна ситуация, когда класс, на основе которого создается контейнер, является обобщенным. На языке рассмотренных ранее примеров это означает, что класс с динамическим массивом должен

быть обобщенным. Создавая на основе обобщенного класса объект, получим возможность специфицировать тип элементов массива, "спрятанного" в объекте.



На заметку

В стандартной библиотеке шаблонов STL имеется множество контейнеров (то есть шаблонных классов, позволяющих создавать на их основе объекты-контейнеры). Среди наиболее востребованных контейнерных классов можно выделить `vector` (вектор - аналог динамического массива переменного размера с возможностью произвольного доступа к элементам), `list` (список - аналог динамического массива переменного размера с последовательным доступом к элементам), или, например, `map` (ассоциативный контейнер, который иногда называют словарем - набор элементов с доступом к ним по ключу).

Вообще, для практического использования лучше прибегнуть к стандартным контейнерам, описанным в библиотеке STL. Однако поскольку нас интересует не только конечный результат, но и "идеологическая подоплека", то мы рассмотрим несколько примеров, призванных по принципу "нарастания" пояснить основные подходы относительно использования контейнеров.

Начнем с очень простой программы. Эта программа - модификация примеров, в которых рассматривались классы с динамическими массивами. Мы постепенно будем усложнять пример, а отправной точкой послужит класс с целочисленным динамическим массивом. Мы традиционно предусмотрим возможность индексировать объекты. Еще в программе реализуется возможность добавить элемент в массив (причем не только в начало или конец динамического массива, но и в указанное место массива), удалять элемент массива с указанным индексом, выполнять полную очистку массива.

Подробности

Поскольку программный код и так получается немаленький, мы существенно сэкономим, не описывая разные варианты конструкторов (в первую очередь конструктор создания копии), не будем описывать операторный метод для оператора присваивания и, соответственно, не будем "тестировать" соответствующие операции. Вместе с тем, при создании "настоящего" контейнера набор утилит должен быть полным. Те, кого интересует вопрос с описанием конструктора создания копии и оператора присваивания, могут освежить в памяти примеры, рассмотренные ранее в книге.

Рассмотрим программу, представленную в листинге 12.1. Основу данного примера составляет код класса `Vector`. У класса два поля: целочисленное поле `length` определяет размер динамического массива, а поле-указатель на целочисленное значение `v` обозначает название динамического массива. Помимо стандартных методов (конструктора, деструктора, операторного

метода для индексирования объектов, и метода для отображения содержимого массива), в классе описаны следующие методы:

- Метод `insert()` позволяет вставить новый элемент в массив. Аргументами методу передаются индекс позиции, в которую вставляется элемент, и значение элемента.
- Метод `push_back()` позволяет вставить новый элемент в конец массив. Аргументом методу передается значение данного элемента.
- Метод `push_front()` позволяет вставить новый элемент в начало массив. Аргументом методу передается значение этого элемента.
- Метод `erase()` позволяет удалить элемент с индексом, переданным аргументом методу.
- Метод `clear()` предназначен для удаления всех элементов из массива.
- Метод `pop_back()` удаляет последний элемент в массиве.
- Метод `pop_front()` удаляет первый элемент в массиве.
- Методом `size()` в качестве результата возвращается количество элементов в массиве.



На заметку

В библиотеке STL есть класс `vector`. Контейнерный класс `Vector`, который мы описываем здесь, представляет собой очень скромную попытку продублировать (в некоторых моментах) библиотечный класс `vector`.

Методы, подразумевающие вставку или удаление элементов, реализуются по одной общей схеме: в соответствии со смыслом операции, которую следует запрограммировать, создается новый динамический массив, заполняется "правильным" образом, после чего этот новый массив становится полем объекта. Последнее действие выполняется очень просто: указателю `vb` в качестве значения присваивается адрес нового созданного массива. Теперь проанализируем программный код:

Листинг 12.1. Контейнер для целых чисел

```
#include <iostream>
using namespace std;
// Аналог контейнерного класса:
class Vector{
private:
    // Размер массива:
```

```

int length;
    // Имя массива:
int *v;
public:
    // Конструктор:
Vector(int s,int val=0){
length=s; // Размер массива
    // Создание массива:
    v=new int[length];
    // Заполнение массива:
for(int i=0;i<length;i++){
v[i]=val;
    }
}
// Деструктор:
~Vector(){
// Удаление массива:
delete [] v;
}
// Метод возвращает размер массива:
int size(){
return length;
}
// Операторный метод для индексирования объектов:
int &operator[](int k){
return v[k];
}
// Метод для вставки элемента в массив:
void insert(int m,int val){
// "Уточнение" места вставки:
int k=m;
if(m<0) k=0;
if(m>length) k=length;
int *t,i;
    // Новый массив:
t=new int[length+1];
    // Заполнение массива:
for(i=0;i<k;i++){
t[i]=v[i];
    }
t[k]=val;
for(i=k+1;i<=length;i++){
t[i]=v[i-1];
    }
    // Удаление "старого" массива:
delete [] v;

```

```

        // Новое значение массива:
        v=t;
        // Новый размер массива:
length++;
    }
    // Метод для очистки массива:
void clear(){
    // Удаление массива:
delete [] v;
    // Нулевой указатель:
    v=0;
    // Нулевая длина массива:
length=0;
}
    // Метод для удаления элемента:
void erase(int m){
    // "Уточнение" индекса удаляемого элемента:
int k=m,i;
if(m<0) k=0;
if(m>=length) k=length-1;
int *t;
    // Новый массив:
t=new int[length-1];
    // Заполнение нового массива:
for(i=0;i<k;i++){
t[i]=v[i];
}
for(i=k;i<length-1;i++){
t[i]=v[i+1];
}
    // Удаление старого массива:
delete [] v;
    // Новое значение массива:
    v=t;
    // Новая длина массива:
length--;
}
    // Метод для добавления элемента в конец массива:
void push_back(intval){
insert(length, val);
}
    // Метод для добавления элемента в начало массива:
void push_front(intval){
insert(0, val);
}
    // Метод для удаления последнего элемента:

```

```

void pop_back() {
erase(length);
}
// Метод для удаления первого элемента:
void pop_front() {
erase(0);
}
// Метод для отображения содержимого массива:
void show() {
if(length==0) {
cout<<"Нет значений.\n";
return;
}
for(int i=0;i<length;i++) {
cout<<v[i]<<" ";
}
cout<<endl;
}
}; // Окончание описания класса
// Главная функция программы:
int main() {
// Объект-контейнер:
Vector a(5,1);
// Размер массива:
cout<<"Количество элементов: "<<a.size()<<endl;
a.show();
// Вставка элемента в массив:
a.insert(3,0);
// Размер массива:
cout<<"Количество элементов: "<<a.size()<<endl;
a.show();
// Очистка массива:
a.clear();
a.show();
// Вставка элемента в пустой массив:
a.insert(0,0);
// Добавление элементов в конец и начало массива:
for(int i=1;i<=5;i++) {
a.push_back(i);
a.push_front(-i);
}
a.show();
// Удаление элемента из массива:
a.erase(5);
a.show();
// Удаление последнего и первого элемента:

```

```

a.pop_back();
a.pop_front();
a.show();
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 12.1)

```

Количество элементов: 5
1  1  1  1  1
Количество элементов: 6
1  1  1  0  1  1
Нет значений.
-5  -4  -3  -2  -1  0  1  2  3  4  5
-5  -4  -3  -2  -1  1  2  3  4  5
-4  -3  -2  -1  1  2  3  4

```

У конструктора два аргумента: первый аргумент определяет количество элементов в массиве, а второй аргумент (с нулевым значением по умолчанию) - значение, присваиваемое элементам массива при создании объекта. В теле конструктора создается динамический массив, элементы которого получают значения.

В деструкторе происходит удаление динамического массива.

Простой код у метода `size()`: результатом метод возвращает значение поля `length` (размер массива).

Операторным методом `operator[]()` для аргумента `k` возвращается ссылка на элемент `v[k]`. Благодаря этому можно индексировать объекты класса `Vector` как при считывании значений, так и при присваивании значений элементам массива.

У метода `insert()` два аргумента: индекс позиции для вставки нового элемента, и значение вставляемого элемента. В теле метода выполняется проверка корректности значения индекса. Для этого объявляется переменная `k`, которой присваивается значение первого аргумента метода. Но затем последовательно проверяется два условия: если первый аргумент отрицательный, переменной `k` присваивается нулевое значение, а если первый аргумент метода больше длины массива, то значение переменной `k` устанавливается равным `length` (длина массива). Таким образом, если индекс указан некорректно и он отрицательный, то вставка нового элемента вы-

полняется в начало массива. Если индекс превышает длину массива, то вставка нового элемента выполняется в конец массива.



На заметку

Подобная процедура "уточнения" индекса выполняется и в методе `erase()`.

В теле метода командой `t=new int[length+1]` создается новый динамический массив, и его длина на единицу больше длины исходного массива, в который выполняется вставка. Далее выполняется заполнение значениями элементов этого нового массива. Поскольку вставка выполняется в позицию с индексом `k`, то в новом и исходном массивах элементы с индексами от 0 до `k-1` совпадают. Поэтому запускается оператор цикла, в нем индексная переменная `i` пробегает значения от 0 до `k-1` включительно, и в теле оператора цикла выполняется команда `t[i]=v[i]`. Значение элементу с индексом `k` присваивается командой `t[k]=val` (через `val` обозначен второй аргумент метода - значение, добавляемое в массив).

Для элементов в новом массиве с индексами, начиная с `k+1`, значения "сдвинуты" по сравнению исходным массивом на одну позицию вправо. Поэтому в следующем цикле индексная переменная `i` пробегает значения от `k+1` до `length`, и в теле оператора выполняется команда `t[i]=v[i-1]`, что соответствует сдвигу элементов на одну позицию.

После того, как новый массив заполнен, командой `delete [] v` удаляется старый массив, а затем командой `v=t` новый массив становится полем объекта. Наконец, командой `length++` в соответствии с увеличением размера массива, на единицу увеличивается и поле `length`.

Метод для очистки массива `clear()` не имеет аргументов и не возвращает результат. В теле метода командой `delete [] v` удаляется массив, полю `v` присваивается нулевой указатель (команда `v=0`), а полю `length` присваивается нулевое значение.

Метод `erase()` предназначен для удаления элемента из массива. Аргумент метода - индекс элемента, который следует удалить. В теле метода значение индекса проверяется на корректность: если указан отрицательный индекс, то удаляется первый элемент (с нулевым индексом), а если индекс слишком большой, то удаляется последний элемент в массиве.

Новый массив (на замену исходному) создается командой `t=new int[length-1]`. В новом массиве на один элемент меньше, чем в исходном (поскольку из исходного один элемент удаляется). Индекс удаляемого элемента определяется переменной `k`. Если индекс `i` элементов в новом массива лежит в диапазоне от 0 до `k-1`, то значение элементов в но-

вом и старом массивах совпадают. Поэтому в соответствующем операторе цикла выполняется команда `t[i]=v[i]`. Если индекс элементов в новом массиве попадает в диапазон от `k` до `length-2`, то индексы смещены на единицу: в операторе цикла выполняется команда `t[i]=v[i+1]`. Старый массив удаляется командой `delete [] v`, после чего командой `v=t` указатель `v` "перебрасывается" на новый массив. Значение поля `length` командой `length--` уменьшается на единицу.

Методы `insert()` и `erase()` используются в методах `push_back()`, `push_front()`, `pop_back()` и `pop_front()` для вставки и удаления элементов в начало и конец массива. Аргументами методам `push_back()` и `push_front()` передаются добавляемые в массив значения, а у методов `pop_back()` и `pop_front()` аргументов нет.

Метод `show()` описан так, что для непустого массива отображается набор значений его элементов, а в случае пустого массива отображается сообщение об отсутствии элементов.

При выполнении программы командой `Vector a(5,1)` создается объект `a`, массив в котором содержит 5 элементов, и значение каждого элемента равно 1. Размер массива в объекте определяем командой `a.size()`. Командой `a.insert(3,0)` в массив четвертым слева (соответствует индексу 3) вставляется элемент с нулевым значением. Командой `a.clear()` выполняется очистка массива, после чего он не содержит элементов. После выполнения команды `a.insert(0,0)` массив будет состоять из одного элемента с нулевым значением (вставляется элемент 0 в позицию с индексом 0). В операторе цикла, в котором индексная переменная `i` пробегает значения от 1 до 5 командой `a.push_back(i)` число `i` дописывается в конец массива, а число `-i` командой `a.push_front(-i)` добавляется в начало массива. В результате после выполнения оператора цикла массив будет состоять из 11 элементов с целочисленными значениями в диапазоне от -5 до 5.

Командой `a.erase(5)` удаляется элемент с индексом 5 (число 0 в середине массива). Командой `a.pop_back()` удаляется последний элемент массива (число 5), а командой `a.pop_front()` удаляется первый элемент массива (число -5).

На следующем этапе нам предстоит "превратить" класс `Vector` в обобщенный класс. Работа эта в основном "техническая" и сводится к тому, чтобы в соответствующих местах программного кода заменить тип `int` на параметр, обозначающий обобщенный тип. Новая версия рассмотренного выше примера представлена в листинге 12.2.

Листинг 12.2. Контейнер на основе обобщенного класса

```

#include <iostream>
using namespace std;
// Обобщенный контейнерный класс:
template <class X> class Vector{
private:
    // Размер массива:
    int length;
    // Имя массива:
    X *v;
public:
    // Конструктор:
    Vector(int s,X val){
        length=s; // Размер массива
        // Создание массива:
        v=new X[length];
        // Заполнение массива:
        for(int i=0;i<length;i++){
            v[i]=val;
        }
    }
    // Деструктор:
    ~Vector(){
        // Удаление массива:
        delete [] v;
    }
    // Метод возвращает размер массива:
    int size(){
        return length;
    }
    // Операторный метод для индексирования объектов:
    X &operator[](int k){
        return v[k];
    }
    // Метод для вставки элемента в массив:
    void insert(intm,Xval){
        // "Уточнение" места вставки:
        int k=m;
        if(m<0) k=0;
        if(m>length) k=length;
        X *t;
        int i;
        // Новый массив:
        t=new X[length+1];
        // Заполнение массива:

```

```

for(i=0;i<k;i++){
t[i]=v[i];
}
t[k]=val;
for(i=k+1;i<=length;i++){
t[i]=v[i-1];
}
// Удаление "старого" массива:
delete [] v;
// Новое значение массива:
v=t;
// Новый размер массива:
length++;
}
// Метод для очистки массива:
void clear(){
// Удаление массива:
delete [] v;
// Нулевой указатель:
v=0;
// Нулевая длина массива:
length=0;
}
// Метод для удаления элемента:
void erase(int m){
// "Уточнение" индекса удаляемого элемента:
int k=m,i;
if(m<0) k=0;
if(m>=length) k=length-1;
X *t;
// Новый массив:
t=new X[length-1];
// Заполнение нового массива:
for(i=0;i<k;i++){
t[i]=v[i];
}
for(i=k;i<length-1;i++){
t[i]=v[i+1];
}
// Удаление старого массива:
delete [] v;
// Новое значение массива:
v=t;
// Новая длина массива:
length--;
}

```

```

    // Метод для добавления элемента в конец массива:
void push_back(X val){
insert(length,val);
}
    // Метод для добавления элемента в начало массива:
void push_front(X val){
insert(0,val);
}
    // Метод для удаления последнего элемента:
void pop_back(){
erase(length);
}
    // Метод для удаления первого элемента:
void pop_front(){
erase(0);
}
    // Метод для отображения содержимого массива:
void show(){
if(length==0){
cout<<"Нет значений.\n";
return;
}
for(int i=0;i<length;i++){
cout<<v[i]<<" ";
}
cout<<endl;
}
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Объект-контейнер:
Vector<int> a(5,1);
    // Размер массива:
cout<<"Количество элементов: "<<a.size()<<endl;
a.show();
    // Вставка элемента в массив:
a.insert(3,0);
    // Размер массива:
cout<<"Количество элементов: "<<a.size()<<endl;
a.show();
    // Очистка массива:
a.clear();
a.show();
    // Вставка элемента в пустой массив:
a.insert(0,0);
    // Добавление элементов в конец и начало массива:

```

```

for(int i=1;i<=5;i++){
a.push_back(i);
    a.push_front(-i);
}
a.show();
// Удаление элемента из массива:
a.erase(5);
a.show();
    // Удаление последнего и первого элемента:
a.pop_back();
a.pop_front();
a.show();
    // Объект-контейнер:
Vector<char> b(5, 'a');
// Размер массива:
cout<<"Количество элементов: "<<b.size()<<endl;
b.show();
    // Вставка элемента в массив:
b.insert(3, 'z');
    // Размер массива:
cout<<"Количество элементов: "<<b.size()<<endl;
b.show();
    // Очистка массива:
b.clear();
b.show();
    // Вставка элемента в пустой массив:
b.insert(0, 'o');
    // Добавление элементов в конец и начало массива:
for(int i=1;i<=5;i++){
b.push_back('o'+i);
b.push_front('o'-i);
}
b.show();
    // Удаление элемента из массива:
b.erase(5);
b.show();
    // Удаление последнего и первого элемента:
b.pop_back();
b.pop_front();
b.show();
return 0;
}

```

Результат выполнения программы следующий:

Результат выполнения программы (из листинга 12.2)

```

Количество элементов: 5
1  1  1  1  1
Количество элементов: 6
1  1  1  0  1  1
Нет значений.
-5  -4  -3  -2  -1  0  1  2  3  4  5
-5  -4  -3  -2  -1  1  2  3  4  5
-4  -3  -2  -1  1  2  3  4
Количество элементов: 5
a  aaaa
Количество элементов: 6
a  aa  z  a  a
Нет значений.
j  k  l  m  n  o  p  q  r  s  t
j  k  l  m  n  p  q  r  s  t
k  l  m  n  p  q  r  s

```

Хочется верить, что особых комментариев эта программа и результаты ее выполнения не требуют. Обратим лишь внимание, что теперь на основе класса `Vector` мы создали два объекта: командой `Vector<int> a(5,1)` создается объект `a` с целочисленным массивом из пяти элементов, у каждого элемента единичное значение, а командой `Vector<char> b(5,'a')` создается объект `b` тоже из пяти элементов, но на этот раз массив символьный и все элементы имеют значение `'a'`. Таким образом, в объекты класса `Vector` мы можем "прятать" самые различные значения (одного типа), включая и объекты других классов.

Далее познакомимся с *итераторами*.

12.2. Знакомство с итераторами

- А я пиратов ищу.
 - Зачем? Тебе что, больше всех надо?
 из к/ф "Гостья из будущего"

Главное, что нужно знать об *итераторе* -то, что это объект. Правда, объект не любой, а немножко специфический. Итератор связан с контейнером и играет примерно такую же роль, как указатели при работе с массивами. Другими словами, итератор - объект, позволяющий получать доступ к элементам, "спрятанным" в контейнере.

Понятно, что может быть очень много разных вариантов, когда некий объект позволяет получить доступ к содержимому контейнера. Но далеко не каждый подобный объект можно назвать итератором. Итератор должен

поддерживать определенный набор операций, большинство из которых по своей сути перекликаются с правилами адресной арифметики. В этом смысле итератор мимикрирует под указатель, но не является таковым по сути.

Итераторы бывают разных типов, но если говорить в общем, то итератор позволяет перемещаться по контейнерному объекту или "перебирать" элементы в контейнерном объекте. Как именно - это уже вопрос другой. Поэтому в зависимости от типа итератора его характеристики могут разниться, но есть некоторые общие "черты".

Итераторы поддерживают операцию *взятия значения*: когда перед итератором ставится звездочка *, результатом должно быть значение элемента в контейнере - значение элемента, на который указывает в данный момент итератор. Обычно итератор поддерживает такие арифметические операции, как *прибавление* к итератору целого числа и *вычитание* из итератора целого числа. Результатом означенных операций является итератор, но он указывает уже на другой элемент в контейнере. Фактически, прибавление к итератору и вычитание из итератора целых чисел - такой способ перемещаться по контейнеру. Также обычно при работе с итераторами используются методы, позволяющие *установить* итератор в *начало* контейнера и *конец* контейнера.

Итератор является объектом некоторого класса (для удобства назовем его *классом итератора*). Контейнер является объектом обобщенного контейнерного класса. Формально итератор и контейнер - разные объекты. Но между ними существует связь. Причем это не только связь "функциональная", обусловленная необходимостью получать доступ через итератор к содержимому контейнера. Здесь важно и то, что контейнерный объект создается на основе обобщенного класса. Сказанное означает, что итератор должен создаваться с учетом шаблона, применявшегося при создании контейнерного объекта. Проще говоря, при создании (и описании) итератора необходимо учитывать обобщенный характер контейнерного класса. Легче всего добиться результата, описав класс итератора *внутренним* классом контейнерного класса. Именно такой подход использован в программе, представленной в листинге 12.3. Программа представляет собой модификацию предыдущего примера. Если ранее основные операции с содержимым контейнера выполнялись через индексирование объекта контейнера, то в рассматриваемом далее примере в основу положено использование итераторов.



На заметку

Поскольку многие фрагменты кода ранее уже комментировались и должны быть понятны читателю, в программе удалены некоторые комментарии. Также обсуж-

дать мы будем в основном те новые фрагменты, что появились в рассматриваемом примере.

Рассмотрим представленный далее программный код:

Листинг 12.3. Использование итератора

```
#include <iostream>
using namespace std;
// Обобщенный контейнерный класс:
template <class X> class Vector{
private:
    int length;
    X *v;
public:
    // Класс итератора:
    class Iterator{
public:
        // Указатель на объект контейнера:
        Vector<X> *t;
        // Индекс элемента в контейнере:
        int index;
        // Конструктор итератора:
        Iterator(){
            t=0;
            index=0;
        }
        // Операторный метод взятия значения
        // (оператор звездочка *):
        X &operator*(){
            // Результат - ссылка на индексированный
            // контейнерный объект (элемент массива):
            return (*t)[index];
        }
        // Операторный метод "равно"
        // (для сравнения итераторов на предмет равенства):
        bool operator==(Iterator obj){
            // Если совпадают значения полей итераторов:
            if((index==obj.index)&&(t==obj.t)) return true;
            // Если значения полей не совпадают:
            else return false;
        }
        // Операторный метод "не равно"
        // (для сравнения итераторов на предмет неравенства):
        bool operator!=(Iterator obj){
            // Если разные индексы:
```

```

if(index!=obj.index) return true;
// Если разные контейнерные объекты:
if(t!=obj.t) return true;
// Иначе:
return false;
    }
    // Операторный метод "сложение"
    // (прибавление к итератору целого числа):
Iterator operator+(int a){
    // Индекс для итератора - результата метода:
int ind=index+a;
    // Объект итератора:
    Iterator p;
    // Значение полей итератора:
p.t=t; // Указатель на контейнерный объект
    // "Уточнение" значения индекса:
if(ind>t->size()) ind=t->size();
p.index=ind; // Индекс итератора
    // Результат метода - итератор:
return p;
    }
    // Операторный метод "вычитание"
    // (для вычитания из итератора целого числа):
Iterator operator-(int a){
    // Индекс для итератора - результата метода:
int ind=index-a;
    // Объект итератора:
    Iterator p;
    // Значения поле итератора:
p.t=t; // Указатель на контейнерный объект
    // "Уточнение" значения индекса:
if(ind<0) ind=0;
p.index=ind; // Индекс итератора
    // Результат метода - итератор:
return p;
    }
    // Операторный метод "инкремент"
    // (префиксная форма):
Iterator operator++(){
    // Увеличение значения индекса:
index++;
    // "Уточнение" значения индекса:
if(index>t->size()) index=t->size();
    // Результат метода:
return *this;
    }

```



```

        // Операторный метод "инкремент"
// (постфиксная форма):
Iterator operator++(int){
// Вызывается префиксная форма метода:
return ++(*this);
    }
    // Операторный метод "декремент"
    // (префиксная форма):
Iterator operator--(){
    // Уменьшение значения индекса:
index--;
    // "Уточнение" значения индекса:
if(index<0) index=0;
    // Результат метода:
return *this;
    }
    // Операторный метод "декремент"
// (постфиксная форма):
Iterator operator--(int){
// Вызывается префиксная версия метода:
return --(*this);
    }
}; // Окончание класса итератора
// Конструктор контейнера:
Vector(int s,X val){
length=s;
    v=new X[length];
for(inti=0;i<length;i++){
v[i]=val;
    }
    }
    // Деструктор:
~Vector(){
delete [] v;
    }
// Метод для получения размера контейнера:
int size(){
return length;
    }
    // Операторный метод для индексирования объектов:
X &operator[](int k){
return v[k];
    }
    // Метод для вставки элемента в массив
    // (аргументы - индекс и значение для вставки):
void insert(int m,X val){

```

```

int k=m;
if(m<0) k=0;
if(m>length) k=length;
X *t;
inti;
        t=new X[length+1];
for(i=0;i<k;i++){
t[i]=v[i];
        }
t[k]=val;
for(i=k+1;i<=length;i++){
t[i]=v[i-1];
        }
delete [] v;
        v=t;
length++;
        }
        // Метод для вставки элемента в массив
        // (аргументы - итератор и значение для вставки):
void insert(Iterator p,X val){
// Вызывается версия метода с первым
        // целочисленным аргументом:
insert(p.index,val);
        }
        // Метод для удаления элемента
        // (аргумент - индекс удаляемого элемента):
void erase(int m){
// Если контейнер пустой:
if(empty())return;
// Если удаляется единственный элемент:
if(length==1){
length=0;
delete [] v;
        v=0;
return;
        }
int k=m,i;
if(m<0) k=0;
if(m>=length) k=length-1;
        X *t;
        t=new X[length-1];
for(i=0;i<k;i++){
t[i]=v[i];
        }
for(i=k;i<length-1;i++){
t[i]=v[i+1];

```

```

    }
delete [] v;
    v=t;
length--;
}
    // Метод для удаления элемента
    // (аргумент - итератор):
void erase(Iterator p){
    // Вызов версии метода
    // с целочисленным аргументом:
erase(p.index);
}
    // Метод для очистки массива
    // (аргументы - итераторы):
void clear(Iterator p,Iterator r){
    // Локальный объект итератора:
    Iterator s;
        // Начальное значение локального итератора:
s=r;
        // Поэлементное удаление элементов из контейнера:
while(s!=p){
        // Уменьшение итератора:
s--;
        // Удаление элемента:
erase(s);
    }
}
    // Метод для очистки массива
    // (безаргументов):
void clear(){
    // Вызов версии метода
        // с аргументами - итераторами:
clear(begin(),end());
}
    // Метод для добавления элемента в конец массива:
void push_back(X val){
insert(length,val);
}
    // Метод для добавления элемента в начало массива:
void push_front(X val){
insert(0,val);
}
    // Метод для удаления последнего элемента:
void pop_back(){
erase(length);
}

```

```

    // Метод для удаления первого элемента:
void pop_front(){
    erase(0);
}
// Проверка контейнера на предмет отсутствия элементов:
bool empty(){
    // Результат сравнения итераторов:
    return begin()==end();
}
// Метод для отображения содержимого массива:
void show(){
    // Если контейнер пустой:
    if(empty()){
        cout<<"Нетзначений.\n";
        return;
    }
    // Локальный итератор установлен
    // в начало контейнера:
    Iterator p=begin();
    // Формально бесконечный цикл:
    while(true){
        // Отображение значения элемента
        // (значение получаем через итератор):
        cout<<*p<<" ";
        // Увеличение итератора:
        p++;
        // Проверка значения итератора:
        if(p==end()) break;
    }
    cout<<endl;
}
// Метод установки итератора в начало контейнера:
Iterator begin(){
    // Локальный итератор:
    Iterator p;
    // Индекс итератора:
    p.index=0;
    // Указатель на контейнерный объект итератора:
    p.t=this;
    // Результат метода - итератор:
    return p;
}
// Метод установки итератора в конец контейнера:
Iterator end(){
    // Локальный итератор:
    Iterator p;

```

```

// Индекс итератора:
p.index=size();
    // Указатель на контейнерный объект итератора:
p.t=this;
    // Результат метода - итератор:
return p;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Начальный размер контейнера:
int n=5;
    // Числовой объект-контейнер (заполнен нулями):
Vector<int> a(n,0);
    // Итератор для числового контейнера:
Vector<int>::Iterator q;
    // Итератор установлен в начало контейнера:
q=a.begin();
cout<<"Содержимое числового контейнера:\n";
    // Отображение содержимого контейнера:
while(q!=a.end()){
cout<<*q<<" ";
q++;
}
cout<<endl;
// Итератор установлен в конец контейнера:
q=a.end();
int k=n;
// Заполнение контейнера (с конца в начало):
while(q!=a.begin()){
q--;
*q=k--;
}
cout<<"Новое содержимое контейнера:\n";
    // Отображение содержимого контейнера:
for(int i=0;i<n;i++){
cout<<*(a.begin()+i)<<" ";
}
cout<<endl;
cout<<"Содержимое контейнера (с конца в начало):\n";
    // Отображение содержимого контейнера
    // (с конца в начало):
for(int i=0;i<n;i++){
cout<<*(a.end()-i-1)<<" ";
}
cout<<endl;

```

```

// Вставка элемента в контейнер:
a.insert(a.begin()+a.size()/2,n+1);
// Вставка элемента в начало контейнера:
a.push_front(0);
// Вставка элемента в начало контейнера:
a.push_front(*(a.begin()));
// Вставка элемента в конец контейнера:
a.push_back(n+2);
// Вставка элемента в конец контейнера:
a.push_back(*(a.end()-1));
// Итератор установлен в начало контейнера:
q=a.begin();
cout<<"После добавления элементов:\n";
// Отображение содержимого контейнера:
while(q!=a.end()){
cout<<*q<<" ";
    q=q+1;
}
cout<<endl;
// Удаление первого элемента:
a.erase(a.begin());
// Удаление последнего элемента:
a.erase(a.end());
cout<<"После удаления крайних элементов:\n";
a.show();
// Удаление элемента внутри контейнера:
a.erase(a.begin()+a.size()/2);
cout<<"После удаления элемента внутри контейнера:\n";
a.show();
// Устанавливается значение итератора:
q=a.begin()+a.size()/2;
// Удаление нескольких элементов:
a.clear(q-1,q+1);
cout<<"Удалено несколько элементов:\n";
a.show();
// Удаление последнего элемента:
a.pop_back();
// Удаление первого элемента:
a.pop_front();
cout<<"Удалены крайние элементы:\n";
a.show();
// Очистка контейнера:
a.clear();
cout<<"После очистки контейнера:\n";
a.show();
// Символьная переменная:

```

```

charm='x';
// Символьный объект-контейнер (заполнен символами 'x'):
Vector<char> b(n,m);
// Итератор для числового контейнера:
Vector<char>::Iterator p;
// Итератор установлен в начало контейнера:
p=b.begin();
cout<<"Содержимое символьного контейнера:\n";
// Отображение содержимого контейнера:
while(p!=b.end()){
cout<<*p<<" ";
p++;
}
cout<<endl;
// Итератор установлен в конец контейнера:
p=b.end();
// Заполнение контейнера (с конца в начало):
while(p!=b.begin()){
p--;
*p=m--;
}
cout<<"Новое содержимое контейнера:\n";
// Отображение содержимого контейнера:
for(int i=0;i<n;i++){
cout<<*(b.begin()+i)<<" ";
}
cout<<endl;
cout<<"Содержимое контейнера (с конца в начало):\n";
// Отображение содержимого контейнера
// (с конца в начало):
for(int i=0;i<n;i++){
cout<<*(b.end()-i-1)<<" ";
}
cout<<endl;
// Вставка элемента в контейнер:
b.insert(b.begin()+b.size()/2,'A');
// Вставка элемента в начало контейнера:
b.push_front('a');
// Вставка элемента в начало контейнера:
b.push_front(*(b.begin()));
// Вставка элемента в конец контейнера:
b.push_back('b');
// Вставка элемента в конец контейнера:
b.push_back(*(b.end()-1));
// Итератор установлен в начало контейнера:
p=b.begin();

```

```

cout<<"После добавления элементов:\n";
// Отображение содержимого контейнера:
while(p!=b.end()){
cout<<*p<<" ";
    p=p+1;
}
cout<<endl;
// Удаление первого элемента:
b.erase(b.begin());
// Удаление последнего элемента:
b.erase(b.end());
cout<<"После удаления крайних элементов:\n";
b.show();
// Удаление элемента внутри контейнера:
b.erase(b.begin()+b.size()/2);
cout<<"После удаления элемента внутри контейнера:\n";
b.show();
// Устанавливается значение итератора:
p=b.begin()+b.size()/2;
// Удаление нескольких элементов:
b.clear(p-1,p+1);
cout<<"Удалено несколько элементов:\n";
b.show();
// Удаление последнего элемента:
b.pop_back();
// Удаление первого элемента:
b.pop_front();
cout<<"Удалены крайние элементы:\n";
b.show();
// Очистка контейнера:
b.clear();
cout<<"После очистки контейнера:\n";
b.show();
return 0;
}

```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 12.3)

```

Содержимое числового контейнера:
0 0 0 0 0
Новое содержимое контейнера:
1 2 3 4 5
Содержимое контейнера (с конца в начало):
5 4 3 2 1

```


После добавления элементов:
 0 0 1 2 6 3 4 5 7 7
 После удаления крайних элементов:
 0 1 2 6 3 4 5 7
 После удаления элемента внутри контейнера:
 0 1 2 6 4 5 7
 Удалено несколько элементов:
 0 1 4 5 7
 Удалены крайние элементы:
 1 4 5
 После очистки контейнера:
 Нет значений.
 Содержимое символьного контейнера:
 x x x x x
 Новое содержимое контейнера:
 t u v w x
 Содержимое контейнера (с конца в начало):
 x w v u t
 После добавления элементов:
 aa t u A v w x b b
 После удаления крайних элементов:
 a t u A v w x b
 После удаления элемента внутри контейнера:
 a t u A w x b
 Удалено несколько элементов:
 a t w x b
 Удалены крайние элементы:
 t w x
 После очистки контейнера:
 Нет значений.

В обобщенном классе `Vector` описан внутренний класс `Iterator`. У класса два поля: указатель `t` на объект контейнерного класса `Vector<X>`, где через `X` обозначен параметр типа обобщенного класса `Vector`. Целочисленное поле `index` предназначено для запоминания индекса элемента в динамическом массиве, на который ссылается объект итератора.



На заметку

Чтобы сократить объем программного кода и немного упростить его, все члены внутреннего класса `Iterator` объявлены открытыми.

В конструкторе итератора полям `t` и `index` присваиваются нулевые значения (хотя в контексте рассматриваемого примера это и не принципиально). А вот операторный метод с прототипом `X &operator*()` имеет принципиальное значение. Речь идет об операторе звездочка `*`. С его помощью

на основе итератора можно получить значение элемента, на который итератор указывает. В качестве значения методом возвращается ссылка (важный момент!) на индексированный контейнерный объект. Благодаря тому, что в контейнерном классе `Vector` определен операторный метод для индексирования объектов контейнера, выражение `(*t)[index]`, возвращаемое результатом метода, имеет смысл.

Подробности

Операторный метод для оператора умножения `operator*()` имеет такое же название, как и операторный метод `operator*()` для оператора "звездочка". Разница в том, что первый является бинарным оператором, а второй - унарный. Проще говоря, ясли мы описываем оператор без аргументов, то это оператор "звездочка". А если мы описываем оператор с аргументом, то это оператор "умножить".

Чтобы итераторы можно было сравнивать на предмет равенства и неравенства, определяем операторные методы `operator==()` и `operator!=()`. Два итератора полагаем равными, если у них совпадают значения полей `index` и `t`, то есть если они указывают на один и тот же элемент в одном и том же контейнере.

Операторный метод `operator+()` с целочисленным аргументом `a` результатом возвращает объект класса `Iterator`. Метод обрабатывает операции по прибавлению к итератору целого числа (аргумент метода). В теле метода целочисленной переменной `ind` присваивается значение `index+a`. Это новое значение для индекса элемента, на который будет ссылаться итератор, возвращаемый результатом метода. Но перед присваиванием значение индекса проверяется на корректность: при выполнении условного оператора `if(ind>t->size()) ind=t->size()` если значение индекса превышает длину массива в контейнерном объекте, значение индекса устанавливается равным длине объекта (то есть на единицу больше, чем индекс последнего элемента). Командой `Iterator p` создается локальный объект для итератора, командами `p.t=t` (новый итератор "связан" с тем же контейнером, что и итератор, к которому прибавляется число) и `p.index=ind` полям итератора присваиваются значения и затем объект `p` возвращается как результат метода.

Аналогично описывается операторный метод `operator-()` для обработки процесса вычитания из итератора целого числа. Принципиальное отличие в том, что, во-первых, число вычитается, а не прибавляется и, во-вторых, если при вычислениях получаем индекс меньше нуля, то индексу присваивается нулевое значение.

Операторный метод `operator++()` для операции инкремента (увеличения значения на единицу) итератора содержит команду `index++`, увели-

чивающей на единицу значение поля `index`, после чего значение индекса "уточняется": если индекс получился больше размера массива в контейнере, индекс устанавливается равным размеру массива. Результатом метода возвращается объект, к которому применяется операция инкремента (инструкция `return *this`).

Подробности

Описанный таким образом операторный метод для операции инкремента обрабатывает префиксную форму вызова оператора, то есть выражения вида `++объект`. Оператор инкремента в постфиксной форме (выражения вида `объект++`) описывается с формальным целочисленным аргументом. Это же относится и к перегрузке оператора декремента: постфиксная форма оператора декремента описывается с формальным целочисленным аргументом.

Операторный метод для операции инкремента в постфиксной форме описывается с целочисленным аргументом и состоит всего из одной команды `return ++(*this)` в теле метода, которой вызывается префиксная форма оператора (то есть префиксная и постфиксная формы оператора инкремента в плане конечного результата эквивалентны).



На заметку

Две версии операторного метода `operator--()` описаны аналогично.

В контейнерном классе `Vector` описан знакомый нам по предыдущему примеру метод `insert()` для вставки нового элемента в контейнер с индексом места вставки и значением вставляемого элемента в качестве аргументов. Вместе с тем появилась еще одна версия метода, в которой первым аргументом передается не индекс, а итератор. В теле данной версии метода вызывается его же версия, но с аргументом-индексом. Индекс определяется на основе итератора как его поле `index`.

Немного изменился программный код метода `erase()`, предназначенный для удаления элементов из контейнера. Есть версия метода, когда аргументом передается индекс удаляемого элемента. Если так, то проверяется несколько условий. Во-первых, с помощью метода `empty()` проверяется, пустой контейнер или нет. Если контейнер пустой (то есть если выполняется попытка удалить элемент из пустого контейнера), выполнение метода завершается без каких-либо манипуляций с массивом в контейнере.



На заметку

Метод `empty()` возвращает значение `true` если контейнер пустой (не содержит элементов) и `false` в противном случае (если в контейнере есть элементы).

Если в контейнере один элемент, выполняется удаление динамического массива, а полям контейнерного объекта присваиваются нулевые значения.

В прочих случаях выполняется практически тот же самый код, что и в аналогичном методе из предыдущего примера.

У метода `erase()` есть версия с аргументом-итератором, указывающим на удаляемый элемент. В теле метода в этом случае вызывается версия метода с целочисленным аргументом. Целочисленный аргумент вычисляется как значение поля `index` итератора, переданного аргументом методу.

Претерпел изменения и метод `clear()`. Теперь методом можно не только выполнять полную очистку контейнера, но и удалить элементы в определенном диапазоне. "Базовой" здесь является версия метода с двумя аргументами. Первый итератор указывает на начальный элемент для удаления (то есть тот элемент, начиная с которого выполняется удаление элементов). Второй аргумент - итератор, указывающий на элемент, первый после последнего удаляемого элемента (то есть второй итератор указывает на первый не удаляемый элемент). В теле метода инструкцией `Iterator s` создается локальный итератор. Командой `s=r` начальное значение итератора устанавливается такое же, как и второй аргумент метода.

Подробности

Мы для класса итератора операцию присваивания не определяли. Поэтому при присваивании итераторов выполняется побитовое копирование. Нас в принципе это устраивает, поскольку у класса `Iterator` два поля (целочисленное `index` и указатель `t` на объект контейнерного класса) и их копирование к ошибкам не приводит.

Таким образом, локальный итератор `s` в начальный момент указывает на элемент в конце удаляемого диапазона, причем сам элемент не удаляется.

Запускается оператор цикла `while` с проверкой условия `s!=p` (через `p` обозначен первый аргумент метода). Следовательно, оператор цикла выполняется до тех пор, пока итератор `s` не будет указывать на первый элемент из удаляемого диапазона.

В теле оператора цикла командой `s--` "уменьшается" итератор, в результате чего итератор "перебрасывается" на элемент слева от текущего, после чего командой `erase(s)` данный элемент удаляется.

Версией метода `clear()` без аргументов выполняется полная очистка контейнера. В теле этой версии метода выполняется команда `clear(begin(),end())` — то есть вызывается версия метода с двумя аргументами. Здесь использованы методы `begin()` и `end()`. Методом `be-`

`gin()` возвращается итератор, указывающий на первый элемент в контейнере. Методом `end()` возвращается итератор, указывающий позицию за последним элементом в контейнере.

Подробности

Получается, что метод `end()` возвращает итератор, указывающий на элемент после последнего элемента в контейнере. Понятно, что такого элемента нет. В этом смысле итератор, возвращаемый методом `end()`, лучше рассматривать, все же, как индикатор окончания контейнера. Также следует отметить, что это стандартный подход, который во многих случаях упрощает реализацию программных кодов.

В теле метода `begin()` создается локальный итератор `p` (возвращается результатом метода), полю `index` итератора присваивается нулевое значение (поскольку итератор должен указывать на первый элемент в контейнере). Полю `t` итератора присваивается значение `this`. Последнее означает, что итератор предназначен для работы с контейнером, из которого вызывается метод `begin()`.

Программный код метода `end()` похож на код метода `begin()`, с той лишь разницей, что полю `index` возвращаемого итератора присваивается в качестве значения результат вызова метода `size()` - то есть значение поля `length` объекта-контейнера.



На заметку

Итератор, возвращаемый методом `end()`, содержит индекс со значением `length`. Это при том, что индекс последнего элемента в массиве контейнерного объекта равен `length-1`.

Метод `empty()` мы уже упоминали выше. Результатом метода `empty()` возвращается выражение `begin()==end()`. Значение выражения равно `true`, если совпадают итераторы, возвращаемые методами `begin()` и `end()` — совпадают в том смысле, что у них значения полей `index` одинаковые (поля `t` совпадают автоматически, поскольку методы вызываются из одного объекта). Но поле `index` итератора, возвращаемого методом `begin()`, всегда равно 0. Поэтому итераторы совпадают, если поле `index` итератора, возвращаемого методом `end()`, также равняется 0. С другой стороны, поле данного итератора совпадает с длиной массива в контейнере. А нулевая длина массива только в пустом контейнере.

Еще мы изменили программный код метода `show()`, которым отображается содержимое контейнера. Теперь весь процесс базируется на использовании итераторов. После проверки контейнера на предмет, не пустой ли он, командой `Iterator p=begin()` создается локальный итератор и устанавливается

ливается в начало контейнера. Далее запускается формально бесконечный оператор цикла (условие `true` в `while`-инструкции, выход из бесконечного цикла реализуется благодаря условному оператору). В операторе цикла командой `cout<<*p<<" "` отображается значение элемента, на который указывает итератор `p`, затем итератор командой `p++` перебрасывается на следующий элемент.



На заметку

Если перед итератором поставить звездочку `*` (например, как в инструкции `*p`), получим значение элемента, на который ссылается итератор.

Если при проверке условия `p==end()` в условном операторе окажется, что итератор установлен в конец контейнера, инструкцией `break` завершается выполнение условного оператора.

При выполнении программы создается контейнерный объект `a` с целочисленным массивом. Все элементы массива заполняются нулями (команда `Vector<int> a(n, 0)`). Для работы с контейнером нам понадобится итератор. Итератор `q` создаем командой `Vector<int>::Iterator q`. Здесь поскольку класс итератора `Iterator` описан внутри класса контейнера `Vector`, указываем оба класса (чрезоператор расширения контекста `::`), причем для контейнерного класса указывается значение `int` для обобщенного параметра. Командой `q=a.begin()` итератор `q` устанавливается в начало контейнера `a`.



На заметку

Напомним, что метод `begin()` устанавливает связь между итератором и тем контейнером, из которого вызывается метод `begin()`. Это же замечание относится и к методу `end()`. В данном случае связь устанавливается между итератором `q` и контейнером `a`.

Для отображения содержимого контейнера запускается оператор цикла. Оператор выполняется, пока истинно условие `q!=a.end()`, то есть пока итератор `q` не установлен в конец контейнера. В самом операторе отображается (команда `cout<<*q<<" "`) значение текущего (на который указывает итератор) элемента, после чего итератор командой `q++` переходит к следующему элементу контейнера.

На следующем этапе мы заполняем контейнер числами. Для этого итератор устанавливается в конец контейнера (команда `q=a.end()`), и запускается еще один оператор цикла. Там сначала командой `q--` итератор смещается на одну позицию влево (перебрасывается на предыдущий элемент по срав-

нению к текущему), после чего командой `*q=k--` данному элементу присваивается значение.



На заметку

Поскольку операторный метод для оператора "звездочка" возвращает не просто значение элемента в контейнере, а ссылку на это значение, то инструкцию вида `*q` можно использовать не только для считывания значения элемента, но и для присваивания элементу значения.

Далее, поскольку в выражении `*q=k--` использована постфиксная форма оператора декремента, то сначала элементу контейнера присваивается текущее значение переменной `k`, и только после этого значение переменной `k` уменьшается на единицу.

Оператор цикла выполняется до тех пор, пока истинно условие `q!=a.begin()` (итератор установлен не в начало контейнера).

Подробности

Не исключено, что кому-то может показаться, будто при выполнении оператора цикла не присваивается новое значение начальному элементу в контейнере (поскольку если итератор `q` установлен в начало контейнера, то оператор цикла завершает свою работу). Разумеется, все не так. Дело в том, что при проверке условия в операторе цикла итератор всегда указывает на тот элемент, которому уже присвоено значение. Поэтому на момент, когда при проверке условия итераторы `q` и `a.begin()` станут совпадать, начальный элемент будет с новым значением (и это значение 1).

После присваивания значений элементам в контейнере, его содержимое снова отображается. Для этого запускаем еще один оператор цикла (с индексной переменной `i`), а доступ к значению элемента получаем с помощью выражения `*(a.begin()+i)`. Вычисляется оно следующим образом: `(a.begin()+i)` - это итератор на элемент, отстоящий от начального на `i` позиций, а соответственно `*(a.begin()+i)` - значение такого элемента.

Похожим образом значения элементов отображаются в обратном порядке: при том же характере изменения индексной переменной `i` значение элементов вычисляется выражением `*(a.end()-i-1)`: результатом выражения `(a.end()-i-1)` является итератор, указывающий на элемент, находящийся на `i` позиций влево от последнего элемента.



На заметку

Напомним, что метод `end()` указывает не на последний элемент, а на "позицию" за последним элементом.

Для вставки нового элемента в контейнер использована команда `a.insert(a.begin()+a.size()/2,n+1)`. Вставляется числовое зна-

чение $n+1$. Позиция, в которую вставляется элемент, определяется итератором `a.begin()+a.size()/2`. Данный итератор указывает на элемент, отстоящий на `a.size()/2` позиций вправо от начального элемента (то есть элемент вставляется где-то примерно посередине массива). Для вставки нулевого элементов в начало контейнера использована команда `a.push_front(0)`. Командой `a.push_front(*a.begin())` в начало контейнера вставляется еще один элемент. Значение элемента вычисляется выражением `*a.begin()`. А это есть ни что иное, как значение начального (на момент выполнения всей команды) элемента (то есть начальный элемент будет продублирован).

В конец контейнера элемент со значением $n+2$ вставляется командой `a.push_back(n+2)`. Еще один такой же элемент вставляется в конец контейнера при выполнении команды `a.push_back(*a.end()-1)`.



На заметку

На последний элемент контейнера `a` указывает итератор `a.end()-1`.

На данном этапе мы вновь отображаем содержимое контейнера. С этой целью командой `q=a.begin()` итератор `q` устанавливается в начало контейнера `a`, после чего в игру вступает оператор цикла. За каждый цикл значение итератора изменяется командой `q=q+1` (прибавление к итератору единицы и присваивание полученного результата), но перед этим значение `*q` отображается в окне вывода. Оператор выполняется при истинном условии `q!=a.end()`.

Затем элементы из контейнера начинают удаляться. Первый элемент контейнера удаляем командой `a.erase(a.begin())`. Здесь вызывается версия метода `erase()` с аргументом `a.begin()` - итератором, указывающим на первый элемент контейнера. Последний элемент из контейнера удаляется командой `a.erase(a.end())`. Хотя аргументом методу `erase()` передан итератор `a.end()`, указывающий на несуществующий элемент, метод `erase()` описан так, что "целеуказание" на удаляемый элемент автоматически переопределяется и удаляется последний элемент контейнера.



На заметку

Далее для проверки содержимого контейнера `a` выполняется команда `a.show()`.

Элемент внутри контейнера удаляется с помощью команды `a.erase(a.begin()+a.size()/2)`. Для удаления нескольких элементов использована команда `a.clear(q-1,q+1)`, причем значение итератора `q` предварительно задано командой `q=a.begin()+a.size()/2`.



На заметку

При выполнении команды `a.clear(q-1, q+1)` первый удаляемый элемент определяется итератором `q-1`, а последний удаляемый элемент определяется итератором `q`. Элемент, на который указывает итератор `q+1`, не удаляется.

Наконец, командой `a.pop_back()` удаляется последний элемент, а командой `a.pop_front()` удаляется первый элемент в контейнере. Для полной очистки контейнера (удаления всех оставшихся элементов) использована команда `a.clear()`.

Во второй части программы командой `Vector<char> b(n, m)` создается контейнерный объект `b` с символьным массивом. Итератор для данного контейнера создаем командой `Vector<char>::Iterator p`. С контейнером `b` и итератором `p` продельваются практически те же манипуляции, что и с контейнером `a` и итератором `q` соответственно (с поправкой на то, что теперь речь идет о символьных значениях). Хочется верить, что результат выполняемых операций читателю будет ясен без дополнительных пояснений.

12.3. Стандартные подходы

*Ну, понимаете, я за кефиром пошел - а тут такие приключения!
из к/ф "Гостья из будущего"*

Теперь рассмотрим предыдущий пример, но только вместо описания собственного контейнерного класса `Vector` с внутренним классом итератора `Iterator` воспользуемся классом `vector` из библиотеки шаблонов STL. Для того чтобы класс `vector` можно было использовать в программе, необходимо включить в программу заголовок `<vector>`.

Сразу перейдем к рассмотрению программного кода, представленного в листинге 12.4.

Листинг 12.4. Использование контейнера `vector`

```
#include <iostream>
#include <vector>
using namespace std;
// Обобщенная функция для отображения содержимого
// контейнера, созданного на основе класса vector:
template <class X> void show(vector<X> v){
if(v.empty()){ // Если контейнер пустой
cout<<"Нет элементов.\n";
return;
}
}
```

```

    // Если контейнер содержит элементы:
    for(int i=0;i<v.size();i++){
    // Объект контейнера можно индексировать:
    cout<<v[i]<<" "; // Отображение значения элемента
    }
    cout<<endl;
    }
    // Главная функция программы:
    int main(){
        // Начальный размер контейнера:
        int n=5;
        // Числовой объект-контейнер (заполнен нулями):
        vector<int> a(n,0);
        // Итератор для числового контейнера:
        vector<int>::iterator q;
        // Итератор установлен в начало контейнера:
        q=a.begin();
        cout<<"Содержимое числового контейнера:\n";
        // Отображение содержимого контейнера:
        while(q!=a.end()){
        cout<<*q<<" ";
        q++;
        }
        cout<<endl;
        // Итератор установлен в конец контейнера:
        q=a.end();
        int k=n;
        // Заполнение контейнера (с конца в начало):
        while(q!=a.begin()){
        q--;
        *q=k--;
        }
        cout<<"Новое содержимое контейнера:\n";
        // Отображение содержимого контейнера:
        for(int i=0;i<n;i++){
        cout<<*(a.begin()+i)<<" ";
        }
        cout<<endl;
        cout<<"Содержимое контейнера (с конца в начало):\n";
        // Отображение содержимого контейнера
        // (с конца в начало):
        for(int i=0;i<n;i++){
        cout<<*(a.end()-i-1)<<" ";
        }
        cout<<endl;
        // Вставка элемента в контейнер:

```

```

a.insert(a.begin()+a.size()/2,n+1);
// Вставка элемента в начало контейнера:
a.insert(a.begin(),0);
// Вставка элемента в начало контейнера:
a.insert(a.begin(),*(a.begin()));
// Вставка элемента в конец контейнера:
a.push_back(n+2);
// Вставка элемента в конец контейнера:
a.push_back(*(a.end()-1));
// Итератор установлен в начало контейнера:
q=a.begin();
cout<<"После добавления элементов:\n";
// Отображение содержимого контейнера:
while(q!=a.end()){
cout<<*q<<" ";
    q=q+1;
}
cout<<endl;
// Удаление первого элемента:
a.erase(a.begin());
// Удаление последнего элемента:
a.erase(a.end()-1);
cout<<"После удаления крайних элементов:\n";
show(a);
// Удаление элемента внутри контейнера:
a.erase(a.begin()+a.size()/2);
cout<<"После удаления элемента внутри контейнера:\n";
show(a);
// Устанавливается значение итератора:
q=a.begin()+a.size()/2;
// Удаление нескольких элементов:
a.erase(q-1,q+1);
cout<<"Удалено несколько элементов:\n";
show(a);
// Удаление последнего элемента:
a.pop_back();
// Удаление первого элемента:
a.erase(a.begin());
cout<<"Удалены крайние элементы:\n";
show(a);
// Очистка контейнера:
a.clear();
cout<<"После очистки контейнера:\n";
show(a);
// Символьная переменная:
char m='x';

```

```

    // Символьный объект-контейнер (заполнен символами 'x'):
    vector<char> b(n,m);
    // Итератор для числового контейнера:
    vector<char>::iterator p;
    // Итератор установлен в начало контейнера:
    p=b.begin();
    cout<<"Содержимое символьного контейнера:\n";
    // Отображение содержимого контейнера:
    while(p!=b.end()){
        cout<<*p<<" ";
        p++;
    }
    cout<<endl;
    // Итератор установлен в конец контейнера:
    p=b.end();
    // Заполнение контейнера (с конца в начало):
    while(p!=b.begin()){
        p--;
        *p=m--;
    }
    cout<<"Новое содержимое контейнера:\n";
    // Отображение содержимого контейнера:
    for(int i=0;i<n;i++){
        cout<<*(b.begin()+i)<<" ";
    }
    cout<<endl;
    cout<<"Содержимое контейнера (с конца в начало):\n";
    // Отображение содержимого контейнера
    // (с конца в начало):
    for(int i=0;i<n;i++){
        cout<<*(b.end()-i-1)<<" ";
    }
    cout<<endl;
    // Вставка элемента в контейнер:
    b.insert(b.begin()+b.size()/2,'A');
    // Вставка элемента в начало контейнера:
    b.insert(b.begin(),'a');
    // Вставка элемента в начало контейнера:
    b.insert(b.begin(),*(b.begin()));
    // Вставка элемента в конец контейнера:
    b.push_back('b');
    // Вставка элемента в конец контейнера:
    b.push_back(*(b.end()-1));
    // Итератор установлен в начало контейнера:
    p=b.begin();
    cout<<"После добавления элементов:\n";

```

```
// Отображение содержимого контейнера:
while(p!=b.end()){
    cout<<*p<<" ";
    p=p+1;
}
cout<<endl;
// Удаление первого элемента:
b.erase(b.begin());
// Удаление последнего элемента:
b.erase(b.end()-1);
cout<<"После удаления крайних элементов:\n";
show(b);
// Удаление элемента внутри контейнера:
b.erase(b.begin()+b.size()/2);
cout<<"После удаления элемента внутри контейнера:\n";
show(b);
// Устанавливается значение итератора:
p=b.begin()+b.size()/2;
// Удаление нескольких элементов:
b.erase(p-1,p+1);
cout<<"Удалено несколько элементов:\n";
show(b);
// Удаление последнего элемента:
b.pop_back();
// Удаление первого элемента:
b.erase(b.begin());
cout<<"Удалены крайние элементы:\n";
show(b);
// Очистка контейнера:
b.clear();
cout<<"После очистки контейнера:\n";
show(b);
return 0;
}
```

Результат выполнения программы фактически такой же, как и в предыдущем случае:

Результат выполнения программы (из листинга 12.4)

```
Содержимое числового контейнера:
0 0 0 0 0
Новое содержимое контейнера:
1 2 3 4 5
Содержимое контейнера (с конца в начало):
5 4 3 2 1
```

```

После добавления элементов:
0 0 1 2 6 3 4 5 7 7
После удаления крайних элементов:
0 1 2 6 3 4 5 7
После удаления элемента внутри контейнера:
0 1 2 6 4 5 7
Удалено несколько элементов:
0 1 4 5 7
Удалены крайние элементы:
1 4 5
После очистки контейнера:
Нет элементов.
Содержимое символьного контейнера:
x xxxx
Новое содержимое контейнера:
t u v w x
Содержимое контейнера (с конца в начало):
x w v u t
После добавления элементов:
aa t u A v w x b b
После удаления крайних элементов:
a t u A v w x b
После удаления элемента внутри контейнера:
a t u A w x b
Удалено несколько элементов:
a t w x b
Удалены крайние элементы:
t w x
После очистки контейнера:
Нет элементов.

```

Если не считать того, что отсутствует описание класса `Vector`, программный код остался практически таким же - имеется в виду главная функция программы `main()`. Правда, появилось описание шаблонной функции `show()`, но там на самом деле нового тоже мало.

Итак, в программе там, где использовался класс `Vector`, теперь используется библиотечный класс `vector`, а там, где мы использовали внутренний класс `Iterator`, используется (определенный в контейнере `vector`) тип `iterator`.



На заметку

Чтобы использование класса `vector` и сопутствующих утилит стало возможным, в шапку программы добавлена инструкция `#include <vector>`.

У контейнерного класса `vector` есть методы со знакомыми нам названиями (все совпадения случайны):

- Метод `empty()` возвращает значение `true` если контейнер пустой, и `false` в противном случае (то есть метод "дает ответ" на вопрос "пустой контейнер или нет").
- Метод `size()` результатом возвращает количество элементов в контейнере.
- Метод `begin()` результатом возвращает итератор, установленный в начало контейнера.
- Метод `end()` возвращает результатом итератор, установленный в конец контейнера - имеется в виду "позиция" за последним элементом в контейнере (то есть с этим методом ситуация такая же, как и с одноименным методом, описанным в предыдущем примере).
- Метод `insert()` предназначен для вставки элемента в контейнер. Аргументами методу могут передаваться итератор, определяющий место вставки элемента, и значение вставляемого элемента.
- Метод `push_back()` выполняет вставку нового элемента (аргумент метода) в конец контейнера.
- Метод `erase()` удаляет элемент или элементы из контейнера. Если методу аргументом передать итератор, то будет удален элемент, на который итератор указывает. Если методу передать аргументами два итератора, то будет удалена группа элементов, начиная с того элемента, на который указывает первый итератор, и до элемента, на который указывает второй итератор (но не включая этот элемент!). Другими словами, элемент, на который указывает второй итератор, остается в контейнере (не удаляется).
- Метод `pop_back()` предназначен для удаления последнего элемента в контейнере.
- Метод `clear()` предназначен для полной очистки контейнера.

Кроме этого, с итераторами, связанными с контейнерным классом `vector`, можно выполнять такие арифметические операции, как прибавление и вычитание целых чисел, инкремент и декремент, можно применять процедуры взятия значения (оператор звездочка `*`). Объекты контейнерного класса `vector` можно индексировать.

Подробности

По сравнению с предыдущими примером, различия в "поведении" методов все же

есть. А есть и просто "потери". Так, у класса `vector` нет методов `push_front()` и `pop_front()`. Поэтому (здесь и далее через `obj` обозначен контейнерный объект):

- В программевместо команд вида `obj.push_front(значение)` использована альтернативная команда `obj.insert(obj.begin(), значение)`.
- Вместо команд формата `obj.pop_front()` использованы выражения вида `obj.erase(obj.begin())`.

Поскольку метод `clear()` вызывается без аргументов, то команды вида `obj.clear(итератор_1, итератор_2)` заменены на `obj.erase(итератор_1, итератор_2)`.

Еще один момент связан с использованием метода `erase()`. Когда мы определяли этот метод самостоятельно, то предусмотрели "уточнение" некорректных аргументов. Например, при выполнении команды вида `obj.erase(obj.end())` в предыдущем примере в контейнере `obj` удалялся бы последний элемент, хотя итератор `obj.end()` указывает на позицию за последним элементом. Метод `erase()` контейнерного класса `vector` подобным демократизмом не обладает. Поэтому для удаления последнего элемента из контейнера `obj` с помощью метода `erase()` используем команду `obj.erase(obj.end() - 1)`.

Поскольку у контейнерного класса `vector` метода `show()` нет, мы для удобства описали обобщенную функцию с таким же названием. Аргументом функции передается объект контейнерного класса. В теле функции сначала проверяется контейнер на предмет того, пустой он или нет. Если контейнер не пустой, запускается оператор цикла, в котором отображаются значения элементов контейнера. Стоит обратить внимание на использование в функции методов `empty()` (метод идентификации пустого контейнера) и `size()` (метод позволяет узнать количество элементов в контейнере). Объекты контейнера индексируются как элементы массива - то есть с контейнерным объектом мы можем обращаться как с массивом, что достаточно удобно.



На заметку

В программе команды вида `obj.show()` заменены командами `show(obj)`, где через `obj` обозначен контейнерный объект.

В завершение главы хочется отметить, что библиотека шаблонов STL представляет собой очень мощное подспорье, позволяющее создавать простые, гибкие и эффективные коды. В этом смысле использование утилит библиотеки на практике только приветствуется.

Глава 13.

ОБРАБОТКА ИСКЛЮЧИТЕЛЬНЫХ СИТУАЦИЙ



Отрицательный результат - это тоже результат.

из к/ф "Гостья из будущего"

Не секрет, что выполнение программ иногда приводит к ошибкам. В принципе, такая ситуация вполне нормальная. Ведь нет идеальных программистов и даже очень хороший разработчик время от времени ошибается. Но дело не только в этом. Бывают ситуации (осознанно или неосознанно созданные), в которых крайне сложно предугадать вероятность возникновения ошибки. Поэтому важно соответствующие "моменты" выявлять и по возможности "блокировать". Главная задача "блокирования" обычно состоит в том, чтобы программа продолжала выполняться, несмотря на ошибки, которые могут возникнуть.

Подробности

Обычно если при выполнении программы возникает ошибка, выполнение программы завершается в "аварийном" порядке. В принципе это представляется логичным. Вместе с тем, хорошо написанная программа завершает работу так, как хочет программист, а не в зависимости от того, как "сложилась звезда".

Существует механизм, позволяющий обрабатывать ситуации, возникающие вследствие тех или иных ошибок. Именно этому механизму посвящена данная глава. Нам понадобится некоторая терминология. Для начала познакомимся с понятием *исключения* или *исключительной ситуации*: в основном будем под этими понятиями подразумевать ситуацию, когда в процессе выполнения программы возникает ошибка. Про обрабатываемую ошибку будем также говорить как о *перехватываемой ошибке* или *перехватываемом исключении*.

13.1. Пример с ошибкой

Лучше бы я упал вместо тебя.

из к/ф "Бриллиантовая рука"

Чтобы понять, как в общих чертах работает механизм обработки исключительных ситуаций (механизм обработки ошибок), рассмотрим небольшой пример. В этом примере при определенных обстоятельствах может возникнуть ошибка.



На заметку

Речь идет об ошибке, возникающей в процессе выполнения программы. То есть при компиляции программы никаких ошибок не возникает.

Мы рассмотрим ситуацию, с которой на самом деле сталкивались многократно: речь идет о создании динамического массива. Программа, с которой мы начнем изучение процесса обработки исключительных ситуаций, очень простая. В ней всего лишь создается динамический массив - мы даже заполнять его не планируем. Единственная особенность программы в том, что размер создаваемого массива определяется пользователем уже в процессе выполнения программы: при запуске программы появляется сообщение с просьбой ввести целое число, число считывается, и в соответствии со считанным значением создается динамический массив.



На заметку

Правильнее было бы вместо фразы "создается массив" использовать фразу "предпринимается попытка создать массив". Но всему свое время.

Рассмотрим программный код, представленный в листинге 13.1. Он очень простой.

Листинг 13.1. Создание массива

```
#include <iostream>
using namespace std;
int main(){
    int *p; // Указатель (имя массива)
    int n; // Размер массива
    cout<<"Введите целое число: ";
    cin>>n; // Считывается размер массива
        // Создание массива:
        p=new int[n];
    cout<<"Создан массив из "<<n<<" элементов."<<endl;
        // Удаление массива:
    delete [] p;
    cout<<"Выполнение программы завершено.\n";
    return 0;
}
```

Многое зависит от того, какое число введет пользователь. Например, если пользователь введет число 10, то результат выполнения программы, скорее всего, будет таким (жирным шрифтом выделено введенное пользователем значение):

Результат выполнения программы (из листинга 13.1)

Введите целое число: **10**
Создан массив из 10 элементов.

Выполнение программы завершено.

Пока ничего интересного. Но все может круто измениться, если мы, скажем, введем отрицательное число. В этом случае результат окажется, например, таким (жирным шрифтом выделен ввод пользователя, курсивом выделено сообщение, которое автоматически отображается в области вывода):

Результат выполнения программы (из листинга 13.1)

Введите целое число: **-2**

```
This application has requested the Runtime to
terminate it in an unusual way. Please contact the
application's support team for more information.
terminate called after throwing an instance of
'std::bad_alloc' what(): std::bad_alloc
```

В данном случае показана примерная реакция программы на введенное пользователем значение **-2**. Конкретный тип сообщения, появляющегося в области вывода, не принципиален. Важно другое: выполнение программы было прервано. Понять это несложно, если обратить внимание, что программой выведена только первая строка из трех ожидаемых.

Итак, мы столкнулись с ошибкой. Причина ее в принципе понятна - ведь мы попытались создать массив отрицательного размера. Вопрос: как ее избежать? Ответ кажется очевидным: размещаем условный оператор, в котором выполняем проверку вводимого пользователем значения, исключив тем самым ситуацию, когда размер массива отрицательный. Проблема, однако, в том, что ошибка может возникнуть и при попытке создать массив очень большого размера. Например, если мы попытаемся создать массив из 1000000000 (миллиард!) элементов, с большой вероятностью увидим такое:

Результат выполнения программы (из листинга 13.1)

Введите целое число: **1000000000**

```
This application has requested the Runtime to
terminate it in an unusual way. Please contact the
application's support team for more information.
```

```
terminate called after throwing an instance of
'std::bad_alloc' what(): std::bad_alloc
```

Здесь важно понимать, что теоретически программа может дать сбой и при создании более скромных (по размеру) массивов. Причина в том, что память для массива выделяется динамически, и если обстоятельства сложатся так, что память задействуется для решения иных задач, ее банально может не хватить даже для очень скромного массива.

Резюме такое: при создании динамического массива может возникнуть ошибка, и эта проблема принципиальная. Решать ее нужно профессионально.

Задействуем механизм обработки исключительных ситуаций. Для этого фрагмент программного кода, при выполнении которого могут возникнуть ошибки, заключаем в `try`-блок: указывается ключевое слово `try`, а после него в фигурных скобках размещается код, "подозреваемый" в вольнодумном поведении (*контролируемый код*). После `try`-блока указывается `catch`-блок: ключевое слово `catch`, в круглых скобках после него трое-точие `...` (называется *эллипсис*), а затем в фигурных скобках указывается программный код, выполняемый при обработке возникшей ошибки (*код обработки*). Вся конструкция выглядит так (основные синтаксические конструкции выделены жирным шрифтом):

```
try{
// контролируемый код
}
catch(...) {
// код обработки
}
```

Функционирует вся эта чудо-система следующим образом:

- Выполняется контролируемый код в `try`-блоке. Если в процессе возникает ошибка, выполнение кода в `try`-блоке прекращается, и управление передается `catch`-блоку. Выполняется код обработки в `catch`-блоке, после чего начинает выполняться программный код после `try-catch` конструкции.
- Если при выполнении контролируемого кода в `try`-блоке ошибок не возникает, то `catch`-блок игнорируется и код обработки не выполняется. После завершения выполнения `try`-блока управление передается первой команде после `try-catch` конструкции.

В листинге 13.2 приведен пример программы, в которой создается динамический массив, но на этот раз еще и выполняется обработка исключительных ситуаций.

Листинг 13.2. Создаем массив и обрабатываем ошибки

```
#include <iostream>
using namespace std;
int main(){
    int *p; // Указатель (имя массива)
    int n; // Размер массива
    try{ // Контролируемый код
        cout<<"Введите целое число: ";
        cin>>n; // Считывается размер массива
        // Создание массива:
        p=newint[n];
        cout<<"Создан массив из "<<n<<" элементов."<<endl;
        // Удаление массива:
        delete [] p;
    } // Завершение try-блока
    catch(...){ // Код обработки
        cout<<"К сожалению, массив не создан.\n";
    } // Завершение catch-блока
    cout<<"Выполнение программы завершено.\n";
    return 0;
}
```

Результат выполнения программы может быть таким:

Результат выполнения программы (из листинга 13.2)

```
Введите целое число: 10
Создан массив из 10 элементов.
Выполнение программы завершено.
```

Или таким:

Результат выполнения программы (из листинга 13.2)

```
Введите целое число: -2
К сожалению, массив не создан.
Выполнение программы завершено.
```

Вводимые пользователем значения выделены жирным шрифтом. Видим, что если пользователь вводит значение **10**, то программа выполняется так, как и в предыдущем примере (при корректном значении параметра для размера массива). Это означает, что выполняются все команды в `try`-блоке,

после чего выполняются команды после `try-catch` конструкции. Команды в `catch`-блоке игнорируются, поскольку в `try`-блоке ошибки не возникло.

В случае, когда пользователь ввел значение `-2`, массив создан не был. При попытке создать динамический массив в `try`-блоке возникла ошибка. Выполнение команд в `try`-блоке было прекращено и начали выполняться команды в `catch`-блоке. Именно при выполнении `catch`-блока появляется сообщение о невозможности создать массив. После выполнения `catch`-блока начинают выполняться команды после `try-catch` конструкции. В частности, появляется сообщение о завершении выполнения программы.

13.2. Персонализируем ошибки

А нельзя, чтоб этот гипс вместо меня поносил кто-то другой?

из к/ф "Бриллиантовая рука"

Троеточие (эллипсис) в круглых скобках после `catch`-инструкции означает, что таким `catch`-блоком перехватываются все ошибки (которые в принципе можно перехватить), возникающие в `try`-блоке. Чтобы пояснить это, рассмотрим пример, представленный в листинге 13.3. В программе мы, по сравнению с предыдущим примером, решаем похожую задачу: создается контейнер для набора целочисленных элементов, у всех элементов значения нулевые, но одному элементу присваивается единичное значение. Размер контейнера и индекс элемента, которому присваивается единичное значение, указываются пользователем в процессе выполнения программы. Контейнер создается на основе контейнерного класса `vector`.

Подробности

Для использования класса `vector` инструкцией `#include <vector>` в шапке программы подключается соответствующий заголовок. Сразу обращаем внимание еще на одну особенность выполнения операций с элементами контейнера: обращение к элементам выполняется с помощью метода `at()`. Метод вызывается из контейнерного объекта, а аргументом методу передается индекс элемента, к которому выполняется обращение (для считывания или присваивания значения).

Рассмотрим программный код:

Листинг 13.3. Создание контейнера

```
#include <iostream>
#include <vector>
using namespace std;
int main(){
    int n,m; // Размер контейнера и индекс элемента
    try{ // Контролируемый код
```

```

cout<<"Укажите размер контейнера: ";
cin>>n; // Считывается размер контейнера
        // Создание контейнера (значения - нули):
vector<int> p(n,0); // Контейнер
cout<<"Укажите индекс элемента: ";
cin>>m; // Считывается индекс элемента
        // Присваивание единичного значения элементу:
p.at(m)=1;
cout<<"Создан контейнер из "<<n<<" элементов:"<<endl;
        // Отображение содержимого контейнера:
for(int i=0;i<p.size();i++){
cout<<p.at(i)<<" ";
}
cout<<endl;
} // Завершение try-блока
catch(...){ // Код обработки
cout<<"К сожалению, произошла ошибка.\n";
} // Завершение catch-блока
cout<<"Выполнение программы завершено.\n";
return 0;
}

```

В программе объявляются две целочисленные переменные: в переменную `n` записывается значение для размера контейнера, а в переменную `m` записывается значение индекса элемента, которому присваивается единичное значение. Значение обеих переменных вводится пользователем с клавиатуры, поэтому на момент компиляции программы значения переменных `n` и `m` неизвестны.

Сначала в `try`-блоке запрашивается и считывается значение переменной `n`. После этого командой `vector<int> p(n,0)` выполняется попытка создать на основе обобщенного класса `vector` контейнер `p` из целочисленных элементов с нулевыми значениями. Количество элементов в контейнере определяется значением переменной `n`.



На заметку

В данном месте может произойти ошибка, связанная с невозможностью выделить в памяти место для контейнера - например, если пользователем введено отрицательное значение для переменной `n`.

Если процесс создания контейнера завершился удачно, выводится запрос на ввод пользователем значения индекса элемента, которому будет присваиваться значение. После того, как значение переменной `m` считывается, командой `p.at(m)=1` элементу с индексом `m` в контейнере `p` присваивается значение 1.



На заметку

При попытке присвоить значение элементу может произойти ошибка - если индекс m не попадает в допустимый диапазон значений от 0 до $n-1$ для контейнера, содержащего n элементов.

При условии, что процесс присваивания значения элементу контейнера прошел без сюрпризов, запускается оператор цикла, а в нем последовательно отображаются элементы контейнера.



На заметку

Для определения размера контейнера используем метод `size()`, а для обращения к элементу контейнера по его индексу используем метод `at()`.

Это основные команды в `try`-блоке. Помимо `try`-блока есть еще `catch`-блок со всего одной командой `cout<<"К сожалению, произошла ошибка.\n"`. Эта команда выполняется только в том случае, если при выполнении `try`-блока возникла ошибка. Команда `cout<<"Выполнение программы завершено.\n"` после `catch`-блока выполняется в любом случае.

Теперь выясним, каким в принципе может быть результат выполнения программы. Ниже приведены сообщения, которые появляются, если оба целочисленных значения введены пользователем корректно (здесь и далее то, что вводится пользователем, выделено жирным шрифтом):

Результат выполнения программы (из листинга 13.3)

```
Укажите размер контейнера: 10
Укажите индекс элемента: 3
Создан контейнер из 10 элементов:
0 0 0 1 0 0 0 0 0 0
Выполнение программы завершено.
```

Еще один случай - когда введено некорректное значение для размера контейнера. Результат выполнения программы мог бы быть таким:

Результат выполнения программы (из листинга 13.3)

```
Укажите размер контейнера: -2
К сожалению, произошла ошибка.
Выполнение программы завершено.
```

Наконец, ниже показано, каким будет результат выполнения программы, если размер контейнера указан корректно, а вот индекс не попадает в допустимый диапазон значений:

Результат выполнения программы (из листинга 13.3)

Укажите размер контейнера: 10
 Укажите индекс элемента: 12
 К сожалению, произошла ошибка.
 Выполнение программы завершено.

В принципе, программа работает достаточно неплохо и успешно "блокирует" проблемные участки кода. Что же нас в ней не устраивает? Не устраивает нас то, что при возникновении *разных* ошибок обрабатываются они *одинаково*. Другими словами, независимо от того, какая возникла ошибка, обработка ошибки выполняется одним и тем же программным блоком. Мы хотим эту ситуацию изменить. Для этого в программе вместо одного `catch`-блока используем два `catch`-блока: по одному `catch`-блоку на каждую из возможных ошибок. Расклад следующий: если возникает ошибка при выделении памяти под контейнер, то обрабатывается она в одном `catch`-блоке, а если ошибка возникнет при попытке присвоить значение элементу, то обрабатывается она в другом `catch`-блоке. Как отмечалось выше, технически все выглядит так, что после `try`-блока не один (как раньше), а два `catch`-блока. Но здесь сразу возникает новый вопрос: а как мы сможем связать ошибку определенного типа с конкретным `catch`-блоком? Чтобы понять механизм данного "связывания", необходимо пополнить багаж теоретических знаний. "Теорию" мы по возможности постараемся упростить, не теряя при этом основного смысла.

Каждая из ошибок относится к определенному *типу*. Если при выполнении программного кода происходит ошибка, автоматически создается объект, ее описывающий. Но мы уже знаем, что объекты создаются на основе классов. Отсюда несложно догадаться, что тип ошибки можно отождествлять с тем классом, на основе которого создается объект, описывающий ошибку данного типа. Примерно так.

**На заметку**

Сформулируем все это иначе. В C++ существует иерархия классов, описывающих ошибки разных типов. При возникновении ошибки на основе класса, которому она соответствует, создается объект. Объект содержит информацию об ошибке.

Создаваемый при возникновении ошибки объект, передается в `catch`-блок для обработки. И вот здесь самое главное: в круглых скобках после инструкции `catch` указывается тип ошибки (точнее, класс), обрабатываемой данным `catch`-блоком.

Подробности

Во всех предыдущих примерах мы использовали один `catch`-блок, а в круглых скобках указывался эллипсис. Это стандартная конструкция, означающая, что данным блоком обрабатываются все исключения, независимо от их типа. На самом же деле `catch`-блок - инструмент "индивидуальный". Он обычно "заточен" под исключения определенного типа.

Пока что с минимальной потерей общего смысла станем отталкиваться от такого шаблона использования `try-catch` конструкции с несколькими `catch`-блоками - их может быть больше двух (жирным шрифтом выделены основные синтаксические элементы):

```
try{
// контролируемый код
}
catch(тип_1) {
// код обработки для ошибок типа_1
}
catch(тип_2) {
// код обработки для ошибок типа_2
}
...
catch(тип_N) {
// код обработки для ошибок типа_N
}
```

Если при исполнении контролируемого кода возникает ошибка, начинается просмотр `catch`-блоков. Ищется совпадение между типом возникшей ошибки и типом ошибки, указанной в `catch`-блоке. Как только совпадение найдено, начинает выполняться программный код соответствующего `catch`-блока. Код прочих `catch`-блоков игнорируется. Если совпадение не найдено, ошибка не перехватывается (остается необработанной).



На заметку

Еще раз подчеркнем, что ключевое слово, указанное в круглых скобках после инструкции `catch`, которое мы отождествляем с типом ошибки, на самом деле обычно является названием класса.

Но вернемся к нашему примеру. При выделении памяти под контейнер может возникнуть ошибка класса `bad_alloc`. При обращении к элементу контейнера с помощью метода `at()` в случае неверно указанного индекса генерируется ошибка, которая может быть перехвачена в `catch`-блоке указанным в нем классом ошибки `out_of_range`.

Подробности

Здесь не так все просто, как кажется на первый взгляд. Но пока что мы в подробности не вдаемся и "перехватываем" ошибку класса `out_of_range`.

Перепишем программный код так, чтобы эти ошибки отлавливались "персонально".

**На заметку**

Для работы с классом `bad_alloc` подключают заголовок `<new>`, а для использования класса `out_of_range` подключают заголовок `<stdexcept>`. Соответственно, в шапке программы появляются инструкции `#include <new>` и `#include <stdexcept>`.

Ошибки класса `bad_alloc` возникают при операциях, связанных с распределением памяти. Что касается ошибок класса `out_of_range`, то здесь речь обычно идет о выходе аргумента или индекса за допустимый диапазон значений. Хотя формально в рассматриваемом примере обсуждается "некорректный индекс", на самом деле речь идет о "некорректном аргументе" метода `at()`.

Обратимся к листингу 13.4, в котором приведен несколько измененный предыдущий пример.

Листинг 13.4. Сортировка ошибок

```
#include <iostream>
#include <vector>
#include <new>
#include <stdexcept>
using namespace std;
int main(){
int n,m; // Размер контейнера и индекс элемента
try{ // Контролируемый код
cout<<"Укажите размер контейнера: ";
cin>>n; // Считывается размер контейнера
// Создание контейнера (значения - нули):
vector<int> p(n,0); // Контейнер
cout<<"Укажите индекс элемента: ";
cin>>m; // Считывается индекс элемента
// Присваивание единичного значения элементу:
p.at(m)=1;
cout<<"Создан контейнер из "<<n<<" элементов:"<<endl;
// Отображение содержимого контейнера:
for(int i=0;i<p.size();i++){
cout<<p.at(i)<<" ";
}
cout<<endl;
} // Завершение try-блока
```

```
catch(bad_alloc){ // Проблемы с выделением памяти
cout<<"Не удалось создать контейнер.\n";
} // Завершение 1-го catch-блока
catch(out_of_range){ // Проблемы с индексом
cout<<"Указан некорректный индекс.\n";
} // Завершение 2-го catch-блока
cout<<"Выполнение программы завершено.\n";
return 0;
}
```

Вместо одного catch-блока с эллипсисом мы использовали два catch-блока с явным указанием типа обрабатываемой ошибки. Первый блок та-
кой:

```
catch(bad_alloc){
cout<<"Не удалось создать контейнер.\n";
}
```

Этот блок выполняется, если сгенерировано исключение класса `bad_alloc` (ошибка при операциях с памятью). Блок для исключения класса `out_of_range` должен отлавливать ошибки, возникающие при несоответствии индекса допустимому диапазону значений. Данный блок выглядит так:

```
catch(out_of_range){
cout<<"Указан некорректный индекс.\n";
}
```

Если при выполнении программы оба параметра (размер контейнера и индекс элемента) указаны корректно, получим следующий результат (здесь и далее жирный шрифт - ввод пользователя):

Результат выполнения программы (из листинга 13.4)

```
Укажите размер контейнера: 10
Укажите индекс элемента: 3
Создан контейнер из 10 элементов:
0 0 0 1 0 0 0 0 0 0
Выполнение программы завершено.
```

Так будет выглядеть результат выполнения программы, если ошибка произошла при создании контейнера:

Результат выполнения программы (из листинга 13.4)

Укажите размер контейнера: -2
 Не удалось создать контейнер.
 Выполнение программы завершено.

Если использован неверный индекс, результат следующий:

Результат выполнения программы (из листинга 13.4)

Укажите размер контейнера: 10
 Укажите индекс элемента: 12
 Указан некорректный индекс.
 Выполнение программы завершено.

Итак, нужный результат достигнут: разные ошибки обрабатываются по-разному. Но на этом возможности системы обработки исключительных ситуаций не исчерпаны.

13.3. Использование объекта исключения

*Живите, как жили - сами клонут.
 из к/ф "Бриллиантовая рука"*

Ранее отмечалось, что при возникновении ошибки создается объект, ее описывающий. Такой *объект ошибки*, или *объект исключения*, можно использовать в программном коде при обработке ошибки. Чтобы понять "механику" процесса, абстрагируемся от реальности и начнем рассматривать `catch`-инструкцию с круглыми скобками как некое подобие вызова функции (хотя это и не совсем корректно). Объект ошибки тогда играл бы роль аргумента функции. Тип ошибки - класс, к которому принадлежит объект ошибки. Ниже приведен фрагмент кода, а в нем в `catch`-блоке явно использован объект ошибки (жирным шрифтом выделены места кода, достойные особого внимания):

```
catch (bad_alloc&e) {
    cout<<"Не удалось создать контейнер.\n";
    cout<<"Ошибка: "<<e.what()<<endl;
}
```

Объект ошибки обозначен через `e`. Амперсанд `&` перед именем объекта исключения имеет тот же смысл, что и при передаче аргументов функциям: объект ошибки передается по ссылке (то есть при передаче объекта ошибки в `catch`-блок передается непосредственно тот объект, что был сгенерирован при возникновении ошибки, а не его копия).



На заметку

В принципе, если амперсанд не указать, то, скорее всего, ничего катастрофического не произойдет. Но все же множить ошибки (пускай даже в виде объектов) на ровном месте как-то нехорошо.

Поскольку в `catch`-блоке объект ошибки явно обозначен, его можно использовать в программном коде обработки ошибки. У всех объектов, описывающих исключения, есть метод `what()`, возвращающий результатом текст с краткой информацией о сути возникшей ошибки. Оттого команда `cout<<"Ошибка: "<<e.what()<<endl` имеет смысл.

Подробности

Классы, описывающие исключительные ситуации, образуют определенную иерархию. Иерархия базируется на наследовании. В вершине этой иерархии находится класс `exception` (для работы с этим классом подключается заголовок `<exception>`). У класса `exception` есть метод `what()`, который наследуется производными классами. Поэтому и у объектов класса `bad_alloc`, и у объектов класса `out_of_range` (равно как и у объектов других классов исключений) есть метод `what()`.

Аналогичные изменения претерпел и блок кода для обработки ошибки, связанной с некорректно заданным индексом:

```
catch(out_of_range&e) {
    cout<<"Указаннекорректныйиндекс.\n";
    cout<<"Ошибка: "<<e.what()<<endl;
}
```

Весь программный код, с новыми версиями блоков обработки исключительных ситуаций (и с удаленными комментариями) приведен в листинге 13.5.

Листинг 13.5. Использование объекта исключения

```
#include<iostream>
#include <vector>
#include <new>
#include <stdexcept>
using namespace std;
int main() {
    int n,m;
    try{
        cout<<"Укажите размер контейнера: ";
        cin>>n;
        vector<int> p(n,0);
        cout<<"Укажите индекс элемента: ";
        cin>>m;
        p.at(m)=1;
```

```

cout<<"Создан контейнер из "<<n<<" элементов:"<<endl;
for(int i=0;i<p.size();i++){
cout<<p.at(i)<<" ";
}
cout<<endl;
}
catch(bad_alloc&e){
cout<<"Не удалось создать контейнер.\n";
cout<<"Ошибка: "<<e.what()<<endl;
}
catch(out_of_range&e){
cout<<"Указан некорректный индекс.\n";
cout<<"Ошибка: "<<e.what()<<endl;
}
cout<<"Выполнение программы завершено.\n";
return 0;
}

```

Если ошибок при выполнении программы не возникает, то результат выполнения программы не отличается от рассмотренного ранее случая. А вот при возникновении ошибок результат выполнения программы меняется (правда, незначительно). Если возникает ошибка при создании контейнера, результат следующий:

Результат выполнения программы (из листинга 13.5)

```

Укажите размер контейнера: -2
Не удалось создать контейнер.
Ошибка: std::bad_alloc
Выполнение программы завершено.

```

Ниже приведены сообщения, появляющиеся в окне вывода, если программа генерирует ошибку при обращении к элементу контейнера:

Результат выполнения программы (из листинга 13.5)

```

Укажите размер контейнера: 10
Укажите индекс элемента: 12
Указан некорректный индекс.
Ошибка: vector::_M_range_check
Выполнение программы завершено.

```

В принципе информация об ошибке, получаемая с помощью метода `what()`, ограничивается в основном названием класса ошибки.

Подробности

Теперь о замечании, в котором утверждалось, что с классом `out_of_range` "не все так просто". Есть одно очень важное свойство, которое мы еще будем обсуждать. Состоит оно в том, что в `catch`-блоке перехватываются ошибки не только класса, непосредственно указанного в блоке, но и ошибки всех производных классов. Поэтому, например, если в `catch`-блоке указать в качестве типа ошибки имя класса `exception`, то данным блоком перехватываются ошибки всех типов. Причина в том, что класс `exception` находится в вершине иерархии наследования классов исключений.

Как следствие вполне может оказаться, что мы перехватывали одно исключение, а перехватили другое. Правда, класс этого другого исключения должен быть подклассом класса, указанного в блоке обработки. Например, мы отлавливали ошибку класса `out_of_range`, а метод `what()` объекта "выловленной" ошибки дает нам текст `vector::M_range_check`. Возникает подозрение, что здесь имеем дело с подклассом класса `out_of_range`.

13.4. Генерирование исключений

- Нет у меня никакого приступа!

- Будет!

из к/ф "Покровские ворота"

Существует возможность не только отлавливать стихийно возникающие исключения, но и собственными руками *генерировать* их. На первый взгляд наличие данного механизма может показаться полным абсурдом, а на самом деле речь идет об очень полезной штуке.



На заметку

Как минимум это позволяет легко изменять логику выполнения программы и существенно расширяет горизонты относительно организации структуры программы.

С прикладной точки зрения сгенерировать исключение исключительно просто. Для этого после инструкции `throw` следует указать какую-нибудь переменную или объект. Такая маленькая и скромная команда приводит к серьезным последствиям: выполнение кода останавливается и управление передается блоку обработки исключительной ситуации. Тип переменной или класс объекта после инструкции `throw` отождествляется с типом ошибки. Сама переменная (объект) отождествляется с объектом ошибки. В принципе это все. Дальше имеет смысл сразу перейти к примерам.



На заметку

Можно создавать собственные классы исключений. Обычно эти классы создаются наследованием класса `exception`. Мы обсудим это немного позже.

В листинге 13.6 приведен небольшой пример программного кода, в котором исключительные ситуации генерируются "искусственно", с помощью инструкции `throw`.

Листинг 13.6. Генерирование исключительных ситуаций

```
#include <iostream>
#include <cstdlib>
#include <string>
using namespace std;
// Класс для создания объекта ошибки:
class MyError{
private:
    // Целочисленное поле:
    int number;
    // Текстовое поле:
    string text;
public:
    // Конструктор:
    MyError(string txt,int num){
        text=txt;
        number=num;
    }
    // Метод для отображения информации
    // об объекте:
    void show(){
        cout<<text;
        cout<<number<<endl;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Инициализация генератора случайных чисел:
    srand(2015);
    // Количество циклов:
    int Nmax=20;
    // Текстовая переменная:
    string t="Кратное трем число: ";
    // Целочисленная переменная:
    int x;
    cout<<"Начало выполнения программы.\n";
    // Оператор цикла:
    for(int i=1;i<=Nmax;i++){
        // Случайное число от 1 до 7 включительно:
        x=rand()%7+1;
        try{ // Контролируемый код:
```

```

        // Генерирование ошибки (класс string):
if(x==2||x==4) throw string("это 2 или 4");
// Генерирование ошибки (тип int):
if(x==5) throw x;
// Генерирование ошибки (класс string):
if(x==7) throw string("семерка");
// Генерирование ошибки (класс MyError):
if(x%3==0) throw MyError(t,x);
    }
    // Обработка ошибки (класс MyError):
catch(MyError&e){
e.show();
}
// Обработка ошибки (класс string):
catch(string &e){
cout<<"Число: "<<e<<endl;
}
// Обработка ошибки (тип int):
catch(int &e){
cout<<"Целое число: "<<e<<endl;
}
}
cout<<"Программа завершила работу.\n";
return 0;
}

```

При выполнении программы получаем такой результат (с поправками на особенности генератора случайных чисел):

Результат выполнения программы (из листинга 13.6)

Начало выполнения программы.

Число: это 2 или 4

Число: семерка

Кратное трем число: 3

Целое число: 5

Кратное трем число: 3

Число: это 2 или 4

Число: семерка

Целое число: 5

Кратное трем число: 3

Кратное трем число: 3

Число: это 2 или 4

Число: это 2 или 4

Число: это 2 или 4

Кратное трем число: 6

Кратное трем число: 6
 Программа завершила работу.

В программе описывается класс `MyError` (объекты этого класса будут "генерироваться" как ошибки - но о создании пользовательских классов ошибок речь пока не идет). В классе описано текстовое поле `text` (объект класса `string`) и целочисленное поле `number`. Еще в классе есть конструктор с двумя аргументами (текст и целое число - значения полей создаваемого объекта), а также метод `show()`, с помощью которого отображается информация об объекте (фактически, отображаются значения полей объекта).

При выполнении программы запускается оператор цикла. Контролируемый в `try`-блоке код и `catch`-блоки обработки исключений находятся внутри оператора цикла. Поэтому генерирование исключений и их обработка происходят на каждом цикле при выполнении оператора цикла.



На заметку

Если на каком-то цикле генерируется исключение, то выполнение цикла приостанавливается и начинается попытка обработать исключение. В случае, когда исключение обработано в том же самом цикле, в котором оно сгенерировано, далее выполняется первая команда после блока обработки исключений того же самого цикла. Если в цикле больше команд нет, начинает выполняться новый цикл.

Резюме следующее: в случае, когда исключения генерируются и обрабатываются в рамках одного цикла, оператор цикла продолжает свою работу.

За каждый цикл целочисленной переменной `x` с помощью генератора случайных чисел присваивается в качестве значения случайное целое число в диапазоне значений от 1 до 7 включительно. Затем идет несколько условных операторов однотипной структуры: проверяется определенное условие и при его истинности с помощью инструкции `throw` генерируется исключение. Возможны такие варианты:

- Если выполнено условие `x==2 || x==4` (значение переменной `x` равно 2 или 4), инструкцией `throw string("это 2 или 4")` генерируется исключение с объектом исключения класса `string`. Здесь мы встречаем выражение `string("это 2 или 4")`. Это есть ни что иное, как вызов конструктора класса `string` с аргументом "это 2 или 4". В результате создается объект класса `string` с текстовым значением "это 2 или 4". Формально такой объект не записывается ни в какую переменную (*анонимный объект*). Но в данном случае это и не нужно. Объект (как бы без имени) передается для обработки в соответствующий `catch`-блок (а там для объекта исключения указано название). Сразу отметим, что в общем

случае для объектов генерируемых исключений нет необходимости быть анонимными.

- Если значение переменной `x` равно 5 (истинно условие `x==5`), исключение генерируется командой `throw x`. Поскольку переменная относится к типу `int`, то и перехватывается исключение в `catch`-блоке для типа `int`. Значение объекта исключения в данном случае - это текущее значение переменной `x` (значение переменной на момент, когда генерировалось исключение).
- Для значения переменной `x`, равного 7 (условие `x==7`), исключение генерируется инструкцией `throw string("семерка")`. В роли объекта исключения выступает анонимный объект класса `string` со значением "семерка". Похожую ситуацию мы уже рассматривали. Здесь важно то, что генерируется еще одно исключение с объектом класса `string`, и обрабатывается данное исключение в том же `catch`-блоке, что и исключение, генерируемое при значениях переменной `x` равных 2 или 4.
- Наконец, в случае, когда значение переменной `x` делится на 3 (остаток от деления равен нулю - соответственно, истинно условие `x%3==0`), исключение генерируется инструкцией `throw MyError(t, x)`. Выражение `MyError(t, x)` представляет собой команду вызова конструктора класса `MyError` с первым текстовым аргументом `t` (которому предварительно присвоено значение "Кратное трем число: ") и вторым целочисленным аргументом `x` (текущее значение данной переменной). Исключение перехватывается в `catch`-блоке, в котором типом исключения указано название класса `MyError`.

Блоков обработки исключений всего три. В каждом из них объект исключения передается по ссылке (наличие амперсанда `&`) и называется одинаково - объект исключения обозначен переменной `e`. Тип (класс) переменной `e` в каждом блоке разный, и вообще, следует понимать, что переменная `e` является "локальной" - существует и доступна только в том блоке, где она объявлена. Поэтому переменные с одинаковым названием `e` в разных блоках на самом деле являются разными переменными.

Подробности

Переменная `e` в каком-то конкретном `catch`-блоке - это фактически объект, указанный после инструкции `throw`, генерирующей ошибку, обрабатываемую в данном `catch`-блоке.

В зависимости от типа сгенерированного исключения обработка происходит следующим образом:

- Для исключения на основе объекта класса `MyError` из объекта исключения `e` командой `e.show()` вызывается метод `show()`. Здесь мы использовали то обстоятельство, что под `e` в данном `catch`-блоке подразумевается объект класса `MyError`, а у объектов класса `MyError` есть метод `show()`.
- В `catch`-блоке для обработки исключений на основе объектов класса `string` просто отображается текстовое значение объекта `e` - ведь в данном случае это текст.
- В `catch`-блоке, предназначенном для обработки исключений типа `int`, также отображается значение переменной `e` (только теперь это целое число).

При генерировании исключительных ситуаций "вручную" совсем не обязательно использовать не всегда подходящие для такого дела объекты и переменные "неисключительного" характера. Генерировать ошибки можно и на основе стандартных классов ошибок. Далее рассматривается небольшой пример, немного напоминающий предыдущий, но только теперь генерирование исключительных ситуаций базируется на создании объектов классов `exception`, `logic_error` и `length_error`.

Подробности

Для использования класса `exception` подключается заголовок `<exception>`, а для использования классов `logic_error` и `length_error` подключается заголовок `<stdexcept>`.

Класс `exception` мы ранее уже упоминали как класс, находящийся в вершине иерархии классов исключений.

Класс `logic_error` является базовым классом для классов исключений, описывающих логические ошибки, связанные со способом организации программы. Грубо говоря, это ошибки, связанные с тем, что формально правильный (и поэтому компилируемый без ошибок) код организован неправильно с точки зрения реализации общего алгоритма. Среди подклассов класса `logic_error` есть:

- класс `invalid_argument` (ошибки, связанные с некорректностью аргумента);
- класс `domain_error` (ошибки, возникающие вследствие несоответствия параметров выделенной области памяти для выполнения операции);
- класс `length_error` (ошибки превышения допустимых размеров объекта);
- класс `out_of_range` (ошибки, возникающие при попытке доступа к элементу вне допустимого диапазона);
- класс `future_error` (ошибки, связанные с асинхронностью выполнения потоков).

Сам класс `logic_error` является подклассом класса `exception`.

К классу `length_error` относятся исключения, возникающие при попытке создать объект слишком большой для его размещения в выделенной области памяти.

В данном конкретном примере для нас не очень важно, какие ошибки относятся к классам `extension`, `length_error` и `logic_error`. Мы просто сгенерируем ошибки соответствующих классов.

В представленной далее программе выполняется оператор цикла, в котором за каждый цикл генерируется случайное число в диапазоне значений от 0 до 9. Затем проверяется полученное случайное значение и в зависимости от этого значения может генерироваться одна из трех ошибок:

- Если случайное число нулевое, генерируется исключение класса `exception`.
- Если случайное число больше 7, генерируется исключение класса `length_error`.
- Если случайное число равняется 5, генерируется исключение класса `logic_error`.

При создании объектов классов `length_error` и `logic_error` аргументам конструкторов передаются текстовые аргументы. Этот текст возвращается впоследствии при обработке ошибки, когда из объекта ошибки вызывается метод `what()`. Рассмотрим программный код, представленный в листинге 13.7.

Листинг 13.7. Название

```
#include<iostream>
#include <exception>
#include <stdexcept>
#include <cstdlib>
using namespace std;
int main() {
    cout<<"Начинается выполнение программы.\n";
    // Инициализация генератора случайных чисел:
    srand(2014);
    // Целочисленные переменные:
    int a, Nmax=15;
    // Оператор цикла:
    for(int i=1; i<Nmax; i++){
        // Случайное число от 0 до 9:
        a=rand()%10;
        try{ // Контролируемый код
            // Исключение класса exception:
```

```

if(a==0) throw exception();
// Исключение класса length_error:
if(a>7) throw length_error("слишкоммного");
// Исключение класса logic_error:
if(a==5) throw logic_error("значение 5");
}
// Обработка исключений класса length_error:
catch(length_error &e){
cout<<"Выход за диапазон: ";
cout<<e.what()<<endl;
}
// Обработкаисключенийклассалogic_error:
catch(logic_error &e){
cout<<"Логическаяошибка: ";
cout<<e.what()<<endl;
}
// Обработкаисключенийклассaeexception:
catch(exception &e){
cout<<"Нулевоезначение: ";
cout<<e.what()<<endl;
}
}
cout<<"Выполнение программы завершено.\n";
return 0;
}

```

Результат выполнения программы может быть, например, таким:

Результат выполнения программы (из листинга 13.7)

```

Начинается выполнение программы.
Логическая ошибка: значение 5
Нулевое значение: std::exception
Логическая ошибка: значение 5
Выход за диапазон: слишком много
Нулевое значение: std::exception
Выход за диапазон: слишком много
Выполнение программы завершено.

```

Фактически, мы в данном примере самостоятельно сгенерировали "стандартные" ошибка - правда, добавив к объектам некоторых ошибок свое описание. Ситуация напоминает ту, когда объект ошибки создавался на основе класса, описанного пользователем. Однако здесь есть одно важное обстоятельство: классы, на основе которых создаются объекты исключений, связаны между собой цепочкой наследования: класс `logic_error` является производным от класса `exception`, а класс `length_error` является

производным от класса `logic_error`. Поэтому блоки перехвата исключений должны располагаться именно в том порядке, как показано в примере. Объяснение простое: в `catch`-блоках на самом деле перехватываются не только ошибки явно указанного в описании блока класса, но и ошибки производных классов. Следовательно, если блок для обработки ошибки класса `logic_error` окажется размещенным до блока, обрабатывающего ошибки класса `length_error`, последний вообще окажется "не у дел". Ведь блок, описанный для перехвата ошибок класса `logic_error`, перехватывает и ошибки класса `length_error`. Причина как раз в том, что класс `length_error` является производным от класса `logic_error`. Ну а если разместить первым блок для обработки ошибок класса `exception`, то он вообще будет перехватывать все ошибки.

13.5. Подклассы ошибок

*История, леденящая кровь. Под маской овцы
скрывался лев!*

из к/ф "Покровские ворота"

Итак, в `catch`-блоках перехватываются ошибки того класса, который указан в описании `catch`-блока, а также ошибки, относящиеся к его подклассам. Как небольшая иллюстрация к сказанному - программный код в листинге 13.8. Традиционно рассматриваемая далее программа является модификацией предыдущего примера.

Листинг 13.8. Подклассы ошибок

```
#include <iostream>
#include <exception>
#include <stdexcept>
#include <cstdlib>
#include <string>
using namespace std;
// Пользовательский класс ошибки:
class my_error{
private:
    // Текстовое поле:
    string txt;
public:
    // Конструктор:
    my_error(string txt){
        this->txt=txt;
    }
    // Метод описания ошибки:
    string what(){
```

```

return txt;
    }
}; // Окончание описания класса
// Главный метод программы:
int main() {
    cout<<"Начинается выполнение программы.\n";
    // Инициализация генератора случайных чисел:
    srand(111);
    // Целочисленные переменные:
    int a, Nmax=20;
    // Операторцикла:
    for(int i=1; i<Nmax; i++){
        // Случайное число от 0 до 9:
        a=rand()%10;
        try{ // Контролируемый код
            // Исключениеклассаexception:
            if(a==0) throw exception();
                // Исключениеклассаmy_error:
            if(a==1) throw my_error("Ошибкаmy_error");
            // Исключениеклассалength_error:
            if(a>7) throw length_error("Ошибкаlength_error");
            // Исключениеклассалогic_error:
            if(a==5) throw logic_error("Ошибкаlogic_error");
        }
        // Обработка исключений класса exception
        //и его подклассов:
        catch(exception &e){
            cout<<"Первый блок. ";
            cout<<e.what()<<endl;
        }
        // Обработка исключений класса my_error:
        catch(my_error &e){
            cout<<"Второй блок. ";
            cout<<e.what()<<endl;
        }
    }
    cout<<"Выполнение программы завершено.\n";
    return 0;
}

```

Результат выполнения программы может быть таким:

Результат выполнения программы (из листинга 13.8)

Начинается выполнение программы.
 Второй блок. Ошибкаmy_error
 Первыйблок. std::exception

```
Первый блок. std::exception
Первый блок. Ошибка length_error
Первый блок. Ошибка length_error
Первый блок. Ошибка logic_error
Второй блок. Ошибка my_error
Первый блок. Ошибка length_error
Выполнение программы завершено.
```

В этом примере описывается пользовательский класс `my_error` с конструктором и методом `what()`. Конструктору при создании передается текстовый аргумент. Аналогичное текстовое значение возвращается методом `what()`. В программу (в блок контролируемого кода) добавлен условный оператор и в случае, если вычисленное случайное число оказалось единичным, генерируется исключение класса `my_error`.

Таким образом, в программе генерируются ошибки классов `exception`, `length_error`, `logic_error` и `my_error`. При этом для обработки ошибок используется всего два `catch`-блока: в первом по очередности блоке перехватываются исключения класса `exception`, а во втором блоке перехватываются исключения класса `my_error`. Важных моментов два:

- В первом блоке на самом деле обрабатываются исключения классов `exception`, `logic_error` и `length_error` (поскольку классы `logic_error` и `length_error` являются подклассами класса `exception` -или прямо, как класс `logic_error`, или опосредованно, как класс `length_error`).
- Исключения класса `my_error` в первом блоке не перехватываются (поскольку данный класс не является подклассом класса `exception`) и обрабатываются во втором блоке.

13.6. Пользовательские классы исключений

*В прах разметал домашний очаг - одни руины.
из к/ф "Покровские ворота"*

Ранее мы уже предпринимали робкие попытки описывать классы для создания на их основе объектов исключений. В принципе подход был неплохой. Тем не менее, есть два момента, которые мы попробуем "зафиксировать". Во-первых, мы сделаем класс ошибки производным от класса `exception`. Во-вторых, мы переопределим метод `what()`, наследуемый из класса `exception`.

Подробности

В принципе, пользовательский класс ошибки может наследовать какой-то другой класс исключения. Напомним, что в вершине иерархии стандартных классов исключений находится класс `exception`. Прочие классы исключений являются производными от этого класса. Например, производными от класса `exception` являются классы `bad_alloc` (ошибка при операциях с памятью) и `bad_cast` (ошибки, связанные с обработкой типов, для использования класса подключается заголовок `<typeinfo>`).

Есть две группы исключений: логические и ошибки времени исполнения. Классы логических исключений мы уже упоминали выше: `invalid_argument` (некорректный аргумент), `domain_error` (проблемы с размещением), `length_error` (некорректный размер), `out_of_range` (некорректный диапазон), `future_error` (проблемы с синхронностью). Эти классы являются производными от класса `logic_error`, который является производным от класса `exception`. Логические ошибки связаны с проблемами в структуре программы. Образно выражаясь, подобные ошибки появляются в "плохо продуманных" программах.

Другая группа ошибок - ошибки времени выполнения. Эти ошибки возникают в процессе выполнения программы и не могут быть "выявлены" до ее запуска.

Класс `runtime_error` является производным от класса `exception` и он же является базовым для классов: `range_error` (некорректный диапазон), `overflow_error` (ошибка переполнения), `underflow_error` (ошибка потери точности), `system_error` (системная ошибка).

Классы (из перечисленных выше) как логических ошибок, так и ошибок времени исполнения доступны при подключении заголовка `<stdexcept>`.

Что касается метода `what()`, то в предыдущем примере мы метод с таким именем в пользовательском классе исключения описывали. Правда, там метод возвращал в качестве результата объект класса `string`. А это не совсем отвечает прототипу метода `what()`, описанному в классе `exception`.

Подробности

Метод `what()` в качестве результата возвращает текст с описанием ошибки. Но дело в том, что есть два способа представления текста: через объект класса `string` и в виде символьного массива. Причем второй путь - основной. Методом `what()` возвращается именно такой текст - текст, соответствующий типу `char*`. Но это не все.

Метод `what()` объявлен как возвращающий постоянное значение (постоянный текст). Отсюда ключевое слово `const` в начале прототипа метода. Но и это не все.

Метод не изменяет объект (константный метод). Поэтому после названия метода и пустых круглых скобок в прототипе метода указывается еще один идентификатор `const`.

Вот здесь многие удивятся, но и это еще не все. Метод `what()` при вызове не генерирует исключений. Данный чудесный факт отображается посредством ключевого слова `noexcept` в прототипе метода (ключевое слово `noexcept` - новшество в стандарте C++11).

Следовательно, шаблон для описания метода `what()` выглядит следующим образом:

```
const char* what() const noexcept{
    // код метода
}
```

В принципе, можно описать метод так, как мы делали раньше (или не описывать его совсем). Ничего страшного не произойдет. Но в некоторых случаях все же разница будет заметна. К этому моменту мы еще вернемся.

Итак, в листинге 13.9 представлен программный код с описанием двух классов для ошибок пользовательского типа. Оба класса наследуют класс `exception`. В каждом классе переопределяется метод `what()`. Первый класс называется `DivZero` и предназначен работы с ошибкой "деление на ноль". У конструктора класса нет аргументов.

Класс `my_error` никак конкретно не специфицирован в плане типа ошибки, для которой создается объект этого класса. У класса есть поле, представляющее собой символьный массив, а конструктору при создании объекта передается текстовый аргумент.

Также в программе описана функция `getNumber()`, не имеющая аргументов и возвращающая (иногда) в качестве результата целое число. Функция предназначена для считывания целочисленного значения, вводимого пользователем с клавиатуры. При выполнении кода функции выводится запрос на ввод пользователем целого числа. После считывания значения оно проверяется: если значение отрицательное (меньше 0) или превышает 5, генерируется исключение класса `my_error` (два разных объекта класса `my_error` с разными текстовыми полями). Если же со считанным значением все в порядке, оно возвращается результатом функции.

Небольшой сюрприз ожидает нас в главной функции программы. Там `try-catch` конструкция заключена во внешний `try`-блок. Для этого `try`-блока указан один `catch`-блок с эллипсисом. Поэтому если какое-то исключение не обработано во внутренней `try-catch` конструкции, данное исключение обрабатывается во внешнем `catch`-блоке с эллипсисом.

При выполнении программы во внутреннем `try`-блоке делается попытка считать числовое значение. Если значение считано без генерирования исключения класса `my_error` в функции `getNumber()`, оно проверяется на предмет равенства нулю. Если число равно нулю, генерируется исключение класса `DivZero`. В противном случае считанное число используется как знаменатель при вычислении частного, а результат вычислений отображается в окне вывода. Теперь рассмотрим программный код:

Листинг 13.9. Пользовательский класс исключения

```

#include <iostream>
#include <exception>
#include <cstring>
using namespace std;
// Класс ошибки "деления на ноль":
class DivZero: public exception{
public:
// Конструктор:
DivZero():exception(){}
// Переопределение метода what():
const char* what() const noexcept{
// Результат метода:
return "Деление на ноль";
}
}; // Окончание класса ошибки
// Класс ошибки:
class my_error: public exception{
private:
// Текстовое поле:
char txt[100];
public:
// Конструктор:
my_error(char* str):exception(){
// Копирование строк:
strcpy(txt, str);
}
// Переопределение метода what():
const char* what() const noexcept{
// Результат метода:
return txt;
}
}; // Окончание класса
// Функция для считывания числа:
int getNumber(){
// Целое число:
int n;
// Текстовые значения:
char str_neg[100]="Ошибка: число меньше 0";
char str_pos[100]="Ошибка: число больше 5";
cout<<"Введите число: ";
cin>>n; // Считывание целого числа
// Проверка условия для генерирования ошибки:
if(n<0) throw my_error(str_neg);
// Проверка условия для генерирования ошибки:

```

```

if(n>5) throw my_error(str_pos);
// Результат метода:
return n;
}
// Главная функция программы:
int main(){
cout<<"Начинается выполнение программы.\n";
    // Целочисленные переменные:
int a=120,b;
    // Внешний контролируемый блок кода:
try{
    // Внутренний контролируемый блок кода:
    try{
        // Попытка считывания числа:
b=getNumber();
        // Генерирование ошибки при нулевом значении:
if(b==0) throwDivZero();
        // Результат деления чисел:
cout<<a<<"/"<<b<<"="<<a/b<<endl;
    }
        // Первый внутренний блок обработки ошибки DivZero:
catch(DivZero&e){
cout<<e.what()<<endl;
    }
        // Второй внутренний блок обработки ошибки exception:
catch(exception &e){
cout<<e.what()<<endl;
// Повторное генерирование исключения:
throw;
    }
}
// Внешний блок обработки ошибок:
catch(...){
cout<<"Число должно быть от 1 до 5\n";
}
cout<<"Выполнение программы завершено.\n";
return 0;
}

```

Таким образом, при выполнении команд во внутреннем `try`-блоке могут возникнуть ошибки двух типов: ошибка класса `my_error` в теле функции `getNumber()` и ошибка класса `DivZero`. Для перехвата этих ошибок предусмотрены два `catch`-блока. Один персонифицирован для ошибок класса `DivZero`, а во втором блоке типом ошибки указан класс `exception`. Но поскольку класс `my_error` является производным от класса `exception`, то данным блоком перехватываются и ошибки класса `my_error`.

**На заметку**

Данный блок перехватывал бы и ошибки класса `DivZero` (класс `DivZero` является производным от класса `exception`), но ошибки такого типа "отлавливаются" в первом `catch`-блоке, так что до второго блока они просто "не доходят".

Во втором `catch`-блоке есть инструкция `throw`. При выполнении этой инструкции происходит *повторное генерирование исключения*. Другими словами, второй `catch`-блок сам создает ошибку. Эту ошибку перехватывает внешний `catch`-блок с эллипсисом.

В зависимости от вводимого пользователем значения (введенное число меньше нуля, больше нуля, равно нулю и попадает в диапазон от 1 до 5), могут быть следующие результаты выполнения программы (жирным шрифтом выделено введенное пользователем значение) -если введенное целое число попадает в диапазон от 1 до 5:

Результат выполнения программы (из листинга 13.9)

Начинается выполнение программы.
Введите число: **3**
120/3=40
Выполнение программы завершено.

Если введенное число меньше нуля:

Результат выполнения программы (из листинга 13.9)

Начинается выполнение программы.
Введите число: **-2**
Ошибка: число меньше 0
Число должно быть от 1 до 5
Выполнение программы завершено.

Если введенное число больше 5:

Результат выполнения программы (из листинга 13.9)

Начинается выполнение программы.
Введите число: **7**
Ошибка: число больше 5
Число должно быть от 1 до 5
Выполнение программы завершено.

Если введенное число равно нулю:

Результат выполнения программы (из листинга 13.9)

Начинается выполнение программы.
Введите число: 0
Деление на ноль
Выполнение программы завершено.

В завершение хочется прокомментировать способ определения метода `what()` в классе `my_error`. Во-первых, поскольку в теле метода происходит копирование текстовых строк, реализованных в виде символьных массивов, для выполнения копирования применяется библиотечная функция `strcpy()`. Первый аргумент функции - строка (массив), в которую выполняется копирование, а второй аргумент - копируемая строка. Для использования функции подключался заголовок `<cstring>`.

Во-вторых, если не определить метод `what()` с таким же прототипом, как в классе `exception`, то поскольку в программе исключение класса `my_error` перехватывается в блоке для исключения класса `exception`, будет вызвана версия метода `what()` из класса `exception`.



На заметку

Чтобы понять причину, рекомендуется освежить в памяти особенности вызова методов из объектов производных классов, записанных в переменную базового класса.

Понятно, что в данном случае ситуация создана искусственно, но все же подобные моменты следует иметь в виду при составлении программ с привлечением системы обработки исключительных ситуаций.

Глава 14.

МНОГОПОТОЧНОЕ ПРОГРАММИРОВАНИЕ



Объясните, наконец: почему ваша невеста придет искать вас у меня дома?

из к/ф "Ирония судьбы или с легким паром"

Эта глава посвящена *многопоточному программированию*. Потоки - это разные части программы, которые могут выполняться одновременно. Ранее мы имели дело только с одним потоком: был какой-то блок или фрагмент программного кода, и он выполнялся - команда за командой. Теперь представим себе несколько фрагментов программного кода, тоже выполняемые команда за командой, но одновременно (или почти одновременно).



На заметку

На самом деле обычно одновременно потоки не выполняются, поскольку это довольно сложно реализовать технически - особенно если у компьютера один процессор. Но зато можно создать иллюзию таких "параллельных" вычислений - например, если процессор периодически (очень быстро) переключается на выполнение разных потоков. Как бы там ни было, нас техническая сторона вопроса не интересует, и мы намерены твердо придерживаться версии об одновременном выполнении потоков.

В стандарте C++11 появились простые и удобные средства для реализации многопоточкового подхода при составлении программных кодов. Их и обсудим.



На заметку

Примеры из данной главы тестировались в среде разработки Microsoft Visual Studio Express 2013. Также следует иметь в виду, что рассматриваемые далее утилиты достаточно новы и пока еще поддерживаются не всеми компиляторами.

Подробности

При работе со средой Microsoft Visual Studio Express 2013 читателю, по всей видимости, придется столкнуться с консольным окном. А работа с консольным окном чревата двумя проблемами. Первая связана с тем, что как только результат выполнения программы в консольное окно выведен, последнее, скорее всего, сразу закроется. И это "сразу" наступает очень быстро. Настолько быстро, что заметить, что же там, в окне, отображалось, с большой вероятностью не удастся. То есть с этим нужно что-то делать. Ситуация усугубляется тем, что указанное поведение консольного окна - в общем-то проблема не связанная с программированием на C++. Другими словами, C++ - это C++, а консольное окно - это консольное окно.

По-хорошему, вопрос надо было бы решать на уровне выполнения настроек консольного окна. С другой стороны, книга о C++ и никак уж не о консольном окне. То есть посвящать раздел книги вопросу выполнения настроек консоли не очень разумно. А с третьей стороны, совсем проигнорировать данную ситуацию - значит поставить для кого-то (кто сам не сможет разобраться с этим вопросом) шлагбаум на пути изучения C++. Мы, как всегда, постараемся найти "золотую середину".

Вторая проблема связана с использованием в командах ввода и вывода кириллического текста. Нет никаких проблем с тем, чтобы использовать текстовые литералы в редакторе кодов. Но вот когда программа компилируется и запускается на выполнение, в консольном окне вместо разумного текста появляются непонятные символы (подчеркнем, что замечание касается только кириллического текста). То есть в редакторе кода все выглядит нормально - а при выводе в консоль все выглядит ужасно.

В чем проблема? Проблема в том, что в редакторе кодов и в консольном окне используется разная кодировка (в редакторах обычно используется кодировка cp 1251, а в консольном окне кодировка cp 866). С одной стороны, можно сохранять файлы в той же кодировке, что и кодировка консольного окна. Для этого в окне среды VS Express следует в меню **Файл** выбрать команду **Дополнительные параметры сохранения** и выбрать в списке Кодировка элемент Кириллица (DOS) - кодовая страница 866. Но на самом деле такой подход не наилучший (хотя и не самый плохой).

Обе проблемы мы будем решать в рамках одного подхода. Воспользуемся функцией `system()` (используется при подключенном заголовке `<cstdlib>`). С помощью команды `system("chcp 1251")` "заставим" консоль использовать кодировку cp 1251. Команда `system("PAUSE")`, размещенная в конце программы (но перед инструкцией `return 0`), позволяет "удержать" консольное окно от закрытия.

14.1. Знакомство с потоками

Я могу пронизать пространство и уйти в прошлое.

из к/ф "Иван Васильевич меняет профессию"

Чтобы понять принципы реализации потокового подхода, постараемся конкретизировать ситуацию. Итак, традиционно у нас есть программа, которую мы отождествляем с функцией `main()`. Отождествляем в том смысле, что выполнение программы - это выполнение кода данной функции. Здесь ничего не меняется. Просто теперь в процессе выполнения главной функции мы можем запустить *поток* или *потоки*. Что происходит при запуске потока? Если в общих чертах, то программа (функция `main()`) продолжает выполняться, как и выполнялась ранее, но при этом начинается процесс

одновременного (с выполнением кода главной функции) выполнения еще одного фрагмента программного кода. Важно здесь то, что выполнение разных фрагментов кода (имеется в виду код главной функции и код потока) происходит *одновременно*.



На заметку

То есть в любом случае мы начинаем с выполнения главной функции программы, а уже после ее запуска возможен запуск параллельных процессов. По этой причине процесс выполнения главной функции нередко называют главным потоком программы, а потоки, запускаемые из главного потока, называются дочерними.

Далее рассмотрим две ситуации. Сначала предлагается самая обычная программа с самой обычной функцией, и эта самая обычная функция вызывается в главной функции программы. Здесь речь о потоках не идет.

Затем все будет практически так же: в программе описывается функция, и она запускается в главной функции программы. Но только в этом случае код вызываемой функции выполняется одновременно с кодом главной функции. Другими словами, мы "превратим" код обычной функции в поток (не буквально, конечно).

В листинге 14.1 приведен пример программы, в которой *нет потоков*. В программе описана функция `say()` без аргументов и не возвращающая результат. Функцией при вызове в окне вывода отображается несколько сообщений. Функция `say()` вызывается в главной функции `main()`. Также главной функцией отображается несколько сообщений. Легко убедиться, что представленная ниже программа очень простая:

Листинг 14.1. Программа без создания потоков

```
#include <iostream>
#include <cstdlib>
using namespace std;
// Функция для вызова в программе:
void say() {
    cout<<"Функция: запуск.\n";
    cout<<"Функция: в процессе выполнения.\n";
    cout<<"Функция: завершение.\n";
}
// Главная функция программы:
int main() {
    // Установка кодировки для консольного окна:
    system("chcp 1251");
    cout<<"Программа: запущена.\n";
    // Вызов функции say() в теле программы:
```

```
say();
cout<< "Программа: функциявызвана.\n";
cout<< "Программа: завершена.\n";
// Командадлязадержки консольного окна
// (спецификасредыVS Express):
system("PAUSE");
return 0;
}
```

Результат выполнения программы тоже легко предугадать:

Результат выполнения программы (из листинга 14.1)

Программа: запущена.
 Функция: запуск.
 Функция: в процессе выполнения.
 Функция: завершение.
 Программа: функция вызвана.
 Программа: завершена.

Подробности

Программный код начинается командой `system("chcp 1251")`. В результате выполнения команды в консольном окне используется кодировка `cp 1251` (такая же, как в редакторе кода). У команды `system("chcp 1251")` есть "побочный эффект": при ее выполнении отображается сообщение `Текущая кодовая страница: 1251`. Чтобы это сообщение не появлялось, рекомендуется использовать команду `system("chcp 1251>nul")`.

Перед инструкцией `return 0` размещена команда `system("PAUSE")`. Команда нужна для того, чтобы задержать на экране консольное окно, которое в противном случае сразу закрывается после завершения выполнения программы. При выполнении команды в консольном окне появляется строка `Для продолжения нажмите любую клавишу`. Чтобы сообщение не отображалось, вместо команды `system("PAUSE")` предлагается использовать команду `system("PAUSE>nul")`.

Во всех примерах главы программный код главной функции программы начинается с инструкции `system("chcp 1251")`, а команда `system("PAUSE")` всегда последняя перед инструкцией `return 0`. При этом мы данные команды в дальнейшем пояснять не будем, равно как и не будем приводить для результатов выполнения программ сообщения, отображаемые данными командами. Также напомним, что для использования функции `system()` подключается заголовок `<cstdlib>`. Поэтому все примеры в главе содержат еще и инструкцию `#include <cstdlib>` в шапке программы.

Что касается самой функции `system()`, то ее действие таково: строка, указанная аргументом функции, передается для исполнения командному процессору операционной системы. Например, с помощью команды `system("DIR")` удастся отобразить содержимое текущей папки (каталога), командой `system("CLS")` выполняется очистка консольного окна, а командой `system("HELP")` в консольном окне отображается справка по командам командной строки, и так далее.

Кроме того, в зависимости от способа создания проекта может понадобиться в шапке программы первой командой указать инструкцию `#include "stdafx.h"`.

Инструкция нужна в том случае, если проект создается с установленной опцией Предварительно скомпилированный заголовок при определении параметров создаваемого приложения.

Ну и, разумеется, все изложенное выше никакого отношения к многопоточковому программированию не имеет.

На что в результатах выполнения программы стоит обратить внимание? Нас интересует последовательность сообщений, появляющихся в окне вывода, а если точнее, то "источник" этих сообщений. Видим, что первое сообщение - следствие выполнения команды в главной функции программы. Следующие три сообщения отображаются при выполнении программного кода функции `say()`. Два последних сообщения - результат выполнения команд, описанных непосредственно в теле функции `main()`. Все прогнозируемо.

Теперь переходим ко второй части "исследования". Нам нужно организовать *поток*. Код, который должен выполняться при выполнении потока - код функции `say()`. Проще говоря, наша задача состоит в том, чтобы в главной функции программы запустить на выполнение поток с программным кодом функции `say()`. Это разрешимая задача. Начиная со стандарта языка C++11 есть стандартные средства разработки, позволяющие воплотить наши планы в реальность.

Система потокового программирования во многом базируется на использовании класса `thread`. Для использования класса в программе подключается заголовок `<thread>`. Если заголовок подключен, можем приступить к созданию потока. Рецепт очень простой: достаточно создать объект класса `thread`, передав аргументом конструктору имя функции, выполняемой в потоке. Более конкретно, если мы хотим, чтобы в потоке выполнялся программный код функции `say()`, то нам для создания и запуска потока достаточно команды `thread t(say)`. Формально командой создается объект `t` класса `thread` (объект `t` будем называть *объектом потока*). А неформально - как только выполнена команда `thread t(say)`, параллельно с главным потоком начнется выполняться дочерний поток. Выполняемый в дочернем потоке код - это код функции `say()` (ее имя указывалось при создании объекта класса `thread`). На практике все выглядит так, как показано в листинге 14.2.

Листинг 14.2. Создаем поток

```
#include <iostream>
#include <cstdlib>
#include <thread>
using namespace std;
```



```
// Функция для выполнения в дочернем потоке:
void say(){
cout<<"Функция: запуск.\n";
cout<< "Функция: в процессе выполнения.\n";
cout<<"Функция: завершение.\n";
}
// Главная функция программы:
int main(){
// Установка кодировки для консольного окна:
system("chcp 1251");
cout<<"Программа: запущена.\n";
// Создание объекта t класса thread (запуск потока).
// Аргумент конструктора - имя функции say(),
// запускаемой при выполнении потока:
thread t(say);
cout<< "Программа: поток запущен.\n";
cout<< "Программа: завершена.\n";
// Ожидание завершения дочернего потока:
t.join();
// Команда для задержки консольного окна
// (специфика среды VS Express):
system("PAUSE");
return 0;
}
```

Если в предыдущем случае результат выполнения программы был жестко "детерминирован", то в данном случае "возможны варианты". Например, результат выполнения программы может быть таким:

Результат выполнения программы (из листинга 14.2)

```
Программа: запущена.
Функция: запуск.
Функция: в процессе выполнения.
Программа: поток запущен.
Программа: завершена.
Функция: завершение.
```

Что мы можем сказать в данном случае о результате выполнения программы? С определенностью можем утверждать, что первым сообщением всегда является текст "Программа: запущена.". Это результат выполнения самой первой команды в главной функции программы. Объяснение простое: на момент выполнения первой команды в функции `main()` не произошло ничего, что давало бы основания поставить под сомнение порядок и способ выполнения команд. Дальше начинается "зона нестабильности" - в том

смысле, что последовательность появления сообщений предугадать очень сложно. Связано такое положение дел с тем, что командой `thread t(say)` запускается дочерний поток, в котором выполняется программный код функции `say()`. Начиная с этого момента команды дочернего потока (а фактически инструкции в теле функции `say()`) и последующие (после команды `thread t(say)`) инструкции главного потока (инструкции в теле функции `main()`) выполняются "одновременно". Оба потока обращаются к консольному окну. И в одном, и в другом потоке выполняется совсем немного команд. Какое сообщение появится раньше, а какое чуть позже - вопрос задержки на доли секунды при проведении вычислений. Поэтому результат спрогнозировать нереально. Единственное, что прогнозируемо - сообщения появляются "вперемешку". И здесь проявляется принципиальное отличие в использовании потоков от последовательного вызова команд. В предыдущем примере при вызове функции `say()` сначала выполнялись команды, описанные в теле функции, и только затем начиналось выполнение команд в функции `main()`, находившихся после вызова `say()`. В рассматриваемом примере запуск потока (на основе функции `say()`) не означает "перерыв" в выполнении команд непосредственно в функции `main()`.

Одна из последних команд в главной функции программы - инструкция `t.join()`. В данном случае из объекта дочернего потока `t` вызывается метод `join()`. Назначение команды такое: управление следующей инструкции не передается до тех пор, пока не будет завершен поток объекта `t`. Проще говоря, команда `t.join()` - команда ждать завершения потока `t`.

Подробности

Если не использовать команду `t.join()`, то, скорее всего, главный поток закончится раньше окончания дочернего потока. Это плохая ситуация, поскольку программа выполнение как бы закончилась, а потоки, запущенные программой, еще не закончились. Нормальным такое завершением работы программы назвать нельзя. Поэтому перед тем, как программа (главный поток) завершит свою работу, следует дождаться завершения дочернего потока.



На заметку

Строго говоря, прежде чем вызывать из объекта потока метод `join()`, следовало бы проверить, не завершился ли уже данный поток. Или, другими словами, имеет смысл ждать завершения лишь незавершенного потока. Если мы попытаемся дождаться завершения заверченного потока, возникнет ошибка.

Есть метод `joinable()`, который вызывается из объекта потока и позволяет установить, завершился поток или нет. Метод `join()` целесообразно вызывать из объекта потока, если результатом вызова метода `joinable()` из этого объекта является значение `true`. Мы пока к помощи метода `joinable()` не прибегали, но и забывать о его существовании не намерены.

14.2. Несколько дочерних потоков

*Маленькая, а с фантазией.
из к/ф "Девчата"*

В рассмотренном примере в главном потоке создавался один дочерний поток. На самом деле потоков может быть несколько, причем у дочерних потоков могут быть свои дочерние потоки - то есть в дочернем потоке можно запустить собственный дочерний поток (или потоки). Небольшой пример, в котором из главного потока запускается два дочерних потока, представлен в листинге 14.3.

Листинг 14.3. Несколько дочерних потоков

```
#include<iostream>
#include <cstdlib>
#include <thread>
using namespace std;
// Переменная для использования
// в операторах цикла:
int n = 5;
// Функция для выполнения в первом потоке:
void one() {
    cout<<"Запускается 1-й дочерний поток.\n";
    cout<<"Выполнение 1-го дочернего потока.\n";
    cout<<"Отображаются числа (1-й поток):\n";
    for(int i=1;i<=n;i++){
        cout<<" * "<<i<<" * \n";
    }
    cout<<"Завершение 1-го дочернего потока.\n";
}
// Функция для выполнения во втором потоке:
void two() {
    cout<<"Запускается 2-й дочерний поток.\n";
    cout<<"Выполнение 2-го дочернего потока.\n";
    cout<<"Отображаются буквы (2-й поток):\n";
    for(int i=0;i<n;i++){
        cout<<" * "<<(char) ('A'+i)<<" * \n";
    }
    cout<<"Завершение 2-го дочернего потока.\n";
}
// Главная функция программы:
int main() {
    // Кодовая страница для консольного окна:
    system("chcp 1251");
    cout<<"Начало выполнения программы.\n";
    // Запуск первого дочернего потока:
```

```

thread a(one);
    // Запуск второго дочернего потока:
thread b(two);
cout<<"Программа еще выполняется.\n";
    // Ожидание завершения первого потока:
if(a.joinable()) a.join();
// Ожидание завершения второго потока:
if(b.joinable()) b.join();
cout<<"Завершение выполнения программы.\n";
    // Задержка консольного окна:
system("PAUSE");
return 0;
}

```

Результат выполнения программы может быть совершенно неожиданным:

Результат выполнения программы (из листинга 14.3)

```

Начало выполнения программы.
Запускается 1-й дочерний поток.
Программа еще выполняется.
Запускается 2-й дочерний поток.
Выполнение 2-го дочернего потока.
Отображаются буквы (2-й поток):
  * A *
Выполнение 1-го дочернего потока.
Отображаются числа (1-й поток):
  * * B *
  * C *
  * D *
  * E *
Завершение 2-го дочернего потока.
1 *
  * 2 *
  * 3 *
  * 4 *
  * 5 *
Завершение 1-го дочернего потока.
Завершение выполнения программы.

```

Хотя в данном конкретном случае вывод программы более-менее структурирован, здесь скорее исключение из правил. Обычно данные выводятся "вперемешку". Но для нас сейчас это не принципиально, поскольку интерес представляет не результат, а сам процесс.

Что касается программного кода, то в программе описаны две функции (`one()` и `two()`) достаточно схожего "содержания": выводятся сообщения

о начале и завершении процесса выполнения потока, а кроме этого отображается ряд натуральных чисел (для первой функции) или букв (для второй функции).

В главной функции программы после сообщения о начале выполнения программы командами `thread a(one)` и `thread b(two)` создается два дочерних потока. Эти потоки выполняются.

Подробности

Выражение `(char)('A'+i)` содержит инструкцию `(char)` для явного приведения выражения `('A'+i)` к типу `char`. Дело в том, что значения типа `char` при арифметических вычислениях интерпретируются на уровне кода соответствующего символа. Поэтому если к символу прибавляется число, то результатом будет числовое значение, получающееся прибавление к коду символа числового операнда суммы. Для превращения такого числа в символ и добавляют инструкцию явного приведения типа. При приведении целого числа к типу `char` приводимое целочисленное значение интерпретируется как код соответствующего символа.

Команды `if(a.joinable()) a.join()` и `if(b.joinable()) b.join()` - на самом деле инструкции дождаться окончания выполнения соответственно первого и второго потоков. Но здесь появляется важное новшество: перед вызовом метода `join()` из объекта потока, мы проверяем данную операцию на корректность. Для этого из объекта потока вызывается метод `joinable()`. Метод возвращает значение `true` если поток находится в стадии выполнения. Если поток уже выполнен или в силу иных причин не является исполняемым потоком, метод возвращает значение `false`.

Небольшая модификация рассмотренного выше примера представлена в листинге 14.4. В предыдущем примере из главного потока вызывалось два дочерних потока. Теперь же схема несколько иная: в главном потоке вызывается дочерний поток, а в данном дочернем потоке вызывается еще один дочерний поток (дочерний поток для дочернего потока). Рассмотрим программный код, представленный ниже (большинство комментариев для уменьшения объема кода удалено, а наиболее важные места кода выделены жирным шрифтом):

Листинг 14.4. Поток вызывается в дочернем потоке

```
#include <iostream>
#include <cstdlib>
#include <thread>
using namespace std;
int n = 5;
void one() {
    cout<<"Запускается дочерний поток.\n";
```

```

cout<<"Выполнение дочернего потока.\n";
cout<<"Отображаются числа (дочерний поток):\n";
for(int i=1;i<=n;i++){
    cout<<" * "<<i<<" * \n";
}
cout<<"Завершение дочернего потока.\n";
}
void two(){
    cout<<"Запускается поток.\n";
    cout<<"Выполнение потока.\n";
    cout<<"Запускается дочерний поток.\n";
    // Запускается дочерний поток:
    thread a(one);
    cout<<"Отображаются буквы:\n";
    for(int i=0;i<n;i++){
        cout<<" * "<<(char)('A'+i)<<" * \n";
    }
    // Ожидание завершения дочернего потока:
    if(a.joinable()) a.join();
    cout<<"Завершение потока.\n";
}
int main(){
    system("chcp 1251");
    cout<<"Начало выполнения программы.\n";
    // Запуск потока:
    thread b(two);
    cout<<"Программа еще выполняется.\n";
    // Ожидание завершения потока:
    if(b.joinable()) b.join();
    cout<<"Завершение выполнения программы.\n";
    system("PAUSE");
    return 0;
}

```

Результат выполнения программы может быть, например, таким:

Результат выполнения программы (из листинга 14.4)

```

Начало выполнения программы.
Программа еще выполняется.
Запускается поток.
Выполнение потока.
Запускается дочерний поток.
Запускается дочерний поток.
Выполнение дочернего потока.
Отображаются числа (дочерний поток):
 * Отображаются буквы:

```

```
* A *
* B1 *
*   *
* 2 *
* 3 *
* 4 *
* 5C *
```

Завершение дочернего потока.

```
*
* D *
* E *
```

Завершение потока.

Завершение выполнения программы.

Принципиально мало что поменялось, но на два момента стоит обратить внимание. Во-первых, как уже отмечалось выше, схема использована следующая:

- в главной функции программы на основе функции `two()` создается поток (команда `thread b(two)`);
- в теле функции `two()` на основе функции `one()` в свою очередь создается еще один поток (команда `thread a(one)`).

Во-вторых, в теле функции `two()` есть команда `if(a.joinable()) a.join()`, вследствие чего поток, созданный на основе функции `two()`, будет ожидать завершения выполнения дочернего потока `a`, запущенного ранее командой `thread a(one)`. В свою очередь в главной функции программы имеется инструкция `if(b.joinable()) b.join()`, благодаря чему главный поток ожидает завершения выполнения потока, созданного и запущенного командой `thread b(two)`.

14.3. Передача аргументов функции потока

Тихо, кричать не надо. Каждая ваша мысль нам известна.

из к/ф "Гостья из будущего"

Выше мы рассматривали примеры, в которых аргументом конструктору класса `thread` при создании объекта этого класса передавалось имя функции. Каждый раз использовались функции без аргументов, не возвращающие результат. Но поток можно создать и на основе функции, не возвращающей результат, но имеющей аргументы. Если функции, на основе которой создается поток, при вызове нужно передавать аргументы, то эти аргументы передаются аргументами конструктору класса `thread`: первым

аргументом конструктору, как и ранее, передается имя функции, а все последующие аргументы - это как раз аргументы, передаваемые функции при вызове. Небольшой пример, иллюстрирующий процесс создания потока на основе функции с аргументами представлен в листинге 14.5.

Листинг 14.5. Создание потока на основе функции с аргументами

```
#include <iostream>
#include <cstdlib>
#include <thread>
using namespace std;
// Функция без аргументов для создания потока:
void alpha(){
    cout<<"Alpha: начало выполнения потока.\n";
    cout<<"Alpha: аргументов у функции потока нет.\n";
    cout<<"Alpha: завершение выполнения потока.\n";
}
// Функция с одним аргументом для создания потока:
void bravo(char s){
    cout <<"Bravo: начало выполнения потока.\n";
    cout<<"Bravo: аргумент функции потока - "<<s<<"\n";
    cout<<"Bravo: завершение выполнения потока.\n";
}
// Функция с двумя аргументами для создания потока:
void charlie(int a,int b){
    cout<<"Charlie: начало выполнения потока.\n";
    cout<<"Charlie: аргументы - "<<a<<" и "<<b<<"\n";
    cout<<"Charlie: завершение выполнения потока.\n";
}
// Главная функция программы:
int main(){
    // Кодировка для консольного окна:
    system("chcp 1251");
    cout<<"Alpha: поток на основе функции без аргументов.\n";
    // Создание первого потока
    // (на основе функции без аргументов):
    thread a(alpha);
    cout<<"Bravo: поток на основе функции с одним аргументом.\n";
    // Создание второго потока
    // (на основе функции с одним аргументом):
    thread b(bravo,'A');
    cout<<"Charlie: поток на основе функции с двумя
аргументами.\n";
    // Создание третьего потока
    // (на основе функции с двумя аргументами):
    thread c(charlie,5,9);
```



```
// Ожидание завершения потоков:
if(a.joinable()) a.join();
if(b.joinable()) b.join();
if(c.joinable()) c.join();
// Задержка консольного окна:
system("PAUSE");
return 0;
}
```

Возможный результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 14.5)

```
Alpha: поток на основе функции без аргументов.
Bravo: поток на основе функции с одним аргументом.
Alpha: начало выполнения потока.
Alpha: аргументов у функции потока нет.
Alpha: завершение выполнения потока.
Bravo: начало выполнения потока.
Bravo: аргумент функции потока - А.
Charlie: поток на основе функции с двумя аргументами.
Bravo: завершение выполнения потока.
Charlie: начало выполнения потока.
Charlie: аргументы - 5 и 9.
Charlie: завершение выполнения потока.
```

В программе описывается три функции: функция `alpha()` без аргументов, функция `bravo()` с одним символьным аргументом и функция `charlie()` с двумя целочисленными аргументами. При создании потока на основе первой функции используется команда `thread a(alpha)` (для создания потока функция `alpha()` вызывается без аргументов). При создании потока на основе второй функции использована команда `thread b(bravo, 'A')` (для создания потока функция `bravo()` вызывается с аргументом `'A'`). При создании потока на основе третьей функции использована команда `thread c(charlie, 5, 9)` (для создания потока функция `charlie()` вызывается с аргументами `5` и `9`). В остальном пример достаточно простой, поэтому хочется верить, что в пояснениях необходимости нет.

14.4. Создание потока на основе функтора

*Это в твоих руках все горит, а в его руках
все работает!*

из к/ф "Покровские ворота"

Для создания потока может использоваться не только функция, но и функтор. Создать поток на основе функтора так же легко, как и на основе функ-

ции. Для этого достаточно описать класс функтора, создать на основе данного класса объект (то есть создать сам функтор), а затем при создании потока вместо имени функции указать название объекта-функтора. Программа в листинге 14.6 иллюстрирует означенный подход.

Листинг 14.6. Создание потоков на основе функтора

```
#include <iostream>
#include <cstdlib>
#include <string>
#include <thread>
using namespace std;
// Класс для создания функтора:
class Threadable{
private:
    // Символьное поле:
    char symb;
    // Целочисленное поле:
    int count;
public:
    // Конструктор:
    Threadable(char s,int n){
        symb=s;
        count=n;
    }
    // Операторный метод для вызова объекта
    // (версия без аргументов):
    void operator() () {
        cout<<"Начало 1-го потока.\n";
        for(int i=0;i<count;i++){
            cout<<(char) (symb+i)<<" ";
        }
        cout<<endl;
        cout<<"Завершение1-го потока.\n";
    }
    // Операторный метод для вызова объекта
    // (версия с текстовым аргументом):
    void operator() (string txt){
        cout<<"Начало 2-го потока.\n";
        for(int i=0;i<count;i++){
            cout<<(char) (symb+i);
            cout<<txt[i%txt.length()]<<" ";
        }
        cout<<endl;
        cout<<"Завершение2-го потока.\n";
    }
}
```

```

}; // Окончание описания класса
// Главная функция программы:
int main() {
    // Кодировка для консольного окна:
    system("chcp 1251");
    cout<<"Начало выполнения программы.\n";
    // Создание объекта функтора:
    Threadable obj('A',10);
    // Создание первого потока
    // (на основе функтора без аргументов):
    thread ta(obj);
    // Ожидание завершения первого потока:
    if(ta.joinable()) ta.join();
    // Создание второго потока
    // (на основе функтора с текстовым аргументом):
    thread tb(obj,"АВВГД");
    // Ожидание завершения второго потока:
    if(tb.joinable()) tb.join();
    cout<<"Завершение выполнения программы.\n";
    // Задержка консольного окна:
    system("PAUSE");
    return 0;
}

```

Хотя в программе мы создаем два дочерних потока, специфика их выполнения такова, что второй поток запускается только после того, как заканчивается первый поток. В этом смысле особенности многопоточного программирования нивелируются полностью (все выглядит так, как если бы мы вызывали функторы один за другим, без создания потоков), зато можно разделить и различить результат выполнения разных потоков. При выполнении программы получим следующее:

Результат выполнения программы (из листинга 14.6)

```

Начало выполнения программы.
Начало 1-го потока.
А В С D E F G H I J
Завершение 1-го потока.
Начало 2-го потока.
АА ВВ СВ DГ ЕД FА GВ НВ IГ JД
Завершение 2-го потока.
Завершение выполнения программы.

```

В программе мы описываем класс `Threadable`. У класса два поля: символьное поле `sym` и целочисленное поле `count`. Конструктору класса пе-

редаются два аргумента, определяющие значения полей.

В классе описаны две версии операторного метода для вызова объектов класса. Версия операторного метода `operator()()` без аргументов содержит оператор цикла, в котором отображается подборка символов, начиная с символа, записанного в символьное поле `symb`, а количество отображаемых символов определяется значением поля `count`. Каждый очередной символ вычисляется как результат выражения `(char)(symb+i)`. Результатом выражения является символ с числовым кодом, получаемым при прибавлении к коду символа `symb` целого числа `i`.

Версия операторного метода `operator()(text)` с текстовым аргументом организована похожим образом, но есть некоторые отличия. Самое главное - символы отображаются парами. Первый символ вычисляется точно так же, как и в версии метода без аргумента (выражение `(char)(symb+i)`). Второй символ берется из текста, переданного аргументом методу. В частности, второй символ вычисляется на основе текстового аргумента `txt` выражением `txt[i%txt.length()]`. Принцип получения значения простой: выполняется индексирование текстового аргумента. Индексом указано значение `i%txt.length()` - это остаток от деления значения индексной переменной `i` на число `txt.length()`, равное длине текста. Данная процедура выполняется по той простой причине, что длина текста, переданного методу, в общем случае не совпадает с количеством отображаемых символов (определяется полем `count`). Вычисляя остаток от деления индекса на длину текста, выполняем циклическую перестановку индексов при определении буквы в тексте.

В главной функции программы командой `Threadable obj('A', 10)` создается объект-функтор `obj`. Потоки на основе функтора `obj` создаются командам `thread ta(obj)` (аргумент функтору не передается) и `thread tb(obj, "АВВГД")` (функтору передается аргумент).

14.5. Создание потока на основе метода класса

*Она хорошо знает дело, которым руководит. Такое тоже бывает.
из к/ф "Служебный роман"*

Мы уже умеем создавать потоки на основе функций и функторов. Оказывается, потоки можно создавать и на основе методов. Схема создания потока на основе нестатического (то есть обычного) метода состоит из нескольких этапов. Во-первых, естественно, необходимо описать соответствующий класс. Во-вторых, на основе такого класса нужно создать объект. После это-

го можно создавать поток. Но если при создании потока на основе функции аргументом конструктору класса `thread` передается имя функции, то при создании потока на основе метода объекта аргументом конструктору класса `thread` передается *указатель на метод* и имя объекта, из которого метод будет вызываться. Напомним, что если нам нужно получить указатель на метод некоторого класса, то соответствующее выражение выглядит как `&класс::метод`. Например, если в классе `Threadable` описан метод `one()`, то указатель на данный метод имеет вид `&Threadable::one`. Чтобы создать и запустить поток `ta` на основе метода `one()` объекта `obj` класса `Threadable`, выполняется команда `thread ta(&Threadable::one, obj)` - это при условии, если методу `one()` не нужно передавать аргументы. Команда `thread tb(two, obj, "АВВГД")` - пример создания потока `tb` на основе метода `two()` объекта `obj` класса `Threadable`, причем методу `two()` передается аргумент "АВВГД".

В листинге 14.7 предыдущий пример "переработан" так, что вместо двух версий операторного метода описываются два различных метода `one()` и `two()`, а в главной функции программы создается объект `obj` класса `Threadable`, и затем на основе методов `one()` и `two()` объекта `obj` создаются потоки (некоторые комментарии в программном коде удалены, а наиболее важные места выделены жирным шрифтом).

Листинг 14.7. Создание потоков на основе методов

```
#include <iostream>
#include <cstdlib>
#include <string>
#include <thread>
using namespace std;
class Threadable{
private:
    char symb;
    int count;
public:
    Threadable(char s,int n){
        symb=s;
        count=n;
    }
    // Метод (без аргументов) для создания потока:
    void one() {
        cout<<"Начало 1-го потока.\n";
        for(int i=0;i<count;i++){
            cout<<(char) (symb+i)<<" ";
        }
        cout<<endl;
    }
    void two() {
        cout<<"Начало 2-го потока.\n";
        for(int i=0;i<count;i++){
            cout<<(char) (symb+i)<<" ";
        }
        cout<<endl;
    }
};
```

```

cout<<"Завершение 1-го потока.\n";
}
// Метод (с текстовым аргументом) для создания потока:
void two(string txt){
cout<<"Начало 2-го потока.\n";
for(int i=0;i<count;i++){
cout<<(char) (symb+i);
cout<<txt[i%txt.length()]<<" ";
}
cout<<endl;
cout<<"Завершение 2-го потока.\n";
}
};
int main(){
system("chcp 1251");
cout<<"Начало выполнения программы.\n";
// Создание объекта:
Threadable obj('A',10);
// Создание первого потока
thread ta(&Threadable::one,obj);
if(ta.joinable()) ta.join();
// Создание второго потока
thread tb(&Threadable::two,obj,"АБВГД");
if(tb.joinable()) tb.join();
cout<<"Завершение выполнения программы.\n";
system("PAUSE");
return 0;
}

```

Результат выполнения программы точно такой же, как и в предыдущем случае:

Результат выполнения программы (из листинга 14.7)

```

Начало выполнения программы.
Начало 1-го потока.
А В С D E F G H I J
Завершение 1-го потока.
Начало 2-го потока.
АА ВВ СВ ДГ ЕД ФА ГВ НВ ИГ ЖД
Завершение 2-го потока.
Завершение выполнения программы.

```

В принципе, программный код претерпел минимальные изменения, но те, что внесены, уже анализировались. Посему детальнее на данном примере останавливаться не будем.

14.6. Временная приостановка потоков

Вперед! Ниже голову!
из к/ф "Старики-разбойники"

Нередко возникает необходимость сделать временный перерыв в выполнении потока - проще говоря, остановить выполнение потока на заданный промежуток времени. После этого поток продолжает свое выполнение. Именно такую задачу рассмотрим далее. Но прежде нам предстоит небольшой экскурс в теорию.

Анализ программного кода - как разгадывание детективной истории. Начинать "раскручивать клубок" обычно лучше с конца. Так поступим и мы. Начнем с простого обстоятельства: для приостановки выполнения потока на определенное время используем функцию `sleep_for()`. Время, на которое задерживается (приостанавливается) выполнение потока, определяется аргументом функции (но это не числовой аргумент, как можно было бы ожидать). Приостанавливается тот поток, в котором вызывается функция.

Нас интересует два вопроса: как вызвать функцию и что передать ей аргументом? Оба вопроса имеют нетривиальные ответы. Начнем с вызова функции.

Дело в том, что функция `sleep_for()` описана в пространстве имен `this_thread`, доступном после подключения заголовка `<thread>`. Поэтому при вызове функции перед ее именем (черезоператор расширения контекста `::`) указывается пространство имен `this_thread`, в котором описана функция. Следовательно, команда вызова функции имеет вид `this_thread::sleep_for()`, и в круглых скобках указывается аргумент.

Теперь по поводу аргумента. Аргумент функции `sleep_for()`, определяющий интервал времени приостановки выполнения потока, должен быть объектом, созданным на основе обобщенного класса `duration`. Класс `duration` доступен в пространстве имен `chrono`, а для использования последнего подключается заголовок `<chrono>`. Поскольку класс `duration` обобщенный, при создании объектов на основе класса ему необходимо передавать параметры. Но нам самостоятельно создавать объекты на основе обобщенного класса `duration` вряд ли придется. Мы пойдем более простым путем - воспользуемся классом, представляющим собой одну из реализаций обобщенного класса `duration`. Более конкретно, далее мы воспользуемся классом `seconds`. При создании объекта класса аргументом конструктору передается целое число - время в секундах, на которое выполняется приостановка в выполнении потока. Поскольку класс `seconds` определен в пространстве имен `chrono`, то перед именем класса указывается название пространства имен `chrono` (а между ними - оператор рас-

ширения контекста) - то есть обращение к классу `seconds` выполняется в формате `chrono::seconds`. Например, команда приостановки текущего потока на *1 секунду* может выглядеть как `this_thread::sleep_for(chrono::seconds(t))`.

Подробности

Пространство имен -достаточно удобный способ разграничить и упорядочить множество идентификаторов, используемых для обозначения функций, классов и прочих утилит. В известном смысле пространство имен отождествимо с областью видимости (то есть областью доступности переменных, функций, классов). Пространства имен могут быть вложенными: одно пространство имен может содержать другое пространство имен.

Если функция или класс определены в каком-то пространстве имен, то они доступны только после подключения этого пространства имен с помощью инструкции `using namespace` (плюс название пространства имен - например, `std`) или при явном указании пространства имен в названии функции или класса.

Пространство имен достаточно легко объявить: после ключевого слова `namespace` указывается имя пространства имен и в фигурных скобках - команды объявления утилит, входящих в пространство имен:

```
namespaceназвание{
    // объявления пространства имен
}
```

Мы будем использовать уже существующие пространства имен.

Теперь рассмотрим программный код, представленный в листинге 14.8. Код в принципе достаточно простой: в программе описывается функция `go()`, на основе которой в главной функции программы запускается дочерний поток. В дочернем потоке последовательно отображается ряд натуральных чисел. При этом в главном потоке синхронно отображаются буквы. Новшество состоит в том, что и в дочернем, и в главном потоке между выводом значений делается фиксированная пауза. Рассмотрим программный код:

Листинг 14.8. Временная приостановка потоков

```
#include <iostream>
#include <cstdlib>
#include <thread>
#include <chrono>
using namespace std;
// Функция для создания потока:
void go(int t,int n){
    cout<<"Дочерний поток - начало.\n";
    for(int i=1;i<=n;i++){
        // Задержка потока на tсекунд:
```



```

this_thread::sleep_for(chrono::seconds(t));
cout<<"Дочерний поток:\t"<<i<<endl;
}
cout<<"Дочерний поток - завершение.\n";
}
// Главная функция программы:
int main(){
    // Кодировка для консольного окна:
    system("chcp 1251");
    cout<<"Начало выполнения программы.\n";
    int t=2,n=5,T=3,N=4;
    // Создание потока:
    thread th(go,t,n);
    for(int i=0;i<N;i++){
        // Задержка потока на T секунд:
        this_thread::sleep_for(chrono::seconds(T));
        cout<<"Главный поток:\t"<<(char) ('A'+i)<<endl;
    }
    // Ожидание завершения потока:
    if(th.joinable()) th.join();
    cout<<"Завершение выполнения программы.\n";
    // Задержка консольного окна:
    system("PAUSE");
    return 0;
}

```

С большой вероятностью результат выполнения программы будет таким:

Результат выполнения программы (из листинга 14.8)

```

Начало выполнения программы.
Дочерний поток - начало.
Дочерний поток: 1
Главный поток:  A
Дочерний поток: 2
Главный поток:  B
Дочерний поток: 3
Дочерний поток: 4
Главный поток:  C
Дочерний поток: 5
Дочерний поток - завершение.
Главный поток:  D
Завершение выполнения программы.

```

У функции `go()` два целочисленных аргумента. Первый аргумент `t` определяет интервал времени (в секундах), а второй аргумент `n` определяет количество циклов при отображении чисел. Более конкретно, аргумент `n` является

верхней границей диапазона изменения индексной переменной в операторе цикла, а аргумент `t` используется в команде `this_thread::sleep_for(chrono::seconds(t))` в теле оператора цикла.

В программе командой `thread th(go, t, n)` запускается дочерний поток, и сразу после этого начинает выполняться оператор цикла, в котором в алфавитном порядке отображаются символьные значения. Интервал времени, на который в главном потоке выполняется пауза, определяется командой `this_thread::sleep_for(chrono::seconds(T))`, отличающейся от аналогичной команды в функции `go()` разве что аргументом конструктора класса `seconds` из пространства имен `chrono`.

Перед завершением главного потока ожидается завершение дочернего потока (команда `if(th.joinable()) th.join()`).



На заметку

В отличие от рассмотренных ранее примеров, в данном случае сообщения в окне вывода появляются с заметной задержкой. Числовые параметры в программе подобраны так, что процесс вывода сообщений программой занимает порядка 10-15 секунд.

14.7. Синхронизация потоков

- Да, как эксперимент это интересно. Но какое практическое применение?

- Господи, именно практическое!

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

В предыдущих примерах мы неоднократно наблюдали ситуацию, когда при параллельном выполнении потоков информация, которая отображалась этими потоками в консольном окне, приобретала иногда очень причудливые формы (желающие могут обратиться к результату выполнения программы в листинге 14.3, например). Имеется в виду ситуация, когда в процесс вывода сообщения одним потоком "вмешивался" другой поток. Это проявление более общей проблемы, связанной с совместным доступом разных потоков к общему ресурсу.

Обычно (не всегда, но очень часто) приходится *синхронизировать потоки*: упорядочивать или разграничивать время доступа потока к тому или иному ресурсу. В данном случае вполне логично было бы добиться такой ситуации: если какой-то поток уже начал вывод сообщения в консоль, то он должен вывод завершить без того, чтобы прочие потоки "вставляли свои пять копеек".

Синхронизацию потоков можно выполнять по-разному. Мы рассмотрим наиболее простой и прагматичный способ, основанный на использовании специальных объектов, создаваемых на основе класса `mutex`. Класс `mutex` доступен после подключения заголовка `<mutex>`, а объекты класса, среди прочих, имеют методы `lock()` и `unlock()`. Методами выполняется соответственно *блокировка* и *разблокировка* ресурсов, используемых в потоке. Если ресурс заблокирован, то никакой другой поток к нему не получит доступ (до тех пор, пока ресурс не будет разблокирован). Для иллюстрации принципа синхронизации потоков через блокировку ресурсов (в данном случае блокируется доступ к консольному окну) рассмотрим программный код в листинге 14.9.

Эта программа - модификация примера из листинга 14.3. Теперь в программном коде появилась инструкция `#include <mutex>` подключения заголовка `<mutex>`, команда `mutex m`, которой создается объект `m` класса `mutex`, а также в большом количестве команды `m.lock()` и `m.unlock()`, выполняющие блокировку и разблокировку ресурса потоком. Команды `m.lock()` и `m.unlock()` попадают парами и их реально много. Прежде, чем пояснить в подробностях их роль в программе, рассмотрим весь программный код и результат (возможный) его выполнения:

Листинг 14.9. Синхронизация потоков

```
#include <iostream>
#include <cstdlib>
#include <thread>
#include <mutex>
using namespace std;
// Переменная для использования
// в операторах цикла:
int n=5;
// Объект класса mutex для синхронизации
// доступа к консоли:
mutex m;
// Функция для выполнения в первом потоке:
void one(){
m.lock(); // Блокировка
cout<<"Запускается 1-й дочерний поток.\n";
m.unlock(); // Разблокировка
m.lock(); // Блокировка
cout<<"Выполнение 1-го дочернего потока.\n";
m.unlock(); // Разблокировка
m.lock(); // Блокировка
cout<<"Отображаются числа (1-й поток):\n";
m.unlock(); // Разблокировка
```

```

for(int i=1;i<=n;i++){
m.lock(); // Блокировка
cout<<" * " <<i<<" * \n";
m.unlock(); // Разблокировка
}
m.lock(); // Блокировка
cout<<"Завершение 1-го дочернего потока.\n";
m.unlock(); // Разблокировка
}
// Функция для выполнения во втором потоке:
void two(){
m.lock(); // Блокировка
cout<<"Запускается 2-й дочерний поток.\n";
cout<<"Выполнение 2-го дочернего потока.\n";
cout<<"Отображаются буквы (2-й поток):\n";
m.unlock(); // Разблокировка
for(int i=0;i<n;i++){
m.lock();
cout<<" * "<<(char)('A'+i)<<" * \n";
m.unlock(); // Разблокировка
}
m.lock(); // Блокировка
cout<<"Завершение 2-го дочернего потока.\n";
m.unlock(); // Разблокировка
}
// Главная функция программы:
int main(){
// Кодовая страница для консольного окна:
system("chcp 1251");
cout<<"Начало выполнения программы.\n";
// Запуск первого дочернего потока:
thread a(one);
// Запуск второго дочернего потока:
thread b(two);
m.lock(); // Блокировка
cout<<"Программа еще выполняется.\n";
m.unlock(); // Разблокировка
// Ожидание завершения первого потока:
if(a.joinable()) a.join();
// Ожидание завершения второго потока:
if(b.joinable()) b.join();
cout<<"Завершение выполнения программы.\n";
// Задержка консольного окна:
system("PAUSE");
return 0;
}

```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 14.9)

```
Начало выполнения программы.
Запускается 1-й дочерний поток.
Запускается 2-й дочерний поток.
Выполнение 2-го дочернего потока.
Отображаются буквы (2-й поток):
Программа еще выполняется.
Выполнение 1-го дочернего потока.
  * A *
Отображаются числа (1-й поток):
  * B *
  * 1 *
  * C *
  * 2 *
  * D *
  * 3 *
  * E *
  * 4 *
Завершение 2-го дочернего потока.
  * 5 *
Завершение 1-го дочернего потока.
Завершение выполнения программы.
```

Как и в результате выполнения программного кода из листинга 14.3, в данном случае сообщения от разных потоков отображаются "вперемешку", но в то же время цельными блоками, так что внутри одних сообщений фрагментов других сообщений нет. Это принципиально. И это благодаря использованию механизма блокировки и разблокировки ресурсов.

Например, в функции `one()`, на основе которой создается первый дочерний поток, перед командой `cout<<"Запускается 1-й дочерний поток.\n"` есть команда `m.lock()`. В результате пока выполняется команда вывода сообщения в консоль, никакой другой поток в консоль "вклинить" не сможет (скорее всего). После вывода сообщения выполняется команда `m.unlock()`. В результате блокировка ресурса (консоли) отменяется, и это шанс для других потоков. Поэтому после сообщения, выводимого в консоль потоком на основе функции `one()`, в консольном окне может появиться сообщение другого потока. Аналогичная ситуация имеет место и при отображении прочих сообщений, включая сообщения, отображаемые при выполнении оператора цикла.

**На заметку**

В функции `two()`, на основе которой создается второй дочерний поток, парой команд `m.lock()` и `m.unlock()` выделена не одна, а целых три команды: `cout<<"Запускается 2-й дочерний поток.\n"`, `cout<<"Выполнение 2-го дочернего потока.\n"` и `cout<<"Отображаются буквы (2-й поток):\n"`. Следовательно, консоль будет заблокирована, пока отображаются эти три сообщения. Поэтому результат выполнения команд появляются в одном "блоке" и сообщения других потоков между данными сообщениями не появятся.

Главная функция программы также содержит в основном знакомый нам код, но некоторые команды (для вывода сообщений) "выделены" инструкциями блокировки и разблокировки. Причины те же, что и в случае с функциями, на основе которых создаются дочерние потоки - блокировка от других потоков консоли на время вывода сообщения.

14.8. Идентификация потоков

*Странный способ украшать дом монограммой королевы.
из к/ф "Приключения Шерлока Холмса и доктора
Ватсона"*

Каждый выполняемый поток имеет свой уникальный *идентификатор*, позволяющий однозначно выделить поток среди множества других потоков. Указанный идентификатор является объектом класса `thread::id` (то есть класс объекта-идентификатора `id` описан в классе `thread`). Для получения объекта-идентификатора для потока можно либо из объекта потока (объект класса `thread`) вызвать метод `get_id()`, либо в самом потоке вызвать функцию `get_id()` из пространства имен `this_thread` (соответствующая команда имеет вид `this_thread::get_id()`). В первом случае (при вызове метода) результатом возвращается объект-идентификатор потока, соответствующего объекту, из которого вызывается метод. Во втором случае результатом возвращается объект-идентификатор потока, в котором вызывается функция. В листинге 14.10 приведен небольшой пример с использованием идентификации нескольких выполняемых потоков.

Подробности

В программе используется ассоциативный контейнер. Ассоциативный контейнер создается на основе обобщенного класса `map` из библиотеки STL. Для использования класса в программе подключается заголовок `<map>`.

Что такое ассоциативный контейнер (или словарь, как его иногда называют)? Это контейнер, доступ к элементам которого получают по ключу. Ключом может быть фактически любой объект (значением элемента - тоже). Например, ключом может быть текстовое значение с именем абонента, а значением - телефонный номер абонента. С некоторой натяжкой ассоциативный контейнер допустимо рассматривать как некое подобие массива, но только роль индекса играет не число, а в общем случае какой-то объект (ключ).

Нам в программе нужен ассоциативный контейнер, у которого ключами будут объекты - идентификаторы потоков (объект класса `thread::id`), а значениями элементов - текстовые названия (тип `string`) для потоков.

При создании объекта на основе класса `map` указывается два параметра для обобщенных типов: тип ключа и тип значения. В нашем случае речь идет об инструкции `map<this::id, string>` в качестве идентификатора типа при объявлении поля класса.

Для получения доступа к значению элемента по ключу используем метод `at()`. Метод вызывается из объекта ассоциативного контейнера и аргументом ему передается объект ключа. Результат вызова метода - значение элемента.

Еще одна важная операция - добавление элемента в контейнер. Особенность операции в том, что добавлять на самом деле нужно два объекта: ключ элемента и значение объекта. Эта "сладкая парочка" предварительно оформляется в один объект, создаваемый на основе обобщенного класса `pair`. Параметром класса указываются типы (классы объектов), формирующих "пару". В рассматриваемом примере объект, состоящий из ключа элемента (идентификатор потока) и значения элемента (текстовое название потока), относится к типу `pair<thread::id, string>`.

Для добавления нового элемента в ассоциативный контейнер, из объекта контейнера вызывается метод `insert()`. Аргументом методу передается `pair`-объект.

Что касается программы, то в ее основу положен следующий "сюжет". В программе выполняется главный поток и два дочерних потока. Еще в программе описывается класс `Counter`, а в нем описано целочисленное поле (назовем его "счетчик"). Поле при создании объекта класса получает нулевое значение. Со "счетчиком" можно выполнять одну операцию: увеличивать значение поля (для этой цели предусмотрен специальный метод `up()`). В программе создается объект класса `Counter`, и данный объект (через указатель на него) передается в каждый из потоков. При выполнении главного и двух дочерних потоков каждый из них пытается увеличить значение поля-"счетчика". Пикантность ситуации в том, что объект, "счетчик" которого увеличивается, идентифицирует поток, изменяющий поле. Поток идентифицируется через объект-идентификатор, возвращаемый при вызове функции `this_thread::get_id()` из выполняемого потока.

Есть еще один момент: поскольку три потока в программе используют один и тот же объект со "счетчиком", в программе выполняется синхронизация потоков. Для этого в программе создается объект `m` класса `mutex`, а затем командами `m.lock()` и `m.unlock()` соответственно выполняется блокировка и разблокировка используемых потоком ресурсов.

Теперь обратимся к программному коду:

Листинг 14.10. Идентификация потоков

```
#include <iostream>
#include <cstdlib>
```

```

#include <string>
#include <thread>
#include <chrono>
#include <mutex>
#include <map>
using namespace std;
// Объект класса mutex для синхронизации
// выполнения потоков:
mutex m;
// Класс для создания объекта со "счетчиком" - общего
// "ресурса" для выполняемых потоков:
class Counter{
private:
    // Целочисленное поле ("счетчик"):
    int count;
public:
    // Поле - ассоциативный контейнер (словарь).
    // Элементы контейнера - пары значений.
    // Ключ элемента контейнера - идентификатор потока.
    // Значение элемента контейнера - название потока.
    map<thread::id,string> thID;
    // Конструктор:
    Counter(){
        count=0;
    }
    // Метод для увеличения значения "счетчика":
    void up(){
        // Увеличивается значение "счетчика":
        count++;
        // Отображение информации о значении "счетчика"
        // и потоке, вызвавшем изменение:
        cout<<"Поток "<<thID.at(this_thread::get_id())<<": ";
        cout<<"новое значение\t-\t"<<count<<".\n";
    }
}; // Окончание описания класса
// Функция для создания потока.
// Аргументы функции: указатель на объект со "счетчиком",
// название потока, интервал задержки потока (в секундах),
// количество циклов при выполнении потока:
void start(Counter *p,stringname,intt,int n){
    m.lock(); // Блокировка ресурсов
    // Добавление нового элемента
    // в ассоциативный контейнер:
    p->thID.insert(pair<thread::id,string>(this_thread::get_
    id(),name));
    cout<<"Начинает выполняться поток "<<name<<".\n";

```



```

m.unlock(); // Разблокировка ресурсов
    // Оператор цикла для увеличения значения "счетчика":
for(int i=1;i<=n;i++){
m.lock(); // Блокировка ресурсов
p->up(); // Увеличение значения "счетчика"
m.unlock(); // Разблокировка ресурсов
    // Приостановка выполнения потока:
this_thread::sleep_for(chrono::seconds(t));
}
m.lock(); // Блокировка ресурсов
cout<<"Поток "<<name<<" завершил выполнение.\n";
m.unlock(); // Разблокировка ресурсов
}
// Главная функция программы:
int main(){
    // Кодировка для консольного окна:
system("chcp 1251");
cout<<"Начинает выполняться программа.\n";
    // Создание объекта со "счетчиком":
Counter cnt;
    // Добавление в поле thID объекта cnt идентификатора
    // и названия для главного потока программы:
cnt.thID.insert(pair<thread::id,string>(this_thread::get_id(),"ГЛАВНЫЙ"));
    // Создание первого дочернего потока:
thread first(start,&cnt,"ПЕРВЫЙ",3,7);
    // Создание второго дочернего потока:
thread second(start,&cnt,"ВТОРОЙ",4,4);
    // Оператор цикла для увеличения значений "счетчика"
    // из главного потока:
for(int i=1;i<=5;i++){
m.lock(); // Блокировка ресурсов
    // Увеличение значения счетчика:
cnt.up();
m.unlock(); // Разблокировка ресурсов
    // Приостановка выполнения главного потока:
this_thread::sleep_for(chrono::seconds(5));
}
    // Ожидание завершения первого дочернего потока:
if(first.joinable()) first.join();
    // Ожидание завершения второго дочернего потока:
if(second.joinable()) second.join();
cout<<"Выполнение программы завершено.\n";
    // Задержка консольного окна:
system("PAUSE");
return 0;
}

```

Результат выполнения программы, скорее всего, будет таким:

Результат выполнения программы (из листинга 14.10)

```
Начинает выполняться программа.
Начинает выполняться поток ПЕРВЫЙ.
Начинает выполняться поток ВТОРОЙ.
Поток ГЛАВНЫЙ: новое значение - 1.
Поток ПЕРВЫЙ: новое значение - 2.
Поток ВТОРОЙ: новое значение - 3.
Поток ПЕРВЫЙ: новое значение - 4.
Поток ВТОРОЙ: новое значение - 5.
Поток ГЛАВНЫЙ: новое значение - 6.
Поток ПЕРВЫЙ: новое значение - 7.
Поток ВТОРОЙ: новое значение - 8.
Поток ПЕРВЫЙ: новое значение - 9.
Поток ГЛАВНЫЙ: новое значение - 10.
Поток ПЕРВЫЙ: новое значение - 11.
Поток ВТОРОЙ: новое значение - 12.
Поток ГЛАВНЫЙ: новое значение - 13.
Поток ПЕРВЫЙ: новое значение - 14.
Поток ВТОРОЙ завершил выполнение.
Поток ПЕРВЫЙ: новое значение - 15.
Поток ГЛАВНЫЙ: новое значение - 16.
Поток ПЕРВЫЙ завершил выполнение.
Выполнение программы завершено.
```

Командой `mutex m` объект для синхронизации потоков создается еще до описания класса `Counter` функций `start()`, используемой для создания двух дочерних потоков.



На заметку

В программе два дочерних потока используются на основе одной функции.

В описании класса `Counter` стоит обратить внимание на поле `thID`, представляющее собой ассоциативный контейнер класса `map<thread::id, string>`. Ключами элементов в контейнере являются объекты класса `thread::id` (объект-идентификатор потока, получаемый при вызове функции `get_id()` из пространства имен `this_thread`), а значения у элементов текстовые (объект класса `string` -название потока, которое задаем самостоятельно).

В теле метода `up()`, предназначенного для увеличения значения "счетчика", командой `count++` увеличивается значение числового поля, после чего

выводится сообщение с данными о названии потока, изменившего значение "счетчика" и о новом значении "счетчика".

Для "получения" имени потока используется инструкция `thID.at(this_thread::get_id())`. В данном случае из объекта ассоциативного контейнера, в котором на момент использования метода `up()` должны быть "спрятаны" названия потоков, вызывается метод `at()`. Методу аргументом передается ключ для идентификации потока. Понятно, что это должен быть идентификатор того потока, в котором вызван метод `up()`. Получить объект-идентификатор выполняемого потока можно, вызвав из него функцию `this_thread::get_id()`, что собственно и делается - результат вызова функции передается аргументом методу `at()`.

Функция для создания потока называется `start()` и у нее три аргумента: указатель `p` на объект класса `Counter` (объект, "счетчик" которого будет увеличиваться при выполнении потока), текстовое значение (объект класса `string`, определяющий название потока), а также два целочисленных значения (первое определяет интервал приостановки потока, а второй задает количество циклов при выполнении потока). В теле функции командой `m.lock()` выполняется блокировка консоли и объекта со "счетчиком", после чего командой `p->thID.insert(pair<thread::id,string>(this_thread::get_id(),name))` в поле-контейнер `thID` объекта, на который ссылается указатель `p`, добавляется новый элемент. Для этого через указатель `p` мы получаем доступ к полю-контейнеру `thID`, и из него вызывается метод `insert()`. Аргументом методу передается "пара" - объект класса `pair<thread::id,string>`. Объект анонимный, создается вызовом конструктора класса `pair<thread::id,string>` с аргументами `this_thread::get_id()` (объект-идентификатор потока) и `name` (аргумент функции `start()`, определяющий название потока).

После отображения сообщения с именем потока и новым значением поля `count`, командой `m.unlock()` выполняется разблокировка ресурсов потоком и "в игру вступает" оператор цикла (верхняя граница `n` для значения индексной переменной цикла - третий аргумент функции `start()`). В теле оператора командой `m.lock()` блокируются ресурсы (в данном случае объект со "счетчиком") для эксклюзивного использования в потоке, командой `p->up()` из объекта со "счетчиком" вызывается метод `up()` (увеличивающий значение поля `count` объекта, и выводящий сообщение о значении поля и названии потока). Далее отменяется блокировка ресурсов потоком (команда `m.unlock()`), а командой `this_thread::sleep_for(chrono::seconds(t))` выполняется приостановка в выполнении потока на время, определяемое переменной `t` (четвертый аргумент функции `start()`).

В функции `main()` командой `Counter cnt` создается объект `cnt` (объект со "счетчиком") класса `Counter`. При создании объекта `cnt` поле `count` получает нулевое значение, но поле-контейнер `thID` элементов не содержит. Поэтому командой `cnt.thID.insert(pair<thread::id, string>(this_thread::get_id(), "ГЛАВНЫЙ"))` в контейнер `thID` объекта `cnt` добавляется элемент с текстовым значением "ГЛАВНЫЙ" и ключом `this_thread::get_id()`, представляющим собой идентификатор для главного потока.

Два дочерних потока создаются командами `thread first(start, &cnt, "ПЕРВЫЙ", 3, 7)` и `thread second(start, &cnt, "ВТОРОЙ", 4, 4)`. В обоих случаях функции `start()` для запуска потока передается адрес `&cnt` объекта `cnt`.

После запуска дочерних потоков в главном потоке начинает выполняться оператор цикла. В теле оператора цикла командой `cnt.up()` из объекта `cnt` вызывается метод `up()` для увеличения значения поля `count` объекта `cnt`.



На заметку

Проще говоря, главный поток также принимает участие в "накручивании" поля "счетчика".

Приостановка выполнения главного потока имеет место вследствие выполнения команды `this_thread::sleep_for(chrono::seconds(5))`.

Традиционно перед завершением выполнения программы размещены инструкции, благодаря которым главный поток ожидает завершения выполнения дочерних потоков.

Глава 15.

ИНФОРМАЦИЯ К РАЗМЫШЛЕНИЮ



Я Вам расскажу, что мне удалось обнаружить в связи с делом физиков.

из к/ф "Семнадцать мгновений весны"

Вопросы, рассмотренные в этой главе, не объединены какой-то общей темой или подходом. Мы поговорим о самых разных вещах, которые по той или иной причине не вошли в предыдущие главы. Есть важный момент, позволяющий в некоторой степени обосновать наличие в книге такой "разнородной" главы, как эта. Состоит он в том, что мы не будем детально описывать или анализировать тот или иной механизм, а только дадим читателю представление о наличии соответствующего механизма и его основных характеристиках. Таким образом, в главе представлен краткий обзор того, что не обсуждалось ранее, но что в принципе может понадобиться при составлении программных кодов в рамках концепции ООП. Но как бы там ни было, далее очень кратко описаны определенные конструкции языка C++ и приемы, которые, не исключено, могут стать читателю полезными сейчас или в будущем.

15.1. Структуры

Может где-нибудь высоко в горах, но не в нашем районе, вы что-нибудь обнаружите для вашей науки.

из к/ф "Кавказская пленница"

Мы во множестве использовали классы и объекты. Мы использовали и обычные переменные. Есть нечто среднее между классом и обычной переменной - *структура*. Структура - набор переменных, объединенных общим именем. В известном смысле структура напоминает класс, но совсем без методов, а с одними только полями. Объявляется структура аналогично объявлению класса, с той лишь разницей, что вместо ключевого слова `class` используется ключевое слово `struct`.

Шаблон описания структуры такой (жирным шрифтом выделены основные ключевые элементы шаблона):

```
struct имя_структуры{
тип_поля_1 поле_1;
тип_поля_2 поле_2;
...
тип_поля_N поле_N;
};
```

Как и в случае класса, после закрывающей скобки в описании структуры можно указать список экземпляров структуры.



На заметку

Следует понимать, что структура задает определенный обобщенный тип данных. То есть структура аналогична классу. То, что создается на основе структуры, будем называть экземпляром структуры. Экземпляр структуры, созданный на основе структуры - аналог объекта, созданного на основе класса.

Обращение к полям экземпляра структуры выполняется так же, как и обращение к полям объекта класса: после имени экземпляра структуры через точку указывается поле экземпляра. Небольшой пример использования структур представлен в листинге 15.1.

Листинг 15.1. Структуры

```
#include <iostream>
#include <string>
using namespace std;
// Структура:
struct card{
    // Поля структуры:
    string name;
    int phone;
    char gender;
};
// Функция для отображения значений
// полей экземпляра структуры:
void show(card a){
    cout<<"Имя: "<<a.name<<endl;
    cout<<"Телефон: "<<a.phone<<endl;
    cout<<"Пол: "<<a.gender<<endl;
}
// Главная функция программы:
int main(){
    // Экземпляры структуры:
    card ivanov,petrova,sidorov;
    // Первый экземпляр структуры - значения полей:
    ivanov.name="Иванов И.И.";
    ivanov.phone=2345678;
    ivanov.gender='М';
    // Второй экземпляр структуры - значения полей:
    petrova.name="Петрова П.П.";
    petrova.phone=8765432;
    petrova.gender='Ж';
    cout<<"Первая запись:\n";
    // Отображение значений полей
    // первого экземпляра структуры:
```

```

show(ivanov);
cout<<"Вторая запись:\n";
    // Отображение значений полей
    // второго экземпляра структуры:
show(petrova);
    // Присваивание экземпляров структур:
sidorov=ivanov;
cout<<"Третья запись:\n";
    // Отображение значений полей
    // третьего экземпляра структуры:
show(sidorov);
    // Изменение значения поля name третьего
    // экземпляра структуры:
sidorov.name="Сидоров С.С.";
    // Массив экземпляров структуры:
card fellows[]={ivanov,petrova,sidorov};
cout<<"Имена:\n";
    // Отображение значений поля name элементов
    // массива, состоящего из экземпляров структуры:
for(int i=0;i<3;i++){
cout<<fellows[i].name<<endl;
}
return 0;
}

```

Результат выполнения программы такой:

Результат выполнения программы (из листинга 15.1)

```

Первая запись:
Имя: Иванов И.И.
Телефон: 2345678
Пол: М
Вторая запись:
Имя: Петрова П.П.
Телефон: 8765432
Пол: Ж
Третья запись:
Имя: Иванов И.И.
Телефон: 2345678
Пол: М
Имена:
Иванов И.И.
Петрова П.П.
Сидоров С.С.

```


В этом примере описывается структура с названием `card`. У структуры `card` имеется несколько полей: поле `name` типа `string`, поле `phone` типа `int` и поле `gender` типа `char`. Также в программе описана функция `show()`, предназначенная для отображения значений полей экземпляра структуры, переданной аргументом функции. В главной функции программы создается три экземпляра (`ivanov`, `petrova` и `sidorov`) структуры `card`. В программе полям первых двух экземпляров присваиваются значения, после чего с помощью функции `show()` значения полей этих экземпляров структур отображаются в консольном окне.

В программе есть пример присваивания экземпляров структур: если одному экземпляру структуры в качестве значения присваивается другой экземпляр структуры (речь идет о команде `sidorov=ivanov`), выполняется побитовое копирование. Экземпляры структуры могут быть элементами массива. В программе командой `card fellows[]={ivanov,petrova,sidorov}` создается (и инициализируется экземплярами `ivanov`, `petrova` и `sidorov`) массив `fellows` (размер массива определяется автоматически по количеству экземпляров инициализации). После создания массива с помощью оператора цикла перебираются его элементы, и для каждого из этих элементов отображается значение поля `name`.

15.2. Альтернативное название для типа

А ты не путай свою личную шерсть с государственной.

из к/ф "Кавказская пленница"

С помощью оператора `typedef` создаются альтернативные названия для типов. Проще говоря, существует возможность создать альтернативу идентификаторам, используемым для обозначения типов. Синтаксис использования оператора `typedef` такой:

```
typedef старый_идентификатор новый_идентификатор;
```

Еще один способ создать альтернативное название для типа - воспользоваться ключевым словом `using`. Общая конструкция определения нового идентификатора для уже существующего типа такая:

```
using новый_идентификатор=старый_идентификатор;
```

В обоих случаях старый или исходный тип "не отменяется": его можно использовать, как и прежде.

Небольшой пример с применением инструкций `typedef` и `using` для создания альтернативных названий для идентификаторов типов (которые в свою очередь "конструируются" на основе обобщенного класса), представлен в листинге 15.2.

Листинг 15.2. Использование альтернативных названий для типов

```
#include <iostream>
#include <string>
using namespace std;
// Обобщенный класс:
template<class X> class MyClass{
private:
    // Поле:
    X value;
public:
    // Конструктор:
    MyClass(X v){
        value=v;
    }
    // Метод для отображения значения поля:
    void show(){
        cout<<"Значение поля: "<<value<<endl;
    }
}; // Завершение описания класса
// Создание альтернативных названий для типов:
typedef MyClass<int> MyInt;
using MyChar=MyClass<char>;
typedef MyClass<string> MyString;
// Главная функция программы:
int main(){
    // Первый объект:
    MyInt a(100);
    a.show();
    // Второй объект:
    MyChar b('Б');
    b.show();
    // Третий объект:
    MyString c("текст");
    c.show();
    // Четвертый объект:
    MyClass<int> d(200);
    d.show();
    // Присваивание объектов "разных" типов:
    a=d;
    a.show();
```

```
return 0;
}
```

Ниже представлен результат выполнения данной программы:

Результат выполнения программы (из листинга 15.2)

```
Значение поля: 100
Значение поля: Б
Значение поля: текст
Значение поля: 200
Значение поля: 200
```

В программе описывается обобщенный класс `MyClass`. В классе описано поле `value` обобщенного типа, конструктор с одним аргументом и метод `show()` для отображения значения поля `value`.

Наибольший интерес представляют команды `typedef MyClass<int> MyInt`, `typedef MyClass<string> MyString` и `using MyChar=MyClass<char>`. В результате вместо идентификаторов `MyClass<int>`, `MyClass<string>` и `MyClass<char>` можно использовать соответственно идентификаторы `MyInt`, `MyString` и `MyChar` (а можно не использовать). В главной функции программы приведены пример использования новых идентификаторов.



На заметку

Стоит обратить внимание, что базовые (исходные) идентификаторы разрешается использовать одновременно с новыми обозначениями для типов. Здесь еще раз уместно подчеркнуть, что рассмотренный выше механизм не подразумевает создание новых типов.

15.3. Перечисления

Три магнитофона, три кинокамеры зарубежных, три портсигара отечественных, куртка замшевая... Три куртки.

из к/ф "Иван Васильевич меняет профессию"

В некоторых случаях приходится иметь дело с переменными, которые в силу своей специфики могут принимать значения только из определенного набора. Например, предположим, что в программе нам приходится иметь дело с переменной, способной принимать только целочисленные значения, являющиеся *простыми числами*.

На заметку

Простое число не имеет никаких иных делителей, кроме единицы и самого себя. Другими словами, простое число делится только на единицу и на себя самого. Простыми являются числа 2, 5, 7, 11, 13, 17, 19, 23 и так далее.

Не прибегая к классам и объектам слишком многого в такой ситуации нам не добиться, но есть достаточно простой прием, позволяющий серьезно ограничить диапазон возможных значений переменной. Состоит прием в объявлении *перечисления*.

Перечисление разумно рассматривать как некое подобие типа. Чтобы понять суть этой "конструкции" имеет смысл начать с ее определения. Итак, объявление перечисления начинается с ключевого слова `enum`. Затем указывается идентификатор, обозначающий название перечисления. Если интерпретировать перечисление как тип, то данный идентификатор - название типа. Наконец, после названия типа в фигурных скобках через запятую перечисляются ключевые слова, определяющие возможные значения переменной, принадлежащей к типу перечисления. Пример объявления перечисления приведен ниже:

```
enum colors {red, green, blue, white, black};
```

Здесь объявляется перечисление `colors`. Переменная, относящаяся к данному типу, может принимать одно из следующих значений: `red`, `green`, `blue`, `white` и `black`. Важный момент состоит в том, что перечисленные идентификаторы (`red`, `green`, `blue`, `white` и `black`) нигде не объявляются - впервые они встречаются в объявлении перечисления `colors`. Поэтому естественным образом возникает вопрос: что это за идентификаторы и что с ними разрешается делать? Ответ простой и состоит в том, что `red`, `green`, `blue`, `white` и `black` - целочисленные константы. У этих констант есть значения. Мы их не присваиваем - они константам присваиваются автоматически. По умолчанию константы в списке объявления перечисления получают значения: значение первой константы равно 0, а значение каждой следующей константы на единицу больше, чем у ее предшественницы. В данном конкретном случае значением константы `red` будет 0, у константы `green` значение 1, у константы `blue` значение 2, у константы `white` значение 3 и, наконец, у константы `black` значение 4.

После объявления перечисления его имя можно использовать как тип данных. Например, командой `colors clr` объявляется переменная `clr` типа `colors`. Последнее означает, что переменная `clr` может принимать значения `red`, `green`, `blue`, `white` и `black`. Скажем, имеет смысл команда `clr=red` или `clr=black`. Но если мы попытаемся отобразить значение

переменной `clr` командой `cout<<clr`, то получим соответственно значение 0 или 4. То есть фактическое значение переменной `clr` -целое число. Но есть важное обстоятельство: хотя мы можем использовать, скажем, команду, `clr=red`, мы не сможем воспользоваться командой `clr=0` (хотя значение константы `red` равно 0). В этой особенности фактически кроется главное свойство перечислений.

Имеется возможность задавать в явном виде значения числовых констант, входящих в список допустимых значений переменной перечисления. Пример такой ситуации приведен ниже:

```
enum primes {two=2,three=3,five=5,seven=7};
```

Перечисление `primes`, объявляемое приведенной выше командой, определяется набором констант `two` (значение 2), `three` (значение 3), `five` (значение 5) и `seven` (значение 7).

Небольшой пример с использованием перечислений представлен в листинге 15.3.

Листинг 15.3. Перечисления

```
#include <iostream>
using namespace std;
// Перечисление (тип colors):
enum colors {red,green,blue,white,black};
// Перечисление (тип primes):
enum primes {two=2,three=3,five=5,seven=7};
// Главная функция программы:
int main(){
    // Переменные типа перечисления:
    colors clr;
    primes num;
    // Присваивание значений переменным типа перечисления:
    clr=blue;
    num=five;
    // Проверка значений переменных типа перечисления:
    cout<<"clr = "<<clr<<endl;
    cout<<"num = "<<num<<endl;
    // Операции со значениями из перечислений:
    cout<<"five + seven = "<<five+seven<<endl;
    cout<<"green + white = "<<green+white<<endl;
    cout<<"five + black = "<<five+black<<endl;
    // Сравнение значений:
    if(num==5) cout<<"five = 5"<<endl;
    int A,B;
```

```
// Целочисленным переменным присваиваются значения
// из перечислений:
A=blue;
B=two;
// Сравнение значений:
if (A==B) cout<<"blue = two"<<endl;
return 0;
}
```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 15.3)

```
clr = 2
num = 5
five + seven = 12
green + white = 4
five + black = 9
five = 5
blue = two
```

Часть команд из программного кода уже комментировались ранее. Остановимся на наиболее интересных моментах. Например:

- при вычислении выражения `five+seven` получаем сумму значений констант `five` и `seven` - то есть сумма 5 и 7;
- аналогично, при вычислении суммы `green+white` суммируются значения констант `green` (значение 1) и `white` (значение 3);
- имеет смысл выражение `five+black` (сумма значения 5 константы `five` и значения 4 константы `black`);
- хотя мы не можем использовать команду `num=5`, но если выполнена команда `num=five`, то значение выражения `num==5` будет `true`.

Фактически, константы из списков значений в объявлении перечислений могут использоваться как целочисленные константы в вычислениях. Например, если `A` и `B` - целочисленные переменные, то допустимы команды `A=blue` и `B=two`, причем понятно, что, учитывая значения констант `blue` и `two`, истинно условие `A==B`.



На заметку

В принципе, если проверить значение выражения `blue==two`, то получим значение `true`, поскольку у констант `blue` и `two` одинаковые числовые значения. Но компилятор при этом, скорее всего, выведет предупреждение о том, что сравниваются константы из разных перечислений.

15.4. Вызов конструктора в конструкторе

*Не мешайте работать, инвентаризуемый.
из к/ф "Гостя из будущего"*

Наиболее новый стандарт языка C++ разрешает такую операцию, как вызов конструктора в конструкторе. Классическая ситуация, при которой данный механизм может понадобиться - когда одна из версий конструктора определяется на основе другой версии конструктора. Формат вызова конструктора в конструкторе (для класса с названием `MyClass`) следующий:

```
MyClass(аргументы) : MyClass(аргументы) {
// команды
}
```

Рассмотрим небольшой пример, представленный в листинге 15.4.

Листинг 15.4. Вызов конструктора в конструкторе

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
private:
    // Целочисленное поле:
    int code;
    // Текстовое поле:
    string text;
public:
    // Конструктор с двумя аргументами:
    MyClass(int n, string txt){
        cout<<"Конструктор с двумя аргументами.\n";
        code=n;
        text=txt;
        // Отображение значения полей:
        show();
    }
    // Конструктор с целочисленным аргументом:
    MyClass(int n):MyClass(n, "Bravo"){
        cout<<"Конструктор с int-аргументом завершен.\n";
    }
    // Конструктор с текстовым аргументом:
    MyClass(string txt):MyClass(3, txt){
        cout<<"Конструктор со string-аргументом завершен.\n";
    }
}
```

```

    }
    // Конструктор без аргументов:
    MyClass():MyClass(1,"Alpha"){
    cout<<"Конструктор без аргументов завершен.\n";
    }
    // Метод для отображения значения полей:
    void show(){
    cout<<"Значения полей:\n";
    cout<<"Числовое: "<<code<<endl;
    cout<<"Текстовое: "<<text<<endl;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Первый объект (конструктор без аргументов):
    MyClass A;
    cout<<endl;
    // Второй объект (конструктор с int-аргументом):
    MyClass B(2);
    cout<<endl;
    // Третий объект (конструктор со string-аргументом):
    MyClass C("Charlie");
    cout<<endl;
    // Четвертый объект (конструктор с двумя аргументами):
    MyClass D(4,"Delta");
    return 0;
}

```

Ниже приведен результат выполнения данной программы:

Результат выполнения программы (из листинга 15.4)

```

Конструктор с двумя аргументами.
Значения полей:
Числовое: 1
Текстовое: Alpha
Конструктор без аргументов завершен.

```

```

Конструктор с двумя аргументами.
Значения полей:
Числовое: 2
Текстовое: Bravo
Конструктор с int-аргументом завершен.

```

```

Конструктор с двумя аргументами.
Значения полей:

```


Числовое: 3
Текстовое: Charlie
Конструктор со string-аргументом завершен.

Конструктор с двумя аргументами.
Значения полей:
Числовое: 4
Текстовое: Delta

В программе мы описываем класс `MyClass` с двумя полями: целочисленным и текстовым. В классе описан метод `show()`, которым значения полей отображаются в окне вывода. Основу кода класса составляет конструктор с двумя аргументами. Выполнение кода конструктора с двумя аргументами начинается с отображения сообщения соответствующего содержания. Также в конструкторе значениям полей присваиваются значения, после чего вызывается метод `show()` и значения полей отображаются в окне вода.

А еще в классе описаны такие версии конструкторов: с целочисленным аргументом, с текстовым аргументом и без аргументов. Во всех этих случаях сначала вызывается версия конструктора с двумя аргументами. Команды, описанные непосредственно в теле каждого из трех конструкторов, выполняются после выполнения кода конструктора с двумя аргументами. В последнем несложно убедиться, проанализировав результат выполнения программного кода: в главной функции программы представлены примеры создания объектов класса `MyClass` путем вызова различных конструкторов (без аргументов, с одним аргументом или с двумя аргументами).

В принципе, описанный подход достаточно удобен и нередко позволяет оптимизировать структуру программного кода.

15.5. Фабрика объектов

При моей впечатлительной натуре и тонкой организации, я должен был бы быть поэтом. А кто я? Я уборщик в институте времени. из к/ф "Гостья из будущего"

Есть одно важное свойство объектных переменных базовых классов - им в качестве значения можно присваивать объекты производных классов. А указатель на объект базового класса может ссылаться на объект производного класса. Эти, простые на первый взгляд, обстоятельства имеют далеко идущие последствия. Некоторые мы уже обсуждали. Здесь же рассмотрим один прием. Состоит он в том, чтобы описать функцию, генерирующую объекты нескольких классов. Все эти классы являются производными классами от определенного базового класса. Результатом функция возвращает

указатель на созданный объект. Но тип указателя определяется через базовый класс. Благодаря тому, что указатель базового класса может ссылаться на объект производного класса, такой подход возможен. Данного типа функции нередко называют "фабриками объектов".



На заметку

Проблема в том, что функция должна возвращать значение определенного типа. Мы же пытаемся создать функцию, которая фактически в зависимости от значения аргумента может возвращать результатом указатель на объекты разных классов. Для устранения этого "противоречия" тип результата функции определяется как указатель на объект базового класса.

В листинге 15.5 приведен пример программы с реализованным в ней описанным выше подходом. В частности, в программе описывается базовый класс `Base` виртуальным методом `show()`. Метод описан виртуальным, поскольку на основе класса `Base` путем наследования создается три производных класса (`Alpha`, `Bravo`, `Charlie`), и в каждом из этих классов метод `show()` переопределяется.

Подробности

Мы собираемся получать доступ к объектам производных классов через указатель базового класса. Если бы в базовом классе переопределяемый метод не был описан как виртуальный, то при обращении к объекту производного класса через указатель базового класса вызывалась бы версия метода из базового класса. Если же метод виртуальный, то вызывается версия, описанная в том классе, к которому относится объект.

Для того чтобы метод стал виртуальным, в его прототипе следует указать ключевое слово `virtual`.

У описанной в программе функции `obj_factory()` всего один целочисленный аргумент. В зависимости от значения аргумента при выполнении кода функции создается динамический объект класса `Base`, `Alpha`, `Bravo` или `Charlie`, и указатель на объект возвращается результатом функции. При этом возвращаемый указатель объявлен как указатель на объект базового типа.

Подробности

При описании функции `obj_factory()` использован оператор выбора `switch()`. Принцип выполнения оператора следующий. После ключевого слова `switch` в круглых скобках указывается выражение, значение которого проверяется в операторе. Вычисленное значение выражения проверяется на предмет совпадения со значениями, указанными после ключевых слов `case`. Как только совпадение найдено, начинают выполняться команды в соответствующем `case`-блоке. Коман-

ды выполняются до инструкции `break` (или если такой инструкции нет - до конца оператора выбора). Необязательный блок `default` выполняется в том случае, если при проверке `case`-инструкций совпадение найдено не было.

Весь программный код примера выглядит, как показано ниже:

Листинг 15.5. Фабрика объектов

```
#include <iostream>
using namespace std;
// Базовый класс:
class Base{
public:
    // Виртуальный метод:
    virtual void show(){
        cout<<"Объект класса Base\n";
    }
}; // Окончание описания базового класса
// Первый производный класс:
class Alpha:public Base{
public:
    // Переопределение метода из базового класса:
    void show(){
        cout<<"Объект класса Alpha\n";
    }
}; // Окончание описания первого производного класса
// Второй производный класс:
class Bravo:public Base{
public:
    // Переопределение метода из базового класса:
    void show(){
        cout<<"Объект класса Bravo\n";
    }
}; // Окончание описания второго производного класса
// Третий производный класс:
class Charlie:public Base{
public:
    // Переопределение метода из базового класса:
    void show(){
        cout<<"Объект класса Charlie\n";
    }
}; // Окончание описания третьего производного класса
// Функция "фабрика объектов":
Base *obj_factory(int n){
    // Указатель на объект базового класса:
    Base *t;
    // Оператор выбора:
```

```

switch(n){
case 1: // Если значение равно 1
        // Динамическое создание объекта класса Alpha:
t=new Alpha;
break;
case 2: // Если значение равно 2
        // Динамическое создание объекта класса Bravo:
t=new Bravo;
break;
case 3: // Если значение равно 3
        // Динамическое создание объекта класса Charlie:
t=new Charlie;
break;
default: // Для всех прочих значений n
        // Динамическое создание объекта класса Base:
t=new Base;
        }
// Результат функции - указатель
return t;
}
// Главная функция программы:
int main(){
    // Указатель на объект базового класса:
Base *pnt;
    // Создание разных объектов:
for(int i=0;i<=4;i++){
    // Создается динамический объект:
pnt=obj_factory(i);
    // Вызов метода из созданного объекта:
pnt->show();
    // Удаление объекта:
delete pnt;
}
return 0;
}

```

Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 15.5)

```

Объект класса Base
Объект класса Alpha
Объект класса Bravo
Объект класса Charlie
Объект класса Base

```

В главной функции программы командой `Base *pnt` объявляется ука-

затель `pnt` на объект базового класса `Base`. Затем запускается оператор цикла, в котором индексная переменная `i` пробегает значения от 0 до 4 включительно. При фиксированном значении индексной переменной в теле оператора цикла командой `pnt=obj_factory(i)` создается динамический объект, а указатель на это объект записывается в переменную `pnt`.



На заметку

Класс объекта, создаваемого при выполнении команды `pnt=obj_factory(i)`, зависит от значения аргумента `i`. Функция `obj_factory()` описана так, что при значении аргумента 1 создается объект класса `Alpha`, при значении аргумента 2 создается объект класса `Bravo`, а при значении 3 создается объект класса `Charlie`. При всех прочих значениях аргумента создается объект класса `Base`. Поскольку индексная переменная `i` принимает значения от 0 до 4, то последовательно создаются объекты классов `Base`, `Alpha`, `Bravo`, `Charlie` и снова `Base`.

Затем командой `pnt->show()` из созданного объекта вызывается метод `show()`. После вызова метода командой `delete pnt` динамический объект удаляется.

15.6. Динамическая идентификация типов

Каюсь, что, хоть не по собственной воле, а по принуждению князя Милославского, временно исполнял обязанности царя.

из к/ф "Иван Васильевич меняет профессию"

Хотя язык C++ характеризуется жесткой типизацией (каждая переменная должна быть объявлена с определенным типом), все же нередко случаются ситуации, когда в процессе выполнения программы необходимо определить тип какого-то объекта. Очень простой пример, сразу приходящий на ум - тот, что рассматривался выше, когда указатель на объект базового класса ссылался на объект производного класса. В этом случае, хотя тип переменной-указателя указан явно и однозначно, тип объекта, на который ссылается указатель, может быть разным. Причем по ходу выполнения программы указатель может "перебрасываться" с одного объекта на другой, и типы этих объектов в принципе могут различаться.

Функция `typeid()` позволяет определять типы объектов. Для использования функции в программе подключают заголовок `<typeinfo>`. Тестируемый объект передается аргументом функции. Результатом функции возвращается ссылка на объект класса `type_info`. Помимо объекта, аргументом функции `typeid()` может быть указан идентификатор, используемый для определения типа данных (например, ключевое слово `int` или название класса).

Объект класса `type_info` содержит описание типа объекта и имеет ряд полезных методов. Наиболее полезным, пожалуй, является метод `name()`, возвращающий результатом текст - название типа анализируемого объекта. Кроме этого, объекты класса `type_info` можно сравнивать на предмет равенства или неравенства (соответственно с помощью операторов `==` и `!=`). Объекты класса `type_info` классифицируются как равные, если они описывают один и тот же тип/класс.



На заметку

Возможность определять тип объекта производного класса через указатель базового класса существует, если базовый класс полиморфный: в таком базовом классе должен быть хотя бы один виртуальный метод. Если в базовом классе виртуальных методов нет, при попытке идентифицировать с помощью функции `typeid()` тип объекта производного класса на основе указателя базового класса получим объект класса `type_info`, описывающий базовый класс.

Небольшой пример, иллюстрирующий применение функции `typeid()` и отсутствующих утилит, представлен в листинге 15.6.



На заметку

Пример тестировался в среде разработки Microsoft Visual Studio Express 2013, поэтому содержит команду `system("chcp 1251")` для изменения кодировки консольного окна и команду `system("PAUSE")` для задержки консольного окна на экране. Результат выполнения программы приводится без учета сообщений, появляющихся в консольном окне при выполнении означенных команд. Функция `system()` доступна после подключения заголовка `<cstdlib>`.

Листинг 15.6. Динамическая идентификация типов

```
#include <iostream>
#include <cstdlib>
#include <string>
#include <typeinfo>
using namespace std;
// Базовый класс:
class Alpha{
public:
    // Виртуальный деструктор:
    virtual ~Alpha(){}
}; // Окончание базового класса
// Производный класс:
class Bravo:public Alpha{};
// Производный класс:
class Charlie:public Bravo{};
// Класс с внутренним классом:
```

```

class Delta{
public:
    // Внутренний класс:
    class Base{};
    // Результат метода - объект внутреннего класса:
    Base getObj(){
        Base obj;
    return obj;
    }
}; // Окончание класса
// Обобщенный класс:
template<class X> class MyClass{};
// Главная функция программы:
int main(){
    // Кодировка для консольного окна
    // (используется при работе с VisualStudioExpress):
    system("chcp 1251");
    // Целочисленная переменная:
    int number;
    // Символьная переменная:
    char symbol;
    // Текстовая переменная:
    string text;
    // Логическая переменная:
    bool test;
    // Объекты на основе обобщенного класса:
    MyClass<int> X;
    MyClass<double> Y;
    // Объект базового класса:
    Alpha A;
    // Объекты производных классов:
    Bravo B;
    Charlie C;
    // Объект класса с внутренним классом:
    Delta D;
    cout<<"Определяются типы переменных:\n";
    // Определяются типы переменных:
    cout<<"Тип переменной number: ";
    cout<<typeid(number).name()<<endl;
    cout<<"Тип переменной symbol: ";
    cout<<typeid(symbol).name()<<endl;
    cout<<"Тип переменной text: ";
    cout<<typeid(text).name()<<endl;
    cout<<"Тип переменной test: ";
    cout<<typeid(test).name()<<endl;
    cout<<"Тип переменной X: ";

```

```

cout<<typeid(X).name()<<endl;
cout<<"Тип переменной Y: ";
cout<<typeid(Y).name()<<endl;
cout<<"Тип переменной A: ";
cout<<typeid(A).name()<<endl;
cout<<"Тип переменной B: ";
cout<<typeid(B).name()<<endl;
cout<<"Тип переменной C: ";
cout<<typeid(C).name()<<endl;
cout<<"Тип переменной D: ";
cout<<typeid(D).name()<<endl;
cout<<"Тип выраженияD.getObj(): ";
cout<<typeid(D.getObj()).name()<<endl;
// Указатели:
int *n; // Указатель на целое число
char *s; // Указатель на символ
    // Указатели на объекты обобщенного класса:
MyClass<int> *x;
MyClass<double> *y;
// Указатель на объект базового класса:
Alpha *a;
    // Указатели на объекты производных классов:
Bravo *b;
Charlie *c;
    // Указатель на объект класса с внутренним классом:
Delta *d;
cout<<"Определяются типы указателей:\n";
    // Определяются типы указателей:
cout<<"Тип указателяn: ";
cout<<typeid(n).name()<<endl;
cout<<"Тип указателя s: ";
cout<<typeid(s).name()<<endl;
cout<<"Тип указателя x: ";
cout<<typeid(x).name()<<endl;
cout<<"Типуказателя y: ";
cout<<typeid(y).name()<<endl;
cout<<"Тип указателя a: ";
cout<<typeid(a).name()<<endl;
cout<<"Тип указателя b: ";
cout<<typeid(b).name()<<endl;
cout<<"Тип указателя c: ";
cout<<typeid(c).name()<<endl;
cout<<"Тип указателя d: ";
cout<<typeid(d).name()<<endl;
cout<<"Тип указателя &D.getObj(): ";
cout<<typeid(&D.getObj()).name()<<endl;

```



```
cout<<"Определение класса объекта по указателю:\n";
    // Указатель базового класса ссылается на объект
    // базового класса:
a=&A;
cout<<"Тип выражения *a (выполнена команда a=&A): ";
cout<<typeid(*a).name()<<endl;
// Указатель базового класса ссылается на объект
// производного класса:
a=&B;
cout<<"Типвыражения *a (выполнена команда a=&B): ";
cout<<typeid(*a).name()<<endl;
// Указатель базового класса ссылается на объект
// производного класса:
a=&C;
cout<<"Тип выражения *a (выполнена команда a=&C): ";
cout<<typeid(*a).name()<<endl;
cout<<"Определение типов по идентификаторам:\n";
// Целочисленный тип:
cout<<"Тип int: ";
cout<<typeid(int).name()<<endl;
// Базовый класс:
cout<<"ТипAlpha: ";
cout<<typeid(Alpha).name()<<endl;
// Производный класс:
cout<<"Тип Charlie: ";
cout<<typeid(Charlie).name()<<endl;
// Обобщенныйкласс:
cout<<"ТипMyClass<int>: ";
cout<<typeid(MyClass<int>).name()<<endl;
// Класс объекта, описывающего тип:
cout<<"Тип выраженияtypeid(int): ";
cout<<typeid(typeid(int)).name()<<endl;
// Задержка консольного окна
// (используется при работе с VisualStudioExpress):
system("PAUSE");
return 0;
}
```

Результат выполнения программы будет таким:

Результат выполнения программы (из листинга 15.6)

Определяются типы переменных:
 Тип переменной number: int
 Типпеременной symbol: char

```

Тип переменной text: class std::basic_
string<char,structstd::char_traits<char>,
classstd::allocator<char>>
Тип переменной test: bool
Тип переменной X: class MyClass<int>
Тип переменной Y: class MyClass<double>
Тип переменной A: class Alpha
Тип переменной B: class Bravo
Тип переменной C: class Charlie
Тип переменной D: class Delta
Тип выраженияD.getObj(): class Delta::Base
Определяются типы указателей:
Тип указателя n: int *
Тип указателя s: char *
Тип указателя x: class MyClass<int> *
Тип указателя y: class MyClass<double> *
Типуказателя a: class Alpha *
Тип указателя b: class Bravo *
Тип указателя c: class Charlie *
Тип указателя d: class Delta *
Тип указателя&D.getObj(): class Delta::Base *
Определение класса объекта по указателю:
Тип выражения *a (выполнена команда a=&A): class Alpha
Тип выражения *a (выполнена команда a=&B): class Bravo
Тип выражения *a (выполнена команда a=&C): class Charlie
Определение типов по идентификаторам:
Тип int: int
Тип Alpha: class Alpha
Тип Charlie: class Charlie
Тип MyClass<int>: class MyClass<int>
Тип выражения typeid(int): class type_info

```

Программный код примера очень однообразный, поэтому прокомментируем только основные моменты. Главная идея состоит в том, что переменные и объекты разных типов передаются аргументом функции `typeid()`, а из результата вызова этой функции вызывается метод `name()`. В результате мы получаем описание типа того или иного выражения.

Тестируется несколько переменных базовых типов. Также в программе описан обобщенный класс `MyClass`, базовый класс `Alpha`, производный от него класс `Bravo`, и производный от класса `Bravo` класс `Charlie`. Класс `Delta` описан с внутренним классом `Base`. Еще в классе `Delta` есть метод `getObj()`, возвращающий результатом объект внутреннего класса `Base`.

На что имеет смысл обратить внимание, исходя из результатов выполнения программы? Выделим наиболее интересные моменты:

- Класс `string` имеет достаточно нетривиальную структуру, судя по информации, отображаемой для переменной данного типа.
- Для объектов, созданных на основе обобщенного класса `MyClass`, в описание типа входит и параметр, передаваемый в класс: например, `MyClass<int>` или `MyClass<double>`.
- Внутренний класс `Base` указывается вместе с классом `Delta`, в котором он описан (выражение `Delta::Base`).
- Тип указателей отображается со звездочной `*` (например, `int*` или `Alpha*`).
- Отдельно стоит выделить выражение `typeid(typeid(int)).name()`. Его результатом является текст, описывающий тип объекта `typeid(int)`. Но значение выражения `typeid(int)` - объект класса `type_info`. Отсюда и результат выполнения соответствующей команды.

В программе есть фрагмент кода, и в нем указателю `a` базового класса `Alpha` (объявляется командой `Alpha *a`) последовательно присваиваются значениями адреса объектов `A`, `B` и `C` соответственно классов `Alpha`, `Bravo` и `Charlie` (имеются в виду команды `a=&A`, `a=&B` и `a=&C`). После выполнения очередной команды каждый раз проверяется тип выражения `*a` (объект, на который ссылается указатель). И хотя каждый раз функции `typeid()` передается указатель одного и того же типа, результатом получаем тип того объекта, на который указатель ссылается на момент вызова функции.

Подробности

Рекомендуется провести смелый эксперимент: в описании базового класса `Alpha` удалить описание деструктора или хотя бы ключевое слово `virtual` в его описании. Результат может оказаться неожиданным: при проверке типа выражения `*a` там, где раньше последовательно появлялись названия классов `Alpha`, `Bravo` и `Charlie`, теперь все три раза будет отображаться название класса `Alpha`. Другими словами, теперь по указателю `a` мы не можем определить тип объекта, на который этот указатель ссылается. Причина в том, что базовый класс (в новой редакции) не содержит виртуальных методов (то есть можно было не описывать деструктор с ключевым словом `virtual` или совсем не описывать деструктор, но все же хоть один виртуальный метод в классе должен быть). Что касается непосредственно виртуальных деструкторов (деструкторов, описанных с ключевым словом `virtual`), то они в системе наследования играют далеко не последнюю роль.

15.7. Виртуальные деструкторы

*Преступление века зашло в тупик.
из к/ф "Старики-разбойники"*

Деструктор, как и обычный метод, можно сделать виртуальным. Формально задача решается очень просто: в описании деструктора перед его именем указывается ключевое слово `virtual`. Но чтобы понять разницу между виртуальным и не виртуальным (обычным) деструктором, рассмотрим небольшой пример, представленный в листинге 15.7. Сначала исследуем "правильный" программный код с виртуальным деструктором. Потом попытаемся выяснить, что произойдет, если деструктор будет не виртуальным.

Прямбула такая: в программе описан вспомогательный класс `MyClass` с текстовым полем, конструктором и деструктором.



На заметку

Деструктор у класса обычный, не виртуальный. Вообще, о виртуальном деструкторе имеет смысл говорить, если в программе используется наследование и доступ к объектам производных классов планируется получать, например, через указатели базовых классов. Поскольку класс `MyClass` не является базовым и не задействован в схеме наследования, то нет потребности описывать в нем виртуальный деструктор.

В конструкторе текстовому полю присваивается значение и выводится сообщение о создании объекта. В деструкторе выводится сообщение об удалении объекта с соответствующим полем. Здесь важно то, что при создании и удалении объекта класса `MyClass` в окне вывода по этому поводу появляется сообщение.

Класс `MyClass` используется в описании классов `Alpha` и `Bravo`. Причем класс `Bravo` является производным от класса `Alpha`. В классе `Alpha` есть поле `A`, являющееся указателем на объект класса `MyClass`. В конструкторе создается динамический объект класса `MyClass` и адрес объекта записывается в поле `A`. В классе `Alpha` описан виртуальный деструктор. В нем удаляется динамический объект, адрес которого записан в указателе `A`.

Класс `Bravo` помимо наследуемого поля из класса `Alpha`, имеет еще одно поле `B`, также являющееся указателем на объект класса `MyClass`. В конструкторе класса `Bravo` сначала вызывается конструктор класса `Alpha`, а затем создается динамический объект класса `MyClass` и его адрес записывается в поле `B`. Деструктор класса `Bravo` содержит команду удаления объекта с адресом в указателе `B`.

В главной функции программы сначала создается динамический объект класса `Bravo` и его адрес записывается в указатель класса `Bravo`. Сразу по-

сле создания объект удаляется. Затем снова создается динамический объект класса `Bravo`, но теперь адрес объекта записывается в указатель базового класса `Alpha`. И этот объект тоже удаляется. Далее рассмотрим программный код:

Листинг 15.7. Виртуальный деструктор

```
#include <iostream>
#include <string>
using namespace std;
// Вспомогательный класс:
class MyClass{
private:
    // Текстовое поле:
    string name;
public:
    // Конструктор:
    MyClass(string txt){
        name=txt;
        cout<<"Создан объект с полем "<<name<<endl;
    }
    // Деструктор:
    ~MyClass(){
        cout<<"Удален объект с полем "<<name<<endl;
    }
}; // Окончание описания вспомогательного класса
// Базовый класс:
class Alpha{
protected:
    // Поле - указатель на объект класса MyClass:
    MyClass *A;
public:
    // Конструктор:
    Alpha(string txt){
        // Создание динамического объекта:
        A=new MyClass(txt);
    }
    // Виртуальный деструктор:
    virtual ~Alpha(){
        // Чтобы деструктор перестал быть виртуальным, нужно
        // закомментировать предыдущую строку кода и отменить
        // комментирование следующей строки:
        //~Alpha(){
        delete A; // Удаление динамического объекта
    }
}; // Окончание описания базового класса
```

```

// Производный класс:
class Bravo:public Alpha{
protected:
// Указатель на объект класса MyClass:
MyClass *B;
public:
    // Конструктор:
    Bravo(string t1,string t2):Alpha(t1){
// Создание динамического объекта:
B=new MyClass(t2);
    }
    // Деструктор:
~Bravo(){
    // Удаление динамического объекта:
delete B;
}
}; // Окончание описания производного класса
// Главная функция программы:
int main(){
cout<<"Начало выполнения программы.\n";
cout<<"Использование указателя производного класса:\n";
    // Создание динамического объекта производного класса.
    // Адрес объекта записывается в указатель
    // производного класса:
    Bravo *p=new Bravo("Альфа","Браво");
// Удаление объекта производного класса через указатель
// производного класса:
delete p;
cout<<"Использование указателя базового класса:\n";
    // Создание динамического объекта производного класса.
    // Адрес объекта записывается в указатель
    // базового класса:
    Alpha *q=new Bravo("Альфа","Браво");
// Удаление объекта производного класса через указатель
// базового класса:
delete q;
cout<<"Завершение выполнения программы.\n";
return 0;
}

```

Чего мы ожидаем от работы программы? В главной функции выполняются однотипные действия: объект создается, удаляется, создается и удаляется. При создании объекта класса `Bravo` создается два динамических объекта класса `MyClass`. При создании каждого из объектов появляется сообщение. При удалении объекта класса `Bravo` удаляются два динамических объекта класса `MyClass`. Отсюда еще два сообщения.

Таким образом, можно ожидать, что появится два сообщения о создании объектов, затем два сообщения об удалении этих объектов, снова два сообщения о создании объектов и два сообщения об удалении объектов. В реальности если в классе `Alpha` деструктор *виртуальный* (описан с ключевым словом `virtual`), то все именно так и происходит:

Результат выполнения программы (из листинга 15.7)

```
Начало выполнения программы.
Использование указателя производного класса:
Создан объект с полем Альфа
Создан объект с полем Bravo
Удален объект с полем Bravo
Удален объект с полем Альфа
Использование указателя базового класса:
Создан объект с полем Альфа
Создан объект с полем Bravo
Удален объект с полем Bravo
Удален объект с полем Альфа
Завершение выполнения программы.
```

Одна из инструкций (второе сообщение об удалении объекта с полем `Bravo`) выделена жирным шрифтом. Это не случайно. Модифицируем программный код, сделав деструктор в классе `Alpha` *не виртуальным* (для этого следует удалить ключевое слово `virtual`). После удаления ключевого слова `virtual` код деструктора должен выглядеть следующим образом:

```
~Alpha() {
delete A;
}
```

Так вот в данном случае одна строка в области вывода "пропадет". Это именно та строка, что была выделена жирным шрифтом. Ниже приведен результат выполнения программы в случае, если деструктор базового класса `Alpha` не является виртуальным:

Результат выполнения программы (из листинга 15.7)

```
Начало выполнения программы.
Использование указателя производного класса:
Создан объект с полем Альфа
Создан объект с полем Bravo
Удален объект с полем Bravo
Удален объект с полем Альфа
Использование указателя базового класса:
```

Создан объект с полем Альфа
 Создан объект с полем Bravo
 Удален объект с полем Альфа
 Завершение выполнения программы.

Основные выводы очевидны:

- Если речь идет о записи адреса объекта класса `Bravo` в указатель класса `Bravo`, то не имеет значения, виртуальный деструктор в базовом классе `Alpha` или нет.
- Если адрес объекта класса `Bravo` записывается в указатель базового класса `Alpha`, то процесс создания объекта особенностей не имеет, а вот при удалении объекта в случае не виртуального деструктора не выполняется деструктор производного класса `Bravo`. В результате в окне вывода нет соответствующего сообщения.

Попробуем пояснить происходящее. Начнем с ситуации, когда динамически создается объект класса `Bravo` и адрес объекта записывается в указатель класса `Alpha`. При создании объекта вызывается конструктор класса `Bravo`, в котором есть явная инструкция вызова конструктора класса `Alpha`. Поэтому создание объекта происходит "штатно". А вот когда объект удаляется, то удаляется он через указатель класса `Alpha`. Указатель такого типа "находит" методы (в том числе и деструктор), описанные в классе `Alpha`. Чтобы указатель вместо "базовых" методов находил их переопределенные в производном классе версии, методы следует сделать виртуальными. Деструктор в этом смысле не является исключением. Следовательно, если деструктор в классе `Alpha` не является виртуальным, то он собственно и вызывается, а деструктор в производном классе игнорируется. Если же деструктор в классе `Alpha` виртуальный, то вызывается деструктор класса `Bravo`, ну а деструктор класса `Alpha` вызывается автоматически по завершении выполнения кода деструктора класса `Bravo`.

Ну и учитывая все вышеозначенное, можно надеяться, что процесс создания и удаления объектов с использованием указателя класса `Bravo` особой интриги не содержит.

15.8. Цикл по коллекции

*Замуровали, демоны!
 из к/ф "Иван Васильевич меняет профессию"*

В стандарте C++11 появилась довольно эффектная разновидность оператора цикла, которую иногда называют *циклом по коллекции*. В качестве коллек-

ции может выступать массив или объект класса, поддерживающий методы `begin()` и `end()` и операции с итераторами. Шаблон объявления оператора цикла в таком формате выглядит примерно следующим образом:

```
for (тип &переменная:коллекция) {
// команды
}
```

После ключевого слова `for` в круглых скобках указывается:

- тип переменной (совпадает с типом элементов коллекции);
- обычно амперсанд `&` (если мы собираемся переменную коллекции использовать для присваивания элементам коллекции значений);
- имя переменной (формальное обозначение для элемента коллекции);
- через двоеточие - название коллекции (например, вектор или массив).

Пример использования оператора цикла данного типа приведен в программе в листинге 15.8.

Листинг 15.8. Цикл по коллекции

```
#include<iostream>
#include <vector>
#include <cstdlib>
using namespace std;
// Главная функция программы:
int main(){
// Инициализация генератора случайных чисел:
srand(2015);
    // Размер массива:
const int n=10;
    // Статический массив:
int  nums[n];
cout<<"Массив:\n";
// Цикл по массиву:
for(int  &v: nums){
    // Присваивается значение элементу:
v=rand()%10;
    // Отображается значение элемента:
cout<<v<<" ";
}
cout<<endl;
```

```

cout<<"Проверка:\n";
// Обычный цикл:
for(int i=0;i<n;i++){
// Отображение значения элемента:
cout<<nums[i]<<" ";
}
cout<<endl;
// Размер вектора:
int m=12;
cout<<"Вектор:\n";
// Создание вектора:
vector<char>A(m);
// Цикл по вектору:
for(char &s: A){
// Значение элемента вектора:
s='A'+rand()%m;
// Отображение значения элемента:
cout<<s<<" ";
}
cout<<endl;
cout<<"Проверка:\n";
// Обычный цикл:
for(int i=0;i<m;i++){
// Отображение значения элемента:
cout<<A[i]<<" ";
}
cout<<endl;
return 0;
}

```

Ниже приведен возможный результат выполнения программы:

Результат выполнения программы (из листинга 15.8)

```

Массив:
8 0 1 2 0 0 6 6 4 0
Проверка:
8 0 1 2 0 0 6 6 4 0
Вектор:
К К В Е Е G В D E К Н F
Проверка:
К К В Е Е G В D E К Н F

```

В одном из циклов, который объявляется с заголовком `for(int &v: nums)`, выполняется перебор массива `nums`. Целочисленная переменная `v` является локальной ссылкой на элемент массива `nums`. При выполнении оператора

цикла переменная последовательно "перебрасывается" на элементы массива.

Аналогичное объявление использовано в операторе с заголовком `for(char &s: A)`. Предварительно вектор `A` с символьными элементами создан командой `vector<char> A(m)`. Символьная ссылочная переменная `s` в операторе цикла последовательно указывает на элементы вектора `A`. Поскольку речь идет именно о ссылке, то значения элементов удастся не только считывать, но и присваивать.

15.9. Автоматическое определение типа

*Баранов - в стойло, холодильник - в дом.
из к/ф "Кавказская пленница"*

Стандарт C++11 существенно расширяет возможности по использованию ключевого слова `auto`. Идентификатор `auto`, кроме прочего, может использовать для замещения идентификатора типа переменной. При этом реальный тип определяется на основе значения (переменной или выражения, через которое определяется значение переменной). Небольшой пример использования данного ключевого слова приведен в программе в листинге 15.9.



На заметку

Пример тестировался в среде разработки Microsoft Visual Studio Express 2013. Команда `system("chcp 1251")` использована для определения кодировки консольного окна, команда `system("PAUSE")` используется для задержки консольного окна, в шапке программы подключается заголовок `<cstdlib>`, а результат выполнения программы приводится без учета выполнения указанных команд.

Листинг 15.9. Автоматическое определение типа

```
#include<iostream>
#include <cstdlib>
#include <typeinfo>
#include <vector>
using namespace std;
// Главная функция программы:
int main() {
    // Кодовая таблица для консольного кода
    // (используется при работе с Visual Studio Express):
    system("chcp 1251");
    // Автоматическая идентификация типа переменных:
    auto A=100;
    auto S='Ю';
```

```

auto B=A+S;
auto X="текст";
cout<<"Тип переменной A: "<<typeid(A).name()<<endl;
cout<<"Тип переменной B: "<<typeid(B).name()<<endl;
cout<<"Тип переменной S: "<<typeid(S).name()<<endl;
cout<<"Тип переменной X: "<<typeid(X).name()<<endl;
// Вектор из 10-ти букв 'Я':
vector<char> V(10, 'Я');
cout<<"Содержимое вектора:\n";
    // Автоматическая идентификация типа
    // для переменной цикла:
for(auto &p: V){
cout<<p<<" ";
}
cout<<endl;
    // Задержка консольного окна
    // (используется при работе с VisualStudioExpress):
system("PAUSE");
return 0;
}

```

Результат выполнения программы представлен ниже:

Результат выполнения программы (из листинга 15.9)

```

Тип переменной A: int
Тип переменной B: int
Тип переменной S: char
Тип переменной X: char const *
Содержимое вектора:
я я я я я я я я я я

```

Программа содержит несколько небольших и простых иллюстраций к объявлению переменных с идентификатором `auto`. Еще приведен пример использования идентификатора `auto` в операторе цикла по коллекции. Коллекцией в данном случае является вектор (объект класса `vector<char>`) с символьными элементами. В этом случае тип переменной цикла определяется автоматически на основе типа элементов вектора.

Подробности

Объявление переменной с идентификатором `auto` не означает, что данная переменная может относиться к любому типу или что тип переменной может быть определен в процессе выполнения программы. Например, некорректной является команда `auto z`, поскольку в данном случае невозможно определить тип переменной `z`. Другим словами, использование инструкции `auto` не отменяет требования для переменной иметь определенный тип.

15.10. Особенности перегрузки оператора присваивания

*Тот, кто нам мешает, тот нам и поможет.
из к/ф "Кавказская пленница"*

В некоторых примерах книги мы перегружали оператор присваивания. Напомним, что речь идет об операторном методе `operator=()`. Вместе с тем, есть один прием, который неплохо иметь в виду при описании данного метода. Это проверка на предмет *присваивания объекта самого себе*.

В принципе, обычно ничего страшного не происходит, если объект сам себе присваивается, да и подобные ситуации встречаются не очень часто. С другой стороны, каждое новое поколение программистов становится все более изобретательным, так что гарантий в столь пикантном деле быть не может. Как бы там ни было, иногда неплохо подстраховаться. Рассмотрим по этому поводу небольшой пример.

Создадим класс `MyClass` с полем-указателем `n` на целое число. В конструкторе класса динамически выделяется память для целочисленного значения, а адрес ячейки памяти заносится в поле-указатель. В классе описан метод `get()`, позволяющий получить значение, на которое указывает поле `n`. Методом `show()` значение, на которое ссылается поле `n`, отображается в окне вывода. Данный метод мы используем для проверки "содержимого" объектов класса `MyClass`. Но самое интересное "происходит" в теле операторного метода `operator=()`, определяющего процедуру присваивания объектов класса `MyClass`. Метод возвращает объект класса `MyClass`, причем это именно тот объект, из которого вызывается операторный метод - а проще говоря, объект слева от оператора присваивания (тот объект, которому присваивается значение). Аргумент (объект класса `MyClass`) операторному методу передается по ссылке. В данном случае такой подход - необходимость, поскольку мы в классе не описываем конструктор создания копии, а в классе при этом используется динамическое выделение памяти.

В теле операторного метода сначала удаляется текущее значение - причем удаляется освобождением памяти. Затем память снова выделяется, и туда записывается копия значения, "хранящегося" в присваиваемом объекте. Понятно, что процесс освобождения и повторного выделения памяти выглядит несколько надуманно, но здесь он служит цели благородной: предупредить нас о потенциальной опасности.

Итак, если объект, которому присваивается значение, отличается от присваиваемого объекта, особых проблем не возникает. Но если объект присваивается сам себе, то происходит следующее: освобождаемая память формально

в объекте, которому присваивается значение, освобождается и в присваиваемом объекте. Далее, когда память вновь выделяется в объекте, которому присваивается значение, одновременно она выделяется для присваиваемого объекта. И только затем предпринимается попытка прочесть значение в присваиваемом объекте - то есть значение в области памяти, которая только что выделена. Значение, записанное в выделенной ячейке - некое случайное число, "оставшееся" от предыдущих вычислений. Поэтому после присваивания самого себе объект получает (через поле-указатель) случайное число. Сказанное иллюстрирует приведенный ниже программный код:

Листинг 15.10. Операторный метод для оператора присваивания без проверки на присваивание объекта самого себе

```
#include <iostream>
using namespace std;
// Класс:
class MyClass{
private:
// Поле-указатель:
int *n;
// Значение, на которое ссылается поле:
int get(){
return *n;
}
public:
// Конструктор:
MyClass(int val){
// Динамическое выделение памяти:
n=new int;
// Значение, на которое ссылается поле:
*n=val;
}
// Метод для отображения значения, на которое
// ссылается поле:
void show(){
cout<<"Число: "<<get()<<endl;
}
// Операторный метод для оператора присваивания:
MyClass operator=(MyClass &obj){
// Удаление текущего значения:
delete n;
// Динамическое выделение памяти:
n=new int;
// В выделенную память записывается значение:
*n=obj.get();
// Результат метода:
```

```
return *this;
}
}; // Окончание описания класса
// Главная функция программы:
int main() {
    // Первый объект:
    MyClass A(10);
    // Второй объект:
    MyClass B(20);
    // Проверка значений:
    A.show();
    B.show();
    // Присваивание объектов:
    A=B;
    // Проверка результата присваивания:
    A.show();
    // Объект присваивается сам себе:
    A=A;
    // Проверка результата присваивания
    // объекта самого себе:
    A.show();
    return 0;
}
```

Результат выполнения программы может быть таким (последнее выводимое числовое значение, выделенное жирным шрифтом, фактически является случайным):

Результат выполнения программы (из листинга 15.10)

```
Число: 10
Число: 20
Число: 20
Число: 3347800
```

Проблему легко решить, если добавить в тело операторного метода `operator=()` проверку присваивания объекта самому себе. Для этого первой инструкцией в теле операторного метода добавляется инструкция `if(this==&obj) return *this`. В команде проверяется равенство адреса объекта, из которого вызывается метод (адресобъекта возвращается указателем `this`) и адреса объекта `obj`, переданного аргументом методу (указатель на объект `&obj`). Если адреса совпадают, то никакие операции не выполняются и объект `*this` сразу возвращается как результат операторного метода. Измененный код программы приведен в листинге 15.11 (для

сокращения объема кода комментарии удалены, а добавленный фрагмент кода выделен жирным шрифтом).

Листинг 15.11. Операторный метод для оператора присваивания с проверкой на присваивание объекта самого себе

```
#include <iostream>
using namespace std;
class MyClass{
private:
int *n;
int get(){
return *n;
}
public:
MyClass(int val){
n=new int;
*n=val;
}
void show(){
cout<<"Число: "<<get()<<endl;
}
MyClassoperator=(MyClass&obj){
// Проверка на присваивание объекта
// самого себе:
if(this==&obj) return *this;
delete n;
n=new int;
*n=obj.get();
return *this;
}
};
int main(){
MyClass A(10);
MyClass B(20);
A.show();
B.show();
A=B;
A.show();
A=A;
A.show();
return 0;
}
```

Теперь результат выполнения программы совершенно однозначен:

Результат выполнения программы (из листинга 15.11)

Число: 10
Число: 20
Число: 20
Число: 20

Как видим, в данном случае операция "самоприсваивания" к катастрофическим последствиям не приводит.

15.11. Перегрузка оператора приведения типа

*Артист обязан себя искать. И время от времени переодеваться.
из к/ф "Покровские ворота"*

Есть такая операция -*приведение типов*. Это когда значение одного типа "насильственно" или не очень приводится к значению другого типа. Приведение может быть *явным* или *неявным*. При явном приведении перед приводимым значением в круглых скобках указывается идентификатор типа, к которому выполняется приведение. При неявном приведении подобная процедура выполняется автоматически исходя из контекста всего выражения.

Подробности

Например, значением выражения `(int) 'A'` есть число 65. Здесь выполняется явное приведение символьного значения 'A' к типу `int`. Результатом приведения является код символа (число 65). Наоборот, результатом выражения `(char) 65`, которым выполняется приведение целочисленного значения 65 к типу `char`, является символ 'A' (символ с кодом 65).

Ситуация бывает не столь очевидной. Так, результатом выражения `5+'A'` является значение 70. Причина в том, что при вычислении выражения `5+'A'` символ 'A' неявно приводится к целочисленному типу (получаем значение 65) и суммируется со значением 5 (числа 5 и 65 в сумме дают 70). Но если бы была объявлена символьная переменная `char s` и затем выполнена команда `s=5+'A'`, то переменная `s` получила бы значение 'F'. Это результат неявного приведения числа 70 к типу `char`.

В классе можно описывать операторы приведения к различным типам. Мы рассмотрим наиболее простой случай, когда объекты класса будут приводиться к типу `int` и типу `string`.



На заметку

Вообще, если речь идет о приведении объектов класса к некоторому типу, то соответствующий операторный метод называется `operator тип()`. Тип результата для такого оператора не указывается, поскольку он содержится в параметре `тип`.

Например, оператор приведения к типу `int` называется `operator int()`, а оператор приведения к типу `string` называется `operator string()`. Оператор `operator int()` результатом должен возвращать значение типа `int`, а оператор `operator string()` должен возвращать результатом значение типа `string`.

В программе, представленной в листинге 15.2, описан класс `MyClass`. У класса есть два поля: целочисленное и текстовое. Конструктору передается два аргумента: целое число и текст (значения полей). Оператор приведения к целочисленному типу описан так, что результатом возвращается значение целочисленного поля. Оператор приведения к текстовому типу возвращает результатом значение текстового поля. В главной функции программы создается несколько объектов и с их помощью проверяются операции по явному и неявному приведению типов. Код примера представлен ниже:

Листинг 15.12. Перегрузка оператора приведения типа

```
#include <iostream>
#include <string>
using namespace std;
// Класс:
class MyClass{
public:
    // Целочисленное поле:
    int num;
    // Текстовое поле:
    string name;
    // Конструктор:
    MyClass(int n,string s){
        num=n;
        name=s;
    }
    // Оператор приведения к типу int:
    operator int(){
        return num;
    }
    // Оператор приведения к типу string:
    operator string(){
        return name;
    }
}; // Окончание описания класса
// Главная функция программы:
int main(){
    // Создание объектов:
    MyClass A(100,"Объект A");
    MyClass B(200,"Объект B");
    // Явное приведение к типу string:
```

```
cout<<(string)A<<endl;
    // Явное приведение к типу int:
cout<<(int)A<<endl;
    // Неявное приведение к типу int:
cout<<1+A<<endl;
cout<<A+B<<endl;
// Создание объекта.
    // Аргументы конструктора - объекты:
MyClass C(A,B);
    // Явное приведение к типу string:
cout<<(string)C<<endl;
// Явное приведение к типу int:
cout<<(int)C<<endl;
return 0;
}
```

Результат выполнения программы приведен ниже:

Результат выполнения программы (из листинга 15.12)

```
Объект A
100
101
300
Объект B
100
```

Командами `MyClass A(100,"Объект A")` и `MyClass B(200,"Объект B")` создается два объекта. Значения их полей - аргументы конструктора. Операция явного приведения типа иллюстрируется на примере объекта `A`. Так, значением выражения `(string)A` является значение текстового поля объекта `A` (значение "Объект A"). Соответственно, значение выражения `(int)A` есть значение целочисленного поля объекта `A` (значение 100).

При вычислении значений выражений `1+A` и `A+B` объекты `A` и `B` неявно приводятся к целочисленному типу. Отсюда значение выражения `1+A` равно 101, а значение выражения `A+B` равняется 300.

Носамая интересная, пожалуй, команда создания объекта `MyClass C(A,B)`. В этой команде аргументом конструктору класса `MyClass` передаются объекты `A` и `B`, хотя на их месте должны быть, соответственно, значения типа `int` и `string`. Именно поэтому первый объект `A` неявно приводится к типу `int` (получаем значение 100), а второй объект `B` неявно приводится к типу

`string` (получаем значение "Объект В"). Как результат, поля созданного объекта `C` получают значения `100` и "Объект В". Значения полей объекта `C` проверяем с помощью инструкций явного приведения типа `(string)C` и `(int)C`.

Глава 16.

ЗАКЛЮЧЕНИЕ



Дорогой Ватсон! Вы в совершенстве познали мой дедуктивный метод. Но, увы, Ваши выводы в большинстве ошибочны.

из к/ф "Приключения Шерлока Холмса и доктора Ватсона"

Как правило, книга - это не цель, а лишь средство в достижении цели (хотя, конечно, бывают и исключения). Поэтому естественным образом после того, как книга прочитана и программные коды разобраны и проанализированы, возникает вопрос: а что же дальше? Вот об этом и поговорим. Немного.

16.1. О языках программирования

Вот ведь, на всех языках говоришь, а по-русски не понимаешь.

из к/ф "Покровские ворота"

Очень редко практикующие программисты ограничиваются одним-единственным языком программирования. Если читатель планирует профессионально (или не очень профессионально) заниматься программированием, то он, помимо C++, рано или поздно столкнется и с другими языками (а может уже и сталкивался). Поэтому хорошо иметь хотя бы общее представление о том, что есть в "меню".

Языков программирования очень много, на самый разнообразный "вкус". Обычно тот или иной язык реализует определенную концепцию. В известном смысле фундаментальной является парадигма ООП. Среди языков, поддерживающих ООП, есть "три кита": C++, Java и C#. Данные языки очень похожи, но в то же время они очень разные. Похожи языки в плане синтаксиса. Отличаются "идеологией", так сказать (при том, что в каждом реализован объектно-ориентированный подход). Хорошая новость: владея навыками программирования в C++, несложно освоить языки программирования Java и C#. Даже в этом смысле язык программирования C++ очень перспективен.

**На заметку**

Что из этого следует? А следует из этого то, что перед читателем открывается замечательная перспектива, которая держится на трех столпах, имя которым C++, C# и Java. Как этой перспективой воспользуется читатель - вопрос другой. Но возможность есть, и о ней нужно знать.

Тот или иной язык программирования выбирают в зависимости от решаемой задачи. Языки C++, C# и Java перекрывают широкий сектор востребованных на сегодня программных средств разработки. Поэтому после изучения языка C++ (хорошо бы на профессиональном уровне) вполне разумный шаг - изучение языков C# и Java. С какого из них начать не очень принципиально.

**На заметку**

Если исходить из популярности языков программирования, то язык Java выглядит все же более предпочтительным.

Что касается концепции ООП, то она хоть и популярна, но не лишена недостатков. У объектно-ориентированного подхода есть немало критиков. С другой стороны, серьезной альтернативы объектно-ориентированному подходу пока что нет. В любом случае связка из C++ и ООП - это мощный задел на будущее. Но и потрудиться придется.

16.2. Приложения с графическим интерфейсом

*Валя, пойдем грабить музей!
из к/ф "Старики-разбойники"*

В книге мы рассматривали многие полезные темы, но все же есть одна, которая заставляет сердце биться чаще у любого новичка в программировании (да и не только новичка). Эта тема: создание приложений с графическим интерфейсом. В книге она не обсуждалась. Причина простая: в стандарт языка C++ средства для разработки приложений с графическим интерфейсом не входят.

Что имеется в виду? Вот, например, в Java есть специальные библиотеки классов AWT и Swing, предназначенные для создания приложений с графическим интерфейсом. В C++ такого богатства нет (пока во всяком случае). Но это совсем не означает, что в C++ нельзя создавать приложения с кнопками, полями ввода, переключателями и прочими полезностями - то есть с графическим интерфейсом. Скорее наоборот. Просто кроме C++ по-

надобится *еще что-то*. Здесь возможны варианты, но на сегодня наиболее популярным является подход, основанный на применении *библиотеки Qt*.



На заметку

Желающие могут воспользоваться адресом www.qt.io страницы проекта Qt для получения более подробной информации о проекте и доступных программных продуктах.

Библиотека Qt не является частью стандарта языка C++. Это своеобразная надстройка, которая в комбинации с C++ позволяет достаточно легко, быстро и эффективно создавать приложения с графическим интерфейсом. Вместе с тем, "идеология", реализованная в библиотеке, близка по духу к концепции ООП, что дает неплохие шансы на успех для тех, кто познал премудрости объектно-ориентированного подхода. Но в любом случае, овладение средствами разработки приложений с графическим интерфейсом - хорошее направление для профессионального роста.

16.3. Программирование и жизнь

Надеюсь, нам повезет, и мы встретим то, что единственное украшает жизнь - настоящую неожиданность.

из к/ф "Приключения принца Флоризеля"

Мы живем в прагматичном и во многом циничном мире. Реалии таковы, что мало быть хорошим профессионалом (в программировании в данном случае), необходимо еще и уметь найти свое "место под солнцем" - то есть на рынке труда, как банально бы это ни звучало. И кроме множества всех прочих моментов, важно быть тренде - обладать теми профессиональными навыками, которые востребованы на рынке на данный момент и будут востребованы в будущем. Целесообразно соотносить свой профессиональный рост с тенденциями рынка программных продуктов. Здесь необходимо *планирование*. Но данная задача не из легких.

Жизнь нередко вносит коррективы в планы людей. С этим ничего не поделаешь. Будущее всегда несет в себе некую долю неопределенности. Сказанное относится и к программированию. Меняются концепции, появляются новые языки программирования, некоторые из них становятся популярными, а некоторые в прошлом популярные языки теряют свою актуальность. Во всем этом несложно запутаться. Нужен "ориентир".

Понятно, что универсальных рецептов нет, но кое-что посоветовать все же можно. Во-первых, актуальность языков программирования удастся оце-

нить по объявлениям, которые размещают работодатели в области программных технологий. Работа по анализу таких объявлений кропотливая, но в принципе полезная. Во-вторых, сейчас довольно много статистической информации о популярности (по годам) разных языков программирования. На основе этих сведений легко спрогнозировать популярность того или иного языка программирования на ближайшие несколько лет.

В завершение же хочется напомнить одну простую и непреложную истину - для того, чтобы научиться программировать, необходимо *постоянно практиковаться*. Это и есть главный закон успеха.

Для заметок

Для заметок

Для заметок

Для заметок



Для заметок

Для заметок

Для заметок

Для заметок

Группа подготовки издания:

Зав. редакцией компьютерной литературы: *М. В. Финков*

Редактор: *Е. В. Финков*

Корректор: *А. В. Громова*

ООО «Наука и Техника»

Лицензия №000350 от 23 декабря 1999 года.

198097, г. Санкт-Петербург, ул. Маршала Говорова, д. 29.

Подписано в печать 04.06.2016. Формат 70х100 1/16.

Бумага газетная. Печать офсетная. Объем 34 п. л.

Тираж 1000. Заказ